

库函数使用说明

1 结构体索引	1
1.1 结构体	1
2 文件索引	3
2.1 文件列表	3
3 结构体说明	5
3.1 CMP_TypeDef结构体 参考	5
3.1.1 详细描述	5
3.1.2 结构体成员变量说明	5
3.1.2.1 BLCWIN	5
3.1.2.2 CFG	6
3.1.2.3 DATA	6
3.1.2.4 IE	6
3.1.2.5 IF	6
3.1.2.6 TCLK	6
3.2 CRC_Model_Params结构体 参考	6
3.2.1 详细描述	7
3.2.2 结构体成员变量说明	7
3.2.2.1 initValue	7
3.2.2.2 polynomial	7
3.2.2.3 reflectIn	7
3.2.2.4 reflectOut	7
3.2.2.5 xorOut	7
3.3 HAL_ADC_TypeDef结构体 参考	7
3.3.1 详细描述	9
3.3.2 结构体成员变量说明	9
3.3.2.1 AMC0	9
3.3.2.2 AMC1	9
3.3.2.3 CFG	9
3.3.2.4 CHN0	9
3.3.2.5 CHN1	9
3.3.2.6 CHN2	10
3.3.2.7 CHN3	10
3.3.2.8 CHNT	10
3.3.2.9 DAT	10
3.3.2.10 DC0	10
3.3.2.11 DC1	10
3.3.2.12 GAIN	11
3.3.2.13 GEN	11
3.3.2.14 HTH	11
3.3.2.15 ICHN	11
3.3.2.16 IDAT	11

3.3.2.17 IE	11
3.3.2.18 IF	12
3.3.2.19 LTH	12
3.3.2.20 null	12
3.3.2.21 null2	12
3.3.2.22 null3	12
3.3.2.23 SWT	12
3.3.2.24 TRIG	13
3.4 HAL_CMP_TypeDef结构体 参考	13
3.4.1 详细描述	13
3.4.2 结构体成员变量说明	13
3.4.2.1 BLCWIN	13
3.4.2.2 CFG	14
3.4.2.3 DATA	14
3.4.2.4 IE	14
3.4.2.5 IF	14
3.4.2.6 TCLK	14
3.5 HAL_CRC_TypeDef结构体 参考	14
3.5.1 详细描述	15
3.5.2 结构体成员变量说明	15
3.5.2.1 CR	15
3.5.2.2 DR	15
3.5.2.3 INIT	15
3.5.2.4 POL	15
3.6 HAL_DMA_TypeDef结构体 参考	16
3.6.1 详细描述	16
3.6.2 结构体成员变量说明	16
3.6.2.1 CCR	16
3.6.2.2 CTMS	16
3.6.2.3 DADR	16
3.6.2.4 REN	17
3.6.2.5 SADR	17
3.7 HAL_DSP_TypeDef结构体 参考	17
3.7.1 详细描述	17
3.7.2 结构体成员变量说明	18
3.7.2.1 ARCTAN	18
3.7.2.2 COS	18
3.7.2.3 DID	18
3.7.2.4 DIS	18
3.7.2.5 MOD	18
3.7.2.6 PC	18
3.7.2.7 QUO	18

3.7.2.8 RAD	19
3.7.2.9 REM	19
3.7.2.10 SC	19
3.7.2.11 SIN	19
3.7.2.12 SQRT	19
3.7.2.13 THETA	19
3.7.2.14 X	19
3.7.2.15 Y	19
3.8 HAL_FLASH_TypeDef结构体 参考	20
3.8.1 详细描述	20
3.8.2 结构体成员变量说明	20
3.8.2.1 ADDR	20
3.8.2.2 CFG	21
3.8.2.3 ERASE	21
3.8.2.4 NADDR	21
3.8.2.5 NCFG	21
3.8.2.6 NERASE	21
3.8.2.7 NKEY	21
3.8.2.8 NRDATA	22
3.8.2.9 NWDATA	22
3.8.2.10 PROTECT	22
3.8.2.11 RDATA	22
3.8.2.12 READY	22
3.8.2.13 SIZE	22
3.8.2.14 WDATA	23
3.9 HAL_GPIO_TypeDef结构体 参考	23
3.9.1 详细描述	23
3.9.2 结构体成员变量说明	24
3.9.2.1 BRR	24
3.9.2.2 BSRR	24
3.9.2.3 F3210	24
3.9.2.4 F7654	24
3.9.2.5 FBA98	24
3.9.2.6 FFEDC	24
3.9.2.7 null1	25
3.9.2.8 PDI	25
3.9.2.9 PDO	25
3.9.2.10 PFLT	25
3.9.2.11 PIE	25
3.9.2.12 PODE	25
3.9.2.13 POE	26
3.9.2.14 PUE	26

3.10 HAL_HALL_TypeDef结构体 参考	26
3.10.1 详细描述	26
3.10.2 结构体成员变量说明	26
3.10.2.1 CFG	26
3.10.2.2 CNT	27
3.10.2.3 INFO	27
3.10.2.4 TH	27
3.10.2.5 WIDTH	27
3.11 HAL_I2C_TypeDef结构体 参考	27
3.11.1 详细描述	28
3.11.2 结构体成员变量说明	28
3.11.2.1 ADDR	28
3.11.2.2 BCR	28
3.11.2.3 BSIZE	28
3.11.2.4 CFG	28
3.11.2.5 DATA	28
3.11.2.6 MSCR	29
3.11.2.7 SCR	29
3.12 HAL_IWDG_TypeDef结构体 参考	29
3.12.1 详细描述	29
3.12.2 结构体成员变量说明	29
3.12.2.1 CFG	29
3.12.2.2 CLR	30
3.12.2.3 CNT	30
3.12.2.4 PSW	30
3.12.2.5 RTH	30
3.12.2.6 WTH	30
3.13 HAL_MCPWM_TypeDef结构体 参考	30
3.13.1 详细描述	32
3.13.2 结构体成员变量说明	32
3.13.2.1 AUEN	32
3.13.2.2 CH_DEF	33
3.13.2.3 CH_MSK	33
3.13.2.4 CNT0	33
3.13.2.5 CNT1	33
3.13.2.6 DTH00	33
3.13.2.7 DTH01	33
3.13.2.8 DTH10	34
3.13.2.9 DTH11	34
3.13.2.10 EIE	34
3.13.2.11 EIF	34
3.13.2.12 EVT0	34

3.13.2.13 EVT1	34
3.13.2.14 FAIL012	35
3.13.2.15 FAIL345	35
3.13.2.16 FCNT	35
3.13.2.17 FLT	35
3.13.2.18 IE0	35
3.13.2.19 IE1	35
3.13.2.20 IF0	36
3.13.2.21 IF1	36
3.13.2.22 IO01	36
3.13.2.23 IO23	36
3.13.2.24 IO45	36
3.13.2.25 null	36
3.13.2.26 PP	37
3.13.2.27 PRT	37
3.13.2.28 RE	37
3.13.2.29 SDCFG	37
3.13.2.30 STT_HYST	37
3.13.2.31 TCLK	37
3.13.2.32 TH	38
3.13.2.33 TH0	38
3.13.2.34 TH1	38
3.13.2.35 TMR0	38
3.13.2.36 TMR1	38
3.13.2.37 TMR2	38
3.13.2.38 TMR3	39
3.13.2.39 UPDATE	39
3.13.2.40 ZCS_DELAY	39
3.14 HAL_QEP_TypeDef结构体 参考	39
3.14.1 详细描述	39
3.14.2 结构体成员变量说明	40
3.14.2.1 CFG	40
3.14.2.2 CNT	40
3.14.2.3 IE	40
3.14.2.4 IF	40
3.14.2.5 TH	40
3.15 HAL_SPI_TypeDef结构体 参考	40
3.15.1 详细描述	41
3.15.2 结构体成员变量说明	41
3.15.2.1 BAUD	41
3.15.2.2 CFG	41
3.15.2.3 IE	41

3.15.2.4 RXDATA	42
3.15.2.5 SIZE	42
3.15.2.6 TXDATA	42
3.16 HAL_SYS_TypeDef结构体 参考	42
3.16.1 详细描述	43
3.16.2 结构体成员变量说明	43
3.16.2.1 ADC	43
3.16.2.2 CLK_CFG	44
3.16.2.3 CLK_DIV0	44
3.16.2.4 CLK_DIV1	44
3.16.2.5 CLK_DIV2	44
3.16.2.6 CLK_FEN	44
3.16.2.7 DAC0	44
3.16.2.8 DAC0_AMC	44
3.16.2.9 DAC0_DC	44
3.16.2.10 DAC1	45
3.16.2.11 DAC1_AMC	45
3.16.2.12 DAC1_DC	45
3.16.2.13 DAC_CTRL	45
3.16.2.14 DBG	45
3.16.2.15 DBG_CFG	45
3.16.2.16 FLSE	45
3.16.2.17 FLSP	45
3.16.2.18 FLST	46
3.16.2.19 INFO	46
3.16.2.20 IO_CFG	46
3.16.2.21 MBIST	46
3.16.2.22 null0	46
3.16.2.23 null1	46
3.16.2.24 null10	46
3.16.2.25 null11	46
3.16.2.26 null12	47
3.16.2.27 null13	47
3.16.2.28 null2	47
3.16.2.29 null3	47
3.16.2.30 null4	47
3.16.2.31 null5	47
3.16.2.32 null6	47
3.16.2.33 null7	47
3.16.2.34 null8	48
3.16.2.35 null9	48
3.16.2.36 REG0	48

3.16.2.37 REG1	48
3.16.2.38 REG2	48
3.16.2.39 REG3	48
3.16.2.40 REG4	48
3.16.2.41 REG5	48
3.16.2.42 REG6	49
3.16.2.43 REG7	49
3.16.2.44 REG8	49
3.16.2.45 REG9	49
3.16.2.46 REGA	49
3.16.2.47 REGB	49
3.16.2.48 REGC	49
3.16.2.49 SBIST	49
3.16.2.50 SFT_RST	50
3.16.2.51 TRIM	50
3.16.2.52 WR_PROTECT	50
3.17 HAL_TIMER_TypeDef结构体 参考	50
3.17.1 详细描述	51
3.17.2 结构体成员变量说明	51
3.17.2.1 CFG	51
3.17.2.2 CMP0	51
3.17.2.3 CMP1	51
3.17.2.4 CNT	51
3.17.2.5 EVT	51
3.17.2.6 FLT	52
3.17.2.7 IE	52
3.17.2.8 IF	52
3.17.2.9 IO	52
3.17.2.10 TH	52
3.18 HAL_UART_TypeDef结构体 参考	52
3.18.1 详细描述	53
3.18.2 结构体成员变量说明	53
3.18.2.1 ADR	53
3.18.2.2 BUFF	53
3.18.2.3 CTRL	54
3.18.2.4 DIVH	54
3.18.2.5 DIVL	54
3.18.2.6 IE	54
3.18.2.7 IF	54
3.18.2.8 IOC	54
3.18.2.9 RE	54
3.18.2.10 STT	54

4 文件说明	55
4.1 hal_lks32mc07x.c 文件参考	55
4.1.1 宏定义说明	56
4.1.1.1 _HAL_LKS32MC05X_C_	56
4.2 hal_lks32mc07x.c	56
4.3 hal_lks32mc07x.h 文件参考	56
4.3.1 宏定义说明	58
4.3.1.1 HAL_ADC_IRQHandler	58
4.3.1.2 HAL_ADC_IRQHandler_00	59
4.3.1.3 HAL_ADC_IRQHandler_10	59
4.3.1.4 HAL_ADC_IRQHandler_20	59
4.3.1.5 HAL_CAN_IRQHandler	59
4.3.1.6 HAL_CAN_IRQHandler_00	59
4.3.1.7 HAL_CL_IRQHandler	59
4.3.1.8 HAL_CL_IRQHandler_00	59
4.3.1.9 HAL_CMP_IRQHandler	59
4.3.1.10 HAL_CMP_IRQHandler_00	60
4.3.1.11 HAL_CMP_IRQHandler_10	60
4.3.1.12 HAL_CMP_IRQHandler_20	60
4.3.1.13 HAL_CMP_IRQHandler_30	60
4.3.1.14 HAL_CMP_IRQHandler_40	60
4.3.1.15 HAL_CMP_IRQHandler_50	60
4.3.1.16 HAL_DMA_IRQHandler	60
4.3.1.17 HAL_DMA_IRQHandler_00	60
4.3.1.18 HAL_DSP_IRQHandler	61
4.3.1.19 HAL_DSP_IRQHandler_00	61
4.3.1.20 HAL_FMAC_IRQHandler_00	61
4.3.1.21 HAL_GPIO_IRQHandler	61
4.3.1.22 HAL_GPIO_IRQHandler_00	61
4.3.1.23 HAL_HALL_IRQHandler	61
4.3.1.24 HAL_HALL_IRQHandler_00	61
4.3.1.25 HAL_HALL_IRQHandler_10	61
4.3.1.26 HAL_I2C_IRQHandler	62
4.3.1.27 HAL_I2C_IRQHandler_00	62
4.3.1.28 HAL_I2C_IRQHandler_10	62
4.3.1.29 HAL_INIT_DEFAULE_MCLK	62
4.3.1.30 HAL_INIT_DEFAULE_MCLK_MS	62
4.3.1.31 HAL_INIT_DEFAULE_MCLK_US	62
4.3.1.32 HAL_INIT_IRQHandler	62
4.3.1.33 HAL_INIT_IRQHandler_00	63
4.3.1.34 HAL_MCPWM_IRQHandler	63
4.3.1.35 HAL_MCPWM_IRQHandler_00	63

4.3.1.36 HAL_MCPWM_IRQHandler_01	63
4.3.1.37 HAL_MCPWM_IRQHandler_10	63
4.3.1.38 HAL_MCPWM_IRQHandler_11	63
4.3.1.39 HAL_PWRDN_IRQHandler	63
4.3.1.40 HAL_PWRDN_IRQHandler_00	63
4.3.1.41 HAL_QEP_IRQHandler	64
4.3.1.42 HAL_QEP_IRQHandler_00	64
4.3.1.43 HAL_QEP_IRQHandler_10	64
4.3.1.44 HAL_QEP_IRQHandler_20	64
4.3.1.45 HAL_QEP_IRQHandler_30	64
4.3.1.46 HAL_SIF_IRQHandler	64
4.3.1.47 HAL_SIF_IRQHandler_00	64
4.3.1.48 HAL_SPI_IRQHandler	64
4.3.1.49 HAL_SPI_IRQHandler_00	65
4.3.1.50 HAL_SPI_IRQHandler_10	65
4.3.1.51 HAL_SW_IRQHandler	65
4.3.1.52 HAL_SW_IRQHandler_00	65
4.3.1.53 HAL_TIMER_IRQHandler	65
4.3.1.54 HAL_TIMER_IRQHandler_00	65
4.3.1.55 HAL_TIMER_IRQHandler_10	65
4.3.1.56 HAL_TIMER_IRQHandler_20	66
4.3.1.57 HAL_TIMER_IRQHandler_30	66
4.3.1.58 HAL_TIMER_IRQHandler_40	66
4.3.1.59 HAL_UART_IRQHandler	66
4.3.1.60 HAL_UART_IRQHandler_00	66
4.3.1.61 HAL_UART_IRQHandler_10	66
4.3.1.62 HAL_UART_IRQHandler_20	66
4.3.1.63 HAL_WAKE_IRQHandler	66
4.3.1.64 HAL_WAKE_IRQHandler_00	67
4.3.1.65 HAL_WWDG_IRQHandler_00	67
4.4 hal_lks32mc07x.h	67
4.5 hal_lksmcu.h 文件参考	68
4.6 hal_lksmcu.h	68
4.7 include/hal_lks32mc07x_reg.h 文件参考	69
4.7.1 变量说明	70
4.7.1.1 HAL_ADC	70
4.7.1.2 HAL_CMP	70
4.7.1.3 HAL_CRC	70
4.7.1.4 HAL_DMA	71
4.7.1.5 HAL_DSP	71
4.7.1.6 HAL_FLASH	71
4.7.1.7 HAL_GPIO	71

4.7.1.8 HAL_HALL	71
4.7.1.9 HAL_I2C	71
4.7.1.10 HAL_IWDG	71
4.7.1.11 HAL_MCPWM	71
4.7.1.12 HAL_QEP	72
4.7.1.13 HAL_SPI	72
4.7.1.14 HAL_SYS	72
4.7.1.15 HAL_TIMER	72
4.7.1.16 HAL_UART	72
4.8 hal_lks32mc07x_reg.h	72
4.9 include/hal_lksmcu_adc.h 文件参考	76
4.9.1 枚举类型说明	78
4.9.1.1 HAL_ADC_PORT_x	78
4.9.1.2 HAL_ADC_REFERENCE_x	79
4.9.1.3 HAL_ADC_SAMP_NUMBER_x	79
4.9.1.4 HAL_ADC_SAMP_SEQUENCE_x	80
4.9.1.5 HAL_ADC_TRIGGER_x	81
4.9.1.6 HAL_ADC_x	81
4.9.2 函数说明	81
4.9.2.1 HAL_ADC_Disable()	81
4.9.2.2 HAL_ADC_Enable()	82
4.9.2.3 HAL_ADC_EndSampling()	82
4.9.2.4 HAL_ADC_GetRange()	82
4.9.2.5 HAL_ADC_GetSamplingValues()	83
4.9.2.6 HAL_ADC_GetValuesUnit()	84
4.9.2.7 HAL_ADC_ManualSampling()	84
4.9.2.8 HAL_ADC_Reset()	85
4.9.2.9 HAL_ADC_SetGpioIn()	85
4.9.2.10 HAL_ADC_SetGpioTriggerOut()	86
4.9.2.11 HAL_ADC_SetReference()	86
4.9.2.12 HAL_ADC_SetSamplingChannel()	87
4.9.2.13 HAL_ADC_SetSamplingQuantity()	88
4.9.2.14 HAL_ADC_SetTriggerSignal()	89
4.9.2.15 HAL_ADC_SoftTrigger()	89
4.10 hal_lksmcu_adc.h	90
4.11 include/hal_lksmcu_can.h 文件参考	92
4.12 hal_lksmcu_can.h	92
4.13 include/hal_lksmcu_clk.h 文件参考	93
4.13.1 枚举类型说明	94
4.13.1.1 HAL_CLK_FREQ_x	94
4.13.2 函数说明	94
4.13.2.1 HAL_CLK_GetMcuFreq()	94

4.13.2.2 HAL_CLK_SetMcuFreq()	95
4.14 hal_lksmcu_clk.h	96
4.15 include/hal_lksmcu_cmp.h 文件参考	96
4.15.1 枚举类型说明	98
4.15.1.1 HAL_CMP_CHANNEL_INx	98
4.15.1.2 HAL_CMP_CHANNEL_IPx	98
4.15.1.3 HAL_CMP_x	99
4.15.2 函数说明	100
4.15.2.1 HAL_CMP_Disable()	100
4.15.2.2 HAL_CMP_Enable()	100
4.15.2.3 HAL_CMP_GetData()	101
4.15.2.4 HAL_CMP_Reset()	101
4.15.2.5 HAL_CMP_SetChannel()	102
4.15.2.6 HAL_CMP_SetDigitalFilter()	102
4.15.2.7 HAL_CMP_SetGpioIn()	102
4.15.2.8 HAL_CMP_SetGpioOut()	103
4.16 hal_lksmcu_cmp.h	104
4.17 include/hal_lksmcu_crc.h 文件参考	105
4.17.1 枚举类型说明	105
4.17.1.1 HAL_CRC_MODEL_x	105
4.17.1.2 HAL_CRC_x	106
4.17.2 函数说明	106
4.17.2.1 HAL_CRC_Calc()	106
4.17.2.2 HAL_CRC_Disable()	107
4.17.2.3 HAL_CRC_Enable()	107
4.17.2.4 HAL_CRC_Reset()	107
4.18 hal_lksmcu_crc.h	108
4.19 include/hal_lksmcu_dac.h 文件参考	108
4.19.1 枚举类型说明	109
4.19.1.1 HAL_DAC_x	109
4.19.2 函数说明	109
4.19.2.1 HAL_DAC_Disable()	109
4.19.2.2 HAL_DAC_Enable()	110
4.19.2.3 HAL_DAC_Out2GpioDisable()	110
4.19.2.4 HAL_DAC_Out2GpioEnable()	110
4.19.2.5 HAL_DAC_Reset()	110
4.19.2.6 HAL_DAC_SetGpioOut()	111
4.19.2.7 HAL_DAC_SetOutVal()	111
4.19.2.8 HAL_DAC_SetOutVoltageV()	111
4.20 hal_lksmcu_dac.h	112
4.21 include/hal_lksmcu_dma.h 文件参考	112
4.21.1 枚举类型说明	113

4.21.1.1 HAL_DMA_x	113
4.21.2 函数说明	113
4.21.2.1 HAL_DMA_Disable()	113
4.21.2.2 HAL_DMA_Enable()	114
4.21.2.3 HAL_DMA_I2cRx()	114
4.21.2.4 HAL_DMA_I2cTx()	114
4.21.2.5 HAL_DMA_MemCopy()	114
4.21.2.6 HAL_DMA_Reset()	114
4.21.2.7 HAL_DMA_SpiRx()	114
4.21.2.8 HAL_DMA_SpiRxTx()	115
4.21.2.9 HAL_DMA_SpiTx()	115
4.21.2.10 HAL_DMA_UartRx()	115
4.21.2.11 HAL_DMA_UartTx()	115
4.22 hal_lksmcu_dma.h	116
4.23 include/hal_lksmcu_dsp.h 文件参考	116
4.23.1 枚举类型说明	117
4.23.1.1 HAL_DSP_x	117
4.23.2 函数说明	117
4.23.2.1 HAL_DSP_Cos()	117
4.23.2.2 HAL_DSP_Disable()	118
4.23.2.3 HAL_DSP_Div()	118
4.23.2.4 HAL_DSP_Enable()	118
4.23.2.5 HAL_DSP_Reset()	118
4.23.2.6 HAL_DSP_Sin()	118
4.23.2.7 HAL_DSP_SinCos()	118
4.23.2.8 HAL_DSP_Sqrt()	119
4.24 hal_lksmcu_dsp.h	119
4.25 include/hal_lksmcu_flash.h 文件参考	119
4.25.1 枚举类型说明	120
4.25.1.1 HAL_FLASH_x	120
4.25.2 函数说明	121
4.25.2.1 HAL_FLASH_Disable()	121
4.25.2.2 HAL_FLASH_DisableCache()	121
4.25.2.3 HAL_FLASH_Enable()	121
4.25.2.4 HAL_FLASH_EnableCache()	121
4.25.2.5 HAL_FLASH_Erase()	122
4.25.2.6 HAL_FLASH_Program()	122
4.25.2.7 HAL_FLASH_Read()	123
4.25.2.8 HAL_FLASH_ReadSectorSize()	124
4.25.2.9 HAL_FLASH_ReadUint16()	124
4.25.2.10 HAL_FLASH_ReadUint32()	125
4.25.2.11 HAL_FLASH_ReadUint8()	125

4.25.2.12 HAL_FLASH_Reset()	126
4.25.2.13 HAL_FLASH_SecurityParameterRead()	126
4.25.2.14 HAL_FLASH_SecurityParameterSave()	127
4.25.2.15 HAL_FLASH_Update()	128
4.26 hal_lksmcu_flash.h	128
4.27 include/hal_lksmcu_gpio.h 文件参考	129
4.27.1 枚举类型说明	131
4.27.1.1 HAL_GPIO_AFx	131
4.27.1.2 HAL_GPIO_MODE_x	132
4.27.1.3 HAL_GPIO_Px	133
4.27.2 函数说明	135
4.27.2.1 HAL_GPIO_Disable()	135
4.27.2.2 HAL_GPIO_Enable()	135
4.27.2.3 HAL_GPIO_GetInData()	135
4.27.2.4 HAL_GPIO_GetOutData()	136
4.27.2.5 HAL_GPIO_Invert()	136
4.27.2.6 HAL_GPIO_Reset()	137
4.27.2.7 HAL_GPIO_RstReuseGpioDisable()	137
4.27.2.8 HAL_GPIO_RstReuseGpioEnable()	137
4.27.2.9 HAL_GPIO_SetAf()	137
4.27.2.10 HAL_GPIO_SetHigh()	139
4.27.2.11 HAL_GPIO_SetLow()	139
4.27.2.12 HAL_GPIO_SetMode()	139
4.27.2.13 HAL_GPIO_SwdReuseGpioDisable()	140
4.27.2.14 HAL_GPIO_SwdReuseGpioEnable()	141
4.27.2.15 HAL_GPIO_Write()	141
4.28 hal_lksmcu_gpio.h	141
4.29 include/hal_lksmcu_hall.h 文件参考	143
4.29.1 枚举类型说明	145
4.29.1.1 HAL_HALL_x	145
4.29.2 函数说明	145
4.29.2.1 HAL_HALL_Disable()	145
4.29.2.2 HAL_HALL_Enable()	145
4.29.2.3 HAL_HALL_ReadHallData()	146
4.29.2.4 HAL_HALL_ReadHallWidth()	146
4.29.2.5 HAL_HALL_ReadTh()	146
4.29.2.6 HAL_HALL_Reset()	147
4.29.2.7 HAL_HALL_SetFilter()	147
4.29.2.8 HAL_HALL_SetFilteruS()	148
4.29.2.9 HAL_HALL_SetFreq()	148
4.29.2.10 HAL_HALL_SetGpio()	149
4.29.2.11 HAL_HALL_SetTh()	149

4.29.2.12 HAL_HALL_StartCounting()	150
4.29.2.13 HAL_HALL_StopCounting()	150
4.30 hal_lksmcu_hall.h	150
4.31 include/hal_lksmcu_i2c.h 文件参考	151
4.31.1 枚举类型说明	152
4.31.1.1 HAL_I2C_x	152
4.31.2 函数说明	152
4.31.2.1 HAL_I2C_BusReset()	152
4.31.2.2 HAL_I2C_Disable()	153
4.31.2.3 HAL_I2C_Enable()	153
4.31.2.4 HAL_I2C_Reset()	154
4.31.2.5 HAL_I2C_RxData()	154
4.31.2.6 HAL_I2C_SetBaud()	154
4.31.2.7 HAL_I2C_SetGpioScl()	155
4.31.2.8 HAL_I2C_SetGpioSda()	156
4.31.2.9 HAL_I2C_TxData()	156
4.32 hal_lksmcu_i2c.h	157
4.33 include/hal_lksmcu_init.h 文件参考	157
4.33.1 枚举类型说明	157
4.33.1.1 HAL_INIT_POWER_x	157
4.33.2 函数说明	158
4.33.2.1 HAL_INIT_PowerReady()	158
4.33.2.2 HAL_INIT_SysInit()	158
4.34 hal_lksmcu_init.h	158
4.35 include/hal_lksmcu_irq.h 文件参考	158
4.35.1 枚举类型说明	159
4.35.1.1 HAL_IRQ_MODULE_x	159
4.35.2 函数说明	160
4.35.2.1 HAL_IRQ_Disable()	160
4.35.2.2 HAL_IRQ_Enable()	161
4.35.2.3 HAL_IRQ_ModuleDisable()	161
4.35.2.4 HAL_IRQ_ModuleEnable()	161
4.35.2.5 HAL_IRQ_Reset()	162
4.36 hal_lksmcu_irq.h	162
4.37 include/hal_lksmcu_irqfun.h 文件参考	163
4.37.1 函数说明	164
4.37.1.1 HAL_ADC_IrqSamp1()	164
4.37.1.2 HAL_ADC_IrqSamp1Clear()	165
4.37.1.3 HAL_ADC_IrqSamp1Disable()	165
4.37.1.4 HAL_ADC_IrqSamp1Enable()	165
4.37.1.5 HAL_ADC_IrqSamp1Flg()	165
4.37.1.6 HAL_ADC_IrqSamp2()	166

4.37.1.7 HAL_ADC_IrqSamp2Clear()	166
4.37.1.8 HAL_ADC_IrqSamp2Disable()	166
4.37.1.9 HAL_ADC_IrqSamp2Enable()	167
4.37.1.10 HAL_ADC_IrqSamp2Flg()	167
4.37.1.11 HAL_HALL_IrqChange()	167
4.37.1.12 HAL_HALL_IrqChangeClear()	168
4.37.1.13 HAL_HALL_IrqChangeDisable()	168
4.37.1.14 HAL_HALL_IrqChangeEnable()	168
4.37.1.15 HAL_HALL_IrqChangeFlg()	169
4.37.1.16 HAL_HALL_IrqOver()	169
4.37.1.17 HAL_HALL_IrqOverClear()	169
4.37.1.18 HAL_HALL_IrqOverDisable()	170
4.37.1.19 HAL_HALL_IrqOverEnable()	170
4.37.1.20 HAL_HALL_IrqOverFlg()	170
4.38 hal_lksmcu_irqfun.h	170
4.39 include/hal_lksmcu_iwdg.h 文件参考	171
4.39.1 枚举类型说明	172
4.39.1.1 HAL_IWDG_x	172
4.39.2 函数说明	172
4.39.2.1 HAL_IWDG_Disable()	172
4.39.2.2 HAL_IWDG_Enable()	173
4.39.2.3 HAL_IWDG_Feed()	173
4.39.2.4 HAL_IWDG_Reset()	174
4.39.2.5 HAL_IWDG_SetResetTimemS()	174
4.40 hal_lksmcu_iwdg.h	174
4.41 include/hal_lksmcu_mcpwm.h 文件参考	175
4.41.1 枚举类型说明	177
4.41.1.1 HAL_MCPWM_DEFAULT_OUTPUT_STATE_x	177
4.41.1.2 HAL_MCPWM_FAIL_x	177
4.41.1.3 HAL_MCPWM_OUTPUT_STATE_x	177
4.41.1.4 HAL_MCPWM_POLARITY_x	178
4.41.1.5 HAL_MCPWM_PORT_x	178
4.41.1.6 HAL_MCPWM_x	179
4.41.2 函数说明	179
4.41.2.1 HAL_MCPWM_Disable()	179
4.41.2.2 HAL_MCPWM_Enable()	179
4.41.2.3 HAL_MCPWM_GetFreq()	179
4.41.2.4 HAL_MCPWM_GetThCnt()	181
4.41.2.5 HAL_MCPWM_IrqFail()	182
4.41.2.6 HAL_MCPWM_IrqFailClear()	182
4.41.2.7 HAL_MCPWM_IrqFailDisable()	182
4.41.2.8 HAL_MCPWM_IrqFailEnable()	182

4.41.2.9 HAL_MCPWM_IrqOverflow()	182
4.41.2.10 HAL_MCPWM_IrqOverflowClear()	182
4.41.2.11 HAL_MCPWM_IrqOverflowDisable()	182
4.41.2.12 HAL_MCPWM_IrqOverflowEnable()	183
4.41.2.13 HAL_MCPWM_IrqZeroCross()	183
4.41.2.14 HAL_MCPWM_IrqZeroCrossClear()	183
4.41.2.15 HAL_MCPWM_IrqZeroCrossDisable()	183
4.41.2.16 HAL_MCPWM_IrqZeroCrossEnable()	183
4.41.2.17 HAL_MCPWM_Reset()	183
4.41.2.18 HAL_MCPWM_SetAdcTrig()	184
4.41.2.19 HAL_MCPWM_SetAdcTrigQ15()	184
4.41.2.20 HAL_MCPWM_SetDead()	184
4.41.2.21 HAL_MCPWM_SetDefaultState()	185
4.41.2.22 HAL_MCPWM_SetDrivePolarity()	185
4.41.2.23 HAL_MCPWM_SetFailSignal()	186
4.41.2.24 HAL_MCPWM_SetFreq()	186
4.41.2.25 HAL_MCPWM_SetGpioFail()	187
4.41.2.26 HAL_MCPWM_SetGpioOut()	187
4.41.2.27 HAL_MCPWM_SetOutputDisable()	188
4.41.2.28 HAL_MCPWM_SetOutputEnable()	188
4.41.2.29 HAL_MCPWM_SetOutState()	189
4.41.2.30 HAL_MCPWM_SetPwm()	189
4.41.2.31 HAL_MCPWM_SetPwmQ15()	190
4.41.2.32 HAL_MCPWM_StartCounting()	190
4.41.2.33 HAL_MCPWM_StopCounting()	191
4.42 hal_lksmcu_mcpwm.h	191
4.43 include/hal_lksmcu_opa.h 文件参考	192
4.43.1 宏定义说明	194
4.43.1.1 HAL_OPA_OPAX_2_ADCCHN	194
4.43.1.2 HAL_OPA_READ_GAIN	194
4.43.2 枚举类型说明	194
4.43.2.1 HAL_OPA_GAIN_x	194
4.43.2.2 HAL_OPA_x	195
4.43.3 函数说明	196
4.43.3.1 HAL_OPA_Disable()	196
4.43.3.2 HAL_OPA_Enable()	196
4.43.3.3 HAL_OPA_Out2GpioDisable()	196
4.43.3.4 HAL_OPA_Out2GpioEnable()	196
4.43.3.5 HAL_OPA_Reset()	197
4.43.3.6 HAL_OPA_SetGain()	197
4.43.3.7 HAL_OPA_SetGpioIn()	197
4.43.3.8 HAL_OPA_SetGpioOut()	197

4.44 hal_lksmcu_opa.h	198
4.45 include/hal_lksmcu_qep.h 文件参考	199
4.45.1 枚举类型说明	200
4.45.1.1 HAL_QEP_MODE_x	200
4.45.1.2 HAL_QEP_x	201
4.45.2 函数说明	201
4.45.2.1 HAL_QEP_Disable()	201
4.45.2.2 HAL_QEP_Enable()	201
4.45.2.3 HAL_QEP_ReadCnt()	202
4.45.2.4 HAL_QEP_Reset()	202
4.45.2.5 HAL_QEP_ResetCnt()	202
4.45.2.6 HAL_QEP_SetGpio()	203
4.45.2.7 HAL_QEP_SetMode()	203
4.45.2.8 HAL_QEP_SetTh()	204
4.46 hal_lksmcu_qep.h	204
4.47 include/hal_lksmcu_reg.h 文件参考	204
4.47.1 详细描述	205
4.47.2 宏定义说明	205
4.47.2.1 HAL_REG_IF_WRITE	205
4.47.2.2 HAL_REG_INVERT	205
4.47.2.3 HAL_REG_READ	205
4.47.2.4 HAL_REG_READ_BIT	206
4.47.2.5 HAL_REG_READ_BIT_POS_MASK	206
4.47.2.6 HAL_REG_RESET	206
4.47.2.7 HAL_REG_SET	206
4.47.2.8 HAL_REG_SYS_LOCK	206
4.47.2.9 HAL_REG_SYS_UNLOCK	206
4.47.2.10 HAL_REG_WRITE	207
4.47.2.11 HAL_REG_WRITE_BIT	207
4.47.2.12 HAL_REG_WRITE_BIT_MASK_POS	207
4.48 hal_lksmcu_reg.h	207
4.49 include/hal_lksmcu_reset.h 文件参考	207
4.49.1 函数说明	208
4.49.1.1 HAL_RESET_ResetCore()	208
4.49.1.2 HAL_RESET_ResetGlobal()	208
4.49.1.3 HAL_RESET_ResetRegister()	208
4.50 hal_lksmcu_reset.h	209
4.51 include/hal_lksmcu_spi.h 文件参考	209
4.51.1 枚举类型说明	210
4.51.1.1 HAL_SPI_MODE_x	210
4.51.1.2 HAL_SPI_x	211
4.51.2 函数说明	211

4.51.2.1 HAL_SPI_Disable()	211
4.51.2.2 HAL_SPI_Enable()	212
4.51.2.3 HAL_SPI_Reset()	212
4.51.2.4 HAL_SPI_RxData()	212
4.51.2.5 HAL_SPI_RxDataWidth16()	213
4.51.2.6 HAL_SPI_RxTxData()	213
4.51.2.7 HAL_SPI_RxTxDataWidth16()	213
4.51.2.8 HAL_SPI_SetBaud()	214
4.51.2.9 HAL_SPI_SetGpioClk()	214
4.51.2.10 HAL_SPI_SetGpioCs()	216
4.51.2.11 HAL_SPI_SetGpioDi()	217
4.51.2.12 HAL_SPI_SetGpioDo()	217
4.51.2.13 HAL_SPI_SetGpioMiso()	218
4.51.2.14 HAL_SPI_SetGpioMosi()	219
4.51.2.15 HAL_SPI_SetMode()	220
4.51.2.16 HAL_SPI_TxData()	220
4.51.2.17 HAL_SPI_TxDataWidth16()	220
4.52 hal_lksmcu_spi.h	221
4.53 include/hal_lksmcu_timer.h 文件参考	221
4.53.1 枚举类型说明	223
4.53.1.1 HAL_TIMER_MODE_x	223
4.53.1.2 HAL_TIMER_x	223
4.53.2 函数说明	224
4.53.2.1 HAL_TIMER_Disable()	224
4.53.2.2 HAL_TIMER_Enable()	224
4.53.2.3 HAL_TIMER_GetFreq()	224
4.53.2.4 HAL_TIMER_GetThCnt()	225
4.53.2.5 HAL_TIMER_IrqCompare0()	225
4.53.2.6 HAL_TIMER_IrqCompare0Clear()	225
4.53.2.7 HAL_TIMER_IrqCompare0Disable()	225
4.53.2.8 HAL_TIMER_IrqCompare0Enable()	225
4.53.2.9 HAL_TIMER_IrqCompare1()	226
4.53.2.10 HAL_TIMER_IrqCompare1Clear()	226
4.53.2.11 HAL_TIMER_IrqCompare1Disable()	226
4.53.2.12 HAL_TIMER_IrqCompare1Enable()	226
4.53.2.13 HAL_TIMER_IrqOverflow()	226
4.53.2.14 HAL_TIMER_IrqOverflowClear()	226
4.53.2.15 HAL_TIMER_IrqOverflowDisable()	226
4.53.2.16 HAL_TIMER_IrqOverflowEnable()	227
4.53.2.17 HAL_TIMER_Reset()	227
4.53.2.18 HAL_TIMER_SetFreq()	227
4.53.2.19 HAL_TIMER_SetGpioIn()	227

4.53.2.20 HAL_TIMER_SetGpioOut()	228
4.53.2.21 HAL_TIMER_SetPeriodCount()	229
4.53.2.22 HAL_TIMER_SetPeriodmS()	229
4.53.2.23 HAL_TIMER_SetPerioduS()	230
4.53.2.24 HAL_TIMER_SetPwm()	230
4.53.2.25 HAL_TIMER_SetPwmQ15()	231
4.53.2.26 HAL_TIMER_SetWorkMode()	231
4.53.2.27 HAL_TIMER_StartCounting()	231
4.53.2.28 HAL_TIMER_StopCounting()	231
4.54 hal_lksmcu_timer.h	232
4.55 include/hal_lksmcu_tmp.h 文件参考	232
4.55.1 枚举类型说明	234
4.55.1.1 HAL_TMP_x	234
4.55.2 函数说明	234
4.55.2.1 HAL_TMP_Adc2Temperature()	234
4.55.2.2 HAL_TMP_Adc2Temperature01()	235
4.55.2.3 HAL_TMP_Disable()	235
4.55.2.4 HAL_TMP_Enable()	236
4.55.2.5 HAL_TMP_GetTemperature()	236
4.55.2.6 HAL_TMP_GetTemperature01()	237
4.55.2.7 HAL_TMP_Reset()	238
4.56 hal_lksmcu_tmp.h	238
4.57 include/hal_lksmcu_trim.h 文件参考	239
4.57.1 函数说明	239
4.57.1.1 HAL_TRIM_Read()	239
4.57.1.2 HAL_TRIM_ReadId()	240
4.57.1.3 HAL_TRIM_ReadIdCrc32()	241
4.58 hal_lksmcu_trim.h	241
4.59 include/hal_lksmcu_uart.h 文件参考	242
4.59.1 枚举类型说明	243
4.59.1.1 HAL_UART_MODE_x	243
4.59.1.2 HAL_UART_x	244
4.59.2 函数说明	244
4.59.2.1 HAL_UART_Disable()	244
4.59.2.2 HAL_UART_Enable()	244
4.59.2.3 HAL_UART_IrqCheckErr()	244
4.59.2.4 HAL_UART_IrqCheckErrClear()	244
4.59.2.5 HAL_UART_IrqCheckErrDisable()	245
4.59.2.6 HAL_UART_IrqCheckErrEnable()	245
4.59.2.7 HAL_UART_IrqReceiveOver()	245
4.59.2.8 HAL_UART_IrqReceiveOverClear()	245
4.59.2.9 HAL_UART_IrqReceiveOverDisable()	245

4.59.2.10 HAL_UART_IrqReceiveOverEnable()	245
4.59.2.11 HAL_UART_IrqSendOver()	245
4.59.2.12 HAL_UART_IrqSendOverClear()	246
4.59.2.13 HAL_UART_IrqSendOverDisable()	246
4.59.2.14 HAL_UART_IrqSendOverEnable()	246
4.59.2.15 HAL_UART_IrqStopErr()	246
4.59.2.16 HAL_UART_IrqStopErrClear()	246
4.59.2.17 HAL_UART_IrqStopErrDisable()	246
4.59.2.18 HAL_UART_IrqStopErrEnable()	246
4.59.2.19 HAL_UART_ReadBuffer()	247
4.59.2.20 HAL_UART_Reset()	247
4.59.2.21 HAL_UART_SetBaud()	247
4.59.2.22 HAL_UART_SetGpioRx()	247
4.59.2.23 HAL_UART_SetGpioTx()	248
4.59.2.24 HAL_UART_SetMode()	248
4.59.2.25 HAL_UART_TxData()	249
4.59.2.26 HAL_UART_WriteBuffer()	249
4.60 hal_lksmcu_uart.h	249
4.61 include/hal_lksmcu_wake.h 文件参考	250
4.61.1 枚举类型说明	251
4.61.1.1 HAL_WAKE_UP_IO_x	251
4.61.2 函数说明	251
4.61.2.1 HAL_WAKE_DeepSleepIoTimeWakeUp()	251
4.61.2.2 HAL_WAKE_DeepSleepIoWake()	252
4.61.2.3 HAL_WAKE_DeepSleepTimeWakeUp()	252
4.62 hal_lksmcu_wake.h	253
4.63 readme.md 文件参考	253
4.64 source/hal_lks32mc07x_adc.c 文件参考	253
4.64.1 详细描述	255
4.64.2 函数说明	255
4.64.2.1 HAL_ADC_Disable()	255
4.64.2.2 HAL_ADC_Enable()	255
4.64.2.3 HAL_ADC_EndSampling()	255
4.64.2.4 HAL_ADC_GetRange()	256
4.64.2.5 HAL_ADC_GetSamplingValues()	257
4.64.2.6 HAL_ADC_GetValuesUnit()	257
4.64.2.7 HAL_ADC_ManualSampling()	258
4.64.2.8 HAL_ADC_Reset()	258
4.64.2.9 HAL_ADC_SetGpioIn()	259
4.64.2.10 HAL_ADC_SetGpioTriggerOut()	259
4.64.2.11 HAL_ADC_SetReference()	260
4.64.2.12 HAL_ADC_SetSamplingChannel()	260

4.64.2.13 HAL_ADC_SetSamplingQuantity()	261
4.64.2.14 HAL_ADC_SetTriggerSignal()	262
4.64.2.15 HAL_ADC_SoftTrigger()	263
4.64.2.16 HAL_ADC_UpdateTrig()	263
4.64.3 变量说明	265
4.64.3.1 adc_soft_reg	265
4.65 hal_lks32mc07x_adc.c	265
4.66 source/hal_lks32mc07x_clk.c 文件参考	269
4.66.1 函数说明	270
4.66.1.1 HAL_CLK_GetMcuFreq()	270
4.66.1.2 HAL_CLK_SetMcuFreq()	271
4.66.2 变量说明	272
4.66.2.1 HAL_CLK_mclk	272
4.67 hal_lks32mc07x_clk.c	272
4.68 source/hal_lks32mc07x_cmp.c 文件参考	273
4.68.1 函数说明	274
4.68.1.1 HAL_CMP_Disable()	274
4.68.1.2 HAL_CMP_Enable()	274
4.68.1.3 HAL_CMP_GetData()	275
4.68.1.4 HAL_CMP_Reset()	275
4.68.1.5 HAL_CMP_SetChannel()	276
4.68.1.6 HAL_CMP_SetDigitalFilter()	276
4.68.1.7 HAL_CMP_SetGpioIn()	276
4.68.1.8 HAL_CMP_SetGpioOut()	277
4.69 hal_lks32mc07x_cmp.c	278
4.70 source/hal_lks32mc07x_crc.c 文件参考	280
4.70.1 函数说明	281
4.70.1.1 HAL_CRC_Calc()	281
4.70.1.2 HAL_CRC_Disable()	282
4.70.1.3 HAL_CRC_Enable()	282
4.70.1.4 HAL_CRC_Reflect()	282
4.70.1.5 HAL_CRC_Reset()	282
4.70.2 变量说明	283
4.70.2.1 crcModels	283
4.71 hal_lks32mc07x_crc.c	283
4.72 source/hal_lks32mc07x_dac.c 文件参考	284
4.72.1 函数说明	285
4.72.1.1 HAL_DAC_Disable()	285
4.72.1.2 HAL_DAC_Enable()	286
4.72.1.3 HAL_DAC_Out2GpioDisable()	286
4.72.1.4 HAL_DAC_Out2GpioEnable()	286
4.72.1.5 HAL_DAC_Reset()	286

4.72.1.6 HAL_DAC_SetGpioOut()	287
4.72.1.7 HAL_DAC_SetOutVal()	287
4.72.1.8 HAL_DAC_SetOutVoltageV()	287
4.73 hal_lks32mc07x_dac.c	288
4.74 source/hal_lks32mc07x_dma.c 文件参考	290
4.74.1 函数说明	291
4.74.1.1 HAL_DMA_Disable()	291
4.74.1.2 HAL_DMA_Enable()	291
4.74.1.3 HAL_DMA_I2cRx()	291
4.74.1.4 HAL_DMA_I2cTx()	292
4.74.1.5 HAL_DMA_MemCopy()	292
4.74.1.6 HAL_DMA_Reset()	292
4.74.1.7 HAL_DMA_SpiRx()	292
4.74.1.8 HAL_DMA_SpiRxTx()	292
4.74.1.9 HAL_DMA_SpiTx()	293
4.74.1.10 HAL_DMA_UartRx()	293
4.74.1.11 HAL_DMA_UartTx()	293
4.74.2 变量说明	293
4.74.2.1 HAL_DMA_ena	293
4.75 hal_lks32mc07x_dma.c	294
4.76 source/hal_lks32mc07x_dsp.c 文件参考	295
4.76.1 函数说明	296
4.76.1.1 HAL_DSP_Cos()	296
4.76.1.2 HAL_DSP_Disable()	296
4.76.1.3 HAL_DSP_Div()	296
4.76.1.4 HAL_DSP_Enable()	297
4.76.1.5 HAL_DSP_Reset()	297
4.76.1.6 HAL_DSP_Sin()	297
4.76.1.7 HAL_DSP_SinCos()	297
4.76.1.8 HAL_DSP_Sqrt()	297
4.77 hal_lks32mc07x_dsp.c	298
4.78 source/hal_lks32mc07x_flash.c 文件参考	298
4.78.1 函数说明	300
4.78.1.1 HAL_FLASH_Disable()	300
4.78.1.2 HAL_FLASH_DisableCache()	300
4.78.1.3 HAL_FLASH_Enable()	300
4.78.1.4 HAL_FLASH_EnableCache()	301
4.78.1.5 HAL_FLASH_Erase()	301
4.78.1.6 HAL_FLASH_Program()	302
4.78.1.7 HAL_FLASH_Read()	302
4.78.1.8 HAL_FLASH_ReadSectorSize()	303
4.78.1.9 HAL_FLASH_ReadUint16()	303

4.78.1.10 HAL_FLASH_ReadUint32()	304
4.78.1.11 HAL_FLASH_ReadUint8()	304
4.78.1.12 HAL_FLASH_Reset()	305
4.78.1.13 HAL_FLASH_SecurityParameterCrc()	305
4.78.1.14 HAL_FLASH_SecurityParameterRead()	306
4.78.1.15 HAL_FLASH_SecurityParameterSave()	306
4.78.1.16 HAL_FLASH_Update()	307
4.78.2 变量说明	308
4.78.2.1 HAL_FLASH_data	308
4.79 hal_lks32mc07x_flash.c	308
4.80 source/hal_lks32mc07x_gpio.c 文件参考	311
4.80.1 函数说明	313
4.80.1.1 HAL_GPIO_Disable()	313
4.80.1.2 HAL_GPIO_Enable()	313
4.80.1.3 HAL_GPIO_GetInData()	313
4.80.1.4 HAL_GPIO_GetOutData()	314
4.80.1.5 HAL_GPIO_Invert()	314
4.80.1.6 HAL_GPIO_Reset()	314
4.80.1.7 HAL_GPIO_RstReuseGpioDisable()	315
4.80.1.8 HAL_GPIO_RstReuseGpioEnable()	315
4.80.1.9 HAL_GPIO_SetAf()	315
4.80.1.10 HAL_GPIO_SetHigh()	316
4.80.1.11 HAL_GPIO_SetLow()	317
4.80.1.12 HAL_GPIO_SetMode()	317
4.80.1.13 HAL_GPIO_SwdReuseGpioDisable()	318
4.80.1.14 HAL_GPIO_SwdReuseGpioEnable()	319
4.80.1.15 HAL_GPIO_Write()	319
4.81 hal_lks32mc07x_gpio.c	319
4.82 source/hal_lks32mc07x_hall.c 文件参考	322
4.82.1 函数说明	323
4.82.1.1 HAL_HALL_Disable()	323
4.82.1.2 HAL_HALL_Enable()	323
4.82.1.3 HAL_HALL_ReadHallData()	324
4.82.1.4 HAL_HALL_ReadHallWidth()	324
4.82.1.5 HAL_HALL_ReadTh()	324
4.82.1.6 HAL_HALL_Reset()	325
4.82.1.7 HAL_HALL_SetFilter()	325
4.82.1.8 HAL_HALL_SetFilteruS()	326
4.82.1.9 HAL_HALL_SetFreq()	326
4.82.1.10 HAL_HALL_SetGpio()	327
4.82.1.11 HAL_HALL_SetTh()	327
4.82.1.12 HAL_HALL_StartCounting()	328

4.82.1.13 HAL_HALL_StopCounting()	328
4.83 hal_lks32mc07x_hall.c	328
4.84 source/hal_lks32mc07x_i2c.c 文件参考	330
4.84.1 函数说明	331
4.84.1.1 HAL_I2C_BusReset()	331
4.84.1.2 HAL_I2C_Disable()	332
4.84.1.3 HAL_I2C_Enable()	332
4.84.1.4 HAL_I2C_Reset()	332
4.84.1.5 HAL_I2C_RxData()	332
4.84.1.6 HAL_I2C_SetBaud()	333
4.84.1.7 HAL_I2C_SetGpioScl()	333
4.84.1.8 HAL_I2C_SetGpioSda()	334
4.84.1.9 HAL_I2C_TxData()	335
4.84.2 变量说明	335
4.84.2.1 hal_baud	335
4.84.2.2 hal_i2cio	336
4.85 hal_lks32mc07x_i2c.c	336
4.86 source/hal_lks32mc07x_init.c 文件参考	338
4.86.1 函数说明	339
4.86.1.1 HAL_INIT_PowerReady()	339
4.86.1.2 HAL_INIT_SysInit()	339
4.87 hal_lks32mc07x_init.c	339
4.88 source/hal_lks32mc07x_irq.c 文件参考	340
4.88.1 函数说明	340
4.88.1.1 HAL_IRQ_Disable()	340
4.88.1.2 HAL_IRQ_Enable()	341
4.88.1.3 HAL_IRQ_ModuleDisable()	341
4.88.1.4 HAL_IRQ_ModuleEnable()	342
4.88.1.5 HAL_IRQ_Reset()	342
4.89 hal_lks32mc07x_irq.c	342
4.90 source/hal_lks32mc07x_irqfun.c 文件参考	343
4.90.1 函数说明	344
4.90.1.1 HAL_ADC_IrqSamp1()	344
4.90.1.2 HAL_ADC_IrqSamp1Clear()	345
4.90.1.3 HAL_ADC_IrqSamp1Disable()	345
4.90.1.4 HAL_ADC_IrqSamp1Enable()	345
4.90.1.5 HAL_ADC_IrqSamp1Flg()	346
4.90.1.6 HAL_ADC_IrqSamp2()	346
4.90.1.7 HAL_ADC_IrqSamp2Clear()	346
4.90.1.8 HAL_ADC_IrqSamp2Disable()	347
4.90.1.9 HAL_ADC_IrqSamp2Enable()	347
4.90.1.10 HAL_ADC_IrqSamp2Flg()	347

4.90.1.11 HAL_HALL_IrqChange()	348
4.90.1.12 HAL_HALL_IrqChangeClear()	348
4.90.1.13 HAL_HALL_IrqChangeDisable()	348
4.90.1.14 HAL_HALL_IrqChangeEnable()	348
4.90.1.15 HAL_HALL_IrqChangeFlg()	350
4.90.1.16 HAL_HALL_IrqOver()	350
4.90.1.17 HAL_HALL_IrqOverClear()	350
4.90.1.18 HAL_HALL_IrqOverDisable()	351
4.90.1.19 HAL_HALL_IrqOverEnable()	351
4.90.1.20 HAL_HALL_IrqOverFlg()	351
4.91 hal_lks32mc07x_irqfun.c	352
4.92 source/hal_lks32mc07x_iwdg.c 文件参考	353
4.92.1 函数说明	354
4.92.1.1 HAL_IWDG_Disable()	354
4.92.1.2 HAL_IWDG_Enable()	355
4.92.1.3 HAL_IWDG_Feed()	355
4.92.1.4 HAL_IWDG_Reset()	356
4.92.1.5 HAL_IWDG_SetResetTimemS()	356
4.93 hal_lks32mc07x_iwdg.c	357
4.94 source/hal_lks32mc07x_mcpwm.c 文件参考	357
4.94.1 函数说明	359
4.94.1.1 HAL_MCPWM_Disable()	359
4.94.1.2 HAL_MCPWM_Enable()	360
4.94.1.3 HAL_MCPWM_GetFreq()	360
4.94.1.4 HAL_MCPWM_GetThCnt()	361
4.94.1.5 HAL_MCPWM_IrqFail()	361
4.94.1.6 HAL_MCPWM_IrqFailClear()	361
4.94.1.7 HAL_MCPWM_IrqFailDisable()	361
4.94.1.8 HAL_MCPWM_IrqFailEnable()	362
4.94.1.9 HAL_MCPWM_IrqOverflow()	362
4.94.1.10 HAL_MCPWM_IrqOverflowClear()	362
4.94.1.11 HAL_MCPWM_IrqOverflowDisable()	362
4.94.1.12 HAL_MCPWM_IrqOverflowEnable()	362
4.94.1.13 HAL_MCPWM_IrqZeroCross()	362
4.94.1.14 HAL_MCPWM_IrqZeroCrossClear()	362
4.94.1.15 HAL_MCPWM_IrqZeroCrossDisable()	363
4.94.1.16 HAL_MCPWM_IrqZeroCrossEnable()	363
4.94.1.17 HAL_MCPWM_Reset()	363
4.94.1.18 HAL_MCPWM_SetAdcTrig()	363
4.94.1.19 HAL_MCPWM_SetAdcTrigQ15()	363
4.94.1.20 HAL_MCPWM_SetDead()	364
4.94.1.21 HAL_MCPWM_SetDefaultState()	364

4.94.1.22 HAL_MCPWM_SetDrivePolarity()	365
4.94.1.23 HAL_MCPWM_SetFailSignal()	366
4.94.1.24 HAL_MCPWM_SetFreq()	366
4.94.1.25 HAL_MCPWM_SetGpioFail()	366
4.94.1.26 HAL_MCPWM_SetGpioOut()	367
4.94.1.27 HAL_MCPWM_SetOutputDisable()	367
4.94.1.28 HAL_MCPWM_SetOutputEnable()	368
4.94.1.29 HAL_MCPWM_SetOutState()	368
4.94.1.30 HAL_MCPWM_SetPwm()	369
4.94.1.31 HAL_MCPWM_SetPwmQ15()	369
4.94.1.32 HAL_MCPWM_StartCounting()	370
4.94.1.33 HAL_MCPWM_StopCounting()	370
4.94.1.34 hal_MCPWM_UpdatePolarityState()	370
4.94.2 变量说明	372
4.94.2.1 mcpwm_polarity_state	372
4.95 hal_lks32mc07x_mcpwm.c	373
4.96 source/hal_lks32mc07x_opa.c 文件参考	377
4.96.1 函数说明	379
4.96.1.1 HAL_OPA_Disable()	379
4.96.1.2 HAL_OPA_Enable()	379
4.96.1.3 HAL_OPA_Out2GpioDisable()	379
4.96.1.4 HAL_OPA_Out2GpioEnable()	379
4.96.1.5 HAL_OPA_Reset()	379
4.96.1.6 HAL_OPA_SetGain()	380
4.96.1.7 HAL_OPA_SetGpioIn()	380
4.96.1.8 HAL_OPA_SetGpioOut()	380
4.97 hal_lks32mc07x_opa.c	381
4.98 source/hal_lks32mc07x_qep.c 文件参考	383
4.98.1 函数说明	384
4.98.1.1 HAL_QEP_Disable()	384
4.98.1.2 HAL_QEP_Enable()	384
4.98.1.3 HAL_QEP_ReadCnt()	385
4.98.1.4 HAL_QEP_Reset()	385
4.98.1.5 HAL_QEP_ResetCnt()	385
4.98.1.6 HAL_QEP_SetGpio()	385
4.98.1.7 HAL_QEP_SetMode()	386
4.98.1.8 HAL_QEP_SetTh()	386
4.99 hal_lks32mc07x_qep.c	387
4.100 source/hal_lks32mc07x_reg.c 文件参考	388
4.100.1 变量说明	389
4.100.1.1 HAL_ADC	389
4.100.1.2 HAL_CMP	390

4.100.1.3 HAL_CRC	390
4.100.1.4 HAL_DMA	390
4.100.1.5 HAL_DSP	390
4.100.1.6 HAL_FLASH	390
4.100.1.7 HAL_GPIO	390
4.100.1.8 HAL_HALL	390
4.100.1.9 HAL_I2C	391
4.100.1.10 HAL_IWDG	391
4.100.1.11 HAL_MCPWM	391
4.100.1.12 HAL_QEP	391
4.100.1.13 HAL_SPI	391
4.100.1.14 HAL_SYS	391
4.100.1.15 HAL_TIMER	391
4.100.1.16 HAL_UART	392
4.101 hal_lks32mc07x_reg.c	392
4.102 source/hal_lks32mc07x_reset.c 文件参考	392
4.102.1 函数说明	393
4.102.1.1 HAL_RESET_ResetCore()	393
4.102.1.2 HAL_RESET_ResetGlobal()	393
4.102.1.3 HAL_RESET_ResetRegister()	394
4.103 hal_lks32mc07x_reset.c	394
4.104 source/hal_lks32mc07x_spi.c 文件参考	394
4.104.1 函数说明	396
4.104.1.1 HAL_SPI_Disable()	396
4.104.1.2 HAL_SPI_Enable()	396
4.104.1.3 HAL_SPI_Reset()	396
4.104.1.4 HAL_SPI_RxData()	396
4.104.1.5 HAL_SPI_RxDataWidth16()	398
4.104.1.6 HAL_SPI_RxTxData()	398
4.104.1.7 HAL_SPI_RxTxDataWidth16()	399
4.104.1.8 HAL_SPI_SetBaud()	399
4.104.1.9 HAL_SPI_SetGpioClk()	400
4.104.1.10 HAL_SPI_SetGpioCs()	400
4.104.1.11 HAL_SPI_SetGpioDi()	401
4.104.1.12 HAL_SPI_SetGpioDo()	402
4.104.1.13 HAL_SPI_SetGpioMiso()	402
4.104.1.14 HAL_SPI_SetGpioMosi()	403
4.104.1.15 HAL_SPI_SetMode()	404
4.104.1.16 HAL_SPI_TxData()	404
4.104.1.17 HAL_SPI_TxDataWidth16()	405
4.105 hal_lks32mc07x_spi.c	405
4.106 source/hal_lks32mc07x_test.c 文件参考	408

4.106.1 函数说明	409
4.106.1.1 HAL_TestRegAddr()	409
4.107 hal_lks32mc07x_test.c	409
4.108 source/hal_lks32mc07x_timer.c 文件参考	409
4.108.1 函数说明	411
4.108.1.1 HAL_TIMER_Disable()	411
4.108.1.2 HAL_TIMER_Enable()	411
4.108.1.3 HAL_TIMER_GetFreq()	411
4.108.1.4 HAL_TIMER_GetThCnt()	412
4.108.1.5 HAL_TIMER_IrqCompare0()	412
4.108.1.6 HAL_TIMER_IrqCompare0Clear()	412
4.108.1.7 HAL_TIMER_IrqCompare0Disable()	412
4.108.1.8 HAL_TIMER_IrqCompare0Enable()	412
4.108.1.9 HAL_TIMER_IrqCompare1()	413
4.108.1.10 HAL_TIMER_IrqCompare1Clear()	413
4.108.1.11 HAL_TIMER_IrqCompare1Disable()	413
4.108.1.12 HAL_TIMER_IrqCompare1Enable()	413
4.108.1.13 HAL_TIMER_IrqOverflow()	413
4.108.1.14 HAL_TIMER_IrqOverflowClear()	413
4.108.1.15 HAL_TIMER_IrqOverflowDisable()	413
4.108.1.16 HAL_TIMER_IrqOverflowEnable()	414
4.108.1.17 HAL_TIMER_Reset()	414
4.108.1.18 HAL_TIMER_SetFreq()	414
4.108.1.19 HAL_TIMER_SetGpioIn()	414
4.108.1.20 HAL_TIMER_SetGpioOut()	415
4.108.1.21 HAL_TIMER_SetPeriodCount()	416
4.108.1.22 HAL_TIMER_SetPeriodmS()	416
4.108.1.23 HAL_TIMER_SetPerioduS()	417
4.108.1.24 HAL_TIMER_SetPwm()	417
4.108.1.25 HAL_TIMER_SetPwmQ15()	418
4.108.1.26 HAL_TIMER_SetWorkMode()	418
4.108.1.27 HAL_TIMER_StartCounting()	418
4.108.1.28 HAL_TIMER_StopCounting()	418
4.109 hal_lks32mc07x_timer.c	419
4.110 source/hal_lks32mc07x_tmp.c 文件参考	422
4.110.1 函数说明	424
4.110.1.1 HAL_TMP_Adc2Temperature()	424
4.110.1.2 HAL_TMP_Adc2Temperature01()	424
4.110.1.3 HAL_TMP_Disable()	425
4.110.1.4 HAL_TMP_Enable()	426
4.110.1.5 HAL_TMP_GetTemperature()	426
4.110.1.6 HAL_TMP_GetTemperature01()	427

4.110.1.7 HAL_TMP_Reset()	427
4.110.2 变量说明	428
4.110.2.1 tmp_coef_a	428
4.110.2.2 tmp_coef_b	428
4.111 hal_lks32mc07x_tmp.c	428
4.112 source/hal_lks32mc07x_trim.c 文件参考	429
4.112.1 函数说明	430
4.112.1.1 HAL_TRIM_Read()	430
4.112.1.2 HAL_TRIM_ReadId()	431
4.112.1.3 HAL_TRIM_ReadIdCrc32()	432
4.112.1.4 HAL_TRIM_ReadMcu07()	432
4.113 hal_lks32mc07x_trim.c	433
4.114 source/hal_lks32mc07x_uart.c 文件参考	434
4.114.1 函数说明	435
4.114.1.1 HAL_UART_Disable()	435
4.114.1.2 HAL_UART_Enable()	435
4.114.1.3 HAL_UART_IrqCheckErr()	435
4.114.1.4 HAL_UART_IrqCheckErrClear()	435
4.114.1.5 HAL_UART_IrqCheckErrDisable()	435
4.114.1.6 HAL_UART_IrqCheckErrEnable()	436
4.114.1.7 HAL_UART_IrqReceiveOver()	436
4.114.1.8 HAL_UART_IrqReceiveOverClear()	436
4.114.1.9 HAL_UART_IrqReceiveOverDisable()	436
4.114.1.10 HAL_UART_IrqReceiveOverEnable()	436
4.114.1.11 HAL_UART_IrqSendOver()	436
4.114.1.12 HAL_UART_IrqSendOverClear()	436
4.114.1.13 HAL_UART_IrqSendOverDisable()	437
4.114.1.14 HAL_UART_IrqSendOverEnable()	437
4.114.1.15 HAL_UART_IrqStopErr()	437
4.114.1.16 HAL_UART_IrqStopErrClear()	437
4.114.1.17 HAL_UART_IrqStopErrDisable()	437
4.114.1.18 HAL_UART_IrqStopErrEnable()	437
4.114.1.19 HAL_UART_ReadBuffer()	437
4.114.1.20 HAL_UART_Reset()	438
4.114.1.21 HAL_UART_SetBaud()	438
4.114.1.22 HAL_UART_SetGpioRx()	438
4.114.1.23 HAL_UART_SetGpioTx()	439
4.114.1.24 HAL_UART_SetMode()	439
4.114.1.25 HAL_UART_TxData()	440
4.114.1.26 HAL_UART_WriteBuffer()	440
4.115 hal_lks32mc07x_uart.c	440
4.116 source/hal_lks32mc07x_wake.c 文件参考	444

4.116.1 函数说明	445
4.116.1.1 HAL_WAKE_DeepSleepToTimeWakeUp()	445
4.116.1.2 HAL_WAKE_DeepSleepToWake()	445
4.116.1.3 HAL_WAKE_DeepSleepTimeWakeUp()	445
4.117 hal_lks32mc07x_wake.c	446

Chapter 1

结构体索引

1.1 结构体

这里列出了所有结构体，并附带简要说明:

CMP_TypeDef	
ADC寄存器结构体句柄	5
CRC_Model_Params	6
HAL_ADC_TypeDef	
ADC0寄存器定义	7
HAL_CMP_TypeDef	
CMP寄存器定义	13
HAL_CRC_TypeDef	
CRC寄存器定义	14
HAL_DMA_TypeDef	16
HAL_DSP_TypeDef	
DSP寄存器定义	17
HAL_FLASH_TypeDef	
FLASH寄存器定义	20
HAL_GPIO_TypeDef	
GPIO寄存器定义	23
HAL_HALL_TypeDef	
HALL寄存器定义	26
HAL_I2C_TypeDef	
I2C寄存器定义	27
HAL_IWDG_TypeDef	29
HAL_MCPWM_TypeDef	
MCPWM寄存器定义	30
HAL_QEP_TypeDef	
QEP寄存器定义	39
HAL_SPI_TypeDef	
SPI寄存器定义	40
HAL_SYS_TypeDef	
SYS寄存器定义	42
HAL_TIMER_TypeDef	
TIMER寄存器定义	50
HAL_UART_TypeDef	
UART寄存器定义	52

Chapter 2

文件索引

2.1 文件列表

这里列出了所有文件，并附带简要说明:

hal_lks32mc07x.c	55
hal_lks32mc07x.h	56
hal_lksmcu.h	68
include/hal_lks32mc07x_reg.h	69
include/hal_lksmcu_adc.h	76
include/hal_lksmcu_can.h	92
include/hal_lksmcu_clk.h	93
include/hal_lksmcu_cmp.h	96
include/hal_lksmcu_crc.h	105
include/hal_lksmcu_dac.h	108
include/hal_lksmcu_dma.h	112
include/hal_lksmcu_dsp.h	116
include/hal_lksmcu_flash.h	119
include/hal_lksmcu_gpio.h	129
include/hal_lksmcu_hall.h	143
include/hal_lksmcu_i2c.h	151
include/hal_lksmcu_init.h	157
include/hal_lksmcu_irq.h	158
include/hal_lksmcu_irqfun.h	163
include/hal_lksmcu_iwdg.h	171
include/hal_lksmcu_mcpwm.h	175
include/hal_lksmcu_opa.h	192
include/hal_lksmcu_qep.h	199
include/hal_lksmcu_reg.h	
HAL库配置和内部参数	204
include/hal_lksmcu_reset.h	207
include/hal_lksmcu_spi.h	209
include/hal_lksmcu_timer.h	221
include/hal_lksmcu_tmp.h	232
include/hal_lksmcu_trim.h	239
include/hal_lksmcu_uart.h	242
include/hal_lksmcu_wake.h	250
source/hal_lks32mc07x_adc.c	
虚拟实现,用来在芯片之外约束函数运行效果	253
source/hal_lks32mc07x_clk.c	269

source/hal_lks32mc07x_cmp.c	273
source/hal_lks32mc07x_crc.c	280
source/hal_lks32mc07x_dac.c	284
source/hal_lks32mc07x_dma.c	290
source/hal_lks32mc07x_dsp.c	295
source/hal_lks32mc07x_flash.c	298
source/hal_lks32mc07x_gpio.c	311
source/hal_lks32mc07x_hall.c	322
source/hal_lks32mc07x_i2c.c	330
source/hal_lks32mc07x_init.c	338
source/hal_lks32mc07x_irq.c	340
source/hal_lks32mc07x_irqfun.c	343
source/hal_lks32mc07x_iwdg.c	353
source/hal_lks32mc07x_mcpwm.c	357
source/hal_lks32mc07x_opa.c	377
source/hal_lks32mc07x_qep.c	383
source/hal_lks32mc07x_reg.c	388
source/hal_lks32mc07x_reset.c	392
source/hal_lks32mc07x_spi.c	394
source/hal_lks32mc07x_test.c	408
source/hal_lks32mc07x_timer.c	409
source/hal_lks32mc07x_tmp.c	422
source/hal_lks32mc07x_trim.c	429
source/hal_lks32mc07x_uart.c	434
source/hal_lks32mc07x_wake.c	444

Chapter 3

结构体说明

3.1 CMP_TypeDef结构体 参考

ADC寄存器结构体句柄

成员变量

- volatile uint32_t **IE**
0x00 比较器中断使能寄存器
- volatile uint32_t **IF**
0x04 比较器中断标志寄存器
- volatile uint32_t **TCLK**
0x08 比较器分频时钟控制寄存器
- volatile uint32_t **CFG**
0x0C 比较器控制寄存器
- volatile uint32_t **BLCWIN**
0x10 比较器开窗控制寄存器
- volatile uint32_t **DATA**
0x14 比较器输出数值寄存器

3.1.1 详细描述

ADC寄存器结构体句柄

在文件 [hal_lks32mc07x_cmp.c](#) 第 9 行定义.

3.1.2 结构体成员变量说明

3.1.2.1 BLCWIN

volatile uint32_t BLCWIN

0x10 比较器开窗控制寄存器

在文件 [hal_lks32mc07x_cmp.c](#) 第 15 行定义.

3.1.2.2 CFG

`volatile uint32_t CFG`

0x0C 比较器控制寄存器

在文件 [hal_lks32mc07x_cmp.c](#) 第 14 行定义.

3.1.2.3 DATA

`volatile uint32_t DATA`

0x14 比较器输出数值寄存器

在文件 [hal_lks32mc07x_cmp.c](#) 第 16 行定义.

3.1.2.4 IE

`volatile uint32_t IE`

0x00 比较器中断使能寄存器

在文件 [hal_lks32mc07x_cmp.c](#) 第 11 行定义.

3.1.2.5 IF

`volatile uint32_t IF`

0x04 比较器中断标志寄存器

在文件 [hal_lks32mc07x_cmp.c](#) 第 12 行定义.

3.1.2.6 TCLK

`volatile uint32_t TCLK`

0x08 比较器分频时钟控制寄存器

在文件 [hal_lks32mc07x_cmp.c](#) 第 13 行定义.

3.2 CRC_Model_Params结构体 参考

成员变量

- `uint32_t polynomial`
- `uint32_t initValue`
- `uint32_t xorOut`
- `uint8_t reflectIn`
- `uint8_t reflectOut`

3.2.1 详细描述

在文件 [hal_lks32mc07x_crc.c](#) 第 20 行定义.

3.2.2 结构体成员变量说明

3.2.2.1 initValue

```
uint32_t initValue
```

在文件 [hal_lks32mc07x_crc.c](#) 第 23 行定义.

3.2.2.2 polynomial

```
uint32_t polynomial
```

在文件 [hal_lks32mc07x_crc.c](#) 第 22 行定义.

3.2.2.3 reflectIn

```
uint8_t reflectIn
```

在文件 [hal_lks32mc07x_crc.c](#) 第 25 行定义.

3.2.2.4 reflectOut

```
uint8_t reflectOut
```

在文件 [hal_lks32mc07x_crc.c](#) 第 26 行定义.

3.2.2.5 xorOut

```
uint32_t xorOut
```

在文件 [hal_lks32mc07x_crc.c](#) 第 24 行定义.

3.3 HAL_ADC_TypeDef结构体 参考

ADC0寄存器定义

```
#include <hal_lks32mc07x_reg.h>
```

成员变量

- volatile uint32_t DAT [14]
第 0 次常规采样数据
- volatile uint32_t IDAT [2]
第 0 次空闲采样数据
- volatile uint32_t ICHN
空闲采样通道配置
- volatile uint32_t null [3]
地址对齐,无意义
- volatile uint32_t CHN0
第 0~3 次常规采样输入信号选择
- volatile uint32_t CHN1
第 4~7 次常规采样输入信号选择
- volatile uint32_t CHN2
第 8~11 次常规采样输入信号选择
- volatile uint32_t CHN3
第 12~13 次常规采样输入信号选择
- volatile uint32_t CHNT
各种触发模式下采样次数
- volatile uint32_t GAIN
增益选择
- volatile uint32_t null2 [3]
地址对齐,无意义
- volatile uint32_t CFG
模式配置
- volatile uint32_t TRIG
采样触发配置
- volatile uint32_t SWT
软件触发
- volatile uint32_t DC0
增益为 0 时 DC offset
- volatile uint32_t AMC0
增益为 0 时增益校正系数
- volatile uint32_t DC1
增益为 1 时 DC offset
- volatile uint32_t AMC1
增益为 1 时增益校正系数
- volatile uint32_t IE
中断使能
- volatile uint32_t IF
中断标志
- volatile uint32_t null3 [11]
地址对齐,无意义
- volatile uint32_t LTH
数据低阈值
- volatile uint32_t HTH
数据高阈值
- volatile uint32_t GEN
阈值监测使能

3.3.1 详细描述

ADC0寄存器定义

在文件 [hal_lks32mc07x_reg.h](#) 第 141 行定义.

3.3.2 结构体成员变量说明

3.3.2.1 AMC0

```
volatile uint32_t AMC0
```

增益为 0 时增益校正系数

在文件 [hal_lks32mc07x_reg.h](#) 第 158 行定义.

3.3.2.2 AMC1

```
volatile uint32_t AMC1
```

增益为 1 时增益校正系数

在文件 [hal_lks32mc07x_reg.h](#) 第 160 行定义.

3.3.2.3 CFG

```
volatile uint32_t CFG
```

模式配置

在文件 [hal_lks32mc07x_reg.h](#) 第 154 行定义.

3.3.2.4 CHN0

```
volatile uint32_t CHN0
```

第 0~3 次常规采样输入信号选择

在文件 [hal_lks32mc07x_reg.h](#) 第 147 行定义.

3.3.2.5 CHN1

```
volatile uint32_t CHN1
```

第 4~7 次常规采样输入信号选择

在文件 [hal_lks32mc07x_reg.h](#) 第 148 行定义.

3.3.2.6 CHN2

```
volatile uint32_t CHN2
```

第 8~11 次常规采样输入信号选择

在文件 [hal_lks32mc07x_reg.h](#) 第 149 行定义.

3.3.2.7 CHN3

```
volatile uint32_t CHN3
```

第 12~13 次常规采样输入信号选择

在文件 [hal_lks32mc07x_reg.h](#) 第 150 行定义.

3.3.2.8 CHNT

```
volatile uint32_t CHNT
```

各种触发模式下采样次数

在文件 [hal_lks32mc07x_reg.h](#) 第 151 行定义.

3.3.2.9 DAT

```
volatile uint32_t DAT[14]
```

第 0 次常规采样数据

在文件 [hal_lks32mc07x_reg.h](#) 第 143 行定义.

3.3.2.10 DC0

```
volatile uint32_t DC0
```

增益为 0 时 DC offset

在文件 [hal_lks32mc07x_reg.h](#) 第 157 行定义.

3.3.2.11 DC1

```
volatile uint32_t DC1
```

增益为 1 时 DC offset

在文件 [hal_lks32mc07x_reg.h](#) 第 159 行定义.

3.3.2.12 GAIN

```
volatile uint32_t GAIN
```

增益选择

在文件 [hal_lks32mc07x_reg.h](#) 第 152 行定义.

3.3.2.13 GEN

```
volatile uint32_t GEN
```

阈值监测使能

在文件 [hal_lks32mc07x_reg.h](#) 第 166 行定义.

3.3.2.14 HTH

```
volatile uint32_t HTH
```

数据高阈值

在文件 [hal_lks32mc07x_reg.h](#) 第 165 行定义.

3.3.2.15 ICHN

```
volatile uint32_t ICHN
```

空闲采样通道配置

在文件 [hal_lks32mc07x_reg.h](#) 第 145 行定义.

3.3.2.16 IDAT

```
volatile uint32_t IDAT[2]
```

第 0 次空闲采样数据

在文件 [hal_lks32mc07x_reg.h](#) 第 144 行定义.

3.3.2.17 IE

```
volatile uint32_t IE
```

中断使能

在文件 [hal_lks32mc07x_reg.h](#) 第 161 行定义.

3.3.2.18 IF

```
volatile uint32_t IF
```

中断标志

在文件 [hal_lks32mc07x_reg.h](#) 第 162 行定义.

3.3.2.19 LTH

```
volatile uint32_t LTH
```

数据低阈值

在文件 [hal_lks32mc07x_reg.h](#) 第 164 行定义.

3.3.2.20 null

```
volatile uint32_t null[3]
```

地址对齐,无意义

在文件 [hal_lks32mc07x_reg.h](#) 第 146 行定义.

3.3.2.21 null2

```
volatile uint32_t null2[3]
```

地址对齐,无意义

在文件 [hal_lks32mc07x_reg.h](#) 第 153 行定义.

3.3.2.22 null3

```
volatile uint32_t null3[11]
```

地址对齐,无意义

在文件 [hal_lks32mc07x_reg.h](#) 第 163 行定义.

3.3.2.23 SWT

```
volatile uint32_t SWT
```

软件触发

在文件 [hal_lks32mc07x_reg.h](#) 第 156 行定义.

3.3.2.24 TRIG

```
volatile uint32_t TRIG
```

采样触发配置

在文件 [hal_lks32mc07x_reg.h](#) 第 155 行定义.

3.4 HAL_CMP_TypeDef结构体 参考

CMP寄存器定义

```
#include <hal_lks32mc07x_reg.h>
```

成员变量

- volatile uint32_t [IE](#)
0x00 中断使能寄存器
- volatile uint32_t [IF](#)
0x04 中断标志寄存器
- volatile uint32_t [TCLK](#)
0x08 时钟分频控制寄存器
- volatile uint32_t [CFG](#)
0x0C 配置寄存器
- volatile uint32_t [BLCWIN](#)
0x10 黑窗口控制寄存器
- volatile uint32_t [DATA](#)
0x14 数据寄存器

3.4.1 详细描述

CMP寄存器定义

在文件 [hal_lks32mc07x_reg.h](#) 第 116 行定义.

3.4.2 结构体成员变量说明

3.4.2.1 BLCWIN

```
volatile uint32_t BLCWIN
```

0x10 黑窗口控制寄存器

在文件 [hal_lks32mc07x_reg.h](#) 第 122 行定义.

3.4.2.2 CFG

```
volatile uint32_t CFG
```

0x0C 配置寄存器

在文件 [hal_lks32mc07x_reg.h](#) 第 121 行定义.

3.4.2.3 DATA

```
volatile uint32_t DATA
```

0x14 数据寄存器

在文件 [hal_lks32mc07x_reg.h](#) 第 123 行定义.

3.4.2.4 IE

```
volatile uint32_t IE
```

0x00 中断使能寄存器

在文件 [hal_lks32mc07x_reg.h](#) 第 118 行定义.

3.4.2.5 IF

```
volatile uint32_t IF
```

0x04 中断标志寄存器

在文件 [hal_lks32mc07x_reg.h](#) 第 119 行定义.

3.4.2.6 TCLK

```
volatile uint32_t TCLK
```

0x08 时钟分频控制寄存器

在文件 [hal_lks32mc07x_reg.h](#) 第 120 行定义.

3.5 HAL_CRC_TypeDef结构体 参考

CRC寄存器定义

```
#include <hal_lks32mc07x_reg.h>
```

成员变量

- volatile uint32_t **DR**
0x00 CRC数据
- volatile uint32_t **CR**
0x04 CRC控制
- volatile uint32_t **INIT**
0x08 CRC初始值
- volatile uint32_t **POL**
0x0C CRC系数

3.5.1 详细描述

CRC寄存器定义

在文件 [hal_lks32mc07x_reg.h](#) 第 269 行定义.

3.5.2 结构体成员变量说明

3.5.2.1 CR

```
volatile uint32_t CR
```

0x04 CRC控制

在文件 [hal_lks32mc07x_reg.h](#) 第 272 行定义.

3.5.2.2 DR

```
volatile uint32_t DR
```

0x00 CRC数据

在文件 [hal_lks32mc07x_reg.h](#) 第 271 行定义.

3.5.2.3 INIT

```
volatile uint32_t INIT
```

0x08 CRC初始值

在文件 [hal_lks32mc07x_reg.h](#) 第 273 行定义.

3.5.2.4 POL

```
volatile uint32_t POL
```

0x0C CRC系数

在文件 [hal_lks32mc07x_reg.h](#) 第 274 行定义.

3.6 HAL_DMA_TypeDef结构体 参考

```
#include <hal_lks32mc07x_reg.h>
```

成员变量

- volatile uint32_t **CCR**
DMA 通道配置寄存器
- volatile uint32_t **REN**
DMA 请求使能寄存器
- volatile uint32_t **CTMS**
DMA传输次数设置寄存器
- volatile uint32_t **SADR**
DMA 源地址设置寄存器
- volatile uint32_t **DADR**
DMA 目的地址设置

3.6.1 详细描述

在文件 [hal_lks32mc07x_reg.h](#) 第 326 行定义.

3.6.2 结构体成员变量说明

3.6.2.1 CCR

```
volatile uint32_t CCR
```

DMA 通道配置寄存器

在文件 [hal_lks32mc07x_reg.h](#) 第 328 行定义.

3.6.2.2 CTMS

```
volatile uint32_t CTMS
```

DMA传输次数设置寄存器

在文件 [hal_lks32mc07x_reg.h](#) 第 330 行定义.

3.6.2.3 DADR

```
volatile uint32_t DADR
```

DMA 目的地址设置

在文件 [hal_lks32mc07x_reg.h](#) 第 332 行定义.

3.6.2.4 REN

```
volatile uint32_t REN
```

DMA 请求使能寄存器

在文件 [hal_lks32mc07x_reg.h](#) 第 329 行定义.

3.6.2.5 SADR

```
volatile uint32_t SADR
```

DMA 源地址设置寄存器

在文件 [hal_lks32mc07x_reg.h](#) 第 331 行定义.

3.7 HAL_DSP_TypeDef结构体 参考

DSP寄存器定义

```
#include <hal_lks32mc07x_reg.h>
```

成员变量

- volatile uint32_t [SC](#)
- volatile uint32_t [THETA](#)
- volatile uint32_t [X](#)
- volatile uint32_t [Y](#)
- volatile uint32_t [SIN](#)
- volatile uint32_t [COS](#)
- volatile uint32_t [MOD](#)
- volatile uint32_t [ARCTAN](#)
- volatile uint32_t [DID](#)
- volatile uint32_t [DIS](#)
- volatile uint32_t [QUO](#)
- volatile uint32_t [REM](#)
- volatile uint32_t [RAD](#)
- volatile uint32_t [SQRT](#)
- volatile uint32_t [PC](#)

3.7.1 详细描述

DSP寄存器定义

在文件 [hal_lks32mc07x_reg.h](#) 第 297 行定义.

3.7.2 结构体成员变量说明

3.7.2.1 ARCTAN

```
volatile uint32_t ARCTAN
```

在文件 [hal_lks32mc07x_reg.h](#) 第 306 行定义.

3.7.2.2 COS

```
volatile uint32_t COS
```

在文件 [hal_lks32mc07x_reg.h](#) 第 304 行定义.

3.7.2.3 DID

```
volatile uint32_t DID
```

在文件 [hal_lks32mc07x_reg.h](#) 第 307 行定义.

3.7.2.4 DIS

```
volatile uint32_t DIS
```

在文件 [hal_lks32mc07x_reg.h](#) 第 308 行定义.

3.7.2.5 MOD

```
volatile uint32_t MOD
```

在文件 [hal_lks32mc07x_reg.h](#) 第 305 行定义.

3.7.2.6 PC

```
volatile uint32_t PC
```

在文件 [hal_lks32mc07x_reg.h](#) 第 313 行定义.

3.7.2.7 QUO

```
volatile uint32_t QUO
```

在文件 [hal_lks32mc07x_reg.h](#) 第 309 行定义.

3.7.2.8 RAD

```
volatile uint32_t RAD
```

在文件 [hal_lks32mc07x_reg.h](#) 第 311 行定义.

3.7.2.9 REM

```
volatile uint32_t REM
```

在文件 [hal_lks32mc07x_reg.h](#) 第 310 行定义.

3.7.2.10 SC

```
volatile uint32_t SC
```

在文件 [hal_lks32mc07x_reg.h](#) 第 299 行定义.

3.7.2.11 SIN

```
volatile uint32_t SIN
```

在文件 [hal_lks32mc07x_reg.h](#) 第 303 行定义.

3.7.2.12 SQRT

```
volatile uint32_t SQRT
```

在文件 [hal_lks32mc07x_reg.h](#) 第 312 行定义.

3.7.2.13 THETA

```
volatile uint32_t THETA
```

在文件 [hal_lks32mc07x_reg.h](#) 第 300 行定义.

3.7.2.14 X

```
volatile uint32_t X
```

在文件 [hal_lks32mc07x_reg.h](#) 第 301 行定义.

3.7.2.15 Y

```
volatile uint32_t Y
```

在文件 [hal_lks32mc07x_reg.h](#) 第 302 行定义.

3.8 HAL_FLASH_TypeDef结构体 参考

FLASH寄存器定义

```
#include <hal_lks32mc07x_reg.h>
```

成员变量

- volatile uint32_t **CFG**
0x00 FLASH配置寄存器
- volatile uint32_t **ADDR**
0x04 地址寄存器
- volatile uint32_t **WDATA**
0x08 写数据寄存器
- volatile uint32_t **RDATA**
0x0C 读数据寄存器
- volatile uint32_t **ERASE**
0x10 擦除控制寄存器
- volatile uint32_t **PROTECT**
0x14 FLASH保护状态寄存器
- volatile uint32_t **READY**
0x18 FLASH工作状态寄存器
- volatile uint32_t **SIZE**
0x1C FLASH容量状态寄存器
- volatile uint32_t **NCFG**
0x20 FLASH配置寄存器（保护状态下，针对NVR操作）
- volatile uint32_t **NADDR**
0x24 地址寄存器（保护状态下，针对NVR操作）
- volatile uint32_t **NWDATA**
0x28 写数据寄存器（保护状态下，针对NVR操作）
- volatile uint32_t **NRDATA**
0x2C 读数据寄存器（保护状态下，针对NVR操作）
- volatile uint32_t **NERASE**
0x30 擦除控制寄存器（保护状态下，针对NVR操作）
- volatile uint32_t **NKEY**
0x34 FLASH密钥寄存器（保护状态下，针对NVR操作）

3.8.1 详细描述

FLASH寄存器定义

在文件 [hal_lks32mc07x_reg.h](#) 第 69 行定义.

3.8.2 结构体成员变量说明

3.8.2.1 ADDR

```
volatile uint32_t ADDR
```

0x04 地址寄存器

在文件 [hal_lks32mc07x_reg.h](#) 第 72 行定义.

3.8.2.2 CFG

```
volatile uint32_t CFG
```

0x00 FLASH配置寄存器

在文件 [hal_lks32mc07x_reg.h](#) 第 71 行定义.

3.8.2.3 ERASE

```
volatile uint32_t ERASE
```

0x10 擦除控制寄存器

在文件 [hal_lks32mc07x_reg.h](#) 第 75 行定义.

3.8.2.4 NADDR

```
volatile uint32_t NADDR
```

0x24 地址寄存器（保护状态下，针对NVR操作）

在文件 [hal_lks32mc07x_reg.h](#) 第 80 行定义.

3.8.2.5 NCFG

```
volatile uint32_t NCFG
```

0x20 FLASH配置寄存器（保护状态下，针对NVR操作）

在文件 [hal_lks32mc07x_reg.h](#) 第 79 行定义.

3.8.2.6 NERASE

```
volatile uint32_t NERASE
```

0x30 擦除控制寄存器（保护状态下，针对NVR操作）

在文件 [hal_lks32mc07x_reg.h](#) 第 83 行定义.

3.8.2.7 NKEY

```
volatile uint32_t NKEY
```

0x34 FLASH密钥寄存器（保护状态下，针对NVR操作）

在文件 [hal_lks32mc07x_reg.h](#) 第 84 行定义.

3.8.2.8 NRDATA

```
volatile uint32_t NRDATA
```

0x2C 读数据寄存器（保护状态下，针对NVR操作）

在文件 [hal_lks32mc07x.reg.h](#) 第 82 行定义.

3.8.2.9 NWDATA

```
volatile uint32_t NWDATA
```

0x28 写数据寄存器（保护状态下，针对NVR操作）

在文件 [hal_lks32mc07x.reg.h](#) 第 81 行定义.

3.8.2.10 PROTECT

```
volatile uint32_t PROTECT
```

0x14 FLASH保护状态寄存器

在文件 [hal_lks32mc07x.reg.h](#) 第 76 行定义.

3.8.2.11 RDATA

```
volatile uint32_t RDATA
```

0x0C 读数据寄存器

在文件 [hal_lks32mc07x.reg.h](#) 第 74 行定义.

3.8.2.12 READY

```
volatile uint32_t READY
```

0x18 FLASH工作状态寄存器

在文件 [hal_lks32mc07x.reg.h](#) 第 77 行定义.

3.8.2.13 SIZE

```
volatile uint32_t SIZE
```

0x1C FLASH容量状态寄存器

在文件 [hal_lks32mc07x.reg.h](#) 第 78 行定义.

3.8.2.14 WDATA

```
volatile uint32_t WDATA
```

0x08 写数据寄存器

在文件 [hal_lks32mc07x_reg.h](#) 第 73 行定义.

3.9 HAL_GPIO_TypeDef结构体 参考

GPIO寄存器定义

```
#include <hal_lks32mc07x_reg.h>
```

成员变量

- volatile uint32_t [PIE](#)
输入使能
- volatile uint32_t [POE](#)
输出使能
- volatile uint32_t [PDI](#)
输入数据
- volatile uint32_t [PDO](#)
输出数据
- volatile uint32_t [PUE](#)
上拉使能
- volatile uint32_t [null1](#)
- volatile uint32_t [PODE](#)
开漏使能
- volatile uint32_t [PFLT](#)
滤波使能
- volatile uint32_t [F3210](#)
功能选择 *GPIO_Pin_0-3*
- volatile uint32_t [F7654](#)
功能选择 *GPIO_Pin_4-7*
- volatile uint32_t [FBA98](#)
功能选择 *GPIO_Pin_8-11*
- volatile uint32_t [FFEDC](#)
功能选择 *GPIO_Pin_12-15*
- volatile uint32_t [BSRR](#)
位置一操作
- volatile uint32_t [BRR](#)
位清零操作

3.9.1 详细描述

GPIO寄存器定义

在文件 [hal_lks32mc07x_reg.h](#) 第 248 行定义.

3.9.2 结构体成员变量说明

3.9.2.1 BRR

```
volatile uint32_t BRR
```

位清零操作

在文件 [hal_lks32mc07x_reg.h](#) 第 263 行定义.

3.9.2.2 BSRR

```
volatile uint32_t BSRR
```

位置一操作

在文件 [hal_lks32mc07x_reg.h](#) 第 262 行定义.

3.9.2.3 F3210

```
volatile uint32_t F3210
```

功能选择 GPIO.Pin_0-3

在文件 [hal_lks32mc07x_reg.h](#) 第 258 行定义.

3.9.2.4 F7654

```
volatile uint32_t F7654
```

功能选择 GPIO.Pin_4-7

在文件 [hal_lks32mc07x_reg.h](#) 第 259 行定义.

3.9.2.5 FBA98

```
volatile uint32_t FBA98
```

功能选择 GPIO.Pin_8-11

在文件 [hal_lks32mc07x_reg.h](#) 第 260 行定义.

3.9.2.6 FFEDC

```
volatile uint32_t FFEDC
```

功能选择 GPIO.Pin_12-15

在文件 [hal_lks32mc07x_reg.h](#) 第 261 行定义.

3.9.2.7 null1

```
volatile uint32_t null1
```

在文件 [hal_lks32mc07x_reg.h](#) 第 255 行定义.

3.9.2.8 PDI

```
volatile uint32_t PDI
```

输入数据

在文件 [hal_lks32mc07x_reg.h](#) 第 252 行定义.

3.9.2.9 PDO

```
volatile uint32_t PDO
```

输出数据

在文件 [hal_lks32mc07x_reg.h](#) 第 253 行定义.

3.9.2.10 PFLT

```
volatile uint32_t PFLT
```

滤波使能

在文件 [hal_lks32mc07x_reg.h](#) 第 257 行定义.

3.9.2.11 PIE

```
volatile uint32_t PIE
```

输入使能

在文件 [hal_lks32mc07x_reg.h](#) 第 250 行定义.

3.9.2.12 PODE

```
volatile uint32_t PODE
```

开漏使能

在文件 [hal_lks32mc07x_reg.h](#) 第 256 行定义.

3.9.2.13 POE

```
volatile uint32_t POE
```

输出使能

在文件 [hal_lks32mc07x_reg.h](#) 第 251 行定义.

3.9.2.14 PUE

```
volatile uint32_t PUE
```

上拉使能

在文件 [hal_lks32mc07x_reg.h](#) 第 254 行定义.

3.10 HAL_HALL_TypeDef结构体 参考

HALL寄存器定义

```
#include <hal_lks32mc07x_reg.h>
```

成员变量

- volatile uint32_t [CFG](#)
0x00 配置寄存器
- volatile uint32_t [INFO](#)
0x04 信息寄存器
- volatile uint32_t [WIDTH](#)
0x08 宽度寄存器
- volatile uint32_t [TH](#)
0x0C 阈值寄存器
- volatile uint32_t [CNT](#)
0x10 计数寄存器

3.10.1 详细描述

HALL寄存器定义

在文件 [hal_lks32mc07x_reg.h](#) 第 129 行定义.

3.10.2 结构体成员变量说明

3.10.2.1 CFG

```
volatile uint32_t CFG
```

0x00 配置寄存器

在文件 [hal_lks32mc07x_reg.h](#) 第 131 行定义.

3.10.2.2 CNT

`volatile uint32_t CNT`

0x10 计数寄存器

在文件 [hal_lks32mc07x_reg.h](#) 第 135 行定义.

3.10.2.3 INFO

`volatile uint32_t INFO`

0x04 信息寄存器

在文件 [hal_lks32mc07x_reg.h](#) 第 132 行定义.

3.10.2.4 TH

`volatile uint32_t TH`

0x0C 阈值寄存器

在文件 [hal_lks32mc07x_reg.h](#) 第 134 行定义.

3.10.2.5 WIDTH

`volatile uint32_t WIDTH`

0x08 宽度寄存器

在文件 [hal_lks32mc07x_reg.h](#) 第 133 行定义.

3.11 HAL_I2C_TypeDef结构体 参考

I2C寄存器定义

```
#include <hal_lks32mc07x_reg.h>
```

成员变量

- `volatile uint32_t ADDR`
0x00 I2C 地址寄存器
- `volatile uint32_t CFG`
0x04 I2C 配置寄存器
- `volatile uint32_t SCR`
0x08 I2C 状态寄存器
- `volatile uint32_t DATA`
0x0C I2C 数据寄存器
- `volatile uint32_t MSCR`
0x10 I2C 主模式寄存器
- `volatile uint32_t BCR`
0x14 I2C 传输控制寄存器
- `volatile uint32_t BSIZE`
0x18 I2C 传输长度寄存器

3.11.1 详细描述

I2C寄存器定义

在文件 [hal_lks32mc07x_reg.h](#) 第 102 行定义.

3.11.2 结构体成员变量说明

3.11.2.1 ADDR

```
volatile uint32_t ADDR
```

0x00 I2C 地址寄存器

在文件 [hal_lks32mc07x_reg.h](#) 第 104 行定义.

3.11.2.2 BCR

```
volatile uint32_t BCR
```

0x14 I2C 传输控制寄存器

在文件 [hal_lks32mc07x_reg.h](#) 第 109 行定义.

3.11.2.3 BSIZE

```
volatile uint32_t BSIZE
```

0x18 I2C 传输长度寄存器

在文件 [hal_lks32mc07x_reg.h](#) 第 110 行定义.

3.11.2.4 CFG

```
volatile uint32_t CFG
```

0x04 I2C 配置寄存器

在文件 [hal_lks32mc07x_reg.h](#) 第 105 行定义.

3.11.2.5 DATA

```
volatile uint32_t DATA
```

0x0C I2C 数据寄存器

在文件 [hal_lks32mc07x_reg.h](#) 第 107 行定义.

3.11.2.6 MSCR

`volatile uint32_t MSCR`

0x10 I2C 主模式寄存器

在文件 [hal_lks32mc07x_reg.h](#) 第 108 行定义.

3.11.2.7 SCR

`volatile uint32_t SCR`

0x08 I2C 状态寄存器

在文件 [hal_lks32mc07x_reg.h](#) 第 106 行定义.

3.12 HAL_IWDG_TypeDef结构体 参考

```
#include <hal_lks32mc07x_reg.h>
```

成员变量

- `volatile uint32_t PSW`
0x00 独立看门狗密码寄存器
- `volatile uint32_t CFG`
0x04 独立看门狗配置寄存器
- `volatile uint32_t CLR`
0x08 独立看门狗清零寄存器
- `volatile uint32_t WTH`
0x0C 独立看门狗计数器定时唤醒门限值寄存器
- `volatile uint32_t RTH`
0x10 独立看门狗计数器超时复位门限值寄存器
- `volatile uint32_t CNT`
0x14 独立看门狗计数器当前计数值寄存器

3.12.1 详细描述

在文件 [hal_lks32mc07x_reg.h](#) 第 316 行定义.

3.12.2 结构体成员变量说明

3.12.2.1 CFG

`volatile uint32_t CFG`

0x04 独立看门狗配置寄存器

在文件 [hal_lks32mc07x_reg.h](#) 第 319 行定义.

3.12.2.2 CLR

```
volatile uint32_t CLR
```

0x08 独立看门狗清零寄存器

在文件 [hal_lks32mc07x_reg.h](#) 第 320 行定义.

3.12.2.3 CNT

```
volatile uint32_t CNT
```

0x14 独立看门狗计数器当前计数值寄存器

在文件 [hal_lks32mc07x_reg.h](#) 第 323 行定义.

3.12.2.4 PSW

```
volatile uint32_t PSW
```

0x00 独立看门狗密码寄存器

在文件 [hal_lks32mc07x_reg.h](#) 第 318 行定义.

3.12.2.5 RTH

```
volatile uint32_t RTH
```

0x10 独立看门狗计数器超时复位门限值寄存器

在文件 [hal_lks32mc07x_reg.h](#) 第 322 行定义.

3.12.2.6 WTH

```
volatile uint32_t WTH
```

0x0C 独立看门狗计数器定时唤醒门限值寄存器

在文件 [hal_lks32mc07x_reg.h](#) 第 321 行定义.

3.13 HAL_MCPWM_TypeDef结构体 参考

MCPWM寄存器定义

```
#include <hal_lks32mc07x_reg.h>
```

成员变量

- volatile uint32_t **TH** [12]
偏移地址: 0x00 MCPWM CH0.P 比较门限值寄存器
- volatile uint32_t **TMR0**
偏移地址: 0x30 ADC 采样定时器比较门限 0 寄存器
- volatile uint32_t **TMR1**
偏移地址: 0x34 ADC 采样定时器比较门限 1 寄存器
- volatile uint32_t **TMR2**
偏移地址: 0x38 ADC 采样定时器比较门限 2 寄存器
- volatile uint32_t **TMR3**
偏移地址: 0x3C ADC 采样定时器比较门限 3 寄存器
- volatile uint32_t **TH0**
偏移地址: 0x40 MCPWM 时基 0 门限值寄存器
- volatile uint32_t **TH1**
偏移地址: 0x44 MCPWM 时基 1 门限值寄存器
- volatile uint32_t **CNT0**
偏移地址: 0x48 MCPWM 时基 0 计数器寄存器
- volatile uint32_t **CNT1**
偏移地址: 0x4C MCPWM 时基 1 计数器寄存器
- volatile uint32_t **UPDATE**
偏移地址: 0x50 MCPWM 加载控制寄存器
- volatile uint32_t **FCNT**
偏移地址: 0x54 MCPWM FAIL 时刻 CNT 值
- volatile uint32_t **EVT0**
偏移地址: 0x58 MCPWM 时基 0 外部触发
- volatile uint32_t **EVT1**
偏移地址: 0x5C MCPWM 时基 1 外部触发
- volatile uint32_t **DTH00**
偏移地址: 0x60 MCPWM CH0/1/2 N 通道死区宽度控制寄存器
- volatile uint32_t **DTH01**
偏移地址: 0x64 MCPWM CH0/1/2 P 通道死区宽度控制寄存器
- volatile uint32_t **DTH10**
偏移地址: 0x68 MCPWM CH3/4/5 N 通道死区宽度控制寄存器
- volatile uint32_t **DTH11**
偏移地址: 0x6C MCPWM CH3/4/5 P 通道死区宽度控制寄存器
- volatile uint32_t **FLT**
偏移地址: 0x70 MCPWM 滤波时钟分频寄存器
- volatile uint32_t **SDCFG**
偏移地址: 0x74 MCPWM 加载配置寄存器
- volatile uint32_t **AUEN**
偏移地址: 0x78 MCPWM 自动更新使能寄存器
- volatile uint32_t **TCLK**
偏移地址: 0x7C MCPWM 时钟分频控制寄存器
- volatile uint32_t **IE0**
偏移地址: 0x80 MCPWM 时基 0 中断控制寄存器
- volatile uint32_t **IF0**
偏移地址: 0x84 MCPWM 时基 0 中断标志位寄存器
- volatile uint32_t **IE1**
偏移地址: 0x88 MCPWM 中断控制寄存器
- volatile uint32_t **IF1**

- 偏移地址: 0x8C MCPWM 中断标志位寄存器
- volatile uint32_t [EIE](#)
- 偏移地址: 0x90 MCPWM 异常中断控制寄存器
- volatile uint32_t [EIF](#)
- 偏移地址: 0x94 MCPWM 异常中断标志位寄存器
- volatile uint32_t [RE](#)
- 偏移地址: 0x98 MCPWM DMA 请求控制寄存器
- volatile uint32_t [PP](#)
- 偏移地址: 0x9C MCPWM 推挽模式使能寄存器
- volatile uint32_t [IO01](#)
- 偏移地址: 0xA0 MCPWM CH0 CH1 IO 控制寄存器
- volatile uint32_t [IO23](#)
- 偏移地址: 0xA4 MCPWM CH2 CH3 IO 控制寄存器
- volatile uint32_t [IO45](#)
- 偏移地址: 0xA8 MCPWM CH4 CH5 IO 控制寄存器
- volatile uint32_t [null](#)
- 偏移地址: 0xAC 地址对齐
- volatile uint32_t [FAIL012](#)
- 偏移地址: 0xB0 MCPWM CH0 CH1 CH2 短路控制寄存器
- volatile uint32_t [FAIL345](#)
- 偏移地址: 0xB4 MCPWM CH3 CH4 CH5 短路控制寄存器
- volatile uint32_t [CH_DEF](#)
- 偏移地址: 0xB8 MCPWM 短路保护通道输出默认值
- volatile uint32_t [CH_MSK](#)
- 偏移地址: 0xBC MCPWM 通道屏蔽寄存器
- volatile uint32_t [PRT](#)
- 偏移地址: 0xC0 MCPWM 保护寄存器
- volatile uint32_t [STT_HYST](#)
- 偏移地址: 0xC8 MCPWM 状态停留延时寄存器
- volatile uint32_t [ZCS_DELAY](#)
- 偏移地址: 0xCC MCPWM ZCS 状态延时寄存器

3.13.1 详细描述

MCPWM寄存器定义

在文件 [hal_lks32mc07x_reg.h](#) 第 200 行定义.

3.13.2 结构体成员变量说明

3.13.2.1 AUEN

```
volatile uint32_t AUEN
```

偏移地址: 0x78 MCPWM 自动更新使能寄存器

在文件 [hal_lks32mc07x_reg.h](#) 第 222 行定义.

3.13.2.2 CH_DEF

```
volatile uint32_t CH_DEF
```

偏移地址: 0xB8 MCPWM 短路保护通道输出默认值

在文件 [hal_lks32mc07x_reg.h](#) 第 238 行定义.

3.13.2.3 CH_MSK

```
volatile uint32_t CH_MSK
```

偏移地址: 0xBC MCPWM 通道屏蔽寄存器

在文件 [hal_lks32mc07x_reg.h](#) 第 239 行定义.

3.13.2.4 CNT0

```
volatile uint32_t CNT0
```

偏移地址: 0x48 MCPWM 时基 0 计数器寄存器

在文件 [hal_lks32mc07x_reg.h](#) 第 210 行定义.

3.13.2.5 CNT1

```
volatile uint32_t CNT1
```

偏移地址: 0x4C MCPWM 时基 1 计数器寄存器

在文件 [hal_lks32mc07x_reg.h](#) 第 211 行定义.

3.13.2.6 DTH00

```
volatile uint32_t DTH00
```

偏移地址: 0x60 MCPWM CH0/1/2 N 通道死区宽度控制寄存器

在文件 [hal_lks32mc07x_reg.h](#) 第 216 行定义.

3.13.2.7 DTH01

```
volatile uint32_t DTH01
```

偏移地址: 0x64 MCPWM CH0/1/2 P 通道死区宽度控制寄存器

在文件 [hal_lks32mc07x_reg.h](#) 第 217 行定义.

3.13.2.8 DTH10

```
volatile uint32_t DTH10
```

偏移地址: 0x68 MCPWM CH3/4/5 N 通道死区宽度控制寄存器

在文件 [hal_lks32mc07x_reg.h](#) 第 218 行定义.

3.13.2.9 DTH11

```
volatile uint32_t DTH11
```

偏移地址: 0x6C MCPWM CH3/4/5 P 通道死区宽度控制寄存器

在文件 [hal_lks32mc07x_reg.h](#) 第 219 行定义.

3.13.2.10 EIE

```
volatile uint32_t EIE
```

偏移地址: 0x90 MCPWM 异常中断控制寄存器

在文件 [hal_lks32mc07x_reg.h](#) 第 228 行定义.

3.13.2.11 EIF

```
volatile uint32_t EIF
```

偏移地址: 0x94 MCPWM 异常中断标志位寄存器

在文件 [hal_lks32mc07x_reg.h](#) 第 229 行定义.

3.13.2.12 EVT0

```
volatile uint32_t EVT0
```

偏移地址: 0x58 MCPWM 时基 0 外部触发

在文件 [hal_lks32mc07x_reg.h](#) 第 214 行定义.

3.13.2.13 EVT1

```
volatile uint32_t EVT1
```

偏移地址: 0x5C MCPWM 时基 1 外部触发

在文件 [hal_lks32mc07x_reg.h](#) 第 215 行定义.

3.13.2.14 FAIL012

```
volatile uint32_t FAIL012
```

偏移地址: 0xB0 MCPWM CH0 CH1 CH2 短路控制寄存器

在文件 [hal_lks32mc07x_reg.h](#) 第 236 行定义.

3.13.2.15 FAIL345

```
volatile uint32_t FAIL345
```

偏移地址: 0xB4 MCPWM CH3 CH4 CH5 短路控制寄存器

在文件 [hal_lks32mc07x_reg.h](#) 第 237 行定义.

3.13.2.16 FCNT

```
volatile uint32_t FCNT
```

偏移地址: 0x54 MCPWM FAIL 时刻 CNT 值

在文件 [hal_lks32mc07x_reg.h](#) 第 213 行定义.

3.13.2.17 FLT

```
volatile uint32_t FLT
```

偏移地址: 0x70 MCPWM 滤波时钟分频寄存器

在文件 [hal_lks32mc07x_reg.h](#) 第 220 行定义.

3.13.2.18 IE0

```
volatile uint32_t IE0
```

偏移地址: 0x80 MCPWM 时基 0 中断控制寄存器

在文件 [hal_lks32mc07x_reg.h](#) 第 224 行定义.

3.13.2.19 IE1

```
volatile uint32_t IE1
```

偏移地址: 0x88 MCPWM 中断控制寄存器

在文件 [hal_lks32mc07x_reg.h](#) 第 226 行定义.

3.13.2.20 IF0

```
volatile uint32_t IF0
```

偏移地址: 0x84 MCPWM 时基 0 中断标志位寄存器

在文件 [hal_lks32mc07x_reg.h](#) 第 225 行定义.

3.13.2.21 IF1

```
volatile uint32_t IF1
```

偏移地址: 0x8C MCPWM 中断标志位寄存器

在文件 [hal_lks32mc07x_reg.h](#) 第 227 行定义.

3.13.2.22 IO01

```
volatile uint32_t IO01
```

偏移地址: 0xA0 MCPWM CH0 CH1 IO 控制寄存器

在文件 [hal_lks32mc07x_reg.h](#) 第 232 行定义.

3.13.2.23 IO23

```
volatile uint32_t IO23
```

偏移地址: 0xA4 MCPWM CH2 CH3 IO 控制寄存器

在文件 [hal_lks32mc07x_reg.h](#) 第 233 行定义.

3.13.2.24 IO45

```
volatile uint32_t IO45
```

偏移地址: 0xA8 MCPWM CH4 CH5 IO 控制寄存器

在文件 [hal_lks32mc07x_reg.h](#) 第 234 行定义.

3.13.2.25 null

```
volatile uint32_t null
```

偏移地址: 0xAC 地址对齐

在文件 [hal_lks32mc07x_reg.h](#) 第 235 行定义.

3.13.2.26 PP

```
volatile uint32_t PP
```

偏移地址: 0x9C MCPWM 推挽模式使能寄存器

在文件 [hal_lks32mc07x_reg.h](#) 第 231 行定义.

3.13.2.27 PRT

```
volatile uint32_t PRT
```

偏移地址: 0xC0 MCPWM 保护寄存器

在文件 [hal_lks32mc07x_reg.h](#) 第 240 行定义.

3.13.2.28 RE

```
volatile uint32_t RE
```

偏移地址: 0x98 MCPWM DMA 请求控制寄存器

在文件 [hal_lks32mc07x_reg.h](#) 第 230 行定义.

3.13.2.29 SDCFG

```
volatile uint32_t SDCFG
```

偏移地址: 0x74 MCPWM 加载配置寄存器

在文件 [hal_lks32mc07x_reg.h](#) 第 221 行定义.

3.13.2.30 STT_HYST

```
volatile uint32_t STT_HYST
```

偏移地址: 0xC8 MCPWM 状态停留延时寄存器

在文件 [hal_lks32mc07x_reg.h](#) 第 241 行定义.

3.13.2.31 TCLK

```
volatile uint32_t TCLK
```

偏移地址: 0x7C MCPWM 时钟分频控制寄存器

在文件 [hal_lks32mc07x_reg.h](#) 第 223 行定义.

3.13.2.32 TH

```
volatile uint32_t TH[12]
```

偏移地址: 0x00 MCPWM CH0.P 比较门限值寄存器

在文件 [hal_lks32mc07x_reg.h](#) 第 203 行定义.

3.13.2.33 TH0

```
volatile uint32_t TH0
```

偏移地址: 0x40 MCPWM 时基 0 门限值寄存器

在文件 [hal_lks32mc07x_reg.h](#) 第 208 行定义.

3.13.2.34 TH1

```
volatile uint32_t TH1
```

偏移地址: 0x44 MCPWM 时基 1 门限值寄存器

在文件 [hal_lks32mc07x_reg.h](#) 第 209 行定义.

3.13.2.35 TMR0

```
volatile uint32_t TMR0
```

偏移地址: 0x30 ADC 采样定时器比较门限 0 寄存器

在文件 [hal_lks32mc07x_reg.h](#) 第 204 行定义.

3.13.2.36 TMR1

```
volatile uint32_t TMR1
```

偏移地址: 0x34 ADC 采样定时器比较门限 1 寄存器

在文件 [hal_lks32mc07x_reg.h](#) 第 205 行定义.

3.13.2.37 TMR2

```
volatile uint32_t TMR2
```

偏移地址: 0x38 ADC 采样定时器比较门限 2 寄存器

在文件 [hal_lks32mc07x_reg.h](#) 第 206 行定义.

3.13.2.38 TMR3

```
volatile uint32_t TMR3
```

偏移地址: 0x3C ADC 采样定时器比较门限 3 寄存器

在文件 [hal_lks32mc07x_reg.h](#) 第 207 行定义.

3.13.2.39 UPDATE

```
volatile uint32_t UPDATE
```

偏移地址: 0x50 MCPWM 加载控制寄存器

在文件 [hal_lks32mc07x_reg.h](#) 第 212 行定义.

3.13.2.40 ZCS_DELAY

```
volatile uint32_t ZCS_DELAY
```

偏移地址: 0xCC MCPWM ZCS 状态延时寄存器

在文件 [hal_lks32mc07x_reg.h](#) 第 242 行定义.

3.14 HAL_QEP_TypeDef结构体 参考

QEP寄存器定义

```
#include <hal_lks32mc07x_reg.h>
```

成员变量

- volatile uint32_t [CFG](#)
0x00 QEP0配置寄存器
- volatile uint32_t [TH](#)
0x04 QEP0计数门限寄存器
- volatile uint32_t [CNT](#)
0x08 QEP0计数值寄存器
- volatile uint32_t [IE](#)
0x0C QEP0中断使能寄存器
- volatile uint32_t [IF](#)
0x10 QEP0中断标志寄存器

3.14.1 详细描述

QEP寄存器定义

在文件 [hal_lks32mc07x_reg.h](#) 第 189 行定义.

3.14.2 结构体成员变量说明

3.14.2.1 CFG

`volatile uint32_t CFG`

0x00 QEP0配置寄存器

在文件 [hal_lks32mc07x_reg.h](#) 第 191 行定义.

3.14.2.2 CNT

`volatile uint32_t CNT`

0x08 QEP0计数值寄存器

在文件 [hal_lks32mc07x_reg.h](#) 第 193 行定义.

3.14.2.3 IE

`volatile uint32_t IE`

0x0C QEP0中断使能寄存器

在文件 [hal_lks32mc07x_reg.h](#) 第 194 行定义.

3.14.2.4 IF

`volatile uint32_t IF`

0x10 QEP0中断标志寄存器

在文件 [hal_lks32mc07x_reg.h](#) 第 195 行定义.

3.14.2.5 TH

`volatile uint32_t TH`

0x04 QEP0计数门限寄存器

在文件 [hal_lks32mc07x_reg.h](#) 第 192 行定义.

3.15 HAL_SPI_TypeDef结构体 参考

SPI寄存器定义

```
#include <hal_lks32mc07x_reg.h>
```


成员变量

- volatile uint32_t **CFG**
0x00 配置寄存器
- volatile uint32_t **IE**
0x04 中断寄存器
- volatile uint32_t **BAUD**
0x08 波特率寄存器
- volatile uint32_t **TXDATA**
0x0C 发送数据寄存器
- volatile uint32_t **RXDATA**
0x10 接收数据寄存器
- volatile uint32_t **SIZE**
0x14 传输数据长度寄存器

3.15.1 详细描述

SPI寄存器定义

在文件 [hal_lks32mc07x_reg.h](#) 第 89 行定义.

3.15.2 结构体成员变量说明

3.15.2.1 BAUD

```
volatile uint32_t BAUD
```

0x08 波特率寄存器

在文件 [hal_lks32mc07x_reg.h](#) 第 93 行定义.

3.15.2.2 CFG

```
volatile uint32_t CFG
```

0x00 配置寄存器

在文件 [hal_lks32mc07x_reg.h](#) 第 91 行定义.

3.15.2.3 IE

```
volatile uint32_t IE
```

0x04 中断寄存器

在文件 [hal_lks32mc07x_reg.h](#) 第 92 行定义.

3.15.2.4 RXDATA

```
volatile uint32_t RXDATA
```

0x10 接收数据寄存器

在文件 [hal_lks32mc07x_reg.h](#) 第 95 行定义.

3.15.2.5 SIZE

```
volatile uint32_t SIZE
```

0x14 传输数据长度寄存器

在文件 [hal_lks32mc07x_reg.h](#) 第 96 行定义.

3.15.2.6 TXDATA

```
volatile uint32_t TXDATA
```

0x0C 发送数据寄存器

在文件 [hal_lks32mc07x_reg.h](#) 第 94 行定义.

3.16 HAL_SYS_TypeDef结构体 参考

SYS寄存器定义

```
#include <hal_lks32mc07x_reg.h>
```

成员变量

- volatile uint32_t [ADC](#)
- volatile uint32_t [INFO](#)
- volatile uint32_t [DBG](#)
- volatile uint32_t [null0](#)
- volatile uint32_t [REG0](#)
- volatile uint32_t [REG1](#)
- volatile uint32_t [REG2](#)
- volatile uint32_t [REG3](#)
- volatile uint32_t [REG4](#)
- volatile uint32_t [REG5](#)
- volatile uint32_t [REG6](#)
- volatile uint32_t [REG7](#)
- volatile uint32_t [REG8](#)
- volatile uint32_t [REG9](#)
- volatile uint32_t [REGA](#)
- volatile uint32_t [REGB](#)
- volatile uint32_t [REGC](#)

- volatile uint32_t null1
- volatile uint32_t null2
- volatile uint32_t null3
- volatile uint32_t null4
- volatile uint32_t null5
- volatile uint32_t null6
- volatile uint32_t DAC_CTRL
- volatile uint32_t DAC0
- volatile uint32_t DAC1
- volatile uint32_t DAC0_AMC
- volatile uint32_t DAC0_DC
- volatile uint32_t DAC1_AMC
- volatile uint32_t DAC1_DC
- volatile uint32_t null7
- volatile uint32_t null8
- volatile uint32_t CLK_CFG
- volatile uint32_t IO_CFG
- volatile uint32_t DBG_CFG
- volatile uint32_t null9
- volatile uint32_t CLK_DIV0
- volatile uint32_t CLK_DIV1
- volatile uint32_t CLK_DIV2
- volatile uint32_t CLK_FEN
- volatile uint32_t TRIM
- volatile uint32_t SFT_RST
- volatile uint32_t WR_PROTECT
- volatile uint32_t MBIST
- volatile uint32_t null10
- volatile uint32_t null11
- volatile uint32_t null12
- volatile uint32_t FLSE
- volatile uint32_t FLSP
- volatile uint32_t FLST
- volatile uint32_t null13
- volatile uint32_t SBIST

3.16.1 详细描述

SYS寄存器定义

在文件 [hal_lks32mc07x_reg.h](#) 第 8 行定义.

3.16.2 结构体成员变量说明

3.16.2.1 ADC

```
volatile uint32_t ADC
```

在文件 [hal_lks32mc07x_reg.h](#) 第 10 行定义.

3.16.2.2 CLK_CFG

```
volatile uint32_t CLK_CFG
```

在文件 [hal_lks32mc07x_reg.h](#) 第 44 行定义.

3.16.2.3 CLK_DIV0

```
volatile uint32_t CLK_DIV0
```

在文件 [hal_lks32mc07x_reg.h](#) 第 48 行定义.

3.16.2.4 CLK_DIV1

```
volatile uint32_t CLK_DIV1
```

在文件 [hal_lks32mc07x_reg.h](#) 第 49 行定义.

3.16.2.5 CLK_DIV2

```
volatile uint32_t CLK_DIV2
```

在文件 [hal_lks32mc07x_reg.h](#) 第 50 行定义.

3.16.2.6 CLK_FEN

```
volatile uint32_t CLK_FEN
```

在文件 [hal_lks32mc07x_reg.h](#) 第 51 行定义.

3.16.2.7 DAC0

```
volatile uint32_t DAC0
```

在文件 [hal_lks32mc07x_reg.h](#) 第 34 行定义.

3.16.2.8 DAC0_AMC

```
volatile uint32_t DAC0_AMC
```

在文件 [hal_lks32mc07x_reg.h](#) 第 36 行定义.

3.16.2.9 DAC0_DC

```
volatile uint32_t DAC0_DC
```

在文件 [hal_lks32mc07x_reg.h](#) 第 37 行定义.

3.16.2.10 DAC1

```
volatile uint32_t DAC1
```

在文件 [hal_lks32mc07x_reg.h](#) 第 35 行定义.

3.16.2.11 DAC1_AMC

```
volatile uint32_t DAC1_AMC
```

在文件 [hal_lks32mc07x_reg.h](#) 第 38 行定义.

3.16.2.12 DAC1_DC

```
volatile uint32_t DAC1_DC
```

在文件 [hal_lks32mc07x_reg.h](#) 第 39 行定义.

3.16.2.13 DAC_CTRL

```
volatile uint32_t DAC_CTRL
```

在文件 [hal_lks32mc07x_reg.h](#) 第 33 行定义.

3.16.2.14 DBG

```
volatile uint32_t DBG
```

在文件 [hal_lks32mc07x_reg.h](#) 第 12 行定义.

3.16.2.15 DBG_CFG

```
volatile uint32_t DBG_CFG
```

在文件 [hal_lks32mc07x_reg.h](#) 第 46 行定义.

3.16.2.16 FLSE

```
volatile uint32_t FLSE
```

在文件 [hal_lks32mc07x_reg.h](#) 第 59 行定义.

3.16.2.17 FLSP

```
volatile uint32_t FLSP
```

在文件 [hal_lks32mc07x_reg.h](#) 第 60 行定义.

3.16.2.18 FLST

```
volatile uint32_t FLST
```

在文件 [hal_lks32mc07x_reg.h](#) 第 61 行定义.

3.16.2.19 INFO

```
volatile uint32_t INFO
```

在文件 [hal_lks32mc07x_reg.h](#) 第 11 行定义.

3.16.2.20 IO_CFG

```
volatile uint32_t IO_CFG
```

在文件 [hal_lks32mc07x_reg.h](#) 第 45 行定义.

3.16.2.21 MBIST

```
volatile uint32_t MBIST
```

在文件 [hal_lks32mc07x_reg.h](#) 第 55 行定义.

3.16.2.22 null0

```
volatile uint32_t null0
```

在文件 [hal_lks32mc07x_reg.h](#) 第 13 行定义.

3.16.2.23 null1

```
volatile uint32_t null1
```

在文件 [hal_lks32mc07x_reg.h](#) 第 27 行定义.

3.16.2.24 null10

```
volatile uint32_t null10
```

在文件 [hal_lks32mc07x_reg.h](#) 第 56 行定义.

3.16.2.25 null11

```
volatile uint32_t null11
```

在文件 [hal_lks32mc07x_reg.h](#) 第 57 行定义.

3.16.2.26 null12

```
volatile uint32_t null12
```

在文件 [hal_lks32mc07x_reg.h](#) 第 58 行定义.

3.16.2.27 null13

```
volatile uint32_t null13
```

在文件 [hal_lks32mc07x_reg.h](#) 第 62 行定义.

3.16.2.28 null2

```
volatile uint32_t null2
```

在文件 [hal_lks32mc07x_reg.h](#) 第 28 行定义.

3.16.2.29 null3

```
volatile uint32_t null3
```

在文件 [hal_lks32mc07x_reg.h](#) 第 29 行定义.

3.16.2.30 null4

```
volatile uint32_t null4
```

在文件 [hal_lks32mc07x_reg.h](#) 第 30 行定义.

3.16.2.31 null5

```
volatile uint32_t null5
```

在文件 [hal_lks32mc07x_reg.h](#) 第 31 行定义.

3.16.2.32 null6

```
volatile uint32_t null6
```

在文件 [hal_lks32mc07x_reg.h](#) 第 32 行定义.

3.16.2.33 null7

```
volatile uint32_t null7
```

在文件 [hal_lks32mc07x_reg.h](#) 第 41 行定义.

3.16.2.34 null8

```
volatile uint32_t null8
```

在文件 [hal_lks32mc07x_reg.h](#) 第 42 行定义.

3.16.2.35 null9

```
volatile uint32_t null9
```

在文件 [hal_lks32mc07x_reg.h](#) 第 47 行定义.

3.16.2.36 REG0

```
volatile uint32_t REG0
```

在文件 [hal_lks32mc07x_reg.h](#) 第 14 行定义.

3.16.2.37 REG1

```
volatile uint32_t REG1
```

在文件 [hal_lks32mc07x_reg.h](#) 第 15 行定义.

3.16.2.38 REG2

```
volatile uint32_t REG2
```

在文件 [hal_lks32mc07x_reg.h](#) 第 16 行定义.

3.16.2.39 REG3

```
volatile uint32_t REG3
```

在文件 [hal_lks32mc07x_reg.h](#) 第 17 行定义.

3.16.2.40 REG4

```
volatile uint32_t REG4
```

在文件 [hal_lks32mc07x_reg.h](#) 第 18 行定义.

3.16.2.41 REG5

```
volatile uint32_t REG5
```

在文件 [hal_lks32mc07x_reg.h](#) 第 19 行定义.

3.16.2.42 REG6

```
volatile uint32_t REG6
```

在文件 [hal_lks32mc07x_reg.h](#) 第 20 行定义.

3.16.2.43 REG7

```
volatile uint32_t REG7
```

在文件 [hal_lks32mc07x_reg.h](#) 第 21 行定义.

3.16.2.44 REG8

```
volatile uint32_t REG8
```

在文件 [hal_lks32mc07x_reg.h](#) 第 22 行定义.

3.16.2.45 REG9

```
volatile uint32_t REG9
```

在文件 [hal_lks32mc07x_reg.h](#) 第 23 行定义.

3.16.2.46 REGA

```
volatile uint32_t REGA
```

在文件 [hal_lks32mc07x_reg.h](#) 第 24 行定义.

3.16.2.47 REGB

```
volatile uint32_t REGB
```

在文件 [hal_lks32mc07x_reg.h](#) 第 25 行定义.

3.16.2.48 REGC

```
volatile uint32_t REGC
```

在文件 [hal_lks32mc07x_reg.h](#) 第 26 行定义.

3.16.2.49 SBIST

```
volatile uint32_t SBIST
```

在文件 [hal_lks32mc07x_reg.h](#) 第 63 行定义.

3.16.2.50 SFT_RST

```
volatile uint32_t SFT_RST
```

在文件 [hal_lks32mc07x_reg.h](#) 第 53 行定义.

3.16.2.51 TRIM

```
volatile uint32_t TRIM
```

在文件 [hal_lks32mc07x_reg.h](#) 第 52 行定义.

3.16.2.52 WR_PROTECT

```
volatile uint32_t WR_PROTECT
```

在文件 [hal_lks32mc07x_reg.h](#) 第 54 行定义.

3.17 HAL_TIMER_TypeDef结构体 参考

TIMER寄存器定义

```
#include <hal_lks32mc07x_reg.h>
```

成员变量

- volatile uint32_t [CFG](#)
定时器配置寄存器
- volatile uint32_t [TH](#)
定时器计数门限寄存器
- volatile uint32_t [CNT](#)
定时器计数值寄存器
- volatile uint32_t [CMP0](#)
定时器比较/捕获寄存器 0
- volatile uint32_t [CMP1](#)
定时器比较/捕获寄存器 1
- volatile uint32_t [EVT](#)
定时器外部事件选择寄存器
- volatile uint32_t [FLT](#)
定时器滤波控制寄存器
- volatile uint32_t [IE](#)
定时器中断使能寄存器
- volatile uint32_t [IF](#)
定时器中断标志寄存器
- volatile uint32_t [IO](#)
定时器IO控制寄存器

3.17.1 详细描述

TIMER寄存器定义

在文件 [hal_lks32mc07x_reg.h](#) 第 172 行定义.

3.17.2 结构体成员变量说明

3.17.2.1 CFG

```
volatile uint32_t CFG
```

定时器配置寄存器

在文件 [hal_lks32mc07x_reg.h](#) 第 174 行定义.

3.17.2.2 CMP0

```
volatile uint32_t CMP0
```

定时器比较/捕获寄存器 0

在文件 [hal_lks32mc07x_reg.h](#) 第 177 行定义.

3.17.2.3 CMP1

```
volatile uint32_t CMP1
```

定时器比较/捕获寄存器 1

在文件 [hal_lks32mc07x_reg.h](#) 第 178 行定义.

3.17.2.4 CNT

```
volatile uint32_t CNT
```

定时器计数值寄存器

在文件 [hal_lks32mc07x_reg.h](#) 第 176 行定义.

3.17.2.5 EVT

```
volatile uint32_t EVT
```

定时器外部事件选择寄存器

在文件 [hal_lks32mc07x_reg.h](#) 第 179 行定义.

3.17.2.6 FLT

```
volatile uint32_t FLT
```

定时器滤波控制寄存器

在文件 [hal_lks32mc07x_reg.h](#) 第 180 行定义.

3.17.2.7 IE

```
volatile uint32_t IE
```

定时器中断使能寄存器

在文件 [hal_lks32mc07x_reg.h](#) 第 181 行定义.

3.17.2.8 IF

```
volatile uint32_t IF
```

定时器中断标志寄存器

在文件 [hal_lks32mc07x_reg.h](#) 第 182 行定义.

3.17.2.9 IO

```
volatile uint32_t IO
```

定时器IO控制寄存器

在文件 [hal_lks32mc07x_reg.h](#) 第 183 行定义.

3.17.2.10 TH

```
volatile uint32_t TH
```

定时器计数门限寄存器

在文件 [hal_lks32mc07x_reg.h](#) 第 175 行定义.

3.18 HAL_UART_TypeDef结构体 参考

UART寄存器定义

```
#include <hal_lks32mc07x_reg.h>
```

成员变量

- volatile uint32_t **CTRL**
UART控制寄存器
- volatile uint32_t **DIVH**
UART波特率设置高字节寄存器
- volatile uint32_t **DIVL**
UART波特率设置低字节寄存器
- volatile uint32_t **BUFF**
UART收发缓冲寄存器
- volatile uint32_t **ADR**
485通信地址匹配寄存器
- volatile uint32_t **STT**
UART状态寄存器
- volatile uint32_t **RE**
UART DMA 请求使能寄存器
- volatile uint32_t **IE**
UART中断使能寄存器
- volatile uint32_t **IF**
UART中断标志寄存器
- volatile uint32_t **IOC**
UART IO翻转使能

3.18.1 详细描述

UART寄存器定义

在文件 [hal_lks32mc07x.reg.h](#) 第 280 行定义.

3.18.2 结构体成员变量说明

3.18.2.1 ADR

```
volatile uint32_t ADR
```

485通信地址匹配寄存器

在文件 [hal_lks32mc07x.reg.h](#) 第 286 行定义.

3.18.2.2 BUFF

```
volatile uint32_t BUFF
```

UART收发缓冲寄存器

在文件 [hal_lks32mc07x.reg.h](#) 第 285 行定义.

3.18.2.3 CTRL

```
volatile uint32_t CTRL
```

UART控制寄存器

在文件 [hal_lks32mc07x_reg.h](#) 第 282 行定义.

3.18.2.4 DIVH

```
volatile uint32_t DIVH
```

UART波特率设置高字节寄存器

在文件 [hal_lks32mc07x_reg.h](#) 第 283 行定义.

3.18.2.5 DIVL

```
volatile uint32_t DIVL
```

UART波特率设置低字节寄存器

在文件 [hal_lks32mc07x_reg.h](#) 第 284 行定义.

3.18.2.6 IE

```
volatile uint32_t IE
```

UART中断使能寄存器

在文件 [hal_lks32mc07x_reg.h](#) 第 289 行定义.

3.18.2.7 IF

```
volatile uint32_t IF
```

UART中断标志寄存器

在文件 [hal_lks32mc07x_reg.h](#) 第 290 行定义.

3.18.2.8 IOC

```
volatile uint32_t IOC
```

UART IO翻转使能

在文件 [hal_lks32mc07x_reg.h](#) 第 291 行定义.

3.18.2.9 RE

```
volatile uint32_t RE
```

UART DMA 请求使能寄存器

在文件 [hal_lks32mc07x_reg.h](#) 第 288 行定义.

3.18.2.10 STT

```
volatile uint32_t STT
```

UART状态寄存器

在文件 [hal_lks32mc07x_reg.h](#) 第 287 行定义.

Chapter 4

文件说明

4.1 hal_lks32mc07x.c 文件参考

```
#include "source/hal_lks32mc07x_adc.c"
#include "source/hal_lks32mc07x_clk.c"
#include "source/hal_lks32mc07x_cmp.c"
#include "source/hal_lks32mc07x_crc.c"
#include "source/hal_lks32mc07x_dac.c"
#include "source/hal_lks32mc07x_dma.c"
#include "source/hal_lks32mc07x_dsp.c"
#include "source/hal_lks32mc07x_flash.c"
#include "source/hal_lks32mc07x_gpio.c"
#include "source/hal_lks32mc07x_hall.c"
#include "source/hal_lks32mc07x_i2c.c"
#include "source/hal_lks32mc07x_init.c"
#include "source/hal_lks32mc07x_iwdg.c"
#include "source/hal_lks32mc07x_irq.c"
#include "source/hal_lks32mc07x_mcpwm.c"
#include "source/hal_lks32mc07x_opa.c"
#include "source/hal_lks32mc07x_qep.c"
#include "source/hal_lks32mc07x_reset.c"
#include "source/hal_lks32mc07x_reg.c"
#include "source/hal_lks32mc07x_spi.c"
#include "source/hal_lks32mc07x_test.c"
#include "source/hal_lks32mc07x_timer.c"
#include "source/hal_lks32mc07x_tmp.c"
#include "source/hal_lks32mc07x_trim.c"
#include "source/hal_lks32mc07x_uart.c"
#include "source/hal_lks32mc07x_wake.c"
#include "source/hal_lks32mc07x_irqfun.c"
```

hal_lks32mc07x.c 的引用(Include)关系图:



宏定义

- #define _HAL_LKS32MC05X_C_

4.1.1 宏定义说明

4.1.1.1 __HAL_LKS32MC05X_C__

```
#define __HAL_LKS32MC05X_C__
```

在文件 `hal_lks32mc07x.c` 第 2 行定义。

4.2 hal_lks32mc07x.c

[浏览该文件的文档。](#)

```
00001 #ifndef __HAL_LKS32MC05X_C__
00002 #define __HAL_LKS32MC05X_C__
00003 #include "source/hal_lks32mc07x_adc.c"
00004 #include "source/hal_lks32mc07x_clk.c"
00005 #include "source/hal_lks32mc07x_cmp.c"
00006 #include "source/hal_lks32mc07x_crc.c"
00007 #include "source/hal_lks32mc07x_dac.c"
00008 #include "source/hal_lks32mc07x_dma.c"
00009 #include "source/hal_lks32mc07x_dsp.c"
00010 #include "source/hal_lks32mc07x_flash.c"
00011 #include "source/hal_lks32mc07x_gpio.c"
00012 #include "source/hal_lks32mc07x_hall.c"
00013 #include "source/hal_lks32mc07x_i2c.c"
00014 #include "source/hal_lks32mc07x_init.c"
00015 #include "source/hal_lks32mc07x_iwdg.c"
00016 #include "source/hal_lks32mc07x_irq.c"
00017 #include "source/hal_lks32mc07x_mcpwm.c"
00018 #include "source/hal_lks32mc07x_opa.c"
00019 #include "source/hal_lks32mc07x_qep.c"
00020 #include "source/hal_lks32mc07x_reset.c"
00021 #include "source/hal_lks32mc07x_reg.c"
00022 #include "source/hal_lks32mc07x_spi.c"
00023 #include "source/hal_lks32mc07x_test.c"
00024 #include "source/hal_lks32mc07x_timer.c"
00025 #include "source/hal_lks32mc07x_tpm.c"
00026 #include "source/hal_lks32mc07x_trim.c"
00027 #include "source/hal_lks32mc07x_uart.c"
00028 #include "source/hal_lks32mc07x_wake.c"
00029 #include "source/hal_lks32mc07x_irqfun.c"
00030
00031 #endif
```

4.3 hal_lks32mc07x.h 文件参考

```
#include "basic.h"
#include "stdint.h"
#include "lks32mc07x.h"
#include "include/hal_lksmcu_adc.h"
#include "include/hal_lksmcu_can.h"
#include "include/hal_lksmcu_clk.h"
#include "include/hal_lksmcu_cmp.h"
#include "include/hal_lksmcu_crc.h"
#include "include/hal_lksmcu_dac.h"
#include "include/hal_lksmcu_dma.h"
#include "include/hal_lksmcu_dsp.h"
#include "include/hal_lksmcu_flash.h"
#include "include/hal_lksmcu_gpio.h"
#include "include/hal_lksmcu_hall.h"
#include "include/hal_lksmcu_i2c.h"
#include "include/hal_lksmcu_irq.h"
#include "include/hal_lksmcu_init.h"
```



```
#include "include/hal_lksmcu_iwdg.h"
#include "include/hal_lksmcu_mcpwm.h"
#include "include/hal_lksmcu_opa.h"
#include "include/hal_lksmcu_qep.h"
#include "include/hal_lksmcu_reg.h"
#include "include/hal_lksmcu_reset.h"
#include "include/hal_lksmcu_spi.h"
#include "include/hal_lksmcu_timer.h"
#include "include/hal_lksmcu_tmp.h"
#include "include/hal_lksmcu_trim.h"
#include "include/hal_lksmcu_uart.h"
#include "include/hal_lksmcu_wake.h"
#include "include/hal_lksmcu_irqfun.h"
```

hal_lks32mc07x.h 的引用(Include)关系图:



此图展示该文件直接或间接的被哪些文件引用了:



宏定义

- **#define HAL_TIMER_IRQHandler_00** TIMER0_IRQHandler

统一中断函数接口 命名方式: **HAL**模块名称**_IRQHandler**.模块序号中断序号。例如**MCPWM**模块有两个**CNT**,会产生两个中断。这里分别对应**MCPWM0_IRQHandler**和**MCPWM1_IRQHandler**。为方便使用,末尾的**_00**可以省略。

- **#define HAL_TIMER_IRQHandler_10** TIMER1_IRQHandler
- **#define HAL_TIMER_IRQHandler_20** TIMER2_IRQHandler
- **#define HAL_TIMER_IRQHandler_30** TIMER3_IRQHandler
- **#define HAL_MCPWM_IRQHandler_00** MCPWM0_IRQHandler
- **#define HAL_MCPWM_IRQHandler_01** MCPWM1_IRQHandler
- **#define HAL_I2C_IRQHandler_00** I2C0_IRQHandler
- **#define HAL_SPI_IRQHandler_00** SPI0_IRQHandler
- **#define HAL_GPIO_IRQHandler_00** GPIO_IRQHandler
- **#define HAL_HALL_IRQHandler_00** HALL0_IRQHandler
- **#define HAL_UART_IRQHandler_00** UART0_IRQHandler
- **#define HAL_UART_IRQHandler_10** UART1_IRQHandler
- **#define HAL_CMP_IRQHandler_00** CMP_IRQHandler
- **#define HAL_ADC_IRQHandler_00** ADC0_IRQHandler
- **#define HAL_ADC_IRQHandler_10** ADC1_IRQHandler
- **#define HAL_WAKE_IRQHandler_00** WAKE_IRQHandler
- **#define HAL_QEP_IRQHandler_00** QEP0_IRQHandler
- **#define HAL_QEP_IRQHandler_10** QEP1_IRQHandler
- **#define HAL_DSP_IRQHandler_00** DSP0_IRQHandler
- **#define HAL_DMA_IRQHandler_00** DMA0_IRQHandler
- **#define HAL_CAN_IRQHandler_00** CAN0_IRQHandler

- #define HAL_SIF_IRQHandler_00 SIF0_IRQHandler
- #define HAL_SW_IRQHandler_00 SW_IRQHandler
- #define HAL_PWRDN_IRQHandler_00 PWRDN_IRQHandler
- #define HAL_CL_IRQHandler_00 CL0_IRQHandler
- #define HAL_INIT_IRQHandler_00 PWRDN_IRQHandler
- #define HAL_TIMER_IRQHandler HAL_TIMER_IRQHandler_00
- #define HAL_MCPWM_IRQHandler HAL_MCPWM_IRQHandler_00
- #define HAL_I2C_IRQHandler HAL_I2C_IRQHandler_00
- #define HAL_SPI_IRQHandler HAL_SPI_IRQHandler_00
- #define HAL_GPIO_IRQHandler HAL_GPIO_IRQHandler_00
- #define HAL_HALL_IRQHandler HAL_HALL_IRQHandler_00
- #define HAL_UART_IRQHandler HAL_UART_IRQHandler_00
- #define HAL_CMP_IRQHandler HAL_CMP_IRQHandler_00
- #define HAL_ADC_IRQHandler HAL_ADC_IRQHandler_00
- #define HAL_DMA_IRQHandler HAL_DMA_IRQHandler_00
- #define HAL_WAKE_IRQHandler HAL_WAKE_IRQHandler_00
- #define HAL_QEP_IRQHandler HAL_QEP_IRQHandler_00
- #define HAL_DSP_IRQHandler HAL_DSP_IRQHandler_00
- #define HAL_CAN_IRQHandler HAL_CAN_IRQHandler_00
- #define HAL_SIF_IRQHandler HAL_SIF_IRQHandler_00
- #define HAL_SW_IRQHandler HAL_SW_IRQHandler_00
- #define HAL_PWRDN_IRQHandler HAL_PWRDN_IRQHandler_00
- #define HAL_CL_IRQHandler HAL_CL_IRQHandler_00
- #define HAL_INIT_IRQHandler HAL_INIT_IRQHandler_00
- #define HAL_UART_IRQHandler_20 HAL_UART_IRQHandler_20_Null
- #define HAL_CMP_IRQHandler_10 HAL_CMP_IRQHandler_10_Null
- #define HAL_I2C_IRQHandler_10 HAL_I2C_IRQHandler_10_Null
- #define HAL_HALL_IRQHandler_10 HAL_HALL_IRQHandler_10_Null
- #define HAL_CMP_IRQHandler_20 HAL_CMP_IRQHandler_20_Null
- #define HAL_CMP_IRQHandler_30 HAL_CMP_IRQHandler_30_Null
- #define HAL_CMP_IRQHandler_40 HAL_CMP_IRQHandler_40_Null
- #define HAL_CMP_IRQHandler_50 HAL_CMP_IRQHandler_50_Null
- #define HAL_MCPWM_IRQHandler_10 HAL_MCPWM_IRQHandler_10_Null
- #define HAL_MCPWM_IRQHandler_11 HAL_MCPWM_IRQHandler_11_Null
- #define HAL_TIMER_IRQHandler_40 HAL_TIMER_IRQHandler_40_Null
- #define HAL_QEP_IRQHandler_20 HAL_QEP_IRQHandler_20_Null
- #define HAL_QEP_IRQHandler_30 HAL_QEP_IRQHandler_30_Null
- #define HAL_ADC_IRQHandler_20 HAL_ADC_IRQHandler_20_Null
- #define HAL_SPI_IRQHandler_10 HAL_SPI_IRQHandler_10_Null
- #define HAL_WWDG_IRQHandler_00 HAL_WWDG_IRQHandler_00_Null
- #define HAL_FMAC_IRQHandler_00 HAL_FMAC_IRQHandler_00_Null
- #define HAL_INIT_DEFAULE_MCLK 96000000
芯片的默认时钟频率
- #define HAL_INIT_DEFAULE_MCLK_US 96
- #define HAL_INIT_DEFAULE_MCLK_MS 96000

4.3.1 宏定义说明

4.3.1.1 HAL_ADC_IRQHandler

```
#define HAL_ADC_IRQHandler HAL_ADC_IRQHandler_00
```

在文件 [hal_iks32mc07x.h](#) 第 53 行定义。

4.3.1.2 HAL_ADC_IRQHandler_00

```
#define HAL_ADC_IRQHandler_00 ADC0_IRQHandler
```

在文件 [hal_lks32mc07x.h](#) 第 30 行定义.

4.3.1.3 HAL_ADC_IRQHandler_10

```
#define HAL_ADC_IRQHandler_10 ADC1_IRQHandler
```

在文件 [hal_lks32mc07x.h](#) 第 31 行定义.

4.3.1.4 HAL_ADC_IRQHandler_20

```
#define HAL_ADC_IRQHandler_20 HAL_ADC_IRQHandler_20_Null
```

在文件 [hal_lks32mc07x.h](#) 第 79 行定义.

4.3.1.5 HAL_CAN_IRQHandler

```
#define HAL_CAN_IRQHandler HAL_CAN_IRQHandler_00
```

在文件 [hal_lks32mc07x.h](#) 第 58 行定义.

4.3.1.6 HAL_CAN_IRQHandler_00

```
#define HAL_CAN_IRQHandler_00 CAN0_IRQHandler
```

在文件 [hal_lks32mc07x.h](#) 第 37 行定义.

4.3.1.7 HAL_CL_IRQHandler

```
#define HAL_CL_IRQHandler HAL_CL_IRQHandler_00
```

在文件 [hal_lks32mc07x.h](#) 第 62 行定义.

4.3.1.8 HAL_CL_IRQHandler_00

```
#define HAL_CL_IRQHandler_00 CL0_IRQHandler
```

在文件 [hal_lks32mc07x.h](#) 第 41 行定义.

4.3.1.9 HAL_CMP_IRQHandler

```
#define HAL_CMP_IRQHandler HAL_CMP_IRQHandler_00
```

在文件 [hal_lks32mc07x.h](#) 第 52 行定义.

4.3.1.10 HAL_CMP_IRQHandler_00

```
#define HAL_CMP_IRQHandler_00 CMP_IRQHandler
```

在文件 [hal_lks32mc07x.h](#) 第 29 行定义.

4.3.1.11 HAL_CMP_IRQHandler_10

```
#define HAL_CMP_IRQHandler_10 HAL_CMP_IRQHandler_10_Null
```

在文件 [hal_lks32mc07x.h](#) 第 67 行定义.

4.3.1.12 HAL_CMP_IRQHandler_20

```
#define HAL_CMP_IRQHandler_20 HAL_CMP_IRQHandler_20_Null
```

在文件 [hal_lks32mc07x.h](#) 第 70 行定义.

4.3.1.13 HAL_CMP_IRQHandler_30

```
#define HAL_CMP_IRQHandler_30 HAL_CMP_IRQHandler_30_Null
```

在文件 [hal_lks32mc07x.h](#) 第 71 行定义.

4.3.1.14 HAL_CMP_IRQHandler_40

```
#define HAL_CMP_IRQHandler_40 HAL_CMP_IRQHandler_40_Null
```

在文件 [hal_lks32mc07x.h](#) 第 72 行定义.

4.3.1.15 HAL_CMP_IRQHandler_50

```
#define HAL_CMP_IRQHandler_50 HAL_CMP_IRQHandler_50_Null
```

在文件 [hal_lks32mc07x.h](#) 第 73 行定义.

4.3.1.16 HAL_DMA_IRQHandler

```
#define HAL_DMA_IRQHandler HAL_DMA_IRQHandler_00
```

在文件 [hal_lks32mc07x.h](#) 第 54 行定义.

4.3.1.17 HAL_DMA_IRQHandler_00

```
#define HAL_DMA_IRQHandler_00 DMA0_IRQHandler
```

在文件 [hal_lks32mc07x.h](#) 第 36 行定义.

4.3.1.18 HAL_DSP_IRQHandler

```
#define HAL_DSP_IRQHandler HAL_DSP_IRQHandler_00
```

在文件 [hal_lks32mc07x.h](#) 第 57 行定义.

4.3.1.19 HAL_DSP_IRQHandler_00

```
#define HAL_DSP_IRQHandler_00 DSP0_IRQHandler
```

在文件 [hal_lks32mc07x.h](#) 第 35 行定义.

4.3.1.20 HAL_FMAC_IRQHandler_00

```
#define HAL_FMAC_IRQHandler_00 HAL_FMAC_IRQHandler_00_Null
```

在文件 [hal_lks32mc07x.h](#) 第 82 行定义.

4.3.1.21 HAL_GPIO_IRQHandler

```
#define HAL_GPIO_IRQHandler HAL_GPIO_IRQHandler_00
```

在文件 [hal_lks32mc07x.h](#) 第 49 行定义.

4.3.1.22 HAL_GPIO_IRQHandler_00

```
#define HAL_GPIO_IRQHandler_00 GPIO_IRQHandler
```

在文件 [hal_lks32mc07x.h](#) 第 25 行定义.

4.3.1.23 HAL_HALL_IRQHandler

```
#define HAL_HALL_IRQHandler HAL_HALL_IRQHandler_00
```

在文件 [hal_lks32mc07x.h](#) 第 50 行定义.

4.3.1.24 HAL_HALL_IRQHandler_00

```
#define HAL_HALL_IRQHandler_00 HALL0_IRQHandler
```

在文件 [hal_lks32mc07x.h](#) 第 26 行定义.

4.3.1.25 HAL_HALL_IRQHandler_10

```
#define HAL_HALL_IRQHandler_10 HAL_HALL_IRQHandler_10_Null
```

在文件 [hal_lks32mc07x.h](#) 第 69 行定义.

4.3.1.26 HAL_I2C_IRQHandler

```
#define HAL_I2C_IRQHandler HAL_I2C_IRQHandler_00
```

在文件 [hal_lks32mc07x.h](#) 第 47 行定义.

4.3.1.27 HAL_I2C_IRQHandler_00

```
#define HAL_I2C_IRQHandler_00 I2C0_IRQHandler
```

在文件 [hal_lks32mc07x.h](#) 第 23 行定义.

4.3.1.28 HAL_I2C_IRQHandler_10

```
#define HAL_I2C_IRQHandler_10 HAL_I2C_IRQHandler_10_Null
```

在文件 [hal_lks32mc07x.h](#) 第 68 行定义.

4.3.1.29 HAL_INIT_DEFAULE_MCLK

```
#define HAL_INIT_DEFAULE_MCLK 96000000
```

芯片的默认时钟频率

在文件 [hal_lks32mc07x.h](#) 第 87 行定义.

4.3.1.30 HAL_INIT_DEFAULE_MCLK_MS

```
#define HAL_INIT_DEFAULE_MCLK_MS 96000
```

在文件 [hal_lks32mc07x.h](#) 第 89 行定义.

4.3.1.31 HAL_INIT_DEFAULE_MCLK_US

```
#define HAL_INIT_DEFAULE_MCLK_US 96
```

在文件 [hal_lks32mc07x.h](#) 第 88 行定义.

4.3.1.32 HAL_INIT_IRQHandler

```
#define HAL_INIT_IRQHandler HAL_INIT_IRQHandler_00
```

在文件 [hal_lks32mc07x.h](#) 第 63 行定义.

4.3.1.33 HAL_INIT_IRQHandler_00

```
#define HAL_INIT_IRQHandler_00 PWRDN_IRQHandler
```

在文件 [hal_lks32mc07x.h](#) 第 42 行定义.

4.3.1.34 HAL_MCPWM_IRQHandler

```
#define HAL_MCPWM_IRQHandler HAL_MCPWM_IRQHandler_00
```

在文件 [hal_lks32mc07x.h](#) 第 46 行定义.

4.3.1.35 HAL_MCPWM_IRQHandler_00

```
#define HAL_MCPWM_IRQHandler_00 MCPWM0_IRQHandler
```

在文件 [hal_lks32mc07x.h](#) 第 21 行定义.

4.3.1.36 HAL_MCPWM_IRQHandler_01

```
#define HAL_MCPWM_IRQHandler_01 MCPWM1_IRQHandler
```

在文件 [hal_lks32mc07x.h](#) 第 22 行定义.

4.3.1.37 HAL_MCPWM_IRQHandler_10

```
#define HAL_MCPWM_IRQHandler_10 HAL_MCPWM_IRQHandler_10_Null
```

在文件 [hal_lks32mc07x.h](#) 第 74 行定义.

4.3.1.38 HAL_MCPWM_IRQHandler_11

```
#define HAL_MCPWM_IRQHandler_11 HAL_MCPWM_IRQHandler_11_Null
```

在文件 [hal_lks32mc07x.h](#) 第 75 行定义.

4.3.1.39 HAL_PWRDN_IRQHandler

```
#define HAL_PWRDN_IRQHandler HAL_PWRDN_IRQHandler_00
```

在文件 [hal_lks32mc07x.h](#) 第 61 行定义.

4.3.1.40 HAL_PWRDN_IRQHandler_00

```
#define HAL_PWRDN_IRQHandler_00 PWRDN_IRQHandler
```

在文件 [hal_lks32mc07x.h](#) 第 40 行定义.

4.3.1.41 HAL_QEP_IRQHandler

```
#define HAL_QEP_IRQHandler HAL_QEP_IRQHandler_00
```

在文件 [hal_lks32mc07x.h](#) 第 56 行定义.

4.3.1.42 HAL_QEP_IRQHandler_00

```
#define HAL_QEP_IRQHandler_00 QEP0_IRQHandler
```

在文件 [hal_lks32mc07x.h](#) 第 33 行定义.

4.3.1.43 HAL_QEP_IRQHandler_10

```
#define HAL_QEP_IRQHandler_10 QEP1_IRQHandler
```

在文件 [hal_lks32mc07x.h](#) 第 34 行定义.

4.3.1.44 HAL_QEP_IRQHandler_20

```
#define HAL_QEP_IRQHandler_20 HAL_QEP_IRQHandler_20_Null
```

在文件 [hal_lks32mc07x.h](#) 第 77 行定义.

4.3.1.45 HAL_QEP_IRQHandler_30

```
#define HAL_QEP_IRQHandler_30 HAL_QEP_IRQHandler_30_Null
```

在文件 [hal_lks32mc07x.h](#) 第 78 行定义.

4.3.1.46 HAL_SIF_IRQHandler

```
#define HAL_SIF_IRQHandler HAL_SIF_IRQHandler_00
```

在文件 [hal_lks32mc07x.h](#) 第 59 行定义.

4.3.1.47 HAL_SIF_IRQHandler_00

```
#define HAL_SIF_IRQHandler_00 SIF0_IRQHandler
```

在文件 [hal_lks32mc07x.h](#) 第 38 行定义.

4.3.1.48 HAL_SPI_IRQHandler

```
#define HAL_SPI_IRQHandler HAL_SPI_IRQHandler_00
```

在文件 [hal_lks32mc07x.h](#) 第 48 行定义.

4.3.1.49 HAL_SPI_IRQHandler_00

```
#define HAL_SPI_IRQHandler_00 SPI0_IRQHandler
```

在文件 [hal_lks32mc07x.h](#) 第 24 行定义。

4.3.1.50 HAL_SPI_IRQHandler_10

```
#define HAL_SPI_IRQHandler_10 HAL_SPI_IRQHandler_10_Null
```

在文件 [hal_lks32mc07x.h](#) 第 80 行定义。

4.3.1.51 HAL_SW_IRQHandler

```
#define HAL_SW_IRQHandler HAL_SW_IRQHandler_00
```

在文件 [hal_lks32mc07x.h](#) 第 60 行定义。

4.3.1.52 HAL_SW_IRQHandler_00

```
#define HAL_SW_IRQHandler_00 SW_IRQHandler
```

在文件 [hal_lks32mc07x.h](#) 第 39 行定义。

4.3.1.53 HAL_TIMER_IRQHandler

```
#define HAL_TIMER_IRQHandler HAL_TIMER_IRQHandler_00
```

在文件 [hal_lks32mc07x.h](#) 第 45 行定义。

4.3.1.54 HAL_TIMER_IRQHandler_00

```
#define HAL_TIMER_IRQHandler_00 TIMER0_IRQHandler
```

统一中断函数接口 命名方式：HAL_模块名称_IRQHandler_模块序号中断序号。例如MCPWM模块有两个CNT,会产生两个中断。这里分别对应MCPWM0_IRQHandler和MCPWM1_IRQHandler。为方便使用,末尾的_00可以省略。

在文件 [hal_lks32mc07x.h](#) 第 17 行定义。

4.3.1.55 HAL_TIMER_IRQHandler_10

```
#define HAL_TIMER_IRQHandler_10 TIMER1_IRQHandler
```

在文件 [hal_lks32mc07x.h](#) 第 18 行定义。

4.3.1.56 HAL_TIMER_IRQHandler_20

```
#define HAL_TIMER_IRQHandler_20 TIMER2_IRQHandler
```

在文件 [hal_lks32mc07x.h](#) 第 19 行定义.

4.3.1.57 HAL_TIMER_IRQHandler_30

```
#define HAL_TIMER_IRQHandler_30 TIMER3_IRQHandler
```

在文件 [hal_lks32mc07x.h](#) 第 20 行定义.

4.3.1.58 HAL_TIMER_IRQHandler_40

```
#define HAL_TIMER_IRQHandler_40 HAL_TIMER_IRQHandler_40_Null
```

在文件 [hal_lks32mc07x.h](#) 第 76 行定义.

4.3.1.59 HAL_UART_IRQHandler

```
#define HAL_UART_IRQHandler HAL_UART_IRQHandler_00
```

在文件 [hal_lks32mc07x.h](#) 第 51 行定义.

4.3.1.60 HAL_UART_IRQHandler_00

```
#define HAL_UART_IRQHandler_00 UART0_IRQHandler
```

在文件 [hal_lks32mc07x.h](#) 第 27 行定义.

4.3.1.61 HAL_UART_IRQHandler_10

```
#define HAL_UART_IRQHandler_10 UART1_IRQHandler
```

在文件 [hal_lks32mc07x.h](#) 第 28 行定义.

4.3.1.62 HAL_UART_IRQHandler_20

```
#define HAL_UART_IRQHandler_20 HAL_UART_IRQHandler_20_Null
```

在文件 [hal_lks32mc07x.h](#) 第 66 行定义.

4.3.1.63 HAL_WAKE_IRQHandler

```
#define HAL_WAKE_IRQHandler HAL_WAKE_IRQHandler_00
```

在文件 [hal_lks32mc07x.h](#) 第 55 行定义.

4.3.1.64 HAL_WAKE_IRQHandler_00

```
#define HAL_WAKE_IRQHandler_00 WAKE_IRQHandler
```

在文件 [hal_lks32mc07x.h](#) 第 32 行定义。

4.3.1.65 HAL_WWDG_IRQHandler_00

```
#define HAL_WWDG_IRQHandler_00 HAL_WWDG_IRQHandler_00_Null
```

在文件 [hal_lks32mc07x.h](#) 第 81 行定义。

4.4 hal_lks32mc07x.h

[浏览该文件的文档。](#)

```
00001 #include "basic.h"
00002 #include "stdint.h"
00003 #include "lks32mc07x.h"
00004
00005 #ifndef __HAL_LKS32MC07X_H__
00006 #define __HAL_LKS32MC07X_H__
00007
00017 #define HAL_TIMER_IRQHandler_00 TIMER0_IRQHandler
00018 #define HAL_TIMER_IRQHandler_10 TIMER1_IRQHandler
00019 #define HAL_TIMER_IRQHandler_20 TIMER2_IRQHandler
00020 #define HAL_TIMER_IRQHandler_30 TIMER3_IRQHandler
00021 #define HAL_MCPWM_IRQHandler_00 MCPWM0_IRQHandler
00022 #define HAL_MCPWM_IRQHandler_01 MCPWM1_IRQHandler
00023 #define HAL_I2C_IRQHandler_00 I2C0_IRQHandler
00024 #define HAL_SPI_IRQHandler_00 SPI0_IRQHandler
00025 #define HAL_GPIO_IRQHandler_00 GPIO_IRQHandler
00026 #define HAL_HALL_IRQHandler_00 HALL0_IRQHandler
00027 #define HAL_UART_IRQHandler_00 UART0_IRQHandler
00028 #define HAL_UART_IRQHandler_10 UART1_IRQHandler
00029 #define HAL_CMP_IRQHandler_00 CMP_IRQHandler
00030 #define HAL_ADC_IRQHandler_00 ADC0_IRQHandler
00031 #define HAL_ADC_IRQHandler_10 ADC1_IRQHandler
00032 #define HAL_WAKE_IRQHandler_00 WAKE_IRQHandler
00033 #define HAL_QEP_IRQHandler_00 QEP0_IRQHandler
00034 #define HAL_QEP_IRQHandler_10 QEP1_IRQHandler
00035 #define HAL_DSP_IRQHandler_00 DSP0_IRQHandler
00036 #define HAL_DMA_IRQHandler_00 DMA0_IRQHandler
00037 #define HAL_CAN_IRQHandler_00 CAN0_IRQHandler
00038 #define HAL_SIF_IRQHandler_00 SIF0_IRQHandler
00039 #define HAL_SW_IRQHandler_00 SW_IRQHandler
00040 #define HAL_PWRDN_IRQHandler_00 PWRDN_IRQHandler
00041 #define HAL_CL_IRQHandler_00 CL0_IRQHandler
00042 #define HAL_INIT_IRQHandler_00 PWRDN_IRQHandler
00043
00044 // 简化写法
00045 #define HAL_TIMER_IRQHandler HAL_TIMER_IRQHandler_00
00046 #define HAL_MCPWM_IRQHandler HAL_MCPWM_IRQHandler_00
00047 #define HAL_I2C_IRQHandler HAL_I2C_IRQHandler_00
00048 #define HAL_SPI_IRQHandler HAL_SPI_IRQHandler_00
00049 #define HAL_GPIO_IRQHandler HAL_GPIO_IRQHandler_00
00050 #define HAL_HALL_IRQHandler HAL_HALL_IRQHandler_00
00051 #define HAL_UART_IRQHandler HAL_UART_IRQHandler_00
00052 #define HAL_CMP_IRQHandler HAL_CMP_IRQHandler_00
00053 #define HAL_ADC_IRQHandler HAL_ADC_IRQHandler_00
00054 #define HAL_DMA_IRQHandler HAL_DMA_IRQHandler_00
00055 #define HAL_WAKE_IRQHandler HAL_WAKE_IRQHandler_00
00056 #define HAL_QEP_IRQHandler HAL_QEP_IRQHandler_00
00057 #define HAL_DSP_IRQHandler HAL_DSP_IRQHandler_00
00058 #define HAL_CAN_IRQHandler HAL_CAN_IRQHandler_00
00059 #define HAL_SIF_IRQHandler HAL_SIF_IRQHandler_00
00060 #define HAL_SW_IRQHandler HAL_SW_IRQHandler_00
00061 #define HAL_PWRDN_IRQHandler HAL_PWRDN_IRQHandler_00
00062 #define HAL_CL_IRQHandler HAL_CL_IRQHandler_00
00063 #define HAL_INIT_IRQHandler HAL_INIT_IRQHandler_00
00064
00065 // 当前芯片不支持的中断函数
00066 #define HAL_UART_IRQHandler_20 HAL_UART_IRQHandler_20_Null
00067 #define HAL_CMP_IRQHandler_10 HAL_CMP_IRQHandler_10_Null
```

```

00068 #define HAL_I2C_IRQHandler_10 HAL_I2C_IRQHandler_10_Null
00069 #define HAL_HALL_IRQHandler_10 HAL_HALL_IRQHandler_10_Null
00070 #define HAL_CMP_IRQHandler_20 HAL_CMP_IRQHandler_20_Null
00071 #define HAL_CMP_IRQHandler_30 HAL_CMP_IRQHandler_30_Null
00072 #define HAL_CMP_IRQHandler_40 HAL_CMP_IRQHandler_40_Null
00073 #define HAL_CMP_IRQHandler_50 HAL_CMP_IRQHandler_50_Null
00074 #define HAL_MCPWM_IRQHandler_10 HAL_MCPWM_IRQHandler_10_Null
00075 #define HAL_MCPWM_IRQHandler_11 HAL_MCPWM_IRQHandler_11_Null
00076 #define HAL_TIMER_IRQHandler_40 HAL_TIMER_IRQHandler_40_Null
00077 #define HAL_QEP_IRQHandler_20 HAL_QEP_IRQHandler_20_Null
00078 #define HAL_QEP_IRQHandler_30 HAL_QEP_IRQHandler_30_Null
00079 #define HAL_ADC_IRQHandler_20 HAL_ADC_IRQHandler_20_Null
00080 #define HAL_SPI_IRQHandler_10 HAL_SPI_IRQHandler_10_Null
00081 #define HAL_WWDG_IRQHandler_00 HAL_WWDG_IRQHandler_00_Null
00082 #define HAL_FMAC_IRQHandler_00 HAL_FMAC_IRQHandler_00_Null
00083
00087 #define HAL_INIT_DEFAULT_MCLK 96000000
00088 #define HAL_INIT_DEFAULT_MCLK_US 96 // 上电后的默认时钟1us分频系数
00089 #define HAL_INIT_DEFAULT_MCLK_MS 96000 // 上电后的默认时钟1ms分频系数
00090
00091 #include "include/hal_lksmcu_adc.h"
00092 #include "include/hal_lksmcu_can.h"
00093 #include "include/hal_lksmcu_clk.h"
00094 #include "include/hal_lksmcu_cmp.h"
00095 #include "include/hal_lksmcu_crc.h"
00096 #include "include/hal_lksmcu_dac.h"
00097 #include "include/hal_lksmcu_dma.h"
00098 #include "include/hal_lksmcu_dsp.h"
00099 #include "include/hal_lksmcu_flash.h"
00100 #include "include/hal_lksmcu_gpio.h"
00101 #include "include/hal_lksmcu_hall.h"
00102 #include "include/hal_lksmcu_i2c.h"
00103 #include "include/hal_lksmcu_irq.h"
00104 #include "include/hal_lksmcu_init.h"
00105 #include "include/hal_lksmcu_iwdg.h"
00106 #include "include/hal_lksmcu_mcpwm.h"
00107 #include "include/hal_lksmcu_opa.h"
00108 #include "include/hal_lksmcu_qep.h"
00109 #include "include/hal_lksmcu_reg.h"
00110 #include "include/hal_lksmcu_reset.h"
00111 #include "include/hal_lksmcu_spi.h"
00112 #include "include/hal_lksmcu_timer.h"
00113 #include "include/hal_lksmcu_tmp.h"
00114 #include "include/hal_lksmcu_trim.h"
00115 #include "include/hal_lksmcu_uart.h"
00116 #include "include/hal_lksmcu_wake.h"
00117 #include "include/hal_lksmcu_irqfun.h"
00118 #endif

```

4.5 hal_lksmcu.h 文件参考

```
#include "hal_lks32mc07x.h"
```

hal_lksmcu.h 的引用(Include)关系图:



4.6 hal_lksmcu.h

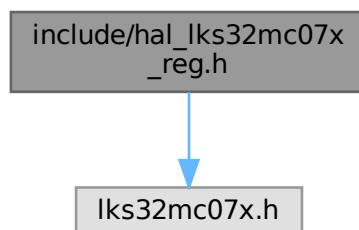
[浏览该文件的文档.](#)

```
00001 #include "hal_lks32mc07x.h"
```

4.7 include/hal_lks32mc07x_reg.h 文件参考

```
#include "lks32mc07x.h"
```

hal_lks32mc07x_reg.h 的引用(Include)关系图:



此图展示该文件直接或间接的被哪些文件引用了:



结构体

- struct [HAL_SYS_TypeDef](#)
SYS寄存器定义
- struct [HAL_FLASH_TypeDef](#)
FLASH寄存器定义
- struct [HAL_SPI_TypeDef](#)
SPI寄存器定义
- struct [HAL_I2C_TypeDef](#)
I2C寄存器定义
- struct [HAL_CMP_TypeDef](#)
CMP寄存器定义
- struct [HAL_HALL_TypeDef](#)
HALL寄存器定义
- struct [HAL_ADC_TypeDef](#)
ADC0寄存器定义
- struct [HAL_TIMER_TypeDef](#)
TIMER寄存器定义
- struct [HAL_QEP_TypeDef](#)
QEP寄存器定义
- struct [HAL_MCPWM_TypeDef](#)
MCPWM寄存器定义
- struct [HAL_GPIO_TypeDef](#)
GPIO寄存器定义
- struct [HAL_CRC_TypeDef](#)

*CRC*寄存器定义

- struct [HAL_UART_TypeDef](#)

*UART*寄存器定义

- struct [HAL_DSP_TypeDef](#)

*DSP*寄存器定义

- struct [HAL_IWDG_TypeDef](#)
- struct [HAL_DMA_TypeDef](#)

变量

- [HAL_SYS_TypeDef](#) *const [HAL_SYS](#) []
- [HAL_FLASH_TypeDef](#) *const [HAL_FLASH](#) []
- [HAL_SPI_TypeDef](#) *const [HAL_SPI](#) []
- [HAL_I2C_TypeDef](#) *const [HAL_I2C](#) []
- [HAL_CMP_TypeDef](#) *const [HAL_CMP](#) []
- [HAL_HALL_TypeDef](#) *const [HAL_HALL](#) []
- [HAL_ADC_TypeDef](#) *const [HAL_ADC](#) []
- [HAL_TIMER_TypeDef](#) *const [HAL_TIMER](#) []
- [HAL_QEP_TypeDef](#) *const [HAL_QEP](#) []
- [HAL_MCPWM_TypeDef](#) *const [HAL_MCPWM](#) []
- [HAL_GPIO_TypeDef](#) *const [HAL_GPIO](#) []
- [HAL_CRC_TypeDef](#) *const [HAL_CRC](#) []
- [HAL_UART_TypeDef](#) *const [HAL_UART](#) []
- [HAL_DSP_TypeDef](#) *const [HAL_DSP](#) []
- [HAL_IWDG_TypeDef](#) *const [HAL_IWDG](#) []
- [HAL_DMA_TypeDef](#) *const [HAL_DMA](#) []

4.7.1 变量说明

4.7.1.1 HAL_ADC

```
HAL\_ADC\_TypeDef* const HAL\_ADC[] [extern]
```

在文件 [hal_lks32mc07x_reg.c](#) 第 12 行定义.

4.7.1.2 HAL_CMP

```
HAL\_CMP\_TypeDef* const HAL\_CMP[] [extern]
```

在文件 [hal_lks32mc07x_reg.c](#) 第 10 行定义.

4.7.1.3 HAL_CRC

```
HAL\_CRC\_TypeDef* const HAL\_CRC[] [extern]
```

在文件 [hal_lks32mc07x_reg.c](#) 第 17 行定义.

4.7.1.4 HAL_DMA

`HAL_DMA_TypeDef* const HAL_DMA[] [extern]`

在文件 `hal_lks32mc07x_reg.c` 第 21 行定义.

4.7.1.5 HAL_DSP

`HAL_DSP_TypeDef* const HAL_DSP[] [extern]`

在文件 `hal_lks32mc07x_reg.c` 第 19 行定义.

4.7.1.6 HAL_FLASH

`HAL_FLASH_TypeDef* const HAL_FLASH[] [extern]`

在文件 `hal_lks32mc07x_reg.c` 第 7 行定义.

4.7.1.7 HAL_GPIO

`HAL_GPIO_TypeDef* const HAL_GPIO[] [extern]`

在文件 `hal_lks32mc07x_reg.c` 第 16 行定义.

4.7.1.8 HAL_HALL

`HAL_HALL_TypeDef* const HAL_HALL[] [extern]`

在文件 `hal_lks32mc07x_reg.c` 第 11 行定义.

4.7.1.9 HAL_I2C

`HAL_I2C_TypeDef* const HAL_I2C[] [extern]`

在文件 `hal_lks32mc07x_reg.c` 第 9 行定义.

4.7.1.10 HAL_IWDG

`HAL_IWDG_TypeDef* const HAL_IWDG[] [extern]`

在文件 `hal_lks32mc07x_reg.c` 第 20 行定义.

4.7.1.11 HAL_MCPWM

`HAL_MCPWM_TypeDef* const HAL_MCPWM[] [extern]`

在文件 `hal_lks32mc07x_reg.c` 第 15 行定义.

4.7.1.12 HAL_QEP

```
HAL_QEP_TypeDef* const HAL_QEP[] [extern]
```

在文件 [hal_lks32mc07x_reg.c](#) 第 14 行定义.

4.7.1.13 HAL_SPI

```
HAL_SPI_TypeDef* const HAL_SPI[] [extern]
```

在文件 [hal_lks32mc07x_reg.c](#) 第 8 行定义.

4.7.1.14 HAL_SYS

```
HAL_SYS_TypeDef* const HAL_SYS[] [extern]
```

在文件 [hal_lks32mc07x_reg.c](#) 第 6 行定义.

4.7.1.15 HAL_TIMER

```
HAL_TIMER_TypeDef* const HAL_TIMER[] [extern]
```

在文件 [hal_lks32mc07x_reg.c](#) 第 13 行定义.

4.7.1.16 HAL_UART

```
HAL_UART_TypeDef* const HAL_UART[] [extern]
```

在文件 [hal_lks32mc07x_reg.c](#) 第 18 行定义.

4.8 hal_lks32mc07x_reg.h

[浏览该文件的文档.](#)

```
00001
00002 #ifndef __HAL_LKS32MC07X_REG_H__
00003 #define __HAL_LKS32MC07X_REG_H__
00004 #include "lks32mc07x.h"
00008 typedef struct
00009 {
00010     volatile uint32_t ADC;          // 0x00
00011     volatile uint32_t INFO;         // 0x04
00012     volatile uint32_t DBG;          // 0x08
00013     volatile uint32_t null0;        // 0x0C
00014     volatile uint32_t REG0;         // 0x10
00015     volatile uint32_t REG1;         // 0x14
00016     volatile uint32_t REG2;         // 0x18
00017     volatile uint32_t REG3;         // 0x1C
00018     volatile uint32_t REG4;         // 0x20
00019     volatile uint32_t REG5;         // 0x24
00020     volatile uint32_t REG6;         // 0x28
00021     volatile uint32_t REG7;         // 0x2C
00022     volatile uint32_t REG8;         // 0x30
00023     volatile uint32_t REG9;         // 0x34
00024     volatile uint32_t REGA;         // 0x38
00025     volatile uint32_t REGB;         // 0x3C
00026     volatile uint32_t REGC;         // 0x40
```



```

00027     volatile uint32_t null1;    // 0x44
00028     volatile uint32_t null2;    // 0x48
00029     volatile uint32_t null3;    // 0x4C
00030     volatile uint32_t null4;    // 0x50
00031     volatile uint32_t null5;    // 0x54
00032     volatile uint32_t null6;    // 0x58
00033     volatile uint32_t DAC_CTRL;  // 0x5C
00034     volatile uint32_t DAC0;     // 0x60
00035     volatile uint32_t DAC1;     // 0x64
00036     volatile uint32_t DAC0_AMC; // 0x68
00037     volatile uint32_t DAC0_DC;  // 0x6C
00038     volatile uint32_t DAC1_AMC; // 0x70
00039     volatile uint32_t DAC1_DC;  // 0x74
00040
00041     volatile uint32_t null7;    // 0x78
00042     volatile uint32_t null8;    // 0x7C
00043
00044     volatile uint32_t CLK_CFG;   // 0x80
00045     volatile uint32_t IO_CFG;    // 0x84
00046     volatile uint32_t DBG_CFG;   // 0x88
00047     volatile uint32_t null9;     // 0x8c
00048     volatile uint32_t CLK_DIV0;  // 0x90 I2C clock div
00049     volatile uint32_t CLK_DIV1;  // 0x94 保留
00050     volatile uint32_t CLK_DIV2;  // 0x98 UART0/1 clock div
00051     volatile uint32_t CLK_FEN;   // 0x9C peripheral clock enable
00052     volatile uint32_t TRIM;      // 0xA0
00053     volatile uint32_t SFT_RST;   // 0xA4
00054     volatile uint32_t WR_PROTECT; // 0xA8
00055     volatile uint32_t MBIST;     // 0xC0
00056     volatile uint32_t null10;    // 0xc4
00057     volatile uint32_t null11;    // 0xc8
00058     volatile uint32_t null12;    // 0xcc
00059     volatile uint32_t FLSE;      // 0xD0
00060     volatile uint32_t FLSP;      // 0xD4
00061     volatile uint32_t FLST;      // 0xD8
00062     volatile uint32_t null13;    // 0xDC
00063     volatile uint32_t SBIST;     // 0xE0
00064 } HAL_SYS_TypeDef;
00065
00069 typedef struct
00070 {
00071     volatile uint32_t CFG;
00072     volatile uint32_t ADDR;
00073     volatile uint32_t WDATA;
00074     volatile uint32_t RDATA;
00075     volatile uint32_t ERASE;
00076     volatile uint32_t PROTECT;
00077     volatile uint32_t READY;
00078     volatile uint32_t SIZE;
00079     volatile uint32_t NCFG;
00080     volatile uint32_t NADDR;
00081     volatile uint32_t NWDATA;
00082     volatile uint32_t NRDATA;
00083     volatile uint32_t NERASE;
00084     volatile uint32_t NKEY;
00085 } HAL_FLASH_TypeDef;
00089 typedef struct
00090 {
00091     volatile uint32_t CFG;
00092     volatile uint32_t IE;
00093     volatile uint32_t BAUD;
00094     volatile uint32_t TXDATA;
00095     volatile uint32_t RXDATA;
00096     volatile uint32_t SIZE;
00097 } HAL_SPI_TypeDef;
00098
00102 typedef struct
00103 {
00104     volatile uint32_t ADDR;
00105     volatile uint32_t CFG;
00106     volatile uint32_t SCR;
00107     volatile uint32_t DATA;
00108     volatile uint32_t MSCR;
00109     volatile uint32_t BCR;
00110     volatile uint32_t BSIZE;
00111 } HAL_I2C_TypeDef;
00112
00116 typedef struct
00117 {
00118     volatile uint32_t IE;
00119     volatile uint32_t IF;
00120     volatile uint32_t TCLK;
00121     volatile uint32_t CFG;
00122     volatile uint32_t BLCWIN;
00123     volatile uint32_t DATA;
00124 } HAL_CMP_TypeDef;
00125

```

```

00129 typedef struct
00130 {
00131     volatile uint32_t CFG;
00132     volatile uint32_t INFO;
00133     volatile uint32_t WIDTH;
00134     volatile uint32_t TH;
00135     volatile uint32_t CNT;
00136 } HAL_HALL_TypeDef;
00137
00141 typedef struct
00142 {
00143     volatile uint32_t DAT[14];
00144     volatile uint32_t IDAT[2];
00145     volatile uint32_t ICHN;
00146     volatile uint32_t null[3];
00147     volatile uint32_t CHN0;
00148     volatile uint32_t CHN1;
00149     volatile uint32_t CHN2;
00150     volatile uint32_t CHN3;
00151     volatile uint32_t CHNT;
00152     volatile uint32_t GAIN;
00153     volatile uint32_t null2[3];
00154     volatile uint32_t CFG;
00155     volatile uint32_t TRIG;
00156     volatile uint32_t SWT;
00157     volatile uint32_t DCO;
00158     volatile uint32_t AMCO;
00159     volatile uint32_t DC1;
00160     volatile uint32_t AMC1;
00161     volatile uint32_t IE;
00162     volatile uint32_t IF;
00163     volatile uint32_t null3[11];
00164     volatile uint32_t LTH;
00165     volatile uint32_t HTH;
00166     volatile uint32_t GEN;
00167 } HAL_ADC_TypeDef;
00168
00172 typedef struct
00173 {
00174     volatile uint32_t CFG;
00175     volatile uint32_t TH;
00176     volatile uint32_t CNT;
00177     volatile uint32_t CMP0;
00178     volatile uint32_t CMP1;
00179     volatile uint32_t EVT;
00180     volatile uint32_t FLT;
00181     volatile uint32_t IE;
00182     volatile uint32_t IF;
00183     volatile uint32_t IO;
00184 } HAL_TIMER_TypeDef;
00185
00189 typedef struct
00190 {
00191     volatile uint32_t CFG;
00192     volatile uint32_t TH;
00193     volatile uint32_t CNT;
00194     volatile uint32_t IE;
00195     volatile uint32_t IF;
00196 } HAL_QEP_TypeDef;
00200 typedef struct
00201 {
00202
00203     volatile uint32_t TH[12];
00204     volatile uint32_t TMR0;
00205     volatile uint32_t TMR1;
00206     volatile uint32_t TMR2;
00207     volatile uint32_t TMR3;
00208     volatile uint32_t TH0;
00209     volatile uint32_t TH1;
00210     volatile uint32_t CNT0;
00211     volatile uint32_t CNT1;
00212     volatile uint32_t UPDATE;
00213     volatile uint32_t FCNT;
00214     volatile uint32_t EVT0;
00215     volatile uint32_t EVT1;
00216     volatile uint32_t DTH00;
00217     volatile uint32_t DTH01;
00218     volatile uint32_t DTH10;
00219     volatile uint32_t DTH11;
00220     volatile uint32_t FLT;
00221     volatile uint32_t SDCFG;
00222     volatile uint32_t AUEN;
00223     volatile uint32_t TCLK;
00224     volatile uint32_t IE0;
00225     volatile uint32_t IF0;
00226     volatile uint32_t IE1;
00227     volatile uint32_t IF1;

```

```

00228     volatile uint32_t EIE;
00229     volatile uint32_t EIF;
00230     volatile uint32_t RE;
00231     volatile uint32_t PP;
00232     volatile uint32_t IO01;
00233     volatile uint32_t IO23;
00234     volatile uint32_t IO45;
00235     volatile uint32_t null;
00236     volatile uint32_t FAIL012;
00237     volatile uint32_t FAIL345;
00238     volatile uint32_t CH_DEF;
00239     volatile uint32_t CH_MSK;
00240     volatile uint32_t PRT;
00241     volatile uint32_t STT_HYST;
00242     volatile uint32_t ZCS_DELAY;
00243 } HAL_MCPWM_TypeDef;
00244
00248 typedef struct
00249 {
00250     volatile uint32_t PIE;
00251     volatile uint32_t POE;
00252     volatile uint32_t PDI;
00253     volatile uint32_t PDO;
00254     volatile uint32_t PUE;
00255     volatile uint32_t null1; // 地址对齐
00256     volatile uint32_t PODE;
00257     volatile uint32_t PFLT;
00258     volatile uint32_t F3210;
00259     volatile uint32_t F7654;
00260     volatile uint32_t FBA98;
00261     volatile uint32_t FFEDC;
00262     volatile uint32_t BSRR;
00263     volatile uint32_t BRR;
00264 } HAL_GPIO_TypeDef;
00265
00269 typedef struct
00270 {
00271     volatile uint32_t DR;
00272     volatile uint32_t CR;
00273     volatile uint32_t INIT;
00274     volatile uint32_t POL;
00275 } HAL_CRC_TypeDef;
00276
00280 typedef struct
00281 {
00282     volatile uint32_t CTRL;
00283     volatile uint32_t DIVH;
00284     volatile uint32_t DIVL;
00285     volatile uint32_t BUFF;
00286     volatile uint32_t ADR;
00287     volatile uint32_t STT;
00288     volatile uint32_t RE;
00289     volatile uint32_t IE;
00290     volatile uint32_t IF;
00291     volatile uint32_t IOC;
00292 } HAL_UART_TypeDef;
00293
00297 typedef struct
00298 {
00299     volatile uint32_t SC; // 0x00
00300     volatile uint32_t THETA; // 0x04
00301     volatile uint32_t X; // 0x08
00302     volatile uint32_t Y; // 0x0C
00303     volatile uint32_t SIN; // 0x10
00304     volatile uint32_t COS; // 0x14
00305     volatile uint32_t MOD; // 0x18
00306     volatile uint32_t ARCTAN; // 0x1C
00307     volatile uint32_t DID; // 0x20
00308     volatile uint32_t DIS; // 0x24
00309     volatile uint32_t QUO; // 0x28
00310     volatile uint32_t REM; // 0x2C
00311     volatile uint32_t RAD; // 0x30
00312     volatile uint32_t SQRT; // 0x34
00313     volatile uint32_t PC; // 0x38
00314 } HAL_DSP_TypeDef;
00315
00316 typedef struct
00317 {
00318     volatile uint32_t PSW;
00319     volatile uint32_t CFG;
00320     volatile uint32_t CLR;
00321     volatile uint32_t WTH;
00322     volatile uint32_t RTH;
00323     volatile uint32_t CNT;
00324 } HAL_IWDG_TypeDef;
00325
00326 typedef struct

```

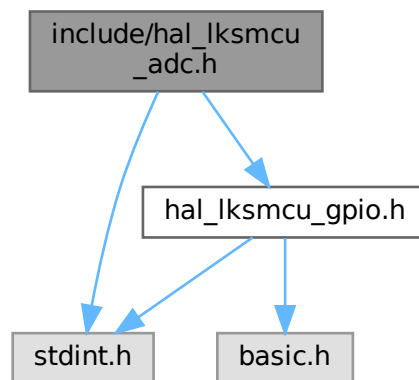
```

00327 {
00328     volatile uint32_t CCR;
00329     volatile uint32_t REN;
00330     volatile uint32_t CTMS;
00331     volatile uint32_t SADR;
00332     volatile uint32_t DADR;
00333 } HAL_DMA_TypeDef;
00334
00335 extern HAL_SYS_TypeDef *const HAL_SYS[];
00336 extern HAL_FLASH_TypeDef *const HAL_FLASH[];
00337 extern HAL_SPI_TypeDef *const HAL_SPI[];
00338 extern HAL_I2C_TypeDef *const HAL_I2C[];
00339 extern HAL_COMP_TypeDef *const HAL_COMP[];
00340 extern HAL_HALL_TypeDef *const HAL_HALL[];
00341 extern HAL_ADC_TypeDef *const HAL_ADC[];
00342 extern HAL_TIMER_TypeDef *const HAL_TIMER[];
00343 extern HAL_QEP_TypeDef *const HAL_QEP[];
00344 extern HAL_MCPWM_TypeDef *const HAL_MCPWM[];
00345 extern HAL_GPIO_TypeDef *const HAL_GPIO[];
00346 extern HAL_CRC_TypeDef *const HAL_CRC[];
00347 extern HAL_UART_TypeDef *const HAL_UART[];
00348 extern HAL_DSP_TypeDef *const HAL_DSP[];
00349 extern HAL_IWDG_TypeDef *const HAL_IWDG[];
00350 extern HAL_DMA_TypeDef *const HAL_DMA[];
00351 #endif

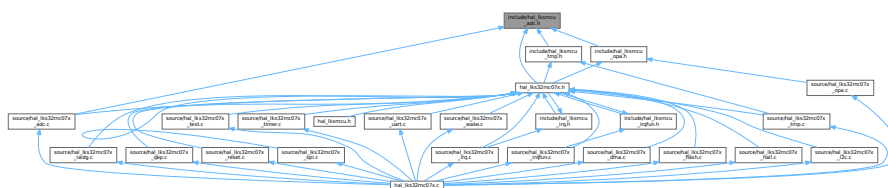
```

4.9 include/hal_lksmcu_adc.h 文件参考

#include "stdint.h"
#include "hal_lksmcu_gpio.h"
hal_lksmcu_adc.h 的引用(Include)关系图:



此图展示该文件直接或间接的被哪些文件引用了:



枚举

- enum HAL_ADC_PORT_x {
 HAL_ADC_PORT_CH0, HAL_ADC_PORT_CH1, HAL_ADC_PORT_CH2, HAL_ADC_PORT_CH3,
 HAL_ADC_PORT_CH4, HAL_ADC_PORT_CH5, HAL_ADC_PORT_CH6, HAL_ADC_PORT_CH7,
 HAL_ADC_PORT_CH8, HAL_ADC_PORT_CH9, HAL_ADC_PORT_CH10, HAL_ADC_PORT_CH11,
 HAL_ADC_PORT_CH12, HAL_ADC_PORT_CH13, HAL_ADC_PORT_CH14, HAL_ADC_PORT_CH15,
 HAL_ADC_PORT_OPA0 = HAL_ADC_PORT_CH0, HAL_ADC_PORT_OPA1 = HAL_ADC_PORT_CH1,
 HAL_ADC_PORT_OPA2 = HAL_ADC_PORT_CH2, HAL_ADC_PORT_OPA3 = HAL_ADC_PORT_CH3,
 HAL_ADC_PORT_DAC0 = HAL_ADC_PORT_CH14, HAL_ADC_PORT_DAC1 = HAL_ADC_PORT_CH15,
 HAL_ADC_PORT_TMP0 = HAL_ADC_PORT_CH14, HAL_ADC_PORT_VDD = HAL_ADC_PORT_CH15,
 HAL_ADC_PORT_REF, HAL_ADC_PORT_AGND, HAL_ADC_PORT_OPA4, HAL_ADC_PORT_OPA5,
 HAL_ADC_PORT_CH16, HAL_ADC_PORT_CH17, HAL_ADC_PORT_CH18, HAL_ADC_PORT_CH19,
 HAL_ADC_PORT_LDO12 }
 主频
- enum HAL_ADC_SAMP_SEQUENCE_x {
 HAL_ADC_SAMP_SEQUENCE_0, HAL_ADC_SAMP_SEQUENCE_1, HAL_ADC_SAMP_SEQUENCE_2,
 HAL_ADC_SAMP_SEQUENCE_3,
 HAL_ADC_SAMP_SEQUENCE_4, HAL_ADC_SAMP_SEQUENCE_5, HAL_ADC_SAMP_SEQUENCE_6,
 HAL_ADC_SAMP_SEQUENCE_7,
 HAL_ADC_SAMP_SEQUENCE_8, HAL_ADC_SAMP_SEQUENCE_9, HAL_ADC_SAMP_SEQUENCE_10,
 HAL_ADC_SAMP_SEQUENCE_11,
 HAL_ADC_SAMP_SEQUENCE_12, HAL_ADC_SAMP_SEQUENCE_13, HAL_ADC_SAMP_SEQUENCE_14,
 HAL_ADC_SAMP_SEQUENCE_15,
 HAL_ADC_SAMP_SEQUENCE_16, HAL_ADC_SAMP_SEQUENCE_17, HAL_ADC_SAMP_SEQUENCE_18,
 HAL_ADC_SAMP_SEQUENCE_19 }
 ADC采样序列
- enum HAL_ADC_x { HAL_ADC_0, HAL_ADC_1, HAL_ADC_2, HAL_ADC_3 }
- enum HAL_ADC_SAMP_NUMBER_x {
 HAL_ADC_SAMP_NUMBER_0, HAL_ADC_SAMP_NUMBER_1, HAL_ADC_SAMP_NUMBER_2, HAL_ADC_SAMP_NUMBER_3,
 HAL_ADC_SAMP_NUMBER_4, HAL_ADC_SAMP_NUMBER_5, HAL_ADC_SAMP_NUMBER_6, HAL_ADC_SAMP_NUMBER_7,
 HAL_ADC_SAMP_NUMBER_8, HAL_ADC_SAMP_NUMBER_9, HAL_ADC_SAMP_NUMBER_10, HAL_ADC_SAMP_NUMBER_11,
 HAL_ADC_SAMP_NUMBER_12, HAL_ADC_SAMP_NUMBER_13, HAL_ADC_SAMP_NUMBER_14,
 HAL_ADC_SAMP_NUMBER_15,
 HAL_ADC_SAMP_NUMBER_MAX = HAL_ADC_SAMP_NUMBER_15 }
 HAL_ADC_SAMP_NUMBER_12, HAL_ADC_SAMP_NUMBER_13, HAL_ADC_SAMP_NUMBER_14,
 HAL_ADC_SAMP_NUMBER_15,
 HAL_ADC_SAMP_NUMBER_MAX = HAL_ADC_SAMP_NUMBER_15 }
- enum HAL_ADC_TRIGGER_x { HAL_ADC_TRIGGER_MCPWM, HAL_ADC_TRIGGER_TIMER, HAL_ADC_TRIGGER_SOFT }
- enum HAL_ADC_REFERENCE_x { HAL_ADC_REFERENCE_INTERNAL, HAL_ADC_REFERENCE_EXTERNAL }

函数

- void HAL_ADC_Reset (HAL_ADC_x)
 初始化指定的ADC模块
- void HAL_ADC_Enable (HAL_ADC_x)
 启用指定的ADC模块
- void HAL_ADC_Disable (HAL_ADC_x)
 禁用指定的ADC模块
- int16_t HAL_ADC_GetSamplingValues (HAL_ADC_x, HAL_ADC_SAMP_SEQUENCE_x)
 获取指定序列号的采样值
- void HAL_ADC_SetSamplingChannel (HAL_ADC_x, HAL_ADC_SAMP_SEQUENCE_x, HAL_ADC_PORT_x)
 设置指定序列号的采样通道

- void `HAL_ADC_SetSamplingQuantity` (`HAL_ADC_x`, `HAL_ADC_SAMP_NUMBER_x`, `HAL_ADC_SAMP_NUMBER_x`)
设置采样数量
- void `HAL_ADC_SetTriggerSignal` (`HAL_ADC_x`, `HAL_ADC_TRIGGER_x`)
设置触发信号
- void `HAL_ADC_EndSampling` (`HAL_ADC_x`)
结束采样
- void `HAL_ADC_SetReference` (`HAL_ADC_x`, `HAL_ADC_REFERENCE_x`)
设置ADC基准源
- float `HAL_ADC_GetRange` (`HAL_ADC_x`)
获取adc量程,使用芯片供电作为adc基准时固定返回5.0
- float `HAL_ADC_GetValuesUnit` (`HAL_ADC_x`)
获取采样值单位
- void `HAL_ADC_SoftTrigger` (`HAL_ADC_x`)
软件触发ADC开始采样
- int16_t `HAL_ADC_ManualSampling` (`HAL_ADC_x`, `HAL_ADC_PORT_x`)
手动采样
- void `HAL_ADC_SetGpioIn` (`HAL_ADC_x`, `HAL_GPIO_Px`)
设置指定的GPIO为ADC输入
- void `HAL_ADC_SetGpioTriggerOut` (`HAL_ADC_x`, `HAL_GPIO_Px`)
设置指定的GPIO为ADC触发信号输出

4.9.1 枚举类型说明

4.9.1.1 HAL_ADC_PORT_x

enum `HAL_ADC_PORT_x`

主频

枚举值

<code>HAL_ADC_PORT_CH0</code>	ADC采样端口0
<code>HAL_ADC_PORT_CH1</code>	ADC采样端口1
<code>HAL_ADC_PORT_CH2</code>	ADC采样端口2
<code>HAL_ADC_PORT_CH3</code>	ADC采样端口3
<code>HAL_ADC_PORT_CH4</code>	ADC采样端口4
<code>HAL_ADC_PORT_CH5</code>	ADC采样端口5
<code>HAL_ADC_PORT_CH6</code>	ADC采样端口6
<code>HAL_ADC_PORT_CH7</code>	ADC采样端口7
<code>HAL_ADC_PORT_CH8</code>	ADC采样端口8
<code>HAL_ADC_PORT_CH9</code>	ADC采样端口9
<code>HAL_ADC_PORT_CH10</code>	ADC采样端口10
<code>HAL_ADC_PORT_CH11</code>	ADC采样端口11
<code>HAL_ADC_PORT_CH12</code>	ADC采样端口12
<code>HAL_ADC_PORT_CH13</code>	ADC采样端口13
<code>HAL_ADC_PORT_CH14</code>	ADC采样端口14
<code>HAL_ADC_PORT_CH15</code>	ADC采样端口15
<code>HAL_ADC_PORT_OPA0</code>	ADC采样端口运放0

枚举值

HAL_ADC_PORT_OPA1	ADC采样端口运放1
HAL_ADC_PORT_OPA2	ADC采样端口运放2
HAL_ADC_PORT_OPA3	ADC采样端口运放3
HAL_ADC_PORT_DAC0	仅ADC0 ADC采样DAC0输出
HAL_ADC_PORT_DAC1	仅ADC0 ADC采样DAC1输出
HAL_ADC_PORT_TMP0	仅ADC1 ADC采样端口温度传感器
HAL_ADC_PORT_VDD	仅ADC1 ADC采样供电
HAL_ADC_PORT_REF	ADC采样芯片内部基准电压 不支持 这里保留接口定义
HAL_ADC_PORT_AGND	ADC采样内部地 不支持 这里保留接口定义
HAL_ADC_PORT_OPA4	ADC采样端口运放4 不支持 这里保留接口定义
HAL_ADC_PORT_OPA5	ADC采样端口运放5 不支持 这里保留接口定义
HAL_ADC_PORT_CH16	ADC采样端口16 不支持 这里保留接口定义
HAL_ADC_PORT_CH17	ADC采样端口17 不支持 这里保留接口定义
HAL_ADC_PORT_CH18	ADC采样端口18 不支持 这里保留接口定义
HAL_ADC_PORT_CH19	ADC采样端口19 不支持 这里保留接口定义
HAL_ADC_PORT_LDO12	ADC采样芯片LDO12(1.2V)内核供电 不支持 这里保留接口定义

在文件 [hal_lksmcu_adc.h](#) 第 6 行定义.

4.9.1.2 HAL_ADC_REFERENCE_x

enum [HAL_ADC_REFERENCE_x](#)

枚举值

HAL_ADC_REFERENCE_INTERNAL	使用内部基准源作为adc基准
HAL_ADC_REFERENCE_EXTERNAL	使用芯片供电作为adc基准

在文件 [hal_lksmcu_adc.h](#) 第 139 行定义.

4.9.1.3 HAL_ADC_SAMP_NUMBER_x

enum [HAL_ADC_SAMP_NUMBER_x](#)

枚举值

HAL_ADC_SAMP_NUMBER_0	ADC采样0次
HAL_ADC_SAMP_NUMBER_1	ADC采样1次
HAL_ADC_SAMP_NUMBER_2	ADC采样2次
HAL_ADC_SAMP_NUMBER_3	ADC采样3次
HAL_ADC_SAMP_NUMBER_4	ADC采样4次
HAL_ADC_SAMP_NUMBER_5	ADC采样5次
HAL_ADC_SAMP_NUMBER_6	ADC采样6次

枚举值

HAL_ADC_SAMP_NUMBER_7	ADC采样7次
HAL_ADC_SAMP_NUMBER_8	ADC采样8次
HAL_ADC_SAMP_NUMBER_9	ADC采样9次
HAL_ADC_SAMP_NUMBER_10	ADC采样10次
HAL_ADC_SAMP_NUMBER_11	ADC采样11次
HAL_ADC_SAMP_NUMBER_12	ADC采样12次
HAL_ADC_SAMP_NUMBER_13	ADC采样13次
HAL_ADC_SAMP_NUMBER_14	ADC采样14次
HAL_ADC_SAMP_NUMBER_15	ADC采样15次
HAL_ADC_SAMP_NUMBER_MAX	ADC采样15次

在文件 [hal_lksmcu_adc.h](#) 第 113 行定义.

4.9.1.4 HAL_ADC_SAMP_SEQUENCE_x

enum [HAL_ADC_SAMP_SEQUENCE_x](#)

ADC采样序列

枚举值

HAL_ADC_SAMP_SEQUENCE_0	ADC第0次采样
HAL_ADC_SAMP_SEQUENCE_1	ADC第1次采样
HAL_ADC_SAMP_SEQUENCE_2	ADC第2次采样
HAL_ADC_SAMP_SEQUENCE_3	ADC第3次采样
HAL_ADC_SAMP_SEQUENCE_4	ADC第4次采样
HAL_ADC_SAMP_SEQUENCE_5	ADC第5次采样
HAL_ADC_SAMP_SEQUENCE_6	ADC第6次采样
HAL_ADC_SAMP_SEQUENCE_7	ADC第7次采样
HAL_ADC_SAMP_SEQUENCE_8	ADC第8次采样
HAL_ADC_SAMP_SEQUENCE_9	ADC第9次采样
HAL_ADC_SAMP_SEQUENCE_10	ADC第10次采样
HAL_ADC_SAMP_SEQUENCE_11	ADC第11次采样
HAL_ADC_SAMP_SEQUENCE_12	ADC第12次采样
HAL_ADC_SAMP_SEQUENCE_13	ADC第13次采样
HAL_ADC_SAMP_SEQUENCE_14	ADC第14次采样
HAL_ADC_SAMP_SEQUENCE_15	ADC第15次采样
HAL_ADC_SAMP_SEQUENCE_16	ADC第16次采样
HAL_ADC_SAMP_SEQUENCE_17	ADC第17次采样
HAL_ADC_SAMP_SEQUENCE_18	ADC第18次采样
HAL_ADC_SAMP_SEQUENCE_19	ADC第19次采样

在文件 [hal_lksmcu_adc.h](#) 第 83 行定义.

4.9.1.5 HAL_ADC_TRIGGER_x

enum HAL_ADC_TRIGGER_x

枚举值

HAL_ADC_TRIGGER_MCPWM	使用MCPWM触发ADC采样
HAL_ADC_TRIGGER_TIMER	使用TIMER触发ADC采样
HAL_ADC_TRIGGER_SOFT	使用软件触发ADC采样

在文件 [hal_lksmcu_adc.h](#) 第 133 行定义.

4.9.1.6 HAL_ADC_x

enum HAL_ADC_x

枚举值

HAL_↔ ADC_0	ADC模块0
HAL_↔ ADC_1	ADC模块1
HAL_↔ ADC_2	ADC模块2
HAL_↔ ADC_3	ADC模块3

在文件 [hal_lksmcu_adc.h](#) 第 106 行定义.

4.9.2 函数说明

4.9.2.1 HAL_ADC_Disable()

```
void HAL_ADC_Disable (
    HAL_ADC_x adc_x )
```

禁用指定的ADC模块

参数

adc_↔ _x	要禁用的ADC模块编号
-------------	-------------

在文件 [hal_lks32mc07x_adc.c](#) 第 114 行定义.

4.9.2.2 HAL_ADC_Enable()

```
void HAL_ADC_Enable (
    HAL_ADC_x adc_x )
```

启用指定的ADC模块

参数

adc↔ _x	要启用的ADC模块编号
------------	-------------

在文件 [hal_lks32mc07x_adc.c](#) 第 87 行定义.

4.9.2.3 HAL_ADC_EndSampling()

```
void HAL_ADC_EndSampling (
    HAL_ADC_x adc_x )
```

结束采样

参数

adc↔ _x	ADC模块编号
------------	---------

在文件 [hal_lks32mc07x_adc.c](#) 第 267 行定义.

这是这个函数的调用关系图:



4.9.2.4 HAL_ADC.GetRange()

```
float HAL_ADC.GetRange (
    HAL_ADC_x adc_x )
```

获取adc量程,使用芯片供电作为adc基准时固定返回5.0

参数

adc↔ _x	ADC模块编号
------------	---------

返回

ADC量程

在文件 [hal_lks32mc07x_adc.c](#) 第 347 行定义.

这是这个函数的调用关系图:



4.9.2.5 HAL_ADC_GetSamplingValues()

```
int16_t HAL_ADC_GetSamplingValues (
    HAL_ADC_x adc_x,
    HAL_ADC_SAMP_SEQUENCE_x sequence_x )
```

获取指定序列号的采样值

参数

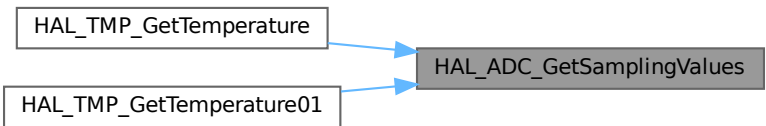
<i>adc_x</i>	ADC模块编号
<i>sequence_x</i>	采样序列号

返回

返回对应序列号的采样值

在文件 [hal_lks32mc07x_adc.c](#) 第 147 行定义.

这是这个函数的调用关系图:



4.9.2.6 HAL_ADC_GetValuesUnit()

```
float HAL_ADC_GetValuesUnit (
    HAL_ADC_x adc_x )
```

获取采样值单位

参数

adc_x	ADC模块编号
----------	---------

返回

采样值单位

在文件 [hal_lks32mc07x_adc.c](#) 第 373 行定义.

函数调用图:



4.9.2.7 HAL_ADC_ManualSampling()

```
int16_t HAL_ADC_ManualSampling (
    HAL_ADC_x adc_x,
    HAL_ADC_PORT_x port_x )
```

手动采样

参数

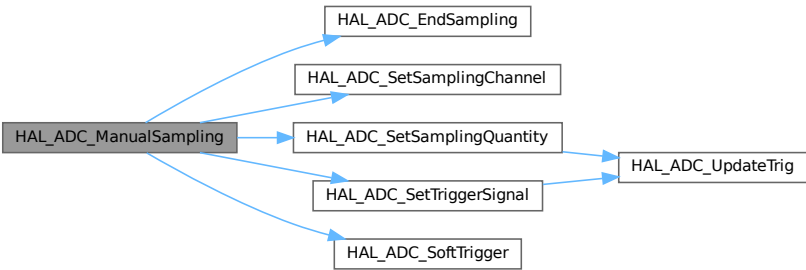
n	采样通道编号
-----	--------

返回

示例返回值,实际应返回采样值

在文件 [hal_lks32mc07x_adc.c](#) 第 285 行定义.

函数调用图:



4.9.2.8 HAL_ADC_Reset()

```
void HAL_ADC_Reset (
    HAL_ADC_x adc_x )
```

初始化指定的ADC模块

参数

<i>adc_x</i>	要初始化的ADC模块编号
--------------	--------------

在文件 [hal_lks32mc07x_adc.c](#) 第 62 行定义.

4.9.2.9 HAL_ADC_SetGpioIn()

```
void HAL_ADC_SetGpioIn (
    HAL_ADC_x adc_x,
    HAL_GPIO_Px pin_x )
```

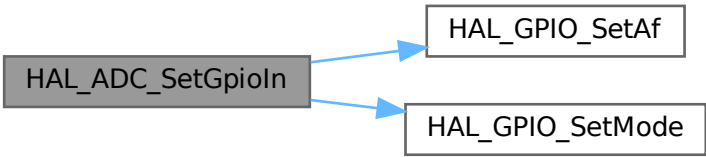
设置指定的GPIO为ADC输入

参数

<i>adc_x</i>	ADC模块编号
<i>pin_x</i>	GPIO编号

在文件 [hal_lks32mc07x_adc.c](#) 第 390 行定义.

函数调用图:



4.9.2.10 HAL_ADC.SetGpioTriggerOut()

```
void HAL_ADC_SetGpioTriggerOut (
    HAL_ADC_x adc_x,
    HAL_GPIO_Px pin_x )
```

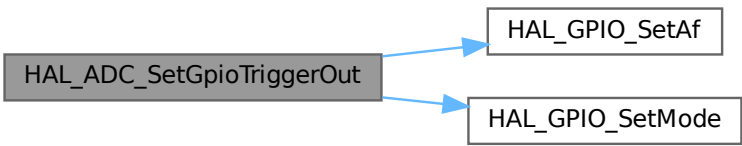
设置指定的GPIO为ADC触发信号输出

参数

<i>adc_x</i>	ADC模块编号
<i>pin_x</i>	GPIO编号

在文件 [hal_lks32mc07x_adc.c](#) 第 401 行定义.

函数调用图:



4.9.2.11 HAL_ADC.SetReference()

```
void HAL_ADC_SetReference (
    HAL_ADC_x adc_x,
    HAL_ADC_REFERENCE_x reference_x )
```

设置ADC基准源

参数

<i>adc_x</i>	ADC模块编号
<i>reference↔_x</i>	基准源

在文件 [hal_lks32mc07x_adc.c](#) 第 315 行定义.

4.9.2.12 HAL_ADC_SetSamplingChannel()

```
void HAL_ADC_SetSamplingChannel (
    HAL_ADC_x adc_x,
    HAL_ADC_SAMP_SEQUENCE_x sequence_x,
    HAL_ADC_PORT_x port_x )
```

设置指定序列号的采样通道

参数

<i>adc_x</i>	ADC模块编号
<i>sequence↔_x</i>	采样序列号
<i>port_x</i>	采样通道端口

- < ADC第0次采样
- < ADC第1次采样
- < ADC第2次采样
- < ADC第3次采样
- < ADC第4次采样
- < ADC第5次采样
- < ADC第6次采样
- < ADC第7次采样
- < ADC第8次采样
- < ADC第9次采样
- < ADC第10次采样
- < ADC第11次采样
- < ADC第12次采样
- < ADC第13次采样

在文件 [hal_lks32mc07x_adc.c](#) 第 158 行定义.

这是这个函数的调用关系图:



4.9.2.13 HAL_ADC_SetSamplingQuantity()

```
void HAL_ADC_SetSamplingQuantity (
    HAL_ADC_x adc_x,
    HAL_ADC_SAMP_NUMBER_x sample1_number,
    HAL_ADC_SAMP_NUMBER_x sample2_number )
```

设置采样数量

参数

<i>adc_x</i>	ADC模块编号
<i>sample_number</i>	采样数量

在文件 [hal_lks32mc07x_adc.c](#) 第 214 行定义.

函数调用图:



这是这个函数的调用关系图:



4.9.2.14 HAL_ADC.SetTriggerSignal()

```
void HAL_ADC_SetTriggerSignal (
    HAL_ADC_x adc_x,
    HAL_ADC_TRIGGER_x trigger )
```

设置触发信号

参数

<i>adc_x</i>	ADC模块编号
<i>trigger</i>	触发信号类型

- < 使用MCPWM触发ADC采样
- < 使用TIMER触发ADC采样
- < 使用软件触发ADC采样

在文件 `hal_lks32mc07x_adc.c` 第 244 行定义.

函数调用图:



这是这个函数的调用关系图:



4.9.2.15 HAL_ADC.SoftTrigger()

```
void HAL_ADC_SoftTrigger (
    HAL_ADC_x adc_x )
```

软件触发ADC开始采样

参数

<code>adc</code> <code>_x</code>	ADC模块编号
-------------------------------------	---------

在文件 [hal_lks32mc07x_adc.c](#) 第 276 行定义.

这是这个函数的调用关系图:



4.10 hal_lksmcu_adc.h

[浏览该文件的文档.](#)

```
00001 #ifndef __HAL_LKSMCU_ADC_H__
00002 #define __HAL_LKSMCU_ADC_H__
00003 #include "stdint.h"
00004 #include "hal_lksmcu_gpio.h"
00006 typedef enum
00007 {
00008     // ADC0采样通道
00009     // 0x0: OPA0输出
00010     // 0x1: OPA1输出
00011     // 0x2: OPA2输出
00012     // 0x3: OPA3输出
00013     // 0x4: ADC01.CH4
00014     // 0x5: ADC01.CH5
00015     // 0x6: ADC01.CH6
00016     // 0x7: ADC01.CH7
00017     // 0x8: ADC01.CH8
00018     // 0x9: ADC01.CH9
00019     // 0xA: ADC0.CH10
00020     // 0xB: ADC0.CH11
00021     // 0xC: ADC0.CH12
00022     // 0xD: ADC0.CH13
00023     // 0xE: DAC0.OUT
00024     // 0xF: DAC1.OUT
00025
00026     // ADC1采样通道
00027     // 0x0: OPA0输出
00028     // 0x1: OPA1输出
00029     // 0x2: OPA2输出
00030     // 0x3: OPA3输出
00031     // 0x4: ADC01.CH4
00032     // 0x5: ADC01.CH5
00033     // 0x6: ADC01.CH6
00034     // 0x7: ADC01.CH7
00035     // 0x8: ADC01.CH8
00036     // 0x9: ADC01.CH9
00037     // 0xA: ADC1.CH10
00038     // 0xB: ADC1.CH11
00039     // 0xC: ADC1.CH12
00040     // 0xD: ADC1.CH13
00041     // 0xE: 温度传感器
00042     // 0xF: 内部电源AVDD
00043
00044     HAL_ADC_PORT.CH0,
00045     HAL_ADC_PORT.CH1,
00046     HAL_ADC_PORT.CH2,
00047     HAL_ADC_PORT.CH3,
00048     HAL_ADC_PORT.CH4,
00049     HAL_ADC_PORT.CH5,
00050     HAL_ADC_PORT.CH6,
```

```

00051     HAL_ADC_PORT.CH7,
00052     HAL_ADC_PORT.CH8,
00053     HAL_ADC_PORT.CH9,
00054     HAL_ADC_PORT.CH10,
00055     HAL_ADC_PORT.CH11,
00056     HAL_ADC_PORT.CH12,
00057     HAL_ADC_PORT.CH13,
00058     HAL_ADC_PORT.CH14,
00059     HAL_ADC_PORT.CH15,
00060
00061     HAL_ADC_PORT.OPA0 = HAL_ADC_PORT.CH0,
00062     HAL_ADC_PORT.OPA1 = HAL_ADC_PORT.CH1,
00063     HAL_ADC_PORT.OPA2 = HAL_ADC_PORT.CH2,
00064     HAL_ADC_PORT.OPA3 = HAL_ADC_PORT.CH3,
00065
00066     HAL_ADC_PORT.DAC0 = HAL_ADC_PORT.CH14,
00067     HAL_ADC_PORT.DAC1 = HAL_ADC_PORT.CH15,
00068
00069     HAL_ADC_PORT.TMP0 = HAL_ADC_PORT.CH14,
00070     HAL_ADC_PORT.VDD = HAL_ADC_PORT.CH15,
00071
00072     HAL_ADC_PORT.REF,
00073     HAL_ADC_PORT.AGND,
00074     HAL_ADC_PORT.OPA4,
00075     HAL_ADC_PORT.OPA5,
00076     HAL_ADC_PORT.CH16,
00077     HAL_ADC_PORT.CH17,
00078     HAL_ADC_PORT.CH18,
00079     HAL_ADC_PORT.CH19,
00080     HAL_ADC_PORT.LD012,
00081 } HAL_ADC_PORT_x;
00083 typedef enum
00084 {
00085     HAL_ADC_SAMP_SEQUENCE_0,
00086     HAL_ADC_SAMP_SEQUENCE_1,
00087     HAL_ADC_SAMP_SEQUENCE_2,
00088     HAL_ADC_SAMP_SEQUENCE_3,
00089     HAL_ADC_SAMP_SEQUENCE_4,
00090     HAL_ADC_SAMP_SEQUENCE_5,
00091     HAL_ADC_SAMP_SEQUENCE_6,
00092     HAL_ADC_SAMP_SEQUENCE_7,
00093     HAL_ADC_SAMP_SEQUENCE_8,
00094     HAL_ADC_SAMP_SEQUENCE_9,
00095     HAL_ADC_SAMP_SEQUENCE_10,
00096     HAL_ADC_SAMP_SEQUENCE_11,
00097     HAL_ADC_SAMP_SEQUENCE_12,
00098     HAL_ADC_SAMP_SEQUENCE_13,
00099     HAL_ADC_SAMP_SEQUENCE_14,
00100     HAL_ADC_SAMP_SEQUENCE_15,
00101     HAL_ADC_SAMP_SEQUENCE_16,
00102     HAL_ADC_SAMP_SEQUENCE_17,
00103     HAL_ADC_SAMP_SEQUENCE_18,
00104     HAL_ADC_SAMP_SEQUENCE_19,
00105 } HAL_ADC_SAMP_SEQUENCE_x;
00106 typedef enum
00107 {
00108     HAL_ADC_0,
00109     HAL_ADC_1,
00110     HAL_ADC_2,
00111     HAL_ADC_3,
00112 } HAL_ADC_x;
00113 typedef enum
00114 {
00115     HAL_ADC_SAMP_NUMBER_0,
00116     HAL_ADC_SAMP_NUMBER_1,
00117     HAL_ADC_SAMP_NUMBER_2,
00118     HAL_ADC_SAMP_NUMBER_3,
00119     HAL_ADC_SAMP_NUMBER_4,
00120     HAL_ADC_SAMP_NUMBER_5,
00121     HAL_ADC_SAMP_NUMBER_6,
00122     HAL_ADC_SAMP_NUMBER_7,
00123     HAL_ADC_SAMP_NUMBER_8,
00124     HAL_ADC_SAMP_NUMBER_9,
00125     HAL_ADC_SAMP_NUMBER_10,
00126     HAL_ADC_SAMP_NUMBER_11,
00127     HAL_ADC_SAMP_NUMBER_12,
00128     HAL_ADC_SAMP_NUMBER_13,
00129     HAL_ADC_SAMP_NUMBER_14,
00130     HAL_ADC_SAMP_NUMBER_15,
00131     HAL_ADC_SAMP_NUMBER_MAX = HAL_ADC_SAMP_NUMBER_15,
00132 } HAL_ADC_SAMP_NUMBER_x;
00133 typedef enum
00134 {
00135     HAL_ADC_TRIGGER_MCPWM,
00136     HAL_ADC_TRIGGER_TIMER,
00137     HAL_ADC_TRIGGER_SOFT,
00138 } HAL_ADC_TRIGGER_x;

```

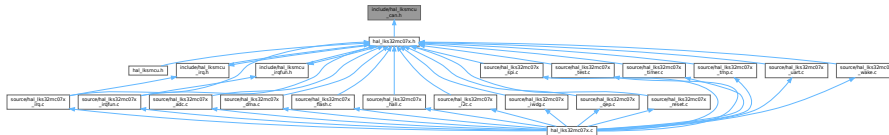
```

00139 typedef enum
00140 {
00141     HAL_ADC_REFERENCE_INTERNAL,
00142     HAL_ADC_REFERENCE_EXTERNAL,
00143 } HAL_ADC_REFERENCE_x;
00144
00145 // 初始化
00146 void HAL_ADC_Reset(HAL_ADC_x); // 模块复位
00147 void HAL_ADC_Enable(HAL_ADC_x); // 模块打开
00148 void HAL_ADC_Disable(HAL_ADC_x); // 模块关闭
00149
00150 // 自动采样配置
00151 int16_t HAL_ADC_GetSamplingValues(HAL_ADC_x, HAL_ADC_SAMP_SEQUENCE_x); // 获取采样值 返回第n次采样结果
00152 void HAL_ADC_SetSamplingChannel(HAL_ADC_x, HAL_ADC_SAMP_SEQUENCE_x, HAL_ADC_PORT_x); // 设置采样通道, 输入参数 第n次采样channel通道
00153 void HAL_ADC_SetSamplingQuantity(HAL_ADC_x, HAL_ADC_SAMP_NUMBER_x, HAL_ADC_SAMP_NUMBER_x); // 设置采样数量
00154 void HAL_ADC_SetTriggerSignal(HAL_ADC_x, HAL_ADC_TRIGGER_x); // 设置触发信号
00155 void HAL_ADC_EndSampling(HAL_ADC_x); // 中止当前采样
00156 void HAL_ADC_SetReference(HAL_ADC_x, HAL_ADC_REFERENCE_x); // 设置ADC基准源
00157 float HAL_ADC_GetRange(HAL_ADC_x); // 获取adc量程, 使用芯片供电作为adc基准时固定返回5.0
00158 float HAL_ADC_GetValuesUnit(HAL_ADC_x); // 获取采样值单位
00159 // 手动采样配置
00160 void HAL_ADC_SoftTrigger(HAL_ADC_x); // 软件触发ADC开始采样
00161 int16_t HAL_ADC_ManualSampling(HAL_ADC_x, HAL_ADC_PORT_x); // 手动采样 需要先设置采样通道为软件触发
00162
00163 // IO配置
00164 void HAL_ADC_SetGpioIn(HAL_ADC_x, HAL_GPIO_Px); // 设置指定的GPIO为ADC输入
00165 void HAL_ADC_SetGpioTriggerOut(HAL_ADC_x, HAL_GPIO_Px); // 设置指定的GPIO为ADC触发信号输出
00166
00167 #endif

```

4.11 include/hal_lksmcu_can.h 文件参考

此图展示该文件直接或间接的被哪些文件引用了:



4.12 hal_lksmcu_can.h

浏览该文件的文档.

```

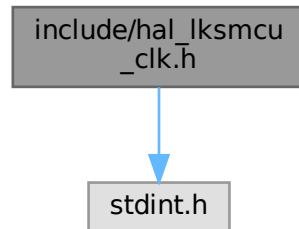
00001 #ifndef __HAL_LKSMCU_CAN_H__
00002 #define __HAL_LKSMCU_CAN_H__
00003 #endif

```

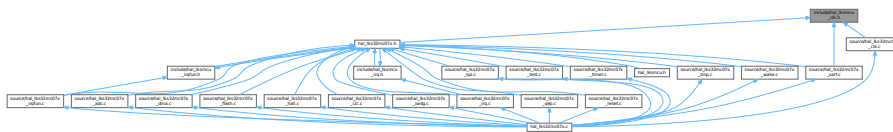
4.13 include/hal_lksmcu_clk.h 文件参考

```
#include "stdint.h"
```

hal_lksmcu_clk.h 的引用(Include)关系图:



此图展示该文件直接或间接的被哪些文件引用了:



枚举

```
enum HAL_CLK_FREQ_x {
    HAL_CLK_FREQ_RC_8MHZ, HAL_CLK_FREQ_RC_9MHZ, HAL_CLK_FREQ_RC_12MHZ, HAL_CLK_FREQ_RC_16MHZ,
    HAL_CLK_FREQ_RC_18MHZ, HAL_CLK_FREQ_RC_24MHZ, HAL_CLK_FREQ_RC_32MHZ, HAL_CLK_FREQ_RC_36MHZ,
    HAL_CLK_FREQ_RC_48MHZ, HAL_CLK_FREQ_RC_64MHZ, HAL_CLK_FREQ_RC_72MHZ, HAL_CLK_FREQ_RC_96MHZ,
    HAL_CLK_FREQ_XTAL_6MHZ, HAL_CLK_FREQ_XTAL_12MHZ, HAL_CLK_FREQ_XTAL_24MHZ,
    HAL_CLK_FREQ_XTAL_48MHZ,
    HAL_CLK_FREQ_XTAL_96MHZ, HAL_CLK_FREQ_RC_4MHZ, HAL_CLK_FREQ_RC_6MHZ, HAL_CLK_FREQ_RC_192MHZ,
    HAL_CLK_FREQ_XTAL_192MHZ }

```

主频

函数

- void HAL_CLK_SetMcuFreq (HAL_CLK_FREQ_x)
设置主频
- uint32_t HAL_CLK_GetMcuFreq (void)
获取主频

4.13.1 枚举类型说明

4.13.1.1 HAL_CLK_FREQ_x

enum [HAL_CLK_FREQ_x](#)

主频

枚举值

HAL_CLK_FREQ_RC_8MHZ	主频8Mhz
HAL_CLK_FREQ_RC_9MHZ	主频9Mhz
HAL_CLK_FREQ_RC_12MHZ	主频12Mhz
HAL_CLK_FREQ_RC_16MHZ	主频16Mhz
HAL_CLK_FREQ_RC_18MHZ	主频18Mhz
HAL_CLK_FREQ_RC_24MHZ	主频24Mhz
HAL_CLK_FREQ_RC_32MHZ	主频32Mhz
HAL_CLK_FREQ_RC_36MHZ	主频36Mhz
HAL_CLK_FREQ_RC_48MHZ	主频48Mhz
HAL_CLK_FREQ_RC_64MHZ	主频64Mhz
HAL_CLK_FREQ_RC_72MHZ	主频72Mhz
HAL_CLK_FREQ_RC_96MHZ	主频96Mhz
HAL_CLK_FREQ_XTAL_6MHZ	主频6Mhz 不支持 这里保留接口定义
HAL_CLK_FREQ_XTAL_12MHZ	主频12Mhz 不支持 这里保留接口定义
HAL_CLK_FREQ_XTAL_24MHZ	主频24Mhz 不支持 这里保留接口定义
HAL_CLK_FREQ_XTAL_48MHZ	主频48Mhz 不支持 这里保留接口定义
HAL_CLK_FREQ_XTAL_96MHZ	主频96Mhz 不支持 这里保留接口定义
HAL_CLK_FREQ_RC_4MHZ	主频4Mhz 不支持 这里保留接口定义
HAL_CLK_FREQ_RC_6MHZ	主频6Mhz 不支持 这里保留接口定义
HAL_CLK_FREQ_RC_192MHZ	主频192Mhz 不支持 这里保留接口定义
HAL_CLK_FREQ_XTAL_192MHZ	主频192Mhz 不支持 这里保留接口定义

在文件 [hal_lksmcu_clk.h](#) 第 6 行定义.

4.13.2 函数说明

4.13.2.1 HAL_CLK_GetMcuFreq()

```
uint32_t HAL_CLK_GetMcuFreq (
    void )
```

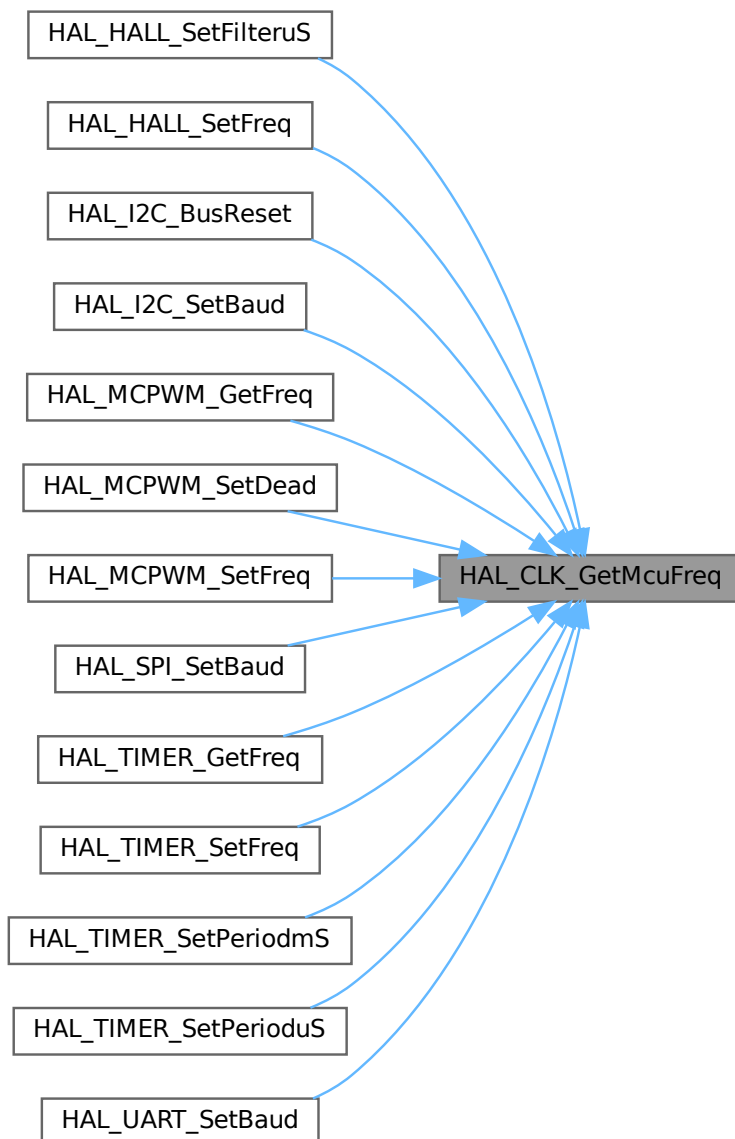
获取主频

返回

主频

在文件 [hal_lks32mc07x.clk.c](#) 第 45 行定义.

这是这个函数的调用关系图:



4.13.2.2 HAL_CLK_SetMcuFreq()

```
void HAL_CLK_SetMcuFreq (  
    HAL\_CLK\_FREQ\_x FREQ_x )
```

设置主频

参数

<i>FREQ</i> _x	主频
--------------------------	----

< 主频4Mhz

< 主频12Mhz

< 主频24Mhz

< 主频48Mhz

< 主频96Mhz

在文件 [hal_lks32mc07x.clk.c](#) 第 11 行定义.

4.14 hal_lksmcu_clk.h

浏览该文件的文档.

```

00001 #ifndef __HAL_LKSMCU_CLK_H__
00002 #define __HAL_LKSMCU_CLK_H__
00003 #include "stdint.h"
00004
00006 typedef enum
00007 {
00008     HAL_CLK_FREQ_RC_8MHZ,
00009     HAL_CLK_FREQ_RC_9MHZ,
00010     HAL_CLK_FREQ_RC_12MHZ,
00011     HAL_CLK_FREQ_RC_16MHZ,
00012     HAL_CLK_FREQ_RC_18MHZ,
00013     HAL_CLK_FREQ_RC_24MHZ,
00014     HAL_CLK_FREQ_RC_32MHZ,
00015     HAL_CLK_FREQ_RC_36MHZ,
00016     HAL_CLK_FREQ_RC_48MHZ,
00017     HAL_CLK_FREQ_RC_64MHZ,
00018     HAL_CLK_FREQ_RC_72MHZ,
00019     HAL_CLK_FREQ_RC_96MHZ,
00020
00021     HAL_CLK_FREQ_XTAL_6MHZ,
00022     HAL_CLK_FREQ_XTAL_12MHZ,
00023     HAL_CLK_FREQ_XTAL_24MHZ,
00024     HAL_CLK_FREQ_XTAL_48MHZ,
00025     HAL_CLK_FREQ_XTAL_96MHZ,
00026
00027     HAL_CLK_FREQ_RC_4MHZ,
00028     HAL_CLK_FREQ_RC_6MHZ,
00029     HAL_CLK_FREQ_RC_192MHZ,
00030     HAL_CLK_FREQ_XTAL_192MHZ,
00031 } HAL_CLK_FREQ_x;
00032 void HAL_CLK_SetMcuFreq(HAL_CLK_FREQ_x);
00033 uint32_t HAL_CLK_GetMcuFreq(void);
00034 #endif

```

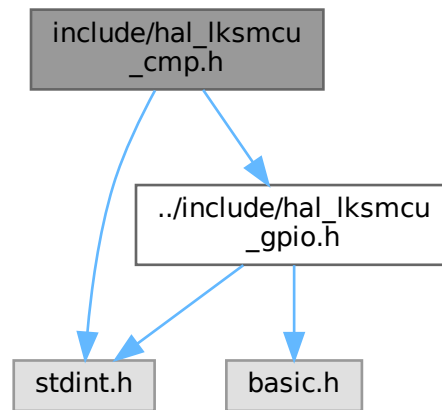
4.15 include/hal_lksmcu_cmp.h 文件参考

```

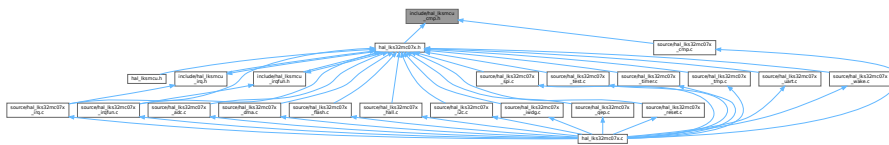
#include "stdint.h"
#include "../include/hal_lksmcu_gpio.h"

```


hal_lksmcu_cmp.h 的引用(Include)关系图:



此图展示该文件直接或间接的被哪些文件引用了:



枚举

- enum HAL_CMP_x {
HAL_CMP_0 , HAL_CMP_1 , HAL_CMP_2 , HAL_CMP_3 ,
HAL_CMP_4 , HAL_CMP_5 }
- enum HAL_CMP_CHANNEL_IPx {
HAL_CMP_CHANNEL_IP0 = 0 , HAL_CMP_CHANNEL_IP1 = 4 , HAL_CMP_CHANNEL_IP2 = 5 ,
HAL_CMP_CHANNEL_IP3 = 6 ,
HAL_CMP_CHANNEL_IP4 = 7 , HAL_CMP_CHANNEL_IP_DAC0 = 7 , HAL_CMP_CHANNEL_IP_OPA0_IP = 1
, HAL_CMP_CHANNEL_IP_OPA3_IP = 1 ,
HAL_CMP_CHANNEL_IP_OPA0_OUTP = 2 , HAL_CMP_CHANNEL_IP_OPA1_OUTP = 3 , HAL_CMP_CHANNEL_IP_OPA2_OUTP
= 2 , HAL_CMP_CHANNEL_IP_OPA3_OUTP = 3 ,
HAL_CMP_CHANNEL_IP5 , HAL_CMP_CHANNEL_IP6 , HAL_CMP_CHANNEL_IP7 , HAL_CMP_CHANNEL_IP_AVSS
,
HAL_CMP_CHANNEL_IP_DAC1 , HAL_CMP_CHANNEL_IP_OPA_OUT_CMP0 , HAL_CMP_CHANNEL_IP_OPA0_IN
, HAL_CMP_CHANNEL_IP_OPA0_OUTN ,
HAL_CMP_CHANNEL_IP_OPA1_IP , HAL_CMP_CHANNEL_IP_OPA1_IN , HAL_CMP_CHANNEL_IP_OPA1_OUTN
, HAL_CMP_CHANNEL_IP_OPA2_IP ,
HAL_CMP_CHANNEL_IP_OPA2_IN , HAL_CMP_CHANNEL_IP_OPA2_OUTN , HAL_CMP_CHANNEL_IP_OPA3_IN
, HAL_CMP_CHANNEL_IP_OPA3_OUTN ,
HAL_CMP_CHANNEL_IP_OPA4_IP , HAL_CMP_CHANNEL_IP_OPA4_IN , HAL_CMP_CHANNEL_IP_OPA4_OUTP
, HAL_CMP_CHANNEL_IP_OPA4_OUTN ,
HAL_CMP_CHANNEL_IP_OPA5_IP , HAL_CMP_CHANNEL_IP_OPA5_IN , HAL_CMP_CHANNEL_IP_OPA5_OUTP
, HAL_CMP_CHANNEL_IP_OPA5_OUTN }
• enum HAL_CMP_CHANNEL_INx { HAL_CMP_CHANNEL_IN_0 , HAL_CMP_CHANNEL_IN_REF , HAL_CMP_CHANNEL_IN_DAC
, HAL_CMP_CHANNEL_IN_MID }

函数

- void [HAL_CMP_Reset](#) (HAL_CMP_x)
模块复位
- void [HAL_CMP_Enable](#) (HAL_CMP_x)
模块打开
- void [HAL_CMP_Disable](#) (HAL_CMP_x)
模块关闭
- void [HAL_CMP_SetChannel](#) (HAL_CMP_x, HAL_CMP_CHANNEL_IPx, HAL_CMP_CHANNEL_INx)
模块关闭
- uint32_t [HAL_CMP_SetDigitalFilter](#) (HAL_CMP_x, uint32_t filter)
获取CMP模块的数字滤波宽度
- void [HAL_CMP_SetGpioOut](#) (HAL_CMP_x cmpx, HAL_GPIO_Px cmpgpio)
设置CMP模块的GPIO
- void [HAL_CMP_SetGpioIn](#) (HAL_CMP_x cmpx, HAL_GPIO_Px cmpgpio)
设置CMP模块的GPIO
- uint8_t [HAL_CMP_GetData](#) (HAL_CMP_x)
获取CMP比较结果

4.15.1 枚举类型说明

4.15.1.1 HAL_CMP_CHANNEL_INx

enum [HAL_CMP_CHANNEL_INx](#)

枚举值

HAL_CMP_CHANNEL_IN_0	CMPN选择为外部负端输入
HAL_CMP_CHANNEL_IN_REF	CMPN选择为内部基准源
HAL_CMP_CHANNEL_IN_DAC	CMPN选择为DAC输出
HAL_CMP_CHANNEL_IN_MID	CMPN选择为反电动势虚拟中心点

在文件 [hal_lksmcu_cmp.h](#) 第 58 行定义。

4.15.1.2 HAL_CMP_CHANNEL_IPx

enum [HAL_CMP_CHANNEL_IPx](#)

枚举值

HAL_CMP_CHANNEL_IP0	CMP模块外部正端输入0
HAL_CMP_CHANNEL_IP1	CMP模块外部正端输入1
HAL_CMP_CHANNEL_IP2	CMP模块外部正端输入2
HAL_CMP_CHANNEL_IP3	CMP模块外部正端输入3
HAL_CMP_CHANNEL_IP4	仅CMP0 CMP模块外部正端输入4
HAL_CMP_CHANNEL_IP_DAC0	仅CMP1 DAC0输出
HAL_CMP_CHANNEL_IP_OPA0_IP	仅CMP0 运放0正端输入

枚举值

HAL_CMP_CHANNEL_IP_OPA3_IP	仅CMP1 运放3正端输入
HAL_CMP_CHANNEL_IP_OPA0_OUTP	仅CMP0 运放0正端输出
HAL_CMP_CHANNEL_IP_OPA1_OUTP	仅CMP0 运放1正端输出
HAL_CMP_CHANNEL_IP_OPA2_OUTP	仅CMP1 运放2正端输出
HAL_CMP_CHANNEL_IP_OPA3_OUTP	仅CMP1 运放3正端输出
HAL_CMP_CHANNEL_IP5	CMP模块外部正端输入5 不支持
HAL_CMP_CHANNEL_IP6	CMP模块外部正端输入6 不支持
HAL_CMP_CHANNEL_IP7	CMP模块外部正端输入7 不支持
HAL_CMP_CHANNEL_IP_AVSS	模拟地 不支持
HAL_CMP_CHANNEL_IP_DAC1	DAC1输出 不支持
HAL_CMP_CHANNEL_IP_OPA_OUT_CMP0	可以软件配置的运放输出0 不支持
HAL_CMP_CHANNEL_IP_OPA0_IN	运放0负端输入 不支持
HAL_CMP_CHANNEL_IP_OPA0_OUTN	运放0负端输出 不支持
HAL_CMP_CHANNEL_IP_OPA1_IP	运放1正端输入 不支持
HAL_CMP_CHANNEL_IP_OPA1_IN	运放1负端输入 不支持
HAL_CMP_CHANNEL_IP_OPA1_OUTN	运放1负端输出 不支持
HAL_CMP_CHANNEL_IP_OPA2_IP	运放2正端输入 不支持
HAL_CMP_CHANNEL_IP_OPA2_IN	运放2负端输入 不支持
HAL_CMP_CHANNEL_IP_OPA2_OUTN	运放2负端输出 不支持
HAL_CMP_CHANNEL_IP_OPA3_IN	运放3负端输入 不支持
HAL_CMP_CHANNEL_IP_OPA3_OUTN	运放3负端输出 不支持
HAL_CMP_CHANNEL_IP_OPA4_IP	运放4正端输入 不支持
HAL_CMP_CHANNEL_IP_OPA4_IN	运放4负端输入 不支持
HAL_CMP_CHANNEL_IP_OPA4_OUTP	运放4正端输出 不支持
HAL_CMP_CHANNEL_IP_OPA4_OUTN	运放4负端输出 不支持
HAL_CMP_CHANNEL_IP_OPA5_IP	运放4正端输入 不支持
HAL_CMP_CHANNEL_IP_OPA5_IN	运放5负端输入 不支持
HAL_CMP_CHANNEL_IP_OPA5_OUTP	运放5正端输出 不支持
HAL_CMP_CHANNEL_IP_OPA5_OUTN	运放5负端输出 不支持

在文件 [hal_lksmcu.cmp.h](#) 第 15 行定义。

4.15.1.3 HAL_CMP_x

enum [HAL_CMP_x](#)

枚举值

HAL_↔ CMP_0	CMP模块0
HAL_↔ CMP_1	CMP模块1
HAL_↔ CMP_2	CMP模块2

枚举值

HAL_↔ CMP_3	CMP模块3
HAL_↔ CMP_4	CMP模块4
HAL_↔ CMP_5	CMP模块5

在文件 [hal_lksmcu_cmp.h](#) 第 5 行定义.

4.15.2 函数说明

4.15.2.1 HAL_CMP_Disable()

```
void HAL_CMP_Disable (
    HAL_CMP_x cmp_x )
```

模块关闭

参数

cmp↔ _x	使用的CMP模块编号
------------	------------

在文件 [hal_lks32mc07x_cmp.c](#) 第 59 行定义.

这是这个函数的调用关系图:



4.15.2.2 HAL_CMP_Enable()

```
void HAL_CMP_Enable (
    HAL_CMP_x cmp_x )
```

模块打开

参数

<i>cmp</i> _↔ <i>_x</i>	使用的CMP模块编号
--------------------------------------	------------

在文件 [hal_lks32mc07x_cmp.c](#) 第 30 行定义.

4.15.2.3 HAL_CMP_GetData()

```
uint8_t HAL_CMP_GetData (
    HAL_CMP_x cmp_x )
```

获取CMP比较结果

参数

<i>cmp</i> _↔ <i>_x</i>	CMP模块编号
--------------------------------------	---------

返回
比较结果

在文件 [hal_lks32mc07x_cmp.c](#) 第 198 行定义.

4.15.2.4 HAL_CMP_Reset()

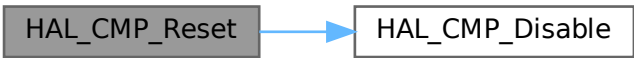
```
void HAL_CMP_Reset (
    HAL_CMP_x cmp_x )
```

模块复位

参数

<i>cmp</i> _↔ <i>_x</i>	使用的CMP模块编号
--------------------------------------	------------

在文件 [hal_lks32mc07x_cmp.c](#) 第 22 行定义.
函数调用图:



4.15.2.5 HAL_CMP_SetChannel()

```
void HAL_CMP_SetChannel (
    HAL_CMP_x cmp_x,
    HAL_CMP_CHANNEL_IPx channel_p_x,
    HAL_CMP_CHANNEL_INx channel_n_x )
```

模块关闭

参数

<i>cmp_x</i>	使用的CMP模块编号
<i>channel_↔p_x</i>	正端通道
<i>channel_↔n_x</i>	负端通道

在文件 [hal_lks32mc07x_cmp.c](#) 第 90 行定义.

4.15.2.6 HAL_CMP_SetDigitalFilter()

```
uint32_t HAL_CMP_SetDigitalFilter (
    HAL_CMP_x cmp_x,
    uint32_t filter )
```

获取CMP模块的数字滤波宽度

参数

<i>cmp_↔_x</i>	CMP模块编号
<i>filter</i>	滤波宽度

返回

实际滤波宽度

在文件 [hal_lks32mc07x_cmp.c](#) 第 136 行定义.

4.15.2.7 HAL_CMP_SetGpioIn()

```
void HAL_CMP_SetGpioIn (
    HAL_CMP_x cmp_x,
    HAL_GPIO_Px cmp_gpio )
```

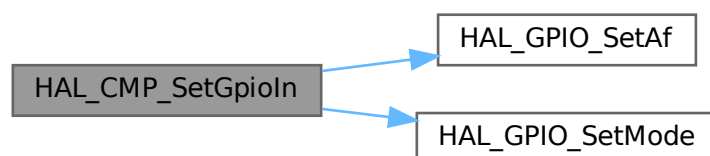
设置CMP模块的GPIO

参数

<i>cmp_x</i>	CMP模块编号
<i>cmpgpio</i>	GPIO编号

在文件 [hal_lks32mc07x.cmp.c](#) 第 186 行定义.

函数调用图:



4.15.2.8 HAL_CMP_SetGpioOut()

```
void HAL_CMP_SetGpioOut (
    HAL_CMP_x cmp_x,
    HAL_GPIO_Px cmpgpio )
```

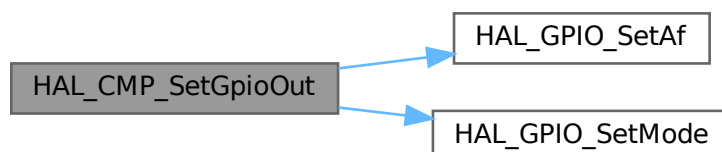
设置CMP模块的GPIO

参数

<i>cmp_x</i>	cmp模块编号
<i>cmpgpio</i>	GPIO编号

在文件 [hal_lks32mc07x.cmp.c](#) 第 174 行定义.

函数调用图:



4.16 hal_lksmcu_cmp.h

浏览该文件的文档.

```

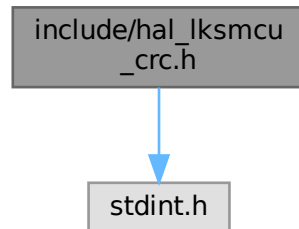
00001 #ifndef __HAL_LKSMCU_CMP_H__
00002 #define __HAL_LKSMCU_CMP_H__
00003 #include "stdint.h"
00004 #include "../include/hal_lksmcu_gpio.h"
00005 typedef enum
00006 {
00007     HAL_CMP_0,
00008     HAL_CMP_1,
00009     HAL_CMP_2,
00010     HAL_CMP_3,
00011     HAL_CMP_4,
00012     HAL_CMP_5,
00013 } HAL_CMP_x;
00014
00015 typedef enum
00016 {
00017
00018     HAL_CMP_CHANNEL_IP0    = 0,
00019     HAL_CMP_CHANNEL_IP1    = 4,
00020     HAL_CMP_CHANNEL_IP2    = 5,
00021     HAL_CMP_CHANNEL_IP3    = 6,
00022     HAL_CMP_CHANNEL_IP4    = 7,
00023     HAL_CMP_CHANNEL_IP_DAC0 = 7,
00024
00025     HAL_CMP_CHANNEL_IP_OPA0_IP = 1,
00026     HAL_CMP_CHANNEL_IP_OPA3_IP = 1,
00027
00028     HAL_CMP_CHANNEL_IP_OPA0_OUTP = 2,
00029     HAL_CMP_CHANNEL_IP_OPA1_OUTP = 3,
00030     HAL_CMP_CHANNEL_IP_OPA2_OUTP = 2,
00031     HAL_CMP_CHANNEL_IP_OPA3_OUTP = 3,
00032
00033     HAL_CMP_CHANNEL_IP5,
00034     HAL_CMP_CHANNEL_IP6,
00035     HAL_CMP_CHANNEL_IP7,
00036     HAL_CMP_CHANNEL_IP_AVSS,
00037     HAL_CMP_CHANNEL_IP_DAC1,
00038     HAL_CMP_CHANNEL_IP_OPA_OUT_CMP0,
00039     HAL_CMP_CHANNEL_IP_OPA0_IN,
00040     HAL_CMP_CHANNEL_IP_OPA0_OUTN,
00041     HAL_CMP_CHANNEL_IP_OPA1_IP,
00042     HAL_CMP_CHANNEL_IP_OPA1_IN,
00043     HAL_CMP_CHANNEL_IP_OPA1_OUTN,
00044     HAL_CMP_CHANNEL_IP_OPA2_IP,
00045     HAL_CMP_CHANNEL_IP_OPA2_IN,
00046     HAL_CMP_CHANNEL_IP_OPA2_OUTN,
00047     HAL_CMP_CHANNEL_IP_OPA3_IN,
00048     HAL_CMP_CHANNEL_IP_OPA3_OUTN,
00049     HAL_CMP_CHANNEL_IP_OPA4_IP,
00050     HAL_CMP_CHANNEL_IP_OPA4_IN,
00051     HAL_CMP_CHANNEL_IP_OPA4_OUTP,
00052     HAL_CMP_CHANNEL_IP_OPA4_OUTN,
00053     HAL_CMP_CHANNEL_IP_OPA5_IP,
00054     HAL_CMP_CHANNEL_IP_OPA5_IN,
00055     HAL_CMP_CHANNEL_IP_OPA5_OUTP,
00056     HAL_CMP_CHANNEL_IP_OPA5_OUTN,
00057 } HAL_CMP_CHANNEL_IPx;
00058 typedef enum
00059 {
00060     HAL_CMP_CHANNEL_IN_0,
00061     HAL_CMP_CHANNEL_IN_REF,
00062     HAL_CMP_CHANNEL_IN_DAC,
00063     HAL_CMP_CHANNEL_IN_MID,
00064 } HAL_CMP_CHANNEL_INx;
00065 // 初始化
00066 void HAL_CMP_Reset(HAL_CMP_x); // 模块复位
00067 void HAL_CMP_Enable(HAL_CMP_x); // 模块打开
00068 void HAL_CMP_Disable(HAL_CMP_x); // 模块关闭
00069
00070 void HAL_CMP_SetChannel(HAL_CMP_x, HAL_CMP_CHANNEL_IPx, HAL_CMP_CHANNEL_INx); // 设置CMP输入信号通道
00071 uint32_t HAL_CMP_SetDigitalFilter(HAL_CMP_x, uint32_t filter); // 设置数字滤波宽度 0-0xffff
// 单位McU时钟周期,根据芯片不同,实际滤波宽度可能会有差异,返回值,实际滤波宽度
00072 void HAL_CMP_SetGpioOut(HAL_CMP_x cmpx, HAL_GPIO_Px cmpgpio);
00073 void HAL_CMP_SetGpioIn(HAL_CMP_x cmpx, HAL_GPIO_Px cmpgpio);
00074 uint8_t HAL_CMP_GetData(HAL_CMP_x);
00075
00076 #endif

```


4.17 include/hal_lksmcu_crc.h 文件参考

```
#include "stdint.h"
```

hal_lksmcu_crc.h 的引用(Include)关系图:



此图展示该文件直接或间接的被哪些文件引用了:



枚举

- enum HAL_CRC_x { HAL_CRC_0 }
- enum HAL_CRC_MODEL_x { HAL_CRC_MODEL_CRC8, HAL_CRC_MODEL_CRC8_ITU, HAL_CRC_MODEL_CRC8_ROHC, HAL_CRC_MODEL_CRC8_MAXIM, HAL_CRC_MODEL_CRC16_IBMx1, HAL_CRC_MODEL_CRC16_MAXIM, HAL_CRC_MODEL_CRC16_USBx1, HAL_CRC_MODEL_CRC16_MODBU, HAL_CRC_MODEL_CRC16_CCITT, HAL_CRC_MODEL_CRC16_CCITT_FALSE, HAL_CRC_MODEL_CRC16_X25, HAL_CRC_MODEL_CRC16_XMODEM, HAL_CRC_MODEL_CRC16_DNP, HAL_CRC_MODEL_CRC32, HAL_CRC_MODEL_CRC32_MPEG_2 }

函数

- void HAL_CRC_Reset (HAL_CRC_x)
- void HAL_CRC_Enable (HAL_CRC_x)
- void HAL_CRC_Disable (HAL_CRC_x)
- uint32_t HAL_CRC_Calc (HAL_CRC_x, HAL_CRC_MODEL_x, uint8_t *data, uint32_t len)

4.17.1 枚举类型说明

4.17.1.1 HAL_CRC_MODEL_x

```
enum HAL_CRC_MODEL_x
```

枚举值

HAL_CRC_MODEL_CRC8	8 07 00 00 false false
HAL_CRC_MODEL_CRC8_ITU	8 07 00 55 false false
HAL_CRC_MODEL_CRC8_ROHC	8 07 FF 00 true true
HAL_CRC_MODEL_CRC8_MAXIMx	8 31 00 00 true true
HAL_CRC_MODEL_CRC16_IBMx1	16 8005 0000 0000 true true
HAL_CRC_MODEL_CRC16_MAXIM	16 8005 0000 FFFF true true
HAL_CRC_MODEL_CRC16_USBx1	16 8005 FFFF FFFF true true
HAL_CRC_MODEL_CRC16_MODBU	16 8005 FFFF 0000 true true
HAL_CRC_MODEL_CRC16_CCITT	16 1021 0000 0000 true true
HAL_CRC_MODEL_CRC16_CCITT_FALSE	16 1021 FFFF 0000 false false
HAL_CRC_MODEL_CRC16_X25	16 1021 FFFF FFFF true true
HAL_CRC_MODEL_CRC16_XMODEM	16 1021 0000 0000 false false
HAL_CRC_MODEL_CRC16_DNP	16 3D65 0000 FFFF true true
HAL_CRC_MODEL_CRC32	32 04C11DB7 FFFFFFFF FFFFFFFF true true
HAL_CRC_MODEL_CRC32_MPEG_2	32 04C11DB7 FFFFFFFF 00000000 false false

在文件 [hal_lksmcu_crc.h](#) 第 10 行定义.

4.17.1.2 HAL_CRC_x

enum [HAL_CRC_x](#)

枚举值

HAL_↔ CRC_0	CRC模块0
----------------	--------

在文件 [hal_lksmcu_crc.h](#) 第 5 行定义.

4.17.2 函数说明

4.17.2.1 HAL_CRC_Calc()

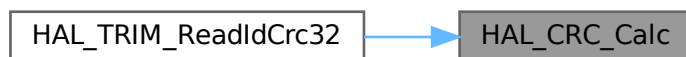
```
uint32_t HAL_CRC_Calc (
    HAL\_CRC\_x crc_x,
    HAL\_CRC\_MODEL\_x model,
    uint8_t * data,
    uint32_t len )
```

在文件 [hal_lks32mc07x_crc.c](#) 第 59 行定义.

函数调用图:



这是这个函数的调用关系图:



4.17.2.2 HAL_CRC.Disable()

```
void HAL_CRC_Disable (
    HAL_CRC_x crc_x )
```

在文件 [hal_lks32mc07x_crc.c](#) 第 16 行定义.

4.17.2.3 HAL_CRC.Enable()

```
void HAL_CRC_Enable (
    HAL_CRC_x crc_x )
```

在文件 [hal_lks32mc07x_crc.c](#) 第 11 行定义.

4.17.2.4 HAL_CRC.Reset()

```
void HAL_CRC_Reset (
    HAL_CRC_x crc_x )
```

在文件 [hal_lks32mc07x_crc.c](#) 第 6 行定义.

4.18 hal_lksmcu_crc.h

浏览该文件的文档.

```

00001 #ifndef __HAL_LKSMCU_CRC_H__
00002 #define __HAL_LKSMCU_CRC_H__
00003
00004 #include "stdint.h"
00005 typedef enum
00006 {
00007     HAL_CRC_0,
00008 } HAL_CRC_x;
00009
00010 typedef enum
00011 {
00012     HAL_CRC_MODEL_CRC8,
00013     HAL_CRC_MODEL_CRC8_ITU,
00014     HAL_CRC_MODEL_CRC8_ROHC,
00015     HAL_CRC_MODEL_CRC8_MAXIMx,
00016     HAL_CRC_MODEL_CRC16_IBMx1,
00017     HAL_CRC_MODEL_CRC16_MAXIM,
00018     HAL_CRC_MODEL_CRC16_USBx1,
00019     HAL_CRC_MODEL_CRC16_MODBU,
00020     HAL_CRC_MODEL_CRC16_CCITT,
00021     HAL_CRC_MODEL_CRC16_CCITT_FALSE,
00022     HAL_CRC_MODEL_CRC16_X25,
00023     HAL_CRC_MODEL_CRC16_XMODEM,
00024     HAL_CRC_MODEL_CRC16_DNP,
00025     HAL_CRC_MODEL_CRC32,
00026     HAL_CRC_MODEL_CRC32_MPEG_2,
00027 } HAL_CRC_MODEL_x;
00028 // 初始化
00029 void HAL_CRC_Reset (HAL_CRC_x); // 模块复位
00030 void HAL_CRC_Enable (HAL_CRC_x); // 模块打开
00031 void HAL_CRC_Disable (HAL_CRC_x); // 模块关闭
00032 // 计算
00033 uint32_t HAL_CRC_Calc (HAL_CRC_x, HAL_CRC_MODEL_x, uint8_t *data, uint32_t len); // 计算crc, 03使用软件实现, 其
    余芯片使用硬件实现
00034 #endif

```

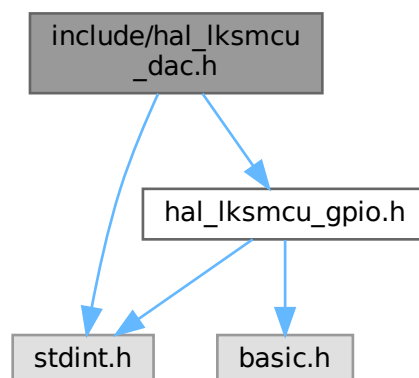
4.19 include/hal_lksmcu_dac.h 文件参考

```

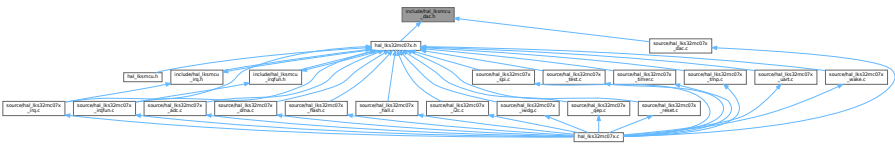
#include "stdint.h"
#include "hal_lksmcu_gpio.h"

```

hal_lksmcu_dac.h 的引用(Include)关系图:



此图展示该文件直接或间接的被哪些文件引用了:



枚举

- enum HAL_DAC_x { HAL_DAC_0 , HAL_DAC_1 , HAL_DAC_2 , HAL_DAC_3 }

函数

- void HAL_DAC_Reset (HAL_DAC_x)
- void HAL_DAC_Enable (HAL_DAC_x)
- void HAL_DAC_Disable (HAL_DAC_x)
- void HAL_DAC_SetOutVal (HAL_DAC_x, uint16_t val)
- void HAL_DAC_SetOutVoltageV (HAL_DAC_x, uint16_t vol_mV)
- void HAL_DAC_Out2GpioEnable (HAL_DAC_x)
- void HAL_DAC_Out2GpioDisable (HAL_DAC_x)
- void HAL_DAC_SetGpioOut (HAL_DAC_x, HAL_GPIO_Px)

设置DAC输出到GPIO

4.19.1 枚举类型说明

4.19.1.1 HAL_DAC_x

enum HAL_DAC_x

枚举值

HAL_↔ DAC_0	DAC模块0
HAL_↔ DAC_1	DAC模块1
HAL_↔ DAC_2	DAC模块2
HAL_↔ DAC_3	DAC模块3

在文件 `hal_lksmcu.dac.h` 第 6 行定义.

4.19.2 函数说明

4.19.2.1 HAL_DAC_Disable()

```
void HAL_DAC_Disable (
    HAL_DAC_x dac_x )
```

在文件 [hal_lks32mc07x.dac.c](#) 第 32 行定义.

这是这个函数的调用关系图:



4.19.2.2 HAL_DAC.Enable()

```
void HAL_DAC_Enable (
    HAL_DAC_x dac_x )
```

在文件 [hal_lks32mc07x.dac.c](#) 第 13 行定义.

4.19.2.3 HAL_DAC.Out2GpioDisable()

```
void HAL_DAC_Out2GpioDisable (
    HAL_DAC_x dac_x )
```

在文件 [hal_lks32mc07x.dac.c](#) 第 169 行定义.

4.19.2.4 HAL_DAC.Out2GpioEnable()

```
void HAL_DAC_Out2GpioEnable (
    HAL_DAC_x dac_x )
```

在文件 [hal_lks32mc07x.dac.c](#) 第 151 行定义.

4.19.2.5 HAL_DAC.Reset()

```
void HAL_DAC_Reset (
    HAL_DAC_x dac_x )
```

在文件 [hal_lks32mc07x.dac.c](#) 第 8 行定义.

函数调用图:



4.19.2.6 HAL_DAC_SetGpioOut()

```
void HAL_DAC_SetGpioOut (
    HAL_DAC_x dac,
    HAL_GPIO_Px px )
```

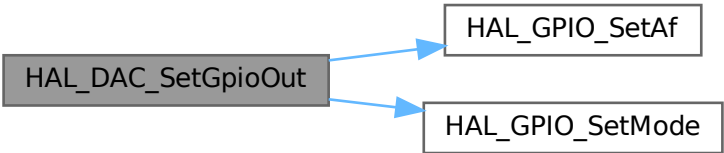
设置DAC输出到GPIO

参数

<i>dac</i>	DAC模块
<i>px</i>	GPIO

在文件 [hal_lks32mc07x.dac.c](#) 第 193 行定义.

函数调用图:



4.19.2.7 HAL_DAC_SetOutVal()

```
void HAL_DAC_SetOutVal (
    HAL_DAC_x dac_x,
    uint16_t val )
```

在文件 [hal_lks32mc07x.dac.c](#) 第 51 行定义.

4.19.2.8 HAL_DAC_SetOutVoltagemV()

```
void HAL_DAC_SetOutVoltagemV (
    HAL_DAC_x dac_x,
    uint16_t vol_mV )
```

在文件 [hal_lks32mc07x.dac.c](#) 第 65 行定义.

函数调用图:



4.20 hal_lksmcu_dac.h

浏览该文件的文档.

```

00001 #ifndef __HAL_LKSMCU_DAC_H__
00002 #define __HAL_LKSMCU_DAC_H__
00003 #include "stdint.h"
00004 #include "hal_lksmcu_gpio.h"
00005
00006 typedef enum
00007 {
00008     HAL_DAC_0,
00009     HAL_DAC_1,
00010     HAL_DAC_2,
00011     HAL_DAC_3,
00012 } HAL_DAC_x;
00013 void HAL_DAC_Reset (HAL_DAC_x); // 模块复位
00014 void HAL_DAC_Enable (HAL_DAC_x); // 模块打开
00015 void HAL_DAC_Disable (HAL_DAC_x); // 模块关闭
00016
00017 void HAL_DAC_SetOutVal (HAL_DAC_x, uint16_t val);
00018 void HAL_DAC_SetOutVoltageMv (HAL_DAC_x, uint16_t vol_mv);
00019 void HAL_DAC_Out2GpioEnable (HAL_DAC_x);
00020 void HAL_DAC_Out2GpioDisable (HAL_DAC_x);
00021 void HAL_DAC_SetGpioOut (HAL_DAC_x, HAL_GPIO_Px);
00022 #endif

```

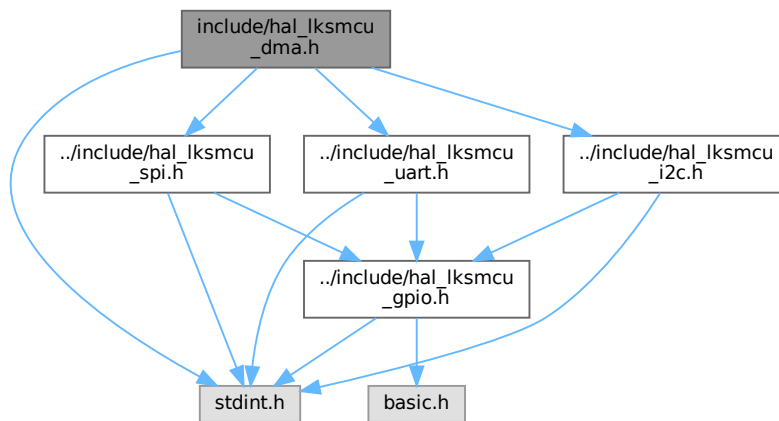
4.21 include/hal_lksmcu_dma.h 文件参考

```

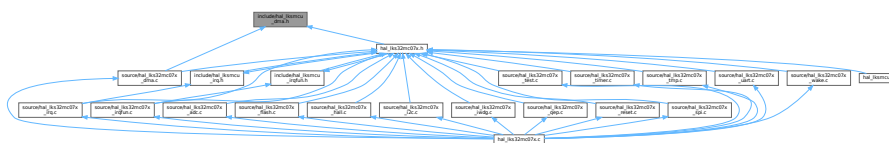
#include "../include/hal_lksmcu_spi.h"
#include "../include/hal_lksmcu_uart.h"
#include "../include/hal_lksmcu_i2c.h"
#include "stdint.h"

```

hal_lksmcu_dma.h 的引用(Include)关系图:



此图展示该文件直接或间接的被哪些文件引用了:



枚举

- enum `HAL_DMA_x` { `HAL_DMA_0` , `HAL_DMA_1` , `HAL_DMA_2` , `HAL_DMA_3` }

函数

- void `HAL_DMA_Reset` (`HAL_DMA_x`)
- void `HAL_DMA_Enable` (`HAL_DMA_x`)
- void `HAL_DMA_Disable` (`HAL_DMA_x`)
- void `HAL_DMA_MemCopy` (`HAL_DMA_x`, void *start, void *end, uint32_t len)
- void `HAL_DMA_UartTx` (`HAL_DMA_x`, `HAL_UART_x`, uint8_t *start, uint32_t len)
- void `HAL_DMA_SpiTx` (`HAL_DMA_x`, `HAL_SPI_x`, uint8_t *start, uint32_t len)
- void `HAL_DMA_I2cTx` (`HAL_DMA_x`, `HAL_I2C_x`, uint8_t *start, uint32_t len)
- void `HAL_DMA_UartRx` (`HAL_DMA_x`, `HAL_UART_x`, uint8_t *start, uint32_t len)
- void `HAL_DMA_SpiRx` (`HAL_DMA_x`, `HAL_SPI_x`, uint8_t *start, uint32_t len)
- void `HAL_DMA_I2cRx` (`HAL_DMA_x`, `HAL_I2C_x`, uint8_t *start, uint32_t len)
- void `HAL_DMA_SpiRxCtx` (`HAL_DMA_x`, `HAL_DMA_x`, `HAL_SPI_x`, uint8_t *txdata, uint8_t *rxdata, uint32_t len)

4.21.1 枚举类型说明

4.21.1.1 HAL_DMA_x

enum `HAL_DMA_x`

枚举值

<code>HAL_DMA_0</code>	DMA模块0
<code>HAL_DMA_1</code>	DMA模块1
<code>HAL_DMA_2</code>	DMA模块2
<code>HAL_DMA_3</code>	DMA模块3

在文件 `hal_lksmcu_dma.h` 第 8 行定义.

4.21.2 函数说明

4.21.2.1 HAL_DMA_Disable()

```
void HAL_DMA_Disable (  
    HAL_DMA_x dma_x )
```

在文件 `hal_lks32mc07x_dma.c` 第 29 行定义.

4.21.2.2 HAL_DMA_Enable()

```
void HAL_DMA_Enable (
    HAL_DMA_x dma_x )
```

在文件 [hal_lks32mc07x_dma.c](#) 第 22 行定义.

4.21.2.3 HAL_DMA_I2cRx()

```
void HAL_DMA_I2cRx (
    HAL_DMA_x dma_x,
    HAL_I2C_x I2Cx,
    uint8_t * start,
    uint32_t len )
```

在文件 [hal_lks32mc07x_dma.c](#) 第 98 行定义.

4.21.2.4 HAL_DMA_I2cTx()

```
void HAL_DMA_I2cTx (
    HAL_DMA_x dma_x,
    HAL_I2C_x I2Cx,
    uint8_t * start,
    uint32_t len )
```

在文件 [hal_lks32mc07x_dma.c](#) 第 68 行定义.

4.21.2.5 HAL_DMA_MemCopy()

```
void HAL_DMA_MemCopy (
    HAL_DMA_x dma_x,
    void * start,
    void * end,
    uint32_t len )
```

在文件 [hal_lks32mc07x_dma.c](#) 第 38 行定义.

4.21.2.6 HAL_DMA_Reset()

```
void HAL_DMA_Reset (
    HAL_DMA_x dma_x )
```

在文件 [hal_lks32mc07x_dma.c](#) 第 10 行定义.

4.21.2.7 HAL_DMA_SpiRx()

```
void HAL_DMA_SpiRx (
    HAL_DMA_x dma_x,
    HAL_SPI_x SPI_x,
    uint8_t * start,
    uint32_t len )
```

在文件 [hal_lks32mc07x_dma.c](#) 第 88 行定义.

4.21.2.8 HAL_DMA_SpiRxDx()

```
void HAL_DMA_SpiRxDx (
    HAL_DMA_x DMA1_x,
    HAL_DMA_x DMA2_x,
    HAL_SPI_x SPI_x,
    uint8_t * txdata,
    uint8_t * rxdata,
    uint32_t len )
```

在文件 [hal_lks32mc07x_dma.c](#) 第 108 行定义.

4.21.2.9 HAL_DMA_SpiTx()

```
void HAL_DMA_SpiTx (
    HAL_DMA_x dma_x,
    HAL_SPI_x SPI_x,
    uint8_t * start,
    uint32_t len )
```

在文件 [hal_lks32mc07x_dma.c](#) 第 58 行定义.

4.21.2.10 HAL_DMA_UartRx()

```
void HAL_DMA_UartRx (
    HAL_DMA_x dma_x,
    HAL_UART_x UART_x,
    uint8_t * start,
    uint32_t len )
```

在文件 [hal_lks32mc07x_dma.c](#) 第 78 行定义.

4.21.2.11 HAL_DMA_UartTx()

```
void HAL_DMA_UartTx (
    HAL_DMA_x dma_x,
    HAL_UART_x UART_x,
    uint8_t * start,
    uint32_t len )
```

在文件 [hal_lks32mc07x_dma.c](#) 第 48 行定义.

4.22 hal_lksmcu_dma.h

[浏览该文件的文档.](#)

```

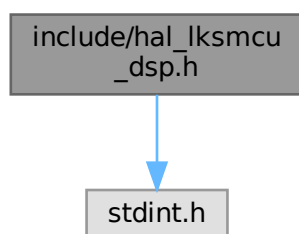
00001 #ifndef __HAL_LKSMCU_DMA_H__
00002 #define __HAL_LKSMCU_DMA_H__
00003
00004 #include "../include/hal_lksmcu_spi.h"
00005 #include "../include/hal_lksmcu_uart.h"
00006 #include "../include/hal_lksmcu_i2c.h"
00007 #include "stdint.h"
00008 typedef enum
00009 {
00010     HAL_DMA_0,
00011     HAL_DMA_1,
00012     HAL_DMA_2,
00013     HAL_DMA_3,
00014 } HAL_DMA_x;
00015
00016 // 初始化
00017 void HAL_DMA_Reset(HAL_DMA_x); // 模块复位
00018 void HAL_DMA_Enable(HAL_DMA_x); // 模块打开
00019 void HAL_DMA_Disable(HAL_DMA_x); // 模块关闭
00020
00021 void HAL_DMA_MemCopy(HAL_DMA_x, void *start, void *end, uint32_t len); // 内存复制
00022
00023 void HAL_DMA_UartTx(HAL_DMA_x, HAL_UART_x, uint8_t *start, uint32_t len); // dma使用uart发送数据,需要先初始化好对应的uart模块
00024 void HAL_DMA_SpiTx(HAL_DMA_x, HAL_SPI_x, uint8_t *start, uint32_t len); // dma使用spi发送数据,需要先初始化好对应的spi模块
00025 void HAL_DMA_I2cTx(HAL_DMA_x, HAL_I2C_x, uint8_t *start, uint32_t len); // dma使用i2c发送数据,需要先初始化好对应的i2c模块
00026 void HAL_DMA_UartRx(HAL_DMA_x, HAL_UART_x, uint8_t *start, uint32_t len); // dma使用uart发送数据,需要先初始化好对应的uart模块
00027 void HAL_DMA_SpiRx(HAL_DMA_x, HAL_SPI_x, uint8_t *start, uint32_t len); // dma使用spi发送数据,需要先初始化好对应的spi模块
00028 void HAL_DMA_I2cRx(HAL_DMA_x, HAL_I2C_x, uint8_t *start, uint32_t len); // dma使用i2c发送数据,需要先初始化好对应的i2c模块
00029
00030 void HAL_DMA_SpiRxTx(HAL_DMA_x, HAL_DMA_x, HAL_SPI_x, uint8_t *txdata, uint8_t *rxdata, uint32_t len); // 使用两个dma模块和一个spi模块,同时发送&接收数据,需要先初始化好对应的spi模块
00031
00032 #endif

```

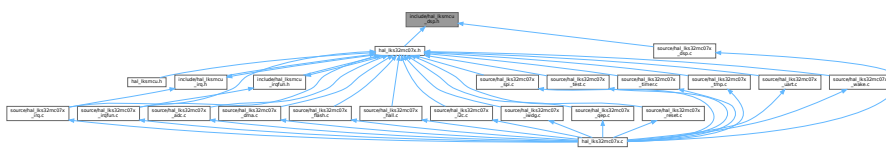
4.23 include/hal_lksmcu_dsp.h 文件参考

```
#include "stdint.h"
```

hal_lksmcu_dsp.h 的引用(Include)关系图:



此图展示该文件直接或间接的被哪些文件引用了:



枚举

- enum HAL_DSP_x { HAL_DSP_0 }

函数

- void [HAL_DSP_Reset](#) (HAL_DSP_x)
- void [HAL_DSP_Enable](#) (HAL_DSP_x)
- void [HAL_DSP_Disable](#) (HAL_DSP_x)
- int16_t [HAL_DSP_Sin](#) (HAL_DSP_x, int16_t angle)
- int16_t [HAL_DSP_Cos](#) (HAL_DSP_x, int16_t angle)
- void [HAL_DSP_SinCos](#) (HAL_DSP_x, int16_t angle, int16_t *sin, int16_t *cos)
- uint16_t [HAL_DSP_Sqrt](#) (HAL_DSP_x, uint32_t)
- int32_t [HAL_DSP_Div](#) (HAL_DSP_x, int32_t, int32_t)

4.23.1 枚举类型说明

4.23.1.1 HAL_DSP_x

```
enum HAL_DSP_x
```

枚举值

HAL↔ DSP_0	DSP模块0
---------------	--------

在文件 `hal_ksmcu_dsp.h` 第 5 行定义.

4.23.2 函数说明

4.23.2.1 HAL_DSP_Cos()

```
int16_t HAL_DSP_Cos (
    HAL_DSP_x dsp_x,
    int16_t angle )
```

在文件 `hal_lks32mc07x_dsp.c` 第 41 行定义.

4.23.2.2 HAL_DSP_Disable()

```
void HAL_DSP_Disable (
    HAL_DSP_x dsp_x )
```

在文件 [hal_lks32mc07x.dsp.c](#) 第 25 行定义.

4.23.2.3 HAL_DSP_Div()

```
int32_t HAL_DSP_Div (
    HAL_DSP_x dsp_x,
    int32_t a,
    int32_t b )
```

在文件 [hal_lks32mc07x.dsp.c](#) 第 62 行定义.

4.23.2.4 HAL_DSP_Enable()

```
void HAL_DSP_Enable (
    HAL_DSP_x dsp_x )
```

在文件 [hal_lks32mc07x.dsp.c](#) 第 16 行定义.

4.23.2.5 HAL_DSP_Reset()

```
void HAL_DSP_Reset (
    HAL_DSP_x dsp_x )
```

在文件 [hal_lks32mc07x.dsp.c](#) 第 6 行定义.

4.23.2.6 HAL_DSP_Sin()

```
int16_t HAL_DSP_Sin (
    HAL_DSP_x dsp_x,
    int16_t angle )
```

在文件 [hal_lks32mc07x.dsp.c](#) 第 34 行定义.

4.23.2.7 HAL_DSP_SinCos()

```
void HAL_DSP_SinCos (
    HAL_DSP_x dsp_x,
    int16_t angle,
    int16_t * sin,
    int16_t * cos )
```

在文件 [hal_lks32mc07x.dsp.c](#) 第 48 行定义.

4.23.2.8 HAL_DSP_Sqrt()

```
uint16_t HAL_DSP_Sqrt (
    HAL_DSP_x dsp_x,
    uint32_t x )
```

在文件 [hal_lks32mc07x.dsp.c](#) 第 56 行定义.

4.24 hal_lksmcu_dsp.h

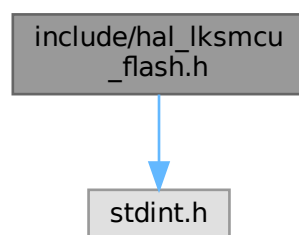
浏览该文件的文档.

```
00001 #ifndef __HAL_LKSMCU_DSP_H__
00002 #define __HAL_LKSMCU_DSP_H__
00003 #include "stdint.h"
00004
00005 typedef enum
00006 {
00007     HAL_DSP_0,
00008 } HAL_DSP_x;
00009 void HAL_DSP_Reset(HAL_DSP_x); // 模块复位
00010 void HAL_DSP_Enable(HAL_DSP_x); // 模块打开
00011 void HAL_DSP_Disable(HAL_DSP_x); // 模块关闭
00012
00013 int16_t HAL_DSP_Sin(HAL_DSP_x, int16_t angle);
00014 int16_t HAL_DSP_Cos(HAL_DSP_x, int16_t angle);
00015 void HAL_DSP_SinCos(HAL_DSP_x, int16_t angle, int16_t *sin, int16_t *cos);
00016 uint16_t HAL_DSP_Sqrt(HAL_DSP_x, uint32_t);
00017 int32_t HAL_DSP_Div(HAL_DSP_x, int32_t, int32_t);
00018 #endif
```

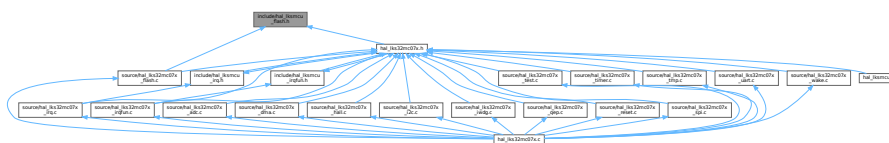
4.25 include/hal_lksmcu_flash.h 文件参考

```
#include "stdint.h"
```

hal_lksmcu_flash.h 的引用(Include)关系图:



此图展示该文件直接或间接的被哪些文件引用了:



枚举

- enum `HAL_FLASH_x` { `HAL_FLASH_0` , `HAL_FLASH_1` }

函数

- void `HAL_FLASH_Reset` (`HAL_FLASH_x`)
模块复位,预留接口
- void `HAL_FLASH_Enable` (`HAL_FLASH_x`)
模块打开,预留接口
- void `HAL_FLASH_Disable` (`HAL_FLASH_x`)
模块关闭,预留接口
- void `HAL_FLASH_EnableCache` (`HAL_FLASH_x`)
使能flash缓存加速
- void `HAL_FLASH_DisableCache` (`HAL_FLASH_x`)
关闭flash缓存加速
- uint32_t `HAL_FLASH_ReadSectorSize` (`HAL_FLASH_x`)
获取一个扇区的大小,和`HAL_FLASH_SECTOR_SIZE`等效
- void `HAL_FLASH_Read` (`HAL_FLASH_x`, uint32_t flashaddr, void *data, uint32_t len)
读FLASH取数据
- uint8_t `HAL_FLASH_ReadUint8` (`HAL_FLASH_x`, uint32_t flashaddr)
读FLASH取数据
- uint16_t `HAL_FLASH_ReadUint16` (`HAL_FLASH_x`, uint32_t flashaddr)
读FLASH取数据
- uint32_t `HAL_FLASH_ReadUint32` (`HAL_FLASH_x`, uint32_t flashaddr)
读FLASH取数据
- void `HAL_FLASH_Erase` (`HAL_FLASH_x`, uint32_t flashaddr, uint32_t len)
扇区擦除,len只能是扇区大小的整数倍,不够一个扇区的部分会被忽略
- void `HAL_FLASH_Program` (`HAL_FLASH_x`, uint32_t flashaddr, uint8_t *data, uint32_t len)
编程
- void `HAL_FLASH_Update` (`HAL_FLASH_x`, uint32_t flashaddr, uint8_t *data, uint32_t len)
更新数据,等效先擦除再编程
- void `HAL_FLASH_SecurityParameterSave` (`HAL_FLASH_x`, void *defaultdata, void *data, uint32_t len)
保存安全参数
- uint32_t `HAL_FLASH_SecurityParameterRead` (`HAL_FLASH_x`, void *defaultdata, void *data, uint32_t len)
读取安全参数

4.25.1 枚举类型说明

4.25.1.1 HAL_FLASH_x

enum `HAL_FLASH_x`

枚举值

<code>HAL_↔ FLASH_0</code>	内部flash
<code>HAL_↔ FLASH_1</code>	外部flash

在文件 [hal_lksmcu_flash.h](#) 第 5 行定义.

4.25.2 函数说明

4.25.2.1 HAL_FLASH_Disable()

```
void HAL_FLASH_Disable (
    HAL_FLASH_X flash )
```

模块关闭,预留接口

参数

<i>flash</i>	模块类型
--------------	------

在文件 [hal_lks32mc07x_flash.c](#) 第 30 行定义.

4.25.2.2 HAL_FLASH_DisableCache()

```
void HAL_FLASH_DisableCache (
    HAL_FLASH_X flash )
```

关闭flash缓存加速

参数

<i>flash</i>	模块类型
--------------	------

在文件 [hal_lks32mc07x_flash.c](#) 第 49 行定义.

4.25.2.3 HAL_FLASH_Enable()

```
void HAL_FLASH_Enable (
    HAL_FLASH_X flash )
```

模块打开,预留接口

参数

<i>flash</i>	模块类型
--------------	------

在文件 [hal_lks32mc07x_flash.c](#) 第 21 行定义.

4.25.2.4 HAL_FLASH_EnableCache()

```
void HAL_FLASH_EnableCache (
    HAL_FLASH_X flash )
```

使能flash缓存加速

参数

<i>flash</i>	模块类型
--------------	------

在文件 [hal_lks32mc07x.flash.c](#) 第 39 行定义.

4.25.2.5 HAL_FLASH_Erase()

```
void HAL_FLASH_Erase (
    HAL_FLASH_X flash,
    uint32_t flashaddr,
    uint32_t len )
```

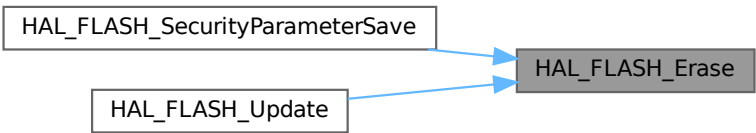
扇区擦除,len只能是扇区大小的整数倍,不够一个扇区的部分会被忽略

参数

<i>flash</i>	模块类型
<i>flashaddr</i>	flash地址
<i>len</i>	擦除长度

在文件 [hal_lks32mc07x.flash.c](#) 第 121 行定义.

这是这个函数的调用关系图:



4.25.2.6 HAL_FLASH_Program()

```
void HAL_FLASH_Program (
    HAL_FLASH_X flash,
    uint32_t flashaddr,
    uint8_t * data,
    uint32_t len )
```

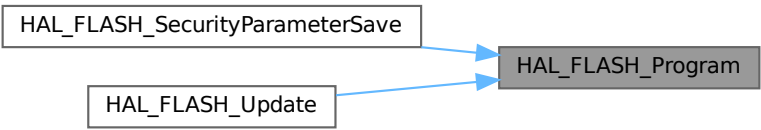
编程

参数

<i>flash</i>	模块类型
<i>flashaddr</i>	flash地址,低两位会被忽略
<i>data</i>	数据指针
<i>len</i>	数据长度

在文件 [hal_lks32mc07x.flash.c](#) 第 141 行定义.

这是这个函数的调用关系图:



4.25.2.7 HAL_FLASH_Read()

```
void HAL_FLASH_Read (
    HAL_FLASH_TypeDef * flash,
    uint32_t flashaddr,
    void * data,
    uint32_t len )
```

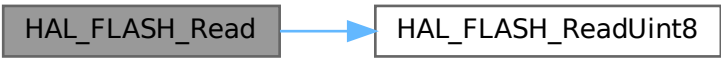
读FLASH取数据

参数

<i>flash</i>	模块类型
<i>flashaddr</i>	flash地址
<i>data</i>	读取的数据指针
<i>len</i>	读取长度

在文件 [hal_lks32mc07x.flash.c](#) 第 73 行定义.

函数调用图:



这是这个函数的调用关系图:



4.25.2.8 HAL_FLASH_ReadSectorSize()

```
uint32_t HAL_FLASH_ReadSectorSize (
    HAL_FLASH_x flash )
```

获取一个扇区的大小,和HAL_FLASH_SECTOR_SIZE等效

参数

flash	模块类型
-------	------

返回

uint32_t 扇区大小

在文件 [hal_lks32mc07x_flash.c](#) 第 60 行定义.

4.25.2.9 HAL_FLASH_ReadUint16()

```
uint16_t HAL_FLASH_ReadUint16 (
    HAL_FLASH_x flash,
    uint32_t flashaddr )
```

读FLASH取数据

参数

flash	模块类型
flashaddr	flash地址

返回

读取的数据

在文件 [hal_lks32mc07x.flash.c](#) 第 98 行定义.

4.25.2.10 HAL_FLASH_ReadUint32()

```
uint32_t HAL_FLASH_ReadUint32 (
    HAL_FLASH_TypeDef * flash,
    uint32_t flashaddr )
```

读FLASH取数据

参数

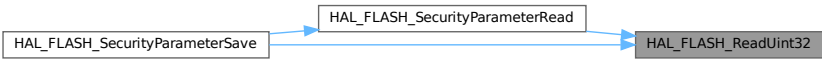
<i>flash</i>	模块类型
<i>flashaddr</i>	flash地址

返回

读取的数据

在文件 [hal_lks32mc07x.flash.c](#) 第 109 行定义.

这是这个函数的调用关系图:



4.25.2.11 HAL_FLASH_ReadUint8()

```
uint8_t HAL_FLASH_ReadUint8 (
    HAL_FLASH_TypeDef * flash,
    uint32_t flashaddr )
```

读FLASH取数据

参数

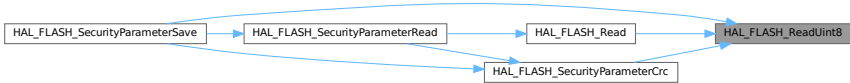
<i>flash</i>	模块类型
<i>flashaddr</i>	flash地址

返回

读取的数据

在文件 [hal_lks32mc07x.flash.c](#) 第 87 行定义.

这是这个函数的调用关系图:



4.25.2.12 HAL_FLASH.Reset()

```
void HAL_FLASH.Reset (
    HAL_FLASH_x flash )
```

模块复位,预留接口

参数

<i>flash</i>	模块类型
--------------	------

在文件 [hal_lks32mc07x.flash.c](#) 第 12 行定义.

4.25.2.13 HAL_FLASH.SecurityParameterRead()

```
uint32_t HAL_FLASH.SecurityParameterRead (
    HAL_FLASH_x flash,
    void * defaultdata,
    void * data,
    uint32_t len )
```

读取安全参数

参数

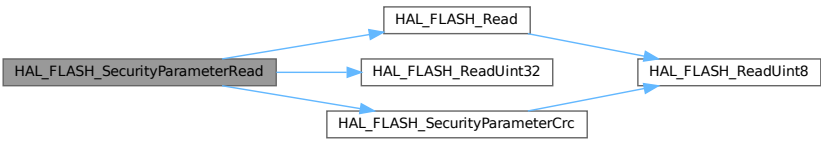
<i>flash</i>	模块类型
<i>data</i>	数据指针
<i>len</i>	数据长度

返回

使用的flash地址

在文件 [hal_lks32mc07x.flash.c](#) 第 297 行定义.

函数调用图:



这是这个函数的调用关系图:



4.25.2.14 HAL_FLASH_SecurityParameterSave()

```
void HAL_FLASH_SecurityParameterSave (
    HAL_FLASH_TypeDef * flash,
    void * defaultdata,
    void * data,
    uint32_t len )
```

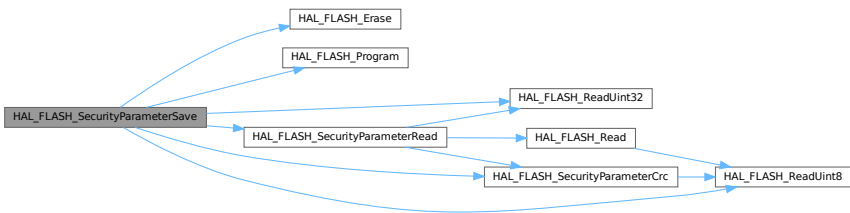
保存安全参数

参数

<i>flash</i>	模块类型
<i>defaultdata</i>	默认值
<i>data</i>	待写入的数据
<i>len</i>	数据长度

在文件 [hal_lks32mc07x_flash.c](#) 第 206 行定义.

函数调用图:



4.25.2.15 HAL_FLASH_Update()

```
void HAL_FLASH_Update (
    HAL_FLASH_x flash,
    uint32_t flashaddr,
    uint8_t * data,
    uint32_t len )
```

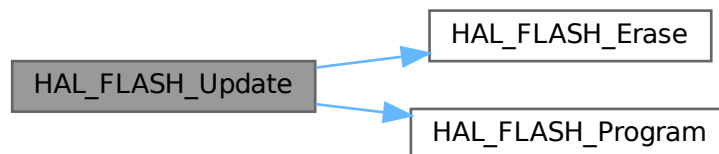
更新数据,等效先擦除再编程

参数

<i>flash</i>	模块类型
<i>flashaddr</i>	flash地址
<i>data</i>	数据指针
<i>len</i>	数据长度

在文件 [hal_lks32mc07x_flash.c](#) 第 164 行定义.

函数调用图:



4.26 hal_lksmcu_flash.h

[浏览该文件的文档.](#)

```
00001 #ifndef __HAL_LKSMCU_FLASH_H__
00002 #define __HAL_LKSMCU_FLASH_H__
00003
00004 #include "stdint.h"
00005 typedef enum
00006 {
00007     HAL_FLASH_0,
00008     HAL_FLASH_1,
00009 } HAL_FLASH_x;
00010 // 初始化
00011 void HAL_FLASH_Reset (HAL_FLASH_x); // 模块复位,预留接口
00012 void HAL_FLASH_Enable (HAL_FLASH_x); // 模块打开,预留接口
00013 void HAL_FLASH_Disable (HAL_FLASH_x); // 模块关闭,预留接口
00014
00015 void HAL_FLASH_EnableCache (HAL_FLASH_x); // 使能flash缓存加
    速
00016 void HAL_FLASH_DisableCache (HAL_FLASH_x); // 关闭flash缓存加
    速
00017 uint32_t HAL_FLASH_ReadSectorSize (HAL_FLASH_x); // 获取一个扇区的大
    小,和HAL_FLASH_SECTOR_SIZE等效
00018 void HAL_FLASH_Read (HAL_FLASH_x, uint32_t flashaddr, void *data, uint32_t len); // 任意长度
00019 uint8_t HAL_FLASH_ReadUint8 (HAL_FLASH_x, uint32_t flashaddr); // 8~(U8)
00020 uint16_t HAL_FLASH_ReadUint16 (HAL_FLASH_x, uint32_t flashaddr); // 16~(U16)
```



```

00021 uint32_t HAL_FLASH_ReadUint32(HAL_FLASH_x, uint32_t flashaddr); // 读(U32)
00022 void HAL_FLASH_Erase(HAL_FLASH_x, uint32_t flashaddr, uint32_t len); // 扇区擦除, len只能是
    扇区大小的整数倍, 不够一个扇区的部分会被忽略
00023 void HAL_FLASH_Program(HAL_FLASH_x, uint32_t flashaddr, uint8_t *data, uint32_t len); // 编程
00024 void HAL_FLASH_Update(HAL_FLASH_x, uint32_t flashaddr, uint8_t *data, uint32_t len); // 更新数据, 等效先擦除
    再编程
00025
00026 // 安全参数 (调用时自动分配地址)
00027 // 支持大小为 500的uint8_t的数据
00028 // 保留3个word用来校验
00029 // 状态+CRC32+保留位
00030 // 调用时必须传入defaultdata (确保极端情况下, data的数据安全)
00031 void HAL_FLASH_SecurityParameterSave(HAL_FLASH_x, void *defaultdata, void *data, uint32_t len); //
    保存数据 (仅变化时保存, 最多可能会关闭中断20ms)
00032 uint32_t HAL_FLASH_SecurityParameterRead(HAL_FLASH_x, void *defaultdata, void *data, uint32_t len); // 读
    数据
00033
00034 #endif

```

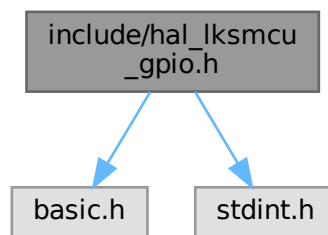
4.27 include/hal_lksmcu_gpio.h 文件参考

```

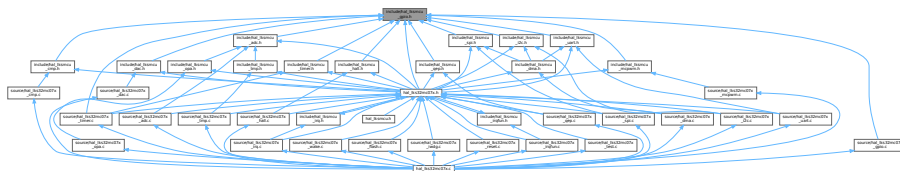
#include "basic.h"
#include "stdint.h"

```

hal_lksmcu_gpio.h 的引用(Include)关系图:



此图展示该文件直接或间接的被哪些文件引用了:



枚举

- enum HAL_GPIO_Px {
 HAL_GPIO_P0_0 = 0x0000, HAL_GPIO_P0_1 = 0x0001, HAL_GPIO_P0_2 = 0x0002, HAL_GPIO_P0_3 = 0x0003,
 HAL_GPIO_P0_4 = 0x0004, HAL_GPIO_P0_5 = 0x0005, HAL_GPIO_P0_6 = 0x0006, HAL_GPIO_P0_7 = 0x0007,
 HAL_GPIO_P0_8 = 0x0008, HAL_GPIO_P0_9 = 0x0009, HAL_GPIO_P0_10 = 0x000a, HAL_GPIO_P0_11 =

```

0x000b ,
HAL_GPIO_P0_12 = 0x000c , HAL_GPIO_P0_13 = 0x000d , HAL_GPIO_P0_14 = 0x000e , HAL_GPIO_P0_15 =
0x000f ,
HAL_GPIO_P1_0 = 0x0100 , HAL_GPIO_P1_1 = 0x0101 , HAL_GPIO_P1_2 = 0x0102 , HAL_GPIO_P1_3 =
0x0103 ,
HAL_GPIO_P1_4 = 0x0104 , HAL_GPIO_P1_5 = 0x0105 , HAL_GPIO_P1_6 = 0x0106 , HAL_GPIO_P1_7 =
0x0107 ,
HAL_GPIO_P1_8 = 0x0108 , HAL_GPIO_P1_9 = 0x0109 , HAL_GPIO_P1_10 = 0x010a , HAL_GPIO_P1_11 =
0x010b ,
HAL_GPIO_P1_12 = 0x010c , HAL_GPIO_P1_13 = 0x010d , HAL_GPIO_P1_14 = 0x010e , HAL_GPIO_P1_15 =
0x010f ,
HAL_GPIO_P2_0 = 0x0200 , HAL_GPIO_P2_1 = 0x0201 , HAL_GPIO_P2_2 = 0x0202 , HAL_GPIO_P2_3 =
0x0203 ,
HAL_GPIO_P2_4 = 0x0204 , HAL_GPIO_P2_5 = 0x0205 , HAL_GPIO_P2_6 = 0x0206 , HAL_GPIO_P2_7 =
0x0207 ,
HAL_GPIO_P2_8 = 0x0208 , HAL_GPIO_P2_9 = 0x0209 , HAL_GPIO_P2_10 = 0x020a , HAL_GPIO_P2_11 =
0x020b ,
HAL_GPIO_P2_12 = 0x020c , HAL_GPIO_P2_13 = 0x020d , HAL_GPIO_P2_14 = 0x020e , HAL_GPIO_P2_15 =
0x020f ,
HAL_GPIO_P3_0 = 0x0300 , HAL_GPIO_P3_1 = 0x0301 , HAL_GPIO_P3_2 = 0x0302 , HAL_GPIO_P3_3 =
0x0303 ,
HAL_GPIO_P3_4 = 0x0304 , HAL_GPIO_P3_5 = 0x0305 , HAL_GPIO_P3_6 = 0x0306 , HAL_GPIO_P3_7 =
0x0307 ,
HAL_GPIO_P3_8 = 0x0308 , HAL_GPIO_P3_9 = 0x0309 , HAL_GPIO_P3_10 = 0x030a , HAL_GPIO_P3_11 =
0x030b ,
HAL_GPIO_P3_12 = 0x030c , HAL_GPIO_P3_13 = 0x030d , HAL_GPIO_P3_14 = 0x030e , HAL_GPIO_P3_15 =
0x030f ,
HAL_GPIO_P4_0 = 0x0400 , HAL_GPIO_P4_1 = 0x0401 , HAL_GPIO_P4_2 = 0x0402 , HAL_GPIO_P4_3 =
0x0403 ,
HAL_GPIO_P4_4 = 0x0404 , HAL_GPIO_P4_5 = 0x0405 , HAL_GPIO_P4_6 = 0x0406 , HAL_GPIO_P4_7 =
0x0407 ,
HAL_GPIO_P4_8 = 0x0408 , HAL_GPIO_P4_9 = 0x0409 , HAL_GPIO_P4_10 = 0x040a , HAL_GPIO_P4_11 =
0x040b ,
HAL_GPIO_P4_12 = 0x040c , HAL_GPIO_P4_13 = 0x040d , HAL_GPIO_P4_14 = 0x040e , HAL_GPIO_P4_15 =
0x040f ,
HAL_GPIO_P5_0 = 0x0500 , HAL_GPIO_P5_1 = 0x0501 , HAL_GPIO_P5_2 = 0x0502 , HAL_GPIO_P5_3 =
0x0503 ,
HAL_GPIO_P5_4 = 0x0504 , HAL_GPIO_P5_5 = 0x0505 , HAL_GPIO_P5_6 = 0x0506 , HAL_GPIO_P5_7 =
0x0507 ,
HAL_GPIO_P5_8 = 0x0508 , HAL_GPIO_P5_9 = 0x0509 , HAL_GPIO_P5_10 = 0x050a , HAL_GPIO_P5_11 =
0x050b ,
HAL_GPIO_P5_12 = 0x050c , HAL_GPIO_P5_13 = 0x050d , HAL_GPIO_P5_14 = 0x050e , HAL_GPIO_P5_15 =
0x050f }

```

GPIO口定义

- enum HAL_GPIO_AFx {
HAL_GPIO_AF_ADC = 0 , HAL_GPIO_AF_OPA = 0 , HAL_GPIO_AF_OPA_OUT = 0 , HAL_GPIO_AF_CMP = 0 ,
HAL_GPIO_AF_DAC = 0 , HAL_GPIO_AF_GPIO = 0 , HAL_GPIO_AF_CLK = 0 , HAL_GPIO_AF_CMP_OUT = 1
,
HAL_GPIO_AF_CLK_OUT = 1 , HAL_GPIO_AF_HALL = 2 , HAL_GPIO_AF_MCPWM = 3 , HAL_GPIO_AF_UART
= 4 ,
HAL_GPIO_AF_SPI = 5 , HAL_GPIO_AF_I2C = 6 , HAL_GPIO_AF_TIMER0 = 7 , HAL_GPIO_AF_TIMER1 = 7 ,
HAL_GPIO_AF_TIMER2 = 8 , HAL_GPIO_AF_TIMER3 = 8 , HAL_GPIO_AF_QEP = 8 , HAL_GPIO_AF_ADC_TRIG
= 9 ,
HAL_GPIO_AF_CAN = 10 , HAL_GPIO_AF_SIF = 11 , HAL_GPIO_AF_CL = 12 , HAL_GPIO_AF_TIMER4 = 0 ,
HAL_GPIO_AF_TIMER5 = 0 }
- enum HAL_GPIO_MODE_x {
HAL_GPIO_MODE_AFE = 0 , HAL_GPIO_MODE_NONE = HAL_GPIO_MODE_AFE , HAL_GPIO_MODE_IN =
BIT0 , HAL_GPIO_MODE_INUP = HAL_GPIO_MODE_IN | BIT2 ,

```

HAL_GPIO_MODE_OUT = BIT1 , HAL_GPIO_MODE_INOUT = HAL_GPIO_MODE_IN | HAL_GPIO_MODE_OUT | BIT3 , HAL_GPIO_MODE_INUPOUT = HAL_GPIO_MODE_INOUT | BIT2 | BIT3 ,
HAL_GPIO_MODE_OUT_0 = HAL_GPIO_MODE_OUT | BIT4 ,
HAL_GPIO_MODE_INOUT_0 = HAL_GPIO_MODE_INOUT | BIT4 , HAL_GPIO_MODE_INUPOUT_0 = HAL_GPIO_MODE_INUPOUT | BIT4 , HAL_GPIO_MODE_OUT_1 = HAL_GPIO_MODE_OUT | BIT5 ,
HAL_GPIO_MODE_INOUT_1 = HAL_GPIO_MODE_INOUT | BIT5 ,
HAL_GPIO_MODE_INUPOUT_1 = HAL_GPIO_MODE_INUPOUT | BIT5 }

```

*GPIO*工作模式定义 *BIT0* 输入使能 *BIT1* 输出使能 *BIT2* 上拉使能 *BIT3* 开漏使能 *BIT4* 初始化输出为低电平 *BIT5* 初始化输出为高电平

函数

- void **HAL_GPIO_Reset** (void)
模块复位,执行后恢复所有的*GPIO*为悬空状态(*SWD*、*JTAG*和*RST*口为默认状态)
- void **HAL_GPIO_Enable** (void)
启用*GPIO*模块,使能*GPIO*时钟和必要的资源
- void **HAL_GPIO_Disable** (void)
禁用*GPIO*模块,关闭*GPIO*时钟和释放资源
- void **HAL_GPIO_SetMode** (HAL_GPIO_Px, HAL_GPIO_MODE_x)
设置指定*GPIO*的工作模式
- void **HAL_GPIO_SetHigh** (HAL_GPIO_Px)
将指定的*GPIO*端口输出高电平(1)
- void **HAL_GPIO_SetLow** (HAL_GPIO_Px)
将指定的*GPIO*端口输出低电平(0)
- void **HAL_GPIO_Write** (HAL_GPIO_Px, uint8_t)
向指定的*GPIO*端口写入0或1
- void **HAL_GPIO_Invert** (HAL_GPIO_Px)
翻转指定的*GPIO*端口的输出状态
- uint8_t **HAL_GPIO_GetOutData** (HAL_GPIO_Px)
获取当前指定*GPIO*端口的输出值
- uint8_t **HAL_GPIO_GetInData** (HAL_GPIO_Px)
获取当前指定*GPIO*端口的输入值
- void **HAL_GPIO_SetAf** (HAL_GPIO_Px, HAL_GPIO_AFx)
配置*GPIO*的特殊功能,如*UART*、*I2C*、*SPI*等
- void **HAL_GPIO_SwdReuseGpioEnable** (void)
将*SWD*接口配置为普通*GPIO*口
- void **HAL_GPIO_SwdReuseGpioDisable** (void)
将*SWD*接口恢复为*SWD*功能
- void **HAL_GPIO_RstReuseGpioEnable** (void)
将*RST*接口配置为普通*GPIO*口
- void **HAL_GPIO_RstReuseGpioDisable** (void)
将*RST*接口恢复为*RST*功能

4.27.1 枚举类型说明

4.27.1.1 HAL_GPIO_AFx

enum **HAL_GPIO_AFx**

枚举值

HAL_GPIO_AF_ADC	
HAL_GPIO_AF_OPA	
HAL_GPIO_AF_OPA_OUT	
HAL_GPIO_AF_CMP	
HAL_GPIO_AF_DAC	
HAL_GPIO_AF_GPIO	
HAL_GPIO_AF_CLK	
HAL_GPIO_AF_CMP_OUT	
HAL_GPIO_AF_CLK_OUT	
HAL_GPIO_AF_HALL	
HAL_GPIO_AF_MCPWM	
HAL_GPIO_AF_UART	
HAL_GPIO_AF_SPI	
HAL_GPIO_AF_I2C	
HAL_GPIO_AF_TIMER0	
HAL_GPIO_AF_TIMER1	
HAL_GPIO_AF_TIMER2	
HAL_GPIO_AF_TIMER3	
HAL_GPIO_AF_QEP	
HAL_GPIO_AF_ADC_TRIG	
HAL_GPIO_AF_CAN	
HAL_GPIO_AF_SIF	
HAL_GPIO_AF_CL	
HAL_GPIO_AF_TIMER4	
HAL_GPIO_AF_TIMER5	

在文件 [hal_lksmcu_gpio.h](#) 第 106 行定义.

4.27.1.2 HAL_GPIO_MODE_x

enum [HAL_GPIO_MODE_x](#)

GPIO工作模式定义 BIT0 输入使能 BIT1 输出使能 BIT2 上拉使能 BIT3 开漏使能 BIT4 初始化输出为低电平 BIT5 初始化输出为高电平

枚举值

HAL_GPIO_MODE_AFE	GPIO设置为模拟接口
HAL_GPIO_MODE_NONE	GPIO设置为不使用 输出输出均关闭
HAL_GPIO_MODE_IN	GPIO设置为输入模式
HAL_GPIO_MODE_INUP	GPIO设置为上拉输入模式(输入模式,打开上拉电阻)
HAL_GPIO_MODE_OUT	GPIO设置为输出模式
HAL_GPIO_MODE_INOUT	GPIO设置为输入输出模式(可以输出为0或悬空,悬空时输入有效)
HAL_GPIO_MODE_INUPOUT	GPIO设置为上拉输入输出模式(可以输出为0或上拉,上拉时输入有效,线与模式)
HAL_GPIO_MODE_OUT_0	GPIO设置为输出模式 初始化完成后io状态为低电平
HAL_GPIO_MODE_INOUT_0	GPIO设置为输入输出模式(可以输出为0或悬空,悬空时输入有效) 初始化完成后io状态为低电平

枚举值

HAL_GPIO_MODE_↔ INUPOUT_0	GPIO设置为上拉输入输出模式(可以输出为0或上拉,上拉时输入有效,线与模式) 初始化完成后io状态为低电平
HAL_GPIO_MODE_OUT_1	GPIO设置为输出模式 初始化完成后io状态为高电平
HAL_GPIO_MODE_INOUT_1	GPIO设置为输入输出模式(可以输出为0或悬空,悬空时输入有效) 初始化完成后io状态为悬空
HAL_GPIO_MODE_↔ INUPOUT_1	GPIO设置为上拉输入输出模式(可以输出为0或上拉,上拉时输入有效,线与模式) 初始化完成后io状态为上拉

在文件 [hal_lksmcu_gpio.h](#) 第 148 行定义.

4.27.1.3 HAL_GPIO_Px

enum [HAL_GPIO_Px](#)

GPIO口定义

枚举值

HAL_GPIO_P0.0	P0.0
HAL_GPIO_P0.1	P0.1
HAL_GPIO_P0.2	P0.2
HAL_GPIO_P0.3	P0.3
HAL_GPIO_P0.4	P0.4
HAL_GPIO_P0.5	P0.5
HAL_GPIO_P0.6	P0.6
HAL_GPIO_P0.7	P0.7
HAL_GPIO_P0.8	P0.8
HAL_GPIO_P0.9	P0.9
HAL_GPIO_P0.10	P0.10
HAL_GPIO_P0.11	P0.11
HAL_GPIO_P0.12	P0.12
HAL_GPIO_P0.13	P0.13
HAL_GPIO_P0.14	P0.14
HAL_GPIO_P0.15	P0.15
HAL_GPIO_P1.0	P1.0
HAL_GPIO_P1.1	P1.1
HAL_GPIO_P1.2	P1.2
HAL_GPIO_P1.3	P1.3
HAL_GPIO_P1.4	P1.4
HAL_GPIO_P1.5	P1.5
HAL_GPIO_P1.6	P1.6
HAL_GPIO_P1.7	P1.7
HAL_GPIO_P1.8	P1.8
HAL_GPIO_P1.9	P1.9
HAL_GPIO_P1.10	P1.10
HAL_GPIO_P1.11	P1.11
HAL_GPIO_P1.12	P1.12
HAL_GPIO_P1.13	P1.13

枚举值

HAL_GPIO_P1_14	P1.14
HAL_GPIO_P1_15	P1.15
HAL_GPIO_P2_0	P2.0
HAL_GPIO_P2_1	P2.1
HAL_GPIO_P2_2	P2.2
HAL_GPIO_P2_3	P2.3
HAL_GPIO_P2_4	P2.4
HAL_GPIO_P2_5	P2.5
HAL_GPIO_P2_6	P2.6
HAL_GPIO_P2_7	P2.7
HAL_GPIO_P2_8	P2.8
HAL_GPIO_P2_9	P2.9
HAL_GPIO_P2_10	P2.10
HAL_GPIO_P2_11	P2.11
HAL_GPIO_P2_12	P2.12
HAL_GPIO_P2_13	P2.13
HAL_GPIO_P2_14	P2.14
HAL_GPIO_P2_15	P2.15
HAL_GPIO_P3_0	P3.0
HAL_GPIO_P3_1	P3.1
HAL_GPIO_P3_2	P3.2
HAL_GPIO_P3_3	P3.3
HAL_GPIO_P3_4	P3.4
HAL_GPIO_P3_5	P3.5
HAL_GPIO_P3_6	P3.6
HAL_GPIO_P3_7	P3.7
HAL_GPIO_P3_8	P3.8
HAL_GPIO_P3_9	P3.9
HAL_GPIO_P3_10	P3.10
HAL_GPIO_P3_11	P3.11
HAL_GPIO_P3_12	P3.12
HAL_GPIO_P3_13	P3.13
HAL_GPIO_P3_14	P3.14
HAL_GPIO_P3_15	P3.15
HAL_GPIO_P4_0	P4.0
HAL_GPIO_P4_1	P4.1
HAL_GPIO_P4_2	P4.2
HAL_GPIO_P4_3	P4.3
HAL_GPIO_P4_4	P4.4
HAL_GPIO_P4_5	P4.5
HAL_GPIO_P4_6	P4.6
HAL_GPIO_P4_7	P4.7
HAL_GPIO_P4_8	P4.8
HAL_GPIO_P4_9	P4.9
HAL_GPIO_P4_10	P4.10
HAL_GPIO_P4_11	P4.11
HAL_GPIO_P4_12	P4.12
HAL_GPIO_P4_13	P4.13
HAL_GPIO_P4_14	P4.14
HAL_GPIO_P4_15	P4.15

枚举值

HAL_GPIO_P5_0	P5.0
HAL_GPIO_P5_1	P5.1
HAL_GPIO_P5_2	P5.2
HAL_GPIO_P5_3	P5.3
HAL_GPIO_P5_4	P5.4
HAL_GPIO_P5_5	P5.5
HAL_GPIO_P5_6	P5.6
HAL_GPIO_P5_7	P5.7
HAL_GPIO_P5_8	P5.8
HAL_GPIO_P5_9	P5.9
HAL_GPIO_P5_10	P5.10
HAL_GPIO_P5_11	P5.11
HAL_GPIO_P5_12	P5.12
HAL_GPIO_P5_13	P5.13
HAL_GPIO_P5_14	P5.14
HAL_GPIO_P5_15	P5.15

在文件 [hal_lksmcu_gpio.h](#) 第 6 行定义.

4.27.2 函数说明

4.27.2.1 HAL_GPIO_Disable()

```
void HAL_GPIO_Disable (
    void )
```

禁用GPIO模块,关闭GPIO时钟和释放资源

在文件 [hal_lks32mc07x_gpio.c](#) 第 35 行定义.

4.27.2.2 HAL_GPIO_Enable()

```
void HAL_GPIO_Enable (
    void )
```

启用GPIO模块,使能GPIO时钟和必要的资源

在文件 [hal_lks32mc07x_gpio.c](#) 第 23 行定义.

4.27.2.3 HAL_GPIO_GetInData()

```
uint8_t HAL_GPIO_GetInData (
    HAL_GPIO_Px px )
```

获取当前指定GPIO端口的输入值

参数

<i>px</i>	指定的GPIO端口
-----------	-----------

返回

当前GPIO端口的输入值(0或1)

在文件 [hal_lks32mc07x_gpio.c](#) 第 177 行定义.

4.27.2.4 HAL_GPIO_GetOutData()

```
uint8_t HAL_GPIO_GetOutData (
    HAL_GPIO_Px px )
```

获取当前指定GPIO端口的输出值

参数

<i>px</i>	指定的GPIO端口
-----------	-----------

返回

当前GPIO端口的输出值(0或1)

在文件 [hal_lks32mc07x_gpio.c](#) 第 155 行定义.

4.27.2.5 HAL_GPIO_Invert()

```
void HAL_GPIO_Invert (
    HAL_GPIO_Px px )
```

翻转指定的GPIO端口的输出状态

参数

<i>px</i>	指定的GPIO端口
-----------	-----------

在文件 [hal_lks32mc07x_gpio.c](#) 第 143 行定义.

这是这个函数的调用关系图:



4.27.2.6 HAL_GPIO_Reset()

```
void HAL_GPIO_Reset (
    void )
```

模块复位,执行后恢复所有的GPIO为悬空状态(SWD、JTAG和RST口为默认状态)

在文件 [hal_lks32mc07x_gpio.c](#) 第 9 行定义.

4.27.2.7 HAL_GPIO_RstReuseGpioDisable()

```
void HAL_GPIO_RstReuseGpioDisable (
    void )
```

将RST接口恢复为RST功能

在文件 [hal_lks32mc07x_gpio.c](#) 第 295 行定义.

4.27.2.8 HAL_GPIO_RstReuseGpioEnable()

```
void HAL_GPIO_RstReuseGpioEnable (
    void )
```

将RST接口配置为普通GPIO口

在文件 [hal_lks32mc07x_gpio.c](#) 第 283 行定义.

4.27.2.9 HAL_GPIO_SetAf()

```
void HAL_GPIO_SetAf (
    HAL_GPIO_Px px,
    HAL_GPIO_Afx af )
```

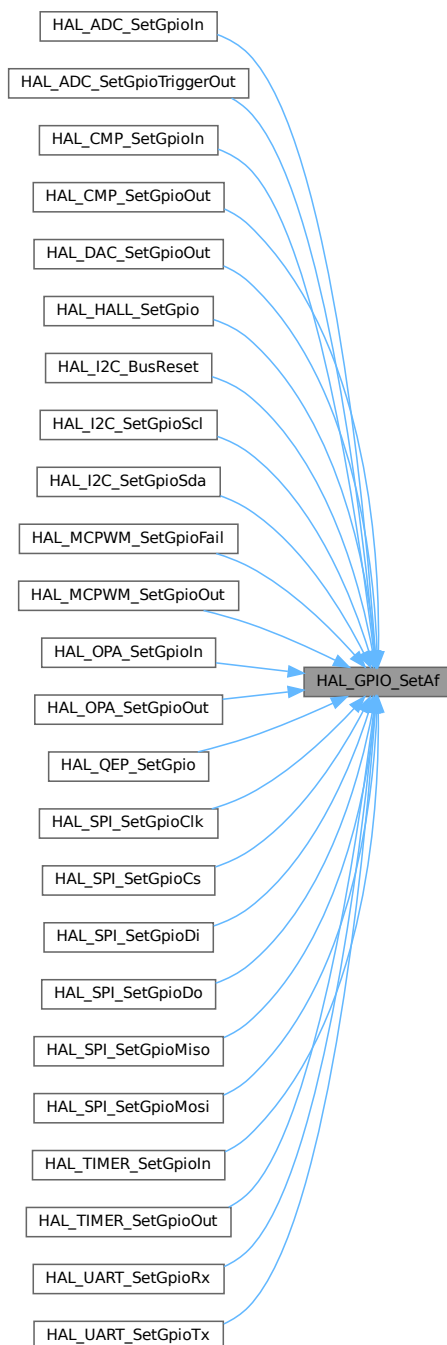
配置GPIO的特殊功能,如UART、I2C、SPI等

参数

<i>px</i>	指定的GPIO端口
<i>af</i>	要配置的特殊功能代码

在文件 [hal_lks32mc07x_gpio.c](#) 第 199 行定义。

这是这个函数的调用关系图：



4.27.2.10 HAL_GPIO_SetHigh()

```
void HAL_GPIO_SetHigh (
    HAL_GPIO_Px px )
```

将指定的GPIO端口输出高电平(1)

参数

<i>px</i>	指定的GPIO端口
-----------	-----------

注解

在INOUT模式下设为悬空,INUPOUT模式下设为上拉

在文件 [hal_lks32mc07x_gpio.c](#) 第 108 行定义.

4.27.2.11 HAL_GPIO_SetLow()

```
void HAL_GPIO_SetLow (
    HAL_GPIO_Px px )
```

将指定的GPIO端口输出低电平(0)

参数

<i>px</i>	指定的GPIO端口
-----------	-----------

在文件 [hal_lks32mc07x_gpio.c](#) 第 119 行定义.

4.27.2.12 HAL_GPIO_SetMode()

```
void HAL_GPIO_SetMode (
    HAL_GPIO_Px px,
    HAL_GPIO_MODE_x mode )
```

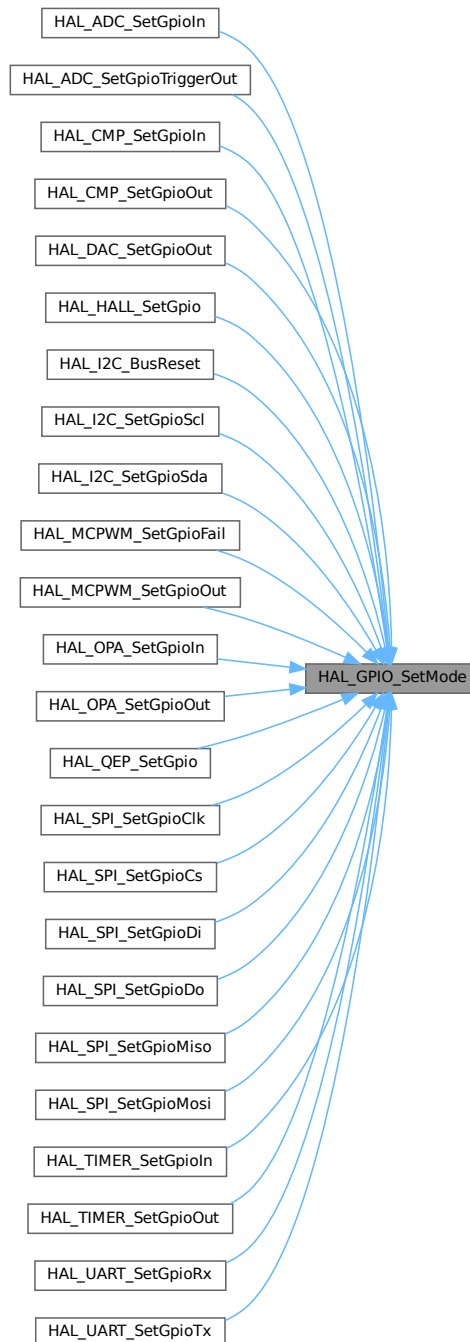
设置指定GPIO的工作模式

参数

<i>px</i>	指定的GPIO端口
<i>mode</i>	要设置的工作模式

在文件 [hal_lks32mc07x_gpio.c](#) 第 49 行定义.

这是这个函数的调用关系图:



4.27.2.13 HAL_GPIO_SwdReuseGpioDisable()

```
void HAL_GPIO_SwdReuseGpioDisable (
    void )
```

将SWD接口恢复为SWD功能

在文件 [hal_lks32mc07x_gpio.c](#) 第 271 行定义.

4.27.2.14 HAL_GPIO_SwdReuseGpioEnable()

```
void HAL_GPIO_SwdReuseGpioEnable (
    void )
```

将SWD接口配置为普通GPIO口

在文件 [hal_lks32mc07x_gpio.c](#) 第 259 行定义.

4.27.2.15 HAL_GPIO_Write()

```
void HAL_GPIO_Write (
    HAL_GPIO_Px px,
    uint8_t value )
```

向指定的GPIO端口写入0或1

参数

<i>px</i>	指定的GPIO端口
<i>value</i>	要写入的值(0或1)

在文件 [hal_lks32mc07x_gpio.c](#) 第 131 行定义.

4.28 hal_lksmcu_gpio.h

[浏览该文件的文档.](#)

```
00001 #ifndef __HAL_LKSMCU_GPIO_H__
00002 #define __HAL_LKSMCU_GPIO_H__
00003 #include "basic.h"
00004 #include "stdint.h"
00006 typedef enum
00007 {
00008     HAL_GPIO_P0_0 = 0x0000,
00009     HAL_GPIO_P0_1 = 0x0001,
00010     HAL_GPIO_P0_2 = 0x0002,
00011     HAL_GPIO_P0_3 = 0x0003,
00012     HAL_GPIO_P0_4 = 0x0004,
00013     HAL_GPIO_P0_5 = 0x0005,
00014     HAL_GPIO_P0_6 = 0x0006,
00015     HAL_GPIO_P0_7 = 0x0007,
00016     HAL_GPIO_P0_8 = 0x0008,
00017     HAL_GPIO_P0_9 = 0x0009,
00018     HAL_GPIO_P0_10 = 0x000a,
00019     HAL_GPIO_P0_11 = 0x000b,
00020     HAL_GPIO_P0_12 = 0x000c,
00021     HAL_GPIO_P0_13 = 0x000d,
00022     HAL_GPIO_P0_14 = 0x000e,
00023     HAL_GPIO_P0_15 = 0x000f,
00024     HAL_GPIO_P1_0 = 0x0100,
00025     HAL_GPIO_P1_1 = 0x0101,
00026     HAL_GPIO_P1_2 = 0x0102,
00027     HAL_GPIO_P1_3 = 0x0103,
00028     HAL_GPIO_P1_4 = 0x0104,
00029     HAL_GPIO_P1_5 = 0x0105,
00030     HAL_GPIO_P1_6 = 0x0106,
00031     HAL_GPIO_P1_7 = 0x0107,
00032     HAL_GPIO_P1_8 = 0x0108,
00033     HAL_GPIO_P1_9 = 0x0109,
00034     HAL_GPIO_P1_10 = 0x010a,
00035     HAL_GPIO_P1_11 = 0x010b,
00036     HAL_GPIO_P1_12 = 0x010c,
00037     HAL_GPIO_P1_13 = 0x010d,
```

```

00038     HAL_GPIO_P1_14 = 0x010e,
00039     HAL_GPIO_P1_15 = 0x010f,
00040     HAL_GPIO_P2_0  = 0x0200,
00041     HAL_GPIO_P2_1  = 0x0201,
00042     HAL_GPIO_P2_2  = 0x0202,
00043     HAL_GPIO_P2_3  = 0x0203,
00044     HAL_GPIO_P2_4  = 0x0204,
00045     HAL_GPIO_P2_5  = 0x0205,
00046     HAL_GPIO_P2_6  = 0x0206,
00047     HAL_GPIO_P2_7  = 0x0207,
00048     HAL_GPIO_P2_8  = 0x0208,
00049     HAL_GPIO_P2_9  = 0x0209,
00050     HAL_GPIO_P2_10 = 0x020a,
00051     HAL_GPIO_P2_11 = 0x020b,
00052     HAL_GPIO_P2_12 = 0x020c,
00053     HAL_GPIO_P2_13 = 0x020d,
00054     HAL_GPIO_P2_14 = 0x020e,
00055     HAL_GPIO_P2_15 = 0x020f,
00056     HAL_GPIO_P3_0  = 0x0300,
00057     HAL_GPIO_P3_1  = 0x0301,
00058     HAL_GPIO_P3_2  = 0x0302,
00059     HAL_GPIO_P3_3  = 0x0303,
00060     HAL_GPIO_P3_4  = 0x0304,
00061     HAL_GPIO_P3_5  = 0x0305,
00062     HAL_GPIO_P3_6  = 0x0306,
00063     HAL_GPIO_P3_7  = 0x0307,
00064     HAL_GPIO_P3_8  = 0x0308,
00065     HAL_GPIO_P3_9  = 0x0309,
00066     HAL_GPIO_P3_10 = 0x030a,
00067     HAL_GPIO_P3_11 = 0x030b,
00068     HAL_GPIO_P3_12 = 0x030c,
00069     HAL_GPIO_P3_13 = 0x030d,
00070     HAL_GPIO_P3_14 = 0x030e,
00071     HAL_GPIO_P3_15 = 0x030f,
00072     HAL_GPIO_P4_0  = 0x0400,
00073     HAL_GPIO_P4_1  = 0x0401,
00074     HAL_GPIO_P4_2  = 0x0402,
00075     HAL_GPIO_P4_3  = 0x0403,
00076     HAL_GPIO_P4_4  = 0x0404,
00077     HAL_GPIO_P4_5  = 0x0405,
00078     HAL_GPIO_P4_6  = 0x0406,
00079     HAL_GPIO_P4_7  = 0x0407,
00080     HAL_GPIO_P4_8  = 0x0408,
00081     HAL_GPIO_P4_9  = 0x0409,
00082     HAL_GPIO_P4_10 = 0x040a,
00083     HAL_GPIO_P4_11 = 0x040b,
00084     HAL_GPIO_P4_12 = 0x040c,
00085     HAL_GPIO_P4_13 = 0x040d,
00086     HAL_GPIO_P4_14 = 0x040e,
00087     HAL_GPIO_P4_15 = 0x040f,
00088     HAL_GPIO_P5_0  = 0x0500,
00089     HAL_GPIO_P5_1  = 0x0501,
00090     HAL_GPIO_P5_2  = 0x0502,
00091     HAL_GPIO_P5_3  = 0x0503,
00092     HAL_GPIO_P5_4  = 0x0504,
00093     HAL_GPIO_P5_5  = 0x0505,
00094     HAL_GPIO_P5_6  = 0x0506,
00095     HAL_GPIO_P5_7  = 0x0507,
00096     HAL_GPIO_P5_8  = 0x0508,
00097     HAL_GPIO_P5_9  = 0x0509,
00098     HAL_GPIO_P5_10 = 0x050a,
00099     HAL_GPIO_P5_11 = 0x050b,
00100     HAL_GPIO_P5_12 = 0x050c,
00101     HAL_GPIO_P5_13 = 0x050d,
00102     HAL_GPIO_P5_14 = 0x050e,
00103     HAL_GPIO_P5_15 = 0x050f,
00104 } HAL_GPIO_Px;
00105
00106 typedef enum
00107 {
00108     HAL_GPIO_AF_ADC      = 0, // ADC输入
00109     HAL_GPIO_AF_OPA      = 0, // 运放输入
00110     HAL_GPIO_AF_OPA_OUT  = 0, // 运放输出
00111     HAL_GPIO_AF_CMP      = 0, // 比较器输入
00112     HAL_GPIO_AF_DAC      = 0, // DAC输出,需要DAC同步设置DAC输出使能
00113     HAL_GPIO_AF_GPIO     = 0, // GPIO功能
00114     HAL_GPIO_AF_CLK      = 0, // 时钟输入
00115
00116     HAL_GPIO_AF_CMP_OUT  = 1, // 比较器比较输出
00117     HAL_GPIO_AF_CLK_OUT  = 1, // 时钟输出
00118
00119     HAL_GPIO_AF_HALL     = 2, // 霍尔输入
00120     HAL_GPIO_AF_MCPWM    = 3, // MCPWMFAIL输入 MCPWM输出
00121     HAL_GPIO_AF_UART     = 4, // UART输入输出
00122     HAL_GPIO_AF_SPI      = 5, // CS CLK MOSI MISO
00123     HAL_GPIO_AF_I2C      = 6, // SCLâ€™ESDA
00124     HAL_GPIO_AF_TIMER0   = 7, // TIMER0输入或输出

```

```

00125     HAL_GPIO_AF_TIMER1 = 7, // TIMER1输入或输出
00126     HAL_GPIO_AF_TIMER2 = 8, // TIMER2输入或输出
00127     HAL_GPIO_AF_TIMER3 = 8, // TIMER3输入或输出
00128     HAL_GPIO_AF_QEP    = 8, // 编码器输入
00129
00130     HAL_GPIO_AF_ADC_TRIG = 9, // adc触发信号输出
00131     HAL_GPIO_AF_CAN      = 10, // CAN输入输出
00132     HAL_GPIO_AF_SIF      = 11, // SIF信号输出
00133     HAL_GPIO_AF_CL       = 12, // CL输出
00134
00135     HAL_GPIO_AF_TIMER4 = 0, // TIMER4输入或输出 不支持 这里保留接口定义
00136     HAL_GPIO_AF_TIMER5 = 0, // TIMER5输入或输出 不支持 这里保留接口定义
00137 } HAL_GPIO_Afx;
00148 typedef enum
00149 {
00150     HAL_GPIO_MODE_AFE          = 0,
00151     HAL_GPIO_MODE_NONE        = HAL_GPIO_MODE_AFE,
00152     HAL_GPIO_MODE_IN          = BIT0,
00153     HAL_GPIO_MODE_INUP        = HAL_GPIO_MODE_IN | BIT2,
00154     HAL_GPIO_MODE_OUT         = BIT1,
00155     HAL_GPIO_MODE_INOUT       = HAL_GPIO_MODE_IN | HAL_GPIO_MODE_OUT | BIT3,
00156     HAL_GPIO_MODE_INUPOUT     = HAL_GPIO_MODE_INOUT | BIT2 | BIT3,
00157     HAL_GPIO_MODE_OUT_0       = HAL_GPIO_MODE_OUT | BIT4,
00158     HAL_GPIO_MODE_INOUT_0     = HAL_GPIO_MODE_INOUT | BIT4,
00159     HAL_GPIO_MODE_INUPOUT_0   = HAL_GPIO_MODE_INUPOUT | BIT4,
00160     HAL_GPIO_MODE_OUT_1       = HAL_GPIO_MODE_OUT | BIT5,
00161     HAL_GPIO_MODE_INOUT_1     = HAL_GPIO_MODE_INOUT | BIT5,
00162     HAL_GPIO_MODE_INUPOUT_1   = HAL_GPIO_MODE_INUPOUT | BIT5,
00163 } HAL_GPIO_MODE_x;
00164 // 模块初始化
00165 void HAL_GPIO_Reset(void); // 模块复位, 执行后恢复所有的GPIO为悬空状态(SWD、JTAG和RST口为默认状态)
00166 void HAL_GPIO_Enable(void); // 模块打开
00167 void HAL_GPIO_Disable(void); // 模块关闭
00168
00169 // 输入输出
00170 void HAL_GPIO_SetMode(HAL_GPIO_Px, HAL_GPIO_MODE_x); // 设置指定GPIO的工作模式
00171 void HAL_GPIO_SetHigh(HAL_GPIO_Px); // 输出1 (INOUT和INUPOUT模式下分别为悬空和上拉)
00172 void HAL_GPIO_SetLow(HAL_GPIO_Px); // 输出0
00173 void HAL_GPIO_Write(HAL_GPIO_Px, uint8_t); // 输出0或1
00174 void HAL_GPIO_Invert(HAL_GPIO_Px); // 翻转指定的GPIO
00175 uint8_t HAL_GPIO_GetOutData(HAL_GPIO_Px); // 获取当前GPIO的输出值, 返回值0或1
00176 uint8_t HAL_GPIO_GetInData(HAL_GPIO_Px); // 获取当前GPIO的输入值, 返回值0或1
00177
00178 // 内部使用
00179 void HAL_GPIO_SetAf(HAL_GPIO_Px, HAL_GPIO_Afx); // 配置GPIO的特殊功能(UART、I2C、SPI等, 由对应模块的初始化调用)
00180 // 特殊GPIO口配置
00181 void HAL_GPIO_SwdReuseGpioEnable(void); // 设置SWD接口为普通GPIO口
00182 void HAL_GPIO_SwdReuseGpioDisable(void); // 设置SWD接口为SWD功能
00183 void HAL_GPIO_RstReuseGpioEnable(void); // 设置RST接口为普通GPIO口
00184 void HAL_GPIO_RstReuseGpioDisable(void); // 设置RST接口为RST功能
00185 #endif

```

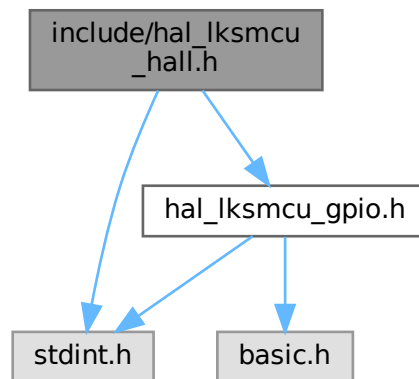
4.29 include/hal_lksmcu_hall.h 文件参考

```

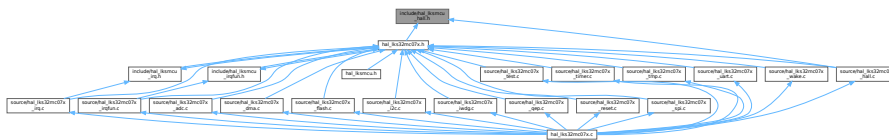
#include "stdint.h"
#include "hal_lksmcu_gpio.h"

```

hal_ksmcu_hall.h 的引用(Include)关系图:



此图展示该文件直接或间接的被哪些文件引用了:



枚举

- enum HAL_HALL_x { HAL_HALL_0 , HAL_HALL_1 }

函数

- void `HAL_HALL_Reset` (`HAL_HALL_x`)
模块复位
- void `HAL_HALL_Enable` (`HAL_HALL_x`)
模块打开
- void `HAL_HALL_Disable` (`HAL_HALL_x`)
模块关闭
- void `HAL_HALL_SetFreq` (`HAL_HALL_x`, `uint32_t freq`)
设置*hall*频率
- void `HAL_HALL_SetTh` (`HAL_HALL_x`, `uint32_t th`)
设置*hall*阈值
- void `HAL_HALL_StartCounting` (`HAL_HALL_x`)
开始计数 *HAL_HALL_Enable*后默认开始计数
- void `HAL_HALL_StopCounting` (`HAL_HALL_x`)
停止计数(同时计数器回到0)
- void `HAL_HALL_SetFilter` (`HAL_HALL_x`, `uint32_t filter_mcklk`)

- 聰剧疆婁ゆ尝
- void HAL_HALL_SetFilteruS (HAL_HALL_x, uint32_t filter_uS)
设置滤波，单位：uS
 - uint32_t HAL_HALL_ReadTh (HAL_HALL_x)
获取当前hall的周期计数值
 - uint8_t HAL_HALL_ReadHallData (HAL_HALL_x)
获取当前hall的数据
 - uint32_t HAL_HALL_ReadHallWidth (HAL_HALL_x)
获取当前hall的宽度
 - void HAL_HALL_SetGpio (HAL_HALL_x, HAL_GPIO_Px)
设置hall输入引脚

4.29.1 枚举类型说明

4.29.1.1 HAL_HALL_x

```
enum HAL_HALL_x
```

枚举值

HAL↔ HALL_0	
HAL↔ HALL_1	

在文件 [hal_lksmcu_hall.h](#) 第 5 行定义.

4.29.2 函数说明

4.29.2.1 HAL_HALL_Disable()

```
void HAL_HALL_Disable (  
    HAL_HALL_x hall )
```

模块关闭

参数

hall	模块号
------	-----

在文件 [hal_lks32mc07x_hall.c](#) 第 39 行定义.

4.29.2.2 HAL_HALL_Enable()

```
void HAL_HALL_Enable (  
    HAL_HALL_x hall )
```

模块打开

参数

<i>hall</i>	模块号
-------------	-----

在文件 [hal_lks32mc07x.hall.c](#) 第 27 行定义.

4.29.2.3 HAL_HALL_ReadHallData()

```
uint8_t HAL_HALL_ReadHallData (
    HAL_HALL_x hall )
```

获取当前hall的数据

参数

<i>hall</i>	模块号
-------------	-----

返回

数据值

在文件 [hal_lks32mc07x.hall.c](#) 第 84 行定义.

4.29.2.4 HAL_HALL_ReadHallWidth()

```
uint32_t HAL_HALL_ReadHallWidth (
    HAL_HALL_x hall )
```

获取当前hall的宽度

参数

<i>hall</i>	模块号
-------------	-----

返回

宽度值

在文件 [hal_lks32mc07x.hall.c](#) 第 94 行定义.

4.29.2.5 HAL_HALL_ReadTh()

```
uint32_t HAL_HALL_ReadTh (
    HAL_HALL_x hall )
```

获取当前hall的周期计数值

参数

<i>hall</i>	模块号
-------------	-----

返回

周期计数值

在文件 [hal_lks32mc07x.hall.c](#) 第 74 行定义.

4.29.2.6 HAL_HALL_Reset()

```
void HAL_HALL_Reset (
    HAL_HALL_x hall )
```

模块复位

参数

<i>hall</i>	模块号
-------------	-----

在文件 [hal_lks32mc07x.hall.c](#) 第 12 行定义.

4.29.2.7 HAL_HALL_SetFilter()

```
void HAL_HALL_SetFilter (
    HAL_HALL_x hall,
    uint32_t filter_mclk )
```

璁剧疆婊0尝

参数

<i>hall</i>	模块号
<i>filter_mclk</i>	滤波值

在文件 [hal_lks32mc07x.hall.c](#) 第 134 行定义.
这是这个函数的调用关系图:



4.29.2.8 HAL_HALL_SetFilteruS()

```
void HAL_HALL_SetFilteruS (
    HAL_HALL_x hall,
    uint32_t filter_uS )
```

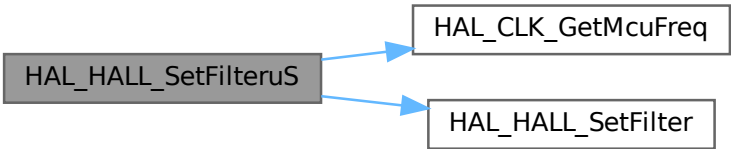
设置滤波，单位：uS

参数

hall	模块号
filter_uS	滤波值

在文件 [hal_lks32mc07x.hall.c](#) 第 166 行定义.

函数调用图:



4.29.2.9 HAL_HALL_SetFreq()

```
void HAL_HALL_SetFreq (
    HAL_HALL_x hall,
    uint32_t freq )
```

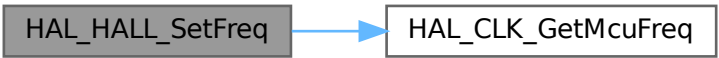
设置hall频率

参数

hall	模块号
freq	频率值

在文件 [hal_lks32mc07x.hall.c](#) 第 52 行定义.

函数调用图:



4.29.2.10 HAL_HALL_SetGpio()

```
void HAL_HALL_SetGpio (
    HAL_HALL_x hall,
    HAL_GPIO_Px px )
```

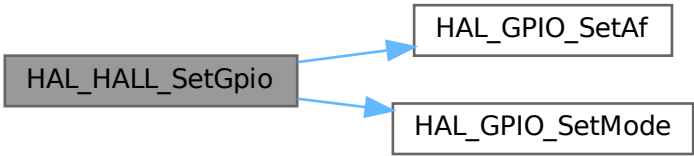
设置hall输入引脚

参数

hall	模块号
px	引脚

在文件 [hal_lks32mc07x_hall.c](#) 第 104 行定义.

函数调用图:



4.29.2.11 HAL_HALL_SetTh()

```
void HAL_HALL_SetTh (
    HAL_HALL_x hall,
    uint32_t th )
```

设置hall阈值

参数

<i>hall</i>	模块号
<i>th</i>	阈值

在文件 [hal_lks32mc07x_hall.c](#) 第 64 行定义.

4.29.2.12 HAL_HALL_StartCounting()

```
void HAL_HALL_StartCounting (
    HAL_HALL_x hall )
```

开始计数 HAL_HALL_Enable后默认开始计数

参数

<i>hall</i>	模块号
-------------	-----

在文件 [hal_lks32mc07x_hall.c](#) 第 115 行定义.

4.29.2.13 HAL_HALL_StopCounting()

```
void HAL_HALL_StopCounting (
    HAL_HALL_x hall )
```

停止计数(同时计数器回到0)

参数

<i>hall</i>	模块号
-------------	-----

在文件 [hal_lks32mc07x_hall.c](#) 第 123 行定义.

4.30 hal_lksmcu_hall.h

[浏览该文件的文档.](#)

```
00001 #ifndef __HAL_LKSMCU_HALL_H__
00002 #define __HAL_LKSMCU_HALL_H__
00003 #include "stdint.h"
00004 #include "hal_lksmcu_gpio.h"
00005 typedef enum
00006 {
00007     HAL_HALL_0,
00008     HAL_HALL_1,
00009 } HAL_HALL_x;
00010 // 初始化
00011 void HAL_HALL_Reset(HAL_HALL_x); // 模块复位
00012 void HAL_HALL_Enable(HAL_HALL_x); // 模块打开
00013 void HAL_HALL_Disable(HAL_HALL_x); // 模块关闭
00014 //
00015 void HAL_HALL_SetFreq(HAL_HALL_x, uint32_t freq); // 设置hall频率 SetFreq和SetTh配置其中一个即可
00016 void HAL_HALL_SetTh(HAL_HALL_x, uint32_t th); // 设置hall阈值 SetFreq和SetTh配置其中一个即可
```

```

00017 void HAL_HALL_StartCounting(HAL_HALL_x); // 开始计数 HAL_HALL_Enable后默认开始计数
00018 void HAL_HALL_StopCounting(HAL_HALL_x); // 停止计数(同时计数器回到0)
00019 void HAL_HALL_SetFilter(HAL_HALL_x, uint32_t filter_mclk); // 设置滤波, 单位: MCU时钟
00020 void HAL_HALL_SetFilteruS(HAL_HALL_x, uint32_t filter_uS); // 设置滤波, 单位: uS
00021
00022 uint32_t HAL_HALL_ReadTh(HAL_HALL_x); // 获取当前hall的周期计数值
00023 uint8_t HAL_HALL_ReadHallData(HAL_HALL_x); // 获取当前hall的数据
00024 uint32_t HAL_HALL_ReadHallWidth(HAL_HALL_x); // 获取当前hall捕获到的宽度
00025
00026 void HAL_HALL_SetGpio(HAL_HALL_x, HAL_GPIO_Px); // 设置hall输入引脚
00027 #endif

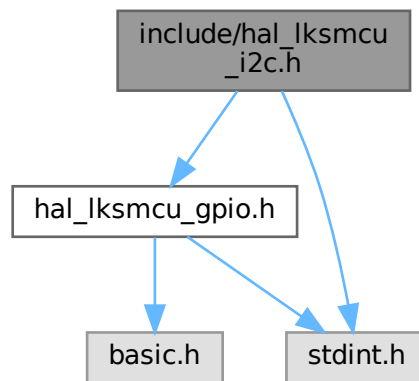
```

4.31 include/hal_lksmcu_i2c.h 文件参考

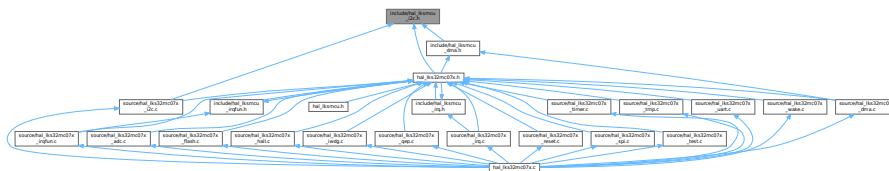
```
#include "hal_lksmcu_gpio.h"
```

```
#include "stdint.h"
```

hal_lksmcu_i2c.h 的引用(Include)关系图:



此图展示该文件直接或间接的被哪些文件引用了:



枚举

- enum HAL_I2C_x { HAL_I2C_0, HAL_I2C_1 }

函数

- void [HAL_I2C_Reset \(HAL_I2C_x\)](#)
复位I2C模块
- void [HAL_I2C_Enable \(HAL_I2C_x\)](#)
打开I2C模块
- void [HAL_I2C_Disable \(HAL_I2C_x\)](#)
关闭I2C模块
- void [HAL_I2C_SetBaud \(HAL_I2C_x, uint32_t baud\)](#)
设置I2C模块波特率
- uint8_t [HAL_I2C_TxData \(HAL_I2C_x, uint8_t addr, uint8_t *data, uint32_t len\)](#)
发送数据到I2C设备
- uint8_t [HAL_I2C_RxData \(HAL_I2C_x, uint8_t addr, uint8_t *data, uint32_t len\)](#)
从I2C设备接收数据
- void [HAL_I2C_SetGpioScl \(HAL_I2C_x, HAL_GPIO_Px i2cgpio\)](#)
设置I2C模块的GPIO
- void [HAL_I2C_SetGpioSda \(HAL_I2C_x, HAL_GPIO_Px i2cgpio\)](#)
设置I2C模块的GPIO
- void [HAL_I2C_BusReset \(HAL_I2C_x\)](#)
复位I2C总线

4.31.1 枚举类型说明

4.31.1.1 HAL_I2C_x

enum [HAL_I2C_x](#)

枚举值

HAL_I2C↔ C_0	
HAL_I2C↔ C_1	

在文件 [hal_lksmcu_i2c.h](#) 第 5 行定义.

4.31.2 函数说明

4.31.2.1 HAL_I2C_BusReset()

```
void HAL_I2C_BusReset (
    HAL\_I2C\_x i2c_x )
```

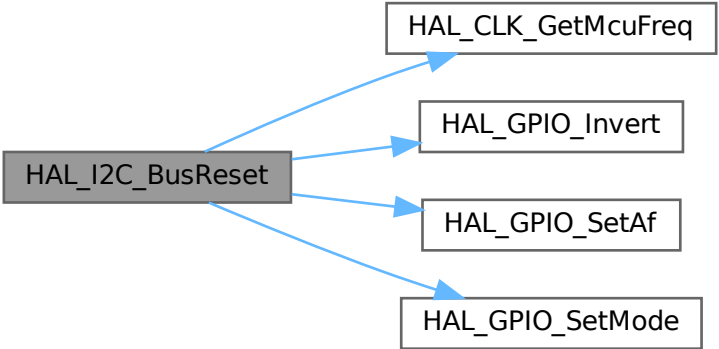
复位I2C总线

参数

$i2c_x$	I2C模块编号
----------	---------

在文件 [hal_lks32mc07x.i2c.c](#) 第 202 行定义.

函数调用图:



4.31.2.2 HAL_I2C.Disable()

```
void HAL_I2C_Disable (
    HAL_I2C_x i2c_x )
```

关闭I2C模块

参数

<i>i2c_x</i>	I2C模块编号
--------------	---------

在文件 [hal_lks32mc07x.i2c.c](#) 第 38 行定义.

4.31.2.3 HAL_I2C.Enable()

```
void HAL_I2C_Enable (
    HAL_I2C_x i2c_x )
```

打开I2C模块

参数

<i>i2c_x</i>	I2C模块编号
--------------	---------

在文件 [hal_lks32mc07x.i2c.c](#) 第 26 行定义.

4.31.2.4 HAL_I2C_Reset()

```
void HAL_I2C_Reset (
    HAL_I2C_TypeDef* i2c_x )
```

复位I2C模块

参数

<i>i2c_x</i>	I2C模块编号
--------------	---------

在文件 [hal_lks32mc07x_i2c.c](#) 第 13 行定义.

4.31.2.5 HAL_I2C_RxData()

```
uint8_t HAL_I2C_RxData (
    HAL_I2C_TypeDef* i2c_x,
    uint8_t* addr,
    uint8_t* data,
    uint32_t len )
```

从I2C设备接收数据

参数

<i>i2c_x</i>	I2C模块编号
<i>addr</i>	设备地址
<i>data</i>	数据缓冲区
<i>len</i>	数据长度

返回

0:成功 非0:失败

在文件 [hal_lks32mc07x_i2c.c](#) 第 130 行定义.

4.31.2.6 HAL_I2C_SetBaud()

```
void HAL_I2C_SetBaud (
    HAL_I2C_TypeDef* i2c_x,
    uint32_t baud )
```

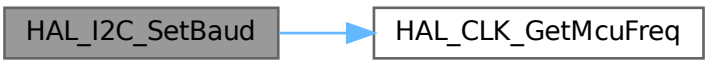
设置I2C模块波特率

参数

<i>i2c_x</i>	I2C模块编号
<i>baud</i>	波特率

在文件 [hal_lks32mc07x.i2c.c](#) 第 50 行定义.

函数调用图:



4.31.2.7 HAL_I2C_SetGpioScl()

```
void HAL_I2C_SetGpioScl (
    HAL_I2C_x i2c_x,
    HAL_GPIO_Px i2cgpio )
```

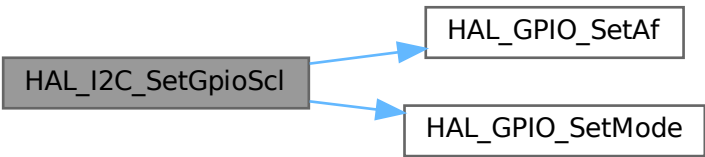
设置I2C模块的GPIO

参数

<i>i2c_x</i>	I2C模块编号
<i>i2cgpio</i>	GPIO编号

在文件 [hal_lks32mc07x.i2c.c](#) 第 179 行定义.

函数调用图:



4.31.2.8 HAL_I2C_SetGpioSda()

```
void HAL_I2C_SetGpioSda (
    HAL_I2C_x i2c_x,
    HAL_GPIO_Px i2cgpio )
```

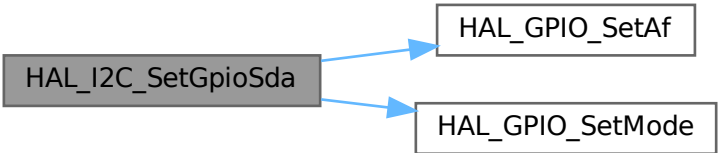
设置I2C模块的GPIO

参数

<i>i2c_x</i>	I2C模块编号
<i>i2cgpio</i>	GPIO编号

在文件 [hal_lks32mc07x_i2c.c](#) 第 191 行定义.

函数调用图:



4.31.2.9 HAL_I2C_TxData()

```
uint8_t HAL_I2C_TxData (
    HAL_I2C_x i2c_x,
    uint8_t addr,
    uint8_t * data,
    uint32_t len )
```

发送数据到I2C设备

参数

<i>i2c_x</i>	I2C模块编号
<i>addr</i>	设备地址
<i>data</i>	数据缓冲区
<i>len</i>	数据长度

返回

0:成功 非0:失败

在文件 `hal_iks32mc07x_i2c.c` 第 69 行定义.

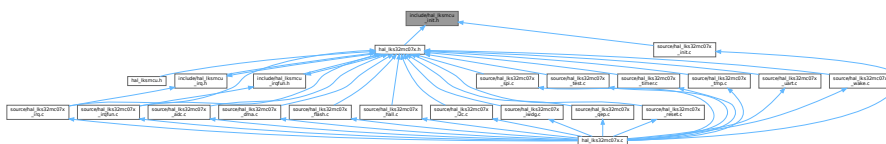
4.32 hal_ksmcu_i2c.h

[浏览该文件的文档.](#)

```
00001 #ifndef __HAL_LKSMCU_I2C_H__
00002 #define __HAL_LKSMCU_I2C_H__
00003 #include "hal_lksmcu_gpio.h"
00004 #include "stdint.h"
00005 typedef enum
00006 {
00007     HAL_I2C_0,
00008     HAL_I2C_1,
00009 } HAL_I2C_x;
00010 // 初始化
00011 void HAL_I2C_Reset(HAL_I2C_x); // 模块复位
00012 void HAL_I2C_Enable(HAL_I2C_x); // 模块打开
00013 void HAL_I2C_Disable(HAL_I2C_x); // 模块关闭
00014 //
00015 void HAL_I2C_SetBaud(HAL_I2C_x, uint32_t baud);
00016 uint8_t HAL_I2C_TxData(HAL_I2C_x, uint8_t addr, uint8_t *data, uint32_t len);
00017 uint8_t HAL_I2C_RxData(HAL_I2C_x, uint8_t addr, uint8_t *data, uint32_t len);
00018
00019 void HAL_I2C_SetGpioScl(HAL_I2C_x, HAL_GPIO_Px i2cgpio);
00020 void HAL_I2C_SetGpioSda(HAL_I2C_x, HAL_GPIO_Px i2cgpio);
00021
00022 void HAL_I2C_BusReset(HAL_I2C_x); // 复位总线
00023
00024 #endif
```

4.33 include/hal_lksmcu_init.h 文件参考

此图展示该文件直接或间接的被哪些文件引用了:



枚举

- enum HAL_INIT_POWER_x { HAL_INIT_POWER_3.3V = 0 , HAL_INIT_POWER_5V }

函数

- void HAL_INIT_Sysinit (void)
- void HAL_INIT_PowerReady (HAL_INIT_POWER_x powerx)

4.33.1 枚举类型说明

4.33.1.1 HAL_INIT_POWER_x

```
enum HAL_INIT_POWER_x
```

枚举值

HAL_INIT_POWER_3_3V	
HAL_INIT_POWER_5V	

在文件 [hal_lksmcu_init.h](#) 第 3 行定义.

4.33.2 函数说明

4.33.2.1 HAL_INIT_PowerReady()

```
void HAL_INIT_PowerReady (
    HAL_INIT_POWER_x powerx )
```

在文件 [hal_lks32mc07x_init.c](#) 第 14 行定义.

4.33.2.2 HAL_INIT_SysInit()

```
void HAL_INIT_SysInit (
    void )
```

在文件 [hal_lks32mc07x_init.c](#) 第 6 行定义.

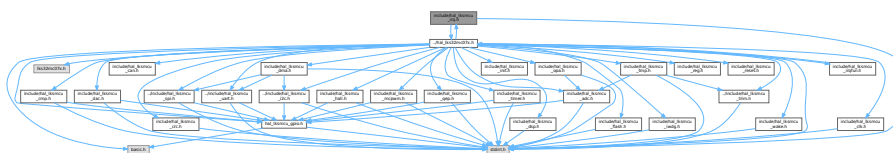
4.34 hal_lksmcu_init.h

[浏览该文件的文档.](#)

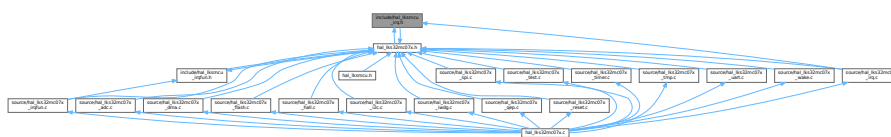
```
00001 #ifndef __HAL_LKSMCU_INIT_H__
00002 #define __HAL_LKSMCU_INIT_H__
00003 typedef enum
00004 {
00005     HAL_INIT_POWER_3_3V = 0,
00006     HAL_INIT_POWER_5V,
00007 } HAL_INIT_POWER_x;
00008 void HAL_INIT_SysInit(void);
00009 void HAL_INIT_PowerReady(HAL_INIT_POWER_x powerx);
00010
00011 #endif
```

4.35 include/hal_lksmcu_irq.h 文件参考

```
#include "stdint.h"
#include "../hal_lks32mc07x.h"
hal_lksmcu_irq.h 的引用(Include)关系图:
```



此图展示该文件直接或间接的被哪些文件引用了:



枚举

- enum **HAL_IRQ_MODULE_x** {
HAL_IRQ_MODULE_NonMaskableInt = NonMaskableInt_IRQn , **HAL_IRQ_MODULE_HardFault** = HardFault_IRQn , **HAL_IRQ_MODULE_SVCALL** = SVCALL_IRQn , **HAL_IRQ_MODULE_PendSV** = PendSV_IRQn ,
HAL_IRQ_MODULE_SysTick = SysTick_IRQn , **HAL_IRQ_MODULE_TIMER_00** = TIMER0_IRQn ,
HAL_IRQ_MODULE_TIMER_10 = TIMER1_IRQn , **HAL_IRQ_MODULE_TIMER_20** = TIMER2_IRQn ,
HAL_IRQ_MODULE_MCPWM_00 = MCPWM0_IRQn , **HAL_IRQ_MODULE_MCPWM_01** = MCPWM1_IRQn ,
HAL_IRQ_MODULE_I2C_00 = I2C0_IRQn , **HAL_IRQ_MODULE_SPI_00** = SPI0_IRQn ,
HAL_IRQ_MODULE_GPIO_00 = GPIO_IRQn , **HAL_IRQ_MODULE_HALL_00** = HALL0_IRQn , **HAL_IRQ_MODULE_UART_00** = UART0_IRQn , **HAL_IRQ_MODULE_UART_10** = UART1_IRQn ,
HAL_IRQ_MODULE_CMP_00 = CMP_IRQn , **HAL_IRQ_MODULE_ADC_00** = ADC0_IRQn , **HAL_IRQ_MODULE_ADC_10** = ADC1_IRQn , **HAL_IRQ_MODULE_WAKE_00** = WAKE_IRQn ,
HAL_IRQ_MODULE_QEP_00 = QEP0_IRQn , **HAL_IRQ_MODULE_DMA_00** = DMA0_IRQn , **HAL_IRQ_MODULE_SIF_00** = SIF0_IRQn , **HAL_IRQ_MODULE_SW_00** = SW_IRQn ,
HAL_IRQ_MODULE_INIT_00 = PWRDN_IRQn , **HAL_IRQ_MODULE_CL_00** = CL0_IRQn , **HAL_IRQ_MODULE_UART_20** , **HAL_IRQ_MODULE_CMP_10** }
 各个模块的中断编号 如果调用不存在中断的模块,则不做任何操作

函数

- void **HAL_IRQ_Reset** (void)
重置中断(等效于关闭中断)
- void **HAL_IRQ_Enable** (void)
使能全局中断
- void **HAL_IRQ_Disable** (void)
关闭全局中断
- void **HAL_IRQ_ModuleEnable** (HAL_IRQ_MODULE_x, uint8_t priority)
使能某个模块的中断
- void **HAL_IRQ_ModuleDisable** (HAL_IRQ_MODULE_x)
关闭某个模块的中断

4.35.1 枚举类型说明

4.35.1.1 HAL_IRQ_MODULE_x

enum **HAL_IRQ_MODULE_x**

各个模块的中断编号 如果调用不存在中断的模块,则不做任何操作

枚举值

HAL_IRQ_MODULE_NonMaskableInt

枚举值

HAL_IRQ_MODULE_HardFault	
HAL_IRQ_MODULE_SVCall	
HAL_IRQ_MODULE_PendSV	
HAL_IRQ_MODULE_SysTick	
HAL_IRQ_MODULE_TIMER_00	
HAL_IRQ_MODULE_TIMER_10	
HAL_IRQ_MODULE_TIMER_20	
HAL_IRQ_MODULE_MCPWM_00	
HAL_IRQ_MODULE_MCPWM_01	
HAL_IRQ_MODULE_I2C_00	
HAL_IRQ_MODULE_SPI_00	
HAL_IRQ_MODULE_GPIO_00	
HAL_IRQ_MODULE_HALL_00	
HAL_IRQ_MODULE_UART_00	
HAL_IRQ_MODULE_UART_10	
HAL_IRQ_MODULE_CMP_00	
HAL_IRQ_MODULE_ADC_00	
HAL_IRQ_MODULE_ADC_10	
HAL_IRQ_MODULE_WAKE_00	
HAL_IRQ_MODULE_QEP_00	
HAL_IRQ_MODULE_DMA_00	
HAL_IRQ_MODULE_SIF_00	
HAL_IRQ_MODULE_SW_00	
HAL_IRQ_MODULE_INIT_00	
HAL_IRQ_MODULE_CL_00	
HAL_IRQ_MODULE_UART_20	
HAL_IRQ_MODULE_CMP_10	

在文件 [hal_lksmcu_irq.h](#) 第 11 行定义.

4.35.2 函数说明

4.35.2.1 HAL_IRQ_Disable()

```
void HAL_IRQ_Disable (
    void )
```

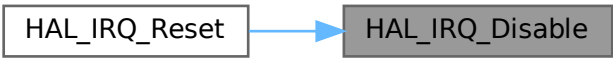
关闭全局中断

参数

<i>irq</i>	中断编号
------------	------

在文件 [hal_lks32mc07x_irq.c](#) 第 29 行定义.

这是这个函数的调用关系图:



4.35.2.2 HAL_IRQ_Enable()

```
void HAL_IRQ_Enable (
    void )
```

使能全局中断

参数

<i>irq</i>	中断编号
------------	------

在文件 [hal_lks32mc07x_irq.c](#) 第 20 行定义.

4.35.2.3 HAL_IRQ_ModuleDisable()

```
void HAL_IRQ_ModuleDisable (
    HAL_IRQ_MODULE_# module )
```

关闭某个模块的中断

参数

<i>irq</i>	中断编号
<i>module</i>	模块编号
<i>number</i>	中断号

在文件 [hal_lks32mc07x_irq.c](#) 第 52 行定义.

4.35.2.4 HAL_IRQ_ModuleEnable()

```
void HAL_IRQ_ModuleEnable (
    HAL_IRQ_MODULE_# module,
    uint8_t priority )
```

使能某个模块的中断

参数

<i>irq</i>	中断编号
<i>module</i>	模块编号
<i>number</i>	中断号
<i>priority</i>	中断优先级

在文件 [hal_lks32mc07x_irq.c](#) 第 41 行定义.

4.35.2.5 HAL_IRQ_Reset()

```
void HAL_IRQ_Reset (
    void )
```

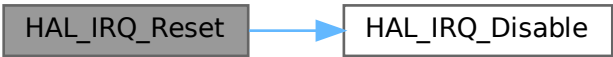
重置中断(等效于关闭中断)

参数

<i>irq</i>	中断编号
------------	------

在文件 [hal_lks32mc07x_irq.c](#) 第 11 行定义.

函数调用图:



4.36 hal_lksmcu_irq.h

[浏览该文件的文档.](#)

```
00001 #ifndef __HAL_LKSMCU_IRQ_H__
00002 #define __HAL_LKSMCU_IRQ_H__
00003
00004 #include "stdint.h"
00005 #include "../hal_lks32mc07x.h"
00006
00011 typedef enum
00012 {
00013     // 来自arm内核的模块
00014     HAL_IRQ_MODULE.NonMaskableInt = NonMaskableInt_IRQn,
00015     HAL_IRQ_MODULE.HardFault      = HardFault_IRQn,
00016     HAL_IRQ_MODULE.SVCall         = SVCall_IRQn,
00017     HAL_IRQ_MODULE.PendSV         = PendSV_IRQn,
00018     HAL_IRQ_MODULE.SysTick        = SysTick_IRQn,
00019     // 芯片外设
00020     HAL_IRQ_MODULE.TIMER_00       = TIMER0_IRQn,
00021     HAL_IRQ_MODULE.TIMER_10       = TIMER1_IRQn,
00022     HAL_IRQ_MODULE.TIMER_20       = TIMER2_IRQn,
```

```

00023 HAL_IRQ_MODULE_MCPWM_00 = MCPWM0_IRQn,
00024 HAL_IRQ_MODULE_MCPWM_01 = MCPWM1_IRQn,
00025 HAL_IRQ_MODULE_I2C_00 = I2C0_IRQn,
00026 HAL_IRQ_MODULE_SPI_00 = SPI0_IRQn,
00027 HAL_IRQ_MODULE_GPIO_00 = GPIO_IRQn,
00028 HAL_IRQ_MODULE_HALL_00 = HALL0_IRQn,
00029 HAL_IRQ_MODULE_UART_00 = UART0_IRQn,
00030 HAL_IRQ_MODULE_UART_10 = UART1_IRQn,
00031 HAL_IRQ_MODULE_CMP_00 = CMP_IRQn,
00032 HAL_IRQ_MODULE_ADC_00 = ADC0_IRQn,
00033 HAL_IRQ_MODULE_ADC_10 = ADC1_IRQn,
00034 HAL_IRQ_MODULE_WAKE_00 = WAKE_IRQn,
00035 HAL_IRQ_MODULE_QEP_00 = QEP0_IRQn,
00036 HAL_IRQ_MODULE_DMA_00 = DMA0_IRQn,
00037 HAL_IRQ_MODULE_SIF_00 = SIF0_IRQn,
00038 HAL_IRQ_MODULE_SW_00 = SW_IRQn,
00039 HAL_IRQ_MODULE_INIT_00 = PWRDN_IRQn,
00040 HAL_IRQ_MODULE_CL_00 = CL0_IRQn,
00041
00042 HAL_IRQ_MODULE_UART_20, // 不支持
00043 HAL_IRQ_MODULE_CMP_10, // 不支持
00044 } HAL_IRQ_MODULE_x;
00045
00046 void HAL_IRQ_Reset(void); // 重置中断(等效于关闭中断)
00047 void HAL_IRQ_Enable(void); // 使能全局中断
00048 void HAL_IRQ_Disable(void); // 关闭全局中断
00049
00050 void HAL_IRQ_ModuleEnable(HAL_IRQ_MODULE_x, uint8_t priority); // 使能某个模块的中断
00051 void HAL_IRQ_ModuleDisable(HAL_IRQ_MODULE_x); // 关闭某个模块的中断
00052
00053 #endif

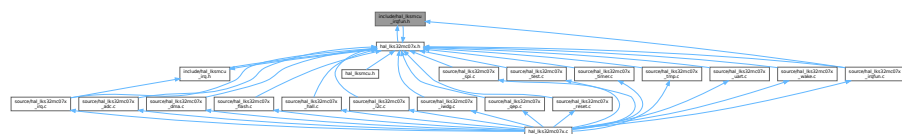
```

4.37 include/hal_lksmcu_irqfun.h 文件参考

#include "../hal_lks32mc07x.h"
hal_lksmcu_irqfun.h 的引用(Include)关系图:



此图展示该文件直接或间接的被哪些文件引用了:



函数

- uint8_t HAL_HALL_IrqChange (HAL_HALL_x)
中断处理函数
- uint8_t HAL_HALL_IrqChangeFlg (HAL_HALL_x hall)
中断信号 (和 HAL_HALL_IrqChange 的区别是不判断中断使能)
- void HAL_HALL_IrqChangeEnable (HAL_HALL_x)
中断使能
- void HAL_HALL_IrqChangeDisable (HAL_HALL_x hall)
中断禁止

- void [HAL_HALL_IrqChangeClear](#) (HAL_HALL_x hall)
中断清除
- uint8_t [HAL_HALL_IrqOver](#) (HAL_HALL_x)
中断处理函数
- uint8_t [HAL_HALL_IrqOverFlg](#) (HAL_HALL_x hall)
中断信号（和 [HAL_HALL_IrqOver](#) 的区别是不判断中断使能）
- void [HAL_HALL_IrqOverEnable](#) (HAL_HALL_x)
中断使能
- void [HAL_HALL_IrqOverDisable](#) (HAL_HALL_x hall)
中断禁止
- void [HAL_HALL_IrqOverClear](#) (HAL_HALL_x hall)
中断清除
- uint8_t [HAL_ADC_IrqSamp1](#) (HAL_ADC_x)
中断处理函数
- uint8_t [HAL_ADC_IrqSamp1Flg](#) (HAL_ADC_x adc)
中断信号（和 [HAL_ADC_IrqSamp1](#) 的区别是不判断中断使能）
- void [HAL_ADC_IrqSamp1Enable](#) (HAL_ADC_x)
中断使能
- void [HAL_ADC_IrqSamp1Disable](#) (HAL_ADC_x adc)
中断禁止
- void [HAL_ADC_IrqSamp1Clear](#) (HAL_ADC_x adc)
中断清除
- uint8_t [HAL_ADC_IrqSamp2](#) (HAL_ADC_x)
中断处理函数
- uint8_t [HAL_ADC_IrqSamp2Flg](#) (HAL_ADC_x adc)
中断信号（和 [HAL_ADC_IrqSamp2](#) 的区别是不判断中断使能）
- void [HAL_ADC_IrqSamp2Enable](#) (HAL_ADC_x)
中断使能
- void [HAL_ADC_IrqSamp2Disable](#) (HAL_ADC_x adc)
中断禁止
- void [HAL_ADC_IrqSamp2Clear](#) (HAL_ADC_x adc)
中断清除

4.37.1 函数说明

4.37.1.1 HAL_ADC_IrqSamp1()

```
uint8_t HAL_ADC_IrqSamp1 (
    HAL_ADC_x adc )
```

中断处理函数

参数

<i>adc</i>	模块编号
------------	------

返回

1表示中断发生，0表示没有中断发生

在文件 [hal_lks32mc07x_irqfun.c](#) 第 128 行定义.

4.37.1.2 HAL_ADC_IrqSamp1Clear()

```
void HAL_ADC_IrqSamp1Clear (  
    HAL_ADC_x adc )
```

中断清除

参数

<i>adc</i>	模块编号
------------	------

在文件 [hal_lks32mc07x_irqfun.c](#) 第 175 行定义.

4.37.1.3 HAL_ADC_IrqSamp1Disable()

```
void HAL_ADC_IrqSamp1Disable (  
    HAL_ADC_x adc )
```

中断禁止

参数

<i>adc</i>	模块编号
------------	------

在文件 [hal_lks32mc07x_irqfun.c](#) 第 167 行定义.

4.37.1.4 HAL_ADC_IrqSamp1Enable()

```
void HAL_ADC_IrqSamp1Enable (  
    HAL_ADC_x adc )
```

中断使能

参数

<i>adc</i>	模块编号
------------	------

在文件 [hal_lks32mc07x_irqfun.c](#) 第 159 行定义.

4.37.1.5 HAL_ADC_IrqSamp1Flg()

```
uint8_t HAL_ADC_IrqSamp1Flg (  
    HAL_ADC_x adc )
```

中断信号（和 HAL_ADC_IrqSamp1 的区别是不判断中断使能）

参数

<i>adc</i>	模块编号
------------	------

返回

1表示中断发生，0表示没有中断发生

在文件 [hal_lks32mc07x_irqfun.c](#) 第 144 行定义.

4.37.1.6 HAL_ADC_IrqSamp2()

```
uint8_t HAL_ADC_IrqSamp2 (
    HAL_ADC_x adc )
```

中断处理函数

参数

<i>adc</i>	模块编号
------------	------

返回

1表示中断发生，0表示没有中断发生

在文件 [hal_lks32mc07x_irqfun.c](#) 第 185 行定义.

4.37.1.7 HAL_ADC_IrqSamp2Clear()

```
void HAL_ADC_IrqSamp2Clear (
    HAL_ADC_x adc )
```

中断清除

参数

<i>adc</i>	模块编号
------------	------

在文件 [hal_lks32mc07x_irqfun.c](#) 第 232 行定义.

4.37.1.8 HAL_ADC_IrqSamp2Disable()

```
void HAL_ADC_IrqSamp2Disable (
    HAL_ADC_x adc )
```

中断禁止

参数

<i>adc</i>	模块编号
------------	------

在文件 [hal_lks32mc07x_irqfun.c](#) 第 224 行定义.

4.37.1.9 HAL_ADC_IrqSamp2Enable()

```
void HAL_ADC_IrqSamp2Enable (
    HAL_ADC_x adc )
```

中断使能

参数

<i>adc</i>	模块编号
------------	------

在文件 [hal_lks32mc07x_irqfun.c](#) 第 216 行定义.

4.37.1.10 HAL_ADC_IrqSamp2Flg()

```
uint8_t HAL_ADC_IrqSamp2Flg (
    HAL_ADC_x adc )
```

中断信号（和 HAL_ADC_IrqSamp2 的区别是不判断中断使能）

参数

<i>adc</i>	模块编号
------------	------

返回

1表示中断发生，0表示没有中断发生

在文件 [hal_lks32mc07x_irqfun.c](#) 第 201 行定义.

4.37.1.11 HAL_HALL_IrqChange()

```
uint8_t HAL_HALL_IrqChange (
    HAL_HALL_x hall )
```

中断处理函数

参数

<i>hall</i>	模块编号
-------------	------

返回

1表示中断发生，0表示没有中断发生

在文件 [hal_lks32mc07x_irqfun.c](#) 第 14 行定义.

4.37.1.12 HAL_HALL_IrqChangeClear()

```
void HAL_HALL_IrqChangeClear (
    HAL_HALLx hall )
```

中断清除

参数

<i>hall</i>	模块编号
-------------	------

在文件 [hal_lks32mc07x_irqfun.c](#) 第 61 行定义.

4.37.1.13 HAL_HALL_IrqChangeDisable()

```
void HAL_HALL_IrqChangeDisable (
    HAL_HALLx hall )
```

中断禁止

参数

<i>hall</i>	模块编号
-------------	------

在文件 [hal_lks32mc07x_irqfun.c](#) 第 53 行定义.

4.37.1.14 HAL_HALL_IrqChangeEnable()

```
void HAL_HALL_IrqChangeEnable (
    HAL_HALLx hall )
```

中断使能

参数

<i>hall</i>	模块编号
-------------	------

在文件 [hal_lks32mc07x_irqfun.c](#) 第 45 行定义.

4.37.1.15 HAL_HALL_IrqChangeFlg()

```
uint8_t HAL_HALL_IrqChangeFlg (
    HAL_HALL_x hall )
```

中断信号（和 HAL_HALL_IrqChange 的区别是不判断中断使能）

参数

<i>hall</i>	模块编号
-------------	------

返回

1表示中断发生，0表示没有中断发生

在文件 [hal_lks32mc07x_irqfun.c](#) 第 30 行定义.

4.37.1.16 HAL_HALL_IrqOver()

```
uint8_t HAL_HALL_IrqOver (
    HAL_HALL_x hall )
```

中断处理函数

参数

<i>hall</i>	模块编号
-------------	------

返回

1表示中断发生，0表示没有中断发生

在文件 [hal_lks32mc07x_irqfun.c](#) 第 71 行定义.

4.37.1.17 HAL_HALL_IrqOverClear()

```
void HAL_HALL_IrqOverClear (
    HAL_HALL_x hall )
```

中断清除

参数

<i>hall</i>	模块编号
-------------	------

在文件 [hal_lks32mc07x_irqfun.c](#) 第 118 行定义.

4.37.1.18 HAL_HALL_IrqOverDisable()

```
void HAL_HALL_IrqOverDisable (
    HAL_HALL_x hall )
```

中断禁止

参数

<i>hall</i>	模块编号
-------------	------

在文件 [hal_lks32mc07x_irqfun.c](#) 第 110 行定义.

4.37.1.19 HAL_HALL_IrqOverEnable()

```
void HAL_HALL_IrqOverEnable (
    HAL_HALL_x hall )
```

中断使能

参数

<i>hall</i>	模块编号
-------------	------

在文件 [hal_lks32mc07x_irqfun.c](#) 第 102 行定义.

4.37.1.20 HAL_HALL_IrqOverFlg()

```
uint8_t HAL_HALL_IrqOverFlg (
    HAL_HALL_x hall )
```

中断信号（和 HAL_HALL_IrqOver 的区别是不判断中断使能）

参数

<i>hall</i>	模块编号
-------------	------

返回

1表示中断发生，0表示没有中断发生

在文件 [hal_lks32mc07x_irqfun.c](#) 第 87 行定义.

4.38 hal_lksmcu_irqfun.h

[浏览该文件的文档.](#)

```

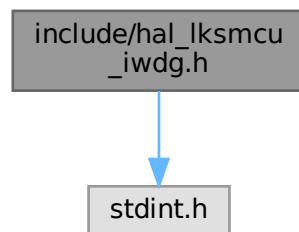
00001
00002 #ifndef __HAL_LKSMCU_IRQFUN_H__
00003 #define __HAL_LKSMCU_IRQFUN_H__
00004
00005 #include "../hal_lks32mc07x.h"
00006
00007 uint8_t HAL_HALL_IrqChange (HAL_HALL_x);
00008 uint8_t HAL_HALL_IrqChangeFlg (HAL_HALL_x hall);
00009 void HAL_HALL_IrqChangeEnable (HAL_HALL_x);
00010 void HAL_HALL_IrqChangeDisable (HAL_HALL_x hall);
00011 void HAL_HALL_IrqChangeClear (HAL_HALL_x hall);
00012
00013 uint8_t HAL_HALL_IrqOver (HAL_HALL_x);
00014 uint8_t HAL_HALL_IrqOverFlg (HAL_HALL_x hall);
00015 void HAL_HALL_IrqOverEnable (HAL_HALL_x);
00016 void HAL_HALL_IrqOverDisable (HAL_HALL_x hall);
00017 void HAL_HALL_IrqOverClear (HAL_HALL_x hall);
00018
00019 uint8_t HAL_ADC_IrqSamp1 (HAL_ADC_x);
00020 uint8_t HAL_ADC_IrqSamp1Flg (HAL_ADC_x adc);
00021 void HAL_ADC_IrqSamp1Enable (HAL_ADC_x);
00022 void HAL_ADC_IrqSamp1Disable (HAL_ADC_x adc);
00023 void HAL_ADC_IrqSamp1Clear (HAL_ADC_x adc);
00024
00025 uint8_t HAL_ADC_IrqSamp2 (HAL_ADC_x);
00026 uint8_t HAL_ADC_IrqSamp2Flg (HAL_ADC_x adc);
00027 void HAL_ADC_IrqSamp2Enable (HAL_ADC_x);
00028 void HAL_ADC_IrqSamp2Disable (HAL_ADC_x adc);
00029 void HAL_ADC_IrqSamp2Clear (HAL_ADC_x adc);
00030
00031 #endif

```

4.39 include/hal_lksmcu_iwdg.h 文件参考

```
#include "stdint.h"
```

hal_lksmcu_iwdg.h 的引用(Include)关系图:



此图展示该文件直接或间接的被哪些文件引用了:



枚举

- enum HAL_IWDG_x { HAL_IWDG_0 }

函数

- void [HAL_IWDG_Reset](#) ([HAL_IWDG_x](#))
模块复位
- void [HAL_IWDG_Enable](#) ([HAL_IWDG_x](#))
模块打开
- void [HAL_IWDG_Disable](#) ([HAL_IWDG_x](#))
模块关闭
- void [HAL_IWDG_SetResetTimemS](#) ([HAL_IWDG_x](#), int32_t ms)
设置看门狗超时复位时间
- void [HAL_IWDG_Feed](#) ([HAL_IWDG_x](#))
喂狗

4.39.1 枚举类型说明

4.39.1.1 HAL_IWDG_x

```
enum HAL\_IWDG\_x
```

枚举值

HAL_↔ IWDG_0	
-----------------	--

在文件 [hal_lksmcu_iwdg.h](#) 第 4 行定义.

4.39.2 函数说明

4.39.2.1 HAL_IWDG_Disable()

```
void HAL_IWDG_Disable (
    HAL\_IWDG\_x iwdg )
```

模块关闭

参数

<i>iwdg</i>	看门狗模块
-------------	-------

在文件 [hal_lks32mc07x_iwdg.c](#) 第 29 行定义.

这是这个函数的调用关系图:



4.39.2.2 HAL_IWDG_Enable()

```
void HAL_IWDG_Enable (
    HAL_IWDG_x iwdg )
```

模块打开

参数

<i>iwdg</i>	看门狗模块
-------------	-------

在文件 [hal_lks32mc07x.iwdg.c](#) 第 19 行定义.

4.39.2.3 HAL_IWDG_Feed()

```
void HAL_IWDG_Feed (
    HAL_IWDG_x iwdg )
```

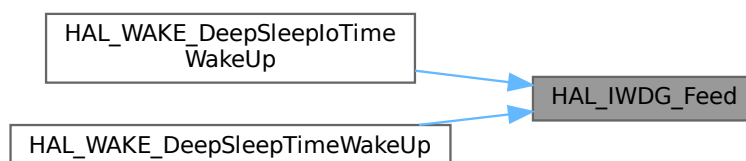
喂狗

参数

<i>iwdg</i>	看门狗模块
-------------	-------

在文件 [hal_lks32mc07x.iwdg.c](#) 第 54 行定义.

这是这个函数的调用关系图:



4.39.2.4 HAL_IWDG_Reset()

```
void HAL_IWDG_Reset (
    HAL_IWDG_x iwdg )
```

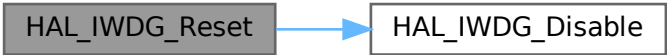
模块复位

参数

<i>iwdg</i>	看门狗模块
-------------	-------

在文件 [hal_lks32mc07x_iwdg.c](#) 第 10 行定义.

函数调用图:



4.39.2.5 HAL_IWDG_SetResetTimemS()

```
void HAL_IWDG_SetResetTimemS (
    HAL_IWDG_x iwdg,
    int32_t ms )
```

设置看门狗超时复位时间

参数

<i>iwdg</i>	看门狗模块
<i>ms</i>	超时复位时间,单位mS

在文件 [hal_lks32mc07x_iwdg.c](#) 第 41 行定义.

4.40 hal_lksmcu_iwdg.h

[浏览该文件的文档.](#)

```
00001 #ifndef __HAL_LKSMCU_IWDG_H__
00002 #define __HAL_LKSMCU_IWDG_H__
00003 #include "stdint.h"
00004 typedef enum
00005 {
00006     HAL_IWDG_0,
00007 } HAL_IWDG_x;
00008 // 初始化
```

```

00009 void HAL_IWDG_Reset(HAL_IWDG_x); // 模块复位
00010 void HAL_IWDG_Enable(HAL_IWDG_x); // 模块打开
00011 void HAL_IWDG_Disable(HAL_IWDG_x); // 模块关闭
00012 //
00013 void HAL_IWDG_SetResetTimemS(HAL_IWDG_x, int32_t ms); // 设置看门狗超时复位时间,单位ms,实际复位时间向上取整,例如
// 设置为130ms,实际一步为128ms,则按照256ms进行配置
00014 void HAL_IWDG_Feed(HAL_IWDG_x); // 喂狗
00015 #endif

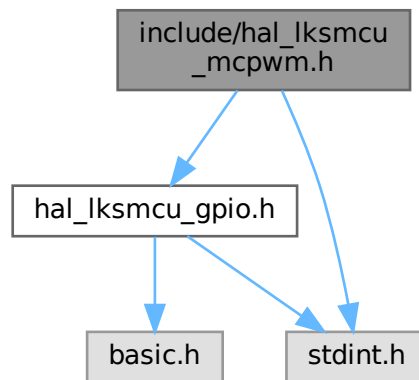
```

4.41 include/hal_lksmcu_mcpwm.h 文件参考

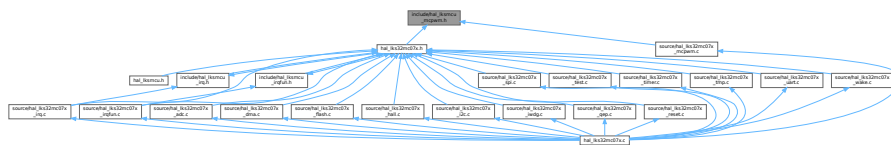
```
#include "hal_lksmcu_gpio.h"
```

```
#include "stdint.h"
```

hal_lksmcu_mcpwm.h 的引用(Include)关系图:



此图展示该文件直接或间接的被哪些文件引用了:



枚举

- enum HAL_MCPWM_x { HAL_MCPWM_0, HAL_MCPWM_1 }
- enum HAL_MCPWM_PORT_x { HAL_MCPWM_PORT_0, HAL_MCPWM_PORT_1, HAL_MCPWM_PORT_2, HAL_MCPWM_PORT_3, HAL_MCPWM_PORT_4, HAL_MCPWM_PORT_5 }
- enum HAL_MCPWM_POLARITY_x { HAL_MCPWM_POLARITY_PL_NL, HAL_MCPWM_POLARITY_PH_NL, HAL_MCPWM_POLARITY_PL_NH, HAL_MCPWM_POLARITY_PH_NH }
- enum HAL_MCPWM_OUTPUT_STATE_x { HAL_MCPWM_OUTPUT_STATE_PWM, HAL_MCPWM_OUTPUT_STATE_OFF, HAL_MCPWM_OUTPUT_STATE_LOW, HAL_MCPWM_OUTPUT_STATE_HIGH, HAL_MCPWM_OUTPUT_STATE_LOW_PWM, HAL_MCPWM_OUTPUT_STATE_HIGH_PWM }

- enum HAL_MCPWM_DEFAULT_OUTPUT_STATE_x { HAL_MCPWM_DEFAULT_OUTPUT_STATE_OFF , HAL_MCPWM_DEFAULT_OUTPUT_STATE_LOW , HAL_MCPWM_DEFAULT_OUTPUT_STATE_HIGH }
- enum HAL_MCPWM_FAIL_x { HAL_MCPWM_FAIL_IO0_HIGH , HAL_MCPWM_FAIL_IO1_HIGH , HAL_MCPWM_FAIL_CMP0_HIGH , HAL_MCPWM_FAIL_CMP1_HIGH , HAL_MCPWM_FAIL_IO0_LOW , HAL_MCPWM_FAIL_IO1_LOW , HAL_MCPWM_FAIL_CMP0_LOW , HAL_MCPWM_FAIL_CMP1_LOW }

函数

- void HAL_MCPWM_Reset (HAL_MCPWM_x)
复位*mcpwm*模块
- void HAL_MCPWM_Enable (HAL_MCPWM_x)
*mcpwm*模块使能
- void HAL_MCPWM_Disable (HAL_MCPWM_x)
*mcpwm*模块关闭
- void HAL_MCPWM_SetFreq (HAL_MCPWM_x, uint32_t freq)
设置*pwm*频率
- uint32_t HAL_MCPWM_GetFreq (HAL_MCPWM_x)
获取*pwm*频率
- uint32_t HAL_MCPWM_GetThCnt (HAL_MCPWM_x)
获取计数器最大值
- void HAL_MCPWM_SetDead (HAL_MCPWM_x, uint32_t pdead_nS, uint32_t ndead_nS)
设置死区
- void HAL_MCPWM_SetOutputDisable (HAL_MCPWM_x)
关闭输出
- void HAL_MCPWM_SetOutputEnable (HAL_MCPWM_x)
打开输出
- void HAL_MCPWM_StartCounting (HAL_MCPWM_x)
开始计数
- void HAL_MCPWM_StopCounting (HAL_MCPWM_x)
停止计数(同时计数器回到-th)
- void HAL_MCPWM_SetPwm (HAL_MCPWM_x, HAL_MCPWM_PORT_x, int32_t t0, int32_t t1)
设置占空比
- void HAL_MCPWM_SetPwmQ15 (HAL_MCPWM_x, HAL_MCPWM_PORT_x, int16_t t0, int16_t t1)
- void HAL_MCPWM_SetOutState (HAL_MCPWM_x, HAL_MCPWM_PORT_x, HAL_MCPWM_OUTPUT_STATE_x)
设置输出状态
- void HAL_MCPWM_SetAdcTrig (HAL_MCPWM_x, int32_t t0, int32_t t1)
- void HAL_MCPWM_SetAdcTrigQ15 (HAL_MCPWM_x, int32_t t0, int32_t t1)
- void HAL_MCPWM_SetDrivePolarity (HAL_MCPWM_x, HAL_MCPWM_POLARITY_x)
设置*pwm*极性
- void HAL_MCPWM_SetDefaultState (HAL_MCPWM_x, HAL_MCPWM_DEFAULT_OUTPUT_STATE_x)
设置默认状态
- void HAL_MCPWM_SetGpioOut (HAL_MCPWM_x, HAL_GPIO_Px)
设置用于输出的*gpio*
- void HAL_MCPWM_SetGpioFail (HAL_MCPWM_x, HAL_GPIO_Px)
- void HAL_MCPWM_SetFailSignal (HAL_MCPWM_x, HAL_MCPWM_FAIL_x)
设置*FAIL*信号来源
- uint8_t HAL_MCPWM_IrqOverflow (HAL_MCPWM_x)
- void HAL_MCPWM_IrqOverflowEnable (HAL_MCPWM_x)
- void HAL_MCPWM_IrqOverflowDisable (HAL_MCPWM_x)

- void [HAL_MCPWM_IrqOverflowClear](#) (HAL_MCPWM_x)
- uint8_t [HAL_MCPWM_IrqZeroCross](#) (HAL_MCPWM_x)
- void [HAL_MCPWM_IrqZeroCrossEnable](#) (HAL_MCPWM_x)
- void [HAL_MCPWM_IrqZeroCrossDisable](#) (HAL_MCPWM_x)
- void [HAL_MCPWM_IrqZeroCrossClear](#) (HAL_MCPWM_x)
- uint8_t [HAL_MCPWM_IrqFail](#) (HAL_MCPWM_x)
- void [HAL_MCPWM_IrqFailEnable](#) (HAL_MCPWM_x)
- void [HAL_MCPWM_IrqFailDisable](#) (HAL_MCPWM_x)
- void [HAL_MCPWM_IrqFailClear](#) (HAL_MCPWM_x)

4.41.1 枚举类型说明

4.41.1.1 HAL_MCPWM_DEFAULT_OUTPUT_STATE_x

enum [HAL_MCPWM_DEFAULT_OUTPUT_STATE_x](#)

枚举值

HAL_MCPWM_DEFAULT_OUTPUT_STATE_OFF	关闭输出
HAL_MCPWM_DEFAULT_OUTPUT_STATE_LOW	下管恒开
HAL_MCPWM_DEFAULT_OUTPUT_STATE_HIGH	上管恒开

在文件 [hal_lksmcu_mcpwm.h](#) 第 36 行定义.

4.41.1.2 HAL_MCPWM_FAIL_x

enum [HAL_MCPWM_FAIL_x](#)

枚举值

HAL_MCPWM_FAIL_IO0_HIGH	
HAL_MCPWM_FAIL_IO1_HIGH	
HAL_MCPWM_FAIL_CMP0_HIGH	
HAL_MCPWM_FAIL_CMP1_HIGH	
HAL_MCPWM_FAIL_IO0_LOW	
HAL_MCPWM_FAIL_IO1_LOW	
HAL_MCPWM_FAIL_CMP0_LOW	
HAL_MCPWM_FAIL_CMP1_LOW	

在文件 [hal_lksmcu_mcpwm.h](#) 第 42 行定义.

4.41.1.3 HAL_MCPWM_OUTPUT_STATE_x

enum [HAL_MCPWM_OUTPUT_STATE_x](#)

枚举值

HAL_MCPWM_OUTPUT_STATE_PWM	上下管对称pwm
HAL_MCPWM_OUTPUT_STATE_OFF	关闭输出
HAL_MCPWM_OUTPUT_STATE_LOW	下管恒开
HAL_MCPWM_OUTPUT_STATE_HIGH	上管恒开
HAL_MCPWM_OUTPUT_STATE_LOW_PWM	下管pwm
HAL_MCPWM_OUTPUT_STATE_HIGH_PWM	上管pwm

在文件 [hal_lksmcu_mcpwm.h](#) 第 27 行定义.

4.41.1.4 HAL_MCPWM_POLARITY_x

enum [HAL_MCPWM_POLARITY_x](#)

枚举值

HAL_MCPWM_POLARITY_PL_NL	上管低有效,下管低有效
HAL_MCPWM_POLARITY_PH_NL	上管高有效,下管低有效
HAL_MCPWM_POLARITY_PL_NH	上管低有效,下管高有效
HAL_MCPWM_POLARITY_PH_NH	上管高有效,下管高有效

在文件 [hal_lksmcu_mcpwm.h](#) 第 20 行定义.

4.41.1.5 HAL_MCPWM_PORT_x

enum [HAL_MCPWM_PORT_x](#)

枚举值

HAL_MCPWM_↔ PORT_0	输出端口
HAL_MCPWM_↔ PORT_1	输出端口
HAL_MCPWM_↔ PORT_2	输出端口
HAL_MCPWM_↔ PORT_3	输出端口
HAL_MCPWM_↔ PORT_4	输出端口
HAL_MCPWM_↔ PORT_5	输出端口

在文件 [hal_lksmcu_mcpwm.h](#) 第 11 行定义.

4.41.1.6 HAL_MCPWM_x

enum HAL_MCPWM_x

枚举值

HAL↔ MCPWM_0	MCPWM模块0
HAL↔ MCPWM_1	MCPWM模块1

在文件 [hal_lksmcu_mcpwm.h](#) 第 5 行定义.

4.41.2 函数说明

4.41.2.1 HAL_MCPWM_Disable()

```
void HAL_MCPWM_Disable (
    HAL_MCPWM_x mcpwm_x )
```

mcpwm模块关闭

参数

mcpwm↔ _x	操作的模块编号
--------------	---------

在文件 [hal_lks32mc07x_mcpwm.c](#) 第 191 行定义.

4.41.2.2 HAL_MCPWM_Enable()

```
void HAL_MCPWM_Enable (
    HAL_MCPWM_x mcpwm_x )
```

mcpwm模块使能

参数

mcpwm↔ _x	操作的模块编号
--------------	---------

在文件 [hal_lks32mc07x_mcpwm.c](#) 第 172 行定义.

4.41.2.3 HAL_MCPWM_GetFreq()

```
uint32_t HAL_MCPWM_GetFreq (
    HAL_MCPWM_x mcpwm_x )
```

获取pwm频率

参数

$mcpwm_{x}$	操作的模块编号
-------------	---------

返回

当前pwm的频率

在文件 [hal_lks32mc07x_mcpwm.c](#) 第 221 行定义.

函数调用图:



4.41.2.4 HAL_MCPWM_GetThCnt()

```
uint32_t HAL_MCPWM_GetThCnt (
    HAL_MCPWM_x mcpwm_x )
```

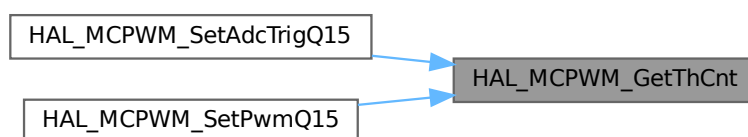
获取计数器最大值

参数

$mcpwm_{x}$	操作的模块编号
-------------	---------

在文件 [hal_lks32mc07x_mcpwm.c](#) 第 232 行定义.

这是这个函数的调用关系图:



4.41.2.5 HAL_MCPWM_IrqFail()

```
uint8_t HAL_MCPWM_IrqFail (
    HAL_MCPWM_TypeDef * mcpwm_x )
```

在文件 [hal_lks32mc07x_mcpwm.c](#) 第 494 行定义.

4.41.2.6 HAL_MCPWM_IrqFailClear()

```
void HAL_MCPWM_IrqFailClear (
    HAL_MCPWM_TypeDef * mcpwm_x )
```

在文件 [hal_lks32mc07x_mcpwm.c](#) 第 517 行定义.

4.41.2.7 HAL_MCPWM_IrqFailDisable()

```
void HAL_MCPWM_IrqFailDisable (
    HAL_MCPWM_TypeDef * mcpwm_x )
```

在文件 [hal_lks32mc07x_mcpwm.c](#) 第 511 行定义.

4.41.2.8 HAL_MCPWM_IrqFailEnable()

```
void HAL_MCPWM_IrqFailEnable (
    HAL_MCPWM_TypeDef * mcpwm_x )
```

在文件 [hal_lks32mc07x_mcpwm.c](#) 第 505 行定义.

4.41.2.9 HAL_MCPWM_IrqOverflow()

```
uint8_t HAL_MCPWM_IrqOverflow (
    HAL_MCPWM_TypeDef * mcpwm_x )
```

在文件 [hal_lks32mc07x_mcpwm.c](#) 第 438 行定义.

4.41.2.10 HAL_MCPWM_IrqOverflowClear()

```
void HAL_MCPWM_IrqOverflowClear (
    HAL_MCPWM_TypeDef * mcpwm_x )
```

在文件 [hal_lks32mc07x_mcpwm.c](#) 第 461 行定义.

4.41.2.11 HAL_MCPWM_IrqOverflowDisable()

```
void HAL_MCPWM_IrqOverflowDisable (
    HAL_MCPWM_TypeDef * mcpwm_x )
```

在文件 [hal_lks32mc07x_mcpwm.c](#) 第 455 行定义.

4.41.2.12 HAL_MCPWM_IrqOverflowEnable()

```
void HAL_MCPWM_IrqOverflowEnable (
    HAL_MCPWM_X mcpwm_x )
```

在文件 [hal_lks32mc07x_mcpwm.c](#) 第 449 行定义.

4.41.2.13 HAL_MCPWM_IrqZeroCross()

```
uint8_t HAL_MCPWM_IrqZeroCross (
    HAL_MCPWM_X mcpwm_x )
```

在文件 [hal_lks32mc07x_mcpwm.c](#) 第 466 行定义.

4.41.2.14 HAL_MCPWM_IrqZeroCrossClear()

```
void HAL_MCPWM_IrqZeroCrossClear (
    HAL_MCPWM_X mcpwm_x )
```

在文件 [hal_lks32mc07x_mcpwm.c](#) 第 489 行定义.

4.41.2.15 HAL_MCPWM_IrqZeroCrossDisable()

```
void HAL_MCPWM_IrqZeroCrossDisable (
    HAL_MCPWM_X mcpwm_x )
```

在文件 [hal_lks32mc07x_mcpwm.c](#) 第 483 行定义.

4.41.2.16 HAL_MCPWM_IrqZeroCrossEnable()

```
void HAL_MCPWM_IrqZeroCrossEnable (
    HAL_MCPWM_X mcpwm_x )
```

在文件 [hal_lks32mc07x_mcpwm.c](#) 第 477 行定义.

4.41.2.17 HAL_MCPWM_Reset()

```
void HAL_MCPWM_Reset (
    HAL_MCPWM_X mcpwm_x )
```

复位mcpwm模块

参数

<i>mcpwm_x</i>	操作的模块编号
----------------	---------

在文件 [hal_lks32mc07x_mcpwm.c](#) 第 158 行定义.

4.41.2.18 HAL_MCPWM_SetAdcTrig()

```
void HAL_MCPWM_SetAdcTrig (
    HAL_MCPWM_X mcpwm_x,
    int32_t t0,
    int32_t t1 )
```

在文件 [hal_lks32mc07x_mcpwm.c](#) 第 422 行定义.

4.41.2.19 HAL_MCPWM_SetAdcTrigQ15()

```
void HAL_MCPWM_SetAdcTrigQ15 (
    HAL_MCPWM_X mcpwm_x,
    int32_t t0,
    int32_t t1 )
```

在文件 [hal_lks32mc07x_mcpwm.c](#) 第 428 行定义.

函数调用图:



4.41.2.20 HAL_MCPWM_SetDead()

```
void HAL_MCPWM_SetDead (
    HAL_MCPWM_X mcpwm_x,
    uint32_t pdead_nS,
    uint32_t ndead_nS )
```

设置死区

参数

<i>mcpwm_x</i>	操作的模块编号
<i>pdead_nS</i>	下管关闭上管打开的死区宽度,单位nS
<i>ndead_nS</i>	上管关闭下管打开的死区宽度,单位nS

在文件 [hal_lks32mc07x_mcpwm.c](#) 第 242 行定义.

函数调用图:



4.41.2.21 HAL_MCPWM_SetDefaultState()

```
void HAL_MCPWM_SetDefaultState (
    HAL_MCPWM_x mcpwm_x,
    HAL_MCPWM_DEFAULT_OUTPUT_STATE_x mcpwm_default_output_state_x )
```

设置默认状态

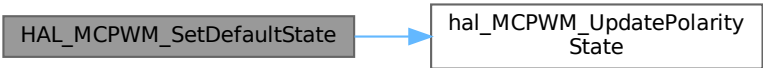
参数

<i>mcpwm_x</i>	操作的模块编号
<i>mcpwm_default_output_state_x</i>	默认输出状态

bit0 通道交换使能 0 交换 1 交换 bit1 上管有效电平 0 上管低有效 1 上管高有效 bit2 下管有效电平 0 下管低有效 1 下管高有效 bit3 默认上管电平 0 关机后上管关闭 1 关机后上管打开 bit4 默认下管电平 0 关机后下管关闭 1 关机后下管打开 bit5-bit7 输出状态 HAL_MCPWM_OUTPUT_STATE_x

在文件 `hal_lks32mc07x.mcpwm.c` 第 362 行定义.

函数调用图:



4.41.2.22 HAL_MCPWM_SetDrivePolarity()

```
void HAL_MCPWM_SetDrivePolarity (
    HAL_MCPWM_x mcpwm_x,
    HAL_MCPWM_POLARITY_x mcpwm_polarity_x )
```

设置pwm极性

参数

<i>mcpwm_x</i>	操作的模块编号
<i>mcpwm_x</i> <i>polarity_x</i>	第x个pwm输出端口

bit0 通道交换使能 0 交换 1 交换 bit1 上管有效电平 0 上管低有效 1 上管高有效 bit2 下管有效电平 0 下管低有效 1 下管高有效 bit3 默认上管电平 0 关机后上管关闭 1 关机后上管打开 bit4 默认下管电平 0 关机后下管关闭 1 关机后下管打开 bit5-bit7 输出状态 HAL_MCPWM_OUTPUT_STATE_x

在文件 [hal_lks32mc07x.mcpwm.c](#) 第 338 行定义.

函数调用图:



4.41.2.23 HAL_MCPWM_SetFailSignal()

```
void HAL_MCPWM_SetFailSignal (
    HAL_MCPWM_x mcpwm_x,
    HAL_MCPWM_FAIL_x fail_x )
```

设置FAIL信号来源

参数

<i>mcpwm_x</i>	操作的模块编号
<i>fail_x</i>	FAIL信号来源

在文件 [hal_lks32mc07x.mcpwm.c](#) 第 416 行定义.

4.41.2.24 HAL_MCPWM_SetFreq()

```
void HAL_MCPWM_SetFreq (
    HAL_MCPWM_x mcpwm_x,
    uint32_t freq )
```

设置pwm频率

参数

<i>mcpwm</i> _{<i>x</i>}	操作的模块编号
----------------------------------	---------

在文件 [hal_lks32mc07x_mcpwm.c](#) 第 207 行定义.

函数调用图:

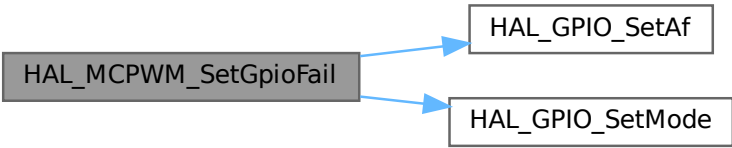


4.41.2.25 HAL_MCPWM_SetGpioFail()

```
void HAL_MCPWM_SetGpioFail (
    HAL_MCPWM_ x mcpwm_x,
    HAL_GPIO_Port px )
```

在文件 [hal_lks32mc07x_mcpwm.c](#) 第 404 行定义.

函数调用图:



4.41.2.26 HAL_MCPWM_SetGpioOut()

```
void HAL_MCPWM_SetGpioOut (
    HAL_MCPWM_ x mcpwm_x,
    HAL_GPIO_Port px )
```

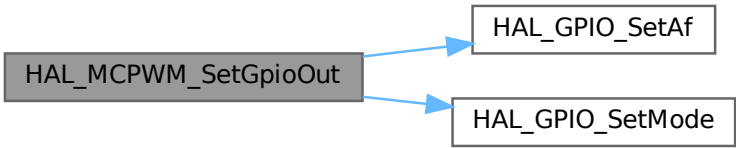
设置用于输出的gpio

参数

<i>mcpwm</i> ↔ <i>_x</i>	MCPWM模块编号
<i>px</i>	使用的GPIO口

在文件 [hal_lks32mc07x_mcpwm.c](#) 第 398 行定义.

函数调用图:



4.41.2.27 HAL_MCPWM_SetOutputDisable()

```
void HAL_MCPWM_SetOutputDisable (
    HAL_MCPWM_x mcpwm_x )
```

关闭输出

参数

<i>mcpwm</i> ↔ <i>_x</i>	操作的模块编号
-----------------------------	---------

在文件 [hal_lks32mc07x_mcpwm.c](#) 第 257 行定义.

4.41.2.28 HAL_MCPWM_SetOutputEnable()

```
void HAL_MCPWM_SetOutputEnable (
    HAL_MCPWM_x mcpwm_x )
```

打开输出

参数

<i>mcpwm</i> ↔ <i>_x</i>	操作的模块编号
-----------------------------	---------

在文件 [hal_lks32mc07x_mcpwm.c](#) 第 267 行定义.

4.41.2.29 HAL_MCPWM_SetOutState()

```
void HAL_MCPWM_SetOutState (
    HAL_MCPWM_x mcpwm_x,
    HAL_MCPWM_PORT_x mcpwm_port_x,
    HAL_MCPWM_OUTPUT_STATE_x mcpwm_output_state_x )
```

设置输出状态

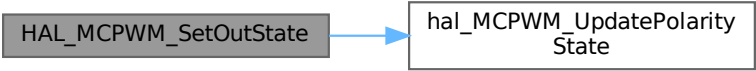
参数

<i>mcpwm_x</i>	操作的模块编号
<i>mcpwm_port_x</i>	第x个pwm输出端口
<i>mcpwm_output_state_x</i>	输出状态

bit0 通道交换使能 0 交换 1 交换 bit1 上管有效电平 0 上管低有效 1 上管高有效 bit2 下管有效电平 0 下管低有效 1 下管高有效 bit3 默认上管电平 0 关机后上管关闭 1 关机后上管打开 bit4 默认下管电平 0 关机后下管关闭 1 关机后下管打开 bit5-bit7 输出状态 HAL_MCPWM_OUTPUT_STATE_x

在文件 [hal_lks32mc07x_mcpwm.c](#) 第 319 行定义.

函数调用图:



4.41.2.30 HAL_MCPWM_SetPwm()

```
void HAL_MCPWM_SetPwm (
    HAL_MCPWM_x mcpwm_x,
    HAL_MCPWM_PORT_x mcpwm_port_x,
    int32_t t0,
    int32_t t1 )
```

设置占空比

参数

<i>mcpwm_x</i>	操作的模块编号
<i>mcpwm_port_x</i>	第x个pwm输出端口
<i>t0</i>	pwm打开时刻
<i>t1</i>	pwm关闭时刻

在文件 [hal_lks32mc07x_mcpwm.c](#) 第 300 行定义.

这是这个函数的调用关系图:

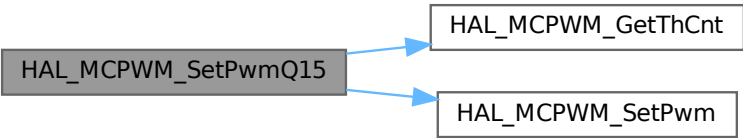


4.41.2.31 HAL.MCPWM.SetPwmQ15()

```
void HAL_MCPWM_SetPwmQ15 (
    HAL_MCPWM_x mcpwm_x,
    HAL_MCPWM_PORT_x mcpwm_port_x,
    int16_t t0,
    int16_t t1 )
```

在文件 [hal_lks32mc07x_mcpwm.c](#) 第 306 行定义.

函数调用图:



4.41.2.32 HAL.MCPWM.StartCounting()

```
void HAL_MCPWM_StartCounting (
    HAL_MCPWM_x mcpwm_x )
```

开始计数

参数

<i>mcpwm_x</i>	操作的模块编号
----------------	---------

在文件 [hal_lks32mc07x_mcpwm.c](#) 第 277 行定义.

4.41.2.33 HAL_MCPWM_StopCounting()

```
void HAL_MCPWM_StopCounting (
    HAL_MCPWM_x mcpwm_x )
```

停止计数(同时计数器回到-th)

参数

<i>mcpwm_x</i>	操作的模块编号
----------------	---------

在文件 `hal_lks32mc07x_mcpwm.c` 第 287 行定义.

4.42 hal_lksmcu_mcpwm.h

[浏览该文件的文档.](#)

```
00001 #ifndef __HAL_LKSMCU_MCPWM_H__
00002 #define __HAL_LKSMCU_MCPWM_H__
00003 #include "hal_lksmcu_gpio.h"
00004 #include "stdint.h"
00005 typedef enum
00006 {
00007     HAL_MCPWM_0,
00008     HAL_MCPWM_1,
00009 } HAL_MCPWM_x;
00010
00011 typedef enum
00012 {
00013     HAL_MCPWM_PORT_0,
00014     HAL_MCPWM_PORT_1,
00015     HAL_MCPWM_PORT_2,
00016     HAL_MCPWM_PORT_3,
00017     HAL_MCPWM_PORT_4,
00018     HAL_MCPWM_PORT_5,
00019 } HAL_MCPWM_PORT_x;
00020 typedef enum
00021 {
00022     HAL_MCPWM_POLARITY_PL_NL,
00023     HAL_MCPWM_POLARITY_PH_NL,
00024     HAL_MCPWM_POLARITY_PL_NH,
00025     HAL_MCPWM_POLARITY_PH_NH,
00026 } HAL_MCPWM_POLARITY_x;
00027 typedef enum
00028 {
00029     HAL_MCPWM_OUTPUT_STATE_PWM,
00030     HAL_MCPWM_OUTPUT_STATE_OFF,
00031     HAL_MCPWM_OUTPUT_STATE_LOW,
00032     HAL_MCPWM_OUTPUT_STATE_HIGH,
00033     HAL_MCPWM_OUTPUT_STATE_LOW_PWM,
00034     HAL_MCPWM_OUTPUT_STATE_HIGH_PWM,
00035 } HAL_MCPWM_OUTPUT_STATE_x;
00036 typedef enum
00037 {
00038     HAL_MCPWM_DEFAULT_OUTPUT_STATE_OFF,
00039     HAL_MCPWM_DEFAULT_OUTPUT_STATE_LOW,
00040     HAL_MCPWM_DEFAULT_OUTPUT_STATE_HIGH,
00041 } HAL_MCPWM_DEFAULT_OUTPUT_STATE_x;
00042 typedef enum
00043 {
00044     HAL_MCPWM_FAIL_IO0_HIGH,
00045     HAL_MCPWM_FAIL_IO1_HIGH,
00046     HAL_MCPWM_FAIL_CMP0_HIGH,
00047     HAL_MCPWM_FAIL_CMP1_HIGH,
00048     HAL_MCPWM_FAIL_IO0_LOW,
00049     HAL_MCPWM_FAIL_IO1_LOW,
00050     HAL_MCPWM_FAIL_CMP0_LOW,
00051     HAL_MCPWM_FAIL_CMP1_LOW,
00052 } HAL_MCPWM_FAIL_x;
00053 // 初始化
00054 void HAL_MCPWM_Reset (HAL_MCPWM_x); // 模块复位
00055 void HAL_MCPWM_Enable (HAL_MCPWM_x); // 模块打开
```

```

00056 void HAL_MCPWM_Disable(HAL_MCPWM_x); // 模块关闭
00057
00058 void HAL_MCPWM_SetFreq(HAL_MCPWM_x, uint32_t freq); // 设置pwm频率
00059 uint32_t HAL_MCPWM_GetFreq(HAL_MCPWM_x); // 获取当前的pwm频率
00060 uint32_t HAL_MCPWM_GetThCnt(HAL_MCPWM_x); // 获取计数器最大值
00061 void HAL_MCPWM_SetDead(HAL_MCPWM_x, uint32_t pdead_nS, uint32_t ndead_nS); // 设置死区
00062
00063 void HAL_MCPWM_SetOutputDisable(HAL_MCPWM_x); // 关闭输出
00064 void HAL_MCPWM_SetOutputEnable(HAL_MCPWM_x); // 打开输出
00065 void HAL_MCPWM_StartCounting(HAL_MCPWM_x); // 开始计数
00066 void HAL_MCPWM_StopCounting(HAL_MCPWM_x); // 停止计数(同时计数
器回到-th)
00067 void HAL_MCPWM_SetPwm(HAL_MCPWM_x, HAL_MCPWM_PORT_x, int32_t t0, int32_t t1); // 设置pwm频率
00068 void HAL_MCPWM_SetPwmQ15(HAL_MCPWM_x, HAL_MCPWM_PORT_x, int16_t t0, int16_t t1); // 设置pwm波形q15格式输入
00069 void HAL_MCPWM_SetOutState(HAL_MCPWM_x, HAL_MCPWM_PORT_x, HAL_MCPWM_OUTPUT_STATE_x); // 设置输出状态
00070 void HAL_MCPWM_SetAdcTrig(HAL_MCPWM_x, int32_t t0, int32_t t1); // 设置pwm输出ADC采样
时间
00071 void HAL_MCPWM_SetAdcTrigQ15(HAL_MCPWM_x, int32_t t0, int32_t t1); // 设置pwm输出ADC采样
时间q15格式输入
00072
00073 void HAL_MCPWM_SetDrivePolarity(HAL_MCPWM_x, HAL_MCPWM_POLARITY_x); // 设置pwm极性
00074 void HAL_MCPWM_SetDefaultState(HAL_MCPWM_x, HAL_MCPWM_DEFAULT_OUTPUT_STATE_x); // 设置默认电平
00075 void HAL_MCPWM_SetGpioOut(HAL_MCPWM_x, HAL_GPIO_Px); // 设置用于输出的gpio
00076 void HAL_MCPWM_SetGpioFail(HAL_MCPWM_x, HAL_GPIO_Px); // 设置用于故障输入的gpio
00077 void HAL_MCPWM_SetFailSignal(HAL_MCPWM_x, HAL_MCPWM_FAIL_x); // 设置FAIL信号来源
00078
00079 uint8_t HAL_MCPWM_IrqOverflow(HAL_MCPWM_x);
00080 void HAL_MCPWM_IrqOverflowEnable(HAL_MCPWM_x);
00081 void HAL_MCPWM_IrqOverflowDisable(HAL_MCPWM_x);
00082 void HAL_MCPWM_IrqOverflowClear(HAL_MCPWM_x);
00083
00084 uint8_t HAL_MCPWM_IrqZeroCross(HAL_MCPWM_x);
00085 void HAL_MCPWM_IrqZeroCrossEnable(HAL_MCPWM_x);
00086 void HAL_MCPWM_IrqZeroCrossDisable(HAL_MCPWM_x);
00087 void HAL_MCPWM_IrqZeroCrossClear(HAL_MCPWM_x);
00088
00089 uint8_t HAL_MCPWM_IrqFail(HAL_MCPWM_x);
00090 void HAL_MCPWM_IrqFailEnable(HAL_MCPWM_x);
00091 void HAL_MCPWM_IrqFailDisable(HAL_MCPWM_x);
00092 void HAL_MCPWM_IrqFailClear(HAL_MCPWM_x);
00093 #endif

```

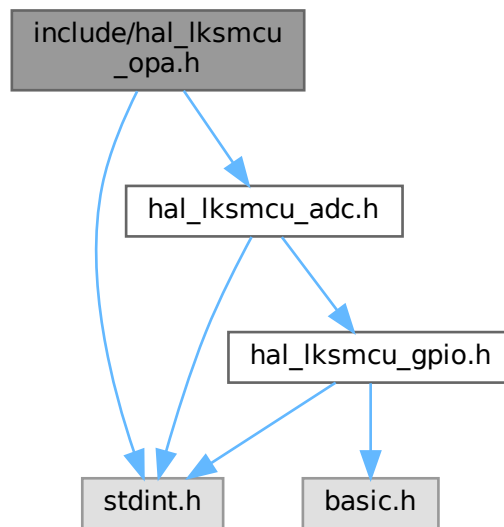
4.43 include/hal_lksmcu_opa.h 文件参考

```

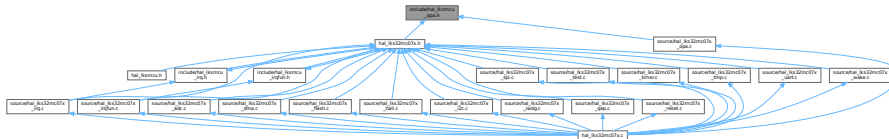
#include "stdint.h"
#include "hal_lksmcu_adc.h"

```


hal_lksmcu_opa.h 的引用(Include)关系图:



此图展示该文件直接或间接的被哪些文件引用了:



宏定义

- `#define HAL_OPA_OPAX_2_ADCCHN(OPAX) ((OPAX) == HAL_OPA_0B ? HAL_ADC_PORT_0 : (HAL_ADC_PORT_x)(OPAX))`
- `#define HAL_OPA_READ_GAIN(GAIN_x, res)`
芯片的运放放大倍数

枚举

- `enum HAL_OPA_x {`
`HAL_OPA_0, HAL_OPA_1, HAL_OPA_2, HAL_OPA_3,`
`HAL_OPA_4, HAL_OPA_5, HAL_OPA_0B }`
- `enum HAL_OPA_GAIN_x {`
`HAL_OPA_GAIN_320_10 = 0, HAL_OPA_GAIN_160_10 = 1, HAL_OPA_GAIN_80_10 = 2, HAL_OPA_GAIN_40_10`
`= 3,`
`HAL_OPA_GAIN_HIGH = HAL_OPA_GAIN_160_10, HAL_OPA_GAIN_MEDIUM = HAL_OPA_GAIN_80_10,`
`HAL_OPA_GAIN_LOW = HAL_OPA_GAIN_40_10, HAL_OPA_GAIN_200_10,`
`HAL_OPA_GAIN_190_20, HAL_OPA_GAIN_180_30, HAL_OPA_GAIN_170_40, HAL_OPA_GAIN_200_10_6,`

```

HAL_OPA_GAIN_190_20_6, HAL_OPA_GAIN_180_30_6, HAL_OPA_GAIN_170_40_6, HAL_OPA_GAIN_200_10_4
,
HAL_OPA_GAIN_190_20_4, HAL_OPA_GAIN_180_30_4, HAL_OPA_GAIN_170_40_4, HAL_OPA_GAIN_660_20
,
HAL_OPA_GAIN_640_40, HAL_OPA_GAIN_320_40, HAL_OPA_GAIN_160_40, HAL_OPA_GAIN_20_10,
HAL_OPA_GAIN_320_5 }

```

函数

- void `HAL_OPA_Reset` (`HAL_OPA_x`)
- void `HAL_OPA_Enable` (`HAL_OPA_x`)
- void `HAL_OPA_Disable` (`HAL_OPA_x`)
- void `HAL_OPA_SetGain` (`HAL_OPA_x`, `HAL_OPA_GAIN_x`)
- void `HAL_OPA_Out2GpioEnable` (`HAL_OPA_x`)
OPA输出到IO
- void `HAL_OPA_Out2GpioDisable` (`HAL_OPA_x`)
OPA停止输出到IO
- void `HAL_OPA_SetGpioIn` (`HAL_OPA_x`, `HAL_GPIO_Px`)
OPA设置IO输入
- void `HAL_OPA_SetGpioOut` (`HAL_OPA_x`, `HAL_GPIO_Px`)
OPA设置IO输出

4.43.1 宏定义说明

4.43.1.1 HAL_OPA_OPAX_2_ADCCHN

```

#define HAL_OPA_OPAX_2_ADCCHN(
    OPAX ) ((OPAX) == HAL_OPA_0B ? HAL_ADC_PORT_0 : (HAL_ADC_PORT_x)(OPAX))

```

在文件 `hal_lksmcu_opa.h` 第 68 行定义.

4.43.1.2 HAL_OPA_READ_GAIN

```

#define HAL_OPA_READ_GAIN(
    GAIN_x,
    res )

```

值:

```

== HAL_OPA_GAIN_HIGH ? (160.0f / (10.0f + (float)res)) : (GAIN_x == HAL_OPA_GAIN_MEDIUM) ? (80.0f /
(10.0f + (float)res)) \
(40.0f / (10.0f + (float)res)))

```

芯片的运放放大倍数

在文件 `hal_lksmcu_opa.h` 第 72 行定义.

4.43.2 枚举类型说明

4.43.2.1 HAL_OPA_GAIN_x

```
enum HAL_OPA_GAIN_x
```

枚举值

HAL_OPA_GAIN_320_10	
HAL_OPA_GAIN_160_10	
HAL_OPA_GAIN_80_10	
HAL_OPA_GAIN_40_10	
HAL_OPA_GAIN_HIGH	高增益
HAL_OPA_GAIN_MEDIUM	中等增益
HAL_OPA_GAIN_LOW	低增益
HAL_OPA_GAIN_200_10	
HAL_OPA_GAIN_190_20	
HAL_OPA_GAIN_180_30	
HAL_OPA_GAIN_170_40	
HAL_OPA_GAIN_200_↔ 10_6	
HAL_OPA_GAIN_190_↔ 20_6	
HAL_OPA_GAIN_180_↔ 30_6	
HAL_OPA_GAIN_170_↔ 40_6	
HAL_OPA_GAIN_200_↔ 10_4	
HAL_OPA_GAIN_190_↔ 20_4	
HAL_OPA_GAIN_180_↔ 30_4	
HAL_OPA_GAIN_170_↔ 40_4	
HAL_OPA_GAIN_660_20	
HAL_OPA_GAIN_640_40	
HAL_OPA_GAIN_320_40	
HAL_OPA_GAIN_160_40	
HAL_OPA_GAIN_20_10	
HAL_OPA_GAIN_320_5	

在文件 [hal_lksmcu_opa.h](#) 第 16 行定义.

4.43.2.2 HAL_OPA_x

enum [HAL_OPA_x](#)

枚举值

HAL_OPA_0	ADC模块0
HAL_OPA_1	ADC模块1
HAL_OPA_2	ADC模块2
HAL_OPA_3	ADC模块3
HAL_OPA_4	ADC模块4
HAL_OPA_5	ADC模块5
HAL_OPA_0B	ADC模块0B

在文件 [hal_lksmcu_opa.h](#) 第 5 行定义.

4.43.3 函数说明

4.43.3.1 HAL_OPA_Disable()

```
void HAL_OPA_Disable (
    HAL_OPA_x opa_x )
```

在文件 [hal_lks32mc07x_opa.c](#) 第 37 行定义.

4.43.3.2 HAL_OPA_Enable()

```
void HAL_OPA_Enable (
    HAL_OPA_x opa_x )
```

在文件 [hal_lks32mc07x_opa.c](#) 第 12 行定义.

4.43.3.3 HAL_OPA_Out2GpioDisable()

```
void HAL_OPA_Out2GpioDisable (
    HAL_OPA_x opa_x )
```

OPA停止输出到IO

参数

opa_x	运放序号
----------	------

在文件 [hal_lks32mc07x_opa.c](#) 第 119 行定义.

4.43.3.4 HAL_OPA_Out2GpioEnable()

```
void HAL_OPA_Out2GpioEnable (
    HAL_OPA_x opa_x )
```

OPA输出到IO

参数

opa_x	运放序号
----------	------

在文件 [hal_lks32mc07x_opa.c](#) 第 91 行定义.

4.43.3.5 HAL_OPA_Reset()

```
void HAL_OPA_Reset (
    HAL_OPA_x opa_x )
```

在文件 [hal_lks32mc07x_opa.c](#) 第 7 行定义.

4.43.3.6 HAL_OPA_SetGain()

```
void HAL_OPA_SetGain (
    HAL_OPA_x opa_x,
    HAL_OPA_GAIN_x gain_x )
```

在文件 [hal_lks32mc07x_opa.c](#) 第 62 行定义.

4.43.3.7 HAL_OPA_SetGpioIn()

```
void HAL_OPA_SetGpioIn (
    HAL_OPA_x opa_x,
    HAL_GPIO_Px px )
```

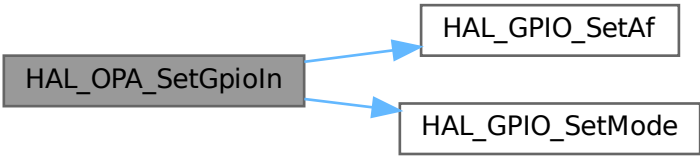
OPA设置IO输入

参数

<i>opa_x</i>	运放序号
<i>px</i>	IO序号

在文件 [hal_lks32mc07x_opa.c](#) 第 134 行定义.

函数调用图:



4.43.3.8 HAL_OPA_SetGpioOut()

```
void HAL_OPA_SetGpioOut (
    HAL_OPA_x opa_x,
    HAL_GPIO_Px px )
```

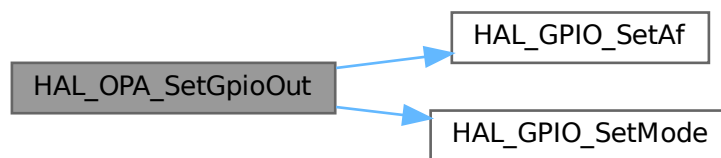
OPA设置IO输出

参数

<i>opa</i> ↔ <i>_x</i>	运放序号
<i>px</i>	IO序号

在文件 `hal_lks32mc07x_opa.c` 第 145 行定义.

函数调用图:



4.44 hal_lksmcu_opa.h

浏览该文件的文档.

```
00001 #ifndef _HAL_LKSMCU_OPA_H_
00002 #define _HAL_LKSMCU_OPA_H_
00003 #include "stdint.h"
00004 #include "hal_lksmcu_adc.h"
00005 typedef enum
00006 {
00007     HAL_OPA_0,
00008     HAL_OPA_1,
00009     HAL_OPA_2,
00010     HAL_OPA_3,
00011     HAL_OPA_4,
00012     HAL_OPA_5,
00013     HAL_OPA_0B,
00014 } HAL_OPA_x;
00015
00016 typedef enum
00017 {
00018     // 07
00019     HAL_OPA_GAIN_320_10 = 0,
00020     HAL_OPA_GAIN_160_10 = 1,
00021     HAL_OPA_GAIN_80_10 = 2,
00022     HAL_OPA_GAIN_40_10 = 3,
00023
00024     HAL_OPA_GAIN_HIGH = HAL_OPA_GAIN_160_10,
00025     HAL_OPA_GAIN_MEDIUM = HAL_OPA_GAIN_80_10,
00026     HAL_OPA_GAIN_LOW = HAL_OPA_GAIN_40_10,
00027
00028     // 03
00029     HAL_OPA_GAIN_200_10,
00030     HAL_OPA_GAIN_190_20,
00031     HAL_OPA_GAIN_180_30,
00032     HAL_OPA_GAIN_170_40,
00033     // 05
00034     HAL_OPA_GAIN_200_10_6,
00035     HAL_OPA_GAIN_190_20_6,
00036     HAL_OPA_GAIN_180_30_6,
00037     HAL_OPA_GAIN_170_40_6,
00038
00039     // 08
```

```

00040     HAL_OPA_GAIN_200_10_4,
00041     HAL_OPA_GAIN_190_20_4,
00042     HAL_OPA_GAIN_180_30_4,
00043     HAL_OPA_GAIN_170_40_4,
00044
00045     // 09
00046     HAL_OPA_GAIN_660_20,
00047     HAL_OPA_GAIN_640_40,
00048     HAL_OPA_GAIN_320_40,
00049     HAL_OPA_GAIN_160_40,
00050
00051     // 45
00052     HAL_OPA_GAIN_20_10,
00053     HAL_OPA_GAIN_320_5,
00054
00055 } HAL_OPA_GAIN_x;
00056
00057 // 初始化
00058 void HAL_OPA_Reset (HAL_OPA_x); // 模块复位
00059 void HAL_OPA_Enable (HAL_OPA_x); // 模块打开
00060 void HAL_OPA_Disable (HAL_OPA_x); // 模块关闭
00061
00062 void HAL_OPA_SetGain (HAL_OPA_x, HAL_OPA_GAIN_x); // 设置增益
00063 void HAL_OPA_Out2GpioEnable (HAL_OPA_x);
00064 void HAL_OPA_Out2GpioDisable (HAL_OPA_x);
00065 void HAL_OPA_SetGpioIn (HAL_OPA_x, HAL_GPIO_Px);
00066 void HAL_OPA_SetGpioOut (HAL_OPA_x, HAL_GPIO_Px);
00067
00068 #define HAL_OPA_OPAX_2_ADCCHN(OPAX) ((OPAX) == HAL_OPA_0B ? HAL_ADC_PORT_0 : (HAL_ADC_PORT_x)(OPAX))
00072 #define HAL_OPA_READ_GAIN(GAIN_x, res) (GAIN_x == HAL_OPA_GAIN_HIGH ? (160.0f / (10.0f + (float)res)) :
    (GAIN_x == HAL_OPA_GAIN_MEDIUM ? (80.0f / (10.0f + (float)res)) \
00073 : (40.0f / (10.0f + (float)res)))
00074
00075 #endif

```

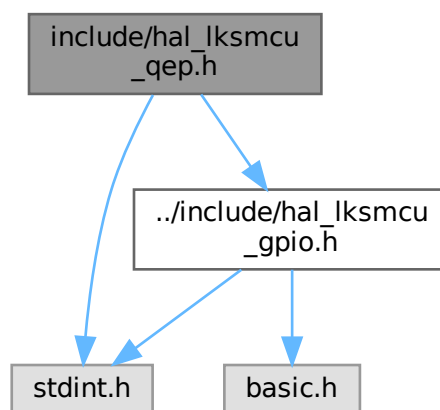
4.45 include/hal_lksmcu_qep.h 文件参考

```

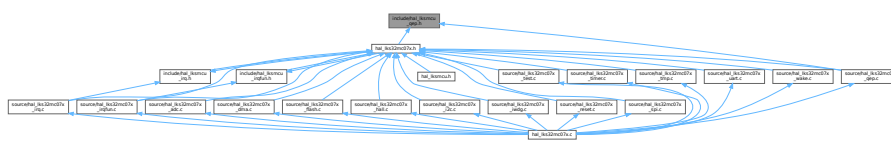
#include <stdint.h>
#include "../include/hal_lksmcu_gpio.h"

```

hal_lksmcu_qep.h 的引用(Include)关系图:



此图展示该文件直接或间接的被哪些文件引用了:



枚举

- enum `HAL_QEP_x` { `HAL_QEP_0` , `HAL_QEP_1` , `HAL_QEP_2` , `HAL_QEP_3` }
- enum `HAL_QEP_MODE_x` { `HAL_QEP_MODE_QUADRATURE` , `HAL_QEP_MODE_SYMBOL_PULSE` , `HAL_QEP_MODE_DUAL_PULSE` }

函数

- void `HAL_QEP_Reset` (`HAL_QEP_x`)
模块复位
- void `HAL_QEP_Enable` (`HAL_QEP_x`)
模块打开
- void `HAL_QEP_Disable` (`HAL_QEP_x`)
模块关闭
- void `HAL_QEP_SetTh` (`HAL_QEP_x`, `uint32_t th`)
设置最大计数值
- void `HAL_QEP_SetMode` (`HAL_QEP_x`, `HAL_QEP_MODE_x`)
设置计数模式
- `uint32_t` `HAL_QEP_ReadCnt` (`HAL_QEP_x`)
获取当前的计数值
- void `HAL_QEP_ResetCnt` (`HAL_QEP_x`)
复位计数值到0
- void `HAL_QEP_SetGpio` (`HAL_QEP_x`, `HAL_GPIO_Px`)
设置QEP模块的引脚

4.45.1 枚举类型说明

4.45.1.1 HAL_QEP_MODE_x

enum `HAL_QEP_MODE_x`

枚举值

<code>HAL_QEP_MODE_QUADRATURE</code>	
<code>HAL_QEP_MODE_SYMBOL_PULSE</code>	
<code>HAL_QEP_MODE_DUAL_PULSE</code>	

在文件 `hal_lksmcu_qep.h` 第 13 行定义.

4.45.1.2 HAL_QEP_x

enum HAL_QEP_x

枚举值

HAL_↔ QEP_0	
HAL_↔ QEP_1	
HAL_↔ QEP_2	
HAL_↔ QEP_3	

在文件 [hal_lksmcu_qep.h](#) 第 5 行定义.

4.45.2 函数说明

4.45.2.1 HAL_QEP_Disable()

```
void HAL_QEP_Disable (
    HAL_QEP_x qep )
```

模块关闭

参数

<i>qep</i>	QEP模块
------------	-------

在文件 [hal_lks32mc07x_qep.c](#) 第 63 行定义.

4.45.2.2 HAL_QEP_Enable()

```
void HAL_QEP_Enable (
    HAL_QEP_x qep )
```

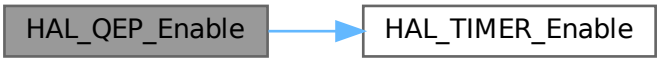
模块打开

参数

<i>qep</i>	QEP模块
------------	-------

在文件 [hal_lks32mc07x_qep.c](#) 第 37 行定义.

函数调用图:



4.45.2.3 HAL_QEP_ReadCnt()

```
uint32_t HAL_QEP_ReadCnt (
    HAL_QEP_x qep )
```

获取当前的计数值

参数

qep	QEP模块
-----	-------

返回

当前的计数值

在文件 [hal_lks32mc07x.qep.c](#) 第 121 行定义.

4.45.2.4 HAL_QEP_Reset()

```
void HAL_QEP_Reset (
    HAL_QEP_x qep )
```

模块复位

参数

qep	QEP模块
-----	-------

在文件 [hal_lks32mc07x.qep.c](#) 第 12 行定义.

4.45.2.5 HAL_QEP_ResetCnt()

```
void HAL_QEP_ResetCnt (
    HAL_QEP_x qep )
```

复位计数值到0

参数

<i>qep</i>	QEP模块
------------	-------

在文件 [hal_lks32mc07x.qep.c](#) 第 130 行定义.

4.45.2.6 HAL_QEP_SetGpio()

```
void HAL_QEP_SetGpio (
    HAL_QEP_x qepx,
    HAL_GPIO_Px px )
```

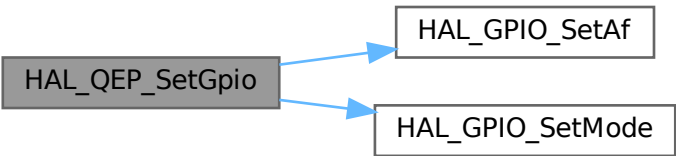
设置QEP模块的引脚

参数

<i>qepx</i>	QEP模块
<i>px</i>	引脚

在文件 [hal_lks32mc07x.qep.c](#) 第 140 行定义.

函数调用图:



4.45.2.7 HAL_QEP_SetMode()

```
void HAL_QEP_SetMode (
    HAL_QEP_x qep,
    HAL_QEP_MODE_x mode )
```

设置计数模式

参数

<i>qep</i>	QEP模块
<i>mode</i>	计数模式

在文件 [hal_lks32mc07x_qep.c](#) 第 98 行定义.

4.45.2.8 HAL_QEP_SetTh()

```
void HAL_QEP_SetTh (
    HAL_QEP_x qep,
    uint32_t th )
```

设置最大计数值

参数

<i>qep</i>	QEP模块
<i>th</i>	最大计数值

在文件 [hal_lks32mc07x_qep.c](#) 第 88 行定义.

4.46 hal_lksmcu_qep.h

[浏览该文件的文档.](#)

```
00001 #ifndef __HAL_LKSMCU_QEP_H__
00002 #define __HAL_LKSMCU_QEP_H__
00003 #include <stdint.h>
00004 #include "../include/hal_lksmcu_gpio.h"
00005 typedef enum
00006 {
00007     HAL_QEP_0,
00008     HAL_QEP_1,
00009     HAL_QEP_2,
00010     HAL_QEP_3,
00011 } HAL_QEP_x;
00012
00013 typedef enum
00014 {
00015     HAL_QEP_MODE_QUADRATURE, // 正交编码计数模式
00016     HAL_QEP_MODE_SYMBOL_PULSE, // 符号加脉冲计数模式
00017     HAL_QEP_MODE_DUAL_PULSE, // 双脉冲计数模式
00018 } HAL_QEP_MODE_x;
00019 void HAL_QEP_Reset(HAL_QEP_x); // 模块复位
00020 void HAL_QEP_Enable(HAL_QEP_x); // 模块打开
00021 void HAL_QEP_Disable(HAL_QEP_x); // 模块关闭
00022
00023 void HAL_QEP_SetTh(HAL_QEP_x, uint32_t th); // 设置最大计数值
00024 void HAL_QEP_SetMode(HAL_QEP_x, HAL_QEP_MODE_x); // 设置计数模式
00025 uint32_t HAL_QEP_ReadCnt(HAL_QEP_x); // 获取当前的计数值
00026 void HAL_QEP_ResetCnt(HAL_QEP_x); // 复位计数值到0
00027
00028 void HAL_QEP_SetGpio(HAL_QEP_x, HAL_GPIO_Px); // 设置指定的GPIO为ADC输入
00029 #endif
```

4.47 include/hal_lksmcu_reg.h 文件参考

HAL库配置和内部参数

此图展示该文件直接或间接的被哪些文件引用了:



宏定义

- `#define HAL_REG_WRITE(reg, mask) reg = (mask)`
- `#define HAL_REG_READ(reg) (reg)`
- `#define HAL_REG_WRITE_BIT(reg, mask, val) reg = ((reg) & ~(mask)) | ((val) & (mask))`
- `#define HAL_REG_WRITE_BIT_MASK_POS(reg, name, val) reg = ((reg) & ~(name##_MASK)) | (name(val))`
- `#define HAL_REG_READ_BIT_POS_MASK(reg, mask) (((reg) & (mask##_MASK)) >> mask##_POS)`
- `#define HAL_REG_READ_BIT(reg, mask) ((reg) & (mask))`
- `#define HAL_REG_SET(reg, mask) reg |= (mask)`
- `#define HAL_REG_RESET(reg, mask) reg &= ~(mask)`
- `#define HAL_REG_INVERT(reg, mask) reg = ((reg) & ~(mask)) | ((~((reg) & (mask))) & (mask))`
- `#define HAL_REG_SYS_LOCK() SYS_WR_PROTECT = 0`
- `#define HAL_REG_SYS_UNLOCK() SYS_WR_PROTECT = 0x7a83`
- `#define HAL_REG_IF_WRITE(reg, mask, condition) reg = (condition) ? ((reg) | (mask)) : ((reg) & ~(mask))`

4.47.1 详细描述

HAL库配置和内部参数

在文件 `hal_lksmcu_reg.h` 中定义.

4.47.2 宏定义说明

4.47.2.1 HAL_REG_IF_WRITE

```
#define HAL_REG_IF_WRITE(  
    reg,  
    mask,  
    condition ) reg = (condition) ? ((reg) | (mask)) : ((reg) & ~(mask))
```

在文件 `hal_lksmcu_reg.h` 第 19 行定义.

4.47.2.2 HAL_REG_INVERT

```
#define HAL_REG_INVERT(  
    reg,  
    mask ) reg = ((reg) & ~(mask)) | ((~((reg) & (mask))) & (mask))
```

在文件 `hal_lksmcu_reg.h` 第 16 行定义.

4.47.2.3 HAL_REG_READ

```
#define HAL_REG_READ(  
    reg ) (reg)
```

在文件 `hal_lksmcu_reg.h` 第 9 行定义.

4.47.2.4 HAL_REG_READ_BIT

```
#define HAL_REG_READ_BIT(  
    reg,  
    mask ) ((reg) & (mask))
```

在文件 [hal_lksmcu_reg.h](#) 第 13 行定义.

4.47.2.5 HAL_REG_READ_BIT_POS_MASK

```
#define HAL_REG_READ_BIT_POS_MASK(  
    reg,  
    mask ) (((reg) & (mask##_MASK)) >> mask##_POS)
```

在文件 [hal_lksmcu_reg.h](#) 第 12 行定义.

4.47.2.6 HAL_REG_RESET

```
#define HAL_REG_RESET(  
    reg,  
    mask ) reg &= ~(mask)
```

在文件 [hal_lksmcu_reg.h](#) 第 15 行定义.

4.47.2.7 HAL_REG_SET

```
#define HAL_REG_SET(  
    reg,  
    mask ) reg |= (mask)
```

在文件 [hal_lksmcu_reg.h](#) 第 14 行定义.

4.47.2.8 HAL_REG_SYS_LOCK

```
#define HAL_REG_SYS_LOCK( ) SYS_WR_PROTECT = 0
```

在文件 [hal_lksmcu_reg.h](#) 第 17 行定义.

4.47.2.9 HAL_REG_SYS_UNLOCK

```
#define HAL_REG_SYS_UNLOCK( ) SYS_WR_PROTECT = 0x7a83
```

在文件 [hal_lksmcu_reg.h](#) 第 18 行定义.

4.47.2.10 HAL_REG_WRITE

```
#define HAL_REG_WRITE(  
    reg,  
    mask ) reg = (mask)
```

在文件 [hal_lksmcu_reg.h](#) 第 8 行定义.

4.47.2.11 HAL_REG_WRITE_BIT

```
#define HAL_REG_WRITE_BIT(  
    reg,  
    mask,  
    val ) reg = ((reg) & ~(mask)) | ((val) & (mask))
```

在文件 [hal_lksmcu_reg.h](#) 第 10 行定义.

4.47.2.12 HAL_REG_WRITE_BIT_MASK_POS

```
#define HAL_REG_WRITE_BIT_MASK_POS(  
    reg,  
    name,  
    val ) reg = ((reg) & ~(name##_MASK)) | (name(val))
```

在文件 [hal_lksmcu_reg.h](#) 第 11 行定义.

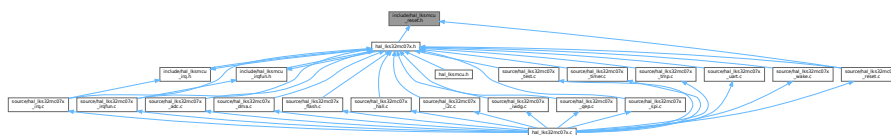
4.48 hal_lksmcu_reg.h

[浏览该文件的文档.](#)

```
00001 #ifndef __HAL_LKSMCU_REG_H__  
00002 #define __HAL_LKSMCU_REG_H__  
00003  
00008 #define HAL_REG_WRITE(reg, mask)                reg = (mask)  
00009 #define HAL_REG_READ(reg)                        (reg)  
00010 #define HAL_REG_WRITE_BIT(reg, mask, val)        reg = ((reg) & ~(mask)) | ((val) & (mask))  
00011 #define HAL_REG_WRITE_BIT_MASK_POS(reg, name, val) reg = ((reg) & ~(name##_MASK)) | (name(val))  
00012 #define HAL_REG_READ_BIT_MASK_POS(reg, mask)    (((reg) & (mask##_MASK)) >> mask##_POS)  
00013 #define HAL_REG_READ_BIT(reg, mask)              ((reg) & (mask))  
00014 #define HAL_REG_SET(reg, mask)                   reg |= (mask)  
00015 #define HAL_REG_RESET(reg, mask)                  reg &= ~(mask)  
00016 #define HAL_REG_INVERT(reg, mask)                 reg = ((reg) & ~(mask)) | (~((reg) & (mask))) &  
    (mask)  
00017 #define HAL_REG_SYS_LOCK()                        SYS_WR_PROTECT = 0  
00018 #define HAL_REG_SYS_UNLOCK()                      SYS_WR_PROTECT = 0x7a83  
00019 #define HAL_REG_IF_WRITE(reg, mask, condition)    reg = (condition) ? ((reg) | (mask)) : ((reg) &  
    ~(mask))  
00020  
00021 #endif
```

4.49 include/hal_lksmcu_reset.h 文件参考

此图展示该文件直接或间接的被哪些文件引用了:



函数

- void [HAL_RESET_ResetCore](#) (void)
复位内核
- void [HAL_RESET_ResetRegister](#) (void)
寄存器复位
- void [HAL_RESET_ResetGlobal](#) (void)
全局复位(先复位寄存器再复位内核)

4.49.1 函数说明

4.49.1.1 HAL_RESET_ResetCore()

```
void HAL_RESET_ResetCore (  
    void )
```

复位内核

在文件 [hal_lks32mc07x.reset.c](#) 第 10 行定义.

4.49.1.2 HAL_RESET_ResetGlobal()

```
void HAL_RESET_ResetGlobal (  
    void )
```

全局复位(先复位寄存器再复位内核)

在文件 [hal_lks32mc07x.reset.c](#) 第 29 行定义.

函数调用图:



4.49.1.3 HAL_RESET_ResetRegister()

```
void HAL_RESET_ResetRegister (  
    void )
```

寄存器复位

在文件 [hal_lks32mc07x.reset.c](#) 第 17 行定义.

这是这个函数的调用关系图:



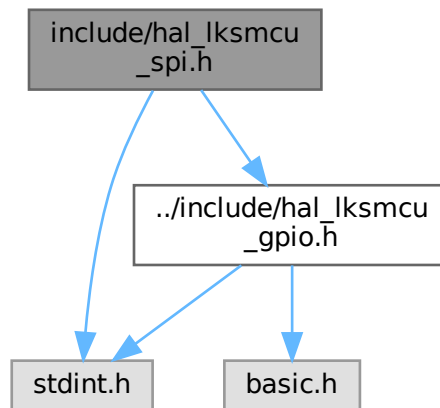
4.50 hal_lksmcu_reset.h

[浏览该文件的文档.](#)

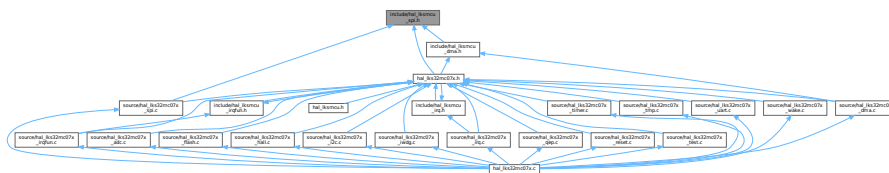
```
00001 #ifndef __HAL_LKSMCU_RESET_H__
00002 #define __HAL_LKSMCU_RESET_H__
00003 void HAL_RESET_ResetCore(void); // 复位内核
00004 void HAL_RESET_ResetRegister(void); // 寄存器复位
00005 void HAL_RESET_ResetGlobal(void); // 全局复位 (先复位寄存器再复位内核)
00006 #endif
```

4.51 include/hal_lksmcu_spi.h 文件参考

```
#include "stdint.h"
#include "../include/hal_lksmcu_gpio.h"
hal_lksmcu_spi.h 的引用(Include)关系图:
```



此图展示该文件直接或间接的被哪些文件引用了:



枚举

- enum `HAL_SPI_x` { `HAL_SPI_0`, `HAL_SPI_1` }

- enum `HAL_SPI_MODE_x` {
`HAL_SPI_MODE_MASTER_LSB_00 = 0` , `HAL_SPI_MODE_MASTER_LSB_01` , `HAL_SPI_MODE_MASTER_LSB_10`
`HAL_SPI_MODE_MASTER_LSB_11` ,
`HAL_SPI_MODE_MASTER_MSB_00` , `HAL_SPI_MODE_MASTER_MSB_01` , `HAL_SPI_MODE_MASTER_MSB_10`
`HAL_SPI_MODE_MASTER_MSB_11` ,
`HAL_SPI_MODE_SLAVE_LSB_00` , `HAL_SPI_MODE_SLAVE_LSB_01` , `HAL_SPI_MODE_SLAVE_LSB_10` ,
`HAL_SPI_MODE_SLAVE_LSB_11` ,
`HAL_SPI_MODE_SLAVE_MSB_00` , `HAL_SPI_MODE_SLAVE_MSB_01` , `HAL_SPI_MODE_SLAVE_MSB_10` ,
`HAL_SPI_MODE_SLAVE_MSB_11` }

函数

- void `HAL_SPI_Reset` (`HAL_SPI_x`)
 模块复位
- void `HAL_SPI_Enable` (`HAL_SPI_x`)
 模块打开
- void `HAL_SPI_Disable` (`HAL_SPI_x`)
 模块关闭
- void `HAL_SPI_SetBaud` (`HAL_SPI_x` spi, uint32_t baud)
 设置波特率
- void `HAL_SPI_SetMode` (`HAL_SPI_x`, `HAL_SPI_MODE_x`)
 设置SPI模式
- void `HAL_SPI_SetGpioCs` (`HAL_SPI_x`, `HAL_GPIO_Px` pin)
 初始化CS引脚
- void `HAL_SPI_SetGpioMosi` (`HAL_SPI_x`, `HAL_GPIO_Px` pin)
 初始化Mosi引脚
- void `HAL_SPI_SetGpioMiso` (`HAL_SPI_x`, `HAL_GPIO_Px` pin)
 初始化Miso引脚
- void `HAL_SPI_SetGpioClk` (`HAL_SPI_x`, `HAL_GPIO_Px` pin)
 初始化Clk引脚
- void `HAL_SPI_SetGpioDi` (`HAL_SPI_x`, `HAL_GPIO_Px` pin)
 初始化SPI的输入引脚
- void `HAL_SPI_SetGpioDo` (`HAL_SPI_x`, `HAL_GPIO_Px` pin)
 初始化SPI的输出引脚
- uint8_t `HAL_SPI_TxData` (`HAL_SPI_x`, uint8_t *data, uint32_t len)
 发送数据
- uint8_t `HAL_SPI_RxData` (`HAL_SPI_x`, uint8_t *data, uint32_t len)
 接收数据
- uint8_t `HAL_SPI_RxTxData` (`HAL_SPI_x`, uint8_t *rxdata, uint8_t *txdata, uint32_t len)
 发送和接收数据
- uint8_t `HAL_SPI_TxDataWidth16` (`HAL_SPI_x`, uint16_t *data, uint32_t len)
 发送数据
- uint8_t `HAL_SPI_RxDataWidth16` (`HAL_SPI_x`, uint16_t *data, uint32_t len)
 接收数据
- uint8_t `HAL_SPI_RxTxDataWidth16` (`HAL_SPI_x`, uint16_t *rxdata, uint16_t *txdata, uint32_t len)
 发送和接收数据

4.51.1 枚举类型说明

4.51.1.1 HAL_SPI_MODE_x

enum `HAL_SPI_MODE_x`

枚举值

HAL_SPI_MODE_MASTER_LSB_00	主模式 低位先发 CPOL0 CPHA0
HAL_SPI_MODE_MASTER_LSB_01	主模式 低位先发 CPOL0 CPHA1
HAL_SPI_MODE_MASTER_LSB_10	主模式 低位先发 CPOL1 CPHA0
HAL_SPI_MODE_MASTER_LSB_11	主模式 低位先发 CPOL1 CPHA1
HAL_SPI_MODE_MASTER_MSB_00	主模式 高位先发 CPOL0 CPHA0
HAL_SPI_MODE_MASTER_MSB_01	主模式 高位先发 CPOL0 CPHA1
HAL_SPI_MODE_MASTER_MSB_10	主模式 高位先发 CPOL1 CPHA0
HAL_SPI_MODE_MASTER_MSB_11	主模式 高位先发 CPOL1 CPHA1
HAL_SPI_MODE_SLAVE_LSB_00	从模式 低位先发 CPOL0 CPHA0
HAL_SPI_MODE_SLAVE_LSB_01	从模式 低位先发 CPOL0 CPHA1
HAL_SPI_MODE_SLAVE_LSB_10	从模式 低位先发 CPOL1 CPHA0
HAL_SPI_MODE_SLAVE_LSB_11	从模式 低位先发 CPOL1 CPHA1
HAL_SPI_MODE_SLAVE_MSB_00	从模式 高位先发 CPOL0 CPHA0
HAL_SPI_MODE_SLAVE_MSB_01	从模式 高位先发 CPOL0 CPHA1
HAL_SPI_MODE_SLAVE_MSB_10	从模式 高位先发 CPOL1 CPHA0
HAL_SPI_MODE_SLAVE_MSB_11	从模式 高位先发 CPOL1 CPHA1

在文件 [hal_lksmcu_spi.h](#) 第 10 行定义.

4.51.1.2 HAL_SPI_x

enum [HAL_SPI_x](#)

枚举值

HAL_↔ SPI_0	
HAL_↔ SPI_1	

在文件 [hal_lksmcu_spi.h](#) 第 5 行定义.

4.51.2 函数说明

4.51.2.1 HAL_SPI_Disable()

```
void HAL_SPI_Disable (  
    HAL\_SPI\_x spi )
```

模块关闭

参数

<i>spi</i>	SPI模块号
------------	--------

在文件 [hal_lks32mc07x_spi.c](#) 第 37 行定义.

4.51.2.2 HAL_SPI_Enable()

```
void HAL_SPI_Enable (
    HAL_SPI_x spi )
```

模块打开

参数

<i>spi</i>	SPI模块号
------------	--------

在文件 [hal_lks32mc07x_spi.c](#) 第 25 行定义.

4.51.2.3 HAL_SPI_Reset()

```
void HAL_SPI_Reset (
    HAL_SPI_x spi )
```

模块复位

参数

<i>spi</i>	SPI模块号
------------	--------

在文件 [hal_lks32mc07x_spi.c](#) 第 12 行定义.

4.51.2.4 HAL_SPI_RxData()

```
uint8_t HAL_SPI_RxData (
    HAL_SPI_x spi,
    uint8_t * data,
    uint32_t len )
```

接收数据

参数

<i>spi</i>	SPI模块号
<i>data</i>	接收缓冲区
<i>len</i>	数据长度

返回

接收状态

在文件 [hal_lks32mc07x_spi.c](#) 第 255 行定义.

4.51.2.5 HAL_SPI_RxDataWidth16()

```
uint8_t HAL_SPI_RxDataWidth16 (
    HAL_SPI_TypeDef * spi,
    uint16_t * data,
    uint32_t len )
```

接收数据

参数

<i>spi</i>	SPI模块号
<i>data</i>	接收缓冲区
<i>len</i>	数据长度

返回

接收状态

在文件 [hal_lks32mc07x_spi.c](#) 第 330 行定义.

4.51.2.6 HAL_SPI_RxTxData()

```
uint8_t HAL_SPI_RxTxData (
    HAL_SPI_TypeDef * spi,
    uint8_t * rxdata,
    uint8_t * txdata,
    uint32_t len )
```

发送和接收数据

参数

<i>spi</i>	SPI模块号
<i>rxdata</i>	接收缓冲区
<i>txdata</i>	要发送的数据
<i>len</i>	数据长度

返回

发送和接收状态

在文件 [hal_lks32mc07x_spi.c](#) 第 282 行定义.

4.51.2.7 HAL_SPI_RxTxDataWidth16()

```
uint8_t HAL_SPI_RxTxDataWidth16 (
    HAL_SPI_TypeDef * spi,
```

```
uint16_t * rxdata,  
uint16_t * txdata,  
uint32_t len )
```

发送和接收数据

参数

<i>spi</i>	SPI模块号
<i>rxdata</i>	接收缓冲区
<i>txdata</i>	要发送的数据
<i>len</i>	数据长度

返回

发送和接收状态

在文件 [hal_lks32mc07x_spi.c](#) 第 355 行定义.

4.51.2.8 HAL_SPI_SetBaud()

```
void HAL_SPI_SetBaud (   
    HAL_SPI_t spi,   
    uint32_t baud )
```

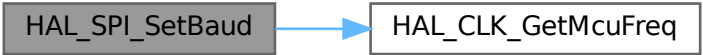
设置波特率

参数

<i>spi</i>	SPI模块号
------------	--------

在文件 [hal_lks32mc07x_spi.c](#) 第 49 行定义.

函数调用图:



4.51.2.9 HAL_SPI_SetGpioClk()

```
void HAL_SPI_SetGpioClk (   
    HAL_SPI_t spi,   
    HAL_GPIO_t pin )
```

初始化Cik引脚

参数

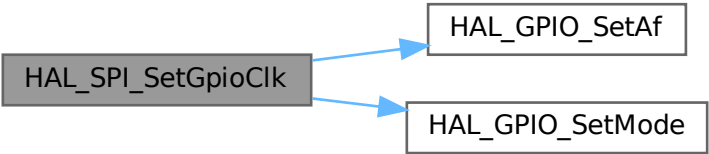
<i>spi</i>	SPI模块号
<i>pin</i>	GPIO引脚

参见

需要先初始化SPI

在文件 [hal_lks32mc07x_spi.c](#) 第 186 行定义.

函数调用图:



4.51.2.10 HAL_SPI.SetGpioCs()

```
void HAL_SPI_SetGpioCs (
    HAL_SPI_TypeDef* spi,
    HAL_GPIO_TypeDef* pin )
```

初始化CS引脚

参数

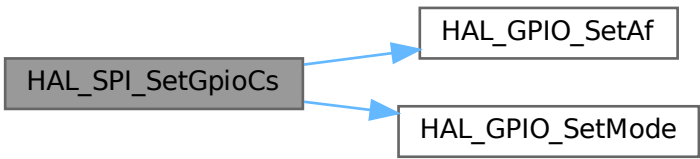
<i>spi</i>	SPI模块号
<i>pin</i>	GPIO引脚

参见

需要先初始化SPI

在文件 [hal_lks32mc07x_spi.c](#) 第 123 行定义.

函数调用图:



4.51.2.11 HAL_SPI.SetGpioDi()

```
void HAL_SPI_SetGpioDi (
    HAL_SPI_x spi,
    HAL_GPIO_Px pin )
```

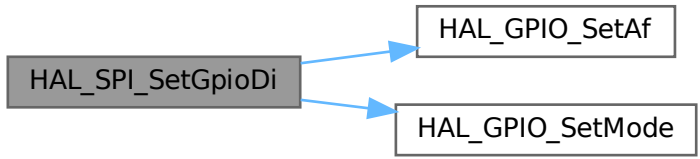
初始化SPI的输入引脚

参数

<i>spi</i>	SPI模块号
<i>pin</i>	GPIO引脚

在文件 [hal_lks32mc07x_spi.c](#) 第 206 行定义.

函数调用图:



4.51.2.12 HAL_SPI.SetGpioDo()

```
void HAL_SPI_SetGpioDo (
    HAL_SPI_x spi,
    HAL_GPIO_Px pin )
```

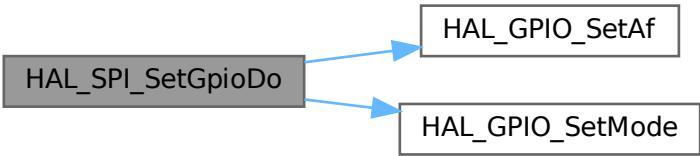
初始化SPI的输出引脚

参数

<i>spi</i>	SPI模块号
<i>pin</i>	GPIO引脚

在文件 [hal_lks32mc07x_spi.c](#) 第 217 行定义.

函数调用图:



4.51.2.13 HAL_SPI_SetGpioMiso()

```
void HAL_SPI_SetGpioMiso (
    HAL_SPI_x spi,
    HAL_GPIO_Px pin )
```

初始化Miso引脚

参数

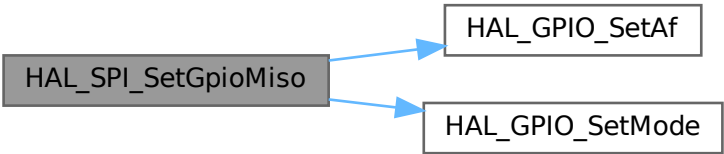
<i>spi</i>	SPI模块号
<i>pin</i>	GPIO引脚

参见

需要先初始化SPI

在文件 [hal_lks32mc07x_spi.c](#) 第 165 行定义.

函数调用图:



4.51.2.14 HAL_SPI.SetGpioMosi()

```
void HAL_SPI_SetGpioMosi (
    HAL_SPI_TypeDef* spi,
    GPIO_TypeDef* pin )
```

初始化Mosi引脚

参数

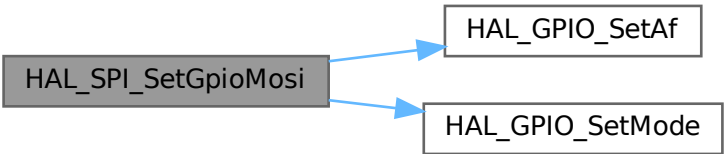
<i>spi</i>	SPI模块号
<i>pin</i>	GPIO引脚

参见

需要先初始化SPI

在文件 [hal_lks32mc07x_spi.c](#) 第 144 行定义.

函数调用图:



4.51.2.15 HAL_SPI_SetMode()

```
void HAL_SPI_SetMode (
    HAL_SPI_TypeDef* spi,
    uint16_t mode )
```

设置SPI模式

参数

<i>spi</i>	SPI模块号
<i>mode</i>	SPI模式

在文件 [hal_lks32mc07x_spi.c](#) 第 60 行定义.

4.51.2.16 HAL_SPI_TxData()

```
uint8_t HAL_SPI_TxData (
    HAL_SPI_TypeDef* spi,
    uint8_t * data,
    uint32_t len )
```

发送数据

参数

<i>spi</i>	SPI模块号
<i>data</i>	要发送的数据
<i>len</i>	数据长度

返回

发送状态

在文件 [hal_lks32mc07x_spi.c](#) 第 231 行定义.

4.51.2.17 HAL_SPI_TxDataWidth16()

```
uint8_t HAL_SPI_TxDataWidth16 (
    HAL_SPI_TypeDef* spi,
    uint16_t * data,
    uint32_t len )
```

发送数据

参数

<i>spi</i>	SPI模块号
<i>data</i>	要发送的数据

返回

发送状态

在文件 `hal_lks32mc07x_spi.c` 第 307 行定义.

4.52 hal_lksmcu_spi.h

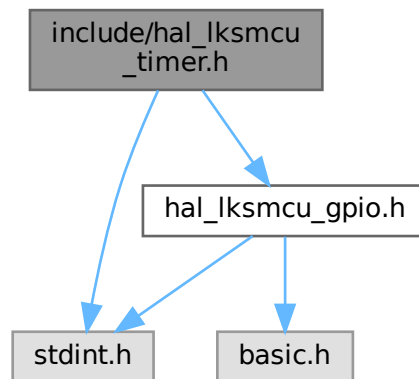
浏览该文件的文档.

```
00001 #ifndef __HAL_LKSMCU_SPI_H__
00002 #define __HAL_LKSMCU_SPI_H__
00003 #include "stdint.h"
00004 #include "../include/hal_lksmcu_gpio.h"
00005 typedef enum
00006 {
00007     HAL_SPI_0,
00008     HAL_SPI_1,
00009 } HAL_SPI_x;
00010 typedef enum
00011 {
00012     HAL_SPI_MODE_MASTER_LSB_00 = 0,
00013     HAL_SPI_MODE_MASTER_LSB_01,
00014     HAL_SPI_MODE_MASTER_LSB_10,
00015     HAL_SPI_MODE_MASTER_LSB_11,
00016     HAL_SPI_MODE_MASTER_MSB_00,
00017     HAL_SPI_MODE_MASTER_MSB_01,
00018     HAL_SPI_MODE_MASTER_MSB_10,
00019     HAL_SPI_MODE_MASTER_MSB_11,
00020     HAL_SPI_MODE_SLAVE_LSB_00,
00021     HAL_SPI_MODE_SLAVE_LSB_01,
00022     HAL_SPI_MODE_SLAVE_LSB_10,
00023     HAL_SPI_MODE_SLAVE_LSB_11,
00024     HAL_SPI_MODE_SLAVE_MSB_00,
00025     HAL_SPI_MODE_SLAVE_MSB_01,
00026     HAL_SPI_MODE_SLAVE_MSB_10,
00027     HAL_SPI_MODE_SLAVE_MSB_11,
00028 } HAL_SPI_MODE_x;
00029 // 初始化
00030 void HAL_SPI_Reset(HAL_SPI_x); // 模块复位
00031 void HAL_SPI_Enable(HAL_SPI_x); // 模块打开
00032 void HAL_SPI_Disable(HAL_SPI_x); // 模块关闭
00033
00034 void HAL_SPI_SetBaud(HAL_SPI_x spi, uint32_t baud);
00035 void HAL_SPI_SetMode(HAL_SPI_x, HAL_SPI_MODE_x);
00036
00037 void HAL_SPI_SetGpioCs(HAL_SPI_x, HAL_GPIO_Px pin);
00038 void HAL_SPI_SetGpioMosi(HAL_SPI_x, HAL_GPIO_Px pin);
00039 void HAL_SPI_SetGpioMiso(HAL_SPI_x, HAL_GPIO_Px pin);
00040 void HAL_SPI_SetGpioClk(HAL_SPI_x, HAL_GPIO_Px pin);
00041
00042 void HAL_SPI_SetGpioDi(HAL_SPI_x, HAL_GPIO_Px pin); // 不自动识别模式, 直接指定输入或输出状态
00043 void HAL_SPI_SetGpioDo(HAL_SPI_x, HAL_GPIO_Px pin); // 不自动识别模式, 直接指定输入或输出状态
00044
00045 uint8_t HAL_SPI_TxData(HAL_SPI_x, uint8_t *data, uint32_t len);
00046 uint8_t HAL_SPI_RxData(HAL_SPI_x, uint8_t *data, uint32_t len);
00047 uint8_t HAL_SPI_RxTxData(HAL_SPI_x, uint8_t *rxdata, uint8_t *txdata, uint32_t len);
00048 uint8_t HAL_SPI_TxDataWidth16(HAL_SPI_x, uint16_t *data, uint32_t len);
00049 uint8_t HAL_SPI_RxDataWidth16(HAL_SPI_x, uint16_t *data, uint32_t len);
00050 uint8_t HAL_SPI_RxTxDataWidth16(HAL_SPI_x, uint16_t *rxdata, uint16_t *txdata, uint32_t len);
00051 #endif
```

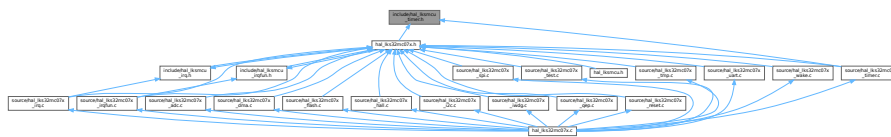
4.53 include/hal_lksmcu_timer.h 文件参考

```
#include "stdint.h"
#include "hal_lksmcu_gpio.h"
```

hal_lksmcu_timer.h 的引用(Include)关系图:



此图展示该文件直接或间接的被哪些文件引用了:



枚举

- enum `HAL_TIMER_x` {
`HAL_TIMER_0` , `HAL_TIMER_1` , `HAL_TIMER_2` , `HAL_TIMER_3` ,
`HAL_TIMER_4` }
- enum `HAL_TIMER_MODE_x` { `HAL_TIMER_MODE_COUNT` , `HAL_TIMER_MODE_PWMOUT` , `HAL_TIMER_MODE_PWMIN` }

函数

- void `HAL_TIMER_Reset` (`HAL_TIMER_x`)
- void `HAL_TIMER_Enable` (`HAL_TIMER_x`)
- void `HAL_TIMER_Disable` (`HAL_TIMER_x`)
- void `HAL_TIMER_SetPeriodCount` (`HAL_TIMER_x`, `uint32_t count`)
 设置定时器周期
- void `HAL_TIMER_SetFreq` (`HAL_TIMER_x`, `uint32_t freq`)
 设置定时器频率
- void `HAL_TIMER_SetPeriodms` (`HAL_TIMER_x`, `uint32_t ms`)
 设置定时器周期(*ms*)
- void `HAL_TIMER_SetPeriodus` (`HAL_TIMER_x`, `uint32_t us`)
 设置定时器周期(*us*)
- `uint8_t HAL_TIMER_IrqOverflow` (`HAL_TIMER_x`)

- void [HAL_TIMER_IrqOverflowEnable](#) ([HAL_TIMER_x](#))
- void [HAL_TIMER_IrqOverflowDisable](#) ([HAL_TIMER_x](#))
- void [HAL_TIMER_IrqOverflowClear](#) ([HAL_TIMER_x](#))
- uint8_t [HAL_TIMER_IrqCompare0](#) ([HAL_TIMER_x](#))
- void [HAL_TIMER_IrqCompare0Enable](#) ([HAL_TIMER_x](#))
- void [HAL_TIMER_IrqCompare0Disable](#) ([HAL_TIMER_x](#))
- void [HAL_TIMER_IrqCompare0Clear](#) ([HAL_TIMER_x](#))
- uint8_t [HAL_TIMER_IrqCompare1](#) ([HAL_TIMER_x](#))
- void [HAL_TIMER_IrqCompare1Enable](#) ([HAL_TIMER_x](#))
- void [HAL_TIMER_IrqCompare1Disable](#) ([HAL_TIMER_x](#))
- void [HAL_TIMER_IrqCompare1Clear](#) ([HAL_TIMER_x](#))
- uint32_t [HAL_TIMER_GetFreq](#) ([HAL_TIMER_x](#))
- uint32_t [HAL_TIMER_GetThCnt](#) ([HAL_TIMER_x](#))
- void [HAL_TIMER_StartCounting](#) ([HAL_TIMER_x](#))
- void [HAL_TIMER_StopCounting](#) ([HAL_TIMER_x](#))
- void [HAL_TIMER_SetPwm](#) ([HAL_TIMER_x](#) timer_x, int32_t t0, int32_t t1)
- void [HAL_TIMER_SetPwmQ15](#) ([HAL_TIMER_x](#) timer_x, int16_t t0, int16_t t1)
- void [HAL_TIMER_SetWorkMode](#) ([HAL_TIMER_x](#) timer_x, [HAL_TIMER_MODE_x](#) mode_x)
- void [HAL_TIMER_SetGpioOut](#) ([HAL_TIMER_x](#) timer_x, [HAL_GPIO_Px](#) px)
设置定时器输出 *gpio*
- void [HAL_TIMER_SetGpioIn](#) ([HAL_TIMER_x](#) timer_x, [HAL_GPIO_Px](#) px)
设置定时器输入 *gpio*

4.53.1 枚举类型说明

4.53.1.1 HAL_TIMER_MODE_x

enum [HAL_TIMER_MODE_x](#)

枚举值

HAL_TIMER_MODE_COUNT	计数模式
HAL_TIMER_MODE_PWMOUT	PWM输出模式
HAL_TIMER_MODE_PWMIN	PWM输入模式

在文件 [hal_lksmcu_timer.h](#) 第 14 行定义.

4.53.1.2 HAL_TIMER_x

enum [HAL_TIMER_x](#)

枚举值

HAL_↔ TIMER_0	TIMER模块0
HAL_↔ TIMER_1	TIMER模块1
HAL_↔ TIMER_2	TIMER模块2

枚举值

HAL_↔ TIMER_3	TIMER模块3
HAL_↔ TIMER_4	TIMER模块3

在文件 [hal_lksmcu_timer.h](#) 第 6 行定义.

4.53.2 函数说明

4.53.2.1 HAL_TIMER_Disable()

```
void HAL_TIMER_Disable (
    HAL_TIMER_x timer_x )
```

在文件 [hal_lks32mc07x_timer.c](#) 第 63 行定义.

4.53.2.2 HAL_TIMER_Enable()

```
void HAL_TIMER_Enable (
    HAL_TIMER_x timer_x )
```

在文件 [hal_lks32mc07x_timer.c](#) 第 38 行定义.

这是这个函数的调用关系图:



4.53.2.3 HAL_TIMER_GetFreq()

```
uint32_t HAL_TIMER_GetFreq (
    HAL_TIMER_x timer_x )
```

在文件 [hal_lks32mc07x_timer.c](#) 第 167 行定义.

函数调用图:



4.53.2.4 HAL_TIMER_GetThCnt()

```
uint32_t HAL_TIMER_GetThCnt (
    HAL_TIMER_x timer_x )
```

在文件 [hal_lks32mc07x.timer.c](#) 第 174 行定义.

这是这个函数的调用关系图:



4.53.2.5 HAL_TIMER_IrqCompare0()

```
uint8_t HAL_TIMER_IrqCompare0 (
    HAL_TIMER_x timer_x )
```

在文件 [hal_lks32mc07x.timer.c](#) 第 250 行定义.

4.53.2.6 HAL_TIMER_IrqCompare0Clear()

```
void HAL_TIMER_IrqCompare0Clear (
    HAL_TIMER_x timer_x )
```

在文件 [hal_lks32mc07x.timer.c](#) 第 272 行定义.

4.53.2.7 HAL_TIMER_IrqCompare0Disable()

```
void HAL_TIMER_IrqCompare0Disable (
    HAL_TIMER_x timer_x )
```

在文件 [hal_lks32mc07x.timer.c](#) 第 268 行定义.

4.53.2.8 HAL_TIMER_IrqCompare0Enable()

```
void HAL_TIMER_IrqCompare0Enable (
    HAL_TIMER_x timer_x )
```

在文件 [hal_lks32mc07x.timer.c](#) 第 264 行定义.

4.53.2.9 HAL_TIMER_IrqCompare1()

```
uint8_t HAL_TIMER_IrqCompare1 (
    HAL_TIMER_x timer_x )
```

在文件 [hal_lks32mc07x_timer.c](#) 第 277 行定义.

4.53.2.10 HAL_TIMER_IrqCompare1Clear()

```
void HAL_TIMER_IrqCompare1Clear (
    HAL_TIMER_x timer_x )
```

在文件 [hal_lks32mc07x_timer.c](#) 第 299 行定义.

4.53.2.11 HAL_TIMER_IrqCompare1Disable()

```
void HAL_TIMER_IrqCompare1Disable (
    HAL_TIMER_x timer_x )
```

在文件 [hal_lks32mc07x_timer.c](#) 第 295 行定义.

4.53.2.12 HAL_TIMER_IrqCompare1Enable()

```
void HAL_TIMER_IrqCompare1Enable (
    HAL_TIMER_x timer_x )
```

在文件 [hal_lks32mc07x_timer.c](#) 第 291 行定义.

4.53.2.13 HAL_TIMER_IrqOverflow()

```
uint8_t HAL_TIMER_IrqOverflow (
    HAL_TIMER_x timer_x )
```

在文件 [hal_lks32mc07x_timer.c](#) 第 223 行定义.

4.53.2.14 HAL_TIMER_IrqOverflowClear()

```
void HAL_TIMER_IrqOverflowClear (
    HAL_TIMER_x timer_x )
```

在文件 [hal_lks32mc07x_timer.c](#) 第 245 行定义.

4.53.2.15 HAL_TIMER_IrqOverflowDisable()

```
void HAL_TIMER_IrqOverflowDisable (
    HAL_TIMER_x timer_x )
```

在文件 [hal_lks32mc07x_timer.c](#) 第 241 行定义.

4.53.2.16 HAL_TIMER_IrqOverflowEnable()

```
void HAL_TIMER_IrqOverflowEnable (  
    HAL_TIMER_x timer_x )
```

在文件 `hal_lks32mc07x.timer.c` 第 237 行定义.

4.53.2.17 HAL_TIMER_Reset()

```
void HAL_TIMER_Reset (  
    HAL_TIMER_x timer_x )
```

在文件 `hal_lks32mc07x.timer.c` 第 10 行定义.

4.53.2.18 HAL_TIMER_SetFreq()

```
void HAL_TIMER_SetFreq (  
    HAL_TIMER_x timer_x,  
    uint32_t freq )
```

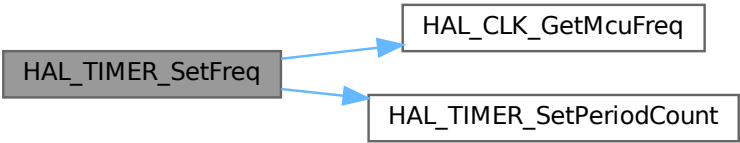
设置定时器频率

参数

<i>timer_x</i>	定时器编号
<i>freq</i>	定时器频率

在文件 `hal_lks32mc07x.timer.c` 第 137 行定义.

函数调用图:



4.53.2.19 HAL_TIMER_SetGpioIn()

```
void HAL_TIMER_SetGpioIn (  
    HAL_TIMER_x timer_x,  
    HAL_GPIO_Px px )
```

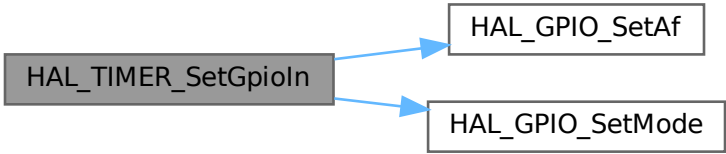
设置定时器输入gpio

参数

<i>timer</i> ↔ _x	定时器编号
<i>px</i>	gpio编号

在文件 [hal_lks32mc07x_timer.c](#) 第 338 行定义.

函数调用图:



4.53.2.20 HAL_TIMER_SetGpioOut()

```
void HAL_TIMER_SetGpioOut (
    HAL_TIMER_x timer_x,
    HAL_GPIO_Px px )
```

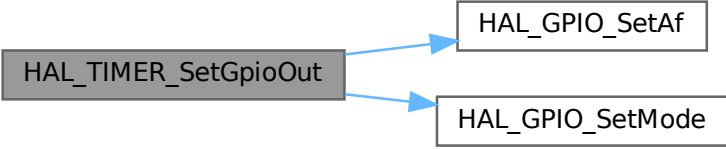
设置定时器输出gpio

参数

<i>timer</i> ↔ _x	定时器编号
<i>px</i>	gpio编号

在文件 [hal_lks32mc07x_timer.c](#) 第 309 行定义.

函数调用图:



4.53.2.21 HAL_TIMER_SetPeriodCount()

```
void HAL_TIMER_SetPeriodCount (
    HAL_TIMER_x timer_x,
    uint32_t count )
```

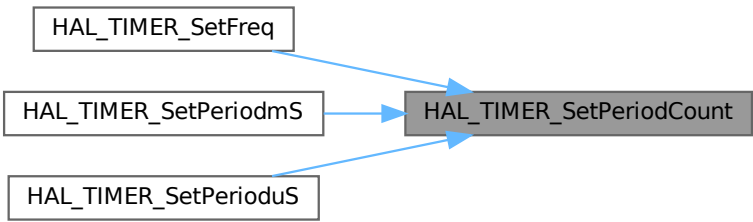
设置定时器周期

参数

<i>timer_x</i>	定时器编号
<i>count</i>	定时器周期

在文件 [hal_lks32mc07x_timer.c](#) 第 92 行定义.

这是这个函数的调用关系图:



4.53.2.22 HAL_TIMER_SetPeriodmS()

```
void HAL_TIMER_SetPeriodmS (
    HAL_TIMER_x timer_x,
    uint32_t ms )
```

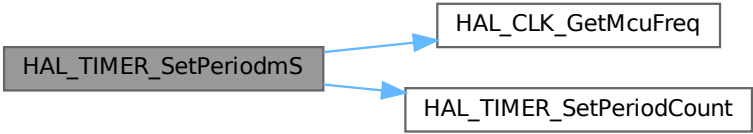
设置定时器周期(ms)

参数

<i>timer_x</i>	定时器编号
<i>ms</i>	定时器周期(ms)

在文件 [hal_lks32mc07x_timer.c](#) 第 149 行定义.

函数调用图:



4.53.2.23 HAL_TIMER_SetPerioduS()

```
void HAL_TIMER_SetPerioduS (
    HAL_TIMER_x timer_x,
    uint32_t us )
```

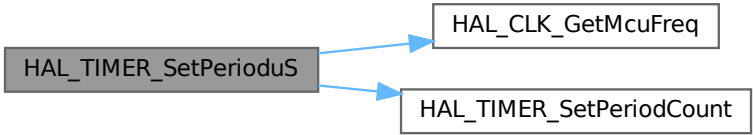
设置定时器周期(us)

参数

<i>timer_x</i>	定时器编号
<i>us</i>	定时器周期(us)

在文件 [hal_lks32mc07x.timer.c](#) 第 160 行定义.

函数调用图:



4.53.2.24 HAL_TIMER_SetPwm()

```
void HAL_TIMER_SetPwm (
    HAL_TIMER_x timer_x,
    int32_t t0,
    int32_t t1 )
```

在文件 [hal_lks32mc07x.timer.c](#) 第 190 行定义.

4.53.2.25 HAL_TIMER_SetPwmQ15()

```
void HAL_TIMER_SetPwmQ15 (
    HAL_TIMER_x timer_x,
    int16_t t0,
    int16_t t1 )
```

在文件 [hal_lks32mc07x.timer.c](#) 第 196 行定义.

函数调用图:



4.53.2.26 HAL_TIMER_SetWorkMode()

```
void HAL_TIMER_SetWorkMode (
    HAL_TIMER_x timer_x,
    HAL_TIMER_MODE_x mode_x )
```

< 计数模式

< PWM输出模式

< PWM输入模式

在文件 [hal_lks32mc07x.timer.c](#) 第 205 行定义.

4.53.2.27 HAL_TIMER_StartCounting()

```
void HAL_TIMER_StartCounting (
    HAL_TIMER_x timer_x )
```

在文件 [hal_lks32mc07x.timer.c](#) 第 179 行定义.

4.53.2.28 HAL_TIMER_StopCounting()

```
void HAL_TIMER_StopCounting (
    HAL_TIMER_x timer_x )
```

在文件 [hal_lks32mc07x.timer.c](#) 第 184 行定义.

4.54 hal_lksmcu_timer.h

浏览该文件的文档.

```

00001 #ifndef __HAL_LKSMCU_TIMER_H__
00002 #define __HAL_LKSMCU_TIMER_H__
00003 #include "stdint.h"
00004 #include "hal_lksmcu_gpio.h"
00005
00006 typedef enum
00007 {
00008     HAL_TIMER_0,
00009     HAL_TIMER_1,
00010     HAL_TIMER_2,
00011     HAL_TIMER_3,
00012     HAL_TIMER_4,
00013 } HAL_TIMER_x;
00014 typedef enum
00015 {
00016     HAL_TIMER_MODE_COUNT,
00017     HAL_TIMER_MODE_PWMOUT,
00018     HAL_TIMER_MODE_PWMIN,
00019 } HAL_TIMER_MODE_x;
00020
00021 void HAL_TIMER_Reset(HAL_TIMER_x); // 模块复位
00022 void HAL_TIMER_Enable(HAL_TIMER_x); // 模块打开
00023 void HAL_TIMER_Disable(HAL_TIMER_x); // 模块关闭
00024
00025 void HAL_TIMER_SetPeriodCount(HAL_TIMER_x, uint32_t count); // 设置周期(计数值)
00026 void HAL_TIMER_SetFreq(HAL_TIMER_x, uint32_t freq); // 设置频率
00027 void HAL_TIMER_SetPeriodms(HAL_TIMER_x, uint32_t ms); // 设置周期(ms)
00028 void HAL_TIMER_SetPeriodus(HAL_TIMER_x, uint32_t us); // 设置周期(us)
00029
00030 uint8_t HAL_TIMER_IrqOverflow(HAL_TIMER_x);
00031 void HAL_TIMER_IrqOverflowEnable(HAL_TIMER_x);
00032 void HAL_TIMER_IrqOverflowDisable(HAL_TIMER_x);
00033 void HAL_TIMER_IrqOverflowClear(HAL_TIMER_x);
00034
00035 uint8_t HAL_TIMER_IrqCompare0(HAL_TIMER_x);
00036 void HAL_TIMER_IrqCompare0Enable(HAL_TIMER_x);
00037 void HAL_TIMER_IrqCompare0Disable(HAL_TIMER_x);
00038 void HAL_TIMER_IrqCompare0Clear(HAL_TIMER_x);
00039
00040 uint8_t HAL_TIMER_IrqCompare1(HAL_TIMER_x);
00041 void HAL_TIMER_IrqCompare1Enable(HAL_TIMER_x);
00042 void HAL_TIMER_IrqCompare1Disable(HAL_TIMER_x);
00043 void HAL_TIMER_IrqCompare1Clear(HAL_TIMER_x);
00044
00045 uint32_t HAL_TIMER_GetFreq(HAL_TIMER_x); // 获取当前的pwm频率
00046 uint32_t HAL_TIMER_GetThCnt(HAL_TIMER_x); // 获取计数器最大值
00047 void HAL_TIMER_StartCounting(HAL_TIMER_x); // 开始计数
00048 void HAL_TIMER_StopCounting(HAL_TIMER_x); // 停止计数(同时计数器回到0)
00049 void HAL_TIMER_SetPwm(HAL_TIMER_x timer_x, int32_t t0, int32_t t1); // 设置pwm波形 仅PWM模式下有效
00050 void HAL_TIMER_SetPwmQ15(HAL_TIMER_x timer_x, int16_t t0, int16_t t1); // 设置pwm波形q15格式输入 仅PWM模
    式下有效
00051 void HAL_TIMER_SetWorkMode(HAL_TIMER_x timer_x, HAL_TIMER_MODE_x mode_x); // 设置工作模式
00052
00053 void HAL_TIMER_SetGpioOut(HAL_TIMER_x timer_x, HAL_GPIO_Px px);
00054 void HAL_TIMER_SetGpioIn(HAL_TIMER_x timer_x, HAL_GPIO_Px px);
00055 #endif

```

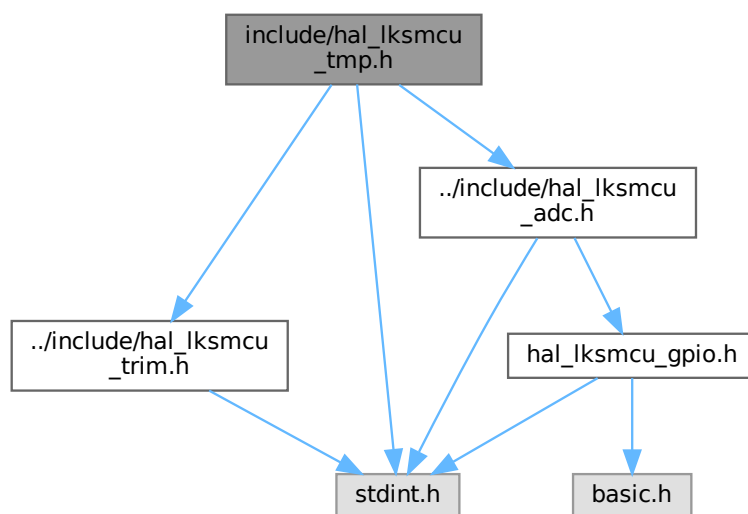
4.55 include/hal_lksmcu_tmp.h 文件参考

```

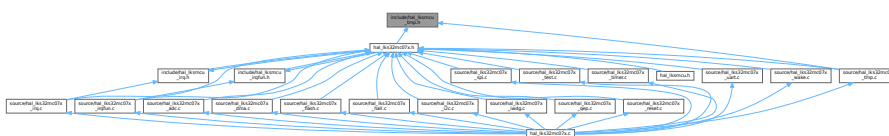
#include "../include/hal_lksmcu_trim.h"
#include "../include/hal_lksmcu_adc.h"
#include "stdint.h"

```


hal_lksmcu_tmp.h 的引用(Include)关系图:



此图展示该文件直接或间接的被哪些文件引用了:



枚举

- enum `HAL_TMP_x` { `HAL_TMP_0` , `HAL_TMP_1` }

函数

- void `HAL_TMP_Reset` (`HAL_TMP_x`)
模块复位
- void `HAL_TMP_Enable` (`HAL_TMP_x`)
模块打开
- void `HAL_TMP_Disable` (`HAL_TMP_x`)
模块关闭
- int32_t `HAL_TMP_GetTemperature` (`HAL_TMP_x`, `HAL_ADC_x`, `HAL_ADC_SAMP_SEQUENCE_x`)
获取当前芯片内部温度传感器测到的温度,单位摄氏度
- int32_t `HAL_TMP_GetTemperature01` (`HAL_TMP_x`, `HAL_ADC_x`, `HAL_ADC_SAMP_SEQUENCE_x`)
获取当前芯片内部温度传感器测到的温度,单位0.1摄氏度
- int32_t `HAL_TMP_Adc2Temperature` (`HAL_TMP_x`, int32_t adc_value)
adc值转温度
- int32_t `HAL_TMP_Adc2Temperature01` (`HAL_TMP_x`, int32_t adc_value)
adc值转温度

4.55.1 枚举类型说明

4.55.1.1 HAL_TMP_x

enum HAL_TMP_x

枚举值

HAL_↔ TMP_0	
HAL_↔ TMP_1	

在文件 [hal_lksmcu.tmp.h](#) 第 7 行定义.

4.55.2 函数说明

4.55.2.1 HAL_TMP_Adc2Temperature()

```
int32_t HAL_TMP_Adc2Temperature (
    HAL_TMP_x TMP_x,
    int32_t adc_value )
```

adc值转温度

参数

<i>TMP_x</i>	模块编号
<i>adc_value</i>	ADC值, 左对齐 (-32767 到32767)

返回

当前温度,单位摄氏度

在文件 [hal_lks32mc07x.tmp.c](#) 第 84 行定义.

函数调用图:



这是这个函数的调用关系图:



4.55.2.2 HAL_TMP_Adc2Temperature01()

```
int32_t HAL_TMP_Adc2Temperature01 (
    HAL_TMP_x TMP_x,
    int32_t adc_value )
```

adc值转温度

参数

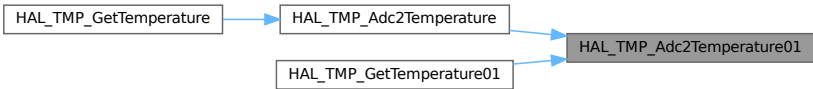
<i>TMP_x</i>	模块编号
<i>adc_value</i>	ADC值, 左对齐 (-32767 到32767)

返回

当前温度,单位0.1摄氏度

在文件 [hal_lks32mc07x.tmp.c](#) 第 94 行定义.

这是这个函数的调用关系图:



4.55.2.3 HAL_TMP_Disable()

```
void HAL_TMP_Disable (
    HAL_TMP_x TMP_x )
```

模块关闭

参数

<i>TMP</i> ↔ _X	模块编号
--------------------	------

在文件 [hal_lks32mc07x.tmp.c](#) 第 37 行定义.

这是这个函数的调用关系图:



4.55.2.4 HAL_TMP.Enable()

```
void HAL_TMP_Enable (
    HAL_TMP_X TMP_X )
```

模块打开

参数

<i>TMP</i> ↔ _X	模块编号
--------------------	------

在文件 [hal_lks32mc07x.tmp.c](#) 第 23 行定义.

函数调用图:



4.55.2.5 HAL_TMP.GetTemperature()

```
int32_t HAL_TMP_GetTemperature (
    HAL_TMP_X TMP_X,
    HAL_ADC_X adc,
    HAL_ADC_SAMP_SEQUENCE_X sequence )
```

获取当前芯片内部温度传感器测到的温度,单位摄氏度

参数

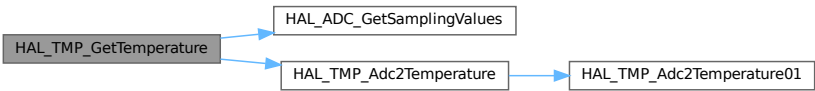
<i>TMP_x</i>	模块编号
<i>adc</i>	ADC编号
<i>sequence</i>	ADC采样序列

返回

当前温度,单位摄氏度

在文件 [hal_lks32mc07x.tmp.c](#) 第 52 行定义.

函数调用图:



4.55.2.6 HAL_TMP_GetTemperature01()

```
int32_t HAL_TMP_GetTemperature01 (
    HAL_TMP_x TMP_x,
    HAL_ADC_x adc,
    HAL_ADC_SAMP_SEQUENCE_x sequence )
```

获取当前芯片内部温度传感器测到的温度,单位0.1摄氏度

参数

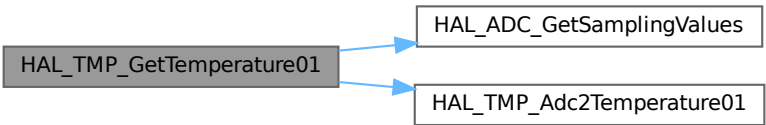
<i>TMP_x</i>	模块编号
<i>adc</i>	ADC编号
<i>sequence</i>	ADC采样序列

返回

当前温度,单位0.1摄氏度

在文件 [hal_lks32mc07x.tmp.c](#) 第 69 行定义.

函数调用图:



4.55.2.7 HAL_TMP_Reset()

```
void HAL_TMP_Reset (
    HAL_TMP_x TMP_x )
```

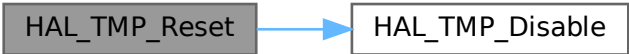
模块复位

参数

<i>TMP</i> ↔	模块编号
<i>_x</i>	

在文件 [hal_lks32mc07x.tmp.c](#) 第 14 行定义.

函数调用图:



4.56 hal_lksmcu.tmp.h

[浏览该文件的文档.](#)

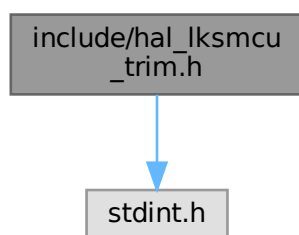
```
00001
00002 #ifndef __HAL_LKSMCU_TMP_H__
00003 #define __HAL_LKSMCU_TMP_H__
00004 #include "../include/hal_lksmcu.trim.h"
00005 #include "../include/hal_lksmcu.adc.h"
00006 #include "stdint.h"
00007 typedef enum
00008 {
00009     HAL_TMP_0,
00010     HAL_TMP_1,
00011 } HAL_TMP_x;
00012 // 初始化
00013 void HAL_TMP_Reset (HAL_TMP_x); // 模块复位
00014 void HAL_TMP_Enable (HAL_TMP_x); // 模块打开
```

```
00015 void HAL_TMP_Disable(HAL_TMP_x); // 模块关闭
00016
00017 int32_t HAL_TMP_GetTemperature(HAL_TMP_x, HAL_ADC_x, HAL_ADC_SAMP_SEQUENCE_x); // 获取当前芯片内部温度传感器
测到的温度,单位摄氏度,需要配置ADC并采样温度传感器通道
00018 int32_t HAL_TMP_GetTemperature01(HAL_TMP_x, HAL_ADC_x, HAL_ADC_SAMP_SEQUENCE_x); // 获取当前芯片内部温度传感器
测到的温度,单位0.1摄氏度,需要配置ADC并采样温度传感器通道
00019
00020 int32_t HAL_TMP_Adc2Temperature(HAL_TMP_x, int32_t adc_value); // 芯片内部温度传感器ADC值转温度 单位度
00021 int32_t HAL_TMP_Adc2Temperature01(HAL_TMP_x, int32_t adc_value); // 芯片内部温度传感器ADC值转温度 单位0.1度
00022 #endif
```

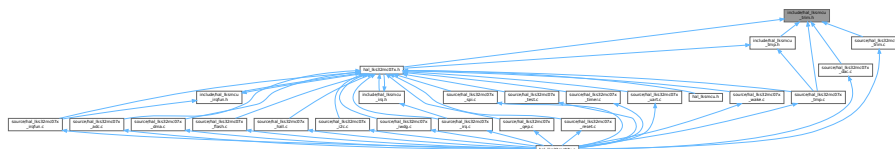
4.57 include/hal_iksmcu_trim.h 文件参考

```
#include "stdint.h"
```

hal_ksmcu_trim.h 的引用(Include)关系图:



此图展示该文件直接或间接的被哪些文件引用了:



函数

- uint32_t **HAL_TRIM_Read** (uint32_t addr)
读取MCU03的校准参数
- uint32_t **HAL_TRIM_ReadId** (uint32_t idx)
读取芯片唯一ID
- uint32_t **HAL_TRIM_ReadIdCrc32** (void)
读取芯片唯一ID的CRC32值

4.57.1 函数说明

4.57.1.1 HAL_TRIM_Read()

```
uint32_t HAL_TRIM_Read (
    uint32_t addr )
```

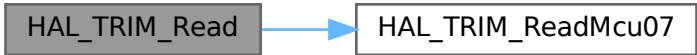
读取MCU03的校准参数

参数

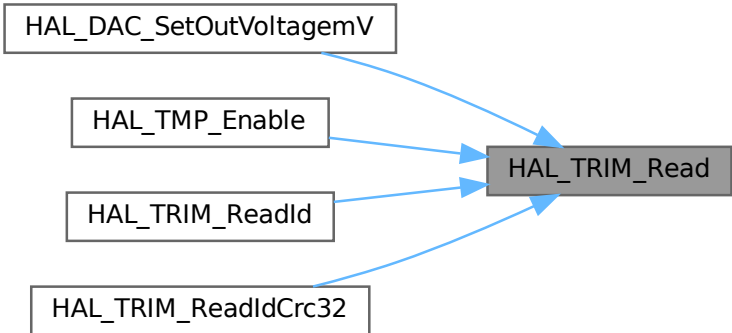
<i>addr</i>	校准参数的地址
-------------	---------

在文件 [hal_lks32mc07x.trim.c](#) 第 68 行定义.

函数调用图:



这是这个函数的调用关系图:



4.57.1.2 HAL_TRIM_ReadId()

```
uint32_t HAL_TRIM_ReadId (
    uint32_t idx )
```

读取芯片唯一ID

参数

<i>idx</i>	0:ID0,1:ID1,2:ID2,3:ID3,4:ID4
------------	-------------------------------

返回
唯一ID

在文件 [hal_lks32mc07x.trim.c](#) 第 78 行定义.

函数调用图:



4.57.1.3 HAL_TRIM_ReadIdCrc32()

```
uint32_t HAL_TRIM_ReadIdCrc32 (
    void )
```

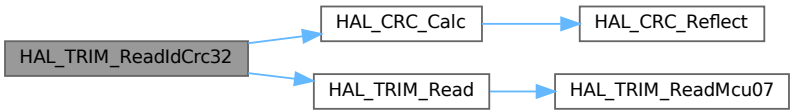
读取芯片唯一ID的CRC32值

返回
唯一ID的CRC32值

注解
相较于HAL_TRIM_ReadId，更方便存储和判断，但理论上存在重复的可能（1/0xffffffff）

在文件 [hal_lks32mc07x.trim.c](#) 第 87 行定义.

函数调用图:



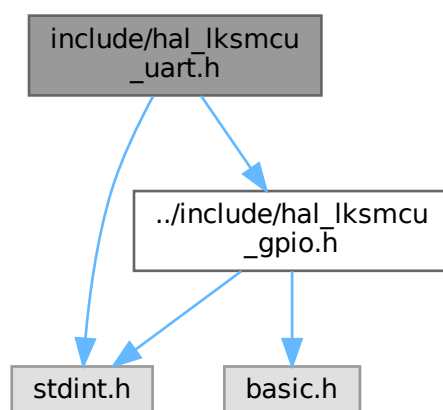
4.58 hal_lksmcu_trim.h

[浏览该文件的文档.](#)

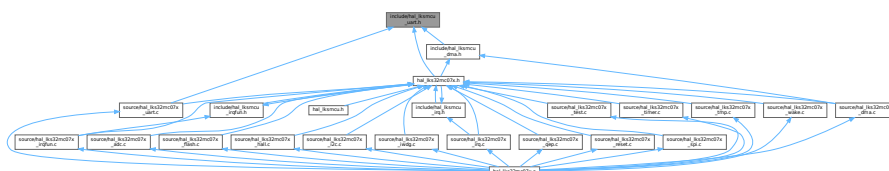
```
00001
00002 #ifndef __HAL_LKSMCU_TRIM_H__
00003 #define __HAL_LKSMCU_TRIM_H__
00004
00005 #include "stdint.h"
00006 uint32_t HAL_TRIM_Read(uint32_t addr);
00007 uint32_t HAL_TRIM_ReadId(uint32_t idx);
00008 uint32_t HAL_TRIM_ReadIdCrc32(void);
00009
00010 #endif
```

4.59 include/hal_lksmcu_uart.h 文件参考

```
#include "stdint.h"
#include "../include/hal_lksmcu_gpio.h"
hal_lksmcu_uart.h 的引用(Include)关系图:
```



此图展示该文件直接或间接的被哪些文件引用了:



枚举

- enum HAL_UART_x { HAL_UART_0 , HAL_UART_1 , HAL_UART_2 }
- enum HAL_UART_MODE_x {
 HAL_UART_MODE_LSB_STOP1_NONE , HAL_UART_MODE_LSB_STOP2_NONE , HAL_UART_MODE_LSB_STOP1_ODD ,
 HAL_UART_MODE_LSB_STOP2_ODD ,
 HAL_UART_MODE_LSB_STOP1_EVEN , HAL_UART_MODE_LSB_STOP2_EVEN , HAL_UART_MODE_MSB_STOP1_NONE ,
 HAL_UART_MODE_MSB_STOP2_NONE ,
 HAL_UART_MODE_MSB_STOP1_ODD , HAL_UART_MODE_MSB_STOP2_ODD , HAL_UART_MODE_MSB_STOP1_EVEN ,
 HAL_UART_MODE_MSB_STOP2_EVEN }

函数

- void HAL_UART_Reset (HAL_UART_x)
- void HAL_UART_Enable (HAL_UART_x)
- void HAL_UART_Disable (HAL_UART_x)

- void `HAL_UART_SetBaud` (`HAL_UART_x`, `uint32_t baud`)
- void `HAL_UART_TxData` (`HAL_UART_x`, `uint8_t *data`, `uint32_t len`)
- void `HAL_UART_SetMode` (`HAL_UART_x`, `HAL_UART_MODE_x`)
- `uint8_t HAL_UART_ReadBuffer` (`HAL_UART_x`)
- void `HAL_UART_WriteBuffer` (`HAL_UART_x`, `uint8_t buff`)
- void `HAL_UART_SetGpioTx` (`HAL_UART_x`, `HAL_GPIO_Px`)
- void `HAL_UART_SetGpioRx` (`HAL_UART_x`, `HAL_GPIO_Px`)
- `uint8_t HAL_UART_IrqSendOver` (`HAL_UART_x`)
- void `HAL_UART_IrqSendOverEnable` (`HAL_UART_x`)
- void `HAL_UART_IrqSendOverDisable` (`HAL_UART_x`)
- void `HAL_UART_IrqSendOverClear` (`HAL_UART_x`)
- `uint8_t HAL_UART_IrqReceiveOver` (`HAL_UART_x`)
- void `HAL_UART_IrqReceiveOverEnable` (`HAL_UART_x`)
- void `HAL_UART_IrqReceiveOverDisable` (`HAL_UART_x`)
- void `HAL_UART_IrqReceiveOverClear` (`HAL_UART_x`)
- `uint8_t HAL_UART_IrqStopErr` (`HAL_UART_x`)
- void `HAL_UART_IrqStopErrEnable` (`HAL_UART_x`)
- void `HAL_UART_IrqStopErrDisable` (`HAL_UART_x`)
- void `HAL_UART_IrqStopErrClear` (`HAL_UART_x`)
- `uint8_t HAL_UART_IrqCheckErr` (`HAL_UART_x`)
- void `HAL_UART_IrqCheckErrEnable` (`HAL_UART_x`)
- void `HAL_UART_IrqCheckErrDisable` (`HAL_UART_x`)
- void `HAL_UART_IrqCheckErrClear` (`HAL_UART_x`)

4.59.1 枚举类型说明

4.59.1.1 HAL_UART_MODE_x

enum `HAL_UART_MODE_x`

枚举值

<code>HAL_UART_MODE_LSB_STOP1_NONE</code>	低位先发 1位停止位 无校验
<code>HAL_UART_MODE_LSB_STOP2_NONE</code>	低位先发 2位停止位 无校验
<code>HAL_UART_MODE_LSB_STOP1_ODD</code>	低位先发 1位停止位 奇校验
<code>HAL_UART_MODE_LSB_STOP2_ODD</code>	低位先发 2位停止位 奇校验
<code>HAL_UART_MODE_LSB_STOP1_EVEN</code>	低位先发 1位停止位 偶校验
<code>HAL_UART_MODE_LSB_STOP2_EVEN</code>	低位先发 2位停止位 偶校验
<code>HAL_UART_MODE_MSB_STOP1_NONE</code>	高位先发 1位停止位 无校验
<code>HAL_UART_MODE_MSB_STOP2_NONE</code>	高位先发 2位停止位 无校验
<code>HAL_UART_MODE_MSB_STOP1_ODD</code>	高位先发 1位停止位 奇校验
<code>HAL_UART_MODE_MSB_STOP2_ODD</code>	高位先发 2位停止位 奇校验
<code>HAL_UART_MODE_MSB_STOP1_EVEN</code>	高位先发 1位停止位 偶校验
<code>HAL_UART_MODE_MSB_STOP2_EVEN</code>	高位先发 2位停止位 偶校验

在文件 `hal_lksmcu_uart.h` 第 12 行定义。

4.59.1.2 HAL_UART_x

enum HAL_UART_x

枚举值

HAL_↔ UART_0	
HAL_↔ UART_1	
HAL_↔ UART_2	

在文件 [hal_lksmcu_uart.h](#) 第 6 行定义.

4.59.2 函数说明

4.59.2.1 HAL_UART_Disable()

```
void HAL_UART_Disable (
    HAL_UART_x UART_x )
```

在文件 [hal_lks32mc07x_uart.c](#) 第 63 行定义.

4.59.2.2 HAL_UART_Enable()

```
void HAL_UART_Enable (
    HAL_UART_x UART_x )
```

在文件 [hal_lks32mc07x_uart.c](#) 第 38 行定义.

4.59.2.3 HAL_UART_IrqCheckErr()

```
uint8_t HAL_UART_IrqCheckErr (
    HAL_UART_x uart_x )
```

在文件 [hal_lks32mc07x_uart.c](#) 第 292 行定义.

4.59.2.4 HAL_UART_IrqCheckErrClear()

```
void HAL_UART_IrqCheckErrClear (
    HAL_UART_x uart_x )
```

在文件 [hal_lks32mc07x_uart.c](#) 第 311 行定义.

4.59.2.5 HAL_UART_IrqCheckErrDisable()

```
void HAL_UART_IrqCheckErrDisable (
    HAL_UART_x uart_x )
```

在文件 [hal_lks32mc07x_uart.c](#) 第 307 行定义.

4.59.2.6 HAL_UART_IrqCheckErrEnable()

```
void HAL_UART_IrqCheckErrEnable (
    HAL_UART_x uart_x )
```

在文件 [hal_lks32mc07x_uart.c](#) 第 303 行定义.

4.59.2.7 HAL_UART_IrqReceiveOver()

```
uint8_t HAL_UART_IrqReceiveOver (
    HAL_UART_x uart_x )
```

在文件 [hal_lks32mc07x_uart.c](#) 第 242 行定义.

4.59.2.8 HAL_UART_IrqReceiveOverClear()

```
void HAL_UART_IrqReceiveOverClear (
    HAL_UART_x uart_x )
```

在文件 [hal_lks32mc07x_uart.c](#) 第 261 行定义.

4.59.2.9 HAL_UART_IrqReceiveOverDisable()

```
void HAL_UART_IrqReceiveOverDisable (
    HAL_UART_x uart_x )
```

在文件 [hal_lks32mc07x_uart.c](#) 第 257 行定义.

4.59.2.10 HAL_UART_IrqReceiveOverEnable()

```
void HAL_UART_IrqReceiveOverEnable (
    HAL_UART_x uart_x )
```

在文件 [hal_lks32mc07x_uart.c](#) 第 253 行定义.

4.59.2.11 HAL_UART_IrqSendOver()

```
uint8_t HAL_UART_IrqSendOver (
    HAL_UART_x uart_x )
```

在文件 [hal_lks32mc07x_uart.c](#) 第 217 行定义.

4.59.2.12 HAL_UART_IrqSendOverClear()

```
void HAL_UART_IrqSendOverClear (
    HAL_UART_x uart_x )
```

在文件 [hal_lks32mc07x_uart.c](#) 第 236 行定义.

4.59.2.13 HAL_UART_IrqSendOverDisable()

```
void HAL_UART_IrqSendOverDisable (
    HAL_UART_x uart_x )
```

在文件 [hal_lks32mc07x_uart.c](#) 第 232 行定义.

4.59.2.14 HAL_UART_IrqSendOverEnable()

```
void HAL_UART_IrqSendOverEnable (
    HAL_UART_x uart_x )
```

在文件 [hal_lks32mc07x_uart.c](#) 第 228 行定义.

4.59.2.15 HAL_UART_IrqStopErr()

```
uint8_t HAL_UART_IrqStopErr (
    HAL_UART_x uart_x )
```

在文件 [hal_lks32mc07x_uart.c](#) 第 267 行定义.

4.59.2.16 HAL_UART_IrqStopErrClear()

```
void HAL_UART_IrqStopErrClear (
    HAL_UART_x uart_x )
```

在文件 [hal_lks32mc07x_uart.c](#) 第 286 行定义.

4.59.2.17 HAL_UART_IrqStopErrDisable()

```
void HAL_UART_IrqStopErrDisable (
    HAL_UART_x uart_x )
```

在文件 [hal_lks32mc07x_uart.c](#) 第 282 行定义.

4.59.2.18 HAL_UART_IrqStopErrEnable()

```
void HAL_UART_IrqStopErrEnable (
    HAL_UART_x uart_x )
```

在文件 [hal_lks32mc07x_uart.c](#) 第 278 行定义.

4.59.2.19 HAL_UART_ReadBuffer()

```
uint8_t HAL_UART_ReadBuffer (
    HAL_UART_TypeDef uart_x )
```

在文件 [hal_lks32mc07x_uart.c](#) 第 195 行定义.

4.59.2.20 HAL_UART_Reset()

```
void HAL_UART_Reset (
    HAL_UART_TypeDef UART_x )
```

在文件 [hal_lks32mc07x_uart.c](#) 第 11 行定义.

4.59.2.21 HAL_UART_SetBaud()

```
void HAL_UART_SetBaud (
    HAL_UART_TypeDef UART_x,
    uint32_t baud )
```

在文件 [hal_lks32mc07x_uart.c](#) 第 87 行定义.

函数调用图:

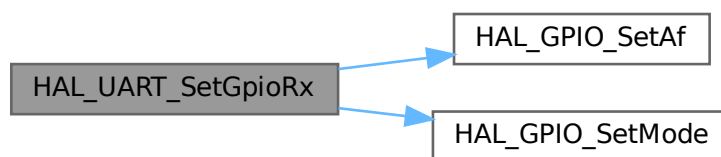


4.59.2.22 HAL_UART_SetGpioRx()

```
void HAL_UART_SetGpioRx (
    HAL_UART_TypeDef uart_x,
    HAL_GPIO_TypeDef px )
```

在文件 [hal_lks32mc07x_uart.c](#) 第 209 行定义.

函数调用图:

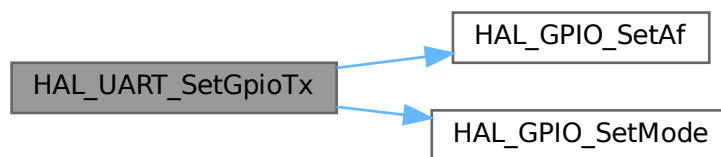


4.59.2.23 HAL_UART_SetGpioTx()

```
void HAL_UART_SetGpioTx (
    HAL_UART_<u>x</u> uart_<u>x</u>,
    HAL_GPIO_P<u>x</u> p<u>x</u> )
```

在文件 [hal_lks32mc07x_uart.c](#) 第 203 行定义.

函数调用图:



4.59.2.24 HAL_UART_SetMode()

```
void HAL_UART_SetMode (
    HAL_UART_<u>x</u> uart_<u>x</u>,
    HAL_UART_MODE_<u>x</u> mode_<u>x</u> )
```

< 低位先发 1位停止位 无校验

< 低位先发 2位停止位 无校验

< 低位先发 1位停止位 奇校验

< 低位先发 2位停止位 奇校验

< 低位先发 1位停止位 偶校验

< 低位先发 2位停止位 偶校验

< 高位先发 1位停止位 无校验

< 高位先发 2位停止位 无校验

< 高位先发 1位停止位 奇校验

< 高位先发 2位停止位 奇校验

< 高位先发 1位停止位 偶校验

< 高位先发 2位停止位 偶校验

在文件 [hal_lks32mc07x_uart.c](#) 第 123 行定义.

4.59.2.25 HAL_UART_TxData()

```
void HAL_UART_TxData (
    HAL_UART_x UART_x,
    uint8_t * data,
    uint32_t len )
```

在文件 `hal_lks32mc07x_uart.c` 第 111 行定义。

4.59.2.26 HAL_UART_WriteBuffer()

```
void HAL_UART_WriteBuffer (
    HAL_UART_x uart_x,
    uint8_t buff )
```

在文件 `hal_lks32mc07x_uart.c` 第 199 行定义。

4.60 hal_lksmcu_uart.h

[浏览该文件的文档.](#)

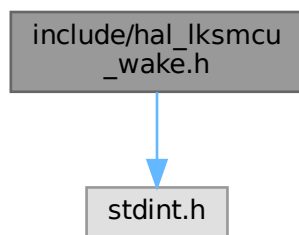
```
00001
00002 #ifndef __HAL_LKSMCU_UART_H__
00003 #define __HAL_LKSMCU_UART_H__
00004 #include "stdint.h"
00005 #include "../include/hal_lksmcu_gpio.h"
00006 typedef enum
00007 {
00008     HAL_UART_0,
00009     HAL_UART_1,
00010     HAL_UART_2,
00011 } HAL_UART_x;
00012 typedef enum
00013 {
00014     HAL_UART_MODE_LSB_STOP1_NONE,
00015     HAL_UART_MODE_LSB_STOP2_NONE,
00016     HAL_UART_MODE_LSB_STOP1_ODD,
00017     HAL_UART_MODE_LSB_STOP2_ODD,
00018     HAL_UART_MODE_LSB_STOP1_EVEN,
00019     HAL_UART_MODE_LSB_STOP2_EVEN,
00020     HAL_UART_MODE_MSB_STOP1_NONE,
00021     HAL_UART_MODE_MSB_STOP2_NONE,
00022     HAL_UART_MODE_MSB_STOP1_ODD,
00023     HAL_UART_MODE_MSB_STOP2_ODD,
00024     HAL_UART_MODE_MSB_STOP1_EVEN,
00025     HAL_UART_MODE_MSB_STOP2_EVEN,
00026 } HAL_UART_MODE_x;
00027 // 初始化
00028 void HAL_UART_Reset(HAL_UART_x); // 模块复位
00029 void HAL_UART_Enable(HAL_UART_x); // 模块打开
00030 void HAL_UART_Disable(HAL_UART_x); // 模块关闭
00031 //
00032 void HAL_UART_SetBaud(HAL_UART_x, uint32_t baud);
00033 void HAL_UART_TxData(HAL_UART_x, uint8_t *data, uint32_t len);
00034 void HAL_UART_SetMode(HAL_UART_x, HAL_UART_MODE_x);
00035 uint8_t HAL_UART_ReadBuffer(HAL_UART_x);
00036 void HAL_UART_WriteBuffer(HAL_UART_x, uint8_t buff);
00037 void HAL_UART_SetGpioTx(HAL_UART_x, HAL_GPIO_Px); // 设置用于输出的gpio
00038 void HAL_UART_SetGpioRx(HAL_UART_x, HAL_GPIO_Px); // 设置用于输出的gpio
00039
00040 // 中断
00041 // 仅支持发送完成、接受完成、停止位错误、校验错误
00042 //
00043 // 以下功能不提供接口, 如果需要使用, 可以使用标准库实现
00044 // 发送缓冲区空
00045 // 发送缓冲区溢出
00046 // 接受缓冲区溢出
00047 // LIN相关
00048
00049 // uart发送完成中断
00050 uint8_t HAL_UART_IrqSendOver(HAL_UART_x);
```

```
00051 void HAL_UART_IrqSendOverEnable (HAL_UART_x);
00052 void HAL_UART_IrqSendOverDisable (HAL_UART_x);
00053 void HAL_UART_IrqSendOverClear (HAL_UART_x);
00054
00055 // uart接受完成中断
00056 uint8_t HAL_UART_IrqReceiveOver (HAL_UART_x);
00057 void HAL_UART_IrqReceiveOverEnable (HAL_UART_x);
00058 void HAL_UART_IrqReceiveOverDisable (HAL_UART_x);
00059 void HAL_UART_IrqReceiveOverClear (HAL_UART_x);
00060
00061 // uart停止位错误中断
00062 uint8_t HAL_UART_IrqStopErr (HAL_UART_x);
00063 void HAL_UART_IrqStopErrEnable (HAL_UART_x);
00064 void HAL_UART_IrqStopErrDisable (HAL_UART_x);
00065 void HAL_UART_IrqStopErrClear (HAL_UART_x);
00066
00067 // uart校验错误中断
00068 uint8_t HAL_UART_IrqCheckErr (HAL_UART_x);
00069 void HAL_UART_IrqCheckErrEnable (HAL_UART_x);
00070 void HAL_UART_IrqCheckErrDisable (HAL_UART_x);
00071 void HAL_UART_IrqCheckErrClear (HAL_UART_x);
00072 #endif
```

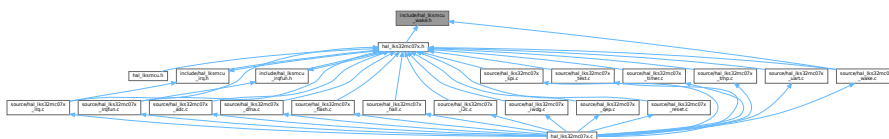
4.61 include/hal_lksmcu_wake.h 文件参考

```
#include "stdint.h"
```

hal_ksmcu_wake.h 的引用(Include)关系图:



此图展示该文件直接或间接的被哪些文件引用了:



枚举

- enum HAL_WAKE_UP_IO_x {
HAL_WAKE_UP_IO_0.Low = 0x0001 , HAL_WAKE_UP_IO_0.High = 0x0101 , HAL_WAKE_UP_IO_1.Low = 0x0002 , HAL_WAKE_UP_IO_1.High = 0x0102 ,
HAL_WAKE_UP_IO_2.Low = 0x0004 , HAL_WAKE_UP_IO_2.High = 0x0104 , HAL_WAKE_UP_IO_3.Low = 0x0008 , HAL_WAKE_UP_IO_3.High = 0x0108 ,
HAL_WAKE_UP_IO_4.Low = 0x0010 , HAL_WAKE_UP_IO_4.High = 0x0110 , HAL_WAKE_UP_IO_5.Low = 0x0020 , HAL_WAKE_UP_IO_5.High = 0x0120 ,
HAL_WAKE_UP_IO_6.Low = 0x0040 , HAL_WAKE_UP_IO_6.High = 0x0140 , HAL_WAKE_UP_IO_7.Low = 0x0080 , HAL_WAKE_UP_IO_7.High = 0x0180 }

函数

- void HAL_WAKE_DeepSleepIoWake (HAL_WAKE_UP_IO_x)
进入深度休眠,并且使用IO唤醒
- void HAL_WAKE_DeepSleepTimeWakeUp (uint32_t ms)
进入深度休眠,并且使用定时唤醒
- void HAL_WAKE_DeepSleepIoTimeWakeUp (HAL_WAKE_UP_IO_x, uint32_t ms)
进入深度休眠,并且使用IO唤醒和定时唤醒

4.61.1 枚举类型说明

4.61.1.1 HAL_WAKE_UP_IO_x

enum HAL_WAKE_UP_IO_x

枚举值

HAL_WAKE_UP_IO_0.Low	
HAL_WAKE_UP_IO_0.High	
HAL_WAKE_UP_IO_1.Low	
HAL_WAKE_UP_IO_1.High	
HAL_WAKE_UP_IO_2.Low	
HAL_WAKE_UP_IO_2.High	
HAL_WAKE_UP_IO_3.Low	
HAL_WAKE_UP_IO_3.High	
HAL_WAKE_UP_IO_4.Low	
HAL_WAKE_UP_IO_4.High	
HAL_WAKE_UP_IO_5.Low	
HAL_WAKE_UP_IO_5.High	
HAL_WAKE_UP_IO_6.Low	
HAL_WAKE_UP_IO_6.High	
HAL_WAKE_UP_IO_7.Low	
HAL_WAKE_UP_IO_7.High	

在文件 [hal_lksmcu_wake.h](#) 第 5 行定义.

4.61.2 函数说明

4.61.2.1 HAL_WAKE_DeepSleepIoTimeWakeUp()

```
void HAL_WAKE_DeepSleepIoTimeWakeUp (
    HAL_WAKE_UP_IO_x wake_up_io,
    uint32_t ms )
```

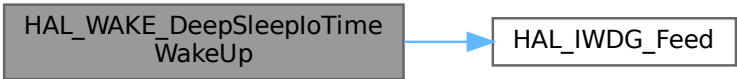
进入深度休眠,并且使用IO唤醒和定时唤醒

参数

<i>wake_up</i> ↔ <i>_io</i>	指定唤醒IO引脚及其电平
<i>ms</i>	指定定时唤醒的时间,单位为毫秒

在文件 [hal_lks32mc07x.wake.c](#) 第 128 行定义.

函数调用图:



4.61.2.2 HAL_WAKE_DeepSleepIoWake()

```
void HAL_WAKE_DeepSleepIoWake (
    HAL_WAKE_UP_IO_x wake_up_io )
```

进入深度休眠,并且使用IO唤醒

参数

<i>wake_up</i> ↔ <i>_io</i>	指定唤醒IO引脚及其电平
--------------------------------	--------------

在文件 [hal_lks32mc07x.wake.c](#) 第 12 行定义.

4.61.2.3 HAL_WAKE_DeepSleepTimeWakeUp()

```
void HAL_WAKE_DeepSleepTimeWakeUp (
    uint32_t ms )
```

进入深度休眠,并且使用定时唤醒

参数

<i>ms</i>	指定定时唤醒的时间,单位为毫秒
-----------	-----------------

在文件 [hal_lks32mc07x.wake.c](#) 第 65 行定义.

函数调用图:



4.62 hal_lksmcu_wake.h

[浏览该文件的文档.](#)

```

00001
00002 #ifndef __HAL_LKSMCU_WEAK_H__
00003 #define __HAL_LKSMCU_WEAK_H__
00004 #include "stdint.h"
00005 typedef enum
00006 {
00007     HAL_WAKE_UP_IO_0_Low = 0x0001,
00008     HAL_WAKE_UP_IO_0_High = 0x0101,
00009     HAL_WAKE_UP_IO_1_Low = 0x0002,
00010     HAL_WAKE_UP_IO_1_High = 0x0102,
00011     HAL_WAKE_UP_IO_2_Low = 0x0004,
00012     HAL_WAKE_UP_IO_2_High = 0x0104,
00013     HAL_WAKE_UP_IO_3_Low = 0x0008,
00014     HAL_WAKE_UP_IO_3_High = 0x0108,
00015     HAL_WAKE_UP_IO_4_Low = 0x0010,
00016     HAL_WAKE_UP_IO_4_High = 0x0110,
00017     HAL_WAKE_UP_IO_5_Low = 0x0020,
00018     HAL_WAKE_UP_IO_5_High = 0x0120,
00019     HAL_WAKE_UP_IO_6_Low = 0x0040,
00020     HAL_WAKE_UP_IO_6_High = 0x0140,
00021     HAL_WAKE_UP_IO_7_Low = 0x0080,
00022     HAL_WAKE_UP_IO_7_High = 0x0180,
00023 } HAL_WAKE_UP_IO_x;
00024
00025 void HAL_WAKE_DeepSleepIoWake(HAL_WAKE_UP_IO_x);
00026 void HAL_WAKE_DeepSleepTimeWakeUp(uint32_t ms);
00027 void HAL_WAKE_DeepSleepIoTimeWakeUp(HAL_WAKE_UP_IO_x, uint32_t ms);
00028 #endif
  
```

4.63 readme.md 文件参考

4.64 source/hal_lks32mc07x_adc.c 文件参考

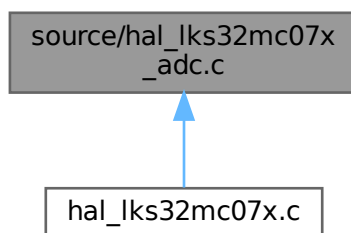
虚拟实现,用来在芯片之外约束函数运行效果

```

#include "../include/hal_lksmcu_adc.h"
#include "../hal_lks32mc07x.h"
#include "lks32mc07x_user_manual.h"
#include "../include/hal_lksmcu_reg.h"
#include "../include/hal_lks32mc07x_reg.h"
hal_lks32mc07x_adc.c 的引用(Include)关系图:
  
```



此图展示该文件直接或间接的被哪些文件引用了:



函数

- static void **HAL_ADC_UpdateTrig** (HAL_ADC_x adc_x)
内部使用,更新`adc_soft_reg`描述的触发配置
- void **HAL_ADC_Reset** (HAL_ADC_x adc_x)
初始化指定的ADC模块
- void **HAL_ADC_Enable** (HAL_ADC_x adc_x)
启用指定的ADC模块
- void **HAL_ADC_Disable** (HAL_ADC_x adc_x)
禁用指定的ADC模块
- int16_t **HAL_ADC_GetSamplingValues** (HAL_ADC_x adc_x, HAL_ADC_SAMP_SEQUENCE_x sequence_x)
获取指定序列号的采样值
- void **HAL_ADC_SetSamplingChannel** (HAL_ADC_x adc_x, HAL_ADC_SAMP_SEQUENCE_x sequence_x, HAL_ADC_PORT_x port_x)
设置指定序列号的采样通道
- void **HAL_ADC_SetSamplingQuantity** (HAL_ADC_x adc_x, HAL_ADC_SAMP_NUMBER_x sample1_number, HAL_ADC_SAMP_NUMBER_x sample2_number)
设置采样数量
- void **HAL_ADC_SetTriggerSignal** (HAL_ADC_x adc_x, HAL_ADC_TRIGGER_x trigger)
设置触发信号
- void **HAL_ADC_EndSampling** (HAL_ADC_x adc_x)
结束采样
- void **HAL_ADC_SoftTrigger** (HAL_ADC_x adc_x)
软件触发ADC开始采样
- int16_t **HAL_ADC_ManualSampling** (HAL_ADC_x adc_x, HAL_ADC_PORT_x port_x)
手动采样
- void **HAL_ADC_SetReference** (HAL_ADC_x adc_x, HAL_ADC_REFERENCE_x reference_x)
设置ADC基准源
- float **HAL_ADC_GetRange** (HAL_ADC_x adc_x)
获取adc量程,使用芯片供电作为adc基准时固定返回5.0
- float **HAL_ADC_GetValuesUnit** (HAL_ADC_x adc_x)
获取采样值单位
- void **HAL_ADC_SetGpioIn** (HAL_ADC_x adc_x, HAL_GPIO_Px pin_x)
设置指定的GPIO为ADC输入
- void **HAL_ADC_SetGpioTriggerOut** (HAL_ADC_x adc_x, HAL_GPIO_Px pin_x)
设置指定的GPIO为ADC触发信号输出

变量

- static volatile uint8_t `adc_soft_reg` [2] = {0, 0}
ADC寄存器结构体句柄

4.64.1 详细描述

虚拟实现,用来在芯片之外约束函数运行效果
在文件 `hal_lks32mc07x_adc.c` 中定义.

4.64.2 函数说明

4.64.2.1 HAL_ADC_Disable()

```
void HAL_ADC_Disable (
    HAL_ADC_x adc_x )
```

禁用指定的ADC模块

参数

<code>adc_x</code>	要禁用的ADC模块编号
--------------------	-------------

在文件 `hal_lks32mc07x_adc.c` 第 114 行定义.

4.64.2.2 HAL_ADC_Enable()

```
void HAL_ADC_Enable (
    HAL_ADC_x adc_x )
```

启用指定的ADC模块

参数

<code>adc_x</code>	要启用的ADC模块编号
--------------------	-------------

在文件 `hal_lks32mc07x_adc.c` 第 87 行定义.

4.64.2.3 HAL_ADC_EndSampling()

```
void HAL_ADC_EndSampling (
    HAL_ADC_x adc_x )
```

结束采样

参数

<code>adc_x</code>	ADC模块编号
--------------------	---------

在文件 [hal_lks32mc07x_adc.c](#) 第 267 行定义.

这是这个函数的调用关系图:



4.64.2.4 HAL_ADC.GetRange()

```
float HAL_ADC_GetRange (
    HAL_ADC_x adc_x )
```

获取adc量程,使用芯片供电作为adc基准时固定返回5.0

参数

<code>adc_x</code>	ADC模块编号
--------------------	---------

返回

ADC量程

在文件 [hal_lks32mc07x_adc.c](#) 第 347 行定义.

这是这个函数的调用关系图:



4.64.2.5 HAL_ADC_GetSamplingValues()

```
int16_t HAL_ADC_GetSamplingValues (
    HAL_ADC_TypeDef ADC_x,
    HAL_ADC_Sequences_TypeDef sequence_x )
```

获取指定序列号的采样值

参数

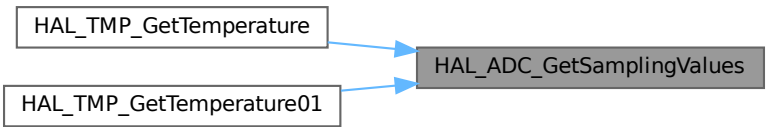
adc_x	ADC模块编号
sequence_x	采样序列号

返回

返回对应序列号的采样值

在文件 [hal_lks32mc07x_adc.c](#) 第 147 行定义.

这是这个函数的调用关系图:



4.64.2.6 HAL_ADC_GetValuesUnit()

```
float HAL_ADC_GetValuesUnit (
    HAL_ADC_TypeDef ADC_x )
```

获取采样值单位

参数

adc_x	ADC模块编号
-------	---------

返回

采样值单位

在文件 [hal_lks32mc07x_adc.c](#) 第 373 行定义.

函数调用图:



4.64.2.7 HAL_ADC.ManualSampling()

```

int16_t HAL_ADC_ManualSampling (
    HAL_ADC_x adc_x,
    HAL_ADC_PORT_x port_x )
  
```

手动采样

参数

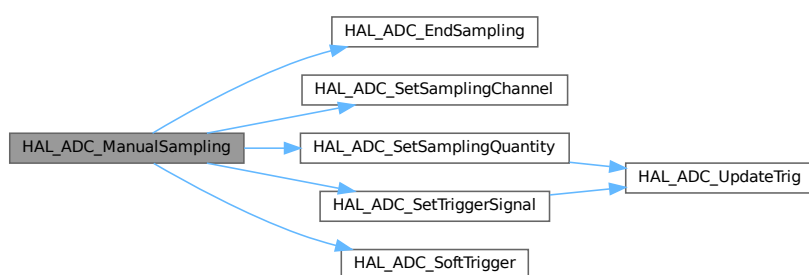
<i>n</i>	采样通道编号
----------	--------

返回

示例返回值,实际应返回采样值

在文件 [hal_lks32mc07x_adc.c](#) 第 285 行定义.

函数调用图:



4.64.2.8 HAL_ADC.Reset()

```

void HAL_ADC_Reset (
    HAL_ADC_x adc_x )
  
```

初始化指定的ADC模块

参数

<i>adc</i> _↔ <i>_x</i>	要初始化的ADC模块编号
--------------------------------------	--------------

在文件 [hal_lks32mc07x_adc.c](#) 第 62 行定义.

4.64.2.9 HAL_ADC_SetGpioIn()

```
void HAL_ADC_SetGpioIn (
    HAL_ADC_t adc_x,
    HAL_GPIO_Px pin_x )
```

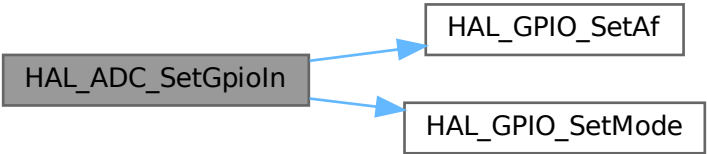
设置指定的GPIO为ADC输入

参数

<i>adc</i> _↔ <i>_x</i>	ADC模块编号
<i>pin</i> _↔ <i>_x</i>	GPIO编号

在文件 [hal_lks32mc07x_adc.c](#) 第 390 行定义.

函数调用图:



4.64.2.10 HAL_ADC_SetGpioTriggerOut()

```
void HAL_ADC_SetGpioTriggerOut (
    HAL_ADC_t adc_x,
    HAL_GPIO_Px pin_x )
```

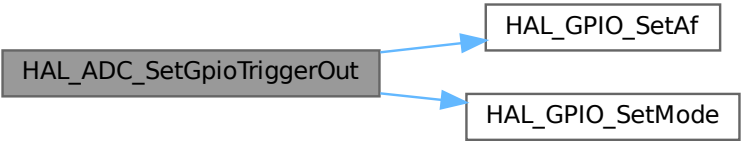
设置指定的GPIO为ADC触发信号输出

参数

<i>adc</i> ↔ _x	ADC模块编号
<i>pin</i> ↔ _x	GPIO编号

在文件 [hal_lks32mc07x_adc.c](#) 第 401 行定义.

函数调用图:



4.64.2.11 HAL_ADC_SetReference()

```
void HAL_ADC_SetReference (
    HAL_ADC_x adc_x,
    HAL_ADC_REFERENCE_x reference_x )
```

设置ADC基准源

参数

<i>adc_x</i>	ADC模块编号
<i>reference</i> ↔ _x	基准源

在文件 [hal_lks32mc07x_adc.c](#) 第 315 行定义.

4.64.2.12 HAL_ADC_SetSamplingChannel()

```
void HAL_ADC_SetSamplingChannel (
    HAL_ADC_x adc_x,
    HAL_ADC_SAMP_SEQUENCE_x sequence_x,
    HAL_ADC_PORT_x port_x )
```

设置指定序列号的采样通道

参数

<i>adc_x</i>	ADC模块编号
<i>sequence_x</i>	采样序列号
<i>port_x</i>	采样通道端口

- < ADC第0次采样
- < ADC第1次采样
- < ADC第2次采样
- < ADC第3次采样
- < ADC第4次采样
- < ADC第5次采样
- < ADC第6次采样
- < ADC第7次采样
- < ADC第8次采样
- < ADC第9次采样
- < ADC第10次采样
- < ADC第11次采样
- < ADC第12次采样
- < ADC第13次采样

在文件 [hal_lks32mc07x_adc.c](#) 第 158 行定义.

这是这个函数的调用关系图:



4.64.2.13 HAL_ADC.SetSamplingQuantity()

```
void HAL_ADC_SetSamplingQuantity (
    HAL_ADC_x adc_x,
    HAL_ADC_SAMP_NUMBER_x sample1_number,
    HAL_ADC_SAMP_NUMBER_x sample2_number )
```

设置采样数量

参数

<i>adc_x</i>	ADC模块编号
<i>sample_number</i>	采样数量

在文件 [hal_lks32mc07x_adc.c](#) 第 214 行定义.

函数调用图:



这是这个函数的调用关系图:



4.64.2.14 HAL_ADC_SetTriggerSignal()

```
void HAL_ADC_SetTriggerSignal (
    HAL_ADC_x adc_x,
    HAL_ADC_TRIGGER_x trigger )
```

设置触发信号

参数

<i>adc_x</i>	ADC模块编号
<i>trigger</i>	触发信号类型

- < 使用MCPWM触发ADC采样
- < 使用TIMER触发ADC采样
- < 使用软件触发ADC采样

在文件 [hal_lks32mc07x_adc.c](#) 第 244 行定义.

函数调用图:



这是这个函数的调用关系图:



4.64.2.15 HAL_ADC_SoftTrigger()

```
void HAL_ADC_SoftTrigger (
    HAL_ADC_x adc_x )
```

软件触发ADC开始采样

参数

<i>adc_x</i>	ADC模块编号
--------------	---------

在文件 [hal_lks32mc07x_adc.c](#) 第 276 行定义.
这是这个函数的调用关系图:



4.64.2.16 HAL_ADC_UpdateTrig()

```
static void HAL_ADC_UpdateTrig (
    HAL_ADC_x adc_x ) [static]
```

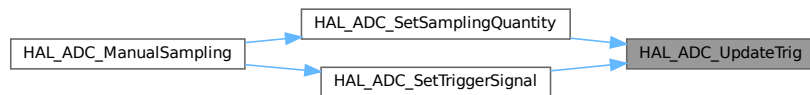
内部使用,更新adc_soft_reg描述的触发配置

参数

<code>adc_x</code>	使用的ADC模块编号
--------------------	------------

在文件 [hal_lks32mc07x_adc.c](#) 第 22 行定义.

这是这个函数的调用关系图:



4.64.3 变量说明

4.64.3.1 adc_soft_reg

```
volatile uint8_t adc_soft_reg[2] = {0, 0} [static]
```

ADC寄存器结构体句柄

在文件 [hal_lks32mc07x_adc.c](#) 第 16 行定义.

4.65 hal_lks32mc07x_adc.c

[浏览该文件的文档.](#)

```

00001
00005 #include "../include/hal_lksmcu_adc.h"
00006 #include "../hal_lks32mc07x.h"
00007 #include "lks32mc07x_user_manual.h"
00008 #include "../include/hal_lksmcu_reg.h"
00009 #include "../include/hal_lks32mc07x_reg.h"
00010
00015 // 虚拟寄存器
00016 static volatile uint8_t adc_soft_reg[2] = {0, 0}; // bit0 adc0使能 bit1 adc1使能 bit2 mcpwm触发 bit3
timer触发 bit4 两段式触发使能
00017
00022 static void HAL_ADC_UpdateTrig(HAL_ADC_x adc_x)
00023 {
00024     // bit2 mcpwm触发 bit3 timer触发 bit4 两段式触发使能
00025     if (HAL_REG_READ_BIT(adc_soft_reg[adc_x], BIT4))
00026     {
00027         HAL_REG_SET(HAL_ADC[adc_x]->CFG, BIT12);
00028         if (HAL_REG_READ_BIT(adc_soft_reg[adc_x], BIT2)) // 两段 mcpwm触发
00029         {
00030             HAL_REG_WRITE_BIT(HAL_ADC[adc_x]->TRIG, BIT0 | BIT1 | BIT4 | BIT5, BIT0 | BIT1);
00031         }
00032         else if (HAL_REG_READ_BIT(adc_soft_reg[adc_x], BIT3)) // 两段 timer触发
00033         {
00034             HAL_REG_WRITE_BIT(HAL_ADC[adc_x]->TRIG, BIT0 | BIT1 | BIT4 | BIT5, BIT4 | BIT5);
00035         }
00036         else // 两段 软件触发
00037         {
00038             HAL_REG_WRITE_BIT(HAL_ADC[adc_x]->TRIG, BIT0 | BIT1 | BIT4 | BIT5, 0);
00039         }
00040     }
00041     else
00042     {

```

```

00043     HAL_REG_RESET(HAL_ADC[adc_x]->CFG, BIT12);
00044     if (HAL_REG_READ_BIT(adc_soft_reg[adc_x], BIT2)) // 一段 mcpwm触发
00045     {
00046         HAL_REG_WRITE_BIT(HAL_ADC[adc_x]->TRIG, BIT0 | BIT1 | BIT4 | BIT5, BIT0);
00047     }
00048     else if (HAL_REG_READ_BIT(adc_soft_reg[adc_x], BIT3)) // 一段 timer触发
00049     {
00050         HAL_REG_WRITE_BIT(HAL_ADC[adc_x]->TRIG, BIT0 | BIT1 | BIT4 | BIT5, BIT4);
00051     }
00052     else // 一段 软件触发
00053     {
00054         HAL_REG_WRITE_BIT(HAL_ADC[adc_x]->TRIG, BIT0 | BIT1 | BIT4 | BIT5, 0);
00055     }
00056 }
00057 }
00062 void HAL_ADC_Reset(HAL_ADC_x adc_x)
00063 {
00064     // 系统寄存器解锁
00065     HAL_REG_SYS_UNLOCK();
00066     switch (adc_x)
00067     {
00068         case HAL_ADC_0:
00069             HAL_REG_SET(SYS_SFT_RST, SYS_SFT_RST_ADC0_SFT_RST_BIT);
00070             HAL_REG_RESET(SYS_SFT_RST, SYS_SFT_RST_ADC0_SFT_RST_BIT);
00071             break;
00072         case HAL_ADC_1:
00073             HAL_REG_SET(SYS_SFT_RST, SYS_SFT_RST_ADC1_SFT_RST_BIT);
00074             HAL_REG_RESET(SYS_SFT_RST, SYS_SFT_RST_ADC1_SFT_RST_BIT);
00075             break;
00076         default:
00077             break;
00078     }
00079     // 系统寄存器上锁
00080     HAL_REG_SYS_LOCK();
00081 }
00082
00087 void HAL_ADC_Enable(HAL_ADC_x adc_x)
00088 {
00089     switch (adc_x)
00090     {
00091         case HAL_ADC_0:
00092             HAL_REG_SET(adc_soft_reg[adc_x], BIT0);
00093             HAL_REG_RESET(adc_soft_reg[adc_x], BIT0);
00094             break;
00095         case HAL_ADC_1:
00096             HAL_REG_SET(adc_soft_reg[adc_x], BIT1);
00097             HAL_REG_RESET(adc_soft_reg[adc_x], BIT1);
00098             break;
00099         default:
00100             break;
00101     }
00102     // 系统寄存器解锁
00103     HAL_REG_SYS_UNLOCK();
00104     // SYS_AFE_REG5[0] ADC 开启使能
00105     HAL_REG_SET(SYS_AFE_REG5, SYS_AFE_REG5_ADCPDN_BIT);
00106     // 系统寄存器上锁
00107     HAL_REG_SYS_LOCK();
00108 }
00109
00114 void HAL_ADC_Disable(HAL_ADC_x adc_x)
00115 {
00116     switch (adc_x)
00117     {
00118         case HAL_ADC_0:
00119             HAL_REG_SET(adc_soft_reg[adc_x], BIT0);
00120             HAL_REG_RESET(adc_soft_reg[adc_x], BIT0);
00121             break;
00122         case HAL_ADC_1:
00123             HAL_REG_SET(adc_soft_reg[adc_x], BIT1);
00124             HAL_REG_RESET(adc_soft_reg[adc_x], BIT1);
00125             break;
00126         default:
00127             break;
00128     }
00129     // 因为adc只有一个使能位,所以
00130     // 只在两个adc同时关闭时才关闭adc
00131     if (HAL_REG_READ_BIT(adc_soft_reg[adc_x], BIT0 | BIT1) == 0)
00132     {
00133         // 系统寄存器解锁
00134         HAL_REG_SYS_UNLOCK();
00135         HAL_REG_RESET(SYS_AFE_REG5, SYS_AFE_REG5_ADCPDN_BIT);
00136         // 系统寄存器上锁
00137         HAL_REG_SYS_LOCK();
00138     }
00139 }
00140
00147 int16_t HAL_ADC_GetSamplingValues(HAL_ADC_x adc_x, HAL_ADC_SAMP_SEQUENCE_x sequence_x)

```

```

00148 {
00149     return HAL_ADC[adc_x]->DAT[sequence_x];
00150 }
00151
00158 void HAL_ADC_SetSamplingChannel(HAL_ADC_x adc_x, HAL_ADC_SAMP_SEQUENCE_x sequence_x, HAL_ADC_PORT_x port_x)
00159 {
00160     switch (sequence_x)
00161     {
00162         case HAL_ADC_SAMP_SEQUENCE_0:
00163             HAL_REG_WRITE_BIT(HAL_ADC[adc_x]->CHN0, 0x000f, port_x);
00164             break;
00165         case HAL_ADC_SAMP_SEQUENCE_1:
00166             HAL_REG_WRITE_BIT(HAL_ADC[adc_x]->CHN0, 0x00f0, port_x << 4);
00167             break;
00168         case HAL_ADC_SAMP_SEQUENCE_2:
00169             HAL_REG_WRITE_BIT(HAL_ADC[adc_x]->CHN0, 0x0f00, port_x << 8);
00170             break;
00171         case HAL_ADC_SAMP_SEQUENCE_3:
00172             HAL_REG_WRITE_BIT(HAL_ADC[adc_x]->CHN0, 0xf000, port_x << 12);
00173             break;
00174         case HAL_ADC_SAMP_SEQUENCE_4:
00175             HAL_REG_WRITE_BIT(HAL_ADC[adc_x]->CHN1, 0x000f, port_x);
00176             break;
00177         case HAL_ADC_SAMP_SEQUENCE_5:
00178             HAL_REG_WRITE_BIT(HAL_ADC[adc_x]->CHN1, 0x00f0, port_x << 4);
00179             break;
00180         case HAL_ADC_SAMP_SEQUENCE_6:
00181             HAL_REG_WRITE_BIT(HAL_ADC[adc_x]->CHN1, 0x0f00, port_x << 8);
00182             break;
00183         case HAL_ADC_SAMP_SEQUENCE_7:
00184             HAL_REG_WRITE_BIT(HAL_ADC[adc_x]->CHN1, 0xf000, port_x << 12);
00185             break;
00186         case HAL_ADC_SAMP_SEQUENCE_8:
00187             HAL_REG_WRITE_BIT(HAL_ADC[adc_x]->CHN2, 0x000f, port_x);
00188             break;
00189         case HAL_ADC_SAMP_SEQUENCE_9:
00190             HAL_REG_WRITE_BIT(HAL_ADC[adc_x]->CHN2, 0x00f0, port_x << 4);
00191             break;
00192         case HAL_ADC_SAMP_SEQUENCE_10:
00193             HAL_REG_WRITE_BIT(HAL_ADC[adc_x]->CHN2, 0x0f00, port_x << 8);
00194             break;
00195         case HAL_ADC_SAMP_SEQUENCE_11:
00196             HAL_REG_WRITE_BIT(HAL_ADC[adc_x]->CHN2, 0xf000, port_x << 12);
00197             break;
00198         case HAL_ADC_SAMP_SEQUENCE_12:
00199             HAL_REG_WRITE_BIT(HAL_ADC[adc_x]->CHN3, 0x000f, port_x);
00200             break;
00201         case HAL_ADC_SAMP_SEQUENCE_13:
00202             HAL_REG_WRITE_BIT(HAL_ADC[adc_x]->CHN3, 0x00f0, port_x << 4);
00203             break;
00204         default:
00205             break;
00206     }
00207 }
00208
00214 void HAL_ADC_SetSamplingQuantity(HAL_ADC_x adc_x, HAL_ADC_SAMP_NUMBER_x sample1_number,
    HAL_ADC_SAMP_NUMBER_x sample2_number)
00215 {
00216     // 第一段采样数量为0时, 芯片不进行采样
00217     if (sample1_number == HAL_ADC_SAMP_NUMBER_0)
00218     {
00219         sample2_number = HAL_ADC_SAMP_NUMBER_0;
00220     }
00221     // 两段采样数量之和不能大于10个
00222     if (sample1_number + sample2_number > 14)
00223     {
00224         sample2_number = (HAL_ADC_SAMP_NUMBER_x)(14 - sample1_number);
00225     }
00226     HAL_REG_WRITE_BIT(HAL_ADC[adc_x]->CHNT, 0x000f, sample1_number);
00227     HAL_REG_WRITE_BIT(HAL_ADC[adc_x]->CHNT, 0x00f0, sample2_number << 4);
00228     if (sample2_number)
00229     {
00230         HAL_REG_SET(adc_soft_reg[adc_x], BIT4);
00231     }
00232     else
00233     {
00234         HAL_REG_RESET(adc_soft_reg[adc_x], BIT4);
00235     }
00236     HAL_ADC_UpdateTrig(adc_x);
00237 }
00238
00244 void HAL_ADC_SetTriggerSignal(HAL_ADC_x adc_x, HAL_ADC_TRIGGER_x trigger)
00245 {
00246     switch (trigger)
00247     {
00248         case HAL_ADC_TRIGGER_MCPWM:
00249             HAL_REG_WRITE_BIT(adc_soft_reg[adc_x], BIT2 | BIT3, BIT2);

```

```

00250         break;
00251     case HAL_ADC_TRIGGER_TIMER:
00252         HAL_REG_WRITE_BIT(adc_soft_reg[adc_x], BIT2 | BIT3, BIT3);
00253         break;
00254     case HAL_ADC_TRIGGER_SOFT:
00255         HAL_REG_WRITE_BIT(adc_soft_reg[adc_x], BIT2 | BIT3, 0);
00256         break;
00257     default:
00258         break;
00259 }
00260 HAL_ADC_UpdateTrig(adc_x);
00261 }
00262
00267 void HAL_ADC_EndSampling(HAL_ADC_x adc_x)
00268 {
00269     HAL_REG_SET(HAL_ADC[adc_x]->CFG, BIT11);
00270 }
00271
00276 void HAL_ADC_SoftTrigger(HAL_ADC_x adc_x)
00277 {
00278     HAL_REG_WRITE(HAL_ADC[adc_x]->SWT, 0x5aa5);
00279 }
00285 int16_t HAL_ADC_ManualSampling(HAL_ADC_x adc_x, HAL_ADC_PORT_x port_x) // 手动采样 需要先设置采样通道为软件触发
00286 {
00287     int16_t data;
00288     HAL_ADC_SetTriggerSignal(adc_x, HAL_ADC_TRIGGER_SOFT);
00289     HAL_ADC_EndSampling(adc_x);
00290     HAL_ADC_SetSamplingQuantity(adc_x, HAL_ADC_SAMP_NUMBER_1, HAL_ADC_SAMP_NUMBER_0);
00291     HAL_ADC_SetSamplingChannel(adc_x, HAL_ADC_SAMP_SEQUENCE_0, port_x);
00292     HAL_REG_WRITE(HAL_ADC[adc_x]->IE, 0);
00293     HAL_REG_WRITE(HAL_ADC[adc_x]->IF, 0xffff);
00294     HAL_ADC_SoftTrigger(adc_x);
00295     uint32_t i = 3000;
00296     while (HAL_ADC[adc_x]->IF == 0)
00297     {
00298         i--;
00299         __NOP();
00300         if (i == 0)
00301         {
00302             break;
00303         }
00304     }
00305     HAL_REG_WRITE(HAL_ADC[adc_x]->IF, 0xffff);
00306     data = HAL_ADC[adc_x]->DAT[0];
00307     return data;
00308 }
00309
00315 void HAL_ADC_SetReference(HAL_ADC_x adc_x, HAL_ADC_REFERENCE_x reference_x)
00316 {
00317     (void)adc_x;
00318     switch (reference_x)
00319     {
00320         case HAL_ADC_REFERENCE_INTERNAL: // 使用内部基准源作为adc基准
00321
00322             // 系统寄存器解锁
00323             HAL_REG_SYS_UNLOCK();
00324             // SYS_AFE_REG1[4] ADC 基准选择
00325             HAL_REG_WRITE_BIT(SYS_AFE_REG1, BIT4, 0);
00326             // 系统寄存器上锁
00327             HAL_REG_SYS_LOCK();
00328
00329             break;
00330         case HAL_ADC_REFERENCE_EXTERNAL:
00331             // 系统寄存器解锁
00332             HAL_REG_SYS_UNLOCK();
00333             // SYS_AFE_REG1[4] ADC 基准选择
00334             HAL_REG_WRITE_BIT(SYS_AFE_REG1, BIT4, BIT4);
00335             // 系统寄存器上锁
00336             HAL_REG_SYS_LOCK();
00337             break;
00338         default:
00339             break;
00340     }
00341 }
00347 float HAL_ADC_GetRange(HAL_ADC_x adc_x)
00348 {
00349     (void)adc_x;
00350     uint32_t val;
00351     float range = 5.0f;
00352     val = HAL_REG_READ_BIT(SYS_AFE_REG1, BIT4);
00353     val >>= 4;
00354     switch (val)
00355     {
00356         case 0:
00357             range = 3.6f;
00358             break;
00359         case 1:

```

```

00360         range = 5.0f;
00361         break;
00362     default:
00363         break;
00364     }
00365
00366     return range;
00367 }
00373 float HAL_ADC_GetValuesUnit(HAL_ADC_x adc_x)
00374 {
00375     if (HAL_ADC[adc_x] -> CFG & BIT10)
00376     {
00377         return HAL_ADC_GetRange(adc_x) / 0x07ff;
00378     }
00379     else
00380     {
00381         return HAL_ADC_GetRange(adc_x) / 0x7fff;
00382     }
00383 }
00384
00390 void HAL_ADC_SetGpioIn(HAL_ADC_x adc_x, HAL_GPIO_Px pin_x)
00391 {
00392     (void)adc_x;
00393     HAL_GPIO_SetAf(pin_x, HAL_GPIO_AF_ADC);
00394     HAL_GPIO_SetMode(pin_x, HAL_GPIO_MODE_AFE);
00395 }
00401 void HAL_ADC_SetGpioTriggerOut(HAL_ADC_x adc_x, HAL_GPIO_Px pin_x)
00402 {
00403     (void)adc_x;
00404     HAL_GPIO_SetAf(pin_x, HAL_GPIO_AF_ADC_TRIG);
00405     HAL_GPIO_SetMode(pin_x, HAL_GPIO_MODE_OUT);
00406 }

```

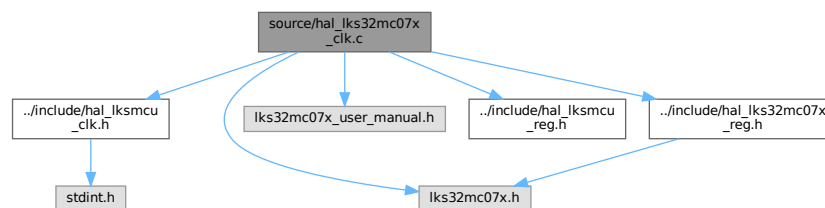
4.66 source/hal_lks32mc07x_clk.c 文件参考

```

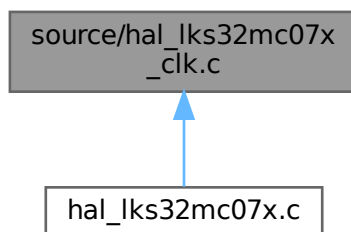
#include "../include/hal_lksmcu_clk.h"
#include "lks32mc07x.h"
#include "lks32mc07x_user_manual.h"
#include "../include/hal_lksmcu_reg.h"
#include "../include/hal_lks32mc07x_reg.h"

```

hal_lks32mc07x_clk.c 的引用(Include)关系图:



此图展示该文件直接或间接的被哪些文件引用了:



函数

- void [HAL_CLK_SetMcuFreq](#) ([HAL_CLK_FREQ_x](#) [FREQ_x](#))
设置主频
- [uint32_t](#) [HAL_CLK_GetMcuFreq](#) (void)
获取主频

变量

- static volatile [uint32_t](#) [HAL_CLK_mclk](#) = 96000000

4.66.1 函数说明

4.66.1.1 HAL_CLK_GetMcuFreq()

```
uint32\_t HAL_CLK_GetMcuFreq (  
    void )
```

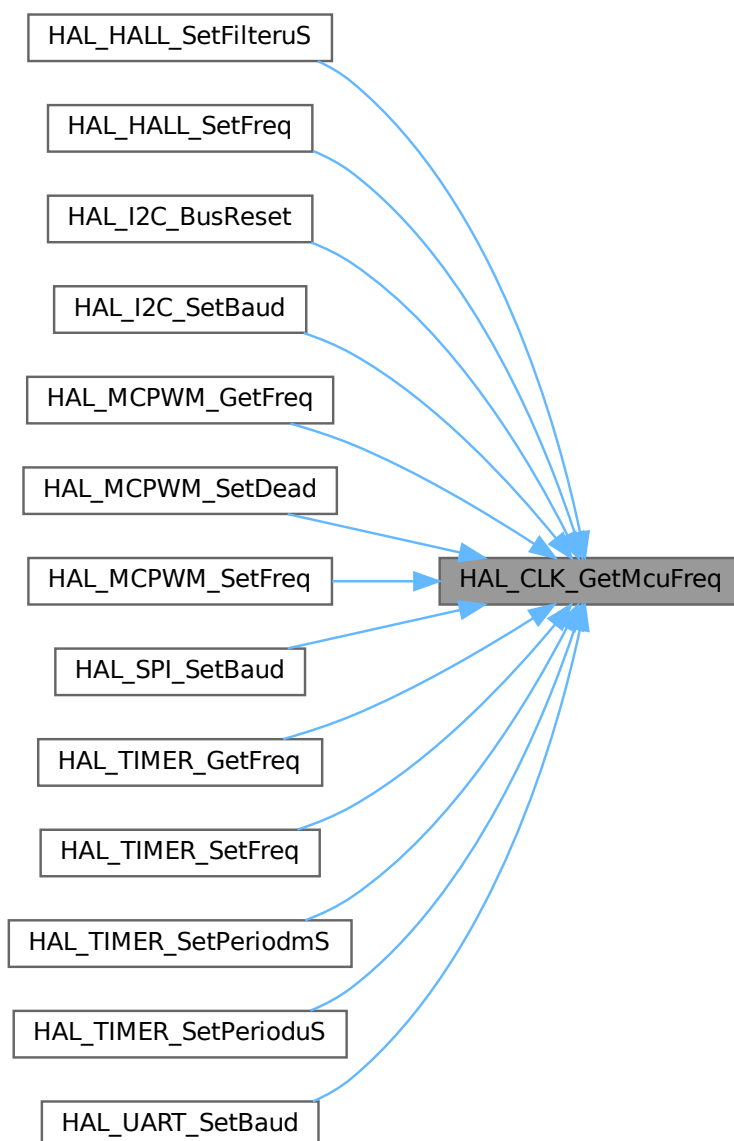
获取主频

返回

主频

在文件 [hal_lks32mc07x_clk.c](#) 第 45 行定义.

这是这个函数的调用关系图:



4.66.1.2 HAL_CLK_SetMcuFreq()

```
void HAL_CLK_SetMcuFreq (
    HAL_CLK_FREQ_x FREQ_x )
```

设置主频

参数

$FREQ_x$	主频
----------	----

< 主频4Mhz

< 主频12Mhz

< 主频24Mhz

< 主频48Mhz

< 主频96Mhz

在文件 [hal_lks32mc07x_clk.c](#) 第 11 行定义.

4.66.2 变量说明

4.66.2.1 HAL_CLK_mclk

```
volatile uint32_t HAL_CLK_mclk = 96000000 [static]
```

在文件 [hal_lks32mc07x_clk.c](#) 第 6 行定义.

4.67 hal_lks32mc07x_clk.c

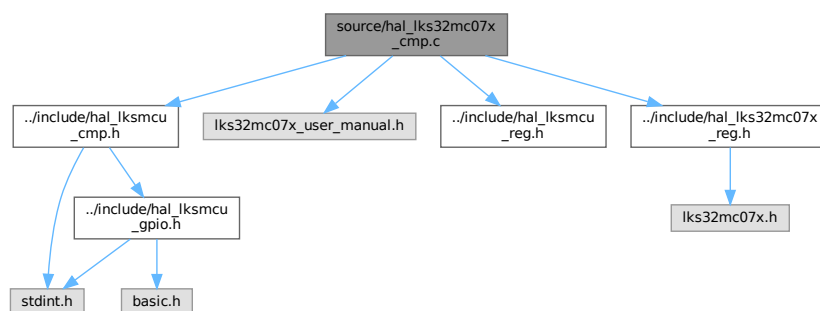
[浏览该文件的文档.](#)

```
00001 #include "../include/hal_lksmcu_clk.h"
00002 #include "lks32mc07x.h"
00003 #include "lks32mc07x_user_manual.h"
00004 #include "../include/hal_lksmcu_reg.h"
00005 #include "../include/hal_lks32mc07x_reg.h"
00006 static volatile uint32_t HAL_CLK_mclk = 96000000;
00011 void HAL_CLK_SetMcuFreq(HAL_CLK_FREQ_x FREQ_x)
00012 {
00013     HAL_REG_SYS_UNLOCK();
00014     switch (FREQ_x)
00015     {
00016         case HAL_CLK_FREQ_RC_4MHZ:
00017             HAL_CLK_mclk = 4000000;
00018             HAL_REG_WRITE(SYS_CLK_CFG, 0);
00019             break;
00020         case HAL_CLK_FREQ_RC_12MHZ:
00021             HAL_CLK_mclk = 12000000;
00022             HAL_REG_WRITE(SYS_CLK_CFG, 0x101);
00023             break;
00024         case HAL_CLK_FREQ_RC_24MHZ:
00025             HAL_CLK_mclk = 24000000;
00026             HAL_REG_WRITE(SYS_CLK_CFG, 0x111);
00027             break;
00028         case HAL_CLK_FREQ_RC_48MHZ:
00029             HAL_CLK_mclk = 48000000;
00030             HAL_REG_WRITE(SYS_CLK_CFG, 0x155);
00031             break;
00032         default:
00033             case HAL_CLK_FREQ_RC_96MHZ:
00034                 HAL_CLK_mclk = 96000000;
00035                 HAL_REG_WRITE(SYS_CLK_CFG, 0x1ff);
00036                 break;
00037     }
00038     HAL_REG_SYS_LOCK();
00039 }
00040
00045 uint32_t HAL_CLK_GetMcuFreq(void)
00046 {
00047     return HAL_CLK_mclk;
00048 }
```

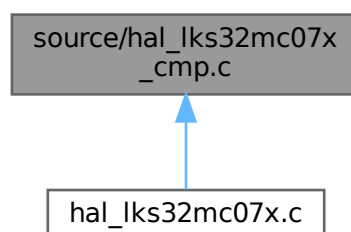

4.68 source/hal_lks32mc07x_cmp.c 文件参考

```
#include "../include/hal_lksmcu_cmp.h"
#include "lks32mc07x_user_manual.h"
#include "../include/hal_lksmcu_reg.h"
#include "../include/hal_lks32mc07x_reg.h"
```

hal_lks32mc07x_cmp.c 的引用(Include)关系图:



此图展示该文件直接或间接的被哪些文件引用了:



结构体

- struct **CMP_TypeDef**
ADC寄存器结构体句柄

函数

- void **HAL_CMP_Reset** (HAL_CMP_x cmp_x)
模块复位
- void **HAL_CMP_Enable** (HAL_CMP_x cmp_x)
模块打开
- void **HAL_CMP_Disable** (HAL_CMP_x cmp_x)

- 模块关闭
- void HAL_CMP_SetChannel (HAL_CMP_x cmp_x, HAL_CMP_CHANNEL_IPx channel_p_x, HAL_CMP_CHANNEL_INx channel_n_x)
模块关闭
 - uint32_t HAL_CMP_SetDigitalFilter (HAL_CMP_x cmpx, uint32_t filter)
获取CMP模块的数字滤波宽度
 - void HAL_CMP_SetGpioOut (HAL_CMP_x cmp_x, HAL_GPIO_Px cmpgpio)
设置CMP模块的GPIO
 - void HAL_CMP_SetGpioIn (HAL_CMP_x cmp_x, HAL_GPIO_Px cmpgpio)
设置CMP模块的GPIO
 - uint8_t HAL_CMP_GetData (HAL_CMP_x cmp_x)
获取CMP比较结果

4.68.1 函数说明

4.68.1.1 HAL_CMP_Disable()

```
void HAL_CMP_Disable (
    HAL_CMP_x cmp_x )
```

模块关闭

参数

<i>cmp_x</i>	使用的CMP模块编号
--------------	------------

在文件 `hal_lks32mc07x.cmp.c` 第 59 行定义.

这是这个函数的调用关系图:



4.68.1.2 HAL_CMP_Enable()

```
void HAL_CMP_Enable (
    HAL_CMP_x cmp_x )
```

模块打开

参数

<i>cmp</i> _↔ <i>_x</i>	使用的CMP模块编号
--------------------------------------	------------

在文件 [hal_lks32mc07x_cmp.c](#) 第 30 行定义.

4.68.1.3 HAL_CMP_GetData()

```
uint8_t HAL_CMP_GetData (
    HAL_CMP_x cmp_x )
```

获取CMP比较结果

参数

<i>cmp</i> _↔ <i>_x</i>	CMP模块编号
--------------------------------------	---------

返回
比较结果

在文件 [hal_lks32mc07x_cmp.c](#) 第 198 行定义.

4.68.1.4 HAL_CMP_Reset()

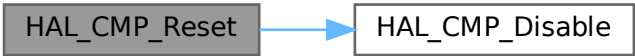
```
void HAL_CMP_Reset (
    HAL_CMP_x cmp_x )
```

模块复位

参数

<i>cmp</i> _↔ <i>_x</i>	使用的CMP模块编号
--------------------------------------	------------

在文件 [hal_lks32mc07x_cmp.c](#) 第 22 行定义.
函数调用图:



4.68.1.5 HAL_CMP_SetChannel()

```
void HAL_CMP_SetChannel (
    HAL_CMP_x cmp_x,
    HAL_CMP_CHANNEL_IPx channel_p_x,
    HAL_CMP_CHANNEL_INx channel_n_x )
```

模块关闭

参数

<i>cmp_x</i>	使用的CMP模块编号
<i>channel_↔ p_x</i>	正端通道
<i>channel_↔ n_x</i>	负端通道

在文件 [hal_lks32mc07x_cmp.c](#) 第 90 行定义.

4.68.1.6 HAL_CMP_SetDigitalFilter()

```
uint32_t HAL_CMP_SetDigitalFilter (
    HAL_CMP_x cmp_x,
    uint32_t filter )
```

获取CMP模块的数字滤波宽度

参数

<i>cmp_↔ _x</i>	CMP模块编号
<i>filter</i>	滤波宽度

返回

实际滤波宽度

在文件 [hal_lks32mc07x_cmp.c](#) 第 136 行定义.

4.68.1.7 HAL_CMP_SetGpioIn()

```
void HAL_CMP_SetGpioIn (
    HAL_CMP_x cmp_x,
    HAL_GPIO_Px cmpgpio )
```

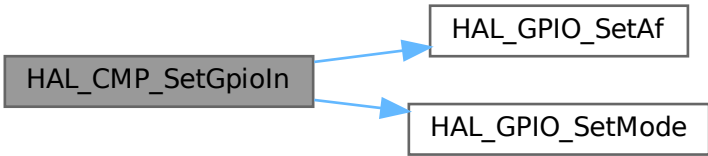
设置CMP模块的GPIO

参数

<i>cmp_x</i>	CMP模块编号
<i>cmpgpio</i>	GPIO编号

在文件 [hal_lks32mc07x_cmp.c](#) 第 186 行定义.

函数调用图:



4.68.1.8 HAL_CMP_SetGpioOut()

```
void HAL_CMP_SetGpioOut (
    HAL_CMP_x cmp_x,
    HAL_GPIO_Px cmpgpio )
```

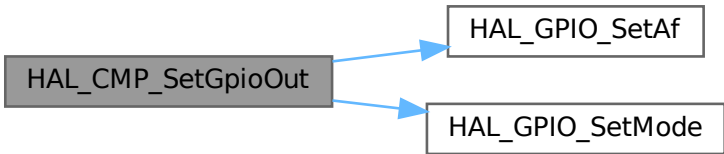
设置CMP模块的GPIO

参数

<i>cmp_x</i>	cmp模块编号
<i>cmpgpio</i>	GPIO编号

在文件 [hal_lks32mc07x_cmp.c](#) 第 174 行定义.

函数调用图:



4.69 hal_lks32mc07x_cmp.c

[浏览该文件的文档。](#)

```

00001 #include "../include/hal_lksmcu_cmp.h"
00002 #include "lks32mc07x_user_manual.h"
00003 #include "../include/hal_lksmcu_reg.h"
00004 #include "../include/hal_lks32mc07x_reg.h"
00005
00009 typedef struct
00010 {
00011     volatile uint32_t IE;
00012     volatile uint32_t IF;
00013     volatile uint32_t TCLK;
00014     volatile uint32_t CFG;
00015     volatile uint32_t BLCWIN;
00016     volatile uint32_t DATA;
00017 } CMP_TypeDef;
00022 void HAL_CMP_Reset (HAL_CMP_x cmp_x)
00023 {
00024     HAL_CMP_Disable (cmp_x);
00025 }
00030 void HAL_CMP_Enable (HAL_CMP_x cmp_x)
00031 {
00032     // 系统寄存器解锁
00033     HAL_REG_SYS_UNLOCK();
00034     switch (cmp_x)
00035     {
00036         case HAL_CMP_0:
00037             HAL_REG_SET (SYS_AFE_REG5, BIT6);
00038             HAL_REG_SET (HAL_CMP[0] -> CFG, CMP_CFG_CMP0_IN_EN_BIT);
00039             break;
00040         case HAL_CMP_1:
00041             HAL_REG_SET (SYS_AFE_REG5, BIT7);
00042             HAL_REG_SET (HAL_CMP[0] -> CFG, CMP_CFG_CMP1_IN_EN_BIT);
00043             break;
00044         case HAL_CMP_2:
00045             HAL_REG_SET (SYS_AFE_REG5, BIT11);
00046             HAL_REG_SET (HAL_CMP[0] -> CFG, CMP_CFG_CMP2_IN_EN_BIT);
00047             break;
00048         default:
00049             break;
00050     }
00051     HAL_REG_SET (SYS_CLK_FEN, SYS_CLK_FEN_CMP_CLK_EN_BIT);
00052     // 系统寄存器上锁
00053     HAL_REG_SYS_LOCK();
00054 }
00059 void HAL_CMP_Disable (HAL_CMP_x cmp_x)
00060 {
00061     // 系统寄存器解锁
00062     HAL_REG_SYS_UNLOCK();
00063     switch (cmp_x)
00064     {
00065         case HAL_CMP_0:
00066             HAL_REG_RESET (SYS_AFE_REG5, BIT6);
00067             HAL_REG_RESET (HAL_CMP[0] -> CFG, CMP_CFG_CMP0_IN_EN_BIT);
00068             break;
00069         case HAL_CMP_1:
00070             HAL_REG_RESET (SYS_AFE_REG5, BIT7);
00071             HAL_REG_RESET (HAL_CMP[0] -> CFG, CMP_CFG_CMP1_IN_EN_BIT);
00072             break;
00073         case HAL_CMP_2:
00074             HAL_REG_RESET (SYS_AFE_REG5, BIT11);
00075             HAL_REG_RESET (HAL_CMP[0] -> CFG, CMP_CFG_CMP2_IN_EN_BIT);
00076             break;
00077         default:
00078             break;
00079     }
00080     HAL_REG_IF_WRITE (SYS_CLK_FEN, SYS_CLK_FEN_CMP_CLK_EN_BIT, !(SYS_AFE_REG5 & (BIT6 | BIT7 | BIT11)));
00081     // 系统寄存器上锁
00082     HAL_REG_SYS_LOCK();
00083 }
00090 void HAL_CMP_SetChannel (HAL_CMP_x cmp_x, HAL_CMP_CHANNEL_IPx channel_p_x, HAL_CMP_CHANNEL_INx channel_n_x)
00091 {
00092     switch (cmp_x)
00093     {
00094         case HAL_CMP_0:
00095             // 系统寄存器解锁
00096             HAL_REG_SYS_UNLOCK();
00097             HAL_REG_WRITE_BIT_MASK_POS (SYS_AFE_REG3, SYS_AFE_REG3_CMP0_SEL_P, channel_p_x);
00098             HAL_REG_WRITE_BIT_MASK_POS (SYS_AFE_REG3, SYS_AFE_REG3_CMP0_SEL_N, channel_n_x);
00099             // 系统寄存器上锁
00100             HAL_REG_SYS_LOCK();
00101             break;
00102         case HAL_CMP_1:
00103             // 系统寄存器解锁

```

```

00104         HAL_REG_SYS_UNLOCK();
00105         HAL_REG_WRITE_BIT_MASK_POS(SYS_AFE_REG3, SYS_AFE_REG3_CMP1_SEL_P, channel_p.x);
00106         HAL_REG_WRITE_BIT_MASK_POS(SYS_AFE_REG3, SYS_AFE_REG3_CMP1_SEL_N, channel_n.x);
00107         // 系统寄存器上锁
00108         HAL_REG_SYS_LOCK();
00109         break;
00110     case HAL_CMP_2:
00111         // 系统寄存器解锁
00112         HAL_REG_SYS_UNLOCK();
00113         if (channel_p.x == HAL_CMP_CHANNEL_IP1)
00114         {
00115             HAL_REG_WRITE_BIT_MASK_POS(SYS_AFE_REG2, SYS_AFE_REG2_CMP2_SEL_P, 1);
00116         }
00117         else
00118         {
00119             HAL_REG_WRITE_BIT_MASK_POS(SYS_AFE_REG2, SYS_AFE_REG2_CMP2_SEL_P, channel_p.x);
00120         }
00121         HAL_REG_WRITE_BIT_MASK_POS(SYS_AFE_REG3, SYS_AFE_REG3_CMP2_SEL_N, channel_n.x);
00122         // 系统寄存器上锁
00123         HAL_REG_SYS_LOCK();
00124         break;
00125     default:
00126         break;
00127 }
00128 }
00129
00136 uint32_t HAL_CMP_SetDigitalFilter(HAL_CMP_x cmpx, uint32_t filter)
00137 {
00138     (void)cmpx;
00139     if (filter == 0)
00140     {
00141         HAL_REG_RESET(HAL_CMP[0]->TCLK, CMP_TCLK_COMM_DIV_MASK | CMP_TCLK_FIL2_CLK_DIV16_MASK |
00142             CMP_TCLK_CLK2_EN_BIT | CMP_TCLK_FIL2_CLK_DIV2_MASK | CMP_TCLK_FIL1_CLK_DIV16_MASK | CMP_TCLK_CLK1_EN_BIT |
00143             CMP_TCLK_FIL1_CLK_DIV2_MASK | CMP_TCLK_FIL0_CLK_DIV16_MASK | CMP_TCLK_CLK0_EN_BIT |
00144             CMP_TCLK_FIL0_CLK_DIV2_MASK);
00145     }
00146     else
00147     {
00148         uint16_t div_2 = 0;
00149         for (; filter > 0xf; )
00150         {
00151             filter >>= 1;
00152             div_2++;
00153         }
00154         if (div_2 > CMP_TCLK_FIL0_CLK_DIV2_MAX)
00155         {
00156             div_2 = CMP_TCLK_FIL0_CLK_DIV2_MAX;
00157             filter = CMP_TCLK_FIL0_CLK_DIV16_MAX;
00158         }
00159         HAL_REG_SET(HAL_CMP[0]->TCLK, CMP_TCLK_CLK0_EN_BIT | CMP_TCLK_CLK1_EN_BIT | CMP_TCLK_CLK2_EN_BIT);
00160         HAL_REG_WRITE_BIT_MASK_POS(HAL_CMP[0]->TCLK, CMP_TCLK_FIL0_CLK_DIV2, div_2);
00161         HAL_REG_WRITE_BIT_MASK_POS(HAL_CMP[0]->TCLK, CMP_TCLK_FIL0_CLK_DIV16, filter);
00162         HAL_REG_WRITE_BIT_MASK_POS(HAL_CMP[0]->TCLK, CMP_TCLK_FIL1_CLK_DIV2, div_2);
00163         HAL_REG_WRITE_BIT_MASK_POS(HAL_CMP[0]->TCLK, CMP_TCLK_FIL1_CLK_DIV16, filter);
00164         HAL_REG_WRITE_BIT_MASK_POS(HAL_CMP[0]->TCLK, CMP_TCLK_FIL2_CLK_DIV2, div_2);
00165         HAL_REG_WRITE_BIT_MASK_POS(HAL_CMP[0]->TCLK, CMP_TCLK_FIL2_CLK_DIV16, filter);
00166     }
00167     filter = filter << div_2;
00168     return filter;
00169 }
00170
00174 void HAL_CMP_SetGpioOut(HAL_CMP_x cmp_x, HAL_GPIO_Px cmp_gpio)
00175 {
00176     (void)cmp_x;
00177     HAL_GPIO_SetAf(cmp_gpio, HAL_GPIO_AF_CMP_OUT);
00178     HAL_GPIO_SetMode(cmp_gpio, HAL_GPIO_MODE_OUT);
00179 }
00180
00186 void HAL_CMP_SetGpioIn(HAL_CMP_x cmp_x, HAL_GPIO_Px cmp_gpio)
00187 {
00188     (void)cmp_x;
00189     HAL_GPIO_SetAf(cmp_gpio, HAL_GPIO_AF_CMP);
00190     HAL_GPIO_SetMode(cmp_gpio, HAL_GPIO_MODE_AFE);
00191 }
00192
00198 uint8_t HAL_CMP_GetData(HAL_CMP_x cmp_x)
00199 {
00200     uint32_t flg = 0;
00201     switch (cmp_x)
00202     {
00203     case HAL_CMP_0:
00204         if (HAL_REG_READ_BIT(HAL_CMP[0]->TCLK, CMP_TCLK_CLK0_EN_BIT))
00205         {
00206             // 打开滤波之后返回滤波后的结果
00207             flg = HAL_REG_READ_BIT(HAL_CMP[0]->DATA, CMP_DATA_CMP0_FLT_DATA_BIT);
00208         }
00209     }
00210 }

```

```

00209         else
00210         {
00211             // 如果没有配置滤波, 则返回滤波前的结果
00212             flg = HAL_REG_READ_BIT(HAL_CMP[0]->DATA, CMP_DATA_CMP0_RAW_DATA_BIT);
00213         }
00214         break;
00215     case HAL_CMP_1:
00216         if (HAL_REG_READ_BIT(HAL_CMP[0]->TCLK, CMP_TCLK_CLK1_EN_BIT))
00217         {
00218             // 打开滤波之后返回滤波后的结果
00219             flg = HAL_REG_READ_BIT(HAL_CMP[0]->DATA, CMP_DATA_CMP1_FLT_DATA_BIT);
00220         }
00221         else
00222         {
00223             // 如果没有配置滤波, 则返回滤波前的结果
00224             flg = HAL_REG_READ_BIT(HAL_CMP[0]->DATA, CMP_DATA_CMP1_RAW_DATA_BIT);
00225         }
00226         break;
00227     case HAL_CMP_2:
00228         if (HAL_REG_READ_BIT(HAL_CMP[0]->TCLK, CMP_TCLK_CLK2_EN_BIT))
00229         {
00230             // 打开滤波之后返回滤波后的结果
00231             flg = HAL_REG_READ_BIT(HAL_CMP[0]->DATA, CMP_DATA_CMP2_FLT_DATA_BIT);
00232         }
00233         else
00234         {
00235             // 如果没有配置滤波, 则返回滤波前的结果
00236             flg = HAL_REG_READ_BIT(HAL_CMP[0]->DATA, CMP_DATA_CMP2_RAW_DATA_BIT);
00237         }
00238         break;
00239     default:
00240         break;
00241     }
00242     if (flg == 0)
00243     {
00244         return 0;
00245     }
00246     else
00247     {
00248         return 1;
00249     }
00250 }

```

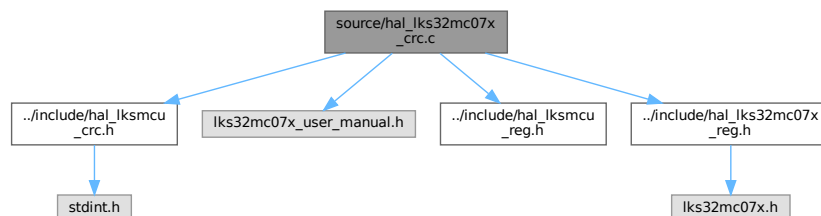
4.70 source/hal_lks32mc07x_crc.c 文件参考

```

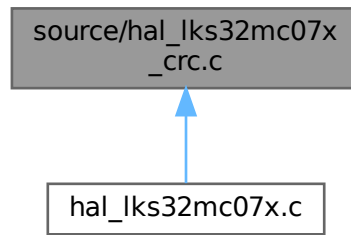
#include "../include/hal_lksmcu_crc.h"
#include "lks32mc07x_user_manual.h"
#include "../include/hal_lksmcu_reg.h"
#include "../include/hal_lks32mc07x_reg.h"

```

hal_lks32mc07x_crc.c 的引用(Include)关系图:



此图展示该文件直接或间接的被哪些文件引用了:



结构体

- struct [CRC_Model_Params](#)

函数

- void [HAL_CRC_Reset](#) ([HAL_CRC_x](#) crc_x)
- void [HAL_CRC_Enable](#) ([HAL_CRC_x](#) crc_x)
- void [HAL_CRC_Disable](#) ([HAL_CRC_x](#) crc_x)
- static uint32_t [HAL_CRC_Reflect](#) (uint32_t data, int width)
- uint32_t [HAL_CRC_Calc](#) ([HAL_CRC_x](#) crc_x, [HAL_CRC_MODEL_x](#) model, uint8_t *data, uint32_t len)

变量

- static const [CRC_Model_Params](#) [crcModels](#) []

4.70.1 函数说明

4.70.1.1 HAL_CRC_Calc()

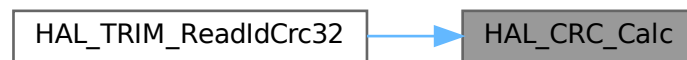
```
uint32_t HAL_CRC_Calc (
    HAL\_CRC\_x crc_x,
    HAL\_CRC\_MODEL\_x model,
    uint8_t * data,
    uint32_t len )
```

在文件 [hal_lks32mc07x_crc.c](#) 第 59 行定义.

函数调用图:



这是这个函数的调用关系图:



4.70.1.2 HAL_CRC.Disable()

```
void HAL_CRC_Disable (
    HAL_CRC_x crc_x )
```

在文件 [hal_lks32mc07x.crc.c](#) 第 16 行定义.

4.70.1.3 HAL_CRC.Enable()

```
void HAL_CRC_Enable (
    HAL_CRC_x crc_x )
```

在文件 [hal_lks32mc07x.crc.c](#) 第 11 行定义.

4.70.1.4 HAL_CRC.Reflect()

```
static uint32_t HAL_CRC_Reflect (
    uint32_t data,
    int width ) [static]
```

在文件 [hal_lks32mc07x.crc.c](#) 第 47 行定义.

这是这个函数的调用关系图:



4.70.1.5 HAL_CRC.Reset()

```
void HAL_CRC_Reset (
    HAL_CRC_x crc_x )
```

在文件 [hal_lks32mc07x.crc.c](#) 第 6 行定义.

4.70.2 变量说明

4.70.2.1 crcModels

```
const CRC_Model_Params crcModels[] [static]
```

初始值:

```
= {
    {0x07, 0x00, 0x00, 0, 0},
    {0x07, 0x00, 0x55, 0, 0},
    {0x07, 0xFF, 0x00, 1, 1},
    {0x31, 0x00, 0x00, 1, 1},
    {0x8005, 0x0000, 0x0000, 1, 1},
    {0x8005, 0x0000, 0xFFFF, 1, 1},
    {0x8005, 0xFFFF, 0xFFFF, 1, 1},
    {0x8005, 0xFFFF, 0x0000, 1, 1},
    {0x1021, 0x0000, 0x0000, 1, 1},
    {0x1021, 0xFFFF, 0x0000, 0, 0},
    {0x1021, 0xFFFF, 0xFFFF, 1, 1},
    {0x1021, 0x0000, 0x0000, 0, 0},
    {0x3D65, 0x0000, 0xFFFF, 1, 1},
    {0x04C11DB7, 0xFFFFFFFF, 0xFFFFFFFF, 1, 1},
    {0x04C11DB7, 0xFFFFFFFF, 0x00000000, 0, 0},
}
```

在文件 `hal_lks32mc07x_crc.c` 第 29 行定义.

4.71 hal_lks32mc07x_crc.c

[浏览该文件的文档.](#)

```
00001 #include "../include/hal_lksmcu_crc.h"
00002 #include "lks32mc07x_user_manual.h"
00003 #include "../include/hal_lksmcu_reg.h"
00004 #include "../include/hal_lks32mc07x_reg.h"
00005 // 模块复位
00006 void HAL_CRC_Reset(HAL_CRC_x crc_x)
00007 {
00008     (void)crc_x;
00009 }
00010 // 模块打开
00011 void HAL_CRC_Enable(HAL_CRC_x crc_x)
00012 {
00013     (void)crc_x;
00014 }
00015 // 模块关闭
00016 void HAL_CRC_Disable(HAL_CRC_x crc_x)
00017 {
00018     (void)crc_x;
00019 }
00020 typedef struct
00021 {
00022     uint32_t polynomial;
00023     uint32_t initValue;
00024     uint32_t xorOut;
00025     uint8_t reflectIn;
00026     uint8_t reflectOut;
00027 } CRC_Model_Params;
00028
00029 static const CRC_Model_Params crcModels[] = {
00030     {0x07, 0x00, 0x00, 0, 0}, // CRC8
00031     {0x07, 0x00, 0x55, 0, 0}, // CRC8-ITU
00032     {0x07, 0xFF, 0x00, 1, 1}, // CRC8-ROHC
00033     {0x31, 0x00, 0x00, 1, 1}, // CRC8-MAXIMx
00034     {0x8005, 0x0000, 0x0000, 1, 1}, // CRC16-IBMx1
00035     {0x8005, 0x0000, 0xFFFF, 1, 1}, // CRC16-MAXIM
00036     {0x8005, 0xFFFF, 0xFFFF, 1, 1}, // CRC16-USBx1
00037     {0x8005, 0xFFFF, 0x0000, 1, 1}, // CRC16-MODBU
00038     {0x1021, 0x0000, 0x0000, 1, 1}, // CRC16-CCITT
00039     {0x1021, 0xFFFF, 0x0000, 0, 0}, // CRC16-CCITT_FALSE
00040     {0x1021, 0xFFFF, 0xFFFF, 1, 1}, // CRC16-X25
00041     {0x1021, 0x0000, 0x0000, 0, 0}, // CRC16-XMODEM
00042     {0x3D65, 0x0000, 0xFFFF, 1, 1}, // CRC16-DNP
00043     {0x04C11DB7, 0xFFFFFFFF, 0xFFFFFFFF, 1, 1}, // CRC32
00044     {0x04C11DB7, 0xFFFFFFFF, 0x00000000, 0, 0}, // CRC32-MPEG-2
00045 };
00046
```

```

00047 static uint32_t HAL_CRC_Reflect(uint32_t data, int width)
00048 {
00049     uint32_t result = 0;
00050     for (int i = 0; i < width; i++)
00051     {
00052         if (data & (1 << i))
00053         {
00054             result |= 1 << (width - 1 - i);
00055         }
00056     }
00057     return result;
00058 }
00059 uint32_t HAL_CRC_Calc(HAL_CRC_t crc_x, HAL_CRC_MODEL_t model, uint8_t *data, uint32_t len)
00060 {
00061     (void)crc_x;
00062     uint32_t crc;
00063     const CRC_Model_Params *params;
00064
00065     if (model >= sizeof(crcModels) / sizeof(crcModels[0]))
00066     {
00067         return 0; // Invalid model
00068     }
00069
00070     params = &crcModels[model];
00071     crc = params->initValue;
00072
00073     for (uint32_t i = 0; i < len; i++)
00074     {
00075         uint8_t byte = data[i];
00076         if (params->reflectIn)
00077         {
00078             byte = HAL_CRC_Reflect(byte, 8);
00079         }
00080
00081         crc ^= (uint32_t)byte << 24;
00082
00083         for (int j = 0; j < 8; j++)
00084         {
00085             if (crc & 0x80000000)
00086             {
00087                 crc = (crc << 1) ^ params->polynomial;
00088             }
00089             else
00090             {
00091                 crc <<= 1;
00092             }
00093         }
00094     }
00095
00096     if (params->reflectOut)
00097     {
00098         crc = HAL_CRC_Reflect(crc, (params->polynomial >> 24) + 1); // Assuming polynomial width
00099         // indicates CRC width
00100     }
00101     crc ^= params->xorOut;
00102     return crc;
00103 }

```

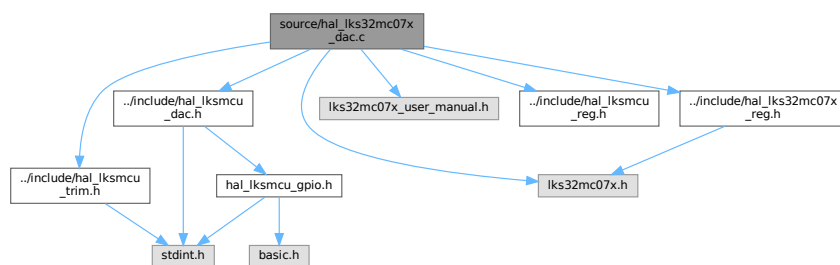
4.72 source/hal_lks32mc07x_dac.c 文件参考

```

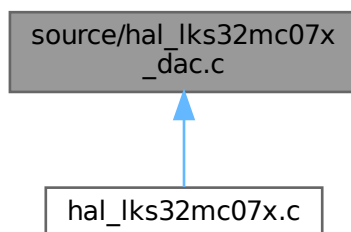
#include "../include/hal_lksmcu.dac.h"
#include "../include/hal_lksmcu.trim.h"
#include "lks32mc07x.h"
#include "lks32mc07x_user_manual.h"
#include "../include/hal_lksmcu.reg.h"
#include "../include/hal_lks32mc07x.reg.h"

```

hal_lks32mc07x_dac.c 的引用(Include)关系图:



此图展示该文件直接或间接的被哪些文件引用了:



函数

- void [HAL_DAC_Reset](#) (HAL_DAC_x dac_x)
- void [HAL_DAC_Enable](#) (HAL_DAC_x dac_x)
- void [HAL_DAC_Disable](#) (HAL_DAC_x dac_x)
- void [HAL_DAC_SetOutVal](#) (HAL_DAC_x dac_x, uint16_t val)
- void [HAL_DAC_SetOutVoltage](#) (HAL_DAC_x dac_x, uint16_t vol_mV)
- void [HAL_DAC_Out2GpioEnable](#) (HAL_DAC_x dac_x)
- void [HAL_DAC_Out2GpioDisable](#) (HAL_DAC_x dac_x)
- void [HAL_DAC_SetGpioOut](#) (HAL_DAC_x dac, HAL_GPIO_Px px)

设置DAC输出到GPIO

4.72.1 函数说明

4.72.1.1 HAL_DAC_Disable()

```
void HAL_DAC_Disable (
    HAL_DAC_x dac_x )
```

在文件 [hal_lks32mc07x_dac.c](#) 第 32 行定义.

这是这个函数的调用关系图:



4.72.1.2 HAL_DAC.Enable()

```
void HAL_DAC_Enable (
    HAL_DAC_TypeDef dac_x )
```

在文件 [hal_lks32mc07x.dac.c](#) 第 13 行定义.

4.72.1.3 HAL_DAC.Out2GpioDisable()

```
void HAL_DAC_Out2GpioDisable (
    HAL_DAC_TypeDef dac_x )
```

在文件 [hal_lks32mc07x.dac.c](#) 第 169 行定义.

4.72.1.4 HAL_DAC.Out2GpioEnable()

```
void HAL_DAC_Out2GpioEnable (
    HAL_DAC_TypeDef dac_x )
```

在文件 [hal_lks32mc07x.dac.c](#) 第 151 行定义.

4.72.1.5 HAL_DAC.Reset()

```
void HAL_DAC_Reset (
    HAL_DAC_TypeDef dac_x )
```

在文件 [hal_lks32mc07x.dac.c](#) 第 8 行定义.

函数调用图:



4.72.1.6 HAL_DAC_SetGpioOut()

```
void HAL_DAC_SetGpioOut (
    HAL_DAC_x dac,
    HAL_GPIO_Px px )
```

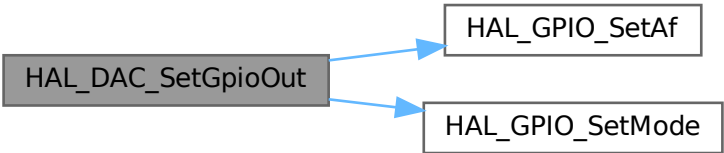
设置DAC输出到GPIO

参数

<i>dac</i>	DAC模块
<i>px</i>	GPIO

在文件 [hal_lks32mc07x.dac.c](#) 第 193 行定义.

函数调用图:



4.72.1.7 HAL_DAC_SetOutVal()

```
void HAL_DAC_SetOutVal (
    HAL_DAC_x dac_x,
    uint16_t val )
```

在文件 [hal_lks32mc07x.dac.c](#) 第 51 行定义.

4.72.1.8 HAL_DAC_SetOutVoltagemV()

```
void HAL_DAC_SetOutVoltagemV (
    HAL_DAC_x dac_x,
    uint16_t vol_mV )
```

在文件 [hal_lks32mc07x.dac.c](#) 第 65 行定义.

函数调用图:



4.73 hal_lks32mc07x_dac.c

[浏览该文件的文档。](#)

```

00001 #include "../include/hal_lksmcu_dac.h"
00002 #include "../include/hal_lksmcu_trim.h"
00003 #include "lks32mc07x.h"
00004 #include "lks32mc07x_user_manual.h"
00005 #include "../include/hal_lksmcu_reg.h"
00006 #include "../include/hal_lks32mc07x_reg.h"
00007 // 模块复位
00008 void HAL_DAC_Reset (HAL_DAC_x dac_x)
00009 {
00010     HAL_DAC_Disable(dac_x);
00011 }
00012 // 模块打开
00013 void HAL_DAC_Enable (HAL_DAC_x dac_x)
00014 {
00015     // 系统寄存器解锁
00016     HAL_REG_SYS_UNLOCK();
00017     switch (dac_x)
00018     {
00019         case HAL_DAC_0:
00020             HAL_REG_SET(SYS_AFE_REG5, BIT12);
00021             break;
00022         case HAL_DAC_1:
00023             HAL_REG_SET(SYS_AFE_REG5, BIT9);
00024             break;
00025         default:
00026             break;
00027     }
00028     // 系统寄存器上锁
00029     HAL_REG_SYS_LOCK();
00030 }
00031 // 模块关闭
00032 void HAL_DAC_Disable (HAL_DAC_x dac_x)
00033 {
00034     // 系统寄存器解锁
00035     HAL_REG_SYS_UNLOCK();
00036     switch (dac_x)
00037     {
00038         case HAL_DAC_0:
00039             HAL_REG_RESET(SYS_AFE_REG5, BIT12);
00040             break;
00041         case HAL_DAC_1:
00042             HAL_REG_RESET(SYS_AFE_REG5, BIT9);
00043             break;
00044         default:
00045             break;
00046     }
00047     // 系统寄存器上锁
00048     HAL_REG_SYS_LOCK();
00049 }
00050
00051 void HAL_DAC_SetOutVal (HAL_DAC_x dac_x, uint16_t val)
00052 {
00053     switch (dac_x)
00054     {
00055         case HAL_DAC_0:
00056             HAL_REG_WRITE(SYS_AFE_DAC0, val);
00057             break;
00058         case HAL_DAC_1:
00059             HAL_REG_WRITE(SYS_AFE_DAC1, val);
00060             break;
00061         default:
00062             break;
00063     }
00064 }
00065 void HAL_DAC_SetOutVoltageV (HAL_DAC_x dac_x, uint16_t vol_mV)
00066 {
00067     // 系统寄存器解锁
00068     int32_t val;
00069     HAL_REG_SYS_UNLOCK();
00070     switch (dac_x)
00071     {
00072         case HAL_DAC_0:
00073             if (vol_mV < 1000)
00074             {
00075                 val = vol_mV;
00076                 val = (val * 27853 + BIT12) >> 13;
00077                 if (val > 4095)
00078                 {
00079                     val = 4095;
00080                 }
00081                 if (val < 0)
00082                 {

```



```

00083         val = 0;
00084     }
00085     SYS_AFE_DAC0_DC = HAL_TRIM_Read(0x0000145C);
00086     SYS_AFE_DAC0_AMC = HAL_TRIM_Read(0x00001458);
00087     HAL_REG_SET(SYS_AFE_REG1, BIT6);
00088     HAL_REG_WRITE(SYS_AFE_DAC0, val);
00089 }
00090 else
00091 {
00092     val = vol_mV;
00093     val = (val * 27674 + BIT14) >> 15;
00094     if (val > 4095)
00095     {
00096         val = 4095;
00097     }
00098     if (val < 0)
00099     {
00100         val = 0;
00101     }
00102     SYS_AFE_DAC0_DC = HAL_TRIM_Read(0x00001454);
00103     SYS_AFE_DAC0_AMC = HAL_TRIM_Read(0x00001450);
00104     HAL_REG_RESET(SYS_AFE_REG1, BIT6);
00105     HAL_REG_WRITE(SYS_AFE_DAC0, val);
00106 }
00107 break;
00108 case HAL_DAC1:
00109     if (vol_mV < 1000)
00110     {
00111         val = vol_mV;
00112         val = (val * 27853 + BIT12) >> 13;
00113         if (val > 4095)
00114         {
00115             val = 4095;
00116         }
00117         if (val < 0)
00118         {
00119             val = 0;
00120         }
00121         SYS_AFE_DAC1_DC = HAL_TRIM_Read(0x0000146C);
00122         SYS_AFE_DAC1_AMC = HAL_TRIM_Read(0x00001468);
00123         HAL_REG_SET(SYS_AFE_REG1, BIT7);
00124         HAL_REG_WRITE(SYS_AFE_DAC1, val);
00125     }
00126     else
00127     {
00128         val = vol_mV;
00129         val = (val * 27674 + BIT14) >> 15;
00130         if (val > 4095)
00131         {
00132             val = 4095;
00133         }
00134         if (val < 0)
00135         {
00136             val = 0;
00137         }
00138         SYS_AFE_DAC1_DC = HAL_TRIM_Read(0x00001464);
00139         SYS_AFE_DAC1_AMC = HAL_TRIM_Read(0x00001460);
00140         HAL_REG_RESET(SYS_AFE_REG1, BIT7);
00141         HAL_REG_WRITE(SYS_AFE_DAC1, val);
00142     }
00143     break;
00144 default:
00145     break;
00146 }
00147 // 系统寄存器上锁
00148 HAL_REG_SYS_LOCK();
00149 }
00150
00151 void HAL_DAC_Out2GpioEnable(HAL_DAC_x dac_x)
00152 {
00153     // 系统寄存器解锁
00154     HAL_REG_SYS_UNLOCK();
00155     switch (dac_x)
00156     {
00157         case HAL_DAC_0:
00158             HAL_REG_SET(SYS_AFE_REG3, BIT11);
00159             break;
00160         case HAL_DAC_1:
00161             HAL_REG_SET(SYS_AFE_REG3, BIT15);
00162             break;
00163         default:
00164             break;
00165     }
00166     // 系统寄存器上锁
00167     HAL_REG_SYS_LOCK();
00168 }
00169 void HAL_DAC_Out2GpioDisable(HAL_DAC_x dac_x)

```

```

00170 {
00171     // 系统寄存器解锁
00172     HAL_REG_SYS_UNLOCK();
00173     switch (dac_x)
00174     {
00175         case HAL_DAC_0:
00176             HAL_REG_RESET(SYS_AFE_REG3, BIT11);
00177             break;
00178         case HAL_DAC_1:
00179             HAL_REG_RESET(SYS_AFE_REG3, BIT15);
00180             break;
00181         default:
00182             break;
00183     }
00184     // 系统寄存器上锁
00185     HAL_REG_SYS_LOCK();
00186 }
00187
00193 void HAL_DAC_SetGpioOut(HAL_DAC_x dac, HAL_GPIO_Px px)
00194 {
00195     (void)dac;
00196
00197     HAL_GPIO_SetAf(px, HAL_GPIO_AF_DAC);
00198     HAL_GPIO_SetMode(px, HAL_GPIO_MODE_AFE);
00199 }

```

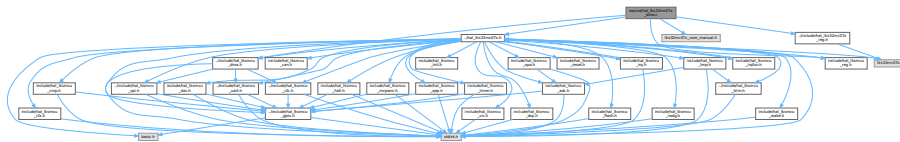
4.74 source/hal_lks32mc07x_dma.c 文件参考

```

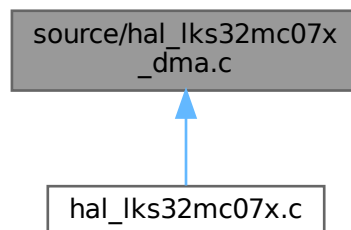
#include "../include/hal_lksmcu_dma.h"
#include "../hal_lks32mc07x.h"
#include "lks32mc07x-user-manual.h"
#include "../include/hal_lksmcu_reg.h"
#include "../include/hal_lks32mc07x_reg.h"

```

hal_lks32mc07x_dma.c 的引用(Include)关系图:



此图展示该文件直接或间接的被哪些文件引用了:



函数

- void [HAL_DMA_Reset](#) ([HAL_DMA_x](#) dma_x)
- void [HAL_DMA_Enable](#) ([HAL_DMA_x](#) dma_x)
- void [HAL_DMA_Disable](#) ([HAL_DMA_x](#) dma_x)
- void [HAL_DMA_MemCopy](#) ([HAL_DMA_x](#) dma_x, void *start, void *end, uint32_t len)
- void [HAL_DMA_UartTx](#) ([HAL_DMA_x](#) dma_x, [HAL_UART_x](#) UART_x, uint8_t *start, uint32_t len)
- void [HAL_DMA_SpiTx](#) ([HAL_DMA_x](#) dma_x, [HAL_SPI_x](#) SPI_x, uint8_t *start, uint32_t len)
- void [HAL_DMA_I2cTx](#) ([HAL_DMA_x](#) dma_x, [HAL_I2C_x](#) I2Cx, uint8_t *start, uint32_t len)
- void [HAL_DMA_UartRx](#) ([HAL_DMA_x](#) dma_x, [HAL_UART_x](#) UART_x, uint8_t *start, uint32_t len)
- void [HAL_DMA_SpiRx](#) ([HAL_DMA_x](#) dma_x, [HAL_SPI_x](#) SPI_x, uint8_t *start, uint32_t len)
- void [HAL_DMA_I2cRx](#) ([HAL_DMA_x](#) dma_x, [HAL_I2C_x](#) I2Cx, uint8_t *start, uint32_t len)
- void [HAL_DMA_SpiRxtx](#) ([HAL_DMA_x](#) DMA1_x, [HAL_DMA_x](#) DMA2_x, [HAL_SPI_x](#) SPI_x, uint8_t *txdata, uint8_t *rxdata, uint32_t len)

变量

- static uint8_t [HAL_DMA_ena](#) = 0

4.74.1 函数说明

4.74.1.1 [HAL_DMA_Disable\(\)](#)

```
void HAL_DMA_Disable (
    HAL\_DMA\_x dma_x )
```

在文件 [hal_lks32mc07x_dma.c](#) 第 29 行定义.

4.74.1.2 [HAL_DMA_Enable\(\)](#)

```
void HAL_DMA_Enable (
    HAL\_DMA\_x dma_x )
```

在文件 [hal_lks32mc07x_dma.c](#) 第 22 行定义.

4.74.1.3 [HAL_DMA_I2cRx\(\)](#)

```
void HAL_DMA_I2cRx (
    HAL\_DMA\_x dma_x,
    HAL\_I2C\_x I2Cx,
    uint8_t * start,
    uint32_t len )
```

在文件 [hal_lks32mc07x_dma.c](#) 第 98 行定义.

4.74.1.4 HAL_DMA_I2cTx()

```
void HAL_DMA_I2cTx (
    HAL_DMA_x dma_x,
    HAL_I2C_x I2Cx,
    uint8_t * start,
    uint32_t len )
```

在文件 [hal_lks32mc07x_dma.c](#) 第 68 行定义.

4.74.1.5 HAL_DMA_MemCopy()

```
void HAL_DMA_MemCopy (
    HAL_DMA_x dma_x,
    void * start,
    void * end,
    uint32_t len )
```

在文件 [hal_lks32mc07x_dma.c](#) 第 38 行定义.

4.74.1.6 HAL_DMA_Reset()

```
void HAL_DMA_Reset (
    HAL_DMA_x dma_x )
```

在文件 [hal_lks32mc07x_dma.c](#) 第 10 行定义.

4.74.1.7 HAL_DMA_SpiRx()

```
void HAL_DMA_SpiRx (
    HAL_DMA_x dma_x,
    HAL_SPI_x SPI_x,
    uint8_t * start,
    uint32_t len )
```

在文件 [hal_lks32mc07x_dma.c](#) 第 88 行定义.

4.74.1.8 HAL_DMA_SpiRxDMA()

```
void HAL_DMA_SpiRxDMA (
    HAL_DMA_x DMA1_x,
    HAL_DMA_x DMA2_x,
    HAL_SPI_x SPI_x,
    uint8_t * txdata,
    uint8_t * rxdata,
    uint32_t len )
```

在文件 [hal_lks32mc07x_dma.c](#) 第 108 行定义.

4.74.1.9 HAL_DMA_SpiTx()

```
void HAL_DMA_SpiTx (
    HAL_DMA_x dma_x,
    HAL_SPI_x SPI_x,
    uint8_t * start,
    uint32_t len )
```

在文件 [hal_lks32mc07x_dma.c](#) 第 58 行定义.

4.74.1.10 HAL_DMA_UartRx()

```
void HAL_DMA_UartRx (
    HAL_DMA_x dma_x,
    HAL_UART_x UART_x,
    uint8_t * start,
    uint32_t len )
```

在文件 [hal_lks32mc07x_dma.c](#) 第 78 行定义.

4.74.1.11 HAL_DMA_UartTx()

```
void HAL_DMA_UartTx (
    HAL_DMA_x dma_x,
    HAL_UART_x UART_x,
    uint8_t * start,
    uint32_t len )
```

在文件 [hal_lks32mc07x_dma.c](#) 第 48 行定义.

4.74.2 变量说明

4.74.2.1 HAL_DMA_ena

```
uint8_t HAL_DMA_ena = 0 [static]
```

在文件 [hal_lks32mc07x_dma.c](#) 第 9 行定义.

4.75 hal_lks32mc07x_dma.c

[浏览该文件的文档。](#)

```

00001 #include "../include/hal_lksmcu_dma.h"
00002 #include "../hal_lks32mc07x.h"
00003 #include "lks32mc07x_user_manual.h"
00004 #include "../include/hal_lksmcu_reg.h"
00005 #include "../include/hal_lks32mc07x_reg.h"
00006
00007 // 初始化
00008 // 模块复位
00009 static uint8_t HAL_DMA_ena = 0;
00010 void HAL_DMA_Reset(HAL_DMA_x dma_x)
00011 {
00012     (void) dma_x;
00013     // 系统寄存器解锁
00014     HAL_REG_SYS_UNLOCK();
00015
00016     HAL_REG_SET(SYS_SFT_RST, SYS_SFT_RST_DMA0_SFT_RST_BIT);
00017     HAL_REG_RESET(SYS_SFT_RST, SYS_SFT_RST_DMA0_SFT_RST_BIT);
00018     // 系统寄存器上锁
00019     HAL_REG_SYS_LOCK();
00020 }
00021 // 模块打开
00022 void HAL_DMA_Enable(HAL_DMA_x dma_x)
00023 {
00024     (void) dma_x;
00025     HAL_DMA_ena |= (1 << dma_x);
00026     HAL_REG_SET(DMA0_CTRL, DMA0_CTRL_EN_BIT);
00027 }
00028 // 模块关闭
00029 void HAL_DMA_Disable(HAL_DMA_x dma_x)
00030 {
00031     HAL_DMA_ena &= ~(1 << dma_x);
00032     if (HAL_DMA_ena == 0)
00033     {
00034         HAL_REG_RESET(DMA0_CTRL, DMA0_CTRL_EN_BIT);
00035     }
00036 }
00037 // 内存复制
00038 void HAL_DMA_MemCopy(HAL_DMA_x dma_x, void *start, void *end, uint32_t len)
00039 {
00040     HAL_DMA[dma_x]->CCR = 0;
00041     HAL_DMA[dma_x]->SADR = (uint32_t)(uintptr_t)start;
00042     HAL_DMA[dma_x]->DADR = (uint32_t)(uintptr_t)end;
00043     HAL_DMA[dma_x]->CTMS = len;
00044     HAL_DMA[dma_x]->CCR = DMA0_CCRx_EN_BIT | DMA0_CCRx_DINC_BIT | DMA0_CCRx_SINC_BIT;
00045     HAL_DMA[dma_x]->REN = BIT31;
00046 }
00047 // dma使用uart发送数据,需要先初始化好对应的uart模块
00048 void HAL_DMA_UartTx(HAL_DMA_x dma_x, HAL_UART_x UART_x, uint8_t *start, uint32_t len)
00049 {
00050     HAL_DMA[dma_x]->CCR = 0;
00051     HAL_DMA[dma_x]->SADR = (uint32_t)(uintptr_t)start;
00052     HAL_DMA[dma_x]->DADR = (uint32_t)(uintptr_t)&(HAL_UART[UART_x]->BUFF);
00053     HAL_DMA[dma_x]->CTMS = len;
00054     HAL_DMA[dma_x]->CCR = DMA0_CCRx_EN_BIT | BIT1 | DMA0_CCRx_SINC_BIT;
00055     HAL_DMA[dma_x]->REN = BIT31 | BIT7;
00056 }
00057 // dma使用spi发送数据,需要先初始化好对应的spi模块
00058 void HAL_DMA_SpiTx(HAL_DMA_x dma_x, HAL_SPI_x SPI_x, uint8_t *start, uint32_t len)
00059 {
00060     HAL_DMA[dma_x]->CCR = 0;
00061     HAL_DMA[dma_x]->SADR = (uint32_t)(uintptr_t)start;
00062     HAL_DMA[dma_x]->DADR = (uint32_t)(uintptr_t)&(HAL_SPI[SPI_x]->TXDATA);
00063     HAL_DMA[dma_x]->CTMS = len;
00064     HAL_DMA[dma_x]->CCR = DMA0_CCRx_EN_BIT | BIT1 | DMA0_CCRx_SINC_BIT;
00065     HAL_DMA[dma_x]->REN = BIT31 | BIT11;
00066 }
00067 // dma使用i2c发送数据,需要先初始化好对应的i2c模块
00068 void HAL_DMA_I2cTx(HAL_DMA_x dma_x, HAL_I2C_x I2Cx, uint8_t *start, uint32_t len)
00069 {
00070     HAL_DMA[dma_x]->CCR = 0;
00071     HAL_DMA[dma_x]->SADR = (uint32_t)(uintptr_t)start;
00072     HAL_DMA[dma_x]->DADR = (uint32_t)(uintptr_t)&(HAL_I2C[I2Cx]->DATA);
00073     HAL_DMA[dma_x]->CTMS = len;
00074     HAL_DMA[dma_x]->CCR = DMA0_CCRx_EN_BIT | BIT1 | DMA0_CCRx_SINC_BIT;
00075     HAL_DMA[dma_x]->REN = BIT31 | BIT11;
00076 }
00077 // dma使用uart发送数据,需要先初始化好对应的uart模块
00078 void HAL_DMA_UartRx(HAL_DMA_x dma_x, HAL_UART_x UART_x, uint8_t *start, uint32_t len)
00079 {
00080     HAL_DMA[dma_x]->CCR = 0;
00081     HAL_DMA[dma_x]->SADR = (uint32_t)(uintptr_t)&(HAL_UART[UART_x]->BUFF);
00082     HAL_DMA[dma_x]->DADR = (uint32_t)(uintptr_t)start;

```

```

00083     HAL_DMA[dma_x]->CTMS = len;
00084     HAL_DMA[dma_x]->CCR = DMA0_CCRx_EN_BIT | DMA0_CCRx_RMODE_BIT | DMA0_CCRx_DINC_BIT;
00085     HAL_DMA[dma_x]->REN = BIT6;
00086 }
00087 // dma使用spi发送数据,需要先初始化好对应的spi模块
00088 void HAL_DMA_SpiRx(HAL_DMA_x dma_x, HAL_SPI_x SPI_x, uint8_t *start, uint32_t len)
00089 {
00090     HAL_DMA[dma_x]->CCR = 0;
00091     HAL_DMA[dma_x]->SADR = (uint32_t)(uintptr_t)(&(HAL_SPI[SPI_x]->RXDATA));
00092     HAL_DMA[dma_x]->DADR = (uint32_t)(uintptr_t)start;
00093     HAL_DMA[dma_x]->CTMS = len;
00094     HAL_DMA[dma_x]->CCR = DMA0_CCRx_EN_BIT | DMA0_CCRx_RMODE_BIT | DMA0_CCRx_DINC_BIT;
00095     HAL_DMA[dma_x]->REN = BIT6;
00096 }
00097 // dma使用i2c发送数据,需要先初始化好对应的i2c模块
00098 void HAL_DMA_I2cRx(HAL_DMA_x dma_x, HAL_I2C_x I2Cx, uint8_t *start, uint32_t len)
00099 {
00100     HAL_DMA[dma_x]->CCR = 0;
00101     HAL_DMA[dma_x]->SADR = (uint32_t)(uintptr_t)(&(HAL_I2C[I2Cx]->DATA));
00102     HAL_DMA[dma_x]->DADR = (uint32_t)(uintptr_t)start;
00103     HAL_DMA[dma_x]->CTMS = len;
00104     HAL_DMA[dma_x]->CCR = DMA0_CCRx_EN_BIT | DMA0_CCRx_RMODE_BIT | DMA0_CCRx_DINC_BIT;
00105     HAL_DMA[dma_x]->REN = BIT6;
00106 }
00107 // 使用两个dma模块和一个spi模块,同时发送&接收数据,需要先初始化好对应的spi模块
00108 void HAL_DMA_SpiRxTx(HAL_DMA_x DMA1_x, HAL_DMA_x DMA2_x, HAL_SPI_x SPI_x, uint8_t *txdata, uint8_t *rxdata,
uint32_t len)
00109 {
00110     HAL_DMA[DMA1_x]->CCR = 0;
00111     HAL_DMA[DMA1_x]->SADR = (uint32_t)(uintptr_t)(&(HAL_SPI[SPI_x]->RXDATA));
00112     HAL_DMA[DMA1_x]->DADR = (uint32_t)(uintptr_t)rxdata;
00113     HAL_DMA[DMA1_x]->CTMS = len;
00114     HAL_DMA[DMA1_x]->CCR = DMA0_CCRx_EN_BIT | DMA0_CCRx_RMODE_BIT | DMA0_CCRx_DINC_BIT;
00115     HAL_DMA[DMA1_x]->REN = BIT6;
00116
00117     HAL_DMA[DMA2_x]->CCR = 0;
00118     HAL_DMA[DMA2_x]->SADR = (uint32_t)(uintptr_t)txdata;
00119     HAL_DMA[DMA2_x]->DADR = (uint32_t)(uintptr_t)(&(HAL_SPI[SPI_x]->TXDATA));
00120     HAL_DMA[DMA2_x]->CTMS = len;
00121     HAL_DMA[DMA2_x]->CCR = DMA0_CCRx_EN_BIT | DMA0_CCRx_RMODE_BIT | DMA0_CCRx_SINC_BIT;
00122     HAL_DMA[DMA2_x]->REN = BIT6;
00123 }

```

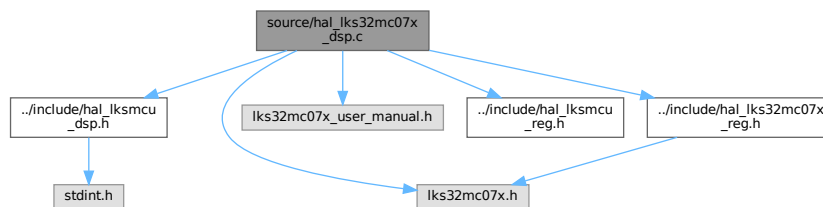
4.76 source/hal_lks32mc07x_dsp.c 文件参考

```

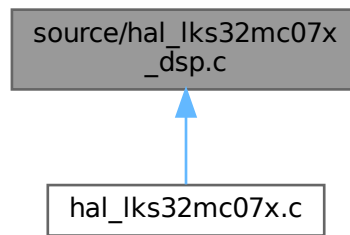
#include "../include/hal_lksmcu_dsp.h"
#include "lks32mc07x.h"
#include "lks32mc07x_user_manual.h"
#include "../include/hal_lksmcu_reg.h"
#include "../include/hal_lks32mc07x_reg.h"

```

hal_lks32mc07x_dsp.c 的引用(Include)关系图:



此图展示该文件直接或间接的被哪些文件引用了:



函数

- void [HAL_DSP_Reset](#) ([HAL_DSP_x](#) dsp_x)
- void [HAL_DSP_Enable](#) ([HAL_DSP_x](#) dsp_x)
- void [HAL_DSP_Disable](#) ([HAL_DSP_x](#) dsp_x)
- int16_t [HAL_DSP_Sin](#) ([HAL_DSP_x](#) dsp_x, int16_t angle)
- int16_t [HAL_DSP_Cos](#) ([HAL_DSP_x](#) dsp_x, int16_t angle)
- void [HAL_DSP_SinCos](#) ([HAL_DSP_x](#) dsp_x, int16_t angle, int16_t *sin, int16_t *cos)
- uint16_t [HAL_DSP_Sqrt](#) ([HAL_DSP_x](#) dsp_x, uint32_t x)
- int32_t [HAL_DSP_Div](#) ([HAL_DSP_x](#) dsp_x, int32_t a, int32_t b)

4.76.1 函数说明

4.76.1.1 HAL_DSP_Cos()

```
int16_t HAL_DSP_Cos (
    HAL\_DSP\_x dsp_x,
    int16_t angle )
```

在文件 [hal_lks32mc07x.dsp.c](#) 第 41 行定义.

4.76.1.2 HAL_DSP_Disable()

```
void HAL_DSP_Disable (
    HAL\_DSP\_x dsp_x )
```

在文件 [hal_lks32mc07x.dsp.c](#) 第 25 行定义.

4.76.1.3 HAL_DSP_Div()

```
int32_t HAL_DSP_Div (
    HAL\_DSP\_x dsp_x,
    int32_t a,
    int32_t b )
```

在文件 [hal_lks32mc07x.dsp.c](#) 第 62 行定义.

4.76.1.4 HAL_DSP_Enable()

```
void HAL_DSP_Enable (
    HAL_DSP_x dsp_x )
```

在文件 [hal_lks32mc07x.dsp.c](#) 第 16 行定义.

4.76.1.5 HAL_DSP_Reset()

```
void HAL_DSP_Reset (
    HAL_DSP_x dsp_x )
```

在文件 [hal_lks32mc07x.dsp.c](#) 第 6 行定义.

4.76.1.6 HAL_DSP_Sin()

```
int16_t HAL_DSP_Sin (
    HAL_DSP_x dsp_x,
    int16_t angle )
```

在文件 [hal_lks32mc07x.dsp.c](#) 第 34 行定义.

4.76.1.7 HAL_DSP_SinCos()

```
void HAL_DSP_SinCos (
    HAL_DSP_x dsp_x,
    int16_t angle,
    int16_t * sin,
    int16_t * cos )
```

在文件 [hal_lks32mc07x.dsp.c](#) 第 48 行定义.

4.76.1.8 HAL_DSP_Sqrt()

```
uint16_t HAL_DSP_Sqrt (
    HAL_DSP_x dsp_x,
    uint32_t x )
```

在文件 [hal_lks32mc07x.dsp.c](#) 第 56 行定义.

4.77 hal_lks32mc07x_dsp.c

浏览该文件的文档。

```

00001 #include "../include/hal_lksmcu_dsp.h"
00002 #include "lks32mc07x.h"
00003 #include "lks32mc07x_user_manual.h"
00004 #include "../include/hal_lksmcu_reg.h"
00005 #include "../include/hal_lks32mc07x_reg.h"
00006 void HAL_DSP_Reset(HAL_DSP_x dsp_x)
00007 {
00008     (void)dsp_x;
00009     // 系统寄存器解锁
00010     HAL_REG_SYS_UNLOCK();
00011     HAL_REG_SET(SYS_SFT_RST, SYS_SFT_RST_DSP0_SFT_RST_BIT);
00012     HAL_REG_RESET(SYS_SFT_RST, SYS_SFT_RST_DSP0_SFT_RST_BIT);
00013     // 系统寄存器上锁
00014     HAL_REG_SYS_LOCK();
00015 }
00016 void HAL_DSP_Enable(HAL_DSP_x dsp_x)
00017 {
00018     (void)dsp_x;
00019     // 系统寄存器解锁
00020     HAL_REG_SYS_UNLOCK();
00021     HAL_REG_SET(SYS_CLK_FEN, SYS_CLK_FEN_DSP0_CLK_EN_BIT);
00022     // 系统寄存器上锁
00023     HAL_REG_SYS_LOCK();
00024 }
00025 void HAL_DSP_Disable(HAL_DSP_x dsp_x)
00026 {
00027     (void)dsp_x;
00028     // 系统寄存器解锁
00029     HAL_REG_SYS_UNLOCK();
00030     HAL_REG_RESET(SYS_CLK_FEN, SYS_CLK_FEN_DSP0_CLK_EN_BIT);
00031     // 系统寄存器上锁
00032     HAL_REG_SYS_LOCK();
00033 }
00034 int16_t HAL_DSP_Sin(HAL_DSP_x dsp_x, int16_t angle)
00035 {
00036     (void)dsp_x;
00037     DSP0_SC = 0x6;
00038     DSP0_THETA = angle;
00039     return DSP0_SIN;
00040 }
00041 int16_t HAL_DSP_Cos(HAL_DSP_x dsp_x, int16_t angle)
00042 {
00043     (void)dsp_x;
00044     DSP0_SC = 0x6;
00045     DSP0_THETA = angle;
00046     return DSP0_COS;
00047 }
00048 void HAL_DSP_SinCos(HAL_DSP_x dsp_x, int16_t angle, int16_t *sin, int16_t *cos)
00049 {
00050     (void)dsp_x;
00051     DSP0_SC = 0x6;
00052     DSP0_THETA = angle;
00053     *sin = DSP0_SIN;
00054     *cos = DSP0_COS;
00055 }
00056 uint16_t HAL_DSP_Sqrt(HAL_DSP_x dsp_x, uint32_t x)
00057 {
00058     (void)dsp_x;
00059     DSP0_RAD = x;
00060     return DSP0_SQRT;
00061 }
00062 int32_t HAL_DSP_Div(HAL_DSP_x dsp_x, int32_t a, int32_t b)
00063 {
00064     (void)dsp_x;
00065     DSP0_DID = a;
00066     DSP0_DIS = b;
00067     return DSP0_QUO;
00068 }

```

4.78 source/hal_lks32mc07x_flash.c 文件参考

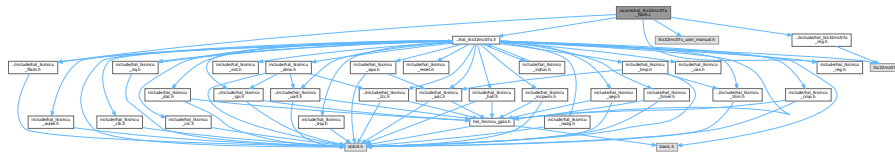
```

#include "../include/hal_lksmcu_flash.h"
#include "../hal_lks32mc07x.h"
#include "lks32mc07x_user_manual.h"
#include "../include/hal_lksmcu_reg.h"

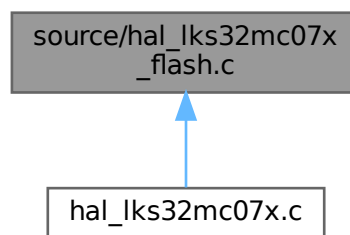
```

```
#include "../include/hal_lks32mc07x_reg.h"
```

hal_lks32mc07x_flash.c 的引用(Include)关系图:



此图展示该文件直接或间接的被哪些文件引用了:



函数

- void **HAL_FLASH_Reset** (HAL_FLASH_x flash)
模块复位,预留接口
- void **HAL_FLASH_Enable** (HAL_FLASH_x flash)
模块打开,预留接口
- void **HAL_FLASH_Disable** (HAL_FLASH_x flash)
模块关闭,预留接口
- void **HAL_FLASH_EnableCache** (HAL_FLASH_x flash)
使能flash缓存加速
- void **HAL_FLASH_DisableCache** (HAL_FLASH_x flash)
关闭flash缓存加速
- uint32_t **HAL_FLASH_ReadSectorSize** (HAL_FLASH_x flash)
获取一个扇区的大小,和HAL_FLASH_SECTOR_SIZE等效
- void **HAL_FLASH_Read** (HAL_FLASH_x flash, uint32_t flashaddr, void *data, uint32_t len)
读FLASH取数据
- uint8_t **HAL_FLASH_ReadUint8** (HAL_FLASH_x flash, uint32_t flashaddr)
读FLASH取数据
- uint16_t **HAL_FLASH_ReadUint16** (HAL_FLASH_x flash, uint32_t flashaddr)
读FLASH取数据
- uint32_t **HAL_FLASH_ReadUint32** (HAL_FLASH_x flash, uint32_t flashaddr)
读FLASH取数据
- void **HAL_FLASH_Erase** (HAL_FLASH_x flash, uint32_t flashaddr, uint32_t len)
扇区擦除,len只能是扇区大小的整数倍,不够一个扇区的部分会被忽略

- void **HAL_FLASH_Program** (**HAL_FLASH_x** flash, uint32_t flashaddr, uint8_t *data, uint32_t len)
编程
- void **HAL_FLASH_Update** (**HAL_FLASH_x** flash, uint32_t flashaddr, uint8_t *data, uint32_t len)
更新数据,等效先擦除再编程
- static uint32_t **HAL_FLASH_SecurityParameterCrc** (**HAL_FLASH_x** flash, uint32_t addr)
- void **HAL_FLASH_SecurityParameterSave** (**HAL_FLASH_x** flash, void *defaultdata, void *data, uint32_t len)
保存安全参数
- uint32_t **HAL_FLASH_SecurityParameterRead** (**HAL_FLASH_x** flash, void *defaultdata, void *data, uint32_t len)
读取安全参数

变量

- static volatile const uint8_t **HAL_FLASH_data** [3 * 512] = {0}

4.78.1 函数说明

4.78.1.1 HAL_FLASH_Disable()

```
void HAL_FLASH_Disable (
    HAL_FLASH_x flash )
```

模块关闭,预留接口

参数

<i>flash</i>	模块类型
--------------	------

在文件 [hal_lks32mc07x_flash.c](#) 第 30 行定义.

4.78.1.2 HAL_FLASH_DisableCache()

```
void HAL_FLASH_DisableCache (
    HAL_FLASH_x flash )
```

关闭flash缓存加速

参数

<i>flash</i>	模块类型
--------------	------

在文件 [hal_lks32mc07x_flash.c](#) 第 49 行定义.

4.78.1.3 HAL_FLASH_Enable()

```
void HAL_FLASH_Enable (
    HAL_FLASH_x flash )
```

模块打开,预留接口

参数

<i>flash</i>	模块类型
--------------	------

在文件 [hal_lks32mc07x.flash.c](#) 第 21 行定义.

4.78.1.4 HAL_FLASH_EnableCache()

```
void HAL_FLASH_EnableCache (
    HAL_FLASH_x flash )
```

使能flash缓存加速

参数

<i>flash</i>	模块类型
--------------	------

在文件 [hal_lks32mc07x.flash.c](#) 第 39 行定义.

4.78.1.5 HAL_FLASH_Erase()

```
void HAL_FLASH_Erase (
    HAL_FLASH_x flash,
    uint32_t flashaddr,
    uint32_t len )
```

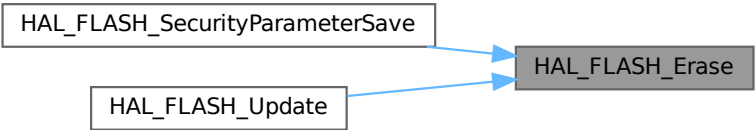
扇区擦除,len只能是扇区大小的整数倍,不够一个扇区的部分会被忽略

参数

<i>flash</i>	模块类型
<i>flashaddr</i>	flash地址
<i>len</i>	擦除长度

在文件 [hal_lks32mc07x.flash.c](#) 第 121 行定义.

这是这个函数的调用关系图:



4.78.1.6 HAL_FLASH_Program()

```
void HAL_FLASH_Program (
    HAL_FLASH_Type * flash,
    uint32_t flashaddr,
    uint8_t * data,
    uint32_t len )
```

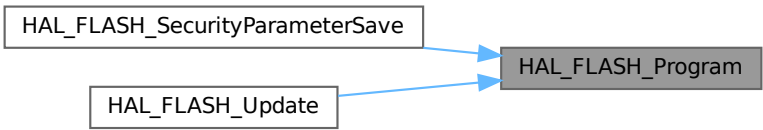
编程

参数

flash	模块类型
flashaddr	flash地址,低两位会被忽略
data	数据指针
len	数据长度

在文件 [hal_lks32mc07x_flash.c](#) 第 141 行定义.

这是这个函数的调用关系图:



4.78.1.7 HAL_FLASH_Read()

```
void HAL_FLASH_Read (
    HAL_FLASH_Type * flash,
    uint32_t flashaddr,
    void * data,
    uint32_t len )
```

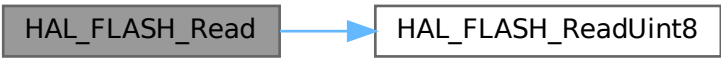
读FLASH取数据

参数

flash	模块类型
flashaddr	flash地址
data	读取的数据指针
len	读取长度

在文件 [hal_lks32mc07x_flash.c](#) 第 73 行定义.

函数调用图:



这是这个函数的调用关系图:



4.78.1.8 HAL_FLASH_ReadSectorSize()

```
uint32_t HAL_FLASH_ReadSectorSize (
    HAL_FLASH_x flash )
```

获取一个扇区的大小,和HAL_FLASH_SECTOR_SIZE等效

参数

flash	模块类型
-------	------

返回

uint32_t 扇区大小

在文件 [hal_lks32mc07x_flash.c](#) 第 60 行定义.

4.78.1.9 HAL_FLASH_ReadUint16()

```
uint16_t HAL_FLASH_ReadUint16 (
    HAL_FLASH_x flash,
    uint32_t flashaddr )
```

读FLASH取数据

参数

flash	模块类型
flashaddr	flash地址

返回

读取的数据

在文件 [hal_lks32mc07x.flash.c](#) 第 98 行定义.

4.78.1.10 HAL_FLASH_ReadUint32()

```
uint32_t HAL_FLASH_ReadUint32 (
    HAL_FLASH_TypeDef * flash,
    uint32_t flashaddr )
```

读FLASH取数据

参数

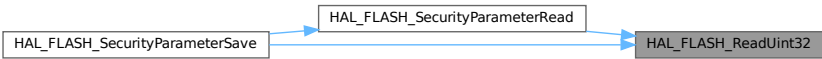
<i>flash</i>	模块类型
<i>flashaddr</i>	flash地址

返回

读取的数据

在文件 [hal_lks32mc07x.flash.c](#) 第 109 行定义.

这是这个函数的调用关系图:



4.78.1.11 HAL_FLASH_ReadUint8()

```
uint8_t HAL_FLASH_ReadUint8 (
    HAL_FLASH_TypeDef * flash,
    uint32_t flashaddr )
```

读FLASH取数据

参数

<i>flash</i>	模块类型
<i>flashaddr</i>	flash地址

返回
读取的数据

在文件 [hal_lks32mc07x.flash.c](#) 第 87 行定义.

这是这个函数的调用关系图:



4.78.1.12 HAL_FLASH.Reset()

```
void HAL_FLASH.Reset (
    HAL_FLASH_TypeDef * flash )
```

模块复位,预留接口

参数

flash	模块类型
-------	------

在文件 [hal_lks32mc07x.flash.c](#) 第 12 行定义.

4.78.1.13 HAL_FLASH.SecurityParameterCrc()

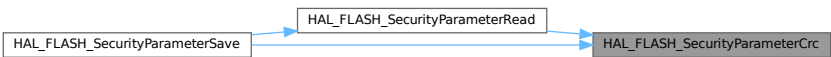
```
static uint32_t HAL_FLASH.SecurityParameterCrc (
    HAL_FLASH_TypeDef * flash,
    uint32_t addr ) [static]
```

在文件 [hal_lks32mc07x.flash.c](#) 第 171 行定义.

函数调用图:



这是这个函数的调用关系图:



4.78.1.14 HAL_FLASH_SecurityParameterRead()

```
uint32_t HAL_FLASH_SecurityParameterRead (
    HAL_FLASH_x flash,
    void * defaultdata,
    void * data,
    uint32_t len )
```

读取安全参数

参数

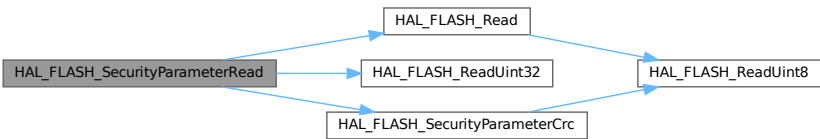
<i>flash</i>	模块类型
<i>data</i>	数据指针
<i>len</i>	数据长度

返回

使用的flash地址

在文件 [hal_lks32mc07x_flash.c](#) 第 297 行定义.

函数调用图:



这是这个函数的调用关系图:



4.78.1.15 HAL_FLASH_SecurityParameterSave()

```
void HAL_FLASH_SecurityParameterSave (
    HAL_FLASH_x flash,
    void * defaultdata,
    void * data,
    uint32_t len )
```

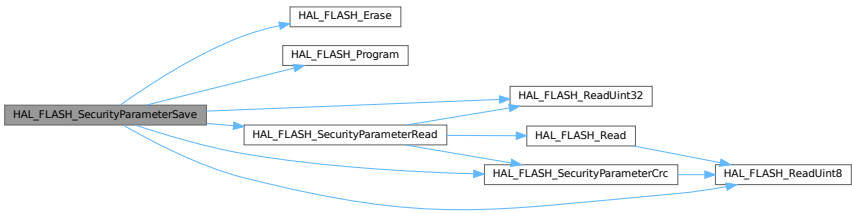
保存安全参数

参数

<i>flash</i>	模块类型
<i>defaultdata</i>	默认值
<i>data</i>	待写入的数据
<i>len</i>	数据长度

在文件 [hal_lks32mc07x.flash.c](#) 第 206 行定义.

函数调用图:



4.78.1.16 HAL.FLASH.Update()

```
void HAL_FLASH.Update (
    HAL_FLASH_x flash,
    uint32_t flashaddr,
    uint8_t * data,
    uint32_t len )
```

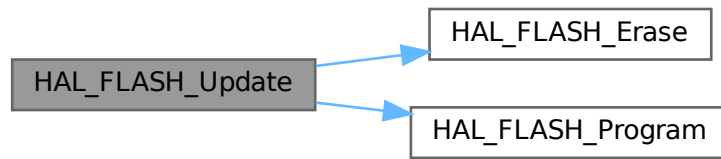
更新数据,等效先擦除再编程

参数

<i>flash</i>	模块类型
<i>flashaddr</i>	flash地址
<i>data</i>	数据指针
<i>len</i>	数据长度

在文件 [hal_lks32mc07x.flash.c](#) 第 164 行定义.

函数调用图:



4.78.2 变量说明

4.78.2.1 HAL_FLASH_data

```
volatile const uint8_t HAL_FLASH_data[3 * 512] = {0} [static]
```

在文件 [hal_lks32mc07x_flash.c](#) 第 7 行定义.

4.79 hal_lks32mc07x_flash.c

[浏览该文件的文档.](#)

```

00001 #include "../include/hal_lksmcu_flash.h"
00002 #include "../hal_lks32mc07x.h"
00003 #include "lks32mc07x.user_manual.h"
00004 #include "../include/hal_lksmcu_reg.h"
00005 #include "../include/hal_lks32mc07x_reg.h"
00006 // 使用三个扇区（确保在随机分配地址时，能分配到至少两个扇区）
00007 static volatile const uint8_t HAL_FLASH_data[3 * 512] = {0};
00012 void HAL_FLASH_Reset(HAL_FLASH_x flash)
00013 {
00014     (void)flash;
00015 }
00016
00021 void HAL_FLASH_Enable(HAL_FLASH_x flash)
00022 {
00023     (void)flash;
00024 }
00025
00030 void HAL_FLASH_Disable(HAL_FLASH_x flash)
00031 {
00032     (void)flash;
00033 }
00034
00039 void HAL_FLASH_EnableCache(HAL_FLASH_x flash)
00040 {
00041     (void)flash;
00042     HAL_REG_SET(FLASH_CFG, BIT19);
00043 }
00044
00049 void HAL_FLASH_DisableCache(HAL_FLASH_x flash)
00050 {
00051     (void)flash;
00052     HAL_REG_RESET(FLASH_CFG, BIT19);
00053 }
00054
00060 uint32_t HAL_FLASH_ReadSectorSize(HAL_FLASH_x flash)
00061 {
00062     (void)flash;
00063     return 512;
00064 }
00065
00073 void HAL_FLASH_Read(HAL_FLASH_x flash, uint32_t flashaddr, void *data, uint32_t len)
  
```

```

00074 {
00075     uint32_t i;
00076     for (i = 0; i < len; i++)
00077     {
00078         ((uint8_t *)data)[i] = HAL_FLASH_ReadUint8(flash, flashaddr + i);
00079     }
00080 }
00087 uint8_t HAL_FLASH_ReadUint8(HAL_FLASH_x flash, uint32_t flashaddr)
00088 {
00089     (void)flash;
00090     return REG8(flashaddr);
00091 }
00098 uint16_t HAL_FLASH_ReadUint16(HAL_FLASH_x flash, uint32_t flashaddr)
00099 {
00100     (void)flash;
00101     return REG16(flashaddr);
00102 }
00109 uint32_t HAL_FLASH_ReadUint32(HAL_FLASH_x flash, uint32_t flashaddr)
00110 {
00111     (void)flash;
00112     return REG32(flashaddr);
00113 }
00114 }
00121 void HAL_FLASH_Erase(HAL_FLASH_x flash, uint32_t flashaddr, uint32_t len)
00122 {
00123     (void)flash;
00124     uint32_t len1 = len;
00125     HAL_REG_WRITE_BIT(FLASH_CFG, BIT31 | BIT15 | BIT11, BIT31);
00126     for (; len1 > 0; len1 -= 512, flashaddr += 512)
00127     {
00128         HAL_REG_WRITE(FLASH_ADDR, flashaddr);
00129         HAL_REG_WRITE(FLASH_ERASE, 0x7654dcba);
00130     }
00131     HAL_REG_WRITE_BIT(FLASH_CFG, BIT31 | BIT15 | BIT11, 0);
00132 }
00133 }
00141 void HAL_FLASH_Program(HAL_FLASH_x flash, uint32_t flashaddr, uint8_t *data, uint32_t len)
00142 {
00143     (void)flash;
00144     uint32_t len1 = len;
00145     HAL_REG_WRITE_BIT(FLASH_CFG, BIT27 | BIT23 | BIT11, BIT27 | BIT23);
00146     flashaddr &= ~0x3;
00147     for (; len1 > 0; len1 -= 4, flashaddr += 4)
00148     {
00149         uint32_t val;
00150         val = data[0] | (data[1] << 8) | (data[2] << 16) | (data[3] << 24);
00151         HAL_REG_WRITE(FLASH_ADDR, flashaddr);
00152         HAL_REG_WRITE(FLASH_WDATA, val);
00153     }
00154     HAL_REG_WRITE_BIT(FLASH_CFG, BIT27 | BIT23 | BIT11, 0);
00155 }
00156 }
00164 void HAL_FLASH_Update(HAL_FLASH_x flash, uint32_t flashaddr, uint8_t *data, uint32_t len)
00165 {
00166     HAL_FLASH_Erase(flash, flashaddr, len);
00167     HAL_FLASH_Program(flash, flashaddr, data, len);
00168 }
00169 }
00170 // 仅内部使用, 计算CRC32
00171 static uint32_t HAL_FLASH_SecurityParameterCrc(HAL_FLASH_x flash, uint32_t addr)
00172 {
00173     uint32_t crc = 0xFFFFFFFF; // CRC32初始值
00174     const uint32_t polynomial = 0xEDB88320; // CRC32多项式
00175
00176     for (uint32_t i = 0; i < 504; i++)
00177     {
00178         uint8_t byte = HAL_FLASH_ReadUint8(flash, addr + i);
00179
00180         // 将字节与CRC的高8位异或
00181         crc ^= byte;
00182
00183         // 处理每个bit
00184         for (uint32_t j = 0; j < 8; j++)
00185         {
00186             if (crc & 1)
00187             {
00188                 crc = (crc >> 1) ^ polynomial;
00189             }
00190             else
00191             {
00192                 crc >>= 1;
00193             }
00194         }
00195     }
00196
00197     return ~crc; // 取反得到最终结果
00198 }

```

```

00206 void HAL_FLASH.SecurityParameterSave(HAL_FLASH_x flash, void *defaultdata, void *data, uint32_t len)
00207 {
00208     uint32_t addr0 = ((uint32_t)(uintptr_t)HAL_FLASH.data + 512) & ~0x1fff;
00209     uint32_t addr1 = addr0 + 512;
00210     uint32_t flg = 0;
00211     uint32_t addr = HAL_FLASH.SecurityParameterRead(flash, 0, 0, 0);
00212     if (addr != 0xffffffff)
00213     {
00214         for (uint32_t i = 0; i < len; i++)
00215         {
00216             // 判断待写入的数据和flash保存的数据是否一致
00217             if (((uint8_t *)data)[i] != HAL_FLASH.ReadUint8(flash, addr + i))
00218             {
00219                 flg = 1;
00220                 break;
00221             }
00222         }
00223     }
00224     else
00225     {
00226         for (uint32_t i = 0; i < len; i++)
00227         {
00228             // 判断待写入的数据和默认数据是否一致
00229             if (((uint8_t *)data)[i] != ((uint8_t *)defaultdata)[i])
00230             {
00231                 flg = 1;
00232                 break;
00233             }
00234         }
00235     }
00236     if (flg != 0) // 需要保存数据
00237     {
00238         uint32_t cnt;
00239         uint32_t cnt1;
00240         uint32_t crc;
00241         if (addr == addr0)
00242         {
00243             cnt = HAL_FLASH.ReadUint32(flash, addr0 + 500);
00244             cnt1 = cnt + 1;
00245             if (cnt1 == 0) // 溢出后从1开始计数
00246             {
00247                 cnt1 = 1;
00248             }
00249             HAL_FLASH.Erase(flash, addr1, 512); // 擦除另一个扇区
00250             HAL_FLASH.Program(flash, addr1, data, len); // 写入用户数据
00251             HAL_FLASH.Program(flash, addr1 + 500, (void *)&cnt1, 4); // 写入计数
00252             crc = HAL_FLASH.SecurityParameterCrc(flash, addr1); // 计算CRC结果
00253             HAL_FLASH.Program(flash, addr1 + 504, (void *)&crc, 4); // 写入crc
00254
00255             if (cnt == 0xffffffff) // 溢出后清空另一个扇区 (确保HAL_FLASH.SecurityParameterRead判断正确)
00256             {
00257                 HAL_FLASH.Erase(flash, addr0, 512);
00258             }
00259         }
00260         else if (addr == addr1)
00261         {
00262             cnt = HAL_FLASH.ReadUint32(flash, addr0 + 500);
00263             cnt1 = cnt + 1;
00264             if (cnt1 == 0) // 溢出后从1开始计数
00265             {
00266                 cnt1 = 1;
00267             }
00268             HAL_FLASH.Erase(flash, addr0, 512);
00269             HAL_FLASH.Program(flash, addr0, data, len);
00270             HAL_FLASH.Program(flash, addr0 + 500, (void *)&cnt1, 4);
00271             crc = HAL_FLASH.SecurityParameterCrc(flash, addr0);
00272             HAL_FLASH.Program(flash, addr0 + 504, (void *)&crc, 4);
00273
00274             if (cnt == 0xffffffff) // 溢出后清空另一个扇区 (确保HAL_FLASH.SecurityParameterRead判断正确)
00275             {
00276                 HAL_FLASH.Erase(flash, addr0, 512);
00277             }
00278         }
00279         else
00280         {
00281             cnt = 1; // 第一次写入时, 写入数据到扇区0, 并
// 从1开始计数
00282             HAL_FLASH.Erase(flash, addr1, 512); // 擦除另一个扇区
00283             HAL_FLASH.Program(flash, addr1, data, len); // 写入用户数据
00284             HAL_FLASH.Program(flash, addr1 + 500, (void *)&cnt, 4); // 写入计数
00285             crc = HAL_FLASH.SecurityParameterCrc(flash, addr1); // 计算CRC结果
00286             HAL_FLASH.Program(flash, addr1 + 504, (void *)&crc, 4); // 写入crc
00287         }
00288     }
00289 }
00297 uint32_t HAL_FLASH.SecurityParameterRead(HAL_FLASH_x flash, void *defaultdata, void *data, uint32_t len)
00298 {

```

```

00299     uint32_t addr0 = ((uint32_t)(uintptr_t)HAL_FLASH_data + 512) & ~0x1fff;
00300     uint32_t addr1 = addr0 + 512;
00301     uint32_t addr = 0;
00302     if (REG32(addr0 + 500) > REG32(addr1 + 500))
00303     {
00304         // 优先使用扇区0
00305         if (HAL_FLASH_SecurityParameterCrc(flash, addr0) == HAL_FLASH_ReadUint32(flash, addr0 + 504))
00306         { // 确保校验通过
00307             addr = addr0;
00308         }
00309         else if (HAL_FLASH_SecurityParameterCrc(flash, addr1) == HAL_FLASH_ReadUint32(flash, addr1 +
504))
00310         {
00311             addr = addr1;
00312         }
00313         else
00314         {
00315             // 两个扇区数据都丢失, 则返回默认参数
00316             // 运行时flash被误擦除
00317             // flash未初始化
00318             // flash保存的参数和默认值一致 (如果flash数据不做任何修改, 则不会触发保存的动作)
00319             addr = 0xffffffff;
00320         }
00321     }
00322 }
00323 else
00324 {
00325     // 优先使用扇区1
00326     if (HAL_FLASH_SecurityParameterCrc(flash, addr1) == HAL_FLASH_ReadUint32(flash, addr1 + 504))
00327     { // 确保校验通过
00328         addr = addr1;
00329     }
00330     else if (HAL_FLASH_SecurityParameterCrc(flash, addr0) == HAL_FLASH_ReadUint32(flash, addr0 +
504))
00331     {
00332         addr = addr0;
00333     }
00334     else
00335     {
00336         // 两个扇区数据都丢失, 则返回默认参数
00337         // 运行时flash被误擦除
00338         // flash未初始化
00339         // flash保存的参数和默认值一致 (如果flash数据不做任何修改, 则不会触发保存的动作)
00340         addr = 0xffffffff;
00341     }
00342 }
00343 if (len != 0)
00344 {
00345     if (addr != 0xffffffff)
00346     {
00347         HAL_FLASH_Read(flash, addr, data, len);
00348     }
00349     else
00350     {
00351         for (uint32_t i = 0; i < len; i++)
00352         {
00353             ((uint8_t *)data)[i] = ((uint8_t *)defaultdata)[i];
00354         }
00355     }
00356 }
00357 return addr;
00358 }

```

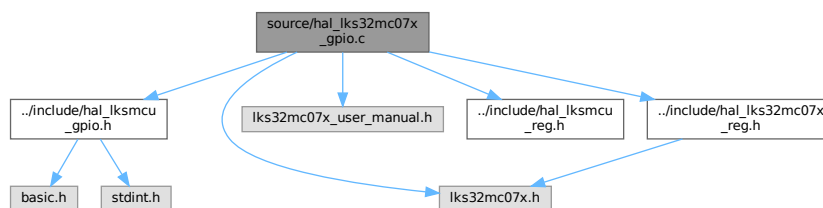
4.80 source/hal_lks32mc07x_gpio.c 文件参考

```

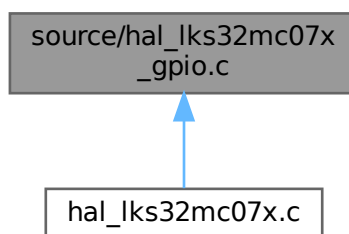
#include "../include/hal_lksmcu_gpio.h"
#include "lks32mc07x.h"
#include "lks32mc07x_user_manual.h"
#include "../include/hal_lksmcu_reg.h"
#include "../include/hal_lks32mc07x_reg.h"

```

hal_lks32mc07x_gpio.c 的引用(Include)关系图:



此图展示该文件直接或间接的被哪些文件引用了:



函数

- void **HAL_GPIO_Reset** ()
模块复位,执行后恢复所有的GPIO为悬空状态(SWD、JTAG和RST口为默认状态)
- void **HAL_GPIO_Enable** ()
启用GPIO模块,使能GPIO时钟和必要的资源
- void **HAL_GPIO_Disable** ()
禁用GPIO模块,关闭GPIO时钟和释放资源
- void **HAL_GPIO_SetMode** (HAL_GPIO_Px px, HAL_GPIO_MODE_x mode)
设置指定GPIO的工作模式
- void **HAL_GPIO_SetHigh** (HAL_GPIO_Px px)
将指定的GPIO端口输出高电平(1)
- void **HAL_GPIO_SetLow** (HAL_GPIO_Px px)
将指定的GPIO端口输出低电平(0)
- void **HAL_GPIO_Write** (HAL_GPIO_Px px, uint8_t value)
向指定的GPIO端口写入0或1
- void **HAL_GPIO_Invert** (HAL_GPIO_Px px)
翻转指定的GPIO端口的输出状态
- uint8_t **HAL_GPIO_GetOutData** (HAL_GPIO_Px px)
获取当前指定GPIO端口的输出值
- uint8_t **HAL_GPIO_GetInData** (HAL_GPIO_Px px)
获取当前指定GPIO端口的输入值

- void [HAL_GPIO_SetAf](#) ([HAL_GPIO_Px](#) px, [HAL_GPIO_Afx](#) af)
配置GPIO的特殊功能,如UART、I2C、SPI等
- void [HAL_GPIO_SwdReuseGpioEnable](#) (void)
将SWD接口配置为普通GPIO口
- void [HAL_GPIO_SwdReuseGpioDisable](#) (void)
将SWD接口恢复为SWD功能
- void [HAL_GPIO_RstReuseGpioEnable](#) (void)
将RST接口配置为普通GPIO口
- void [HAL_GPIO_RstReuseGpioDisable](#) (void)
将RST接口恢复为RST功能

4.80.1 函数说明

4.80.1.1 HAL_GPIO_Disable()

```
void HAL_GPIO_Disable (
    void )
```

禁用GPIO模块,关闭GPIO时钟和释放资源

在文件 [hal_lks32mc07x_gpio.c](#) 第 35 行定义.

4.80.1.2 HAL_GPIO_Enable()

```
void HAL_GPIO_Enable (
    void )
```

启用GPIO模块,使能GPIO时钟和必要的资源

在文件 [hal_lks32mc07x_gpio.c](#) 第 23 行定义.

4.80.1.3 HAL_GPIO_GetInData()

```
uint8_t HAL_GPIO_GetInData (
    HAL\_GPIO\_Px px )
```

获取当前指定GPIO端口的输入值

参数

<i>px</i>	指定的GPIO端口
-----------	-----------

返回

当前GPIO端口的输入值(0或1)

在文件 [hal_lks32mc07x_gpio.c](#) 第 177 行定义.

4.80.1.4 HAL_GPIO_GetOutData()

```
uint8_t HAL_GPIO_GetOutData (
    HAL_GPIO_Px px )
```

获取当前指定GPIO端口的输出值

参数

<i>px</i>	指定的GPIO端口
-----------	-----------

返回

当前GPIO端口的输出值(0或1)

在文件 [hal_lks32mc07x_gpio.c](#) 第 155 行定义.

4.80.1.5 HAL_GPIO_Invert()

```
void HAL_GPIO_Invert (
    HAL_GPIO_Px px )
```

翻转指定的GPIO端口的输出状态

参数

<i>px</i>	指定的GPIO端口
-----------	-----------

在文件 [hal_lks32mc07x_gpio.c](#) 第 143 行定义.

这是这个函数的调用关系图:



4.80.1.6 HAL_GPIO_Reset()

```
void HAL_GPIO_Reset (
    void )
```

模块复位,执行后恢复所有的GPIO为悬空状态(SWD、JTAG和RST口为默认状态)

在文件 [hal_lks32mc07x_gpio.c](#) 第 9 行定义.

4.80.1.7 HAL_GPIO_RstReuseGpioDisable()

```
void HAL_GPIO_RstReuseGpioDisable (
    void )
```

将RST接口恢复为RST功能

在文件 [hal_lks32mc07x_gpio.c](#) 第 295 行定义.

4.80.1.8 HAL_GPIO_RstReuseGpioEnable()

```
void HAL_GPIO_RstReuseGpioEnable (
    void )
```

将RST接口配置为普通GPIO口

在文件 [hal_lks32mc07x_gpio.c](#) 第 283 行定义.

4.80.1.9 HAL_GPIO_SetAf()

```
void HAL_GPIO_SetAf (
    HAL_GPIO_Px px,
    HAL_GPIO_AFx af )
```

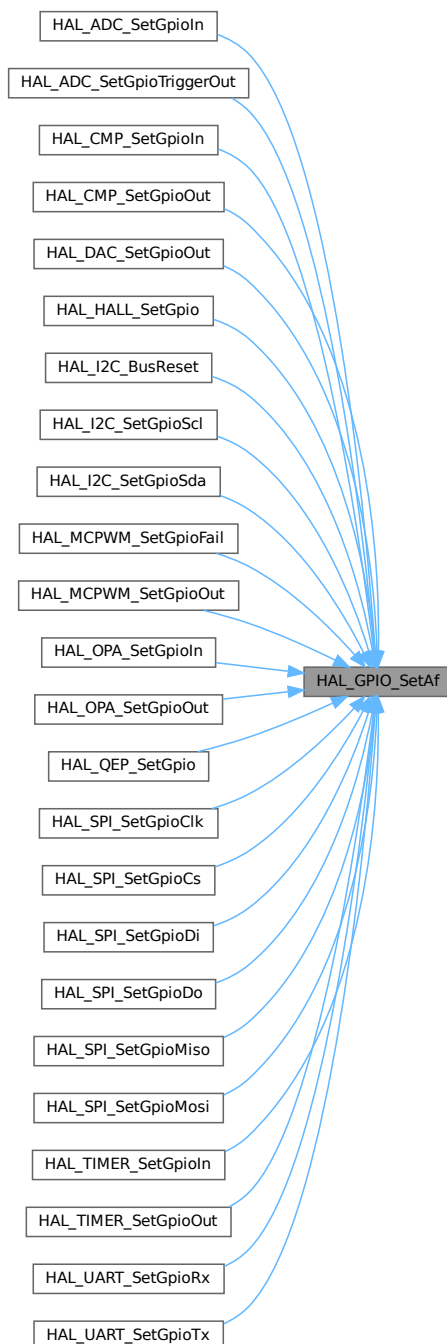
配置GPIO的特殊功能,如UART、I2C、SPI等

参数

<i>px</i>	指定的GPIO端口
<i>af</i>	要配置的特殊功能代码

在文件 [hal_lks32mc07x_gpio.c](#) 第 199 行定义.

这是这个函数的调用关系图:



4.80.1.10 HAL_GPIO_SetHigh()

```
void HAL_GPIO_SetHigh (
    HAL_GPIO_Px px )
```

将指定的GPIO端口输出高电平(1)

参数

<i>px</i>	指定的GPIO端口
-----------	-----------

注解

在INOUT模式下设为悬空,INUPOUT模式下设为上拉

在文件 [hal_lks32mc07x_gpio.c](#) 第 108 行定义.

4.80.1.11 HAL_GPIO_SetLow()

```
void HAL_GPIO_SetLow (
    HAL_GPIO_Px px )
```

将指定的GPIO端口输出低电平(0)

参数

<i>px</i>	指定的GPIO端口
-----------	-----------

在文件 [hal_lks32mc07x_gpio.c](#) 第 119 行定义.

4.80.1.12 HAL_GPIO_SetMode()

```
void HAL_GPIO_SetMode (
    HAL_GPIO_Px px,
    HAL_GPIO_MODE_x mode )
```

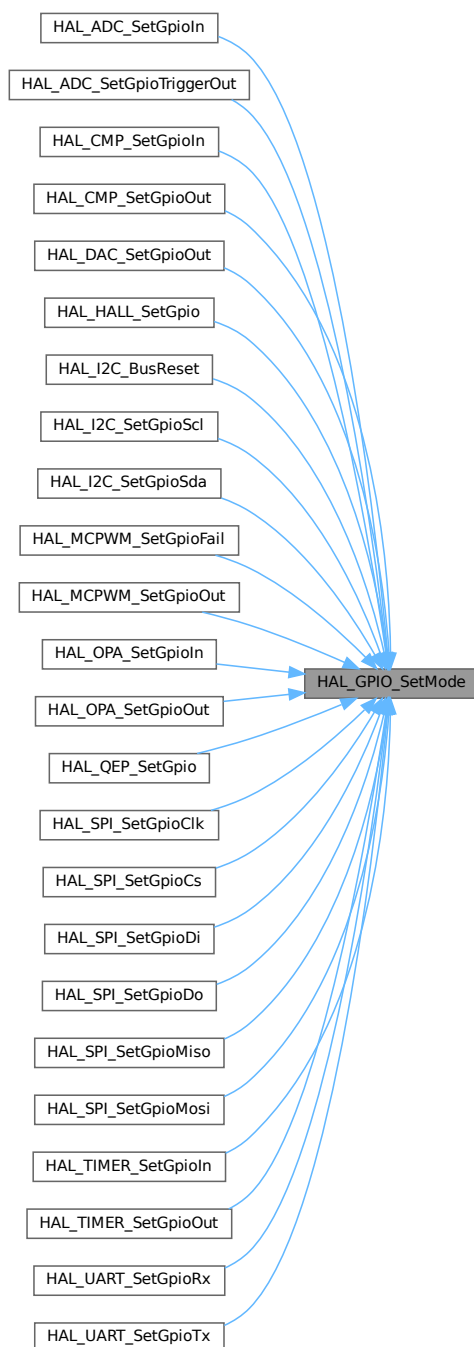
设置指定GPIO的工作模式

参数

<i>px</i>	指定的GPIO端口
<i>mode</i>	要设置的工作模式

在文件 [hal_lks32mc07x_gpio.c](#) 第 49 行定义.

这是这个函数的调用关系图:



4.80.1.13 HAL_GPIO_SwdReuseGpioDisable()

```
void HAL_GPIO_SwdReuseGpioDisable (
    void )
```

将SWD接口恢复为SWD功能

在文件 [hal_lks32mc07x_gpio.c](#) 第 271 行定义.

4.80.1.14 HAL_GPIO_SwdReuseGpioEnable()

```
void HAL_GPIO_SwdReuseGpioEnable (
    void )
```

将SWD接口配置为普通GPIO口

在文件 [hal_lks32mc07x_gpio.c](#) 第 259 行定义.

4.80.1.15 HAL_GPIO_Write()

```
void HAL_GPIO_Write (
    HAL_GPIO_Px px,
    uint8_t value )
```

向指定的GPIO端口写入0或1

参数

<i>px</i>	指定的GPIO端口
<i>value</i>	要写入的值(0或1)

在文件 [hal_lks32mc07x_gpio.c](#) 第 131 行定义.

4.81 hal_lks32mc07x_gpio.c

[浏览该文件的文档.](#)

```
00001 #include "../include/hal_lksmcu_gpio.h"
00002 #include "lks32mc07x.h"
00003 #include "lks32mc07x_user_manual.h"
00004 #include "../include/hal_lksmcu_reg.h"
00005 #include "../include/hal_lks32mc07x_reg.h"
00009 void HAL_GPIO_Reset()
00010 {
00011     // 系统寄存器解锁
00012     HAL_REG_SYS_UNLOCK();
00013     HAL_REG_SET(SYS_SFT_RST, SYS_SFT_RST_GPIO_SFT_RST_BIT);
00014     HAL_REG_RESET(SYS_SFT_RST, SYS_SFT_RST_GPIO_SFT_RST_BIT);
00015     HAL_REG_RESET(SYS_IO_CFG, SYS_IO_CFG_SWDMUX_BIT);
00016     HAL_REG_RESET(SYS_IO_CFG, SYS_IO_CFG_RST_IO_BIT);
00017     // 系统寄存器上锁
00018     HAL_REG_SYS_LOCK();
00019 }
00023 void HAL_GPIO_Enable()
00024 {
00025     // 系统寄存器解锁
00026     HAL_REG_SYS_UNLOCK();
00027     // GPIO.CLK_EN[11] GPIO 模块时钟控制信号,默认关闭 GPIO 模块时钟
00028     HAL_REG_SET(SYS_CLK_FEN, SYS_CLK_FEN_GPIO_CLK_EN_BIT);
00029     // 系统寄存器上锁
00030     HAL_REG_SYS_LOCK();
00031 }
00035 void HAL_GPIO_Disable()
00036 {
00037     // 系统寄存器解锁
00038     HAL_REG_SYS_UNLOCK();
00039     // GPIO.CLK_EN[11] GPIO 模块时钟控制信号,默认关闭 GPIO 模块时钟
00040     HAL_REG_RESET(SYS_CLK_FEN, SYS_CLK_FEN_GPIO_CLK_EN_BIT);
00041     // 系统寄存器上锁
00042     HAL_REG_SYS_LOCK();
00043 }
00049 void HAL_GPIO_SetMode(HAL_GPIO_Px px, HAL_GPIO_MODE_x mode)
00050 {
```

```

00051     uint8_t p      = (px >> 8) & 0xff;
00052     uint8_t x      = (px >> 0) & 0xff;
00053     uint16_t bitx = 1 << x;
00054     // 默认电平设置
00055     if (mode & BIT4)
00056     {
00057         HAL_REG_WRITE(HAL_GPIO[p]->BRR, bitx);
00058     }
00059     else if (mode & BIT5)
00060     {
00061         HAL_REG_WRITE(HAL_GPIO[p]->BSRR, bitx);
00062     }
00063     else
00064     {
00065     }
00066     // 开漏设置
00067     if (mode & BIT3)
00068     {
00069         HAL_REG_SET(HAL_GPIO[p]->PODE, bitx);
00070     }
00071     else
00072     {
00073         HAL_REG_RESET(HAL_GPIO[p]->PODE, bitx);
00074     }
00075     // 上拉设置
00076     if (mode & BIT2)
00077     {
00078         HAL_REG_SET(HAL_GPIO[p]->PUE, bitx);
00079     }
00080     else
00081     {
00082         HAL_REG_RESET(HAL_GPIO[p]->PUE, bitx);
00083     }
00084     // 输出设置
00085     if (mode & BIT1)
00086     {
00087         HAL_REG_SET(HAL_GPIO[p]->POE, bitx);
00088     }
00089     else
00090     {
00091         HAL_REG_RESET(HAL_GPIO[p]->POE, bitx);
00092     }
00093     // 输入使能
00094     if (mode & BIT0)
00095     {
00096         HAL_REG_SET(HAL_GPIO[p]->PIE, bitx);
00097     }
00098     else
00099     {
00100         HAL_REG_RESET(HAL_GPIO[p]->PIE, bitx);
00101     }
00102 }
00108 void HAL_GPIO_SetHigh(HAL_GPIO_Px px)
00109 {
00110     uint8_t p      = (px >> 8) & 0xff;
00111     uint8_t x      = (px >> 0) & 0xff;
00112     uint32_t bitx = 1 << x;
00113     HAL_REG_WRITE(HAL_GPIO[p]->BSRR, bitx);
00114 }
00119 void HAL_GPIO_SetLow(HAL_GPIO_Px px)
00120 {
00121     uint8_t p      = (px >> 8) & 0xff;
00122     uint8_t x      = (px >> 0) & 0xff;
00123     uint32_t bitx = 1 << x;
00124     HAL_REG_WRITE(HAL_GPIO[p]->BRR, bitx);
00125 }
00131 void HAL_GPIO_Write(HAL_GPIO_Px px, uint8_t value)
00132 {
00133     uint8_t p      = (px >> 8) & 0xff;
00134     uint8_t x      = (px >> 0) & 0xff;
00135     uint32_t bitx = 1 << x;
00136     uint32_t valx = value << x;
00137     HAL_REG_WRITE_BIT(HAL_GPIO[p]->PDO, bitx, valx);
00138 }
00143 void HAL_GPIO_Invert(HAL_GPIO_Px px)
00144 {
00145     uint8_t p      = (px >> 8) & 0xff;
00146     uint8_t x      = (px >> 0) & 0xff;
00147     uint32_t bitx = 1 << x;
00148     HAL_REG_INVERT(HAL_GPIO[p]->PDO, bitx);
00149 }
00155 uint8_t HAL_GPIO_GetOutData(HAL_GPIO_Px px)
00156 {
00157     uint8_t p      = (px >> 8) & 0xff;
00158     uint8_t x      = (px >> 0) & 0xff;
00159     uint32_t bitx = 1 << x;
00160     uint32_t valx = HAL_REG_READ_BIT(HAL_GPIO[p]->PDO, bitx);

```



```

00161     uint8_t result;
00162     if (valx)
00163     {
00164         result = 1;
00165     }
00166     else
00167     {
00168         result = 0;
00169     }
00170     return result;
00171 }
00177 uint8_t HAL_GPIO.GetInData(HAL_GPIO_Px px)
00178 {
00179     uint8_t p = (px >> 8) & 0xff;
00180     uint8_t x = (px >> 0) & 0xff;
00181     uint32_t bitx = 1 << x;
00182     uint32_t valx = HAL_REG_READ_BIT(HAL_GPIO[p]->PDI, bitx);
00183     uint8_t result;
00184     if (valx)
00185     {
00186         result = 1;
00187     }
00188     else
00189     {
00190         result = 0;
00191     }
00192     return result;
00193 }
00199 void HAL_GPIO.SetAf(HAL_GPIO_Px px, HAL_GPIO_Af af)
00200 {
00201     uint8_t p = (px >> 8) & 0xff;
00202     uint8_t x = (px >> 0) & 0xff;
00203     af &= 0xf; // 限制af的取值范围为0-15
00204     switch (x)
00205     {
00206     case 0:
00207         HAL_REG_WRITE_BIT(HAL_GPIO[p]->F3210, 0x000f, af);
00208         break;
00209     case 1:
00210         HAL_REG_WRITE_BIT(HAL_GPIO[p]->F3210, 0x00f0, af << 4);
00211         break;
00212     case 2:
00213         HAL_REG_WRITE_BIT(HAL_GPIO[p]->F3210, 0x0f00, af << 8);
00214         break;
00215     case 3:
00216         HAL_REG_WRITE_BIT(HAL_GPIO[p]->F3210, 0xf000, af << 12);
00217         break;
00218     case 4:
00219         HAL_REG_WRITE_BIT(HAL_GPIO[p]->F7654, 0x000f, af);
00220         break;
00221     case 5:
00222         HAL_REG_WRITE_BIT(HAL_GPIO[p]->F7654, 0x00f0, af << 4);
00223         break;
00224     case 6:
00225         HAL_REG_WRITE_BIT(HAL_GPIO[p]->F7654, 0x0f00, af << 8);
00226         break;
00227     case 7:
00228         HAL_REG_WRITE_BIT(HAL_GPIO[p]->F7654, 0xf000, af << 12);
00229         break;
00230     case 8:
00231         HAL_REG_WRITE_BIT(HAL_GPIO[p]->FBA98, 0x000f, af);
00232         break;
00233     case 9:
00234         HAL_REG_WRITE_BIT(HAL_GPIO[p]->FBA98, 0x00f0, af << 4);
00235         break;
00236     case 10:
00237         HAL_REG_WRITE_BIT(HAL_GPIO[p]->FBA98, 0x0f00, af << 8);
00238         break;
00239     case 11:
00240         HAL_REG_WRITE_BIT(HAL_GPIO[p]->FBA98, 0xf000, af << 12);
00241         break;
00242     case 12:
00243         HAL_REG_WRITE_BIT(HAL_GPIO[p]->FFEDC, 0x000f, af);
00244         break;
00245     case 13:
00246         HAL_REG_WRITE_BIT(HAL_GPIO[p]->FFEDC, 0x00f0, af << 4);
00247         break;
00248     case 14:
00249         HAL_REG_WRITE_BIT(HAL_GPIO[p]->FFEDC, 0x0f00, af << 8);
00250         break;
00251     case 15:
00252         HAL_REG_WRITE_BIT(HAL_GPIO[p]->FFEDC, 0xf000, af << 12);
00253         break;
00254     }
00255 }
00259 void HAL_GPIO.SwdReuseGpioEnable(void)
00260 {

```

```

00261 // 系统寄存器解锁
00262 HAL_REG_SYS_UNLOCK();
00263 // SYS.IO.CFG[6] SWD 复用控制信号,默认配置为 SWD 1:P1.8 复用为 SWCLK,P1.9 复用为 SWDIO
00264 HAL_REG_RESET(SYS.IO.CFG, BIT6);
00265 // 系统寄存器上锁
00266 HAL_REG_SYS_LOCK();
00267 }
00271 void HAL_GPIO_SwdReuseGpioDisable(void)
00272 {
00273 // 系统寄存器解锁
00274 HAL_REG_SYS_UNLOCK();
00275 // SYS.IO.CFG[6] SWD 复用控制信号,默认配置为 SWD 1:P1.8 复用为 SWCLK,P1.9 复用为 SWDIO
00276 HAL_REG_SET(SYS.IO.CFG, BIT6);
00277 // 系统寄存器上锁
00278 HAL_REG_SYS_LOCK();
00279 }
00283 void HAL_GPIO_RstReuseGpioEnable(void)
00284 {
00285 // 系统寄存器解锁
00286 HAL_REG_SYS_UNLOCK();
00287 // SYS.IO.CFG[5] RSTn/P0.2 复用控制信号,默认配置为 RSTn 0:RSTn
00288 HAL_REG_SET(SYS.IO.CFG, BIT5);
00289 // 系统寄存器上锁
00290 HAL_REG_SYS_LOCK();
00291 }
00295 void HAL_GPIO_RstReuseGpioDisable(void)
00296 {
00297 // 系统寄存器解锁
00298 HAL_REG_SYS_UNLOCK();
00299 // SYS.IO.CFG[5] RSTn/P0.2 复用控制信号,默认配置为 RSTn 0:RSTn
00300 HAL_REG_RESET(SYS.IO.CFG, BIT5);
00301 // 系统寄存器上锁
00302 HAL_REG_SYS_LOCK();
00303 }

```

4.82 source/hal_lks32mc07x_hall.c 文件参考

```

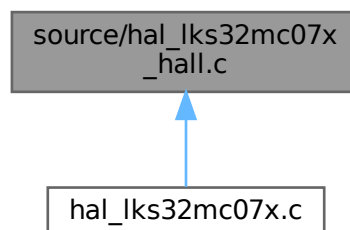
#include "../include/hal_lksmcu_hall.h"
#include "lks32mc07x_user_manual.h"
#include "../include/hal_lksmcu_reg.h"
#include "../include/hal_lks32mc07x_reg.h"
#include "../hal_lks32mc07x.h"

```

hal_lks32mc07x_hall.c 的引用(Include)关系图:



此图展示该文件直接或间接的被哪些文件引用了:



函数

- void [HAL_HALL_Reset](#) ([HAL_HALL_x](#) hall)
模块复位
- void [HAL_HALL_Enable](#) ([HAL_HALL_x](#) hall)
模块打开
- void [HAL_HALL_Disable](#) ([HAL_HALL_x](#) hall)
模块关闭
- void [HAL_HALL_SetFreq](#) ([HAL_HALL_x](#) hall, uint32_t freq)
设置hall频率
- void [HAL_HALL_SetTh](#) ([HAL_HALL_x](#) hall, uint32_t th)
设置hall阈值
- uint32_t [HAL_HALL_ReadTh](#) ([HAL_HALL_x](#) hall)
获取当前hall的周期计数值
- uint8_t [HAL_HALL_ReadHallData](#) ([HAL_HALL_x](#) hall)
获取当前hall的数据
- uint32_t [HAL_HALL_ReadHallWidth](#) ([HAL_HALL_x](#) hall)
获取当前hall的宽度
- void [HAL_HALL_SetGpio](#) ([HAL_HALL_x](#) hall, [HAL_GPIO_Px](#) px)
设置hall输入引脚
- void [HAL_HALL_StartCounting](#) ([HAL_HALL_x](#) hall)
开始计数 [HAL_HALL_Enable](#)后默认开始计数
- void [HAL_HALL_StopCounting](#) ([HAL_HALL_x](#) hall)
停止计数(同时计数器回到0)
- void [HAL_HALL_SetFilter](#) ([HAL_HALL_x](#) hall, uint32_t filter_mclk)
臆刷疆婊ゆ尝
- void [HAL_HALL_SetFilteruS](#) ([HAL_HALL_x](#) hall, uint32_t filter_uS)
设置滤波, 单位: μS

4.82.1 函数说明

4.82.1.1 HAL_HALL_Disable()

```
void HAL_HALL_Disable (
    HAL\_HALL\_x hall )
```

模块关闭

参数

hall	模块号
------	-----

在文件 [hal_lks32mc07x_hall.c](#) 第 39 行定义.

4.82.1.2 HAL_HALL_Enable()

```
void HAL_HALL_Enable (
    HAL\_HALL\_x hall )
```

模块打开

参数

<i>hall</i>	模块号
-------------	-----

在文件 [hal_lks32mc07x.hall.c](#) 第 27 行定义.

4.82.1.3 HAL_HALL_ReadHallData()

```
uint8_t HAL_HALL_ReadHallData (
    HAL_HALL_x hall )
```

获取当前hall的数据

参数

<i>hall</i>	模块号
-------------	-----

返回

数据值

在文件 [hal_lks32mc07x.hall.c](#) 第 84 行定义.

4.82.1.4 HAL_HALL_ReadHallWidth()

```
uint32_t HAL_HALL_ReadHallWidth (
    HAL_HALL_x hall )
```

获取当前hall的宽度

参数

<i>hall</i>	模块号
-------------	-----

返回

宽度值

在文件 [hal_lks32mc07x.hall.c](#) 第 94 行定义.

4.82.1.5 HAL_HALL_ReadTh()

```
uint32_t HAL_HALL_ReadTh (
    HAL_HALL_x hall )
```

获取当前hall的周期计数值



参数

<i>hall</i>	模块号
-------------	-----

返回

周期计数值

在文件 [hal_lks32mc07x.hall.c](#) 第 74 行定义.

4.82.1.6 HAL_HALL_Reset()

```
void HAL_HALL_Reset (
    HAL_HALL_x hall )
```

模块复位

参数

<i>hall</i>	模块号
-------------	-----

在文件 [hal_lks32mc07x.hall.c](#) 第 12 行定义.

4.82.1.7 HAL_HALL_SetFilter()

```
void HAL_HALL_SetFilter (
    HAL_HALL_x hall,
    uint32_t filter_mclk )
```

璁剧疆婊0尝

参数

<i>hall</i>	模块号
<i>filter_mclk</i>	滤波值

在文件 [hal_lks32mc07x.hall.c](#) 第 134 行定义.
这是这个函数的调用关系图:



4.82.1.8 HAL_HALL_SetFilteruS()

```
void HAL_HALL_SetFilteruS (
    HAL_HALL_x hall,
    uint32_t filter_uS )
```

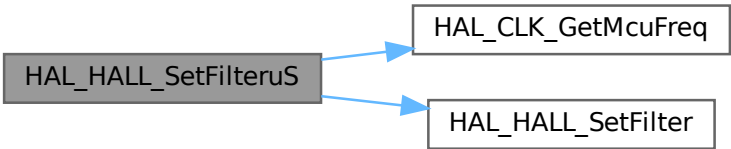
设置滤波，单位：uS

参数

hall	模块号
filter_uS	滤波值

在文件 [hal_lks32mc07x.hall.c](#) 第 166 行定义.

函数调用图:



4.82.1.9 HAL_HALL_SetFreq()

```
void HAL_HALL_SetFreq (
    HAL_HALL_x hall,
    uint32_t freq )
```

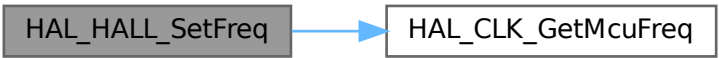
设置hall频率

参数

hall	模块号
freq	频率值

在文件 [hal_lks32mc07x.hall.c](#) 第 52 行定义.

函数调用图:



4.82.1.10 HAL_HALL_SetGpio()

```
void HAL_HALL_SetGpio (
    HAL_HALL_x hall,
    HAL_GPIO_Px px )
```

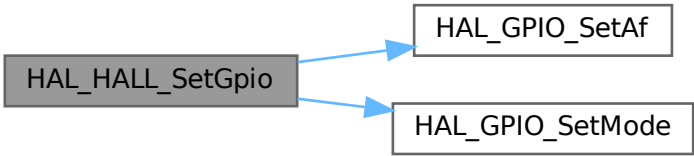
设置hall输入引脚

参数

hall	模块号
px	引脚

在文件 [hal_lks32mc07x_hall.c](#) 第 104 行定义.

函数调用图:



4.82.1.11 HAL_HALL_SetTh()

```
void HAL_HALL_SetTh (
    HAL_HALL_x hall,
    uint32_t th )
```

设置hall阈值

参数

<i>hall</i>	模块号
<i>th</i>	阈值

在文件 [hal_lks32mc07x_hall.c](#) 第 64 行定义.

4.82.1.12 HAL_HALL_StartCounting()

```
void HAL_HALL_StartCounting (
    HAL_HALL_x hall )
```

开始计数 HAL_HALL_Enable后默认开始计数

参数

<i>hall</i>	模块号
-------------	-----

在文件 [hal_lks32mc07x_hall.c](#) 第 115 行定义.

4.82.1.13 HAL_HALL_StopCounting()

```
void HAL_HALL_StopCounting (
    HAL_HALL_x hall )
```

停止计数(同时计数器回到0)

参数

<i>hall</i>	模块号
-------------	-----

在文件 [hal_lks32mc07x_hall.c](#) 第 123 行定义.

4.83 hal_lks32mc07x_hall.c

[浏览该文件的文档.](#)

```
00001
00002 #include "../include/hal_lksmcu_hall.h"
00003 #include "lks32mc07x-user-manual.h"
00004 #include "../include/hal_lksmcu_reg.h"
00005 #include "../include/hal_lks32mc07x_reg.h"
00006 #include "../hal_lks32mc07x.h"
00007
00012 void HAL_HALL_Reset(HAL_HALL_x hall)
00013 {
00014     (void)hall;
00015     // 系统寄存器解锁
00016     HAL_REG_SYS_UNLOCK();
00017     HAL_REG_SET(SYS_SFT_RST, SYS_SFT_RST_HALL0_SFT_RST_BIT);
00018     HAL_REG_RESET(SYS_SFT_RST, SYS_SFT_RST_HALL0_SFT_RST_BIT);
00019     // 系统寄存器上锁
00020     HAL_REG_SYS_LOCK();
```



```

00021 }
00022
00027 void HAL_HALL_Enable(HAL_HALLx hall)
00028 {
00029     HAL_REG_SYS_UNLOCK();
00030     HAL_REG_SET(SYS_CLK_FEN, SYS_CLK_FEN_HALL0_CLK_EN_BIT);
00031     HAL_REG_SET(HAL_HALL[hall]->CFG, HALL0_CFG_HALL_EN_BIT);
00032     HAL_REG_SYS_LOCK();
00033 }
00034
00039 void HAL_HALL_Disable(HAL_HALLx hall)
00040 {
00041     HAL_REG_SYS_UNLOCK();
00042     HAL_REG_RESET(HAL_HALL[hall]->CFG, HALL0_CFG_HALL_EN_BIT);
00043     HAL_REG_RESET(SYS_CLK_FEN, SYS_CLK_FEN_HALL0_CLK_EN_BIT);
00044     HAL_REG_SYS_LOCK();
00045 }
00046
00052 void HAL_HALL_SetFreq(HAL_HALLx hall, uint32_t freq)
00053 {
00054     uint32_t mclk = HAL_CLK_GetMcuFreq();
00055     uint32_t th = (mclk / freq);
00056     HAL_REG_WRITE(HAL_HALL[hall]->TH, th);
00057 }
00058
00064 void HAL_HALL_SetTh(HAL_HALLx hall, uint32_t th)
00065 {
00066     HAL_REG_WRITE(HAL_HALL[hall]->TH, th);
00067 }
00068
00074 uint32_t HAL_HALL_ReadTh(HAL_HALLx hall)
00075 {
00076     return HAL_REG_READ(HAL_HALL[hall]->TH);
00077 }
00078
00084 uint8_t HAL_HALL_ReadHallData(HAL_HALLx hall)
00085 {
00086     return HAL_REG_READ(HAL_HALL[hall]->INFO);
00087 }
00088
00094 uint32_t HAL_HALL_ReadHallWidth(HAL_HALLx hall)
00095 {
00096     return HAL_REG_READ(HAL_HALL[hall]->WIDTH);
00097 }
00098
00104 void HAL_HALL_SetGpio(HAL_HALLx hall, HAL_GPIO_Px px)
00105 {
00106     (void)hall;
00107     HAL_GPIO_SetMode(px, HAL_GPIO_MODE_IN);
00108     HAL_GPIO_SetAf(px, HAL_GPIO_AF_HALL);
00109 }
00110
00115 void HAL_HALL_StartCounting(HAL_HALLx hall)
00116 {
00117     HAL_REG_SET(HAL_HALL[hall]->CFG, HALL0_CFG_HALL_EN_BIT);
00118 }
00123 void HAL_HALL_StopCounting(HAL_HALLx hall)
00124 {
00125     HAL_REG_RESET(HAL_HALL[hall]->CFG, HALL0_CFG_HALL_EN_BIT);
00126     HAL_REG_WRITE(HAL_HALL[hall]->CNT, 0);
00127 }
00128
00134 void HAL_HALL_SetFilter(HAL_HALLx hall, uint32_t filter_mclk)
00135 {
00136     if (filter_mclk < 5)
00137     {
00138         HAL_REG_RESET(HAL_HALL[hall]->CFG, HALL0_CFG_FIL_75_BIT); // 小于5个cnt关闭7/5滤波
00139         HAL_REG_WRITE_BIT_MASK_POS(HAL_HALL[hall]->CFG, HALL0_CFG_CLK_DIV, 0);
00140         HAL_REG_WRITE_BIT_MASK_POS(HAL_HALL[hall]->CFG, HALL0_CFG_FIL_LEN, filter_mclk);
00141     }
00142     else
00143     {
00144         uint32_t div = 0;
00145         HAL_REG_SET(HAL_HALL[hall]->CFG, HALL0_CFG_FIL_75_BIT); // 大于5个cnt打开7/5滤波
00146         // 自动计算分配系数
00147         for (; filter_mclk > HALL0_CFG_FIL_LEN_MAX;)
00148         {
00149             filter_mclk >>= 1;
00150             div++;
00151         }
00152         if (div > HALL0_CFG_CLK_DIV_MAX)
00153         {
00154             div = HALL0_CFG_CLK_DIV_MAX;
00155             filter_mclk = HALL0_CFG_FIL_LEN_MAX;
00156         }
00157         HAL_REG_WRITE_BIT_MASK_POS(HAL_HALL[hall]->CFG, HALL0_CFG_CLK_DIV, div);
00158         HAL_REG_WRITE_BIT_MASK_POS(HAL_HALL[hall]->CFG, HALL0_CFG_FIL_LEN, filter_mclk);

```

```

00159     }
00160 }
00166 void HAL_I2C_SetFilter(u32_t filter_uS)
00167 {
00168     uint32_t clk = HAL_RCC_GetMcuFreq();
00169     HAL_I2C_SetFilter(hall, (clk / 1000000) * filter_uS);
00170 }

```

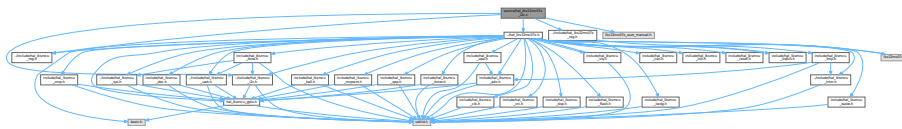
4.84 source/hal_lks32mc07x_i2c.c 文件参考

```

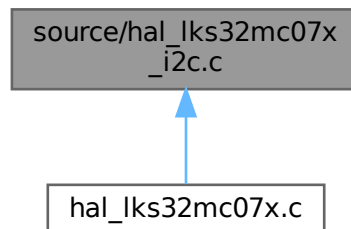
#include "../include/hal_lksmcu_reg.h"
#include "../include/hal_lksmcu_i2c.h"
#include "../hal_lks32mc07x.h"
#include "lks32mc07x_user_manual.h"
#include "../include/hal_lks32mc07x_reg.h"

```

hal_lks32mc07x_i2c.c 的引用(Include)关系图:



此图展示该文件直接或间接的被哪些文件引用了:



函数

- void **HAL_I2C_Reset** (HAL_I2C_x i2c_x)
复位I2C模块
- void **HAL_I2C_Enable** (HAL_I2C_x i2c_x)
打开I2C模块
- void **HAL_I2C_Disable** (HAL_I2C_x i2c_x)
关闭I2C模块
- void **HAL_I2C_SetBaud** (HAL_I2C_x i2c_x, uint32_t baud)
设置I2C模块波特率
- uint8_t **HAL_I2C_TxData** (HAL_I2C_x i2c_x, uint8_t addr, uint8_t *data, uint32_t len)
发送数据到I2C设备
- uint8_t **HAL_I2C_RxData** (HAL_I2C_x i2c_x, uint8_t addr, uint8_t *data, uint32_t len)

- 从I2C设备接收数据
- void HAL_I2C_SetGpioScl (HAL_I2C_x i2c_x, HAL_GPIO_Px i2cgpio)
设置I2C模块的GPIO
 - void HAL_I2C_SetGpioSda (HAL_I2C_x i2c_x, HAL_GPIO_Px i2cgpio)
设置I2C模块的GPIO
 - void HAL_I2C_BusReset (HAL_I2C_x i2c_x)
复位I2C总线

变量

- static HAL_GPIO_Px hal_i2cio [1][2]
- static uint32_t hal_baud [1]

4.84.1 函数说明

4.84.1.1 HAL_I2C_BusReset()

```
void HAL_I2C_BusReset (
    HAL_I2C_x i2c_x )
```

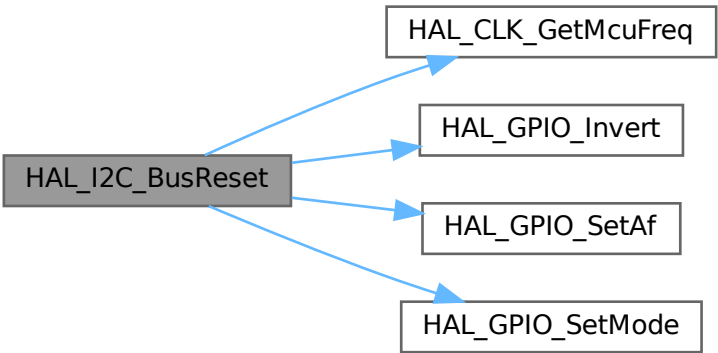
复位I2C总线

参数

i2c_x	I2C模块编号
-------	---------

在文件 [hal_lks32mc07x_i2c.c](#) 第 202 行定义.

函数调用图:



4.84.1.2 HAL_I2C_Disable()

```
void HAL_I2C_Disable (
    HAL_I2C_X i2c_x )
```

关闭I2C模块

参数

$i2c_x$	I2C模块编号
----------	---------

在文件 [hal_lks32mc07x_i2c.c](#) 第 38 行定义.

4.84.1.3 HAL_I2C_Enable()

```
void HAL_I2C_Enable (
    HAL_I2C_X i2c_x )
```

打开I2C模块

参数

$i2c_x$	I2C模块编号
----------	---------

在文件 [hal_lks32mc07x_i2c.c](#) 第 26 行定义.

4.84.1.4 HAL_I2C_Reset()

```
void HAL_I2C_Reset (
    HAL_I2C_X i2c_x )
```

复位I2C模块

参数

$i2c_x$	I2C模块编号
----------	---------

在文件 [hal_lks32mc07x_i2c.c](#) 第 13 行定义.

4.84.1.5 HAL_I2C_RxData()

```
uint8_t HAL_I2C_RxData (
    HAL_I2C_X i2c_x,
    uint8_t addr,
```

```
uint8_t * data,
uint32_t len )
```

从I2C设备接收数据

参数

<i>i2c_x</i>	I2C模块编号
<i>addr</i>	设备地址
<i>data</i>	数据缓冲区
<i>len</i>	数据长度

返回

0:成功 非0:失败

在文件 [hal_lks32mc07x_i2c.c](#) 第 130 行定义.

4.84.1.6 HAL_I2C.SetBaud()

```
void HAL_I2C_SetBaud (
    HAL_I2C_x i2c_x,
    uint32_t baud )
```

设置I2C模块波特率

参数

<i>i2c_x</i>	I2C模块编号
<i>baud</i>	波特率

在文件 [hal_lks32mc07x_i2c.c](#) 第 50 行定义.

函数调用图:



4.84.1.7 HAL_I2C.SetGpioScl()

```
void HAL_I2C_SetGpioScl (
```

```
HAL_I2C_x i2c_x,  
HAL_GPIO_Px i2cgpio )
```

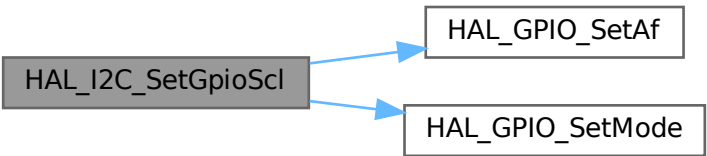
设置I2C模块的GPIO

参数

<i>i2c_x</i>	I2C模块编号
<i>i2cgpio</i>	GPIO编号

在文件 [hal_lks32mc07x_i2c.c](#) 第 179 行定义.

函数调用图:



4.84.1.8 HAL_I2C.SetGpioSda()

```
void HAL_I2C_SetGpioSda (  
    HAL_I2C_x i2c_x,  
    HAL_GPIO_Px i2cgpio )
```

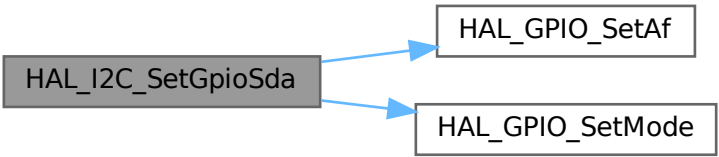
设置I2C模块的GPIO

参数

<i>i2c_x</i>	I2C模块编号
<i>i2cgpio</i>	GPIO编号

在文件 [hal_lks32mc07x_i2c.c](#) 第 191 行定义.

函数调用图:



4.84.1.9 HAL_I2C_TxData()

```
uint8_t HAL_I2C_TxData (
    HAL_I2C_x i2c_x,
    uint8_t addr,
    uint8_t * data,
    uint32_t len )
```

发送数据到I2C设备

参数

<i>i2c_x</i>	I2C模块编号
<i>addr</i>	设备地址
<i>data</i>	数据缓冲区
<i>len</i>	数据长度

返回

0:成功 非0:失败

在文件 [hal_lks32mc07x_i2c.c](#) 第 69 行定义.

4.84.2 变量说明

4.84.2.1 hal_baud

```
uint32_t hal_baud[1] [static]
```

在文件 [hal_lks32mc07x_i2c.c](#) 第 8 行定义.

4.84.2.2 hal_i2cio

HAL_GPIO_Px hal_i2cio[1][2] [static]

在文件 `hal_lks32mc07x_i2c.c` 第 7 行定义.

4.85 hal_lks32mc07x_i2c.c

浏览该文件的文档.

```
00001 #include "../include/hal_lksmcu_reg.h"
00002 #include "../include/hal_lksmcu_i2c.h"
00003 #include "../hal_lks32mc07x.h"
00004 #include "lks32mc07x_user_manual.h"
00005 #include "../include/hal_lksmcu_reg.h"
00006 #include "../include/hal_lks32mc07x_reg.h"
00007 static HAL_GPIO_Px hal_i2cio[1][2];
00008 static uint32_t hal_baud[1];
00013 void HAL_I2C_Reset(HAL_I2C_x i2c_x)
00014 {
00015     (void)i2c_x;
00016     HAL_REG_SYS_UNLOCK();
00017     HAL_REG_SET(SYS_SFT_RST, SYS_SFT_RST_I2C_0SFT_RST_BIT);
00018     HAL_REG_RESET(SYS_SFT_RST, SYS_SFT_RST_I2C_0SFT_RST_BIT);
00019     HAL_REG_SYS_LOCK();
00020 }
00021
00026 void HAL_I2C_Enable(HAL_I2C_x i2c_x)
00027 {
00028     (void)i2c_x;
00029     HAL_REG_SYS_UNLOCK();
00030     HAL_REG_SET(SYS_CLK_FEN, SYS_CLK_FEN_I2C0_CLK_EN_BIT);
00031     HAL_REG_SYS_LOCK();
00032 }
00033
00038 void HAL_I2C_Disable(HAL_I2C_x i2c_x)
00039 {
00040     (void)i2c_x;
00041     HAL_REG_SYS_UNLOCK();
00042     HAL_REG_RESET(SYS_CLK_FEN, SYS_CLK_FEN_I2C0_CLK_EN_BIT);
00043     HAL_REG_SYS_LOCK();
00044 }
00050 void HAL_I2C_SetBaud(HAL_I2C_x i2c_x, uint32_t baud)
00051 {
00052     (void)i2c_x;
00053     u32 BaudRate;
00054     hal_baud[i2c_x] = baud;
00055     BaudRate = ((u32)HAL_CLK_GetMcuFreq() + ((baud * 17) >> 1)) / (baud * 17) - 1;
00056     HAL_REG_SYS_UNLOCK();
00057     HAL_REG_WRITE(SYS_CLK_DIV0, BaudRate);
00058     HAL_REG_SYS_LOCK();
00059 }
00060
00069 uint8_t HAL_I2C_TxData(HAL_I2C_x i2c_x, uint8_t addr, uint8_t *data, uint32_t len)
00070 {
00071     uint32_t i = 0;
00072     HAL_REG_SET(HAL_I2C[i2c_x]->CFG, I2C0_CFG_MST_MODE_BIT);
00073     if (HAL_I2C[i2c_x]->MSCR & BIT3)
00074     {
00075         return 1; // 发送数据时,i2c总线忙碌
00076     }
00077     if (len == 0)
00078     {
00079         return 1; // 不支持仅发送一个地址
00080     }
00081     HAL_I2C[i2c_x]->SCR = 0;
00082     HAL_I2C[i2c_x]->MSCR = 1; // 触发I2C发送数据
00083     while ((HAL_I2C[i2c_x]->MSCR & BIT2) == 0)
00084     {
00085     }
00086     HAL_I2C[i2c_x]->DATA = addr;
00087     while (HAL_I2C[i2c_x]->SCR == 0) // 等待地址发送完成
00088     {
00089     }
00090
00091     if (HAL_I2C[i2c_x]->SCR & BIT1) // 收到nack
00092     {
00093         HAL_I2C[i2c_x]->SCR = 0;
00094         while (HAL_I2C[i2c_x]->MSCR & BIT3) // 等待stop信号发送完成
```



```

00095     {
00096     }
00097     return 1;
00098 }
00099 for (i = 0; i < len; i++)
00100 {
00101     HAL_I2C[i2c_x]->DATA = data[i];
00102     HAL_I2C[i2c_x]->SCR = BIT2; // 触发发送
00103     while ((HAL_I2C[i2c_x]->SCR & BIT0) == 0) // 等待数据发送完成
00104     {
00105     }
00106     if (HAL_I2C[i2c_x]->SCR & BIT1) // 收到nack
00107     {
00108         HAL_I2C[i2c_x]->SCR = 0;
00109         while (HAL_I2C[i2c_x]->MSCR & BIT3) // 等待stop信号发送完成
00110         {
00111         }
00112         return 1;
00113     }
00114 }
00115 HAL_I2C[i2c_x]->SCR = 0;
00116 while (HAL_I2C[i2c_x]->MSCR & BIT3) // 等待stop信号发送完成
00117 {
00118 }
00119 return 0;
00120 }
00121
00130 uint8_t HAL_I2C_RxData(HAL_I2C_TypeDef* i2c_x, uint8_t* addr, uint8_t* data, uint32_t len)
00131 {
00132     uint32_t i;
00133     HAL_REG_SET(HAL_I2C[i2c_x]->CFG, I2C0_CFG_MST_MODE_BIT);
00134     if (HAL_I2C[i2c_x]->MSCR & BIT3)
00135     {
00136         return 1; // 发送数据时,i2c总线忙碌
00137     }
00138     if (len == 0)
00139     {
00140         return 1; // 不支持仅发送一个地址
00141     }
00142     HAL_I2C[i2c_x]->SCR = 0;
00143     HAL_I2C[i2c_x]->MSCR = 1; // 触发I2C发送数据
00144     while ((HAL_I2C[i2c_x]->MSCR & BIT2) == 0) // 等待start信号发送完成
00145     {
00146     }
00147     HAL_I2C[i2c_x]->DATA = addr; // 发送地址数据
00148     while (HAL_I2C[i2c_x]->SCR == 0) // 等待地址发送完成
00149     {
00150     }
00151     if (HAL_I2C[i2c_x]->SCR & BIT1) // 收到nack
00152     {
00153         HAL_I2C[i2c_x]->SCR = 0;
00154         while (HAL_I2C[i2c_x]->MSCR & BIT3) // 等待stop信号发送完成
00155         {
00156         }
00157         return 1;
00158     }
00159     for (i = 0; i < len; i++)
00160     {
00161         HAL_I2C[i2c_x]->SCR = BIT4; // 向从机发送ack响应
00162         while ((HAL_I2C[i2c_x]->SCR & BIT0) == 0) // 等待数据发送完成
00163         {
00164         }
00165         data[i] = HAL_I2C[i2c_x]->DATA;
00166     }
00167     HAL_I2C[i2c_x]->SCR = 0;
00168     while (HAL_I2C[i2c_x]->MSCR & BIT3) // 等待stop信号发送完成
00169     {
00170     }
00171     return 0;
00172 }
00173
00179 void HAL_I2C_SetGpioScl(HAL_I2C_TypeDef* i2c_x, HAL_GPIO_TypeDef* i2cgpio)
00180 {
00181     (void)i2c_x;
00182     HAL_GPIO_SetAf(i2cgpio, HAL_GPIO_AF_I2C);
00183     HAL_GPIO_SetMode(i2cgpio, HAL_GPIO_MODE_IN);
00184     hal_i2cio[i2c_x][0] = i2cgpio; // 保存I2C使用的IO口
00185 }
00191 void HAL_I2C_SetGpioSda(HAL_I2C_TypeDef* i2c_x, HAL_GPIO_TypeDef* i2cgpio)
00192 {
00193     (void)i2c_x;
00194     HAL_GPIO_SetAf(i2cgpio, HAL_GPIO_AF_I2C);
00195     HAL_GPIO_SetMode(i2cgpio, HAL_GPIO_MODE_IN);
00196     hal_i2cio[i2c_x][1] = i2cgpio; // 保存I2C使用的IO口
00197 }
00202 void HAL_I2C_BusReset(HAL_I2C_TypeDef* i2c_x)
00203 {

```

```

00204     uint32_t div;
00205     uint32_t i;
00206     uint32_t j;
00207     // 还原成普通IO口
00208     HAL_GPIO_SetAf(hal_i2cio[i2c_x][0], HAL_GPIO_AF_GPIO);
00209     HAL_GPIO_SetAf(hal_i2cio[i2c_x][1], HAL_GPIO_AF_GPIO);
00210     HAL_GPIO_SetMode(hal_i2cio[i2c_x][0], HAL_GPIO_MODE_INUPOUT_1);
00211     HAL_GPIO_SetMode(hal_i2cio[i2c_x][1], HAL_GPIO_MODE_INUP);
00212     // 翻转10次SCL
00213     div = (HAL_CLK_GetMcuFreq() / hal_baud[i2c_x]) >> 4;
00214     for (i = 0; i < 20; i++)
00215     {
00216         HAL_GPIO_Invert(hal_i2cio[i2c_x][0]);
00217         for (j = div; j--;) // 一个大于波特率的软件延迟
00218         {
00219             __NOP();
00220         }
00221     }
00222     // 复用回I2C
00223     HAL_GPIO_SetAf(hal_i2cio[i2c_x][0], HAL_GPIO_AF_I2C);
00224     HAL_GPIO_SetAf(hal_i2cio[i2c_x][1], HAL_GPIO_AF_I2C);
00225     HAL_GPIO_SetMode(hal_i2cio[i2c_x][0], HAL_GPIO_MODE_IN);
00226     HAL_GPIO_SetMode(hal_i2cio[i2c_x][1], HAL_GPIO_MODE_IN);
00227 }

```

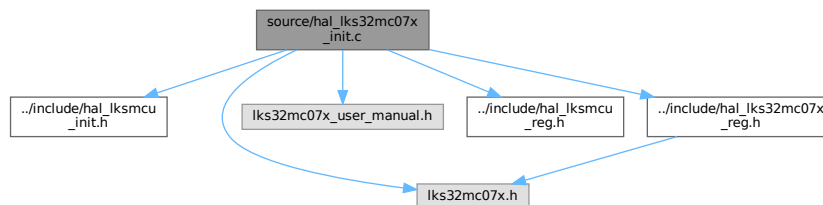
4.86 source/hal_lks32mc07x_init.c 文件参考

```

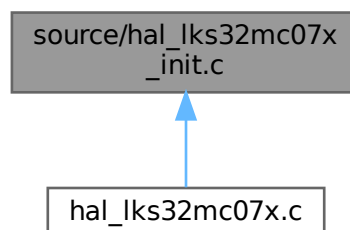
#include "../include/hal_lksmcu_init.h"
#include "lks32mc07x.h"
#include "lks32mc07x_user_manual.h"
#include "../include/hal_lksmcu_reg.h"
#include "../include/hal_lks32mc07x_reg.h"

```

hal_lks32mc07x_init.c 的引用(Include)关系图:



此图展示该文件直接或间接的被哪些文件引用了:



函数

- void [HAL_INIT_SysInit](#) (void)
- void [HAL_INIT_PowerReady](#) (HAL_INIT_POWER_x powerx)

4.86.1 函数说明

4.86.1.1 HAL_INIT_PowerReady()

```
void HAL_INIT_PowerReady (
    HAL_INIT_POWER_x powerx )
```

在文件 [hal_lks32mc07x_init.c](#) 第 14 行定义.

4.86.1.2 HAL_INIT_SysInit()

```
void HAL_INIT_SysInit (
    void )
```

在文件 [hal_lks32mc07x_init.c](#) 第 6 行定义.

4.87 hal_lks32mc07x_init.c

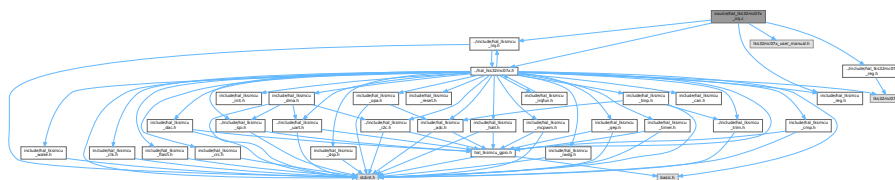
[浏览该文件的文档.](#)

```
00001 #include "../include/hal_lksmcu_init.h"
00002 #include "lks32mc07x.h"
00003 #include "lks32mc07x_user_manual.h"
00004 #include "../include/hal_lksmcu_reg.h"
00005 #include "../include/hal_lks32mc07x_reg.h"
00006 void HAL_INIT_SysInit(void)
00007 {
00008     IWDG.CFG      = 0x3c00;
00009     SYS_WR_PROTECT = 0x7a83; /* 解除系统寄存器写保护 */
00010     SYS_AFE_REG5   = BIT15; /* BIT15: PLLPDN & PPLL */
00011     SYS_CLK_CFG    = 0x000001ff; /* BIT8:0: CLK_HS, 1: PLL | BIT[7:0] CLK_DIV | 1ff对应96M时钟 */
00012 }
00013
00014 void HAL_INIT_PowerReady(HAL_INIT_POWER_x powerx)
00015 {
00016     switch (powerx)
00017     {
00018         case HAL_INIT_POWER_3_3V: // 等待3.3V电源就绪, 判断阈值为2.8V
00019             __disable_irq();
00020             // 设置掉电检测电压
00021             HAL_REG_SYS_UNLOCK();
00022             HAL_REG_WRITE_BIT_MASK_POS(HAL_SYS[0] -> REG6, SYS_AFE_REG6_PVDSEL, 3);
00023             HAL_REG_RESET(HAL_SYS[0] -> REG6, (SYS_AFE_REG6_VSR_PDT_BIT | SYS_AFE_REG6_PD_PDT_BIT));
00024             HAL_REG_SYS_LOCK();
00025             // 等待电源就绪
00026             while (HAL_REG_READ_BIT(HAL_SYS[0] -> DBG, SYS_AFE_DBG_PWR_WEAK_BIT))
00027             {
00028                 __NOP();
00029             }
00030             break;
00031         case HAL_INIT_POWER_5V: // 等待5.0V电源就绪, 判断阈值为4.0V
00032             __disable_irq();
00033             // 设置掉电检测电压
00034             HAL_REG_SYS_UNLOCK();
00035             HAL_REG_WRITE_BIT_MASK_POS(HAL_SYS[0] -> REG6, SYS_AFE_REG6_PVDSEL, 0);
00036             HAL_REG_RESET(HAL_SYS[0] -> REG6, (SYS_AFE_REG6_VSR_PDT_BIT | SYS_AFE_REG6_PD_PDT_BIT));
00037             HAL_REG_SYS_LOCK();
00038             // 等待电源就绪
00039             while (HAL_REG_READ_BIT(HAL_SYS[0] -> DBG, SYS_AFE_DBG_PWR_WEAK_BIT))
00040             {
00041                 __NOP();
00042             }
00043             break;
00044     }
00045     // 清除中断标志位
00046     NVIC_ClearPendingIRQ(PWRDN_IRQn);
00047 }
```

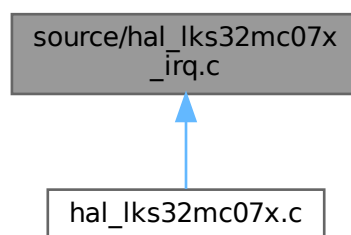
4.88 source/hal_lks32mc07x_irq.c 文件参考

```
#include "../include/hal_lksmcu_irq.h"
#include "../hal_lks32mc07x.h"
#include "lks32mc07x_user_manual.h"
#include "../include/hal_lksmcu_reg.h"
#include "../include/hal_lks32mc07x_reg.h"
```

hal_lks32mc07x_irq.c 的引用(Include)关系图:



此图展示该文件直接或间接的被哪些文件引用了:



函数

- void **HAL_IRQ_Reset** (void)
重置中断(等效于关闭中断)
- void **HAL_IRQ_Enable** (void)
使能全局中断
- void **HAL_IRQ_Disable** (void)
关闭全局中断
- void **HAL_IRQ_ModuleEnable** (HAL_IRQ_MODULE_x module, uint8_t priority)
使能某个模块的中断
- void **HAL_IRQ_ModuleDisable** (HAL_IRQ_MODULE_x module)
关闭某个模块的中断

4.88.1 函数说明

4.88.1.1 HAL_IRQ_Disable()

```
void HAL_IRQ_Disable (
    void )
```

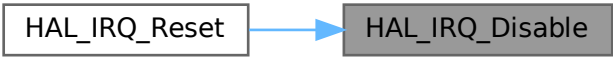
关闭全局中断

参数

<i>irq</i>	中断编号
------------	------

在文件 [hal_lks32mc07x_irq.c](#) 第 29 行定义.

这是这个函数的调用关系图:



4.88.1.2 HAL_IRQ_Enable()

```
void HAL_IRQ_Enable (
    void )
```

使能全局中断

参数

<i>irq</i>	中断编号
------------	------

在文件 [hal_lks32mc07x_irq.c](#) 第 20 行定义.

4.88.1.3 HAL_IRQ_ModuleDisable()

```
void HAL_IRQ_ModuleDisable (
    HAL_IRQ_MODULE_x module )
```

关闭某个模块的中断

参数

<i>irq</i>	中断编号
<i>module</i>	模块编号
<i>number</i>	中断号

在文件 [hal_lks32mc07x_irq.c](#) 第 52 行定义.

4.88.1.4 HAL_IRQ_ModuleEnable()

```
void HAL_IRQ_ModuleEnable (
    HAL_IRQ_MODULE_x module,
    uint8_t priority )
```

使能某个模块的中断

参数

<i>irq</i>	中断编号
<i>module</i>	模块编号
<i>number</i>	中断号
<i>priority</i>	中断优先级

在文件 [hal_lks32mc07x_irq.c](#) 第 41 行定义.

4.88.1.5 HAL_IRQ_Reset()

```
void HAL_IRQ_Reset (
    void )
```

重置中断(等效于关闭中断)

参数

<i>irq</i>	中断编号
------------	------

在文件 [hal_lks32mc07x_irq.c](#) 第 11 行定义.

函数调用图:



4.89 hal_lks32mc07x_irq.c

[浏览该文件的文档.](#)

```
00001 #include "../include/hal_lksmcu_irq.h"
00002 #include "../hal_lks32mc07x.h"
00003 #include "lks32mc07x_user_manual.h"
00004 #include "../include/hal_lksmcu_reg.h"
00005 #include "../include/hal_lks32mc07x_reg.h"
```

```

00006
00011 void HAL_IRQ_Reset(void)
00012 {
00013     HAL_IRQ_Disable();
00014 }
00015
00020 void HAL_IRQ_Enable(void)
00021 {
00022     __enable_irq();
00023 }
00024
00029 void HAL_IRQ_Disable(void)
00030 {
00031     __disable_irq();
00032 }
00033
00041 void HAL_IRQ_ModuleEnable(HAL_IRQ_MODULE_x module, uint8_t priority)
00042 {
00043     NVIC_SetPriority((IRQn_Type)module, priority);
00044     NVIC_EnableIRQ((IRQn_Type)module);
00045 }
00052 void HAL_IRQ_ModuleDisable(HAL_IRQ_MODULE_x module)
00053 {
00054     NVIC_DisableIRQ((IRQn_Type)module);
00055 }

```

4.90 source/hal_lks32mc07x_irqfun.c 文件参考

```

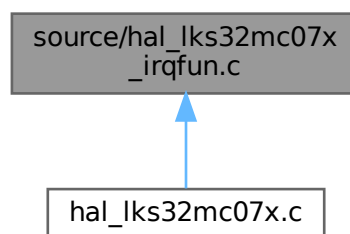
#include "../hal_lks32mc07x.h"
#include "lks32mc07x_user_manual.h"
#include "../include/hal_lksmcu_reg.h"
#include "../include/hal_lks32mc07x_reg.h"
#include "../include/hal_lksmcu_irqfun.h"

```

hal_lks32mc07x_irqfun.c 的引用(Include)关系图:



此图展示该文件直接或间接的被哪些文件引用了:



函数

- `uint8_t HAL_HALL_IrqChange (HAL_HALL_x hall)`
中断处理函数
- `uint8_t HAL_HALL_IrqChangeFlg (HAL_HALL_x hall)`
中断信号（和 `HAL_HALL_IrqChange` 的区别是不判断中断使能）
- `void HAL_HALL_IrqChangeEnable (HAL_HALL_x hall)`
中断使能
- `void HAL_HALL_IrqChangeDisable (HAL_HALL_x hall)`
中断禁止
- `void HAL_HALL_IrqChangeClear (HAL_HALL_x hall)`
中断清除
- `uint8_t HAL_HALL_IrqOver (HAL_HALL_x hall)`
中断处理函数
- `uint8_t HAL_HALL_IrqOverFlg (HAL_HALL_x hall)`
中断信号（和 `HAL_HALL_IrqOver` 的区别是不判断中断使能）
- `void HAL_HALL_IrqOverEnable (HAL_HALL_x hall)`
中断使能
- `void HAL_HALL_IrqOverDisable (HAL_HALL_x hall)`
中断禁止
- `void HAL_HALL_IrqOverClear (HAL_HALL_x hall)`
中断清除
- `uint8_t HAL_ADC_IrqSamp1 (HAL_ADC_x adc)`
中断处理函数
- `uint8_t HAL_ADC_IrqSamp1Flg (HAL_ADC_x adc)`
中断信号（和 `HAL_ADC_IrqSamp1` 的区别是不判断中断使能）
- `void HAL_ADC_IrqSamp1Enable (HAL_ADC_x adc)`
中断使能
- `void HAL_ADC_IrqSamp1Disable (HAL_ADC_x adc)`
中断禁止
- `void HAL_ADC_IrqSamp1Clear (HAL_ADC_x adc)`
中断清除
- `uint8_t HAL_ADC_IrqSamp2 (HAL_ADC_x adc)`
中断处理函数
- `uint8_t HAL_ADC_IrqSamp2Flg (HAL_ADC_x adc)`
中断信号（和 `HAL_ADC_IrqSamp2` 的区别是不判断中断使能）
- `void HAL_ADC_IrqSamp2Enable (HAL_ADC_x adc)`
中断使能
- `void HAL_ADC_IrqSamp2Disable (HAL_ADC_x adc)`
中断禁止
- `void HAL_ADC_IrqSamp2Clear (HAL_ADC_x adc)`
中断清除

4.90.1 函数说明

4.90.1.1 HAL_ADC_IrqSamp1()

```
uint8_t HAL_ADC_IrqSamp1 (
    HAL_ADC_x adc )
```

中断处理函数

参数

<i>adc</i>	模块编号
------------	------

返回

1表示中断发生，0表示没有中断发生

在文件 [hal_lks32mc07x_irqfun.c](#) 第 128 行定义.

4.90.1.2 HAL_ADC_IrqSamp1Clear()

```
void HAL_ADC_IrqSamp1Clear (  
    HAL_ADC_x adc )
```

中断清除

参数

<i>adc</i>	模块编号
------------	------

在文件 [hal_lks32mc07x_irqfun.c](#) 第 175 行定义.

4.90.1.3 HAL_ADC_IrqSamp1Disable()

```
void HAL_ADC_IrqSamp1Disable (  
    HAL_ADC_x adc )
```

中断禁止

参数

<i>adc</i>	模块编号
------------	------

在文件 [hal_lks32mc07x_irqfun.c](#) 第 167 行定义.

4.90.1.4 HAL_ADC_IrqSamp1Enable()

```
void HAL_ADC_IrqSamp1Enable (  
    HAL_ADC_x adc )
```

中断使能

参数

<i>adc</i>	模块编号
------------	------

在文件 [hal_lks32mc07x_irqfun.c](#) 第 159 行定义.

4.90.1.5 HAL_ADC_IrqSamp1Flg()

```
uint8_t HAL_ADC_IrqSamp1Flg (  
    HAL_ADC_x adc )
```

中断信号（和 HAL_ADC_IrqSamp1 的区别是不判断中断使能）

参数

<i>adc</i>	模块编号
------------	------

返回

1表示中断发生，0表示没有中断发生

在文件 [hal_lks32mc07x_irqfun.c](#) 第 144 行定义.

4.90.1.6 HAL_ADC_IrqSamp2()

```
uint8_t HAL_ADC_IrqSamp2 (  
    HAL_ADC_x adc )
```

中断处理函数

参数

<i>adc</i>	模块编号
------------	------

返回

1表示中断发生，0表示没有中断发生

在文件 [hal_lks32mc07x_irqfun.c](#) 第 185 行定义.

4.90.1.7 HAL_ADC_IrqSamp2Clear()

```
void HAL_ADC_IrqSamp2Clear (  
    HAL_ADC_x adc )
```

中断清除

参数

<i>adc</i>	模块编号
------------	------

在文件 [hal_lks32mc07x_irqfun.c](#) 第 232 行定义.

4.90.1.8 HAL_ADC_IrqSamp2Disable()

```
void HAL_ADC_IrqSamp2Disable (
    HAL_ADC_x adc )
```

中断禁止

参数

<i>adc</i>	模块编号
------------	------

在文件 [hal_lks32mc07x_irqfun.c](#) 第 224 行定义.

4.90.1.9 HAL_ADC_IrqSamp2Enable()

```
void HAL_ADC_IrqSamp2Enable (
    HAL_ADC_x adc )
```

中断使能

参数

<i>adc</i>	模块编号
------------	------

在文件 [hal_lks32mc07x_irqfun.c](#) 第 216 行定义.

4.90.1.10 HAL_ADC_IrqSamp2Flg()

```
uint8_t HAL_ADC_IrqSamp2Flg (
    HAL_ADC_x adc )
```

中断信号（和 HAL_ADC_IrqSamp2 的区别是不判断中断使能）

参数

<i>adc</i>	模块编号
------------	------

返回

1表示中断发生，0表示没有中断发生

在文件 [hal_lks32mc07x_irqfun.c](#) 第 201 行定义.

4.90.1.11 HAL_HALL_IrqChange()

```
uint8_t HAL_HALL_IrqChange (
    HAL_HALL_x hall )
```

中断处理函数

参数

<i>hall</i>	模块编号
-------------	------

返回

1表示中断发生，0表示没有中断发生

在文件 [hal_lks32mc07x_irqfun.c](#) 第 14 行定义.

4.90.1.12 HAL_HALL_IrqChangeClear()

```
void HAL_HALL_IrqChangeClear (
    HAL_HALL_x hall )
```

中断清除

参数

<i>hall</i>	模块编号
-------------	------

在文件 [hal_lks32mc07x_irqfun.c](#) 第 61 行定义.

4.90.1.13 HAL_HALL_IrqChangeDisable()

```
void HAL_HALL_IrqChangeDisable (
    HAL_HALL_x hall )
```

中断禁止

参数

<i>hall</i>	模块编号
-------------	------

在文件 [hal_lks32mc07x_irqfun.c](#) 第 53 行定义.

4.90.1.14 HAL_HALL_IrqChangeEnable()

```
void HAL_HALL_IrqChangeEnable (
    HAL_HALL_x hall )
```

中断使能

参数

<i>hall</i>	模块编号
-------------	------

在文件 [hal_lks32mc07x_irqfun.c](#) 第 45 行定义.

4.90.1.15 HAL_HALL_IrqChangeFlg()

```
uint8_t HAL_HALL_IrqChangeFlg (
    HAL_HALL_x hall )
```

中断信号（和 HAL_HALL_IrqChange 的区别是不判断中断使能）

参数

<i>hall</i>	模块编号
-------------	------

返回

1表示中断发生，0表示没有中断发生

在文件 [hal_lks32mc07x_irqfun.c](#) 第 30 行定义.

4.90.1.16 HAL_HALL_IrqOver()

```
uint8_t HAL_HALL_IrqOver (
    HAL_HALL_x hall )
```

中断处理函数

参数

<i>hall</i>	模块编号
-------------	------

返回

1表示中断发生，0表示没有中断发生

在文件 [hal_lks32mc07x_irqfun.c](#) 第 71 行定义.

4.90.1.17 HAL_HALL_IrqOverClear()

```
void HAL_HALL_IrqOverClear (
    HAL_HALL_x hall )
```

中断清除

参数

<i>hall</i>	模块编号
-------------	------

在文件 [hal_lks32mc07x_irqfun.c](#) 第 118 行定义.

4.90.1.18 HAL_HALL_IrqOverDisable()

```
void HAL_HALL_IrqOverDisable (
    HAL_HALL_x hall )
```

中断禁止

参数

<i>hall</i>	模块编号
-------------	------

在文件 [hal_lks32mc07x_irqfun.c](#) 第 110 行定义.

4.90.1.19 HAL_HALL_IrqOverEnable()

```
void HAL_HALL_IrqOverEnable (
    HAL_HALL_x hall )
```

中断使能

参数

<i>hall</i>	模块编号
-------------	------

在文件 [hal_lks32mc07x_irqfun.c](#) 第 102 行定义.

4.90.1.20 HAL_HALL_IrqOverFlg()

```
uint8_t HAL_HALL_IrqOverFlg (
    HAL_HALL_x hall )
```

中断信号（和 HAL_HALL_IrqOver 的区别是不判断中断使能）

参数

<i>hall</i>	模块编号
-------------	------

返回

1表示中断发生，0表示没有中断发生

在文件 `hal_lks32mc07x_irqfun.c` 第 87 行定义.

4.91 hal_lks32mc07x_irqfun.c

[浏览该文件的文档.](#)

```

00001
00002
00003 #include "../hal_lks32mc07x.h"
00004 #include "lks32mc07x_user_manual.h"
00005 #include "../include/hal_lksmcu_reg.h"
00006 #include "../include/hal_lks32mc07x_reg.h"
00007 #include "../include/hal_lksmcu_irqfun.h"
00008
00014 uint8_t HAL_HALL_IrqChange(HAL_HALLx hall)
00015 {
00016     if ((HAL_REG_READ_BIT(HAL_HALL[hall]->CFG, HALL0_CFG_CHG_IE_BIT) &&
00017         (HAL_REG_READ_BIT(HAL_HALL[hall]->INFO, HALL0_INFO_CHG_IF_BIT)))
00018     {
00019         return 1;
00020     }
00021     else
00022     {
00023         return 0;
00024     }
00025 }
00030 uint8_t HAL_HALL_IrqChangeFlg(HAL_HALLx hall)
00031 {
00032     if (HAL_REG_READ_BIT(HAL_HALL[hall]->INFO, HALL0_INFO_CHG_IF_BIT))
00033     {
00034         return 1;
00035     }
00036     else
00037     {
00038         return 0;
00039     }
00040 }
00045 void HAL_HALL_IrqChangeEnable(HAL_HALLx hall)
00046 {
00047     HAL_REG_SET(HAL_HALL[hall]->CFG, HALL0_CFG_CHG_IE_BIT);
00048 }
00053 void HAL_HALL_IrqChangeDisable(HAL_HALLx hall)
00054 {
00055     HAL_REG_RESET(HAL_HALL[hall]->CFG, HALL0_CFG_CHG_IE_BIT);
00056 }
00061 void HAL_HALL_IrqChangeClear(HAL_HALLx hall)
00062 {
00063     HAL_REG_WRITE(HAL_HALL[hall]->INFO, HALL0_INFO_CHG_IF_BIT);
00064 }
00065
00071 uint8_t HAL_HALL_IrqOver(HAL_HALLx hall)
00072 {
00073     if ((HAL_REG_READ_BIT(HAL_HALL[hall]->CFG, HALL0_CFG_CHG_IE_BIT) &&
00074         (HAL_REG_READ_BIT(HAL_HALL[hall]->INFO, HALL0_INFO_OV_IF_BIT)))
00075     {
00076         return 1;
00077     }
00078     else
00079     {
00080         return 0;
00081     }
00082 }
00087 uint8_t HAL_HALL_IrqOverFlg(HAL_HALLx hall)
00088 {
00089     if (HAL_REG_READ_BIT(HAL_HALL[hall]->INFO, HALL0_INFO_OV_IF_BIT))
00090     {
00091         return 1;
00092     }
00093     else
00094     {
00095         return 0;
00096     }
00097 }
00102 void HAL_HALL_IrqOverEnable(HAL_HALLx hall)
00103 {
00104     HAL_REG_SET(HAL_HALL[hall]->CFG, HALL0_CFG_CHG_IE_BIT);
00105 }
00110 void HAL_HALL_IrqOverDisable(HAL_HALLx hall)
00111 {
00112     HAL_REG_RESET(HAL_HALL[hall]->CFG, HALL0_CFG_CHG_IE_BIT);
00113 }
00118 void HAL_HALL_IrqOverClear(HAL_HALLx hall)

```



```

00119 {
00120     HAL_REG_WRITE(HAL_HALL[hal1]->INFO, HALL0_INFO_OV_IF_BIT);
00121 }
00122
00128 uint8_t HAL_ADC_IrqSamp1(HAL_ADC_x adc)
00129 {
00130     if ((HAL_REG_READ_BIT(HAL_ADC[adc]->IE, ADCx_IE_SF1_IE_BIT)) && (HAL_REG_READ_BIT(HAL_ADC[adc]->IF,
00131     ADCx_IF_SF1_IF_BIT)))
00132     {
00133         return 1;
00134     }
00135     else
00136     {
00137         return 0;
00138     }
00139 }
00144 uint8_t HAL_ADC_IrqSamp1Flg(HAL_ADC_x adc)
00145 {
00146     if (HAL_REG_READ_BIT(HAL_ADC[adc]->IF, ADCx_IF_SF1_IF_BIT))
00147     {
00148         return 1;
00149     }
00150     else
00151     {
00152         return 0;
00153     }
00154 }
00159 void HAL_ADC_IrqSamp1Enable(HAL_ADC_x adc)
00160 {
00161     HAL_REG_SET(HAL_ADC[adc]->IE, ADCx_IE_SF1_IE_BIT);
00162 }
00167 void HAL_ADC_IrqSamp1Disable(HAL_ADC_x adc)
00168 {
00169     HAL_REG_RESET(HAL_ADC[adc]->IE, ADCx_IE_SF1_IE_BIT);
00170 }
00175 void HAL_ADC_IrqSamp1Clear(HAL_ADC_x adc)
00176 {
00177     HAL_REG_WRITE(HAL_ADC[adc]->IF, ADCx_IF_SF1_IF_BIT);
00178 }
00179
00185 uint8_t HAL_ADC_IrqSamp2(HAL_ADC_x adc)
00186 {
00187     if ((HAL_REG_READ_BIT(HAL_ADC[adc]->IE, ADCx_IE_SF2_IE_BIT)) && (HAL_REG_READ_BIT(HAL_ADC[adc]->IF,
00188     ADCx_IF_SF2_IF_BIT)))
00189     {
00190         return 1;
00191     }
00192     else
00193     {
00194         return 0;
00195     }
00196 }
00201 uint8_t HAL_ADC_IrqSamp2Flg(HAL_ADC_x adc)
00202 {
00203     if (HAL_REG_READ_BIT(HAL_ADC[adc]->IF, ADCx_IF_SF2_IF_BIT))
00204     {
00205         return 1;
00206     }
00207     else
00208     {
00209         return 0;
00210     }
00211 }
00216 void HAL_ADC_IrqSamp2Enable(HAL_ADC_x adc)
00217 {
00218     HAL_REG_SET(HAL_ADC[adc]->IE, ADCx_IE_SF2_IE_BIT);
00219 }
00224 void HAL_ADC_IrqSamp2Disable(HAL_ADC_x adc)
00225 {
00226     HAL_REG_RESET(HAL_ADC[adc]->IE, ADCx_IE_SF2_IE_BIT);
00227 }
00232 void HAL_ADC_IrqSamp2Clear(HAL_ADC_x adc)
00233 {
00234     HAL_REG_WRITE(HAL_ADC[adc]->IF, ADCx_IF_SF2_IF_BIT);
00235 }

```

4.92 source/hal_lks32mc07x_iwdg.c 文件参考

```

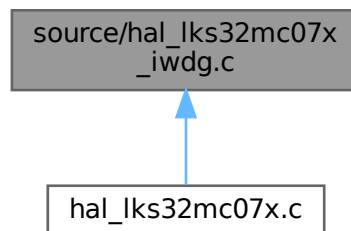
#include "../include/hal_lksmcu_iwdg.h"
#include "../hal_lks32mc07x.h"

```

```
#include "lks32mc07x_user_manual.h"
#include "../include/hal_lksmcu_reg.h"
#include "../include/hal_lks32mc07x_reg.h"
hal_lks32mc07x_iwdg.c 的引用(Include)关系图:
```



此图展示该文件直接或间接的被哪些文件引用了:



函数

- void **HAL_IWDG.Reset** (HAL_IWDG_x iwdg)
模块复位
- void **HAL_IWDG.Enable** (HAL_IWDG_x iwdg)
模块打开
- void **HAL_IWDG.Disable** (HAL_IWDG_x iwdg)
模块关闭
- void **HAL_IWDG.SetResetTimemS** (HAL_IWDG_x iwdg, int32_t ms)
设置看门狗超时复位时间
- void **HAL_IWDG.Feed** (HAL_IWDG_x iwdg)
喂狗

4.92.1 函数说明

4.92.1.1 HAL_IWDG.Disable()

```
void HAL_IWDG_Disable (
    HAL_IWDG_x iwdg )
```

模块关闭

参数

<i>iwdg</i>	看门狗模块
-------------	-------

在文件 [hal_lks32mc07x_iwdg.c](#) 第 29 行定义.

这是这个函数的调用关系图:



4.92.1.2 HAL_IWDG_Enable()

```
void HAL_IWDG_Enable (
    HAL_IWDG_x iwdg )
```

模块打开

参数

<i>iwdg</i>	看门狗模块
-------------	-------

在文件 [hal_lks32mc07x_iwdg.c](#) 第 19 行定义.

4.92.1.3 HAL_IWDG_Feed()

```
void HAL_IWDG_Feed (
    HAL_IWDG_x iwdg )
```

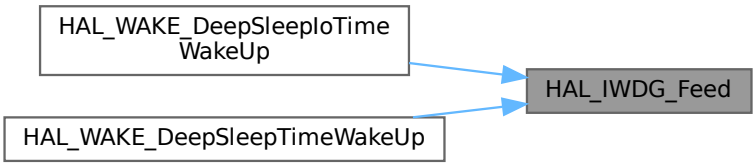
喂狗

参数

<i>iwdg</i>	看门狗模块
-------------	-------

在文件 [hal_lks32mc07x_iwdg.c](#) 第 54 行定义.

这是这个函数的调用关系图:



4.92.1.4 HAL_IWDG_Reset()

```
void HAL_IWDG_Reset (
    HAL_IWDG_x iwdg )
```

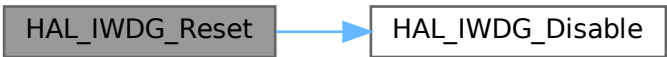
模块复位

参数

<i>iwdg</i>	看门狗模块
-------------	-------

在文件 [hal_lks32mc07x_iwdg.c](#) 第 10 行定义.

函数调用图:



4.92.1.5 HAL_IWDG_SetResetTimemS()

```
void HAL_IWDG_SetResetTimemS (
    HAL_IWDG_x iwdg,
    int32_t ms )
```

设置看门狗超时复位时间

参数

<i>iwdg</i>	看门狗模块
<i>ms</i>	超时复位时间,单位mS

在文件 `hal_lks32mc07x_iwdg.c` 第 41 行定义.

4.93 hal_lks32mc07x_iwdg.c

[浏览该文件的文档.](#)

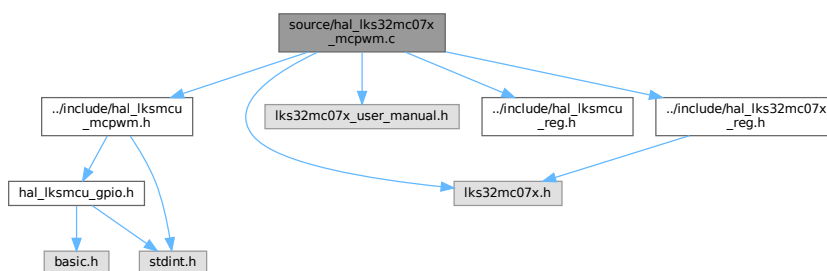
```
00001 #include "../include/hal_lksmcu_iwdg.h"
00002 #include "../hal_lks32mc07x.h"
00003 #include "lks32mc07x_user_manual.h"
00004 #include "../include/hal_lksmcu_reg.h"
00005 #include "../include/hal_lks32mc07x_reg.h"
00010 void HAL_IWDG_Reset(HAL_IWDG_x iwdg)
00011 {
00012     HAL_IWDG_Disable(iwdg);
00013 }
00014
00019 void HAL_IWDG_Enable(HAL_IWDG_x iwdg)
00020 {
00021     (void)iwdg;
00022     HAL_REG_WRITE(HAL_IWDG[0]->CFG, IWDG_CFG_WDG_EN_BIT);
00023 }
00024
00029 void HAL_IWDG_Disable(HAL_IWDG_x iwdg)
00030 {
00031     (void)iwdg;
00032     // 写0同时向[15:8]写入0x3c可清零
00033     HAL_REG_WRITE_BIT(IWDG_CFG, 0x3c00 | IWDG_CFG_WDG_EN_BIT, 0x3c00);
00034 }
00035
00041 void HAL_IWDG_SetResetTimemS(HAL_IWDG_x iwdg, int32_t ms)
00042 {
00043     (void)iwdg;
00044     uint32_t th;
00045     th = ((uint32_t)(32 * ms + 0x800) & 0x1fff000);
00046     HAL_REG_WRITE(HAL_IWDG[0]->PSW, 0xA6B4);
00047     HAL_REG_WRITE(HAL_IWDG[0]->RTH, th);
00048 }
00049
00054 void HAL_IWDG_Feed(HAL_IWDG_x iwdg)
00055 {
00056     (void)iwdg;
00057     if (HAL_REG_READ_BIT(HAL_SYS[0]->CLK_CFG, SYS_CLK_CFG_CLK_SEL_MASK | SYS_CLK_CFG_CLK_DIV_MASK) == 0x1fff)
00058     {
00059         // 全速运行时直接喂狗
00060         HAL_REG_WRITE(HAL_IWDG[iwdg]->PSW, 0xA6B4);
00061         HAL_REG_WRITE(HAL_IWDG[iwdg]->CLR, 0x798D);
00062     }
00063     else
00064     {
00065         int32_t flg = __get_PRIMASK();
00066         uint32_t clkcfg;
00067         __disable_irq();
00068         HAL_REG_SYS_UNLOCK();
00069
00070         clkcfg = HAL_REG_READ(HAL_SYS[0]->CLK_CFG);
00071
00072         HAL_REG_WRITE_BIT_MASK_POS(HAL_SYS[0]->CLK_CFG, SYS_CLK_CFG_CLK_DIV, 0xff);
00073         HAL_REG_WRITE(HAL_IWDG[iwdg]->PSW, 0xA6B4);
00074         HAL_REG_WRITE(HAL_IWDG[iwdg]->CLR, 0x798D);
00075         HAL_REG_WRITE(HAL_SYS[0]->CLK_CFG, clkcfg);
00076
00077         HAL_REG_SYS_LOCK();
00078
00079         __set_PRIMASK(flg);
00080     }
00081 }
```

4.94 source/hal_lks32mc07x_mcpwm.c 文件参考

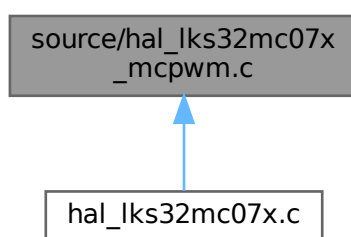
```
#include "../include/hal_lksmcu_mcpwm.h"
#include "lks32mc07x.h"
#include "lks32mc07x_user_manual.h"
#include "../include/hal_lksmcu_reg.h"
```

```
#include "../include/hal_lks32mc07x_reg.h"
```

hal_lks32mc07x_mcpwm.c 的引用(Include)关系图:



此图展示该文件直接或间接的被哪些文件引用了:



函数

- void **hal_MCPWM.UpdatePolarityState** (HAL_MCPWM_x mcpwm_x, HAL_MCPWM_PORT_x mcpwm_port_x)
内部使用,输出端口的状态
- void **HAL_MCPWM.Reset** (HAL_MCPWM_x mcpwm_x)
复位mcpwm模块
- void **HAL_MCPWM.Enable** (HAL_MCPWM_x mcpwm_x)
mcpwm模块使能
- void **HAL_MCPWM.Disable** (HAL_MCPWM_x mcpwm_x)
mcpwm模块关闭
- void **HAL_MCPWM.SetFreq** (HAL_MCPWM_x mcpwm_x, uint32_t freq)
设置pwm频率
- uint32_t **HAL_MCPWM.GetFreq** (HAL_MCPWM_x mcpwm_x)
获取pwm频率
- uint32_t **HAL_MCPWM.GetThCnt** (HAL_MCPWM_x mcpwm_x)
获取计数器最大值
- void **HAL_MCPWM.SetDead** (HAL_MCPWM_x mcpwm_x, uint32_t pdead_nS, uint32_t ndead_nS)
设置死区
- void **HAL_MCPWM.SetOutputDisable** (HAL_MCPWM_x mcpwm_x)

- 关闭输出
- void `HAL_MCPWM_SetOutputEnable` (`HAL_MCPWM_x` `mcpwm_x`)
- 打开输出
- void `HAL_MCPWM_StartCounting` (`HAL_MCPWM_x` `mcpwm_x`)
- 开始计数
- void `HAL_MCPWM_StopCounting` (`HAL_MCPWM_x` `mcpwm_x`)
- 停止计数(同时计数器回到-*th*)
- void `HAL_MCPWM_SetPwm` (`HAL_MCPWM_x` `mcpwm_x`, `HAL_MCPWM_PORT_x` `mcpwm_port_x`, `int32_t` `t0`, `int32_t` `t1`)
- 设置占空比
- void `HAL_MCPWM_SetPwmQ15` (`HAL_MCPWM_x` `mcpwm_x`, `HAL_MCPWM_PORT_x` `mcpwm_port_x`, `int16_t` `t0`, `int16_t` `t1`)
- void `HAL_MCPWM_SetOutState` (`HAL_MCPWM_x` `mcpwm_x`, `HAL_MCPWM_PORT_x` `mcpwm_port_x`, `HAL_MCPWM_OUTPUT_STATE_x` `mcpwm_output_state_x`)
- 设置输出状态
- void `HAL_MCPWM_SetDrivePolarity` (`HAL_MCPWM_x` `mcpwm_x`, `HAL_MCPWM_POLARITY_x` `mcpwm_polarity_x`)
- 设置
- void `HAL_MCPWM_SetDefaultState` (`HAL_MCPWM_x` `mcpwm_x`, `HAL_MCPWM_DEFAULT_OUTPUT_STATE_x` `mcpwm_default_output_state_x`)
- 设置默认状态
- void `HAL_MCPWM_SetGpioOut` (`HAL_MCPWM_x` `mcpwm_x`, `HAL_GPIO_Px` `px`)
- 设置用于输出的gpio
- void `HAL_MCPWM_SetGpioFail` (`HAL_MCPWM_x` `mcpwm_x`, `HAL_GPIO_Px` `px`)
- void `HAL_MCPWM_SetFailSignal` (`HAL_MCPWM_x` `mcpwm_x`, `HAL_MCPWM_FAIL_x` `fail_x`)
- 设置FAIL信号来源
- void `HAL_MCPWM_SetAdcTrig` (`HAL_MCPWM_x` `mcpwm_x`, `int32_t` `t0`, `int32_t` `t1`)
- void `HAL_MCPWM_SetAdcTrigQ15` (`HAL_MCPWM_x` `mcpwm_x`, `int32_t` `t0`, `int32_t` `t1`)
- uint8_t `HAL_MCPWM_IrqOverflow` (`HAL_MCPWM_x` `mcpwm_x`)
- void `HAL_MCPWM_IrqOverflowEnable` (`HAL_MCPWM_x` `mcpwm_x`)
- void `HAL_MCPWM_IrqOverflowDisable` (`HAL_MCPWM_x` `mcpwm_x`)
- void `HAL_MCPWM_IrqOverflowClear` (`HAL_MCPWM_x` `mcpwm_x`)
- uint8_t `HAL_MCPWM_IrqZeroCross` (`HAL_MCPWM_x` `mcpwm_x`)
- void `HAL_MCPWM_IrqZeroCrossEnable` (`HAL_MCPWM_x` `mcpwm_x`)
- void `HAL_MCPWM_IrqZeroCrossDisable` (`HAL_MCPWM_x` `mcpwm_x`)
- void `HAL_MCPWM_IrqZeroCrossClear` (`HAL_MCPWM_x` `mcpwm_x`)
- uint8_t `HAL_MCPWM_IrqFail` (`HAL_MCPWM_x` `mcpwm_x`)
- void `HAL_MCPWM_IrqFailEnable` (`HAL_MCPWM_x` `mcpwm_x`)
- void `HAL_MCPWM_IrqFailDisable` (`HAL_MCPWM_x` `mcpwm_x`)
- void `HAL_MCPWM_IrqFailClear` (`HAL_MCPWM_x` `mcpwm_x`)

变量

- static uint16_t `mcpwm_polarity_state` [1][6] = {{0, 0, 0, 0, 0, 0}}
- bit0* 通道交换使能 0 交换 1 交换 *bit1* 上管有效电平 0 上管低有效 1 上管高有效 *bit2* 下管有效电平 0 下管低有效 1 下管高有效 *bit3* 默认上管电平 0 关机后上管关闭 1 关机后上管打开 *bit4* 默认下管电平 0 关机后下管关闭 1 关机后下管打开 *bit5-bit7* 输出状态 `HAL_MCPWM_OUTPUT_STATE_x`

4.94.1 函数说明

4.94.1.1 HAL_MCPWM_Disable()

```
void HAL_MCPWM_Disable (
    HAL_MCPWM_x mcpwm_x )
```

mcpwm模块关闭

参数

<i>mcpwm</i> ↔ _x	操作的模块编号
----------------------	---------

在文件 [hal_lks32mc07x.mcpwm.c](#) 第 191 行定义.

4.94.1.2 HAL_MCPWM_Enable()

```
void HAL_MCPWM_Enable (
    HAL_MCPWM_x mcpwm_x )
```

mcpwm模块使能

参数

<i>mcpwm</i> ↔ _x	操作的模块编号
----------------------	---------

在文件 [hal_lks32mc07x.mcpwm.c](#) 第 172 行定义.

4.94.1.3 HAL_MCPWM_GetFreq()

```
uint32_t HAL_MCPWM_GetFreq (
    HAL_MCPWM_x mcpwm_x )
```

获取pwm频率

参数

<i>mcpwm</i> ↔ _x	操作的模块编号
----------------------	---------

返回

当前pwm的频率

在文件 [hal_lks32mc07x.mcpwm.c](#) 第 221 行定义.

函数调用图:



4.94.1.4 HAL_MCPWM_GetThCnt()

```
uint32_t HAL_MCPWM_GetThCnt (
    HAL_MCPWM_x mcpwm_x )
```

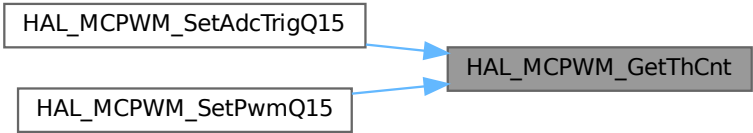
获取计数器最大值

参数

mcpwm_x↔	操作的模块编号
----------	---------

在文件 [hal_lks32mc07x_mcpwm.c](#) 第 232 行定义.

这是这个函数的调用关系图:



4.94.1.5 HAL_MCPWM_IrqFail()

```
uint8_t HALMCPWM_IrqFail (
    HAL_MCPWM_x mcpwm_x )
```

在文件 [hal_lks32mc07x_mcpwm.c](#) 第 494 行定义.

4.94.1.6 HAL_MCPWM_IrqFailClear()

```
void HALMCPWM_IrqFailClear (
    HAL_MCPWM_x mcpwm_x )
```

在文件 [hal_lks32mc07x_mcpwm.c](#) 第 517 行定义.

4.94.1.7 HAL_MCPWM_IrqFailDisable()

```
void HALMCPWM_IrqFailDisable (
    HAL_MCPWM_x mcpwm_x )
```

在文件 [hal_lks32mc07x_mcpwm.c](#) 第 511 行定义.

4.94.1.8 HAL_MCPWM_IrqFailEnable()

```
void HAL_MCPWM_IrqFailEnable (
    HAL_MCPWM_X mcpwm_x )
```

在文件 [hal_lks32mc07x_mcpwm.c](#) 第 505 行定义.

4.94.1.9 HAL_MCPWM_IrqOverflow()

```
uint8_t HAL_MCPWM_IrqOverflow (
    HAL_MCPWM_X mcpwm_x )
```

在文件 [hal_lks32mc07x_mcpwm.c](#) 第 438 行定义.

4.94.1.10 HAL_MCPWM_IrqOverflowClear()

```
void HAL_MCPWM_IrqOverflowClear (
    HAL_MCPWM_X mcpwm_x )
```

在文件 [hal_lks32mc07x_mcpwm.c](#) 第 461 行定义.

4.94.1.11 HAL_MCPWM_IrqOverflowDisable()

```
void HAL_MCPWM_IrqOverflowDisable (
    HAL_MCPWM_X mcpwm_x )
```

在文件 [hal_lks32mc07x_mcpwm.c](#) 第 455 行定义.

4.94.1.12 HAL_MCPWM_IrqOverflowEnable()

```
void HAL_MCPWM_IrqOverflowEnable (
    HAL_MCPWM_X mcpwm_x )
```

在文件 [hal_lks32mc07x_mcpwm.c](#) 第 449 行定义.

4.94.1.13 HAL_MCPWM_IrqZeroCross()

```
uint8_t HAL_MCPWM_IrqZeroCross (
    HAL_MCPWM_X mcpwm_x )
```

在文件 [hal_lks32mc07x_mcpwm.c](#) 第 466 行定义.

4.94.1.14 HAL_MCPWM_IrqZeroCrossClear()

```
void HAL_MCPWM_IrqZeroCrossClear (
    HAL_MCPWM_X mcpwm_x )
```

在文件 [hal_lks32mc07x_mcpwm.c](#) 第 489 行定义.

4.94.1.15 HAL_MCPWM_IrqZeroCrossDisable()

```
void HAL_MCPWM_IrqZeroCrossDisable (
    HAL_MCPWM_X mcpwm_x )
```

在文件 [hal_lks32mc07x_mcpwm.c](#) 第 483 行定义.

4.94.1.16 HAL_MCPWM_IrqZeroCrossEnable()

```
void HAL_MCPWM_IrqZeroCrossEnable (
    HAL_MCPWM_X mcpwm_x )
```

在文件 [hal_lks32mc07x_mcpwm.c](#) 第 477 行定义.

4.94.1.17 HAL_MCPWM_Reset()

```
void HAL_MCPWM_Reset (
    HAL_MCPWM_X mcpwm_x )
```

复位mcpwm模块

参数

<i>mcpwm_x</i>	操作的模块编号
----------------	---------

在文件 [hal_lks32mc07x_mcpwm.c](#) 第 158 行定义.

4.94.1.18 HAL_MCPWM_SetAdcTrig()

```
void HAL_MCPWM_SetAdcTrig (
    HAL_MCPWM_X mcpwm_x,
    int32_t t0,
    int32_t t1 )
```

在文件 [hal_lks32mc07x_mcpwm.c](#) 第 422 行定义.

4.94.1.19 HAL_MCPWM_SetAdcTrigQ15()

```
void HAL_MCPWM_SetAdcTrigQ15 (
    HAL_MCPWM_X mcpwm_x,
    int32_t t0,
    int32_t t1 )
```

在文件 [hal_lks32mc07x_mcpwm.c](#) 第 428 行定义.

函数调用图:



4.94.1.20 HAL_MCPWM_SetDead()

```
void HALMCPWM_SetDead (
    HAL_MCPWM_x mcpwm_x,
    uint32_t pdead_nS,
    uint32_t ndead_nS )
```

设置死区

参数

<i>mcpwm_x</i>	操作的模块编号
<i>pdead_nS</i>	下管关闭上管打开的死区宽度,单位nS
<i>ndead_nS</i>	上管关闭下管打开的死区宽度,单位nS

在文件 [hal_lks32mc07x.mcpwm.c](#) 第 242 行定义.

函数调用图:



4.94.1.21 HAL_MCPWM_SetDefaultState()

```
void HALMCPWM_SetDefaultState (
    HAL_MCPWM_x mcpwm_x,
    HAL_MCPWM_DEFAULT_OUTPUT_STATE_x mcpwm_default_output_state_x )
```

设置默认状态

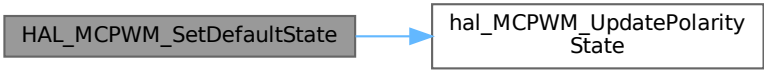
参数

<i>mcpwm_x</i>	操作的模块编号
<i>mcpwm_default_output_state_x</i>	默认输出状态

bit0 通道交换使能 0 交换 1 交换 bit1 上管有效电平 0 上管低有效 1 上管高有效 bit2 下管有效电平 0 下管低有效 1 下管高有效 bit3 默认上管电平 0 关机后上管关闭 1 关机后上管打开 bit4 默认下管电平 0 关机后下管关闭 1 关机后下管打开 bit5-bit7 输出状态 HAL_MCPWM_OUTPUT_STATE_x

在文件 [hal_lks32mc07x.mcpwm.c](#) 第 362 行定义.

函数调用图:



4.94.1.22 HAL_MCPWM_SetDrivePolarity()

```
void HALMCPWM.SetDrivePolarity (
    HALMCPWM.x mcpwm_x,
    HALMCPWM.POLARITY.x mcpwm.polarity_x )
```

设置pwm极性

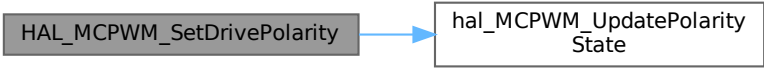
参数

<i>mcpwm_x</i>	操作的模块编号
<i>mcpwm_polarity_x</i>	第x个pwm输出端口

bit0 通道交换使能 0 交换 1 交换 bit1 上管有效电平 0 上管低有效 1 上管高有效 bit2 下管有效电平 0 下管低有效 1 下管高有效 bit3 默认上管电平 0 关机后上管关闭 1 关机后上管打开 bit4 默认下管电平 0 关机后下管关闭 1 关机后下管打开 bit5-bit7 输出状态 HAL_MCPWM_OUTPUT_STATE_x

在文件 [hal_lks32mc07x.mcpwm.c](#) 第 338 行定义.

函数调用图:



4.94.1.23 HAL_MCPWM_SetFailSignal()

```
void HAL_MCPWM_SetFailSignal (
    HAL_MCPWM_X mcpwm_x,
    HAL_MCPWM_FAIL_X fail_x )
```

设置FAIL信号来源

参数

mcpwm_x	操作的模块编号
fail_x	FAIL信号来源

在文件 [hal_lks32mc07x_mcpwm.c](#) 第 416 行定义.

4.94.1.24 HAL_MCPWM_SetFreq()

```
void HAL_MCPWM_SetFreq (
    HAL_MCPWM_X mcpwm_x,
    uint32_t freq )
```

设置pwm频率

参数

mcpwm_x	操作的模块编号
---------	---------

在文件 [hal_lks32mc07x_mcpwm.c](#) 第 207 行定义.

函数调用图:

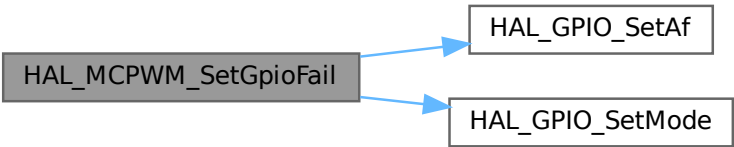


4.94.1.25 HAL_MCPWM_SetGpioFail()

```
void HAL_MCPWM_SetGpioFail (
    HAL_MCPWM_X mcpwm_x,
    HAL_GPIO_Px px )
```

在文件 [hal_lks32mc07x_mcpwm.c](#) 第 404 行定义.

函数调用图:



4.94.1.26 HAL_MCPWM_SetGpioOut()

```
void HAL_MCPWM_SetGpioOut (
    HAL_MCPWM_TypeDef* mcpwm_x,
    GPIO_TypeDef* px )
```

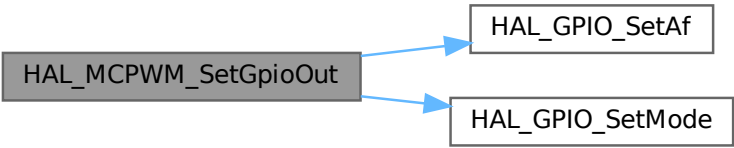
设置用于输出的gpio

参数

<i>mcpwm_x</i>	MCPWM模块编号
<i>px</i>	使用的GPIO口

在文件 `hal_lks32mc07x_mcpwm.c` 第 398 行定义.

函数调用图:



4.94.1.27 HAL_MCPWM_SetOutputDisable()

```
void HAL_MCPWM_SetOutputDisable (
    HAL_MCPWM_TypeDef* mcpwm_x )
```

关闭输出

参数

<i>mcpwm</i> ↔ <i>_x</i>	操作的模块编号
-----------------------------	---------

在文件 [hal_lks32mc07x_mcpwm.c](#) 第 257 行定义.

4.94.1.28 HAL_MCPWM_SetOutputEnable()

```
void HAL_MCPWM_SetOutputEnable (
    HAL_MCPWM_x mcpwm_x )
```

打开输出

参数

<i>mcpwm</i> ↔ <i>_x</i>	操作的模块编号
-----------------------------	---------

在文件 [hal_lks32mc07x_mcpwm.c](#) 第 267 行定义.

4.94.1.29 HAL_MCPWM_SetOutState()

```
void HAL_MCPWM_SetOutState (
    HAL_MCPWM_x mcpwm_x,
    HAL_MCPWM_PORT_x mcpwm_port_x,
    HAL_MCPWM_OUTPUT_STATE_x mcpwm_output_state_x )
```

设置输出状态

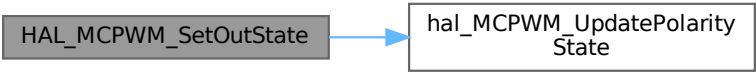
参数

<i>mcpwm_x</i>	操作的模块编号
<i>mcpwm_port_x</i>	第x个pwm输出端口
<i>mcpwm_output</i> ↔ <i>state_x</i>	输出状态

bit0 通道交换使能 0 交换 1 交换 bit1 上管有效电平 0 上管低有效 1 上管高有效 bit2 下管有效电平 0 下管低有效 1 下管高有效 bit3 默认上管电平 0 关机后上管关闭 1 关机后上管打开 bit4 默认下管电平 0 关机后下管关闭 1 关机后下管打开 bit5-bit7 输出状态 HAL_MCPWM_OUTPUT_STATE_x

在文件 [hal_lks32mc07x_mcpwm.c](#) 第 319 行定义.

函数调用图:



4.94.1.30 HAL_MCPWM_SetPwm()

```
void HAL_MCPWM_SetPwm (
    HAL_MCPWM_x mcpwm_x,
    HAL_MCPWM_PORT_x mcpwm_port_x,
    int32_t t0,
    int32_t t1 )
```

设置占空比

参数

<i>mcpwm_x</i>	操作的模块编号
<i>mcpwm_↔port_x</i>	第x个pwm输出端口
<i>t0</i>	pwm打开时刻
<i>t1</i>	pwm关闭时刻

在文件 [hal_lks32mc07x.mcpwm.c](#) 第 300 行定义.

这是这个函数的调用关系图:

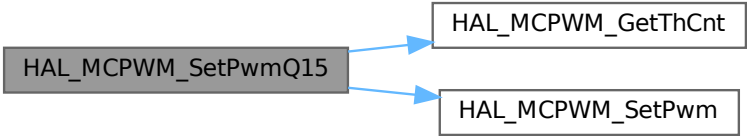


4.94.1.31 HAL_MCPWM_SetPwmQ15()

```
void HAL_MCPWM_SetPwmQ15 (
    HAL_MCPWM_x mcpwm_x,
    HAL_MCPWM_PORT_x mcpwm_port_x,
    int16_t t0,
    int16_t t1 )
```

在文件 [hal_lks32mc07x_mcpwm.c](#) 第 306 行定义.

函数调用图:



4.94.1.32 HAL_MCPWM_StartCounting()

```
void HALMCPWM_StartCounting (
    HAL_MCPWM_x mcpwm_x )
```

开始计数

参数

<i>mcpwm_x</i>	操作的模块编号
----------------	---------

在文件 [hal_lks32mc07x_mcpwm.c](#) 第 277 行定义.

4.94.1.33 HAL_MCPWM_StopCounting()

```
void HALMCPWM_StopCounting (
    HAL_MCPWM_x mcpwm_x )
```

停止计数(同时计数器回到-th)

参数

<i>mcpwm_x</i>	操作的模块编号
----------------	---------

在文件 [hal_lks32mc07x_mcpwm.c](#) 第 287 行定义.

4.94.1.34 hal_MCPWM_UpdatePolarityState()

```
void halMCPWM_UpdatePolarityState (
    HAL_MCPWM_x mcpwm_x,
    HAL_MCPWM_PORT_x mcpwm_port_x )
```

内部使用,输出端口的状态

参数

<i>mcpwm_x</i>	操作的模块编号
<i>mcpwm_x</i> <i>port_x</i>	第x个pwm输出端口

< 上下管对称pwm

< 关闭输出

< 下管恒开

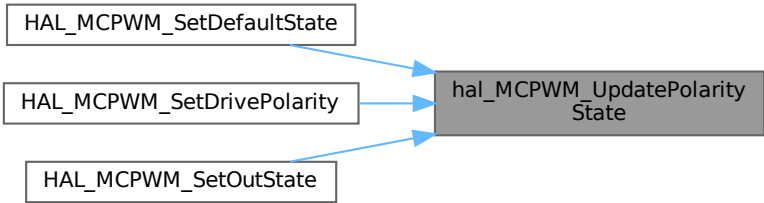
< 上管恒开

< 下管pwm

< 上管pwm

在文件 [hal_lks32mc07x.mcpwm.c](#) 第 22 行定义.

这是这个函数的调用关系图:



4.94.2 变量说明

4.94.2.1 mcpwm_polarity_state

```
uint16_t mcpwm_polarity_state[1][6] = {{0, 0, 0, 0, 0, 0}} [static]
```

bit0 通道交换使能 0 交换 1 交换 bit1 上管有效电平 0 上管低有效 1 上管高有效 bit2 下管有效电平 0 下管低有效 1 下管高有效 bit3 默认上管电平 0 关机后上管关闭 1 关机后上管打开 bit4 默认下管电平 0 关机后下管关闭 1 关机后下管打开 bit5-bit7 输出状态 HAL_MCPWM_OUTPUT_STATE_x

在文件 [hal_lks32mc07x.mcpwm.c](#) 第 15 行定义.

4.95 hal_lks32mc07x_mcpwm.c

[浏览该文件的文档.](#)

```

00001 #include "../include/hal_lksmcu_mcpwm.h"
00002 #include "lks32mc07x.h"
00003 #include "lks32mc07x_user_manual.h"
00004 #include "../include/hal_lksmcu_reg.h"
00005 #include "../include/hal_lks32mc07x_reg.h"
00006
00015 static uint16_t mcpwm_polarity_state[1][6] = {{0, 0, 0, 0, 0, 0}};
00016
00022 void hal_MCPWM_UpdatePolarityState(HAL_MCPWM_x mcpwm_x, HAL_MCPWM_PORT_x mcpwm_port_x)
00023 {
00024     (void)mcpwm_x;
00025     (void)mcpwm_port_x;
00026     uint8_t tmp[6] = {0, 0, 0, 0, 0, 0};
00027     uint8_t tmp1[6] = {0, 0, 0, 0, 0, 0};
00028     int32_t i;
00029     uint32_t ch_def = 0;
00030     uint32_t io01 = 0;
00031     uint32_t io23 = 0;
00032     uint32_t io45 = 0;
00033     HAL_REG_IF_WRITE(io01, BIT6, mcpwm_polarity_state[0][0] & BIT0); // 打开通道交换
00034     HAL_REG_IF_WRITE(io01, BIT14, mcpwm_polarity_state[0][1] & BIT0); // 打开通道交换
00035     HAL_REG_IF_WRITE(io23, BIT6, mcpwm_polarity_state[0][2] & BIT0); // 打开通道交换
00036     HAL_REG_IF_WRITE(io23, BIT14, mcpwm_polarity_state[0][3] & BIT0); // 打开通道交换
00037     HAL_REG_IF_WRITE(io45, BIT6, mcpwm_polarity_state[0][4] & BIT0); // 打开通道交换
00038     HAL_REG_IF_WRITE(io45, BIT14, mcpwm_polarity_state[0][5] & BIT0); // 打开通道交换
00039
00040     HAL_REG_IF_WRITE(io01, BIT1, !(mcpwm_polarity_state[0][0] & BIT1)); // 上管高有效
00041     HAL_REG_IF_WRITE(io01, BIT9, !(mcpwm_polarity_state[0][1] & BIT1)); // 上管高有效
00042     HAL_REG_IF_WRITE(io23, BIT1, !(mcpwm_polarity_state[0][2] & BIT1)); // 上管高有效
00043     HAL_REG_IF_WRITE(io23, BIT9, !(mcpwm_polarity_state[0][3] & BIT1)); // 上管高有效
00044     HAL_REG_IF_WRITE(io45, BIT1, !(mcpwm_polarity_state[0][4] & BIT1)); // 上管高有效
00045     HAL_REG_IF_WRITE(io45, BIT9, !(mcpwm_polarity_state[0][5] & BIT1)); // 上管高有效
00046
00047     HAL_REG_IF_WRITE(io01, BIT0, !(mcpwm_polarity_state[0][0] & BIT2)); // 下管高有效
00048     HAL_REG_IF_WRITE(io01, BIT8, !(mcpwm_polarity_state[0][1] & BIT2)); // 下管高有效
00049     HAL_REG_IF_WRITE(io23, BIT0, !(mcpwm_polarity_state[0][2] & BIT2)); // 下管高有效
00050     HAL_REG_IF_WRITE(io23, BIT8, !(mcpwm_polarity_state[0][3] & BIT2)); // 下管高有效
00051     HAL_REG_IF_WRITE(io45, BIT0, !(mcpwm_polarity_state[0][4] & BIT2)); // 下管高有效
00052     HAL_REG_IF_WRITE(io45, BIT8, !(mcpwm_polarity_state[0][5] & BIT2)); // 下管高有效
00053
00054     for (i = 0; i < 6; i++)
00055     {
00056         if (mcpwm_polarity_state[0][i] & BIT4)
00057         {
00058             // 关机后上管打开
00059             if (mcpwm_polarity_state[0][i] & BIT0) // gpio交换使能
00060             {
00061                 HAL_REG_IF_WRITE(tmp[i], BIT0, (mcpwm_polarity_state[0][i] & BIT1)); // CHO N 通道默认值
00062                 HAL_REG_IF_WRITE(tmp[i], BIT1, !(mcpwm_polarity_state[0][i] & BIT2)); // CHO P 通道默认值
00063             }
00064             else
00065             {
00066                 HAL_REG_IF_WRITE(tmp[i], BIT0, !(mcpwm_polarity_state[0][i] & BIT2)); // CHO N 通道默认值
00067                 HAL_REG_IF_WRITE(tmp[i], BIT1, (mcpwm_polarity_state[0][i] & BIT1)); // CHO P 通道默认值
00068             }
00069         }
00070         else if (mcpwm_polarity_state[0][i] & BIT3)
00071         {
00072             // 关机后下管打开
00073             if (mcpwm_polarity_state[0][i] & BIT0) // gpio交换使能
00074             {
00075                 HAL_REG_IF_WRITE(tmp[i], BIT0, !(mcpwm_polarity_state[0][i] & BIT1)); // CHO N 通道默认值
00076                 HAL_REG_IF_WRITE(tmp[i], BIT1, (mcpwm_polarity_state[0][i] & BIT2)); // CHO P 通道默认值
00077             }
00078             else
00079             {
00080                 HAL_REG_IF_WRITE(tmp[i], BIT0, (mcpwm_polarity_state[0][i] & BIT2)); // CHO N 通道默认值
00081                 HAL_REG_IF_WRITE(tmp[i], BIT1, !(mcpwm_polarity_state[0][i] & BIT1)); // CHO P 通道默认值
00082             }
00083         }
00084         else
00085         {
00086             // 关机后gpio悬空
00087             // 关机后下管打开
00088             if (mcpwm_polarity_state[0][i] & BIT0) // gpio交换使能
00089             {
00090                 HAL_REG_IF_WRITE(tmp[i], BIT0, !(mcpwm_polarity_state[0][i] & BIT1)); // CHO N 通道默认值
00091                 HAL_REG_IF_WRITE(tmp[i], BIT1, !(mcpwm_polarity_state[0][i] & BIT2)); // CHO P 通道默认值
00092             }
00093             else
00094             {
00095                 HAL_REG_IF_WRITE(tmp[i], BIT0, !(mcpwm_polarity_state[0][i] & BIT2)); // CHO N 通道默认值

```

```

00096         HAL_REG_IF_WRITE(tmp[i], BIT1, !(mcpwm.polarity_state[0][i] & BIT1)); // CHO P 通道默认值
00097     }
00098 }
00099 }
00100 HAL_REG_IF_WRITE(ch_def, BIT0, tmp[0] & BIT0); // 默认输出状态
00101 HAL_REG_IF_WRITE(ch_def, BIT1, tmp[0] & BIT1); // 默认输出状态
00102 HAL_REG_IF_WRITE(ch_def, BIT2, tmp[1] & BIT0); // 默认输出状态
00103 HAL_REG_IF_WRITE(ch_def, BIT3, tmp[1] & BIT1); // 默认输出状态
00104 HAL_REG_IF_WRITE(ch_def, BIT4, tmp[2] & BIT0); // 默认输出状态
00105 HAL_REG_IF_WRITE(ch_def, BIT5, tmp[2] & BIT1); // 默认输出状态
00106 HAL_REG_IF_WRITE(ch_def, BIT6, tmp[3] & BIT0); // 默认输出状态
00107 HAL_REG_IF_WRITE(ch_def, BIT7, tmp[3] & BIT1); // 默认输出状态
00108 HAL_REG_IF_WRITE(ch_def, BIT8, tmp[4] & BIT0); // 默认输出状态
00109 HAL_REG_IF_WRITE(ch_def, BIT9, tmp[4] & BIT1); // 默认输出状态
00110 HAL_REG_IF_WRITE(ch_def, BIT10, tmp[5] & BIT0); // 默认输出状态
00111 HAL_REG_IF_WRITE(ch_def, BIT11, tmp[5] & BIT1); // 默认输出状态
00112
00113 for (i = 0; i < 6; i++)
00114 {
00115     HAL_MCPWM_OUTPUT_STATE_x out_state;
00116     out_state = (HAL_MCPWM_OUTPUT_STATE_x)(mcpwm.polarity_state[0][i] >> 5);
00117     switch (out_state)
00118     {
00119         case HAL_MCPWM_OUTPUT_STATE_PWM:
00120             tmp1[i] = 0;
00121             break;
00122         case HAL_MCPWM_OUTPUT_STATE_OFF:
00123             tmp1[i] = BIT2 | BIT3;
00124             break;
00125         case HAL_MCPWM_OUTPUT_STATE_LOW:
00126             tmp1[i] = BIT2 | BIT3 | BIT4;
00127             break;
00128         case HAL_MCPWM_OUTPUT_STATE_HIGH:
00129             tmp1[i] = BIT2 | BIT3 | BIT5;
00130             break;
00131         case HAL_MCPWM_OUTPUT_STATE_LOW_PWM:
00132             tmp1[i] = BIT3;
00133             break;
00134         case HAL_MCPWM_OUTPUT_STATE_HIGH_PWM:
00135             tmp1[i] = BIT2;
00136             break;
00137     }
00138 }
00139 HAL_REG_WRITE_BIT(io01, 0x003c, tmp1[0]);
00140 HAL_REG_WRITE_BIT(io01, 0x3c00, tmp1[1] << 8);
00141 HAL_REG_WRITE_BIT(io23, 0x003c, tmp1[2]);
00142 HAL_REG_WRITE_BIT(io23, 0x3c00, tmp1[3] << 8);
00143 HAL_REG_WRITE_BIT(io45, 0x003c, tmp1[4]);
00144 HAL_REG_WRITE_BIT(io45, 0x3c00, tmp1[5] << 8);
00145
00146 HAL_REG_WRITE(HAL_MCPWM[mcpwm_x]->PRT, 0xdead);
00147 HAL_REG_WRITE_BIT(HAL_MCPWM[mcpwm_x]->CH_DEF, 0xff00, ch_def);
00148 HAL_REG_WRITE_BIT(HAL_MCPWM[mcpwm_x]->IO01, 0x7f7f, io01);
00149 HAL_REG_WRITE_BIT(HAL_MCPWM[mcpwm_x]->IO23, 0x7f7f, io23);
00150 HAL_REG_WRITE_BIT(HAL_MCPWM[mcpwm_x]->IO45, 0x7f7f, io45);
00151 HAL_REG_WRITE(HAL_MCPWM[mcpwm_x]->PRT, 0);
00152 }
00153
00154 void HAL_MCPWM_Reset(HAL_MCPWM_x mcpwm_x)
00155 {
00156     (void)mcpwm_x;
00157     // 系统寄存器解锁
00158     HAL_REG_SYS_UNLOCK();
00159     HAL_REG_SET(SYS_SFT_RST, SYS_SFT_RST_MCPWM0_SFT_RST_BIT);
00160     HAL_REG_RESET(SYS_SFT_RST, SYS_SFT_RST_MCPWM0_SFT_RST_BIT);
00161     // 系统寄存器上锁
00162     HAL_REG_SYS_LOCK();
00163 }
00164
00165 void HAL_MCPWM_Enable(HAL_MCPWM_x mcpwm_x)
00166 {
00167     // 系统寄存器解锁
00168     HAL_REG_SYS_UNLOCK();
00169     // SYS_CLK_FEN[4] MCPWM 模块整体时钟控制信号
00170     HAL_REG_SET(SYS_CLK_FEN, SYS_CLK_FEN_MCPWM0_CLK_EN_BIT);
00171     // 系统寄存器上锁
00172     HAL_REG_SYS_LOCK();
00173
00174     HAL_REG_WRITE(HAL_MCPWM[mcpwm_x]->PRT, 0xdead);
00175     HAL_REG_SET(HAL_MCPWM[mcpwm_x]->TCLK, BIT2);
00176     // HAL_REG_WRITE_BIT(HAL_MCPWM[mcpwm_x]->AUEN, 0xffff, 0x43f);
00177     HAL_REG_SET(HAL_MCPWM[mcpwm_x]->SDCFG, BIT4);
00178     HAL_REG_WRITE(HAL_MCPWM[mcpwm_x]->PRT, 0);
00179 }
00180
00181 void HAL_MCPWM_Disable(HAL_MCPWM_x mcpwm_x)
00182 {
00183     HAL_REG_WRITE(HAL_MCPWM[mcpwm_x]->PRT, 0xdead);
00184     HAL_REG_RESET(HAL_MCPWM[mcpwm_x]->TCLK, BIT2);
00185 }

```

```

00195     HAL_REG_WRITE(HAL_MCPWM[mcpwm_x] -> PRT, 0);
00196     // 系统寄存器解锁
00197     HAL_REG_SYS_UNLOCK();
00198     // SYS_CLK_FEN[4] MCPWM 模块整体时钟控制信号
00199     HAL_REG_RESET(SYS_CLK_FEN, SYS_CLK_FEN_MCPWM0_CLK_EN_BIT);
00200     // 系统寄存器上锁
00201     HAL_REG_SYS_LOCK();
00202 }
00207 void HAL_MCPWM_SetFreq(HAL_MCPWM_x mcpwm_x, uint32_t freq)
00208 {
00209     uint32_t mclk = HAL_CLK_GetMcuFreq();
00210     uint32_t th = ((mclk / freq) - 1) / 2;
00211     HAL_REG_WRITE(HAL_MCPWM[mcpwm_x] -> PRT, 0xdead);
00212     HAL_REG_WRITE(HAL_MCPWM[mcpwm_x] -> TH0, th);
00213     HAL_REG_WRITE(HAL_MCPWM[mcpwm_x] -> PRT, 0);
00214     HAL_REG_WRITE(HAL_MCPWM[mcpwm_x] -> UPDATE, BIT12);
00215 }
00221 uint32_t HAL_MCPWM_GetFreq(HAL_MCPWM_x mcpwm_x)
00222 {
00223     uint32_t mclk = HAL_CLK_GetMcuFreq();
00224     uint16_t th = HAL_MCPWM[mcpwm_x] -> TH0;
00225     uint32_t freq = mclk / (th * 2 + 1);
00226     return freq;
00227 }
00232 uint32_t HAL_MCPWM_GetThCnt(HAL_MCPWM_x mcpwm_x)
00233 {
00234     return HAL_MCPWM[mcpwm_x] -> TH0;
00235 }
00242 void HAL_MCPWM_SetDead(HAL_MCPWM_x mcpwm_x, uint32_t pdead_ns, uint32_t ndead_ns)
00243 {
00244     uint32_t mclk = HAL_CLK_GetMcuFreq();
00245     uint16_t pdead = (int64_t)pdead_ns * mclk / 1000000000;
00246     uint16_t ndead = (int64_t)ndead_ns * mclk / 1000000000;
00247     HAL_REG_WRITE(HAL_MCPWM[mcpwm_x] -> PRT, 0xdead);
00248     HAL_REG_WRITE(HAL_MCPWM[mcpwm_x] -> DTH00, pdead);
00249     HAL_REG_WRITE(HAL_MCPWM[mcpwm_x] -> DTH01, ndead);
00250     HAL_REG_WRITE(HAL_MCPWM[mcpwm_x] -> PRT, 0);
00251 }
00252
00257 void HAL_MCPWM_SetOutputDisable(HAL_MCPWM_x mcpwm_x)
00258 {
00259     HAL_REG_WRITE(HAL_MCPWM[mcpwm_x] -> PRT, 0xdead);
00260     HAL_REG_RESET(HAL_MCPWM[mcpwm_x] -> FAIL012, BIT6);
00261     HAL_REG_WRITE(HAL_MCPWM[mcpwm_x] -> PRT, 0);
00262 }
00267 void HAL_MCPWM_SetOutputEnable(HAL_MCPWM_x mcpwm_x)
00268 {
00269     HAL_REG_WRITE(HAL_MCPWM[mcpwm_x] -> PRT, 0xdead);
00270     HAL_REG_SET(HAL_MCPWM[mcpwm_x] -> FAIL012, BIT6);
00271     HAL_REG_WRITE(HAL_MCPWM[mcpwm_x] -> PRT, 0);
00272 }
00277 void HAL_MCPWM_StartCounting(HAL_MCPWM_x mcpwm_x)
00278 {
00279     HAL_REG_WRITE(HAL_MCPWM[mcpwm_x] -> PRT, 0xdead);
00280     HAL_REG_SET(HAL_MCPWM[mcpwm_x] -> TCLK, BIT6);
00281     HAL_REG_WRITE(HAL_MCPWM[mcpwm_x] -> PRT, 0);
00282 }
00287 void HAL_MCPWM_StopCounting(HAL_MCPWM_x mcpwm_x)
00288 {
00289     HAL_REG_WRITE(HAL_MCPWM[mcpwm_x] -> PRT, 0xdead);
00290     HAL_REG_RESET(HAL_MCPWM[mcpwm_x] -> TCLK, BIT6);
00291     HAL_REG_WRITE(HAL_MCPWM[mcpwm_x] -> PRT, 0);
00292 }
00300 void HAL_MCPWM_SetPwm(HAL_MCPWM_x mcpwm_x, HAL_MCPWM_PORT_x mcpwm_port_x, int32_t t0, int32_t t1)
00301 {
00302     // 使用公式写可以在内联的情况下, 实现和直接配置寄存器一样的效果
00303     HAL_REG_WRITE(HAL_MCPWM[mcpwm_x] -> TH[mcpwm_port_x * 2], t0);
00304     HAL_REG_WRITE(HAL_MCPWM[mcpwm_x] -> TH[mcpwm_port_x * 2 + 1], t1);
00305 }
00306 void HAL_MCPWM_SetPwmQ15(HAL_MCPWM_x mcpwm_x, HAL_MCPWM_PORT_x mcpwm_port_x, int16_t t0, int16_t t1)
00307 {
00308     uint32_t th = HAL_MCPWM_GetThCnt(mcpwm_x);
00309     int32_t cnt0 = (t0 * th) >> 15;
00310     int32_t cnt1 = (t1 * th) >> 15;
00311     HAL_MCPWM_SetPwm(mcpwm_x, mcpwm_port_x, cnt0, cnt1);
00312 }
00319 void HAL_MCPWM_SetOutState(HAL_MCPWM_x mcpwm_x, HAL_MCPWM_PORT_x mcpwm_port_x, HAL_MCPWM_OUTPUT_STATE_x
mcpwm_output_state_x)
00320 {
00321     HAL_REG_WRITE_BIT(mcpwm_polarity_state[mcpwm_x][mcpwm_port_x], BIT5 | BIT6 | BIT7,
mcpwm_output_state_x << 5);
00322     HAL_MCPWM_UpdatePolarityState(mcpwm_x, mcpwm_port_x);
00323 }
00324
00338 void HAL_MCPWM_SetDrivePolarity(HAL_MCPWM_x mcpwm_x, HAL_MCPWM_POLARITY_x mcpwm_polarity_x)
00339 {
00340     HAL_REG_WRITE_BIT(mcpwm_polarity_state[mcpwm_x][HAL_MCPWM_PORT_0], BIT1 | BIT2, mcpwm_polarity_x <<

```

```

1);
00349 HAL_REG_WRITE_BIT(mcpwm_polarity_state[mcpwm_x][HAL_MCPWM_PORT_1], BIT1 | BIT2, mcpwm_polarity_x <<
1);
00350 HAL_REG_WRITE_BIT(mcpwm_polarity_state[mcpwm_x][HAL_MCPWM_PORT_2], BIT1 | BIT2, mcpwm_polarity_x <<
1);
00351 HAL_REG_WRITE_BIT(mcpwm_polarity_state[mcpwm_x][HAL_MCPWM_PORT_3], BIT1 | BIT2, mcpwm_polarity_x <<
1);
00352 HAL_MCPWM_UpdatePolarityState(mcpwm_x, HAL_MCPWM_PORT_0);
00353 HAL_MCPWM_UpdatePolarityState(mcpwm_x, HAL_MCPWM_PORT_1);
00354 HAL_MCPWM_UpdatePolarityState(mcpwm_x, HAL_MCPWM_PORT_2);
00355 HAL_MCPWM_UpdatePolarityState(mcpwm_x, HAL_MCPWM_PORT_3);
00356 }
00362 void HAL_MCPWM_SetDefaultState(HAL_MCPWM_x mcpwm_x, HAL_MCPWM_DEFAULT_OUTPUT_STATE_x
mcpwm_default_output_state_x)
00363 {
00364     uint32_t val = 0;
00365     switch (mcpwm_default_output_state_x)
00366     {
00367         case HAL_MCPWM_DEFAULT_OUTPUT_STATE_OFF:
00368             break;
00369         case HAL_MCPWM_DEFAULT_OUTPUT_STATE_LOW:
00370             HAL_REG_SET(val, BIT4);
00371             break;
00372         case HAL_MCPWM_DEFAULT_OUTPUT_STATE_HIGH:
00373             HAL_REG_SET(val, BIT3);
00374             break;
00375     }
00384 HAL_REG_WRITE_BIT(mcpwm_polarity_state[mcpwm_x][HAL_MCPWM_PORT_0], BIT3 | BIT4, val);
00385 HAL_REG_WRITE_BIT(mcpwm_polarity_state[mcpwm_x][HAL_MCPWM_PORT_1], BIT3 | BIT4, val);
00386 HAL_REG_WRITE_BIT(mcpwm_polarity_state[mcpwm_x][HAL_MCPWM_PORT_2], BIT3 | BIT4, val);
00387 HAL_REG_WRITE_BIT(mcpwm_polarity_state[mcpwm_x][HAL_MCPWM_PORT_3], BIT3 | BIT4, val);
00388 HAL_MCPWM_UpdatePolarityState(mcpwm_x, HAL_MCPWM_PORT_0);
00389 HAL_MCPWM_UpdatePolarityState(mcpwm_x, HAL_MCPWM_PORT_1);
00390 HAL_MCPWM_UpdatePolarityState(mcpwm_x, HAL_MCPWM_PORT_2);
00391 HAL_MCPWM_UpdatePolarityState(mcpwm_x, HAL_MCPWM_PORT_3);
00392 }
00398 void HAL_MCPWM_SetGpioOut(HAL_MCPWM_x mcpwm_x, HAL_GPIO_Px px)
00399 {
00400     (void)mcpwm_x;
00401     HAL_GPIO_SetAf(px, HAL_GPIO_AF_MCPWM);
00402     HAL_GPIO_SetMode(px, HAL_GPIO_MODE_OUT);
00403 }
00404 void HAL_MCPWM_SetGpioFail(HAL_MCPWM_x mcpwm_x, HAL_GPIO_Px px) // 设置用于故障输入的gpio
00405 {
00406     (void)mcpwm_x;
00407     HAL_GPIO_SetAf(px, HAL_GPIO_AF_MCPWM);
00408     HAL_GPIO_SetMode(px, HAL_GPIO_MODE_IN);
00409 }
00410
00416 void HAL_MCPWM_SetFailSignal(HAL_MCPWM_x mcpwm_x, HAL_MCPWM_FAIL_x fail_x)
00417 {
00418     (void)mcpwm_x;
00419     (void)fail_x;
00420 }
00421 // 设置pwm输出ADC采样时间
00422 void HAL_MCPWM_SetAdcTrig(HAL_MCPWM_x mcpwm_x, int32_t t0, int32_t t1)
00423 {
00424     HAL_REG_WRITE(HAL_MCPWM[mcpwm_x]->TMR0, t0);
00425     HAL_REG_WRITE(HAL_MCPWM[mcpwm_x]->TMR1, t1);
00426 }
00427 // 设置pwm输出ADC采样时间q15格式输入
00428 void HAL_MCPWM_SetAdcTrigQ15(HAL_MCPWM_x mcpwm_x, int32_t t0, int32_t t1)
00429 {
00430     uint32_t th = HAL_MCPWM_GetThCnt(mcpwm_x);
00431     int16_t tmr0 = ((int32_t)t0 * th) >> 15;
00432     int16_t tmr1 = ((int32_t)t1 * th) >> 15;
00433     HAL_REG_WRITE(HAL_MCPWM[mcpwm_x]->PRT, 0xdead);
00434     HAL_REG_WRITE(HAL_MCPWM[mcpwm_x]->TMR0, tmr0);
00435     HAL_REG_WRITE(HAL_MCPWM[mcpwm_x]->TMR1, tmr1);
00436 }
00437
00438 uint8_t HAL_MCPWM_IrqOverflow(HAL_MCPWM_x mcpwm_x)
00439 {
00440     if (HAL_MCPWM[mcpwm_x]->IE0 & HAL_MCPWM[mcpwm_x]->IF0 & BIT0)
00441     {
00442         return 1;
00443     }
00444     else
00445     {
00446         return 0;
00447     }
00448 }
00449 void HAL_MCPWM_IrqOverflowEnable(HAL_MCPWM_x mcpwm_x)
00450 {
00451     HAL_REG_WRITE(HAL_MCPWM[mcpwm_x]->PRT, 0xdead);
00452     HAL_REG_SET(HAL_MCPWM[mcpwm_x]->IE0, BIT0);
00453     HAL_REG_WRITE(HAL_MCPWM[mcpwm_x]->PRT, 0);

```



```

00454 }
00455 void HAL_MCPWM_IrqOverflowDisable(HAL_MCPWM_x mcpwm_x)
00456 {
00457     HAL_REG_WRITE(HAL_MCPWM[mcpwm_x] -> PRT, 0xdead);
00458     HAL_REG_RESET(HAL_MCPWM[mcpwm_x] -> IE0, BIT0);
00459     HAL_REG_WRITE(HAL_MCPWM[mcpwm_x] -> PRT, 0);
00460 }
00461 void HAL_MCPWM_IrqOverflowClear(HAL_MCPWM_x mcpwm_x)
00462 {
00463     HAL_REG_WRITE(HAL_MCPWM[mcpwm_x] -> IF0, BIT0);
00464 }
00465
00466 uint8_t HAL_MCPWM_IrqZeroCross(HAL_MCPWM_x mcpwm_x)
00467 {
00468     if (HAL_MCPWM[mcpwm_x] -> IE0 & HAL_MCPWM[mcpwm_x] -> IF0 & BIT1)
00469     {
00470         return 1;
00471     }
00472     else
00473     {
00474         return 0;
00475     }
00476 }
00477 void HAL_MCPWM_IrqZeroCrossEnable(HAL_MCPWM_x mcpwm_x)
00478 {
00479     HAL_REG_WRITE(HAL_MCPWM[mcpwm_x] -> PRT, 0xdead);
00480     HAL_REG_SET(HAL_MCPWM[mcpwm_x] -> IE0, BIT1);
00481     HAL_REG_WRITE(HAL_MCPWM[mcpwm_x] -> PRT, 0);
00482 }
00483 void HAL_MCPWM_IrqZeroCrossDisable(HAL_MCPWM_x mcpwm_x)
00484 {
00485     HAL_REG_WRITE(HAL_MCPWM[mcpwm_x] -> PRT, 0xdead);
00486     HAL_REG_RESET(HAL_MCPWM[mcpwm_x] -> IE0, BIT1);
00487     HAL_REG_WRITE(HAL_MCPWM[mcpwm_x] -> PRT, 0);
00488 }
00489 void HAL_MCPWM_IrqZeroCrossClear(HAL_MCPWM_x mcpwm_x)
00490 {
00491     HAL_REG_WRITE(HAL_MCPWM[mcpwm_x] -> IF0, BIT1);
00492 }
00493
00494 uint8_t HAL_MCPWM_IrqFail(HAL_MCPWM_x mcpwm_x)
00495 {
00496     if (HAL_MCPWM[mcpwm_x] -> EIE & HAL_MCPWM[mcpwm_x] -> EIF & (BIT4 | BIT5))
00497     {
00498         return 1;
00499     }
00500     else
00501     {
00502         return 0;
00503     }
00504 }
00505 void HAL_MCPWM_IrqFailEnable(HAL_MCPWM_x mcpwm_x)
00506 {
00507     HAL_REG_WRITE(HAL_MCPWM[mcpwm_x] -> PRT, 0xdead);
00508     HAL_REG_SET(HAL_MCPWM[mcpwm_x] -> IE0, BIT4 | BIT5);
00509     HAL_REG_WRITE(HAL_MCPWM[mcpwm_x] -> PRT, 0);
00510 }
00511 void HAL_MCPWM_IrqFailDisable(HAL_MCPWM_x mcpwm_x)
00512 {
00513     HAL_REG_WRITE(HAL_MCPWM[mcpwm_x] -> PRT, 0xdead);
00514     HAL_REG_RESET(HAL_MCPWM[mcpwm_x] -> IE0, BIT4 | BIT5);
00515     HAL_REG_WRITE(HAL_MCPWM[mcpwm_x] -> PRT, 0);
00516 }
00517 void HAL_MCPWM_IrqFailClear(HAL_MCPWM_x mcpwm_x)
00518 {
00519     HAL_REG_WRITE(HAL_MCPWM[mcpwm_x] -> IF0, BIT4 | BIT5);
00520 }

```

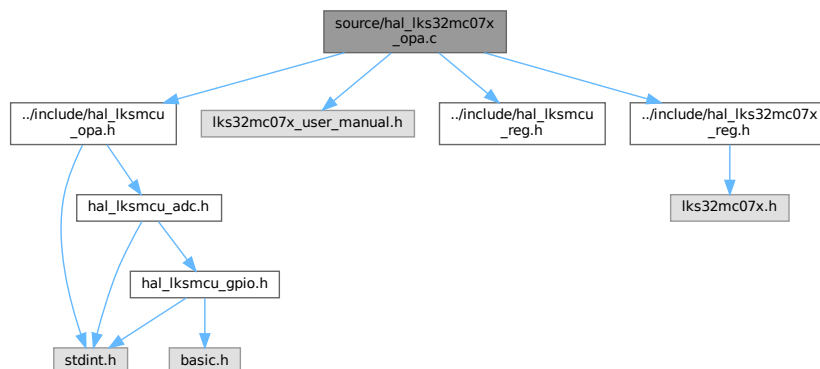
4.96 source/hal_lks32mc07x_opa.c 文件参考

```

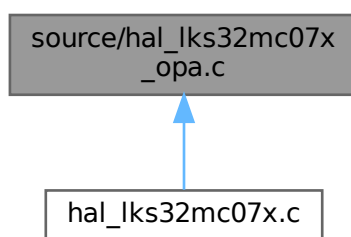
#include "../include/hal_lksmcu_opa.h"
#include "lks32mc07x_user_manual.h"
#include "../include/hal_lksmcu_reg.h"
#include "../include/hal_lks32mc07x_reg.h"

```

hal_lks32mc07x_opa.c 的引用(Include)关系图:



此图展示该文件直接或间接的被哪些文件引用了:



函数

- void `HAL_OPA_Reset` (`HAL_OPA_x` opa_x)
- void `HAL_OPA_Enable` (`HAL_OPA_x` opa_x)
- void `HAL_OPA_Disable` (`HAL_OPA_x` opa_x)
- void `HAL_OPA_SetGain` (`HAL_OPA_x` opa_x, `HAL_OPA_GAIN_x` gain_x)
- void `HAL_OPA_Out2GpioEnable` (`HAL_OPA_x` opa_x)
OPA输出到IO
- void `HAL_OPA_Out2GpioDisable` (`HAL_OPA_x` opa_x)
OPA停止输出到IO
- void `HAL_OPA_SetGpioIn` (`HAL_OPA_x` opa_x, `HAL_GPIO_Px` px)
OPA设置IO输入
- void `HAL_OPA_SetGpioOut` (`HAL_OPA_x` opa_x, `HAL_GPIO_Px` px)
OPA设置IO输出

4.96.1 函数说明

4.96.1.1 HAL_OPA_Disable()

```
void HAL_OPA_Disable (
    HAL_OPA_x opa_x )
```

在文件 [hal_lks32mc07x_opa.c](#) 第 37 行定义.

4.96.1.2 HAL_OPA_Enable()

```
void HAL_OPA_Enable (
    HAL_OPA_x opa_x )
```

在文件 [hal_lks32mc07x_opa.c](#) 第 12 行定义.

4.96.1.3 HAL_OPA_Out2GpioDisable()

```
void HAL_OPA_Out2GpioDisable (
    HAL_OPA_x opa_x )
```

OPA停止输出到IO

参数

<i>opa_x</i>	运放序号
--------------	------

在文件 [hal_lks32mc07x_opa.c](#) 第 119 行定义.

4.96.1.4 HAL_OPA_Out2GpioEnable()

```
void HAL_OPA_Out2GpioEnable (
    HAL_OPA_x opa_x )
```

OPA输出到IO

参数

<i>opa_x</i>	运放序号
--------------	------

在文件 [hal_lks32mc07x_opa.c](#) 第 91 行定义.

4.96.1.5 HAL_OPA_Reset()

```
void HAL_OPA_Reset (
    HAL_OPA_x opa_x )
```

在文件 [hal_lks32mc07x_opa.c](#) 第 7 行定义.

4.96.1.6 HAL_OPA_SetGain()

```
void HAL_OPA_SetGain (
    HAL_OPA_x opa_x,
    HAL_OPA_GAIN_x gain_x )
```

在文件 [hal_lks32mc07x_opa.c](#) 第 62 行定义.

4.96.1.7 HAL_OPA_SetGpioIn()

```
void HAL_OPA_SetGpioIn (
    HAL_OPA_x opa_x,
    HAL_GPIO_Px px )
```

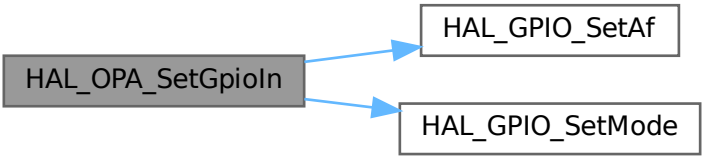
OPA设置IO输入

参数

<i>opa_x</i>	运放序号
<i>px</i>	IO序号

在文件 [hal_lks32mc07x_opa.c](#) 第 134 行定义.

函数调用图:



4.96.1.8 HAL_OPA_SetGpioOut()

```
void HAL_OPA_SetGpioOut (
    HAL_OPA_x opa_x,
    HAL_GPIO_Px px )
```

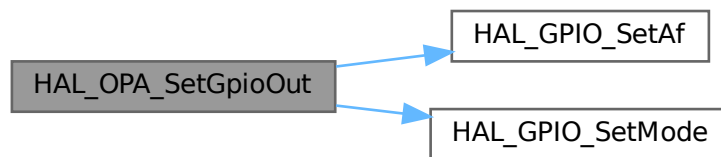
OPA设置IO输出

参数

<i>opa_x</i>	运放序号
<i>px</i>	IO序号

在文件 [hal_lks32mc07x_opa.c](#) 第 145 行定义。

函数调用图:



4.97 hal_lks32mc07x_opa.c

[浏览该文件的文档。](#)

```

00001 #include "../include/hal_lksmcu_opa.h"
00002 #include "lks32mc07x_user_manual.h"
00003 #include "../include/hal_lksmcu_reg.h"
00004 #include "../include/hal_lks32mc07x_reg.h"
00005
00006 // 模块复位
00007 void HAL_OPA_Reset(HAL_OPA_x opa_x)
00008 {
00009     (void)opa_x;
00010 }
00011 // 模块打开
00012 void HAL_OPA_Enable(HAL_OPA_x opa_x)
00013 {
00014     // 系统寄存器解锁
00015     HAL_REG_SYS_UNLOCK();
00016     switch (opa_x)
00017     {
00018         case HAL_OPA_0:
00019             HAL_REG_SET(SYS_AFE_REG5, SYS_AFE_REG5_OPA0PDN_BIT);
00020             break;
00021         case HAL_OPA_1:
00022             HAL_REG_SET(SYS_AFE_REG5, SYS_AFE_REG5_OPA1PDN_BIT);
00023             break;
00024         case HAL_OPA_2:
00025             HAL_REG_SET(SYS_AFE_REG5, SYS_AFE_REG5_OPA2PDN_BIT);
00026             break;
00027         case HAL_OPA_3:
00028             HAL_REG_SET(SYS_AFE_REG5, SYS_AFE_REG5_OPA3PDN_BIT);
00029             break;
00030         default:
00031             break;
00032     }
00033     // 系统寄存器上锁
00034     HAL_REG_SYS_LOCK();
00035 }
00036 // 模块关闭
00037 void HAL_OPA_Disable(HAL_OPA_x opa_x)
00038 {
00039     // 系统寄存器解锁
00040     HAL_REG_SYS_UNLOCK();
00041     switch (opa_x)
00042     {
  
```

```

00043         case HAL_OPA_0:
00044             HAL_REG_RESET(SYS_AFE_REG5, SYS_AFE_REG5_OPA0PDN_BIT);
00045             break;
00046         case HAL_OPA_1:
00047             HAL_REG_RESET(SYS_AFE_REG5, SYS_AFE_REG5_OPA1PDN_BIT);
00048             break;
00049         case HAL_OPA_2:
00050             HAL_REG_RESET(SYS_AFE_REG5, SYS_AFE_REG5_OPA2PDN_BIT);
00051             break;
00052         case HAL_OPA_3:
00053             HAL_REG_RESET(SYS_AFE_REG5, SYS_AFE_REG5_OPA3PDN_BIT);
00054             break;
00055         default:
00056             break;
00057     }
00058     // 系统寄存器上锁
00059     HAL_REG_SYS_LOCK();
00060 }
00061 // 设置增益
00062 void HAL_OPA_SetGain(HAL_OPA_x opa_x, HAL_OPA_GAIN_x gain_x)
00063 {
00064     // 系统寄存器解锁
00065     HAL_REG_SYS_UNLOCK();
00066     switch (opa_x)
00067     {
00068         case HAL_OPA_0:
00069             HAL_REG_WRITE_BIT_MASK_POS(SYS_AFE_REG0, SYS_AFE_REG0_RES_OPA0, gain_x);
00070             break;
00071         case HAL_OPA_1:
00072             HAL_REG_WRITE_BIT_MASK_POS(SYS_AFE_REG0, SYS_AFE_REG0_RES_OPA1, gain_x);
00073             break;
00074         case HAL_OPA_2:
00075             HAL_REG_WRITE_BIT_MASK_POS(SYS_AFE_REG0, SYS_AFE_REG0_RES_OPA2, gain_x);
00076             break;
00077         case HAL_OPA_3:
00078             HAL_REG_WRITE_BIT_MASK_POS(SYS_AFE_REG0, SYS_AFE_REG0_RES_OPA3, gain_x);
00079             break;
00080         default:
00081             break;
00082     }
00083     // 系统寄存器上锁
00084     HAL_REG_SYS_LOCK();
00085 }
00086
00091 void HAL_OPA_Out2GpioEnable(HAL_OPA_x opa_x)
00092 {
00093     // [2:0] 使能OPAx输出信号送至IO口p2-7 000:不输出; 001:输出OPA0信号到IO口; 010:输出OPA1信号到IO口; 011:输出OPA2信号到IO口; 100:输出OPA3信号到IO口; 101~111: 不允许此配置
00094
00095     HAL_REG_SYS_UNLOCK();
00096     switch (opa_x)
00097     {
00098         case HAL_OPA_0:
00099             HAL_REG_WRITE_BIT_MASK_POS(SYS_AFE_REG2, SYS_AFE_REG2_OPAOUT_EN, 1);
00100             break;
00101         case HAL_OPA_1:
00102             HAL_REG_WRITE_BIT_MASK_POS(SYS_AFE_REG2, SYS_AFE_REG2_OPAOUT_EN, 2);
00103             break;
00104         case HAL_OPA_2:
00105             HAL_REG_WRITE_BIT_MASK_POS(SYS_AFE_REG2, SYS_AFE_REG2_OPAOUT_EN, 3);
00106             break;
00107         case HAL_OPA_3:
00108             HAL_REG_WRITE_BIT_MASK_POS(SYS_AFE_REG2, SYS_AFE_REG2_OPAOUT_EN, 4);
00109             break;
00110         default:
00111             break;
00112     }
00113     HAL_REG_SYS_LOCK();
00114 }
00119 void HAL_OPA_Out2GpioDisable(HAL_OPA_x opa_x)
00120 {
00121     (void)opa_x;
00122     // [2:0] 使能OPAx输出信号送至IO口p2-7 000:不输出; 001:输出OPA0信号到IO口; 010:输出OPA1信号到IO口; 011:输出OPA2信号到IO口; 100:输出OPA3信号到IO口; 101~111: 不允许此配置
00123
00124     HAL_REG_SYS_UNLOCK();
00125     HAL_REG_WRITE_BIT_MASK_POS(SYS_AFE_REG2, SYS_AFE_REG2_OPAOUT_EN, 0);
00126     HAL_REG_SYS_LOCK();
00127 }
00128
00134 void HAL_OPA_SetGpioIn(HAL_OPA_x opa_x, HAL_GPIO_Px px)
00135 {
00136     (void)opa_x;
00137     HAL_GPIO_SetAf(px, HAL_GPIO_AF_OPA);
00138     HAL_GPIO_SetMode(px, HAL_GPIO_MODE_AFE);
00139 }
00145 void HAL_OPA_SetGpioOut(HAL_OPA_x opa_x, HAL_GPIO_Px px)

```

```

00146 {
00147     (void) opa_x;
00148     HAL_GPIO_SetAf(px, HAL_GPIO_AF_OPA);
00149     HAL_GPIO_SetMode(px, HAL_GPIO_MODE_AFE);
00150 }

```

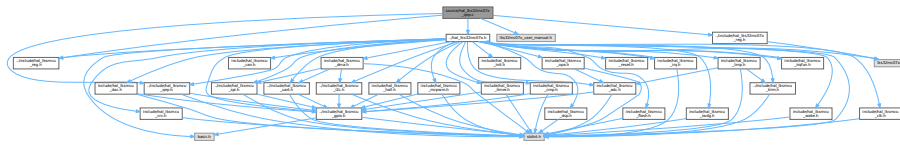
4.98 source/hal_lks32mc07x_qep.c 文件参考

```

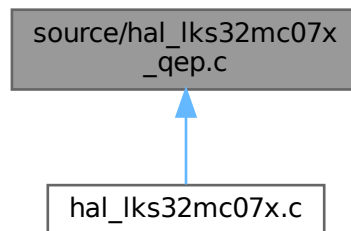
#include "../include/hal_lksmcu_reg.h"
#include "../include/hal_lksmcu_qep.h"
#include "../hal_lks32mc07x.h"
#include "lks32mc07x_user_manual.h"
#include "../include/hal_lks32mc07x_reg.h"

```

hal_lks32mc07x_qep.c 的引用(Include)关系图:



此图展示该文件直接或间接的被哪些文件引用了:



函数

- void [HAL_QEP_Reset](#) ([HAL_QEP_x](#) qep)
模块复位
- void [HAL_QEP_Enable](#) ([HAL_QEP_x](#) qep)
模块打开
- void [HAL_QEP_Disable](#) ([HAL_QEP_x](#) qep)
模块关闭
- void [HAL_QEP_SetTh](#) ([HAL_QEP_x](#) qep, uint32_t th)
设置最大计数值
- void [HAL_QEP_SetMode](#) ([HAL_QEP_x](#) qep, [HAL_QEP_MODE_x](#) mode)
设置计数模式
- uint32_t [HAL_QEP_ReadCnt](#) ([HAL_QEP_x](#) qep)

- 获取当前的计数值
- void HAL_QEP_ResetCnt (HAL_QEP_x qep)
复位计数值到0
 - void HAL_QEP_SetGpio (HAL_QEP_x qepx, HAL_GPIO_Px px)
设置QEP模块的引脚

4.98.1 函数说明

4.98.1.1 HAL_QEP_Disable()

```
void HAL_QEP_Disable (
    HAL_QEP_x qep )
```

模块关闭

参数

qep	QEP模块
-----	-------

在文件 [hal_lks32mc07x_qep.c](#) 第 63 行定义.

4.98.1.2 HAL_QEP_Enable()

```
void HAL_QEP_Enable (
    HAL_QEP_x qep )
```

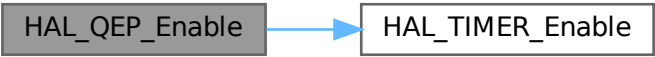
模块打开

参数

qep	QEP模块
-----	-------

在文件 [hal_lks32mc07x_qep.c](#) 第 37 行定义.

函数调用图:



4.98.1.3 HAL_QEP_ReadCnt()

```
uint32_t HAL_QEP_ReadCnt (
    HAL_QEP_x qep )
```

获取当前的计数值

参数

<i>qep</i>	QEP模块
------------	-------

返回

当前的计数值

在文件 [hal_lks32mc07x_qep.c](#) 第 121 行定义.

4.98.1.4 HAL_QEP_Reset()

```
void HAL_QEP_Reset (
    HAL_QEP_x qep )
```

模块复位

参数

<i>qep</i>	QEP模块
------------	-------

在文件 [hal_lks32mc07x_qep.c](#) 第 12 行定义.

4.98.1.5 HAL_QEP_ResetCnt()

```
void HAL_QEP_ResetCnt (
    HAL_QEP_x qep )
```

复位计数值到0

参数

<i>qep</i>	QEP模块
------------	-------

在文件 [hal_lks32mc07x_qep.c](#) 第 130 行定义.

4.98.1.6 HAL_QEP_SetGpio()

```
void HAL_QEP_SetGpio (
    HAL_QEP_x qepx,
    HAL_GPIO_Px px )
```

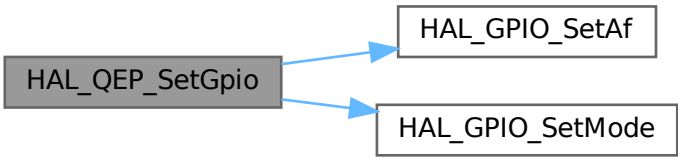
设置QEP模块的引脚

参数

<i>qepx</i>	QEP模块
<i>px</i>	引脚

在文件 [hal_lks32mc07x.qep.c](#) 第 140 行定义.

函数调用图:



4.98.1.7 HAL_QEP_SetMode()

```
void HAL_QEP_SetMode (
    HAL_QEP_x qep,
    HAL_QEP_MODE_x mode )
```

设置计数模式

参数

<i>qep</i>	QEP模块
<i>mode</i>	计数模式

在文件 [hal_lks32mc07x.qep.c](#) 第 98 行定义.

4.98.1.8 HAL_QEP_SetTh()

```
void HAL_QEP_SetTh (
    HAL_QEP_x qep,
    uint32_t th )
```

设置最大计数值

参数

<i>qep</i>	QEP模块
<i>th</i>	最大计数值

在文件 [hal_lks32mc07x_qep.c](#) 第 88 行定义.

4.99 hal_lks32mc07x_qep.c

[浏览该文件的文档.](#)

```

00001 #include "../include/hal_lksmcu.reg.h"
00002 #include "../include/hal_lksmcu.qep.h"
00003 #include "../hal_lks32mc07x.h"
00004 #include "lks32mc07x_user_manual.h"
00005 #include "../include/hal_lksmcu.reg.h"
00006 #include "../include/hal_lks32mc07x_reg.h"
00007
00012 void HAL_QEP_Reset (HAL_QEP_x qep)
00013 {
00014     switch (qep)
00015     {
00016         case HAL_QEP_0:
00017             HAL_REG_SYS_UNLOCK();
00018             HAL_REG_SET(SYS_SFT_RST, SYS_SFT_RST_QEP0_SFT_RST_BIT);
00019             HAL_REG_RESET(SYS_SFT_RST, SYS_SFT_RST_QEP0_SFT_RST_BIT);
00020             HAL_REG_SYS_LOCK();
00021             break;
00022         case HAL_QEP_1:
00023             HAL_REG_SYS_UNLOCK();
00024             HAL_REG_SET(SYS_SFT_RST, SYS_SFT_RST_QEP1_SFT_RST_BIT);
00025             HAL_REG_RESET(SYS_SFT_RST, SYS_SFT_RST_QEP1_SFT_RST_BIT);
00026             HAL_REG_SYS_LOCK();
00027             break;
00028         default:
00029             break;
00030     }
00031 }
00032
00037 void HAL_QEP_Enable (HAL_QEP_x qep)
00038 {
00039     switch (qep)
00040     {
00041         case HAL_QEP_0:
00042             HAL_REG_SYS_UNLOCK();
00043             HAL_REG_SET(SYS_CLK_FEN, SYS_CLK_FEN_QEP0_CLK_EN_BIT);
00044             HAL_REG_SYS_LOCK();
00045             HAL_TIMER_Enable (HAL_TIMER_2); // qep0使用时必须使能TIMER2
00046             break;
00047         case HAL_QEP_1:
00048             HAL_REG_SYS_UNLOCK();
00049             HAL_REG_SET(SYS_CLK_FEN, SYS_CLK_FEN_QEP1_CLK_EN_BIT);
00050             HAL_REG_SYS_LOCK();
00051             HAL_TIMER_Enable (HAL_TIMER_3); // qep1使用时必须使能TIMER3
00052             break;
00053         default:
00054             break;
00055     }
00056     HAL_REG_SET (HAL_QEP[qep]->CFG, QEP0_CFG_EN_BIT);
00057 }
00058
00063 void HAL_QEP_Disable (HAL_QEP_x qep)
00064 {
00065     switch (qep)
00066     {
00067         case HAL_QEP_0:
00068             HAL_REG_SYS_UNLOCK();
00069             HAL_REG_RESET (SYS_CLK_FEN, SYS_CLK_FEN_QEP0_CLK_EN_BIT);
00070             HAL_REG_SYS_LOCK();
00071             break;
00072         case HAL_QEP_1:
00073             HAL_REG_SYS_UNLOCK();
00074             HAL_REG_RESET (SYS_CLK_FEN, SYS_CLK_FEN_QEP1_CLK_EN_BIT);
00075             HAL_REG_SYS_LOCK();
00076             break;
00077         default:
00078             break;

```

```

00079     }
00080     HAL_REG_RESET(HAL_QEP[qep]->CFG, QEP0_CFG_EN_BIT);
00081 }
00082
00088 void HAL_QEP_SetTh(HAL_QEP_x qep, uint32_t th)
00089 {
00090     HAL_QEP[qep]->TH = th;
00091 }
00092
00098 void HAL_QEP_SetMode(HAL_QEP_x qep, HAL_QEP_MODE_x mode)
00099 {
00100     switch (mode)
00101     {
00102         case HAL_QEP_MODE_QUADRATURE:
00103             HAL_REG_WRITE_BIT_MASK_POS(HAL_QEP[qep]->CFG, QEP0_CFG_MODE, 1);
00104             break;
00105         case HAL_QEP_MODE_SYMBOL_PULSE:
00106             HAL_REG_WRITE_BIT_MASK_POS(HAL_QEP[qep]->CFG, QEP0_CFG_MODE, 2);
00107             break;
00108         case HAL_QEP_MODE_DUAL_PULSE:
00109             HAL_REG_WRITE_BIT_MASK_POS(HAL_QEP[qep]->CFG, QEP0_CFG_MODE, 3);
00110             break;
00111         default:
00112             break;
00113     }
00114 }
00115
00121 uint32_t HAL_QEP_ReadCnt(HAL_QEP_x qep)
00122 {
00123     return HAL_QEP[qep]->CNT;
00124 }
00125
00130 void HAL_QEP_ResetCnt(HAL_QEP_x qep)
00131 {
00132     HAL_QEP[qep]->CNT = 0;
00133 }
00134
00140 void HAL_QEP_SetGpio(HAL_QEP_x qepx, HAL_GPIO_Px px)
00141 {
00142     (void)qepx;
00143     HAL_GPIO_SetMode(px, HAL_GPIO_MODE_IN);
00144     HAL_GPIO_SetAf(px, HAL_GPIO_AF_QEP);
00145 }

```

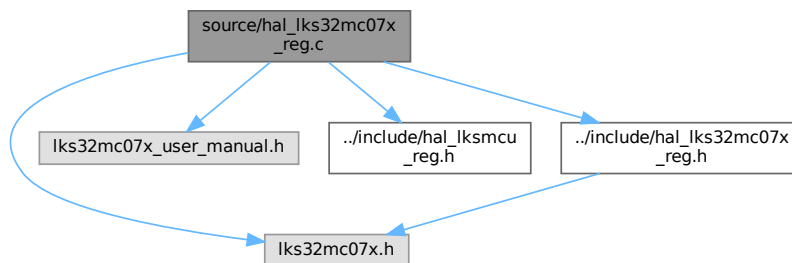
4.100 source/hal_lks32mc07x_reg.c 文件参考

```

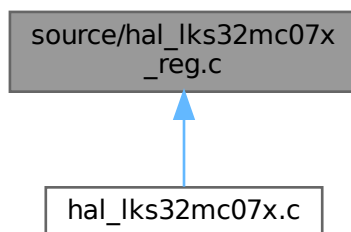
#include "lks32mc07x.h"
#include "lks32mc07x_user_manual.h"
#include "../include/hal_lksmcu_reg.h"
#include "../include/hal_lks32mc07x_reg.h"

```

hal_lks32mc07x_reg.c 的引用(Include)关系图:



此图展示该文件直接或间接的被哪些文件引用了：



变量

- `HAL_SYS_TypeDef *const HAL_SYS [] = {(HAL_SYS_TypeDef *)SYS_BASE}`
- `HAL_FLASH_TypeDef *const HAL_FLASH [] = {(HAL_FLASH_TypeDef *)FLASH_BASE}`
- `HAL_SPI_TypeDef *const HAL_SPI [] = {(HAL_SPI_TypeDef *)SPI0_BASE}`
- `HAL_I2C_TypeDef *const HAL_I2C [] = {(HAL_I2C_TypeDef *)I2C0_BASE}`
- `HAL_CMP_TypeDef *const HAL_CMP [] = {(HAL_CMP_TypeDef *)CMP_BASE}`
- `HAL_HALL_TypeDef *const HAL_HALL [] = {(HAL_HALL_TypeDef *)HALL0_BASE}`
- `HAL_ADC_TypeDef *const HAL_ADC [] = {(HAL_ADC_TypeDef *)ADC0_BASE, (HAL_ADC_TypeDef *)ADC1_BASE}`
- `HAL_TIMER_TypeDef *const HAL_TIMER [] = {(HAL_TIMER_TypeDef *)TIMER0_BASE, (HAL_TIMER_TypeDef *)TIMER1_BASE, (HAL_TIMER_TypeDef *)TIMER2_BASE, (HAL_TIMER_TypeDef *)TIMER3_BASE}`
- `HAL_QEP_TypeDef *const HAL_QEP [] = {(HAL_QEP_TypeDef *)QEP0_BASE, (HAL_QEP_TypeDef *)QEP1_BASE}`
- `HAL_MCPWM_TypeDef *const HAL_MCPWM [] = {(HAL_MCPWM_TypeDef *)MCPWM0_BASE}`
- `HAL_GPIO_TypeDef *const HAL_GPIO [] = {(HAL_GPIO_TypeDef *)GPIO0_BASE, (HAL_GPIO_TypeDef *)GPIO1_BASE, (HAL_GPIO_TypeDef *)GPIO2_BASE, (HAL_GPIO_TypeDef *)GPIO3_BASE}`
- `HAL_CRC_TypeDef *const HAL_CRC [] = {(HAL_CRC_TypeDef *)CRC0_BASE}`
- `HAL_UART_TypeDef *const HAL_UART [] = {(HAL_UART_TypeDef *)UART0_BASE, (HAL_UART_TypeDef *)UART1_BASE}`
- `HAL_DSP_TypeDef *const HAL_DSP [] = {(HAL_DSP_TypeDef *)DSP0_BASE}`
- `HAL_IWDG_TypeDef *const HAL_IWDG [] = {(HAL_IWDG_TypeDef *)IWDG_BASE}`
- `HAL_DMA_TypeDef *const HAL_DMA [] = {(HAL_DMA_TypeDef *) (DMA0_BASE + 0), (HAL_DMA_TypeDef *) (DMA0_BASE + 0x20), (HAL_DMA_TypeDef *) (DMA0_BASE + 0x40), (HAL_DMA_TypeDef *) (DMA0_BASE + 0x60)}`

4.100.1 变量说明

4.100.1.1 HAL_ADC

`HAL_ADC_TypeDef* const HAL_ADC[] = {(HAL_ADC_TypeDef *)ADC0_BASE, (HAL_ADC_TypeDef *)ADC1_BASE}`

在文件 `hal_lks32mc07x_reg.c` 第 12 行定义。

4.100.1.2 HAL_CMP

```
HAL_CMP_TypeDef* const HAL_CMP[] = {(HAL_CMP_TypeDef *)CMP_BASE}
```

在文件 [hal_lks32mc07x_reg.c](#) 第 10 行定义.

4.100.1.3 HAL_CRC

```
HAL_CRC_TypeDef* const HAL_CRC[] = {(HAL_CRC_TypeDef *)CRC0_BASE}
```

在文件 [hal_lks32mc07x_reg.c](#) 第 17 行定义.

4.100.1.4 HAL_DMA

```
HAL_DMA_TypeDef* const HAL_DMA[] = {(HAL_DMA_TypeDef *) (DMA0_BASE + 0), (HAL_DMA_TypeDef *) (DMA0←  
_BASE + 0x20), (HAL_DMA_TypeDef *) (DMA0_BASE + 0x40), (HAL_DMA_TypeDef *) (DMA0_BASE + 0x60)}
```

在文件 [hal_lks32mc07x_reg.c](#) 第 21 行定义.

4.100.1.5 HAL_DSP

```
HAL_DSP_TypeDef* const HAL_DSP[] = {(HAL_DSP_TypeDef *)DSP0_BASE}
```

在文件 [hal_lks32mc07x_reg.c](#) 第 19 行定义.

4.100.1.6 HAL_FLASH

```
HAL_FLASH_TypeDef* const HAL_FLASH[] = {(HAL_FLASH_TypeDef *)FLASH_BASE}
```

在文件 [hal_lks32mc07x_reg.c](#) 第 7 行定义.

4.100.1.7 HAL_GPIO

```
HAL_GPIO_TypeDef* const HAL_GPIO[] = {(HAL_GPIO_TypeDef *)GPIO0_BASE, (HAL_GPIO_TypeDef *)GPIO1←  
_BASE, (HAL_GPIO_TypeDef *)GPIO2_BASE, (HAL_GPIO_TypeDef *)GPIO3_BASE}
```

在文件 [hal_lks32mc07x_reg.c](#) 第 16 行定义.

4.100.1.8 HAL_HALL

```
HAL_HALL_TypeDef* const HAL_HALL[] = {(HAL_HALL_TypeDef *)HALL0_BASE}
```

在文件 [hal_lks32mc07x_reg.c](#) 第 11 行定义.

4.100.1.9 HAL_I2C

```
HAL_I2C_TypeDef* const HAL_I2C[] = {(HAL_I2C_TypeDef *)I2C0_BASE}
```

在文件 [hal_lks32mc07x_reg.c](#) 第 9 行定义.

4.100.1.10 HAL_IWDG

```
HAL_IWDG_TypeDef* const HAL_IWDG[] = {(HAL_IWDG_TypeDef *)IWDG_BASE}
```

在文件 [hal_lks32mc07x_reg.c](#) 第 20 行定义.

4.100.1.11 HAL_MCPWM

```
HAL_MCPWM_TypeDef* const HAL_MCPWM[] = {(HAL_MCPWM_TypeDef *)MCPWM0_BASE}
```

在文件 [hal_lks32mc07x_reg.c](#) 第 15 行定义.

4.100.1.12 HAL_QEP

```
HAL_QEP_TypeDef* const HAL_QEP[] = {(HAL_QEP_TypeDef *)QEP0_BASE, (HAL_QEP_TypeDef *)QEP1_BASE}
```

在文件 [hal_lks32mc07x_reg.c](#) 第 14 行定义.

4.100.1.13 HAL_SPI

```
HAL_SPI_TypeDef* const HAL_SPI[] = {(HAL_SPI_TypeDef *)SPI0_BASE}
```

在文件 [hal_lks32mc07x_reg.c](#) 第 8 行定义.

4.100.1.14 HAL_SYS

```
HAL_SYS_TypeDef* const HAL_SYS[] = {(HAL_SYS_TypeDef *)SYS_BASE}
```

在文件 [hal_lks32mc07x_reg.c](#) 第 6 行定义.

4.100.1.15 HAL_TIMER

```
HAL_TIMER_TypeDef* const HAL_TIMER[] = {(HAL_TIMER_TypeDef *)TIMER0_BASE, (HAL_TIMER_TypeDef *)TIMER1_BASE, (HAL_TIMER_TypeDef *)TIMER2_BASE, (HAL_TIMER_TypeDef *)TIMER3_BASE}
```

在文件 [hal_lks32mc07x_reg.c](#) 第 13 行定义.

4.100.1.16 HAL_UART

```
HAL_UART_TypeDef* const HAL_UART[] = {(HAL_UART_TypeDef *)UART0_BASE, (HAL_UART_TypeDef *)UART1_←
BASE}
```

在文件 `hal_lks32mc07x_reg.c` 第 18 行定义。

4.101 hal_lks32mc07x_reg.c

浏览该文件的文档。

```
00001 #include "lks32mc07x.h"
00002 #include "lks32mc07x.user_manual.h"
00003 #include "../include/hal_lksmcu_reg.h"
00004 #include "../include/hal_lks32mc07x_reg.h"
00005
00006 HAL_SYS_TypeDef *const HAL_SYS[] = {(HAL_SYS_TypeDef *)SYS_BASE};
00007 HAL_FLASH_TypeDef *const HAL_FLASH[] = {(HAL_FLASH_TypeDef *)FLASH_BASE};
00008 HAL_SPI_TypeDef *const HAL_SPI[] = {(HAL_SPI_TypeDef *)SPI0_BASE};
00009 HAL_I2C_TypeDef *const HAL_I2C[] = {(HAL_I2C_TypeDef *)I2C0_BASE};
00010 HAL_CMP_TypeDef *const HAL_CMP[] = {(HAL_CMP_TypeDef *)CMP_BASE};
00011 HAL_HALL_TypeDef *const HAL_HALL[] = {(HAL_HALL_TypeDef *)HALL0_BASE};
00012 HAL_ADC_TypeDef *const HAL_ADC[] = {(HAL_ADC_TypeDef *)ADC0_BASE, (HAL_ADC_TypeDef *)ADC1_BASE};
00013 HAL_TIMER_TypeDef *const HAL_TIMER[] = {(HAL_TIMER_TypeDef *)TIMER0_BASE, (HAL_TIMER_TypeDef
*)TIMER1_BASE, (HAL_TIMER_TypeDef *)TIMER2_BASE, (HAL_TIMER_TypeDef *)TIMER3_BASE};
00014 HAL_QEP_TypeDef *const HAL_QEP[] = {(HAL_QEP_TypeDef *)QEP0_BASE, (HAL_QEP_TypeDef *)QEP1_BASE};
00015 HAL_MCPWM_TypeDef *const HAL_MCPWM[] = {(HAL_MCPWM_TypeDef *)MCPWM0_BASE};
00016 HAL_GPIO_TypeDef *const HAL_GPIO[] = {(HAL_GPIO_TypeDef *)GPIO0_BASE, (HAL_GPIO_TypeDef *)GPIO1_BASE,
(HAL_GPIO_TypeDef *)GPIO2_BASE, (HAL_GPIO_TypeDef *)GPIO3_BASE};
00017 HAL_CRC_TypeDef *const HAL_CRC[] = {(HAL_CRC_TypeDef *)CRC0_BASE};
00018 HAL_UART_TypeDef *const HAL_UART[] = {(HAL_UART_TypeDef *)UART0_BASE, (HAL_UART_TypeDef *)UART1_BASE};
00019 HAL_DSP_TypeDef *const HAL_DSP[] = {(HAL_DSP_TypeDef *)DSP0_BASE};
00020 HAL_IWDG_TypeDef *const HAL_IWDG[] = {(HAL_IWDG_TypeDef *)IWDG_BASE};
00021 HAL_DMA_TypeDef *const HAL_DMA[] = {(HAL_DMA_TypeDef *) (DMA0_BASE + 0), (HAL_DMA_TypeDef *) (DMA0_BASE
+ 0x20), (HAL_DMA_TypeDef *) (DMA0_BASE + 0x40), (HAL_DMA_TypeDef *) (DMA0_BASE + 0x60)};
```

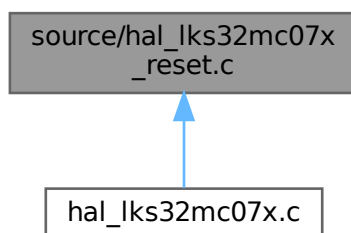
4.102 source/hal_lks32mc07x_reset.c 文件参考

```
#include "../include/hal_lksmcu_reset.h"
#include "../hal_lks32mc07x.h"
#include "lks32mc07x.user_manual.h"
#include "../include/hal_lksmcu_reg.h"
#include "../include/hal_lks32mc07x_reg.h"
```

hal_lks32mc07x_reset.c 的引用(Include)关系图:



此图展示该文件直接或间接的被哪些文件引用了:



函数

- void **HAL_RESET_ResetCore** ()
复位内核
- void **HAL_RESET_ResetRegister** ()
寄存器复位
- void **HAL_RESET_ResetGlobal** ()
全局复位(先复位寄存器再复位内核)

4.102.1 函数说明

4.102.1.1 HAL_RESET_ResetCore()

```
void HAL_RESET_ResetCore (  
    void )
```

复位内核

在文件 [hal_lks32mc07x.reset.c](#) 第 10 行定义.

4.102.1.2 HAL_RESET_ResetGlobal()

```
void HAL_RESET_ResetGlobal (  
    void )
```

全局复位(先复位寄存器再复位内核)

在文件 [hal_lks32mc07x.reset.c](#) 第 29 行定义.

函数调用图:



4.102.1.3 HAL_RESET_ResetRegister()

```
void HAL_RESET_ResetRegister (
    void )
```

寄存器复位

在文件 [hal_lks32mc07x.reset.c](#) 第 17 行定义.

这是这个函数的调用关系图:



4.103 hal_lks32mc07x.reset.c

[浏览该文件的文档.](#)

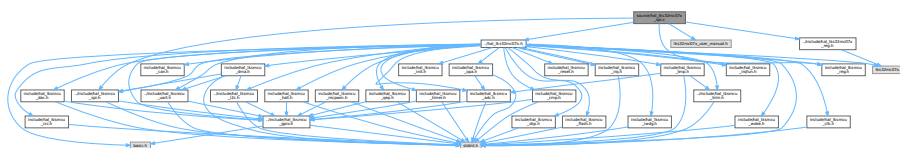
```

00001
00002 #include "../include/hal_lksmcu.reset.h"
00003 #include "../hal_lks32mc07x.h"
00004 #include "lks32mc07x.user_manual.h"
00005 #include "../include/hal_lksmcu.reg.h"
00006 #include "../include/hal_lks32mc07x.reg.h"
00010 void HAL_RESET_ResetCore()
00011 {
00012     NVIC.SystemReset();
00013 }
00017 void HAL_RESET_ResetRegister()
00018 {
00019     // 系统寄存器解锁
00020     HAL_REG_SYS_UNLOCK();
00021     HAL_REG_SET(SYS_SFT_RST, 0xffff);
00022     HAL_REG_RESET(SYS_SFT_RST, 0);
00023     // 系统寄存器上锁
00024     HAL_REG_SYS_LOCK();
00025 }
00029 void HAL_RESET_ResetGlobal()
00030 {
00031     HAL_RESET_ResetRegister();
00032     NVIC.SystemReset();
00033 }
```

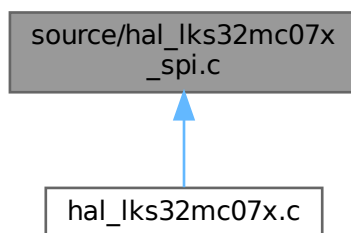
4.104 source/hal_lks32mc07x_spi.c 文件参考

```

#include "../include/hal_lksmcu.spi.h"
#include "../hal_lks32mc07x.h"
#include "lks32mc07x.user_manual.h"
#include "../include/hal_lksmcu.reg.h"
#include "../include/hal_lks32mc07x.reg.h"
hal_lks32mc07x_spi.c 的引用(Include)关系图:
```



此图展示该文件直接或间接的被哪些文件引用了：



函数

- void **HAL_SPI_Reset** (HAL_SPI_x spi)
模块复位
- void **HAL_SPI_Enable** (HAL_SPI_x spi)
模块打开
- void **HAL_SPI_Disable** (HAL_SPI_x spi)
模块关闭
- void **HAL_SPI_SetBaud** (HAL_SPI_x spi, uint32_t baud)
设置波特率
- void **HAL_SPI_SetMode** (HAL_SPI_x spi, HAL_SPI_MODE_x mode)
设置SPI模式
- void **HAL_SPI_SetGpioCs** (HAL_SPI_x spi, HAL_GPIO_Px pin)
初始化CS引脚
- void **HAL_SPI_SetGpioMosi** (HAL_SPI_x spi, HAL_GPIO_Px pin)
初始化Mosi引脚
- void **HAL_SPI_SetGpioMiso** (HAL_SPI_x spi, HAL_GPIO_Px pin)
初始化Miso引脚
- void **HAL_SPI_SetGpioClk** (HAL_SPI_x spi, HAL_GPIO_Px pin)
初始化Clk引脚
- void **HAL_SPI_SetGpioDi** (HAL_SPI_x spi, HAL_GPIO_Px pin)
初始化SPI的输入引脚
- void **HAL_SPI_SetGpioDo** (HAL_SPI_x spi, HAL_GPIO_Px pin)
初始化SPI的输出引脚
- uint8_t **HAL_SPI_TxData** (HAL_SPI_x spi, uint8_t *data, uint32_t len)
发送数据
- uint8_t **HAL_SPI_RxData** (HAL_SPI_x spi, uint8_t *data, uint32_t len)
接收数据
- uint8_t **HAL_SPI_RxTxData** (HAL_SPI_x spi, uint8_t *rxdata, uint8_t *txdata, uint32_t len)
发送和接收数据
- uint8_t **HAL_SPI_TxDataWidth16** (HAL_SPI_x spi, uint16_t *data, uint32_t len)
发送数据
- uint8_t **HAL_SPI_RxDataWidth16** (HAL_SPI_x spi, uint16_t *data, uint32_t len)
接收数据
- uint8_t **HAL_SPI_RxTxDataWidth16** (HAL_SPI_x spi, uint16_t *rxdata, uint16_t *txdata, uint32_t len)
发送和接收数据

4.104.1 函数说明

4.104.1.1 HAL_SPI_Disable()

```
void HAL_SPI_Disable (
    HAL_SPI_TypeDef* spi )
```

模块关闭

参数

<i>spi</i>	SPI模块号
------------	--------

在文件 [hal_lks32mc07x_spi.c](#) 第 37 行定义.

4.104.1.2 HAL_SPI_Enable()

```
void HAL_SPI_Enable (
    HAL_SPI_TypeDef* spi )
```

模块打开

参数

<i>spi</i>	SPI模块号
------------	--------

在文件 [hal_lks32mc07x_spi.c](#) 第 25 行定义.

4.104.1.3 HAL_SPI_Reset()

```
void HAL_SPI_Reset (
    HAL_SPI_TypeDef* spi )
```

模块复位

参数

<i>spi</i>	SPI模块号
------------	--------

在文件 [hal_lks32mc07x_spi.c](#) 第 12 行定义.

4.104.1.4 HAL_SPI_RxData()

```
uint8_t HAL_SPI_RxData (
    HAL_SPI_TypeDef* spi,
```

```
uint8_t * data,  
uint32_t len )
```

接收数据

参数

<i>spi</i>	SPI模块号
<i>data</i>	接收缓冲区
<i>len</i>	数据长度

返回

接收状态

在文件 [hal_lks32mc07x_spi.c](#) 第 255 行定义.

4.104.1.5 HAL_SPI_RxDataWidth16()

```
uint8_t HAL_SPI_RxDataWidth16 (
    HAL_SPI_TypeDef * spi,
    uint16_t * data,
    uint32_t len )
```

接收数据

参数

<i>spi</i>	SPI模块号
<i>data</i>	接收缓冲区
<i>len</i>	数据长度

返回

接收状态

在文件 [hal_lks32mc07x_spi.c](#) 第 330 行定义.

4.104.1.6 HAL_SPI_RxTxData()

```
uint8_t HAL_SPI_RxTxData (
    HAL_SPI_TypeDef * spi,
    uint8_t * rxdata,
    uint8_t * txdata,
    uint32_t len )
```

发送和接收数据

参数

<i>spi</i>	SPI模块号
<i>rxdata</i>	接收缓冲区
<i>txdata</i>	要发送的数据

返回

发送和接收状态

在文件 [hal_lks32mc07x_spi.c](#) 第 282 行定义.

4.104.1.7 HAL_SPI_RxTxDataWidth16()

```
uint8_t HAL_SPI_RxTxDataWidth16 (
    HAL_SPI_TypeDef * spi,
    uint16_t * rxdata,
    uint16_t * txdata,
    uint32_t len )
```

发送和接收数据

参数

<i>spi</i>	SPI模块号
<i>rxdata</i>	接收缓冲区
<i>txdata</i>	要发送的数据
<i>len</i>	数据长度

返回

发送和接收状态

在文件 [hal_lks32mc07x_spi.c](#) 第 355 行定义.

4.104.1.8 HAL_SPI_SetBaud()

```
void HAL_SPI_SetBaud (
    HAL_SPI_TypeDef * spi,
    uint32_t baud )
```

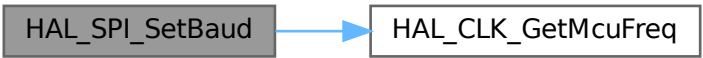
设置波特率

参数

<i>spi</i>	SPI模块号
------------	--------

在文件 [hal_lks32mc07x_spi.c](#) 第 49 行定义.

函数调用图:



4.104.1.9 HAL_SPI_SetGpioClk()

```
void HAL_SPI_SetGpioClk (
    HAL_SPI_TypeDef* spi,
    GPIO_TypeDef* pin )
```

初始化Clk引脚

参数

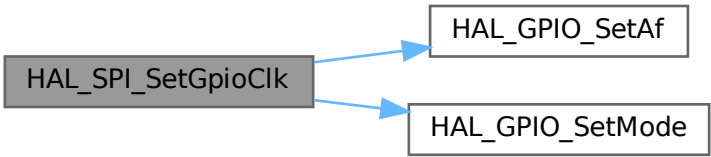
spi	SPI模块号
pin	GPIO引脚

参见

需要先初始化SPI

在文件 [hal_lks32mc07x_spi.c](#) 第 186 行定义.

函数调用图:



4.104.1.10 HAL_SPI_SetGpioCs()

```
void HAL_SPI_SetGpioCs (
    HAL_SPI_TypeDef* spi,
    GPIO_TypeDef* pin )
```

初始化CS引脚

参数

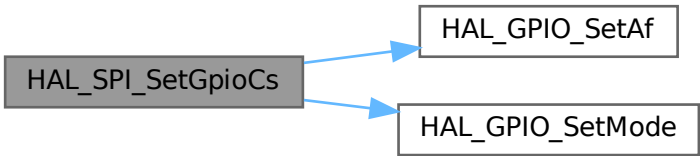
<i>spi</i>	SPI模块号
<i>pin</i>	GPIO引脚

参见

需要先初始化SPI

在文件 [hal_lks32mc07x_spi.c](#) 第 123 行定义.

函数调用图:



4.104.1.11 HAL_SPI_SetGpioDi()

```
void HAL_SPI_SetGpioDi (
    HAL_SPI_x spi,
    HAL_GPIO_Px pin )
```

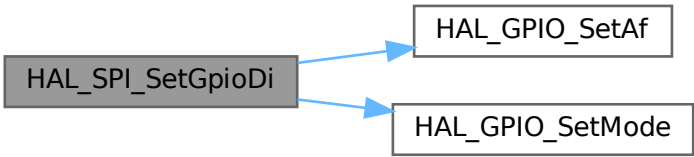
初始化SPI的输入引脚

参数

<i>spi</i>	SPI模块号
<i>pin</i>	GPIO引脚

在文件 [hal_lks32mc07x_spi.c](#) 第 206 行定义.

函数调用图:



4.104.1.12 HAL_SPI_SetGpioDo()

```
void HAL_SPI_SetGpioDo (
    HAL_SPI_x spi,
    HAL_GPIO_Px pin )
```

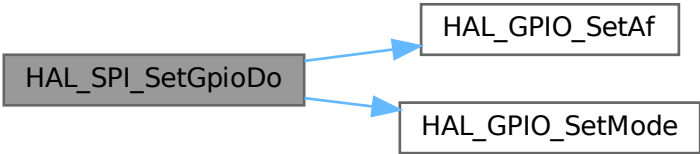
初始化SPI的输出引脚

参数

<i>spi</i>	SPI模块号
<i>pin</i>	GPIO引脚

在文件 [hal_lks32mc07x_spi.c](#) 第 217 行定义.

函数调用图:



4.104.1.13 HAL_SPI_SetGpioMiso()

```
void HAL_SPI_SetGpioMiso (
    HAL_SPI_x spi,
    HAL_GPIO_Px pin )
```

初始化Miso引脚

参数

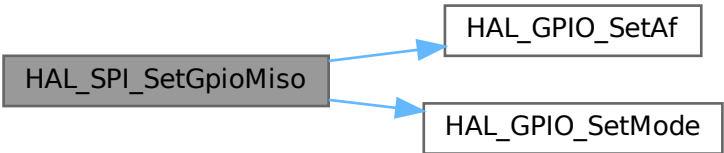
<i>spi</i>	SPI模块号
<i>pin</i>	GPIO引脚

参见

需要先初始化SPI

在文件 [hal_lks32mc07x_spi.c](#) 第 165 行定义.

函数调用图:



4.104.1.14 HAL_SPI.SetGpioMosi()

```
void HAL_SPI_SetGpioMosi (
    HAL_SPI_x spi,
    HAL_GPIO_Px pin )
```

初始化Mosi引脚

参数

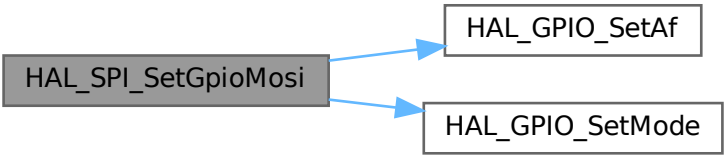
<i>spi</i>	SPI模块号
<i>pin</i>	GPIO引脚

参见

需要先初始化SPI

在文件 [hal_lks32mc07x_spi.c](#) 第 144 行定义.

函数调用图:



4.104.1.15 HAL_SPI_SetMode()

```
void HAL_SPI_SetMode (
    HAL_SPI_x spi,
    HAL_SPI_MODE_x mode )
```

设置SPI模式

参数

<i>spi</i>	SPI模块号
<i>mode</i>	SPI模式

在文件 [hal_lks32mc07x_spi.c](#) 第 60 行定义.

4.104.1.16 HAL_SPI_TxData()

```
uint8_t HAL_SPI_TxData (
    HAL_SPI_x spi,
    uint8_t * data,
    uint32_t len )
```

发送数据

参数

<i>spi</i>	SPI模块号
<i>data</i>	要发送的数据
<i>len</i>	数据长度

返回

发送状态

在文件 [hal_lks32mc07x_spi.c](#) 第 231 行定义.

4.104.1.17 HAL_SPI_TxDataWidth16()

```
uint8_t HAL_SPI_TxDataWidth16 (
    HAL_SPI_TypeDef* spi,
    uint16_t * data,
    uint32_t len )
```

发送数据

参数

<i>spi</i>	SPI模块号
<i>data</i>	要发送的数据
<i>len</i>	数据长度

返回

发送状态

在文件 [hal_lks32mc07x_spi.c](#) 第 307 行定义.

4.105 hal_lks32mc07x_spi.c

[浏览该文件的文档.](#)

```
00001
00002 #include "../include/hal_lksmcu_spi.h"
00003 #include "../hal_lks32mc07x.h"
00004 #include "lks32mc07x_user_manual.h"
00005 #include "../include/hal_lksmcu_reg.h"
00006 #include "../include/hal_lks32mc07x_reg.h"
00007
00012 void HAL_SPI_Reset(HAL_SPI_TypeDef* spi)
00013 {
00014     (void)spi;
00015     HAL_REG_SYS_UNLOCK();
00016     HAL_REG_SET(SYS_SFT_RST, SYS_SFT_RST_SPI0_SFT_RST_BIT);
00017     HAL_REG_RESET(SYS_SFT_RST, SYS_SFT_RST_SPI0_SFT_RST_BIT);
00018     HAL_REG_SYS_LOCK();
00019 }
00020
00025 void HAL_SPI_Enable(HAL_SPI_TypeDef* spi)
00026 {
00027     HAL_REG_SYS_UNLOCK();
00028     HAL_REG_SET(SYS_CLK_FEN, SYS_CLK_FEN_SPI0_CLK_EN_BIT);
00029     HAL_REG_SYS_LOCK();
00030     HAL_REG_SET(HAL_SPI[spi]->CFG, SPI0_CFG_EN_BIT);
00031 }
00032
00037 void HAL_SPI_Disable(HAL_SPI_TypeDef* spi)
00038 {
00039     HAL_REG_RESET(HAL_SPI[spi]->CFG, SPI0_CFG_EN_BIT);
00040     HAL_REG_SYS_UNLOCK();
00041     HAL_REG_RESET(SYS_CLK_FEN, SYS_CLK_FEN_SPI0_CLK_EN_BIT);
00042     HAL_REG_SYS_LOCK();
00043 }
00044
00049 void HAL_SPI_SetBaud(HAL_SPI_TypeDef* spi, uint32_t baud)
00050 {
00051     uint32_t mclk = HAL_CLK_GetMcuFreq();
00052     uint32_t cnt = (mclk / (2 * baud)) - 1;
00053     HAL_REG_WRITE_BIT_MASK_POS(HAL_SPI[spi]->BAUD, SPI0_BAUD_BAUD, cnt);
00054 }
00060 void HAL_SPI_SetMode(HAL_SPI_TypeDef* spi, HAL_SPI_Mode_TypeDef mode)
00061 {
00062     uint32_t mask = SPI0_CFG_MS_BIT | SPI0_CFG_CPHA_BIT | SPI0_CFG_CPOL_BIT | SPI0_CFG_ENDIAN_BIT;
00063     switch (mode)
```

```

00064     {
00065         case HAL_SPI_MODE_MASTER_LSB_00:
00066             HAL_REG_WRITE_BIT(HAL_SPI[spi]->CFG, mask, SPI0_CFG_MS_BIT | SPI0_CFG_ENDIAN_BIT);
00067             break;
00068         case HAL_SPI_MODE_MASTER_LSB_01:
00069             HAL_REG_WRITE_BIT(HAL_SPI[spi]->CFG, mask, SPI0_CFG_MS_BIT | SPI0_CFG_CPOL_BIT |
SPI0_CFG_ENDIAN_BIT);
00070             break;
00071         case HAL_SPI_MODE_MASTER_LSB_10:
00072             HAL_REG_WRITE_BIT(HAL_SPI[spi]->CFG, mask, SPI0_CFG_MS_BIT | SPI0_CFG_CPHA_BIT |
SPI0_CFG_ENDIAN_BIT);
00073             break;
00074         case HAL_SPI_MODE_MASTER_LSB_11:
00075             HAL_REG_WRITE_BIT(HAL_SPI[spi]->CFG, mask, SPI0_CFG_MS_BIT | SPI0_CFG_CPHA_BIT |
SPI0_CFG_CPOL_BIT | SPI0_CFG_ENDIAN_BIT);
00076             break;
00077         case HAL_SPI_MODE_MASTER_MSB_00:
00078             HAL_REG_WRITE_BIT(HAL_SPI[spi]->CFG, mask, SPI0_CFG_MS_BIT | 0);
00079             break;
00080         case HAL_SPI_MODE_MASTER_MSB_01:
00081             HAL_REG_WRITE_BIT(HAL_SPI[spi]->CFG, mask, SPI0_CFG_MS_BIT | SPI0_CFG_CPOL_BIT);
00082             break;
00083         case HAL_SPI_MODE_MASTER_MSB_10:
00084             HAL_REG_WRITE_BIT(HAL_SPI[spi]->CFG, mask, SPI0_CFG_MS_BIT | SPI0_CFG_CPHA_BIT);
00085             break;
00086         case HAL_SPI_MODE_MASTER_MSB_11:
00087             HAL_REG_WRITE_BIT(HAL_SPI[spi]->CFG, mask, SPI0_CFG_MS_BIT | SPI0_CFG_CPHA_BIT |
SPI0_CFG_CPOL_BIT);
00088             break;
00089         case HAL_SPI_MODE_SLAVE_LSB_00:
00090             HAL_REG_WRITE_BIT(HAL_SPI[spi]->CFG, mask, SPI0_CFG_ENDIAN_BIT);
00091             break;
00092         case HAL_SPI_MODE_SLAVE_LSB_01:
00093             HAL_REG_WRITE_BIT(HAL_SPI[spi]->CFG, mask, SPI0_CFG_CPOL_BIT | SPI0_CFG_ENDIAN_BIT);
00094             break;
00095         case HAL_SPI_MODE_SLAVE_LSB_10:
00096             HAL_REG_WRITE_BIT(HAL_SPI[spi]->CFG, mask, SPI0_CFG_CPHA_BIT | SPI0_CFG_ENDIAN_BIT);
00097             break;
00098         case HAL_SPI_MODE_SLAVE_LSB_11:
00099             HAL_REG_WRITE_BIT(HAL_SPI[spi]->CFG, mask, SPI0_CFG_CPHA_BIT | SPI0_CFG_CPOL_BIT |
SPI0_CFG_ENDIAN_BIT);
00100             break;
00101         case HAL_SPI_MODE_SLAVE_MSB_00:
00102             HAL_REG_WRITE_BIT(HAL_SPI[spi]->CFG, mask, 0);
00103             break;
00104         case HAL_SPI_MODE_SLAVE_MSB_01:
00105             HAL_REG_WRITE_BIT(HAL_SPI[spi]->CFG, mask, SPI0_CFG_CPOL_BIT);
00106             break;
00107         case HAL_SPI_MODE_SLAVE_MSB_10:
00108             HAL_REG_WRITE_BIT(HAL_SPI[spi]->CFG, mask, SPI0_CFG_CPHA_BIT);
00109             break;
00110         case HAL_SPI_MODE_SLAVE_MSB_11:
00111             HAL_REG_WRITE_BIT(HAL_SPI[spi]->CFG, mask, SPI0_CFG_CPHA_BIT | SPI0_CFG_CPOL_BIT);
00112             break;
00113         default:
00114             break;
00115     }
00116 }
00123 void HAL_SPI_SetGpioCs(HAL_SPI_x spi, HAL_GPIO_Px pin)
00124 {
00125     if (HAL_REG_READ_BIT(HAL_SPI[spi]->CFG, SPI0_CFG_MS_BIT) == 0)
00126     {
00127         // 从模式
00128         HAL_GPIO_SetAf(pin, HAL_GPIO_AF_SPI);
00129         HAL_GPIO_SetMode(pin, HAL_GPIO_MODE_IN);
00130     }
00131     else
00132     {
00133         // 主模式
00134         HAL_GPIO_SetAf(pin, HAL_GPIO_AF_SPI);
00135         HAL_GPIO_SetMode(pin, HAL_GPIO_MODE_OUT);
00136     }
00137 }
00144 void HAL_SPI_SetGpioMosi(HAL_SPI_x spi, HAL_GPIO_Px pin)
00145 {
00146     if (HAL_REG_READ_BIT(HAL_SPI[spi]->CFG, SPI0_CFG_MS_BIT) == 0)
00147     {
00148         // 从模式
00149         HAL_GPIO_SetAf(pin, HAL_GPIO_AF_SPI);
00150         HAL_GPIO_SetMode(pin, HAL_GPIO_MODE_IN);
00151     }
00152     else
00153     {
00154         // 主模式
00155         HAL_GPIO_SetAf(pin, HAL_GPIO_AF_SPI);
00156         HAL_GPIO_SetMode(pin, HAL_GPIO_MODE_OUT);
00157     }

```

```

00158 }
00165 void HAL_SPI_SetGpioMiso(HAL_SPI_TypeDef spi, HAL_GPIO_TypeDef pin)
00166 {
00167     if (HAL_REG_READ_BIT(HAL_SPI[spi] -> CFG, SPI0_CFG_MS_BIT) == 0)
00168     {
00169         // 从模式
00170         HAL_GPIO_SetAf(pin, HAL_GPIO_AF_SPI);
00171         HAL_GPIO_SetMode(pin, HAL_GPIO_MODE_OUT);
00172     }
00173     else
00174     {
00175         // 主模式
00176         HAL_GPIO_SetAf(pin, HAL_GPIO_AF_SPI);
00177         HAL_GPIO_SetMode(pin, HAL_GPIO_MODE_IN);
00178     }
00179 }
00186 void HAL_SPI_SetGpioClk(HAL_SPI_TypeDef spi, HAL_GPIO_TypeDef pin)
00187 {
00188     if (HAL_REG_READ_BIT(HAL_SPI[spi] -> CFG, SPI0_CFG_MS_BIT) == 0)
00189     {
00190         // 从模式
00191         HAL_GPIO_SetAf(pin, HAL_GPIO_AF_SPI);
00192         HAL_GPIO_SetMode(pin, HAL_GPIO_MODE_IN);
00193     }
00194     else
00195     {
00196         // 主模式
00197         HAL_GPIO_SetAf(pin, HAL_GPIO_AF_SPI);
00198         HAL_GPIO_SetMode(pin, HAL_GPIO_MODE_OUT);
00199     }
00200 }
00206 void HAL_SPI_SetGpioDi(HAL_SPI_TypeDef spi, HAL_GPIO_TypeDef pin)
00207 {
00208     (void) spi;
00209     HAL_GPIO_SetAf(pin, HAL_GPIO_AF_SPI);
00210     HAL_GPIO_SetMode(pin, HAL_GPIO_MODE_IN);
00211 }
00217 void HAL_SPI_SetGpioDo(HAL_SPI_TypeDef spi, HAL_GPIO_TypeDef pin)
00218 {
00219     (void) spi;
00220     HAL_GPIO_SetAf(pin, HAL_GPIO_AF_SPI);
00221     HAL_GPIO_SetMode(pin, HAL_GPIO_MODE_OUT);
00222 }
00223
00231 uint8_t HAL_SPI_TxData(HAL_SPI_TypeDef spi, uint8_t *data, uint32_t len)
00232 {
00233     uint32_t i = 0;
00234     HAL_REG_WRITE(HAL_SPI[spi] -> SIZE, 8);
00235     HAL_REG_SET(HAL_SPI[spi] -> BAUD, SPI0_BAUD_TRANS_MODE_BIT);
00236     HAL_REG_WRITE_BIT_MASK_POS(HAL_SPI[spi] -> CFG, SPI0_CFG_DUPLEX, 2);
00237     for (i = 0; i < len; i++)
00238     {
00239         HAL_REG_WRITE_BIT(HAL_SPI[spi] -> IE, 0x07, BIT2);
00240         HAL_REG_WRITE(HAL_SPI[spi] -> TXDATA, data[i]);
00241         while (HAL_REG_READ(HAL_SPI[spi] -> IE & BIT2) == 0)
00242         {
00243         }
00244     }
00245     return 0;
00246 }
00247
00255 uint8_t HAL_SPI_RxData(HAL_SPI_TypeDef spi, uint8_t *data, uint32_t len)
00256 {
00257     uint32_t i = 0;
00258     HAL_REG_WRITE(HAL_SPI[spi] -> SIZE, 8);
00259     HAL_REG_SET(HAL_SPI[spi] -> BAUD, SPI0_BAUD_TRANS_MODE_BIT);
00260     HAL_REG_WRITE_BIT_MASK_POS(HAL_SPI[spi] -> CFG, SPI0_CFG_DUPLEX, 3);
00261     for (i = 0; i < len; i++)
00262     {
00263         HAL_REG_WRITE_BIT(HAL_SPI[spi] -> IE, 0x07, BIT2);
00264         HAL_REG_WRITE(HAL_SPI[spi] -> TXDATA, 0); // 发送一个空字节, 以触发接收
00265         while (HAL_REG_READ(HAL_SPI[spi] -> IE & BIT2) == 0)
00266         {
00267         }
00268         data[i] = HAL_REG_READ(HAL_SPI[spi] -> RXDATA);
00269     }
00270     // 实现接收数据的代码
00271     return 0; // 返回接收状态
00272 }
00273
00282 uint8_t HAL_SPI_RxTxData(HAL_SPI_TypeDef spi, uint8_t *rxdata, uint8_t *txdata, uint32_t len)
00283 {
00284     uint32_t i = 0;
00285     HAL_REG_WRITE(HAL_SPI[spi] -> SIZE, 8);
00286     HAL_REG_SET(HAL_SPI[spi] -> BAUD, SPI0_BAUD_TRANS_MODE_BIT);
00287     HAL_REG_WRITE_BIT_MASK_POS(HAL_SPI[spi] -> CFG, SPI0_CFG_DUPLEX, 0);
00288     for (i = 0; i < len; i++)

```

```

00289     {
00290         HAL_REG_WRITE_BIT(HAL_SPI[spi]->IE, 0x07, BIT2);
00291         HAL_REG_WRITE(HAL_SPI[spi]->TXDATA, txdata[i]);
00292         while (HAL_REG_READ(HAL_SPI[spi]->IE & BIT2) == 0)
00293         {
00294         }
00295         rxdata[i] = HAL_REG_READ(HAL_SPI[spi]->RXDATA);
00296     }
00297     return 0; // 返回发送和接收状态
00298 }
00299
00300 uint8_t HAL_SPI_TxDataWidth16(HAL_SPI_x spi, uint16_t *data, uint32_t len)
00301 {
00302     uint32_t i = 0;
00303     HAL_REG_WRITE(HAL_SPI[spi]->SIZE, 16);
00304     HAL_REG_SET(HAL_SPI[spi]->BAUD, SPI0_BAUD_TRANS_MODE_BIT);
00305     HAL_REG_WRITE_BIT_MASK_POS(HAL_SPI[spi]->CFG, SPI0_CFG_DUPLEX, 2);
00306     for (i = 0; i < len; i++)
00307     {
00308         HAL_REG_WRITE_BIT(HAL_SPI[spi]->IE, 0x07, BIT2);
00309         HAL_REG_WRITE(HAL_SPI[spi]->TXDATA, data[i]);
00310         while (HAL_REG_READ(HAL_SPI[spi]->IE & BIT2) == 0)
00311         {
00312         }
00313     }
00314     return 0;
00315 }
00316
00317 uint8_t HAL_SPI_RxDataWidth16(HAL_SPI_x spi, uint16_t *data, uint32_t len)
00318 {
00319     uint32_t i = 0;
00320     HAL_REG_WRITE(HAL_SPI[spi]->SIZE, 16);
00321     HAL_REG_SET(HAL_SPI[spi]->BAUD, SPI0_BAUD_TRANS_MODE_BIT);
00322     HAL_REG_WRITE_BIT_MASK_POS(HAL_SPI[spi]->CFG, SPI0_CFG_DUPLEX, 3);
00323     for (i = 0; i < len; i++)
00324     {
00325         HAL_REG_WRITE_BIT(HAL_SPI[spi]->IE, 0x07, BIT2);
00326         HAL_REG_WRITE(HAL_SPI[spi]->TXDATA, 0); // 发送一个空字节,以触发接收
00327         while (HAL_REG_READ(HAL_SPI[spi]->IE & BIT2) == 0)
00328         {
00329         }
00330         data[i] = HAL_REG_READ(HAL_SPI[spi]->RXDATA);
00331     }
00332     return 0; // 返回接收状态
00333 }
00334
00335 uint8_t HAL_SPI_RxTxDataWidth16(HAL_SPI_x spi, uint16_t *rxdata, uint16_t *txdata, uint32_t len)
00336 {
00337     uint32_t i = 0;
00338     HAL_REG_WRITE(HAL_SPI[spi]->SIZE, 16);
00339     HAL_REG_SET(HAL_SPI[spi]->BAUD, SPI0_BAUD_TRANS_MODE_BIT);
00340     HAL_REG_WRITE_BIT_MASK_POS(HAL_SPI[spi]->CFG, SPI0_CFG_DUPLEX, 0);
00341     for (i = 0; i < len; i++)
00342     {
00343         HAL_REG_WRITE_BIT(HAL_SPI[spi]->IE, 0x07, BIT2);
00344         HAL_REG_WRITE(HAL_SPI[spi]->TXDATA, txdata[i]);
00345         while (HAL_REG_READ(HAL_SPI[spi]->IE & BIT2) == 0)
00346         {
00347         }
00348         rxdata[i] = HAL_REG_READ(HAL_SPI[spi]->RXDATA);
00349     }
00350     return 0; // 返回发送和接收状态
00351 }
00352 }

```

4.106 source/hal_lks32mc07x_test.c 文件参考

```

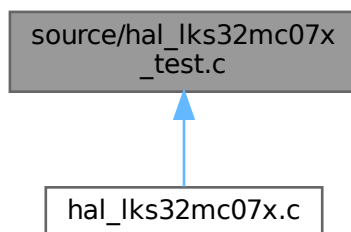
#include "../hal_lks32mc07x.h"
#include "lks32mc07x_user_manual.h"
#include "../include/hal_lksmcu_reg.h"
#include "../include/hal_lks32mc07x_reg.h"

```

hal_lks32mc07x_test.c 的引用(Include)关系图:



此图展示该文件直接或间接的被哪些文件引用了:



函数

- uint32_t [HAL_TestRegAddr](#) (void)

4.106.1 函数说明

4.106.1.1 HAL_TestRegAddr()

```
uint32_t HAL_TestRegAddr (  
    void )
```

在文件 [hal_lks32mc07x_test.c](#) 第 6 行定义.

4.107 hal_lks32mc07x_test.c

[浏览该文件的文档.](#)

```
00001  
00002 #include "../hal_lks32mc07x.h"  
00003 #include "lks32mc07x-user-manual.h"  
00004 #include "../include/hal_lksmcu-reg.h"  
00005 #include "../include/hal_lks32mc07x-reg.h"  
00006 uint32_t HAL_TestRegAddr(void)  
00007 {  
00008     return 0;  
00009 }
```

4.108 source/hal_lks32mc07x_timer.c 文件参考

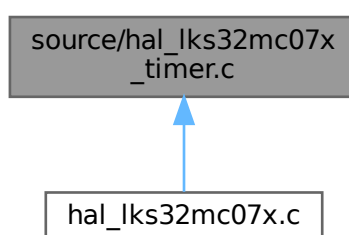
```
#include "../include/hal_lksmcu-timer.h"  
#include "../include/hal_lksmcu-gpio.h"  
#include "../hal_lks32mc07x.h"  
#include "lks32mc07x-user-manual.h"  
#include "../include/hal_lksmcu-reg.h"
```

```
#include "../include/hal_lks32mc07x_reg.h"
```

hal_lks32mc07x_timer.c 的引用(Include)关系图:



此图展示该文件直接或间接的被哪些文件引用了:



函数

- void [HAL_TIMER_Reset](#) (HAL_TIMER_x timer_x)
- void [HAL_TIMER_Enable](#) (HAL_TIMER_x timer_x)
- void [HAL_TIMER_Disable](#) (HAL_TIMER_x timer_x)
- void [HAL_TIMER_SetPeriodCount](#) (HAL_TIMER_x timer_x, uint32_t count)
设置定时器周期
- void [HAL_TIMER_SetFreq](#) (HAL_TIMER_x timer_x, uint32_t freq)
设置定时器频率
- void [HAL_TIMER_SetPeriodmS](#) (HAL_TIMER_x timer_x, uint32_t ms)
设置定时器周期(ms)
- void [HAL_TIMER_SetPerioduS](#) (HAL_TIMER_x timer_x, uint32_t us)
设置定时器周期(us)
- uint32_t [HAL_TIMER_GetFreq](#) (HAL_TIMER_x timer_x)
- uint32_t [HAL_TIMER_GetThCnt](#) (HAL_TIMER_x timer_x)
- void [HAL_TIMER_StartCounting](#) (HAL_TIMER_x timer_x)
- void [HAL_TIMER_StopCounting](#) (HAL_TIMER_x timer_x)
- void [HAL_TIMER_SetPwm](#) (HAL_TIMER_x timer_x, int32_t t0, int32_t t1)
- void [HAL_TIMER_SetPwmQ15](#) (HAL_TIMER_x timer_x, int16_t t0, int16_t t1)
- void [HAL_TIMER_SetWorkMode](#) (HAL_TIMER_x timer_x, HAL_TIMER_MODE_x mode_x)
- uint8_t [HAL_TIMER_IrqOverflow](#) (HAL_TIMER_x timer_x)
- void [HAL_TIMER_IrqOverflowEnable](#) (HAL_TIMER_x timer_x)
- void [HAL_TIMER_IrqOverflowDisable](#) (HAL_TIMER_x timer_x)
- void [HAL_TIMER_IrqOverflowClear](#) (HAL_TIMER_x timer_x)
- uint8_t [HAL_TIMER_IrqCompare0](#) (HAL_TIMER_x timer_x)
- void [HAL_TIMER_IrqCompare0Enable](#) (HAL_TIMER_x timer_x)

- void [HAL_TIMER_IrqCompare0Disable](#) ([HAL_TIMER_x](#) timer_x)
- void [HAL_TIMER_IrqCompare0Clear](#) ([HAL_TIMER_x](#) timer_x)
- uint8_t [HAL_TIMER_IrqCompare1](#) ([HAL_TIMER_x](#) timer_x)
- void [HAL_TIMER_IrqCompare1Enable](#) ([HAL_TIMER_x](#) timer_x)
- void [HAL_TIMER_IrqCompare1Disable](#) ([HAL_TIMER_x](#) timer_x)
- void [HAL_TIMER_IrqCompare1Clear](#) ([HAL_TIMER_x](#) timer_x)
- void [HAL_TIMER_SetGpioOut](#) ([HAL_TIMER_x](#) timer_x, [HAL_GPIO_Px](#) px)
设置定时器输出 *gpio*
- void [HAL_TIMER_SetGpioIn](#) ([HAL_TIMER_x](#) timer_x, [HAL_GPIO_Px](#) px)
设置定时器输入 *gpio*

4.108.1 函数说明

4.108.1.1 HAL_TIMER_Disable()

```
void HAL_TIMER_Disable (
    HAL_TIMER_x timer_x )
```

在文件 [hal_lks32mc07x.timer.c](#) 第 63 行定义.

4.108.1.2 HAL_TIMER_Enable()

```
void HAL_TIMER_Enable (
    HAL_TIMER_x timer_x )
```

在文件 [hal_lks32mc07x.timer.c](#) 第 38 行定义.

这是这个函数的调用关系图:



4.108.1.3 HAL_TIMER_GetFreq()

```
uint32_t HAL_TIMER_GetFreq (
    HAL_TIMER_x timer_x )
```

在文件 [hal_lks32mc07x.timer.c](#) 第 167 行定义.

函数调用图:



4.108.1.4 HAL_TIMER_GetThCnt()

```
uint32_t HAL_TIMER_GetThCnt (
    HAL_TIMER_x timer_x )
```

在文件 [hal_lks32mc07x.timer.c](#) 第 174 行定义.

这是这个函数的调用关系图:



4.108.1.5 HAL_TIMER_IrqCompare0()

```
uint8_t HAL_TIMER_IrqCompare0 (
    HAL_TIMER_x timer_x )
```

在文件 [hal_lks32mc07x.timer.c](#) 第 250 行定义.

4.108.1.6 HAL_TIMER_IrqCompare0Clear()

```
void HAL_TIMER_IrqCompare0Clear (
    HAL_TIMER_x timer_x )
```

在文件 [hal_lks32mc07x.timer.c](#) 第 272 行定义.

4.108.1.7 HAL_TIMER_IrqCompare0Disable()

```
void HAL_TIMER_IrqCompare0Disable (
    HAL_TIMER_x timer_x )
```

在文件 [hal_lks32mc07x.timer.c](#) 第 268 行定义.

4.108.1.8 HAL_TIMER_IrqCompare0Enable()

```
void HAL_TIMER_IrqCompare0Enable (
    HAL_TIMER_x timer_x )
```

在文件 [hal_lks32mc07x.timer.c](#) 第 264 行定义.

4.108.1.9 HAL_TIMER_IrqCompare1()

```
uint8_t HAL_TIMER_IrqCompare1 (
    HAL_TIMER_x timer_x )
```

在文件 [hal_lks32mc07x_timer.c](#) 第 277 行定义.

4.108.1.10 HAL_TIMER_IrqCompare1Clear()

```
void HAL_TIMER_IrqCompare1Clear (
    HAL_TIMER_x timer_x )
```

在文件 [hal_lks32mc07x_timer.c](#) 第 299 行定义.

4.108.1.11 HAL_TIMER_IrqCompare1Disable()

```
void HAL_TIMER_IrqCompare1Disable (
    HAL_TIMER_x timer_x )
```

在文件 [hal_lks32mc07x_timer.c](#) 第 295 行定义.

4.108.1.12 HAL_TIMER_IrqCompare1Enable()

```
void HAL_TIMER_IrqCompare1Enable (
    HAL_TIMER_x timer_x )
```

在文件 [hal_lks32mc07x_timer.c](#) 第 291 行定义.

4.108.1.13 HAL_TIMER_IrqOverflow()

```
uint8_t HAL_TIMER_IrqOverflow (
    HAL_TIMER_x timer_x )
```

在文件 [hal_lks32mc07x_timer.c](#) 第 223 行定义.

4.108.1.14 HAL_TIMER_IrqOverflowClear()

```
void HAL_TIMER_IrqOverflowClear (
    HAL_TIMER_x timer_x )
```

在文件 [hal_lks32mc07x_timer.c](#) 第 245 行定义.

4.108.1.15 HAL_TIMER_IrqOverflowDisable()

```
void HAL_TIMER_IrqOverflowDisable (
    HAL_TIMER_x timer_x )
```

在文件 [hal_lks32mc07x_timer.c](#) 第 241 行定义.

4.108.1.16 HAL_TIMER_IrqOverflowEnable()

```
void HAL_TIMER_IrqOverflowEnable (
    HAL_TIMER_x timer_x )
```

在文件 `hal_lks32mc07x.timer.c` 第 237 行定义.

4.108.1.17 HAL_TIMER_Reset()

```
void HAL_TIMER_Reset (
    HAL_TIMER_x timer_x )
```

在文件 `hal_lks32mc07x.timer.c` 第 10 行定义.

4.108.1.18 HAL_TIMER_SetFreq()

```
void HAL_TIMER_SetFreq (
    HAL_TIMER_x timer_x,
    uint32_t freq )
```

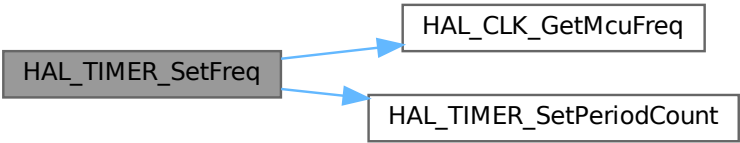
设置定时器频率

参数

<i>timer_x</i>	定时器编号
<i>freq</i>	定时器频率

在文件 `hal_lks32mc07x.timer.c` 第 137 行定义.

函数调用图:



4.108.1.19 HAL_TIMER_SetGpioIn()

```
void HAL_TIMER_SetGpioIn (
    HAL_TIMER_x timer_x,
    HAL_GPIO_Px px )
```

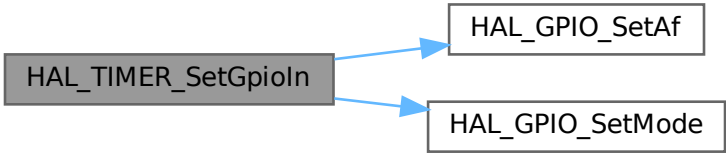
设置定时器输入gpio

参数

<i>timer</i> ↔ _x	定时器编号
<i>px</i>	gpio编号

在文件 [hal_lks32mc07x_timer.c](#) 第 338 行定义.

函数调用图:



4.108.1.20 HAL_TIMER_SetGpioOut()

```
void HAL_TIMER_SetGpioOut (
    HAL_TIMER_x timer_x,
    HAL_GPIO_Px px )
```

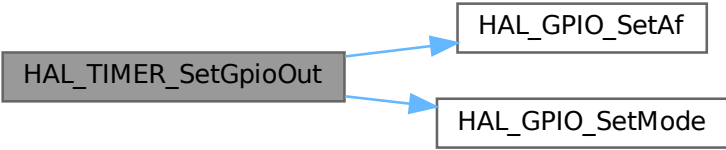
设置定时器输出gpio

参数

<i>timer</i> ↔ _x	定时器编号
<i>px</i>	gpio编号

在文件 [hal_lks32mc07x_timer.c](#) 第 309 行定义.

函数调用图:



4.108.1.21 HAL_TIMER.SetPeriodCount()

```
void HAL_TIMER.SetPeriodCount (
    HAL_TIMER.x timer_x,
    uint32_t count )
```

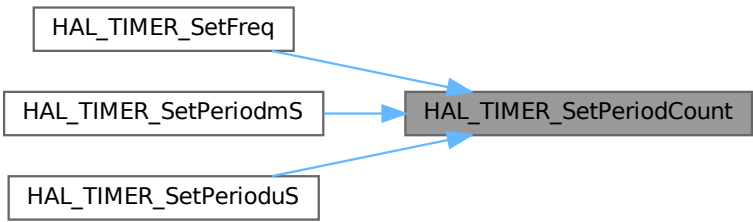
设置定时器周期

参数

<i>timer_x</i>	定时器编号
<i>count</i>	定时器周期

在文件 [hal_lks32mc07x.timer.c](#) 第 92 行定义.

这是这个函数的调用关系图:



4.108.1.22 HAL_TIMER.SetPeriodmS()

```
void HAL_TIMER.SetPeriodmS (
    HAL_TIMER.x timer_x,
    uint32_t ms )
```

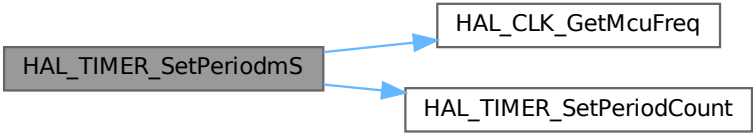
设置定时器周期(ms)

参数

<i>timer_x</i>	定时器编号
<i>ms</i>	定时器周期(ms)

在文件 [hal_lks32mc07x.timer.c](#) 第 149 行定义.

函数调用图:



4.108.1.23 HAL_TIMER_SetPerioduS()

```
void HAL_TIMER_SetPerioduS (
    HAL_TIMER_x timer_x,
    uint32_t us )
```

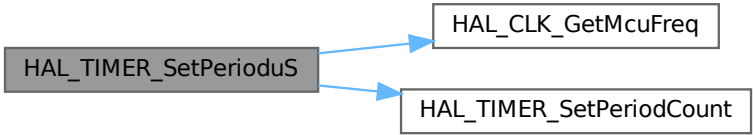
设置定时器周期(us)

参数

<i>timer_x</i>	定时器编号
<i>us</i>	定时器周期(us)

在文件 [hal_lks32mc07x.timer.c](#) 第 160 行定义.

函数调用图:



4.108.1.24 HAL_TIMER_SetPwm()

```
void HAL_TIMER_SetPwm (
    HAL_TIMER_x timer_x,
    int32_t t0,
    int32_t t1 )
```

在文件 [hal_lks32mc07x.timer.c](#) 第 190 行定义.

4.108.1.25 HAL_TIMER_SetPwmQ15()

```
void HAL_TIMER_SetPwmQ15 (
    HAL_TIMER_x timer_x,
    int16_t t0,
    int16_t t1 )
```

在文件 [hal_lks32mc07x.timer.c](#) 第 196 行定义.

函数调用图:



4.108.1.26 HAL_TIMER_SetWorkMode()

```
void HAL_TIMER_SetWorkMode (
    HAL_TIMER_x timer_x,
    HAL_TIMER_MODE_x mode_x )
```

< 计数模式

< PWM输出模式

< PWM输入模式

在文件 [hal_lks32mc07x.timer.c](#) 第 205 行定义.

4.108.1.27 HAL_TIMER_StartCounting()

```
void HAL_TIMER_StartCounting (
    HAL_TIMER_x timer_x )
```

在文件 [hal_lks32mc07x.timer.c](#) 第 179 行定义.

4.108.1.28 HAL_TIMER_StopCounting()

```
void HAL_TIMER_StopCounting (
    HAL_TIMER_x timer_x )
```

在文件 [hal_lks32mc07x.timer.c](#) 第 184 行定义.

4.109 hal_lks32mc07x_timer.c

[浏览该文件的文档.](#)

```

00001
00002 #include "../include/hal_lksmcu_timer.h"
00003 #include "../include/hal_lksmcu_gpio.h"
00004 #include "../hal_lks32mc07x.h"
00005 #include "lks32mc07x_user_manual.h"
00006 #include "../include/hal_lksmcu_reg.h"
00007 #include "../include/hal_lks32mc07x_reg.h"
00008
00009 // 模块复位
00010 void HAL_TIMER_Reset(HAL_TIMER_x timer_x)
00011 {
00012     HAL_REG_SYS_UNLOCK();
00013     switch (timer_x)
00014     {
00015         case HAL_TIMER_0:
00016             HAL_REG_SET(SYS_SFT_RST, SYS_SFT_RST_TIMER0_SFT_RST_BIT);
00017             HAL_REG_RESET(SYS_SFT_RST, SYS_SFT_RST_TIMER0_SFT_RST_BIT);
00018             break;
00019         case HAL_TIMER_1:
00020             HAL_REG_SET(SYS_SFT_RST, SYS_SFT_RST_TIMER1_SFT_RST_BIT);
00021             HAL_REG_RESET(SYS_SFT_RST, SYS_SFT_RST_TIMER1_SFT_RST_BIT);
00022             break;
00023         case HAL_TIMER_2:
00024             HAL_REG_SET(SYS_SFT_RST, SYS_SFT_RST_TIMER2_SFT_RST_BIT);
00025             HAL_REG_RESET(SYS_SFT_RST, SYS_SFT_RST_TIMER2_SFT_RST_BIT);
00026             break;
00027         case HAL_TIMER_3:
00028             HAL_REG_SET(SYS_SFT_RST, SYS_SFT_RST_TIMER3_SFT_RST_BIT);
00029             HAL_REG_RESET(SYS_SFT_RST, SYS_SFT_RST_TIMER3_SFT_RST_BIT);
00030             break;
00031         default:
00032             break;
00033     }
00034     // 系统寄存器上锁
00035     HAL_REG_SYS_LOCK();
00036 }
00037 // 模块打开
00038 void HAL_TIMER_Enable(HAL_TIMER_x timer_x)
00039 {
00040     // 系统寄存器解锁
00041     HAL_REG_SYS_UNLOCK();
00042     switch (timer_x)
00043     {
00044         case HAL_TIMER_0:
00045             HAL_REG_SET(SYS_CLK_FEN, SYS_CLK_FEN_TIMER0_CLK_EN_BIT);
00046             break;
00047         case HAL_TIMER_1:
00048             HAL_REG_SET(SYS_CLK_FEN, SYS_CLK_FEN_TIMER1_CLK_EN_BIT);
00049             break;
00050         case HAL_TIMER_2:
00051             HAL_REG_SET(SYS_CLK_FEN, SYS_CLK_FEN_TIMER2_CLK_EN_BIT);
00052             break;
00053         case HAL_TIMER_3:
00054             HAL_REG_SET(SYS_CLK_FEN, SYS_CLK_FEN_TIMER3_CLK_EN_BIT);
00055             break;
00056         default:
00057             break;
00058     }
00059     // 系统寄存器上锁
00060     HAL_REG_SYS_LOCK();
00061 }
00062 // 模块关闭
00063 void HAL_TIMER_Disable(HAL_TIMER_x timer_x)
00064 {
00065     // 系统寄存器解锁
00066     HAL_REG_SYS_UNLOCK();
00067     switch (timer_x)
00068     {
00069         case HAL_TIMER_0:
00070             HAL_REG_RESET(SYS_CLK_FEN, SYS_CLK_FEN_TIMER0_CLK_EN_BIT);
00071             break;
00072         case HAL_TIMER_1:
00073             HAL_REG_RESET(SYS_CLK_FEN, SYS_CLK_FEN_TIMER1_CLK_EN_BIT);
00074             break;
00075         case HAL_TIMER_2:
00076             HAL_REG_RESET(SYS_CLK_FEN, SYS_CLK_FEN_TIMER2_CLK_EN_BIT);
00077             break;
00078         case HAL_TIMER_3:
00079             HAL_REG_RESET(SYS_CLK_FEN, SYS_CLK_FEN_TIMER3_CLK_EN_BIT);
00080             break;
00081         default:
00082             break;

```

```

00083     }
00084     // 系统寄存器上锁
00085     HAL_REG_SYS_LOCK();
00086 }
00092 void HAL_TIMER_SetPeriodCount(HAL_TIMER_x timer_x, uint32_t count)
00093 {
00094     if (count > 0)
00095     {
00096         count = count - 1;
00097     }
00098     if ((timer_x == HAL_TIMER_0) || (timer_x == HAL_TIMER_1))
00099     {
00100
00101         uint32_t div = 0;
00102         for (; count > TIMER0_TH_TH_MAX;)
00103         {
00104             count >>= 1;
00105             div++;
00106         }
00107         if (div > TIMER0_CFG_CLK_DIV_MAX)
00108         {
00109             div = TIMER0_CFG_CLK_DIV_MAX;
00110             count = TIMER0_TH_TH_MAX;
00111         }
00112         HAL_REG_WRITE_BIT_MASK_POS(HAL_TIMER[timer_x]->CFG, TIMER0_CFG_CLK_DIV, div);
00113         HAL_REG_WRITE_BIT_MASK_POS(HAL_TIMER[timer_x]->TH, TIMER0_TH_TH, count);
00114     }
00115     else
00116     {
00117         uint32_t div = 0;
00118         for (; count > TIMER2_TH_TH_MAX;)
00119         {
00120             count >>= 1;
00121             div++;
00122         }
00123         if (div > TIMER2_CFG_CLK_DIV_MAX)
00124         {
00125             div = TIMER2_CFG_CLK_DIV_MAX;
00126             count = TIMER2_TH_TH_MAX;
00127         }
00128         HAL_REG_WRITE_BIT_MASK_POS(HAL_TIMER[timer_x]->CFG, TIMER2_CFG_CLK_DIV, div);
00129         HAL_REG_WRITE_BIT_MASK_POS(HAL_TIMER[timer_x]->TH, TIMER2_TH_TH, count);
00130     }
00131 }
00137 void HAL_TIMER_SetFreq(HAL_TIMER_x timer_x, uint32_t freq)
00138 {
00139     uint32_t mclk = HAL_CLK_GetMcuFreq();
00140     uint32_t th = (mclk / freq);
00141     HAL_TIMER_SetPeriodCount(timer_x, th);
00142 }
00143
00149 void HAL_TIMER_SetPeriodms(HAL_TIMER_x timer_x, uint32_t ms)
00150 {
00151     uint32_t mclk = HAL_CLK_GetMcuFreq();
00152     uint32_t th = (mclk / 1000) * ms;
00153     HAL_TIMER_SetPeriodCount(timer_x, th);
00154 }
00160 void HAL_TIMER_SetPeriodus(HAL_TIMER_x timer_x, uint32_t us)
00161 {
00162     uint32_t mclk = HAL_CLK_GetMcuFreq();
00163     uint32_t th = (mclk / 1000000) * us;
00164     HAL_TIMER_SetPeriodCount(timer_x, th);
00165 }
00166 // 获取当前的pwm频率
00167 uint32_t HAL_TIMER_GetFreq(HAL_TIMER_x timer_x)
00168 {
00169     uint32_t mclk = HAL_CLK_GetMcuFreq();
00170     uint32_t freq = (mclk / HAL_TIMER[timer_x]->TH);
00171     return freq;
00172 }
00173 // 获取计数器最大值
00174 uint32_t HAL_TIMER_GetThCnt(HAL_TIMER_x timer_x)
00175 {
00176     return HAL_TIMER[timer_x]->TH;
00177 }
00178 // 开始计数
00179 void HAL_TIMER_StartCounting(HAL_TIMER_x timer_x)
00180 {
00181     HAL_REG_SET(HAL_TIMER[timer_x]->CFG, BIT31);
00182 }
00183 // 停止计数 (同时计数器回到0)
00184 void HAL_TIMER_StopCounting(HAL_TIMER_x timer_x)
00185 {
00186     HAL_REG_RESET(HAL_TIMER[timer_x]->CFG, BIT31);
00187     HAL_TIMER[timer_x]->CNT = 0;
00188 }
00189 // 设置pwm波形 仅PWM模式下有效

```

```

00190 void HAL_TIMER_SetPwm(HAL_TIMER_x timer_x, int32_t t0, int32_t t1)
00191 {
00192     HAL_REG_WRITE(HAL_TIMER[timer_x]->CMP0, t0);
00193     HAL_REG_WRITE(HAL_TIMER[timer_x]->CMP1, t1);
00194 }
00195 // 设置pwm波形q15格式输入 仅PWM模式下有效
00196 void HAL_TIMER_SetPwmQ15(HAL_TIMER_x timer_x, int16_t t0, int16_t t1)
00197 {
00198     uint32_t th = HAL_TIMER_GetThCnt(timer_x);
00199     t0 = t0 * th >> 15;
00200     t1 = t1 * th >> 15;
00201     HAL_REG_WRITE(HAL_TIMER[timer_x]->CMP0, t0);
00202     HAL_REG_WRITE(HAL_TIMER[timer_x]->CMP1, t1);
00203 }
00204 // 设置工作模式
00205 void HAL_TIMER_SetWorkMode(HAL_TIMER_x timer_x, HAL_TIMER_MODE_x mode_x)
00206 {
00207     switch (mode_x)
00208     {
00209         case HAL_TIMER_MODE_COUNT:
00210             HAL_REG_WRITE_BIT(HAL_TIMER[timer_x]->CFG, 0xff, 0);
00211             break;
00212         case HAL_TIMER_MODE_PWMOUT:
00213             HAL_REG_WRITE_BIT(HAL_TIMER[timer_x]->CFG, 0xff, BIT7 | BIT3);
00214             break;
00215         case HAL_TIMER_MODE_PWMIN:
00216             HAL_REG_WRITE_BIT(HAL_TIMER[timer_x]->CFG, 0xff, BIT6 | BIT5 | BIT2 | BIT0);
00217             break;
00218         default:
00219             break;
00220     }
00221 }
00222
00223 uint8_t HAL_TIMER_IrqOverflow(HAL_TIMER_x timer_x)
00224 {
00225     uint32_t val1;
00226
00227     val1 = HAL_TIMER[timer_x]->IE & HAL_TIMER[timer_x]->IF;
00228     if (val1 & BIT2)
00229     {
00230         return 1;
00231     }
00232     else
00233     {
00234         return 0;
00235     }
00236 }
00237 void HAL_TIMER_IrqOverflowEnable(HAL_TIMER_x timer_x)
00238 {
00239     HAL_REG_SET(HAL_TIMER[timer_x]->IE, BIT2);
00240 }
00241 void HAL_TIMER_IrqOverflowDisable(HAL_TIMER_x timer_x)
00242 {
00243     HAL_REG_RESET(HAL_TIMER[timer_x]->IE, BIT2);
00244 }
00245 void HAL_TIMER_IrqOverflowClear(HAL_TIMER_x timer_x)
00246 {
00247     HAL_REG_WRITE(HAL_TIMER[timer_x]->IF, BIT2);
00248 }
00249
00250 uint8_t HAL_TIMER_IrqCompare0(HAL_TIMER_x timer_x)
00251 {
00252     uint32_t val1;
00253
00254     val1 = HAL_TIMER[timer_x]->IE & HAL_TIMER[timer_x]->IF;
00255     if (val1 & BIT0)
00256     {
00257         return 1;
00258     }
00259     else
00260     {
00261         return 0;
00262     }
00263 }
00264 void HAL_TIMER_IrqCompare0Enable(HAL_TIMER_x timer_x)
00265 {
00266     HAL_REG_SET(HAL_TIMER[timer_x]->IE, BIT0);
00267 }
00268 void HAL_TIMER_IrqCompare0Disable(HAL_TIMER_x timer_x)
00269 {
00270     HAL_REG_RESET(HAL_TIMER[timer_x]->IE, BIT0);
00271 }
00272 void HAL_TIMER_IrqCompare0Clear(HAL_TIMER_x timer_x)
00273 {
00274     HAL_REG_WRITE(HAL_TIMER[timer_x]->IF, BIT0);
00275 }
00276

```

```

00277 uint8_t HAL_TIMER_IrqCompare1(HAL_TIMER_x timer_x)
00278 {
00279     uint32_t val1;
00280
00281     val1 = HAL_TIMER[timer_x]->IE & HAL_TIMER[timer_x]->IF;
00282     if (val1 & BIT1)
00283     {
00284         return 1;
00285     }
00286     else
00287     {
00288         return 0;
00289     }
00290 }
00291 void HAL_TIMER_IrqCompare1Enable(HAL_TIMER_x timer_x)
00292 {
00293     HAL_REG_SET(HAL_TIMER[timer_x]->IE, BIT1);
00294 }
00295 void HAL_TIMER_IrqCompare1Disable(HAL_TIMER_x timer_x)
00296 {
00297     HAL_REG_RESET(HAL_TIMER[timer_x]->IE, BIT1);
00298 }
00299 void HAL_TIMER_IrqCompare1Clear(HAL_TIMER_x timer_x)
00300 {
00301     HAL_REG_WRITE(HAL_TIMER[timer_x]->IF, BIT1);
00302 }
00303
00309 void HAL_TIMER_SetGpioOut(HAL_TIMER_x timer_x, HAL_GPIO_Px px)
00310 {
00311     switch (timer_x)
00312     {
00313         case HAL_TIMER_0:
00314             HAL_GPIO_SetAf(px, HAL_GPIO_AF_TIMER0);
00315             HAL_GPIO_SetMode(px, HAL_GPIO_MODE_OUT);
00316             break;
00317         case HAL_TIMER_1:
00318             HAL_GPIO_SetAf(px, HAL_GPIO_AF_TIMER1);
00319             HAL_GPIO_SetMode(px, HAL_GPIO_MODE_OUT);
00320             break;
00321         case HAL_TIMER_2:
00322             HAL_GPIO_SetAf(px, HAL_GPIO_AF_TIMER2);
00323             HAL_GPIO_SetMode(px, HAL_GPIO_MODE_OUT);
00324             break;
00325         case HAL_TIMER_3:
00326             HAL_GPIO_SetAf(px, HAL_GPIO_AF_TIMER3);
00327             HAL_GPIO_SetMode(px, HAL_GPIO_MODE_OUT);
00328             break;
00329         default:
00330             break;
00331     }
00332 }
00338 void HAL_TIMER_SetGpioIn(HAL_TIMER_x timer_x, HAL_GPIO_Px px)
00339 {
00340     switch (timer_x)
00341     {
00342         case HAL_TIMER_0:
00343             HAL_GPIO_SetAf(px, HAL_GPIO_AF_TIMER0);
00344             HAL_GPIO_SetMode(px, HAL_GPIO_MODE_IN);
00345             break;
00346         case HAL_TIMER_1:
00347             HAL_GPIO_SetAf(px, HAL_GPIO_AF_TIMER1);
00348             HAL_GPIO_SetMode(px, HAL_GPIO_MODE_IN);
00349             break;
00350         case HAL_TIMER_2:
00351             HAL_GPIO_SetAf(px, HAL_GPIO_AF_TIMER2);
00352             HAL_GPIO_SetMode(px, HAL_GPIO_MODE_IN);
00353             break;
00354         case HAL_TIMER_3:
00355             HAL_GPIO_SetAf(px, HAL_GPIO_AF_TIMER3);
00356             HAL_GPIO_SetMode(px, HAL_GPIO_MODE_IN);
00357             break;
00358         default:
00359             break;
00360     }
00361 }

```

4.110 source/hal_lks32mc07x_tmp.c 文件参考

```

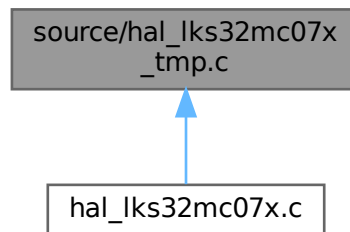
#include "../include/hal_lksmcu_tmp.h"
#include "../include/hal_lksmcu_trim.h"

```

```
#include "../hal_lks32mc07x.h"
#include "lks32mc07x_user_manual.h"
#include "../include/hal_lksmcu_reg.h"
#include "../include/hal_lks32mc07x_reg.h"
hal_lks32mc07x_tmp.c 的引用(Include)关系图:
```



此图展示该文件直接或间接的被哪些文件引用了:



函数

- void **HAL_TMP_Reset** (HAL_TMP_x TMP_x)
模块复位
- void **HAL_TMP_Enable** (HAL_TMP_x TMP_x)
模块打开
- void **HAL_TMP_Disable** (HAL_TMP_x TMP_x)
模块关闭
- int32_t **HAL_TMP_GetTemperature** (HAL_TMP_x TMP_x, HAL_ADC_x adc, HAL_ADC_SAMP_SEQUENCE_x sequence)
获取当前芯片内部温度传感器测到的温度,单位摄氏度
- int32_t **HAL_TMP_GetTemperature01** (HAL_TMP_x TMP_x, HAL_ADC_x adc, HAL_ADC_SAMP_SEQUENCE_x sequence)
获取当前芯片内部温度传感器测到的温度,单位0.1摄氏度
- int32_t **HAL_TMP_Adc2Temperature** (HAL_TMP_x TMP_x, int32_t adc_value)
adc值转温度
- int32_t **HAL_TMP_Adc2Temperature01** (HAL_TMP_x TMP_x, int32_t adc_value)
adc值转温度

变量

- static int32_t **tmp_coef_a** = 0
- static int32_t **tmp_coef_b** = 0

4.110.1 函数说明

4.110.1.1 HAL_TMP_Adc2Temperature()

```
int32_t HAL_TMP_Adc2Temperature (
    HAL_TMP_x TMP_x,
    int32_t adc_value )
```

adc值转温度

参数

<i>TMP_x</i>	模块编号
<i>adc_value</i>	ADC值，左对齐（-32767 到32767）

返回
当前温度,单位摄氏度

在文件 [hal_lks32mc07x.tmp.c](#) 第 84 行定义.

函数调用图:



这是这个函数的调用关系图:



4.110.1.2 HAL_TMP_Adc2Temperature01()

```
int32_t HAL_TMP_Adc2Temperature01 (
    HAL_TMP_x TMP_x,
    int32_t adc_value )
```

adc值转温度

参数

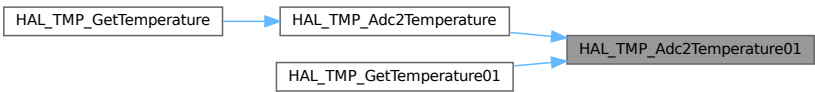
<i>TMP_x</i>	模块编号
<i>adc_value</i>	ADC值, 左对齐 (-32767 到32767)

返回

当前温度,单位0.1摄氏度

在文件 [hal_lks32mc07x.tmp.c](#) 第 94 行定义.

这是这个函数的调用关系图:



4.110.1.3 HAL_TMP_Disable()

```
void HAL_TMP_Disable (
    HAL_TMP_x TMP_x )
```

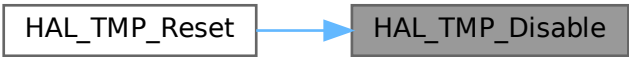
模块关闭

参数

<i>TMP_x</i>	模块编号
--------------	------

在文件 [hal_lks32mc07x.tmp.c](#) 第 37 行定义.

这是这个函数的调用关系图:



4.110.1.4 HAL_TMP_Enable()

```
void HAL_TMP_Enable (
    HAL_TMP_x TMP_x )
```

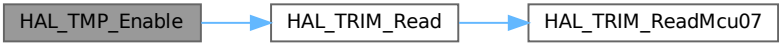
模块打开

参数

<i>TMP_x</i>	模块编号
--------------	------

在文件 `hal_lks32mc07x.tmp.c` 第 23 行定义.

函数调用图:



4.110.1.5 HAL_TMP_GetTemperature()

```
int32_t HAL_TMP_GetTemperature (
    HAL_TMP_x TMP_x,
    HAL_ADC_x adc,
    HAL_ADC_SAMP_SEQUENCE_x sequence )
```

获取当前芯片内部温度传感器测到的温度,单位摄氏度

参数

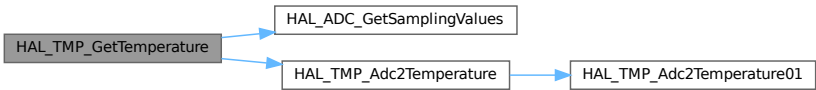
<i>TMP_x</i>	模块编号
<i>adc</i>	ADC编号
<i>sequence</i>	ADC采样序列

返回

当前温度,单位摄氏度

在文件 `hal_lks32mc07x.tmp.c` 第 52 行定义.

函数调用图:



4.110.1.6 HAL_TMP_GetTemperature01()

```
int32_t HAL_TMP_GetTemperature01 (
    HAL_TMP_x TMP_x,
    HAL_ADC_x adc,
    HAL_ADC_SAMP_SEQUENCE_x sequence )
```

获取当前芯片内部温度传感器测到的温度,单位0.1摄氏度

参数

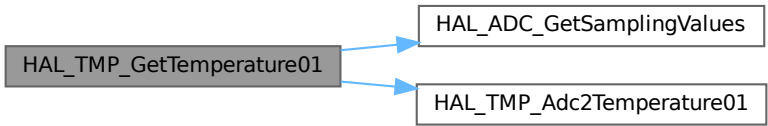
<i>TMP_x</i>	模块编号
<i>adc</i>	ADC编号
<i>sequence</i>	ADC采样序列

返回

当前温度,单位0.1摄氏度

在文件 [hal_lks32mc07x.tmp.c](#) 第 69 行定义.

函数调用图:



4.110.1.7 HAL_TMP_Reset()

```
void HAL_TMP_Reset (
    HAL_TMP_x TMP_x )
```

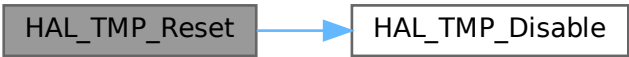
模块复位

参数

<i>TMP</i> ↔ _x	模块编号
--------------------	------

在文件 [hal_lks32mc07x.tmp.c](#) 第 14 行定义.

函数调用图:



4.110.2 变量说明

4.110.2.1 tmp_coef.a

```
int32_t tmp_coef.a = 0 [static]
```

在文件 [hal_lks32mc07x.tmp.c](#) 第 8 行定义.

4.110.2.2 tmp_coef.b

```
int32_t tmp_coef.b = 0 [static]
```

在文件 [hal_lks32mc07x.tmp.c](#) 第 9 行定义.

4.111 hal_lks32mc07x.tmp.c

[浏览该文件的文档.](#)

```
00001
00002 #include "../include/hal_lksmcu.tmp.h"
00003 #include "../include/hal_lksmcu.trim.h"
00004 #include "../hal_lks32mc07x.h"
00005 #include "lks32mc07x.user.manual.h"
00006 #include "../include/hal_lksmcu.reg.h"
00007 #include "../include/hal_lks32mc07x.reg.h"
00008 static int32_t tmp_coef.a = 0;
00009 static int32_t tmp_coef.b = 0;
00014 void HAL_TMP_Reset (HAL_TMP_x TMP_x)
00015 {
00016     HAL_TMP_Disable (TMP_x);
00017 }
00018
00023 void HAL_TMP_Enable (HAL_TMP_x TMP_x)
00024 {
00025     (void) TMP_x;
00026     HAL_REG_SYS_UNLOCK();
00027     HAL_REG_SET (SYS_AFE_REG5, SYS_AFE_REG5_TMPPDN_BIT);
00028     HAL_REG_SYS_LOCK();
00029     tmp_coef.a = -HAL_TRIM_Read (0x000014D0);
```

```

00030     tmp_coef_b = HAL_TRIM_Read(0x000014D4);
00031 }
00032
00037 void HAL_TMP_Disable(HAL_TMP_x TMP_x)
00038 {
00039     (void)TMP_x;
00040     HAL_REG_SYS_UNLOCK();
00041     HAL_REG_RESET(SYS_AFE_REG5, SYS_AFE_REG5_TMP_PDN_BIT);
00042     HAL_REG_SYS_LOCK();
00043 }
00044
00052 int32_t HAL_TMP_GetTemperature(HAL_TMP_x TMP_x, HAL_ADC_x adc, HAL_ADC_SAMP_SEQUENCE_x sequence)
00053 {
00054     // 实现获取温度的代码,单位摄氏度
00055     int32_t adc_data;
00056     int32_t tmp;
00057     adc_data = HAL_ADC_GetSamplingValues(adc, sequence);
00058     tmp = HAL_TMP_Adc2Temperature(TMP_x, adc_data);
00059     return tmp; // 返回温度值
00060 }
00061
00069 int32_t HAL_TMP_GetTemperature01(HAL_TMP_x TMP_x, HAL_ADC_x adc, HAL_ADC_SAMP_SEQUENCE_x sequence)
00070 {
00071     // 实现获取温度的代码,单位0.1摄氏度
00072     int32_t adc_data;
00073     int32_t tmp;
00074     adc_data = HAL_ADC_GetSamplingValues(adc, sequence);
00075     tmp = HAL_TMP_Adc2Temperature01(TMP_x, adc_data);
00076     return tmp; // 返回温度值
00077 }
00084 int32_t HAL_TMP_Adc2Temperature(HAL_TMP_x TMP_x, int32_t adc_value)
00085 {
00086     return HAL_TMP_Adc2Temperature01(TMP_x, adc_value) / 10;
00087 }
00094 int32_t HAL_TMP_Adc2Temperature01(HAL_TMP_x TMP_x, int32_t adc_value)
00095 {
00096     (void)TMP_x;
00097     int32_t tmp;
00098     tmp = (int32_t)tmp_coef_a * (adc_value >> 4) / 1000 + (int32_t)tmp_coef_b;
00099     return tmp; // 返回温度值
00100 }

```

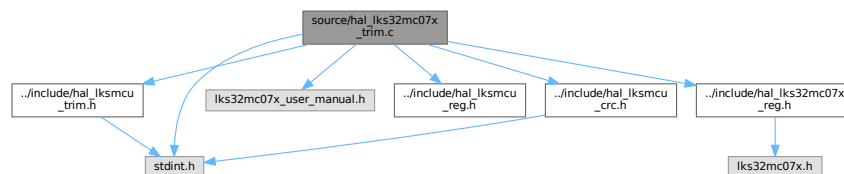
4.112 source/hal_lks32mc07x_trim.c 文件参考

```

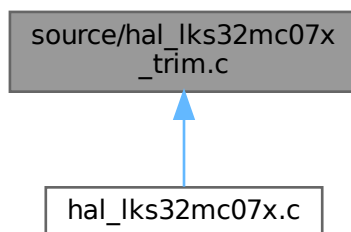
#include "../include/hal_lksmcu_trim.h"
#include "stdint.h"
#include "lks32mc07x_user_manual.h"
#include "../include/hal_lksmcu_reg.h"
#include "../include/hal_lks32mc07x_reg.h"
#include "../include/hal_lksmcu_crc.h"

```

hal_lks32mc07x_trim.c 的引用(Include)关系图:



此图展示该文件直接或间接的被哪些文件引用了:



函数

- uint32_t [HAL_TRIM.ReadMcu07](#) (uint32_t)
- uint32_t [HAL_TRIM.Read](#) (uint32_t addr)
读取MCU03的校准参数
- uint32_t [HAL_TRIM.ReadId](#) (uint32_t idx)
读取芯片唯一ID
- uint32_t [HAL_TRIM.ReadIdCrc32](#) (void)
读取芯片唯一ID的CRC32值

4.112.1 函数说明

4.112.1.1 HAL_TRIM.Read()

```
uint32_t HAL_TRIM_Read (  
    uint32_t addr )
```

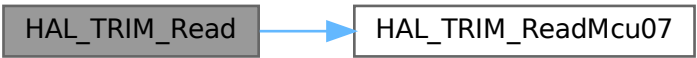
读取MCU03的校准参数

参数

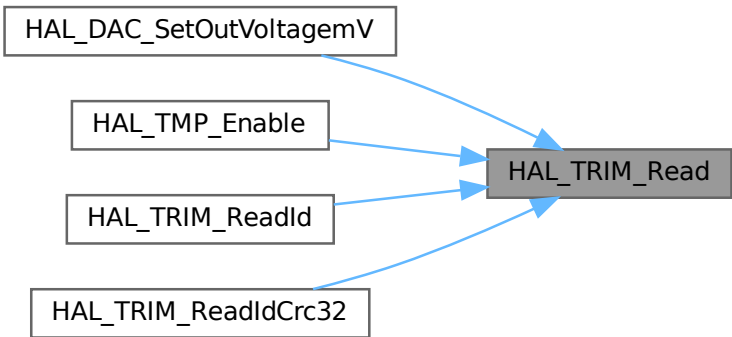
<i>addr</i>	校准参数的地址
-------------	---------

在文件 [hal_lks32mc07x_trim.c](#) 第 68 行定义.

函数调用图:



这是这个函数的调用关系图:



4.112.1.2 HAL_TRIM_ReadId()

```
uint32_t HAL_TRIM_ReadId (
    uint32_t idx )
```

读取芯片唯一ID

参数

<i>idx</i>	0:ID0,1:ID1,2:ID2,3:ID3,4:ID4
------------	-------------------------------

返回

唯一ID

在文件 [hal_lks32mc07x.trim.c](#) 第 78 行定义.

函数调用图:



4.112.1.3 HAL_TRIM_ReadIdCrc32()

```
uint32_t HAL_TRIM_ReadIdCrc32 (
    void )
```

读取芯片唯一ID的CRC32值

返回

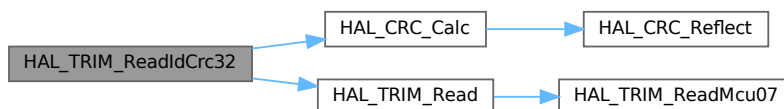
唯一ID的CRC32值

注解

相较于HAL_TRIM_ReadId，更方便存储和判断，但理论上存在重复的可能（1/0xffffffff）

在文件 [hal_lks32mc07x_trim.c](#) 第 87 行定义.

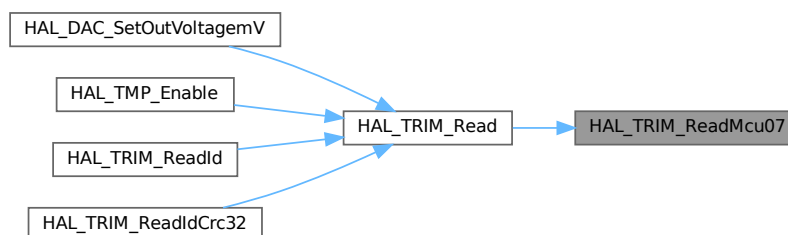
函数调用图:



4.112.1.4 HAL_TRIM_ReadMcu07()

```
uint32_t HAL_TRIM_ReadMcu07 (
    uint32_t )
```

这是这个函数的调用关系图:



4.113 hal_lks32mc07x_trim.c

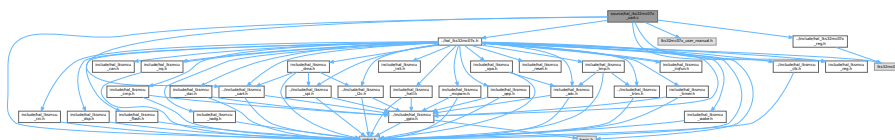
[浏览该文件的文档。](#)

```
00001 #include "../include/hal_lksmcu_trim.h"
00002 #include "stdint.h"
00003 #include "lks32mc07x_user_manual.h"
00004 #include "../include/hal_lksmcu_reg.h"
00005 #include "../include/hal_lks32mc07x_reg.h"
00006 #include "../include/hal_lksmcu_crc.h"
00007 uint32_t HAL_TRIM_ReadMcu07(uint32_t);
00008 // 地址 内容
00009 // 0x00001420 ADC0_DC0 校准值, 3.6V量程OFFSET
00010 // 0x00001424 ADC0_AMC0 校准值, 3.6V量程GAIN
00011 // 0x00001428 ADC0_DC1 校准值, 7.2V量程OFFSET
00012 // 0x0000142C ADC0_AMC1 校准值, 7.2V量程GAIN
00013 // 0x00001430 ADC1_DC0 校准值, 3.6V量程OFFSET
00014 // 0x00001434 ADC1_AMC0 校准值, 3.6V量程GAIN
00015 // 0x00001438 ADC1_DC1 校准值, 7.2V量程OFFSET
00016 // 0x0000143C ADC1_AMC1 校准值, 7.2V量程GAIN
00017 // 0x00001450 DAC0 选择4.85V 档位, SYS_AFE_DAC_AMC 校准值(扩大512 倍结果)
00018 // 0x00001454 DAC0 选择4.85V 档位, SYS_AFE_DAC_DC 校准值
00019 // 0x00001458 DAC0 选择1.20V 档位, SYS_AFE_DAC_AMC 校准值(扩大512 倍结果)
00020 // 0x0000145C DAC0 选择1.20V 档位, SYS_AFE_DAC_DC 校准值
00021 // 0x00001460 DAC1 选择4.85V 档位, SYS_AFE_DAC_AMC 校准值(扩大512 倍结果)
00022 // 0x00001464 DAC1 选择4.85V 档位, SYS_AFE_DAC_DC 校准值
00023 // 0x00001468 DAC1 选择1.20V 档位, SYS_AFE_DAC_AMC 校准值(扩大512 倍结果)
00024 // 0x0000146C DAC1 选择1.20V 档位, SYS_AFE_DAC_DC 校准值
00025 // 0x00001480 OPA0, 320K 欧姆 VS 10K 欧姆, GAIN 校准值(扩大1000 倍结果), R0为0欧姆。
00026 // 0x00001484 OPA0, 160K 欧姆 VS 10K 欧姆, GAIN 校准值(扩大1000 倍结果), R0为0欧姆。
00027 // 0x00001488 OPA0, 080K 欧姆 VS 10K 欧姆, GAIN 校准值(扩大1000 倍结果), R0为0欧姆。
00028 // 0x0000148C OPA0, 040K 欧姆 VS 10K 欧姆, GAIN 校准值(扩大1000 倍结果), R0为0欧姆。
00029 // 0x00001490 OPA1, 320K 欧姆 VS 10K 欧姆, GAIN 校准值(扩大1000 倍结果), R0为0欧姆。
00030 // 0x00001494 OPA1, 160K 欧姆 VS 10K 欧姆, GAIN 校准值(扩大1000 倍结果), R0为0欧姆。
00031 // 0x00001498 OPA1, 080K 欧姆 VS 10K 欧姆, GAIN 校准值(扩大1000 倍结果), R0为0欧姆。
00032 // 0x0000149C OPA1, 040K 欧姆 VS 10K 欧姆, GAIN 校准值(扩大1000 倍结果), R0为0欧姆。
00033 // 0x000014A0 OPA2, 320K 欧姆 VS 10K 欧姆, GAIN 校准值(扩大1000 倍结果), R0为0欧姆。
00034 // 0x000014A4 OPA2, 160K 欧姆 VS 10K 欧姆, GAIN 校准值(扩大1000 倍结果), R0为0欧姆。
00035 // 0x000014A8 OPA2, 080K 欧姆 VS 10K 欧姆, GAIN 校准值(扩大1000 倍结果), R0为0欧姆。
00036 // 0x000014AC OPA2, 040K 欧姆 VS 10K 欧姆, GAIN 校准值(扩大1000 倍结果), R0为0欧姆。
00037 // 0x000014B0 OPA3, 320K 欧姆 VS 10K 欧姆, GAIN 校准值(扩大1000 倍结果), R0为0欧姆。
00038 // 0x000014B4 OPA3, 160K 欧姆 VS 10K 欧姆, GAIN 校准值(扩大1000 倍结果), R0为0欧姆。
00039 // 0x000014B8 OPA3, 080K 欧姆 VS 10K 欧姆, GAIN 校准值(扩大1000 倍结果), R0为0欧姆。
00040 // 0x000014BC OPA3, 040K 欧姆 VS 10K 欧姆, GAIN 校准值(扩大1000 倍结果), R0为0欧姆。
00041 // 0x000014C0 OPA0, 共模电压(扩大10000 倍结果)
00042 // 0x000014C4 OPA1, 共模电压(扩大10000 倍结果)
00043 // 0x000014C8 OPA2, 共模电压(扩大10000 倍结果)
00044 // 0x000014CC OPA3, 共模电压(扩大10000 倍结果)
00045 // 0x000014D0 温度传感器, 斜率校准值
00046 // 0x000014D4 温度传感器, 偏置校准值
00047 // 0x00001500 OPA0, 320K 欧姆档位。高16位存放R2实际阻值, 低16位存放R1实际阻值(扩大100 倍结果)
00048 // 0x00001504 OPA0, 160K 欧姆档位。高16位存放R2实际阻值, 低16位存放R1实际阻值(扩大100 倍结果)
00049 // 0x00001508 OPA0, 080K 欧姆档位。高16位存放R2实际阻值, 低16位存放R1实际阻值(扩大100 倍结果)
00050 // 0x0000150C OPA0, 040K 欧姆档位。高16位存放R2实际阻值, 低16位存放R1实际阻值(扩大100 倍结果)
00051 // 0x00001510 OPA1, 320K 欧姆档位。高16位存放R2实际阻值, 低16位存放R1实际阻值(扩大100 倍结果)
00052 // 0x00001514 OPA1, 160K 欧姆档位。高16位存放R2实际阻值, 低16位存放R1实际阻值(扩大100 倍结果)
00053 // 0x00001518 OPA1, 080K 欧姆档位。高16位存放R2实际阻值, 低16位存放R1实际阻值(扩大100 倍结果)
00054 // 0x0000151C OPA1, 040K 欧姆档位。高16位存放R2实际阻值, 低16位存放R1实际阻值(扩大100 倍结果)
00055 // 0x00001520 OPA2, 320K 欧姆档位。高16位存放R2实际阻值, 低16位存放R1实际阻值(扩大100 倍结果)
00056 // 0x00001524 OPA2, 160K 欧姆档位。高16位存放R2实际阻值, 低16位存放R1实际阻值(扩大100 倍结果)
00057 // 0x00001528 OPA2, 080K 欧姆档位。高16位存放R2实际阻值, 低16位存放R1实际阻值(扩大100 倍结果)
00058 // 0x0000152C OPA2, 040K 欧姆档位。高16位存放R2实际阻值, 低16位存放R1实际阻值(扩大100 倍结果)
00059 // 0x00001530 OPA3, 320K 欧姆档位。高16位存放R2实际阻值, 低16位存放R1实际阻值(扩大100 倍结果)
00060 // 0x00001534 OPA3, 160K 欧姆档位。高16位存放R2实际阻值, 低16位存放R1实际阻值(扩大100 倍结果)
00061 // 0x00001538 OPA3, 080K 欧姆档位。高16位存放R2实际阻值, 低16位存放R1实际阻值(扩大100 倍结果)
00062 // 0x0000153C OPA3, 040K 欧姆档位。高16位存放R2实际阻值, 低16位存放R1实际阻值(扩大100 倍结果)
00063
00068 uint32_t HAL_TRIM_Read(uint32_t addr)
00069 {
00070     return HAL_TRIM_ReadMcu07(addr);
00071 }
00072
00078 uint32_t HAL_TRIM_ReadId(uint32_t idx)
00079 {
00080     return HAL_TRIM_Read(0x00001E84 + idx * 4);
00081 }
00087 uint32_t HAL_TRIM_ReadIdCrc32(void)
00088 {
00089     uint32_t id[5];
00090     for (int i = 0; i < 5; i++)
00091     {
00092         id[i] = HAL_TRIM_Read(0x00001E84 + i * 4);
00093     }
00094     return HAL_CRC_Calc(HAL_CRC_0, HAL_CRC_MODEL_CRC32, (uint8_t *)id, sizeof(id));
00095 }
```

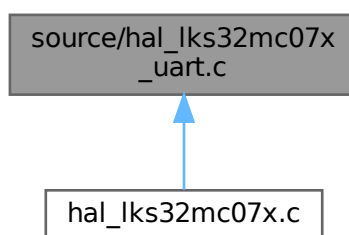
4.114 source/hal_lks32mc07x_uart.c 文件参考

```
#include "../include/hal_lksmcu_clk.h"
#include "../include/hal_lksmcu_uart.h"
#include "../hal_lks32mc07x.h"
#include "lks32mc07x-user_manual.h"
#include "../include/hal_lksmcu_reg.h"
#include "../include/hal_lks32mc07x_reg.h"
#include "stdint.h"
```

hal_lks32mc07x_uart.c 的引用(Include)关系图:



此图展示该文件直接或间接的被哪些文件引用了:



函数

- void [HAL_UART_Reset](#) (HAL_UART_x UART_x)
- void [HAL_UART_Enable](#) (HAL_UART_x UART_x)
- void [HAL_UART_Disable](#) (HAL_UART_x UART_x)
- void [HAL_UART_SetBaud](#) (HAL_UART_x UART_x, uint32_t baud)
- void [HAL_UART_TxData](#) (HAL_UART_x UART_x, uint8_t *data, uint32_t len)
- void [HAL_UART_SetMode](#) (HAL_UART_x uart_x, HAL_UART_MODE_x mode_x)
- uint8_t [HAL_UART_ReadBuffer](#) (HAL_UART_x uart_x)
- void [HAL_UART_WriteBuffer](#) (HAL_UART_x uart_x, uint8_t buff)
- void [HAL_UART_SetGpioTx](#) (HAL_UART_x uart_x, HAL_GPIO_Px px)
- void [HAL_UART_SetGpioRx](#) (HAL_UART_x uart_x, HAL_GPIO_Px px)
- uint8_t [HAL_UART_IrqSendOver](#) (HAL_UART_x uart_x)
- void [HAL_UART_IrqSendOverEnable](#) (HAL_UART_x uart_x)
- void [HAL_UART_IrqSendOverDisable](#) (HAL_UART_x uart_x)
- void [HAL_UART_IrqSendOverClear](#) (HAL_UART_x uart_x)
- uint8_t [HAL_UART_IrqReceiveOver](#) (HAL_UART_x uart_x)
- void [HAL_UART_IrqReceiveOverEnable](#) (HAL_UART_x uart_x)

- void HAL_UART_IrqReceiveOverDisable (HAL_UART_x uart_x)
- void HAL_UART_IrqReceiveOverClear (HAL_UART_x uart_x)
- uint8_t HAL_UART_IrqStopErr (HAL_UART_x uart_x)
- void HAL_UART_IrqStopErrEnable (HAL_UART_x uart_x)
- void HAL_UART_IrqStopErrDisable (HAL_UART_x uart_x)
- void HAL_UART_IrqStopErrClear (HAL_UART_x uart_x)
- uint8_t HAL_UART_IrqCheckErr (HAL_UART_x uart_x)
- void HAL_UART_IrqCheckErrEnable (HAL_UART_x uart_x)
- void HAL_UART_IrqCheckErrDisable (HAL_UART_x uart_x)
- void HAL_UART_IrqCheckErrClear (HAL_UART_x uart_x)

4.114.1 函数说明

4.114.1.1 HAL_UART_Disable()

```
void HAL_UART_Disable (  
    HAL_UART_x UART_x )
```

在文件 [hal_lks32mc07x_uart.c](#) 第 63 行定义.

4.114.1.2 HAL_UART_Enable()

```
void HAL_UART_Enable (  
    HAL_UART_x UART_x )
```

在文件 [hal_lks32mc07x_uart.c](#) 第 38 行定义.

4.114.1.3 HAL_UART_IrqCheckErr()

```
uint8_t HAL_UART_IrqCheckErr (  
    HAL_UART_x uart_x )
```

在文件 [hal_lks32mc07x_uart.c](#) 第 292 行定义.

4.114.1.4 HAL_UART_IrqCheckErrClear()

```
void HAL_UART_IrqCheckErrClear (  
    HAL_UART_x uart_x )
```

在文件 [hal_lks32mc07x_uart.c](#) 第 311 行定义.

4.114.1.5 HAL_UART_IrqCheckErrDisable()

```
void HAL_UART_IrqCheckErrDisable (  
    HAL_UART_x uart_x )
```

在文件 [hal_lks32mc07x_uart.c](#) 第 307 行定义.

4.114.1.6 HAL_UART_IrqCheckErrEnable()

```
void HAL_UART_IrqCheckErrEnable (
    HAL_UART_x uart_x )
```

在文件 [hal_lks32mc07x_uart.c](#) 第 303 行定义.

4.114.1.7 HAL_UART_IrqReceiveOver()

```
uint8_t HAL_UART_IrqReceiveOver (
    HAL_UART_x uart_x )
```

在文件 [hal_lks32mc07x_uart.c](#) 第 242 行定义.

4.114.1.8 HAL_UART_IrqReceiveOverClear()

```
void HAL_UART_IrqReceiveOverClear (
    HAL_UART_x uart_x )
```

在文件 [hal_lks32mc07x_uart.c](#) 第 261 行定义.

4.114.1.9 HAL_UART_IrqReceiveOverDisable()

```
void HAL_UART_IrqReceiveOverDisable (
    HAL_UART_x uart_x )
```

在文件 [hal_lks32mc07x_uart.c](#) 第 257 行定义.

4.114.1.10 HAL_UART_IrqReceiveOverEnable()

```
void HAL_UART_IrqReceiveOverEnable (
    HAL_UART_x uart_x )
```

在文件 [hal_lks32mc07x_uart.c](#) 第 253 行定义.

4.114.1.11 HAL_UART_IrqSendOver()

```
uint8_t HAL_UART_IrqSendOver (
    HAL_UART_x uart_x )
```

在文件 [hal_lks32mc07x_uart.c](#) 第 217 行定义.

4.114.1.12 HAL_UART_IrqSendOverClear()

```
void HAL_UART_IrqSendOverClear (
    HAL_UART_x uart_x )
```

在文件 [hal_lks32mc07x_uart.c](#) 第 236 行定义.

4.114.1.13 HAL_UART_IrqSendOverDisable()

```
void HAL_UART_IrqSendOverDisable (
    HAL_UART_x uart_x )
```

在文件 [hal_lks32mc07x_uart.c](#) 第 232 行定义.

4.114.1.14 HAL_UART_IrqSendOverEnable()

```
void HAL_UART_IrqSendOverEnable (
    HAL_UART_x uart_x )
```

在文件 [hal_lks32mc07x_uart.c](#) 第 228 行定义.

4.114.1.15 HAL_UART_IrqStopErr()

```
uint8_t HAL_UART_IrqStopErr (
    HAL_UART_x uart_x )
```

在文件 [hal_lks32mc07x_uart.c](#) 第 267 行定义.

4.114.1.16 HAL_UART_IrqStopErrClear()

```
void HAL_UART_IrqStopErrClear (
    HAL_UART_x uart_x )
```

在文件 [hal_lks32mc07x_uart.c](#) 第 286 行定义.

4.114.1.17 HAL_UART_IrqStopErrDisable()

```
void HAL_UART_IrqStopErrDisable (
    HAL_UART_x uart_x )
```

在文件 [hal_lks32mc07x_uart.c](#) 第 282 行定义.

4.114.1.18 HAL_UART_IrqStopErrEnable()

```
void HAL_UART_IrqStopErrEnable (
    HAL_UART_x uart_x )
```

在文件 [hal_lks32mc07x_uart.c](#) 第 278 行定义.

4.114.1.19 HAL_UART_ReadBuffer()

```
uint8_t HAL_UART_ReadBuffer (
    HAL_UART_x uart_x )
```

在文件 [hal_lks32mc07x_uart.c](#) 第 195 行定义.

4.114.1.20 HAL_UART_Reset()

```
void HAL_UART_Reset (
    HAL_UART_TypeDef UARTx )
```

在文件 [hal_lks32mc07x_uart.c](#) 第 11 行定义.

4.114.1.21 HAL_UART_SetBaud()

```
void HAL_UART_SetBaud (
    HAL_UART_TypeDef UARTx,
    uint32_t baud )
```

在文件 [hal_lks32mc07x_uart.c](#) 第 87 行定义.

函数调用图:

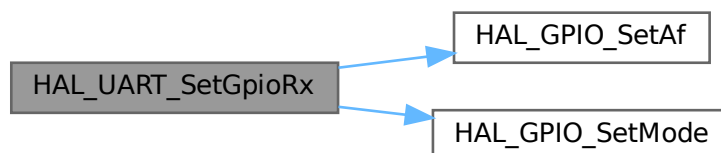


4.114.1.22 HAL_UART_SetGpioRx()

```
void HAL_UART_SetGpioRx (
    HAL_UART_TypeDef uartx,
    HAL_GPIO_TypeDef px )
```

在文件 [hal_lks32mc07x_uart.c](#) 第 209 行定义.

函数调用图:

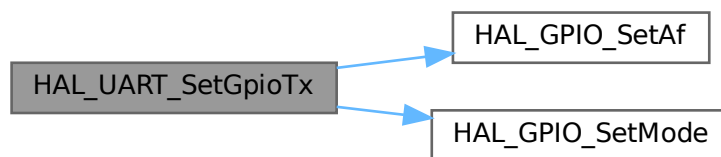


4.114.1.23 HAL_UART_SetGpioTx()

```
void HAL_UART_SetGpioTx (
    HAL_UART_<u>x</u> uart_<u>x</u>,
    HAL_GPIO_P<u>x</u> p<u>x</u> )
```

在文件 [hal_lks32mc07x_uart.c](#) 第 203 行定义.

函数调用图:



4.114.1.24 HAL_UART_SetMode()

```
void HAL_UART_SetMode (
    HAL_UART_<u>x</u> uart_<u>x</u>,
    HAL_UART_MODE_<u>x</u> mode_<u>x</u> )
```

< 低位先发 1位停止位 无校验

< 低位先发 2位停止位 无校验

< 低位先发 1位停止位 奇校验

< 低位先发 2位停止位 奇校验

< 低位先发 1位停止位 偶校验

< 低位先发 2位停止位 偶校验

< 高位先发 1位停止位 无校验

< 高位先发 2位停止位 无校验

< 高位先发 1位停止位 奇校验

< 高位先发 2位停止位 奇校验

< 高位先发 1位停止位 偶校验

< 高位先发 2位停止位 偶校验

在文件 [hal_lks32mc07x_uart.c](#) 第 123 行定义.

4.114.1.25 HAL_UART_TxData()

```
void HAL_UART_TxData (
    HAL_UART_x UART_x,
    uint8_t * data,
    uint32_t len )
```

在文件 [hal_lks32mc07x_uart.c](#) 第 111 行定义。

4.114.1.26 HAL_UART_WriteBuffer()

```
void HAL_UART_WriteBuffer (
    HAL_UART_x uart_x,
    uint8_t buff )
```

在文件 [hal_lks32mc07x_uart.c](#) 第 199 行定义。

4.115 hal_lks32mc07x_uart.c

[浏览该文件的文档。](#)

```
00001 #include "../include/hal_lksmcu_clk.h"
00002 #include "../include/hal_lksmcu_uart.h"
00003 #include "../hal_lks32mc07x.h"
00004 #include "lks32mc07x.user_manual.h"
00005 #include "../include/hal_lksmcu_reg.h"
00006 #include "../include/hal_lks32mc07x_reg.h"
00007 #include "stdint.h"
00008
00009 // 初始化
00010 // 模块复位
00011 void HAL_UART_Reset(HAL_UART_x UART_x)
00012 {
00013     switch (UART_x)
00014     {
00015         case HAL_UART_0:
00016
00017             // 系统寄存器解锁
00018             HAL_REG_SYS_UNLOCK();
00019             HAL_REG_SET(SYS_SFT_RST, SYS_SFT_RST_UART0_SFT_RST_BIT);
00020             HAL_REG_RESET(SYS_SFT_RST, SYS_SFT_RST_UART0_SFT_RST_BIT);
00021             // 系统寄存器上锁
00022             HAL_REG_SYS_LOCK();
00023             break;
00024         case HAL_UART_1:
00025
00026             // 系统寄存器解锁
00027             HAL_REG_SYS_UNLOCK();
00028             HAL_REG_SET(SYS_SFT_RST, SYS_SFT_RST_UART1_SFT_RST_BIT);
00029             HAL_REG_RESET(SYS_SFT_RST, SYS_SFT_RST_UART1_SFT_RST_BIT);
00030             // 系统寄存器上锁
00031             HAL_REG_SYS_LOCK();
00032             break;
00033         default:
00034             break;
00035     }
00036 }
00037 // 模块打开
00038 void HAL_UART_Enable(HAL_UART_x UART_x)
00039 {
00040     switch (UART_x)
00041     {
00042         case HAL_UART_0:
00043
00044             // 系统寄存器解锁
00045             HAL_REG_SYS_UNLOCK();
00046             HAL_REG_SET(SYS_CLK_FEN, SYS_CLK_FEN_UART0_CLK_EN_BIT);
00047             // 系统寄存器上锁
00048             HAL_REG_SYS_LOCK();
00049             break;
00050         case HAL_UART_1:
```



```

00051
00052     // 系统寄存器解锁
00053     HAL_REG_SYS_UNLOCK();
00054     HAL_REG_SET(SYS_CLK_FEN, SYS_CLK_FEN_UART1_CLK_EN_BIT);
00055     // 系统寄存器上锁
00056     HAL_REG_SYS_LOCK();
00057     break;
00058     default:
00059         break;
00060 }
00061 }
00062 // 模块关闭
00063 void HAL_UART_Disable(HAL_UART_x UART_x)
00064 {
00065     switch (UART_x)
00066     {
00067         case HAL_UART_0:
00068             // 系统寄存器解锁
00069             HAL_REG_SYS_UNLOCK();
00070             HAL_REG_RESET(SYS_CLK_FEN, SYS_CLK_FEN_UART0_CLK_EN_BIT);
00071             // 系统寄存器上锁
00072             HAL_REG_SYS_LOCK();
00073             break;
00074         case HAL_UART_1:
00075             // 系统寄存器解锁
00076             HAL_REG_SYS_UNLOCK();
00077             HAL_REG_RESET(SYS_CLK_FEN, SYS_CLK_FEN_UART1_CLK_EN_BIT);
00078             // 系统寄存器上锁
00079             HAL_REG_SYS_LOCK();
00080             break;
00081         default:
00082             break;
00083     }
00084 }
00085 }
00086 }
00087 void HAL_UART_SetBaud(HAL_UART_x UART_x, uint32_t baud)
00088 {
00089     switch (UART_x)
00090     {
00091         case HAL_UART_0:
00092         {
00093             uint32_t mclk = HAL_CLK_GetMcuFreq();
00094             uint32_t div = mclk / baud;
00095             uint32_t div2 = 0;
00096             while (div & 0xffff0000)
00097             {
00098                 div2++;
00099                 div >>= 1;
00100             }
00101             HAL_REG_SYS_UNLOCK();
00102             HAL_REG_WRITE(SYS_CLK_DIV2, div2);
00103             HAL_REG_SYS_LOCK();
00104             HAL_REG_WRITE(HAL_UART[UART_x]->DIVH, div >> 8);
00105             HAL_REG_WRITE(HAL_UART[UART_x]->DIVL, div & 0xff);
00106         }
00107         default:
00108             break;
00109     }
00110 }
00111 void HAL_UART_TxData(HAL_UART_x UART_x, uint8_t *data, uint32_t len)
00112 {
00113     for (; len > 0; len--)
00114     {
00115         HAL_REG_WRITE(HAL_UART[UART_x]->BUFF, *data);
00116         data++;
00117         while (HAL_REG_READ_BIT(HAL_UART[UART_x]->STT, BIT0) == 0)
00118         {
00119         }
00120     }
00121 }
00122 }
00123 void HAL_UART_SetMode(HAL_UART_x uart_x, HAL_UART_MODE_x mode_x)
00124 {
00125     uint32_t mask = (UARTx_CTRL_CK_EN_BIT |
00126                     UARTx_CTRL_CK_TYPE_BIT |
00127                     UARTx_CTRL_BIT_ORDER_BIT |
00128                     UARTx_CTRL_STOP_LEN_BIT |
00129                     UARTx_CTRL_BYTE_LEN_BIT);
00130     switch (mode_x)
00131     {
00132         case HAL_UART_MODE_LSB_STOP1_NONE:
00133         {
00134             HAL_REG_WRITE_BIT(HAL_UART[uart_x]->CTRL, mask, 0);
00135             break;
00136         }
00137     }

```

```

00138         case HAL_UART_MODE_LSB_STOP2_NONE:
00139         {
00140             HAL_REG_WRITE_BIT(HAL_UART[uart_x]->CTRL, mask, UARTx_CTRL_STOP_LEN_BIT);
00141             break;
00142         }
00143         case HAL_UART_MODE_LSB_STOP1_ODD:
00144         {
00145             HAL_REG_WRITE_BIT(HAL_UART[uart_x]->CTRL, mask, UARTx_CTRL_CK_EN_BIT | UARTx_CTRL_CK_TYPE_BIT |
UARTx_CTRL_STOP_LEN_BIT);
00146             break;
00147         }
00148         case HAL_UART_MODE_LSB_STOP2_ODD:
00149         {
00150             HAL_REG_WRITE_BIT(HAL_UART[uart_x]->CTRL, mask, UARTx_CTRL_CK_EN_BIT | UARTx_CTRL_CK_TYPE_BIT |
UARTx_CTRL_STOP_LEN_BIT | UARTx_CTRL_BYTE_LEN_BIT);
00151             break;
00152         }
00153         case HAL_UART_MODE_LSB_STOP1_EVEN:
00154         {
00155             HAL_REG_WRITE_BIT(HAL_UART[uart_x]->CTRL, mask, UARTx_CTRL_CK_EN_BIT |
UARTx_CTRL_BYTE_LEN_BIT);
00156             break;
00157         }
00158         case HAL_UART_MODE_LSB_STOP2_EVEN:
00159         {
00160             HAL_REG_WRITE_BIT(HAL_UART[uart_x]->CTRL, mask, UARTx_CTRL_CK_EN_BIT | UARTx_CTRL_STOP_LEN_BIT
| UARTx_CTRL_BYTE_LEN_BIT);
00161             break;
00162         }
00163         case HAL_UART_MODE_MSB_STOP1_NONE:
00164         {
00165             HAL_REG_WRITE_BIT(HAL_UART[uart_x]->CTRL, mask, UARTx_CTRL_BIT_ORDER_BIT);
00166             break;
00167         }
00168         case HAL_UART_MODE_MSB_STOP2_NONE:
00169         {
00170             HAL_REG_WRITE_BIT(HAL_UART[uart_x]->CTRL, mask, UARTx_CTRL_BIT_ORDER_BIT |
UARTx_CTRL_STOP_LEN_BIT);
00171             break;
00172         }
00173         case HAL_UART_MODE_MSB_STOP1_ODD:
00174         {
00175             HAL_REG_WRITE_BIT(HAL_UART[uart_x]->CTRL, mask, UARTx_CTRL_CK_EN_BIT | UARTx_CTRL_CK_TYPE_BIT |
UARTx_CTRL_BIT_ORDER_BIT | UARTx_CTRL_BYTE_LEN_BIT);
00176             break;
00177         }
00178         case HAL_UART_MODE_MSB_STOP2_ODD:
00179         {
00180             HAL_REG_WRITE_BIT(HAL_UART[uart_x]->CTRL, mask, UARTx_CTRL_CK_EN_BIT | UARTx_CTRL_CK_TYPE_BIT |
UARTx_CTRL_BIT_ORDER_BIT | UARTx_CTRL_STOP_LEN_BIT | UARTx_CTRL_BYTE_LEN_BIT);
00181             break;
00182         }
00183         case HAL_UART_MODE_MSB_STOP1_EVEN:
00184         {
00185             HAL_REG_WRITE_BIT(HAL_UART[uart_x]->CTRL, mask, UARTx_CTRL_CK_EN_BIT | UARTx_CTRL_BIT_ORDER_BIT
| UARTx_CTRL_BYTE_LEN_BIT);
00186             break;
00187         }
00188         case HAL_UART_MODE_MSB_STOP2_EVEN:
00189         {
00190             HAL_REG_WRITE_BIT(HAL_UART[uart_x]->CTRL, mask, UARTx_CTRL_CK_EN_BIT | UARTx_CTRL_BIT_ORDER_BIT
| UARTx_CTRL_STOP_LEN_BIT | UARTx_CTRL_BYTE_LEN_BIT);
00191             break;
00192         }
00193     }
00194 }
00195 uint8_t HAL_UART_ReadBuffer(HAL_UART_x uart_x)
00196 {
00197     return HAL_REG_READ(HAL_UART[uart_x]->BUFF);
00198 }
00199 void HAL_UART_WriteBuffer(HAL_UART_x uart_x, uint8_t buff)
00200 {
00201     HAL_REG_WRITE(HAL_UART[uart_x]->BUFF, buff);
00202 }
00203 void HAL_UART_SetGpioTx(HAL_UART_x uart_x, HAL_GPIO_Px px)
00204 {
00205     (void)uart_x;
00206     HAL_GPIO_SetAf(px, HAL_GPIO_AF_UART);
00207     HAL_GPIO_SetMode(px, HAL_GPIO_MODE_OUT);
00208 }
00209 void HAL_UART_SetGpioRx(HAL_UART_x uart_x, HAL_GPIO_Px px)
00210 {
00211     (void)uart_x;
00212     HAL_GPIO_SetAf(px, HAL_GPIO_AF_UART);
00213     HAL_GPIO_SetMode(px, HAL_GPIO_MODE_IN);
00214 }
00215

```

```

00216 // uart发送完成中断
00217 uint8_t HAL_UART_IrqSendOver(HAL_UART_TypeDef* uartx)
00218 {
00219     if (HAL_UART[uartx]-->IE & HAL_UART[uartx]-->IF & BIT0)
00220     {
00221         return 1;
00222     }
00223     else
00224     {
00225         return 0;
00226     }
00227 }
00228 void HAL_UART_IrqSendOverEnable(HAL_UART_TypeDef* uartx)
00229 {
00230     HAL_REG_SET(HAL_UART[uartx]-->IE, BIT0);
00231 }
00232 void HAL_UART_IrqSendOverDisable(HAL_UART_TypeDef* uartx)
00233 {
00234     HAL_REG_RESET(HAL_UART[uartx]-->IE, BIT0);
00235 }
00236 void HAL_UART_IrqSendOverClear(HAL_UART_TypeDef* uartx)
00237 {
00238     HAL_REG_WRITE(HAL_UART[uartx]-->IF, BIT0);
00239 }
00240
00241 // uart接受完成中断
00242 uint8_t HAL_UART_IrqReceiveOver(HAL_UART_TypeDef* uartx)
00243 {
00244     if (HAL_UART[uartx]-->IE & HAL_UART[uartx]-->IF & UARTx_IF_RX_DONE_BIT)
00245     {
00246         return 1;
00247     }
00248     else
00249     {
00250         return 0;
00251     }
00252 }
00253 void HAL_UART_IrqReceiveOverEnable(HAL_UART_TypeDef* uartx)
00254 {
00255     HAL_REG_SET(HAL_UART[uartx]-->IE, UARTx_IE_RX_DONE_BIT);
00256 }
00257 void HAL_UART_IrqReceiveOverDisable(HAL_UART_TypeDef* uartx)
00258 {
00259     HAL_REG_RESET(HAL_UART[uartx]-->IE, UARTx_IE_RX_DONE_BIT);
00260 }
00261 void HAL_UART_IrqReceiveOverClear(HAL_UART_TypeDef* uartx)
00262 {
00263     HAL_REG_WRITE(HAL_UART[uartx]-->IF, UARTx_IF_RX_DONE_BIT);
00264 }
00265
00266 // uart停止位错误中断
00267 uint8_t HAL_UART_IrqStopErr(HAL_UART_TypeDef* uartx)
00268 {
00269     if (HAL_UART[uartx]-->IE & HAL_UART[uartx]-->IF & UARTx_IF_STOP_ERR_BIT)
00270     {
00271         return 1;
00272     }
00273     else
00274     {
00275         return 0;
00276     }
00277 }
00278 void HAL_UART_IrqStopErrEnable(HAL_UART_TypeDef* uartx)
00279 {
00280     HAL_REG_SET(HAL_UART[uartx]-->IE, UARTx_IE_STOP_ERR_BIT);
00281 }
00282 void HAL_UART_IrqStopErrDisable(HAL_UART_TypeDef* uartx)
00283 {
00284     HAL_REG_RESET(HAL_UART[uartx]-->IE, UARTx_IE_STOP_ERR_BIT);
00285 }
00286 void HAL_UART_IrqStopErrClear(HAL_UART_TypeDef* uartx)
00287 {
00288     HAL_REG_WRITE(HAL_UART[uartx]-->IF, UARTx_IF_STOP_ERR_BIT);
00289 }
00290
00291 // uart校验错误中断
00292 uint8_t HAL_UART_IrqCheckErr(HAL_UART_TypeDef* uartx)
00293 {
00294     if (HAL_UART[uartx]-->IE & HAL_UART[uartx]-->IF & BIT4)
00295     {
00296         return 1;
00297     }
00298     else
00299     {
00300         return 0;
00301     }
00302 }

```

```

00303 void HAL_UART_IrqCheckErrEnable(HAL_UART_x uart_x)
00304 {
00305     HAL_REG_SET(HAL_UART[uart_x]->IE, BIT4);
00306 }
00307 void HAL_UART_IrqCheckErrDisable(HAL_UART_x uart_x)
00308 {
00309     HAL_REG_RESET(HAL_UART[uart_x]->IE, BIT4);
00310 }
00311 void HAL_UART_IrqCheckErrClear(HAL_UART_x uart_x)
00312 {
00313     HAL_REG_WRITE(HAL_UART[uart_x]->IF, BIT4);
00314 }

```

4.116 source/hal_lks32mc07x_wake.c 文件参考

```

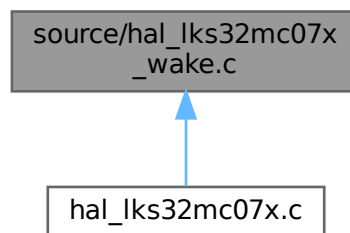
#include "../include/hal_lksmcu_iwdg.h"
#include "../include/hal_lksmcu_wake.h"
#include "../hal_lks32mc07x.h"
#include "lks32mc07x_user_manual.h"
#include "../include/hal_lksmcu_reg.h"
#include "../include/hal_lks32mc07x_reg.h"

```

hal_lks32mc07x_wake.c 的引用(Include)关系图:



此图展示该文件直接或间接的被哪些文件引用了:



函数

- void **HAL_WAKE_DeepSleepIoWake** (HAL_WAKE_UP_IO_x wake_up_io)
进入深度休眠,并且使用IO唤醒
- void **HAL_WAKE_DeepSleepTimeWakeUp** (uint32_t ms)
进入深度休眠,并且使用定时唤醒
- void **HAL_WAKE_DeepSleepIoTimeWakeUp** (HAL_WAKE_UP_IO_x wake_up_io, uint32_t ms)
进入深度休眠,并且使用IO唤醒和定时唤醒

4.116.1 函数说明

4.116.1.1 HAL_WAKE_DeepSleepIoTimeWakeUp()

```
void HAL_WAKE_DeepSleepIoTimeWakeUp (
    HAL_WAKE_UP_IO_x wake_up_io,
    uint32_t ms )
```

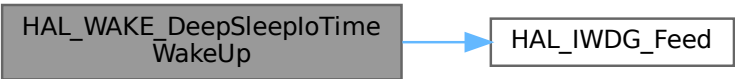
进入深度休眠,并且使用IO唤醒和定时唤醒

参数

<i>wake_up_io</i>	指定唤醒IO引脚及其电平
<i>ms</i>	指定定时唤醒的时间,单位为毫秒

在文件 [hal_lks32mc07x.wake.c](#) 第 128 行定义.

函数调用图:



4.116.1.2 HAL_WAKE_DeepSleepIoWake()

```
void HAL_WAKE_DeepSleepIoWake (
    HAL_WAKE_UP_IO_x wake_up_io )
```

进入深度休眠,并且使用IO唤醒

参数

<i>wake_up_io</i>	指定唤醒IO引脚及其电平
-------------------	--------------

在文件 [hal_lks32mc07x.wake.c](#) 第 12 行定义.

4.116.1.3 HAL_WAKE_DeepSleepTimeWakeUp()

```
void HAL_WAKE_DeepSleepTimeWakeUp (
    uint32_t ms )
```

进入深度休眠,并且使用定时唤醒

参数

ms	指定定时唤醒的时间,单位为毫秒
-----------	-----------------

在文件 [hal_lks32mc07x_wake.c](#) 第 65 行定义.

函数调用图:



4.117 hal_lks32mc07x_wake.c

[浏览该文件的文档.](#)

```

00001 #include "../include/hal_lksmcu_iwdg.h"
00002 #include "../include/hal_lksmcu_wake.h"
00003 #include "../hal_lks32mc07x.h"
00004 #include "lks32mc07x_user_manual.h"
00005 #include "../include/hal_lksmcu_reg.h"
00006 #include "../include/hal_lks32mc07x_reg.h"
00007
00012 void HAL_WAKE_DeepSleepIoWake(HAL_WAKE_UP_IO_x wake_up_io)
00013 {
00014     (void)wake_up_io;
00015     u32 t_cnt;
00016     uint32_t reg5;          // 模拟配置
00017     uint32_t clkcfg;        // 时钟配置
00018     uint32_t clkfen;        // 模块使能配置
00019     uint32_t saved_enable;  // 中断配置
00020
00021     __disable_irq();        // 关闭全局中断，确保执行休眠期间，不会进入其他任何中断
00022     reg5 = SYS_AFE_REG5;    // 保存模拟使能配置
00023     clkcfg = SYS_CLK_CFG;   // 保存时钟配置
00024     clkfen = SYS_CLK_FEN;   // 保存模块时钟使能配置
00025     saved_enable = NVIC->ISER[0]; // 保存当前中断配置
00026
00027     SYS_WR_PROTECT = 0x7a83; // 开启寄存器写使能
00028     NVIC->ICER[0] = 0xFFFFFFFF; // 关闭所有中断
00029     NVIC_EnableIRQ(WAKE_IRQn); // 打开唤醒中断
00030     IWDG_PSW = 0xA6B4;       // 打开看门狗使能
00031     IWDG_CLR = 0x798D;       // 喂狗
00032     SYS_CLK_CFG = 0;         // 切换到HRC时钟
00033     for (t_cnt = 0; t_cnt < 4000; t_cnt++) // 等待时钟稳定
00034     {
00035         __NOP();
00036     }
00037     SYS_CLK_FEN = 0;         // 关闭数字模块时钟
00038     SYS_AFE_REG5 = BIT10 | BIT8; // 准备关闭 PLL, BGP, 以及其它模拟电路使能
00039     SYS_WR_PROTECT = 0x0;    // 关闭系统寄存器写使能
00040     __enable_irq();          // 打开全局中断
00041     SCB->SCR |= (1UL << 2); // 使能深度休眠
00042
00043     __WFI();                 // 休眠 wakeak
00044     __disable_irq();         // 唤醒后关闭全局中断
00045     NVIC_DisableIRQ(WAKE_IRQn); // 关闭唤醒中断
00046
00047     SYS_WR_PROTECT = 0x7a83; // 开启寄存器写使能
00048     SYS_AFE_REG5 = reg5;     // 开启 PLL, BGP
00049
00050     for (t_cnt = 0; t_cnt < 400; t_cnt++) // 此时使用hrc，等待>1ms，确保PLL稳定
00051     {
00052         __NOP();
00053     }
00054     SYS_CLK_CFG = clkcfg;    // 恢复时钟配置
00055     NVIC->ISER[0] = saved_enable; // 恢复中断使能
  
```

```

00056     SYS_CLK_FEN    = clk_fen;           // 打开其他模块
00057     SYS_WR_PROTECT = 0x0;               // 打开写保护
00058     __enable_irq();                      // 使能全局中断
00059 }
00060
00065 void HAL_WAKE_DeepSleepTimeWakeUp(uint32_t ms)
00066 {
00067     u32 t_cnt;
00068     uint32_t reg5;           // 模拟配置
00069     uint32_t clkcfg;         // 时钟配置
00070     uint32_t clk_fen;        // 模块使能配置
00071     uint32_t saved_enable;   // 中断配置
00072
00073     __disable_irq();         // 关闭全局中断，确保执行休眠期间，不会进入其他任何中断
00074     reg5 = SYS_AFE_REG5;     // 保存模拟使能配置
00075     clkcfg = SYS_CLK_CFG;    // 保存时钟配置
00076     clk_fen = SYS_CLK_FEN;   // 保存模块时钟使能配置
00077     saved_enable = NVIC->ISER[0]; // 保存当前中断配置
00078
00079     SYS_WR_PROTECT = 0x7a83; // 开启寄存器写使能
00080     NVIC->ICER[0] = 0xFFFFFFFF; // 关闭所有中断
00081     NVIC_EnableIRQ(WAKE_IRQn); // 打开唤醒中断
00082     {
00083         int32_t th;
00084         th = ((uint32_t)(32 * ms + 0x800) & 0x1fff000);
00085         th = HAL_REG_READ(HAL_IWDG[HAL_IWDG_0]->RTH) - th;
00086         if (th < 0)
00087         {
00088             th = 0;
00089         }
00090         HAL_REG_WRITE(HAL_IWDG[HAL_IWDG_0]->WTH, th);
00091     }
00092     HAL_IWDG_Feed(HAL_IWDG_0); // 喂狗
00093     SYS_CLK_CFG = 0;           // 切换到HRC时钟
00094     for (t_cnt = 0; t_cnt < 4000; t_cnt++) // 等待时钟稳定
00095     {
00096         __NOP();
00097     }
00098     SYS_CLK_FEN = 0;           // 关闭数字模块时钟
00099     SYS_AFE_REG5 = BIT10 | BIT8; // 准备关闭 PLL, BGP, 以及其它模拟电路使能
00100     SYS_WR_PROTECT = 0x0;      // 关闭系统寄存器写使能
00101
00102     __enable_irq();           // 打开全局中断
00103     SCB->SCR |= (1UL << 2); // 使能深度休眠
00104
00105     __WFI();                  // 休眠 wakeak
00106     __disable_irq();          // 唤醒后关闭全局中断
00107     NVIC_DisableIRQ(WAKE_IRQn); // 关闭唤醒中断
00108
00109     SYS_WR_PROTECT = 0x7a83; // 开启寄存器写使能
00110     SYS_AFE_REG5 = reg5;     // 开启 PLL, BGP
00111
00112     for (t_cnt = 0; t_cnt < 400; t_cnt++) // 此时使用hrc，等待>1ms，确保PLL稳定
00113     {
00114         __NOP();
00115     }
00116     SYS_CLK_CFG = clkcfg;     // 恢复时钟配置
00117     NVIC->ISER[0] = saved_enable; // 恢复中断使能
00118     SYS_CLK_FEN = clk_fen;    // 打开其他模块
00119     SYS_WR_PROTECT = 0x0;     // 打开写保护
00120     __enable_irq();           // 使能全局中断
00121 }
00122
00128 void HAL_WAKE_DeepSleepIoTimeWakeUp(HAL_WAKE_UP_IO_x wake_up_io, uint32_t ms)
00129 {
00130     u32 t_cnt;
00131     uint32_t reg5;           // 模拟配置
00132     uint32_t clkcfg;         // 时钟配置
00133     uint32_t clk_fen;        // 模块使能配置
00134     uint32_t saved_enable;   // 中断配置
00135
00136     __disable_irq();         // 关闭全局中断，确保执行休眠期间，不会进入其他任何中断
00137     reg5 = SYS_AFE_REG5;     // 保存模拟使能配置
00138     clkcfg = SYS_CLK_CFG;    // 保存时钟配置
00139     clk_fen = SYS_CLK_FEN;   // 保存模块时钟使能配置
00140     saved_enable = NVIC->ISER[0]; // 保存当前中断配置
00141
00142     SYS_WR_PROTECT = 0x7a83; // 开启寄存器写使能
00143     NVIC->ICER[0] = 0xFFFFFFFF; // 关闭所有中断
00144     NVIC_EnableIRQ(WAKE_IRQn); // 打开唤醒中断
00145     {
00146         int32_t th;
00147         th = ((uint32_t)(64 * ms + 0x800) & 0x1fff000);
00148         th = HAL_REG_READ(HAL_IWDG[HAL_IWDG_0]->RTH) - th;
00149         if (th < 0)
00150         {
00151             th = 0;

```

```

00152     }
00153     HAL_REG_WRITE(HAL_IWDG[HAL_IWDG_0]->WTH, th);
00154 }
00155 HAL_IWDG_Feed(HAL_IWDG_0);           // 喂狗
00156 SYS_CLK_CFG = 0;                     // 切换到HRC时钟
00157 for (t_cnt = 0; t_cnt < 4000; t_cnt++) // 等待时钟稳定
00158 {
00159     __NOP();
00160 }
00161 SYS_CLK_FEN = 0;                     // 关闭数字模块时钟
00162 SYS_AFE_REG5 = BIT10 | BIT8;         // 准备关闭 PLL,BGP, 以及其它模拟电路使能
00163 SYS_WR_PROTECT = 0x0;               // 关闭系统寄存器写使能
00164
00165 AON_IO_WAKE_POL = wake_up_io >> 8;  // 设置IO唤醒电平
00166 AON_IO_WAKE_EN = wake_up_io & 0xFF; // 设置IO唤醒引脚
00167
00168 __enable_irq();                      // 打开全局中断
00169 SCB->SCR |= (1UL << 2);             // 使能深度休眠
00170
00171 __WFI();                             // 休眠 wakeak
00172 __disable_irq();                     // 唤醒后关闭全局中断
00173 NVIC_DisableIRQ(WAKE_IRQn);         // 关闭唤醒中断
00174
00175 SYS_WR_PROTECT = 0x7a83;             // 开启寄存器写使能
00176 SYS_AFE_REG5 = reg5;                // 开启 PLL,BGP
00177
00178 for (t_cnt = 0; t_cnt < 400; t_cnt++) // 此时使用hrc, 等待>1mS, 确保PLL稳定
00179 {
00180     __NOP();
00181 }
00182 SYS_CLK_CFG = clkcfg;               // 恢复时钟配置
00183 NVIC->ISER[0] = saved_enable;        // 恢复中断使能
00184 SYS_CLK_FEN = clk_fen;              // 打开其他模块
00185 SYS_WR_PROTECT = 0x0;               // 打开写保护
00186 __enable_irq();                     // 使能全局中断
00187 }

```


Index

__HAL_LKS32MC05X_C__
 hal_lks32mc07x.c, [56](#)

ADC
 HAL_SYS_TypeDef, [43](#)

adc_soft_reg
 hal_lks32mc07x_adc.c, [265](#)

ADDR
 HAL_FLASH_TypeDef, [20](#)
 HAL_I2C_TypeDef, [28](#)

ADR
 HAL_UART_TypeDef, [53](#)

AMC0
 HAL_ADC_TypeDef, [9](#)

AMC1
 HAL_ADC_TypeDef, [9](#)

ARCTAN
 HAL_DSP_TypeDef, [18](#)

AUEN
 HAL_MCPWM_TypeDef, [32](#)

BAUD
 HAL_SPI_TypeDef, [41](#)

BCR
 HAL_I2C_TypeDef, [28](#)

BLCWIN
 CMP_TypeDef, [5](#)
 HAL_CMP_TypeDef, [13](#)

BRR
 HAL_GPIO_TypeDef, [24](#)

BSIZE
 HAL_I2C_TypeDef, [28](#)

BSRR
 HAL_GPIO_TypeDef, [24](#)

BUFF
 HAL_UART_TypeDef, [53](#)

CCR
 HAL_DMA_TypeDef, [16](#)

CFG
 CMP_TypeDef, [5](#)
 HAL_ADC_TypeDef, [9](#)
 HAL_CMP_TypeDef, [13](#)
 HAL_FLASH_TypeDef, [20](#)
 HAL_HALL_TypeDef, [26](#)
 HAL_I2C_TypeDef, [28](#)
 HAL_IWDG_TypeDef, [29](#)
 HAL_QEP_TypeDef, [40](#)
 HAL_SPI_TypeDef, [41](#)
 HAL_TIMER_TypeDef, [51](#)

CH_DEF
 HAL_MCPWM_TypeDef, [32](#)

CH_MSK
 HAL_MCPWM_TypeDef, [33](#)

CHN0
 HAL_ADC_TypeDef, [9](#)

CHN1
 HAL_ADC_TypeDef, [9](#)

CHN2
 HAL_ADC_TypeDef, [9](#)

CHN3
 HAL_ADC_TypeDef, [10](#)

CHNT
 HAL_ADC_TypeDef, [10](#)

CLK_CFG
 HAL_SYS_TypeDef, [43](#)

CLK_DIV0
 HAL_SYS_TypeDef, [44](#)

CLK_DIV1
 HAL_SYS_TypeDef, [44](#)

CLK_DIV2
 HAL_SYS_TypeDef, [44](#)

CLK_FEN
 HAL_SYS_TypeDef, [44](#)

CLR
 HAL_IWDG_TypeDef, [29](#)

CMP0
 HAL_TIMER_TypeDef, [51](#)

CMP1
 HAL_TIMER_TypeDef, [51](#)

CMP_TypeDef, [5](#)
 BLCWIN, [5](#)
 CFG, [5](#)
 DATA, [6](#)
 IE, [6](#)
 IF, [6](#)
 TCLK, [6](#)

CNT
 HAL_HALL_TypeDef, [26](#)
 HAL_IWDG_TypeDef, [30](#)
 HAL_QEP_TypeDef, [40](#)
 HAL_TIMER_TypeDef, [51](#)

CNT0
 HAL_MCPWM_TypeDef, [33](#)

CNT1
 HAL_MCPWM_TypeDef, [33](#)

COS
 HAL_DSP_TypeDef, [18](#)

CR

HAL_CRC_TypeDef, 15
 CRC_Model_Params, 6
 initValue, 7
 polynomial, 7
 reflectIn, 7
 reflectOut, 7
 xorOut, 7
 crcModels
 hal_lks32mc07x_crc.c, 283
 CTMS
 HAL_DMA_TypeDef, 16
 CTRL
 HAL_UART_TypeDef, 53

 DAC0
 HAL_SYS_TypeDef, 44
 DAC0_AMC
 HAL_SYS_TypeDef, 44
 DAC0_DC
 HAL_SYS_TypeDef, 44
 DAC1
 HAL_SYS_TypeDef, 44
 DAC1_AMC
 HAL_SYS_TypeDef, 45
 DAC1_DC
 HAL_SYS_TypeDef, 45
 DAC_CTRL
 HAL_SYS_TypeDef, 45
 DADR
 HAL_DMA_TypeDef, 16
 DAT
 HAL_ADC_TypeDef, 10
 DATA
 CMP_TypeDef, 6
 HAL_CMP_TypeDef, 14
 HAL_I2C_TypeDef, 28
 DBG
 HAL_SYS_TypeDef, 45
 DBG_CFG
 HAL_SYS_TypeDef, 45
 DC0
 HAL_ADC_TypeDef, 10
 DC1
 HAL_ADC_TypeDef, 10
 DID
 HAL_DSP_TypeDef, 18
 DIS
 HAL_DSP_TypeDef, 18
 DIVH
 HAL_UART_TypeDef, 54
 DIVL
 HAL_UART_TypeDef, 54
 DR
 HAL_CRC_TypeDef, 15
 DTH00
 HAL_MCPWM_TypeDef, 33
 DTH01
 HAL_MCPWM_TypeDef, 33
 DTH10
 HAL_MCPWM_TypeDef, 33
 DTH11
 HAL_MCPWM_TypeDef, 34

 EIE
 HAL_MCPWM_TypeDef, 34
 EIF
 HAL_MCPWM_TypeDef, 34
 ERASE
 HAL_FLASH_TypeDef, 21
 EVT
 HAL_TIMER_TypeDef, 51
 EVT0
 HAL_MCPWM_TypeDef, 34
 EVT1
 HAL_MCPWM_TypeDef, 34

 F3210
 HAL_GPIO_TypeDef, 24
 F7654
 HAL_GPIO_TypeDef, 24
 FAIL012
 HAL_MCPWM_TypeDef, 34
 FAIL345
 HAL_MCPWM_TypeDef, 35
 FBA98
 HAL_GPIO_TypeDef, 24
 FCNT
 HAL_MCPWM_TypeDef, 35
 FFEDC
 HAL_GPIO_TypeDef, 24
 FLSE
 HAL_SYS_TypeDef, 45
 FLSP
 HAL_SYS_TypeDef, 45
 FLST
 HAL_SYS_TypeDef, 45
 FLT
 HAL_MCPWM_TypeDef, 35
 HAL_TIMER_TypeDef, 51

 GAIN
 HAL_ADC_TypeDef, 10
 GEN
 HAL_ADC_TypeDef, 11

 HAL_ADC
 hal_lks32mc07x_reg.c, 389
 hal_lks32mc07x_reg.h, 70
 HAL_ADC_0
 hal_lksmcu_adc.h, 81
 HAL_ADC_1
 hal_lksmcu_adc.h, 81
 HAL_ADC_2
 hal_lksmcu_adc.h, 81
 HAL_ADC_3
 hal_lksmcu_adc.h, 81
 HAL_ADC_Disable
 hal_lks32mc07x_adc.c, 255

hal_lksmcu_adc.h, 81

HAL_ADC_Enable
hal_lks32mc07x_adc.c, 255
hal_lksmcu_adc.h, 81

HAL_ADC_EndSampling
hal_lks32mc07x_adc.c, 255
hal_lksmcu_adc.h, 82

HAL_ADC_GetRange
hal_lks32mc07x_adc.c, 256
hal_lksmcu_adc.h, 82

HAL_ADC_GetSamplingValues
hal_lks32mc07x_adc.c, 256
hal_lksmcu_adc.h, 83

HAL_ADC_GetValuesUnit
hal_lks32mc07x_adc.c, 257
hal_lksmcu_adc.h, 83

HAL_ADC_IRQHandler
hal_lks32mc07x.h, 58

HAL_ADC_IRQHandler_00
hal_lks32mc07x.h, 58

HAL_ADC_IRQHandler_10
hal_lks32mc07x.h, 59

HAL_ADC_IRQHandler_20
hal_lks32mc07x.h, 59

HAL_ADC_IrqSamp1
hal_lks32mc07x_irqfun.c, 344
hal_lksmcu_irqfun.h, 164

HAL_ADC_IrqSamp1Clear
hal_lks32mc07x_irqfun.c, 345
hal_lksmcu_irqfun.h, 165

HAL_ADC_IrqSamp1Disable
hal_lks32mc07x_irqfun.c, 345
hal_lksmcu_irqfun.h, 165

HAL_ADC_IrqSamp1Enable
hal_lks32mc07x_irqfun.c, 345
hal_lksmcu_irqfun.h, 165

HAL_ADC_IrqSamp1Flg
hal_lks32mc07x_irqfun.c, 346
hal_lksmcu_irqfun.h, 165

HAL_ADC_IrqSamp2
hal_lks32mc07x_irqfun.c, 346
hal_lksmcu_irqfun.h, 166

HAL_ADC_IrqSamp2Clear
hal_lks32mc07x_irqfun.c, 346
hal_lksmcu_irqfun.h, 166

HAL_ADC_IrqSamp2Disable
hal_lks32mc07x_irqfun.c, 347
hal_lksmcu_irqfun.h, 166

HAL_ADC_IrqSamp2Enable
hal_lks32mc07x_irqfun.c, 347
hal_lksmcu_irqfun.h, 167

HAL_ADC_IrqSamp2Flg
hal_lks32mc07x_irqfun.c, 347
hal_lksmcu_irqfun.h, 167

HAL_ADC_ManualSampling
hal_lks32mc07x_adc.c, 258
hal_lksmcu_adc.h, 84

HAL_ADC_PORT_AGND
hal_lksmcu_adc.h, 79

HAL_ADC_PORT_CH0
hal_lksmcu_adc.h, 78

HAL_ADC_PORT_CH1
hal_lksmcu_adc.h, 78

HAL_ADC_PORT_CH10
hal_lksmcu_adc.h, 78

HAL_ADC_PORT_CH11
hal_lksmcu_adc.h, 78

HAL_ADC_PORT_CH12
hal_lksmcu_adc.h, 78

HAL_ADC_PORT_CH13
hal_lksmcu_adc.h, 78

HAL_ADC_PORT_CH14
hal_lksmcu_adc.h, 78

HAL_ADC_PORT_CH15
hal_lksmcu_adc.h, 78

HAL_ADC_PORT_CH16
hal_lksmcu_adc.h, 79

HAL_ADC_PORT_CH17
hal_lksmcu_adc.h, 79

HAL_ADC_PORT_CH18
hal_lksmcu_adc.h, 79

HAL_ADC_PORT_CH19
hal_lksmcu_adc.h, 79

HAL_ADC_PORT_CH2
hal_lksmcu_adc.h, 78

HAL_ADC_PORT_CH3
hal_lksmcu_adc.h, 78

HAL_ADC_PORT_CH4
hal_lksmcu_adc.h, 78

HAL_ADC_PORT_CH5
hal_lksmcu_adc.h, 78

HAL_ADC_PORT_CH6
hal_lksmcu_adc.h, 78

HAL_ADC_PORT_CH7
hal_lksmcu_adc.h, 78

HAL_ADC_PORT_CH8
hal_lksmcu_adc.h, 78

HAL_ADC_PORT_CH9
hal_lksmcu_adc.h, 78

HAL_ADC_PORT_DAC0
hal_lksmcu_adc.h, 79

HAL_ADC_PORT_DAC1
hal_lksmcu_adc.h, 79

HAL_ADC_PORT_LDO12
hal_lksmcu_adc.h, 79

HAL_ADC_PORT_OPA0
hal_lksmcu_adc.h, 78

HAL_ADC_PORT_OPA1
hal_lksmcu_adc.h, 79

HAL_ADC_PORT_OPA2
hal_lksmcu_adc.h, 79

HAL_ADC_PORT_OPA3
hal_lksmcu_adc.h, 79

HAL_ADC_PORT_OPA4
hal_lksmcu_adc.h, 79

HAL_ADC_PORT_OPA5

hal_lksmcu_adc.h, 79
 HAL_ADC_PORT_REF
 hal_lksmcu_adc.h, 79
 HAL_ADC_PORT_TMP0
 hal_lksmcu_adc.h, 79
 HAL_ADC_PORT_VDD
 hal_lksmcu_adc.h, 79
 HAL_ADC_PORT_x
 hal_lksmcu_adc.h, 78
 HAL_ADC_REFERENCE_EXTERNAL
 hal_lksmcu_adc.h, 79
 HAL_ADC_REFERENCE_INTERNAL
 hal_lksmcu_adc.h, 79
 HAL_ADC_REFERENCE_x
 hal_lksmcu_adc.h, 79
 HAL_ADC_Reset
 hal_lks32mc07x_adc.c, 258
 hal_lksmcu_adc.h, 85
 HAL_ADC_SAMP_NUMBER_0
 hal_lksmcu_adc.h, 79
 HAL_ADC_SAMP_NUMBER_1
 hal_lksmcu_adc.h, 79
 HAL_ADC_SAMP_NUMBER_10
 hal_lksmcu_adc.h, 80
 HAL_ADC_SAMP_NUMBER_11
 hal_lksmcu_adc.h, 80
 HAL_ADC_SAMP_NUMBER_12
 hal_lksmcu_adc.h, 80
 HAL_ADC_SAMP_NUMBER_13
 hal_lksmcu_adc.h, 80
 HAL_ADC_SAMP_NUMBER_14
 hal_lksmcu_adc.h, 80
 HAL_ADC_SAMP_NUMBER_15
 hal_lksmcu_adc.h, 80
 HAL_ADC_SAMP_NUMBER_2
 hal_lksmcu_adc.h, 79
 HAL_ADC_SAMP_NUMBER_3
 hal_lksmcu_adc.h, 79
 HAL_ADC_SAMP_NUMBER_4
 hal_lksmcu_adc.h, 79
 HAL_ADC_SAMP_NUMBER_5
 hal_lksmcu_adc.h, 79
 HAL_ADC_SAMP_NUMBER_6
 hal_lksmcu_adc.h, 79
 HAL_ADC_SAMP_NUMBER_7
 hal_lksmcu_adc.h, 80
 HAL_ADC_SAMP_NUMBER_8
 hal_lksmcu_adc.h, 80
 HAL_ADC_SAMP_NUMBER_9
 hal_lksmcu_adc.h, 80
 HAL_ADC_SAMP_NUMBER_MAX
 hal_lksmcu_adc.h, 80
 HAL_ADC_SAMP_NUMBER_x
 hal_lksmcu_adc.h, 79
 HAL_ADC_SAMP_SEQUENCE_0
 hal_lksmcu_adc.h, 80
 HAL_ADC_SAMP_SEQUENCE_1
 hal_lksmcu_adc.h, 80
 HAL_ADC_SAMP_SEQUENCE_10
 hal_lksmcu_adc.h, 80
 HAL_ADC_SAMP_SEQUENCE_11
 hal_lksmcu_adc.h, 80
 HAL_ADC_SAMP_SEQUENCE_12
 hal_lksmcu_adc.h, 80
 HAL_ADC_SAMP_SEQUENCE_13
 hal_lksmcu_adc.h, 80
 HAL_ADC_SAMP_SEQUENCE_14
 hal_lksmcu_adc.h, 80
 HAL_ADC_SAMP_SEQUENCE_15
 hal_lksmcu_adc.h, 80
 HAL_ADC_SAMP_SEQUENCE_16
 hal_lksmcu_adc.h, 80
 HAL_ADC_SAMP_SEQUENCE_17
 hal_lksmcu_adc.h, 80
 HAL_ADC_SAMP_SEQUENCE_18
 hal_lksmcu_adc.h, 80
 HAL_ADC_SAMP_SEQUENCE_19
 hal_lksmcu_adc.h, 80
 HAL_ADC_SAMP_SEQUENCE_2
 hal_lksmcu_adc.h, 80
 HAL_ADC_SAMP_SEQUENCE_3
 hal_lksmcu_adc.h, 80
 HAL_ADC_SAMP_SEQUENCE_4
 hal_lksmcu_adc.h, 80
 HAL_ADC_SAMP_SEQUENCE_5
 hal_lksmcu_adc.h, 80
 HAL_ADC_SAMP_SEQUENCE_6
 hal_lksmcu_adc.h, 80
 HAL_ADC_SAMP_SEQUENCE_7
 hal_lksmcu_adc.h, 80
 HAL_ADC_SAMP_SEQUENCE_8
 hal_lksmcu_adc.h, 80
 HAL_ADC_SAMP_SEQUENCE_9
 hal_lksmcu_adc.h, 80
 HAL_ADC_SAMP_SEQUENCE_x
 hal_lksmcu_adc.h, 80
 HAL_ADC_SetGpioIn
 hal_lks32mc07x_adc.c, 259
 hal_lksmcu_adc.h, 85
 HAL_ADC_SetGpioTriggerOut
 hal_lks32mc07x_adc.c, 259
 hal_lksmcu_adc.h, 86
 HAL_ADC_SetReference
 hal_lks32mc07x_adc.c, 260
 hal_lksmcu_adc.h, 86
 HAL_ADC_SetSamplingChannel
 hal_lks32mc07x_adc.c, 260
 hal_lksmcu_adc.h, 87
 HAL_ADC_SetSamplingQuantity
 hal_lks32mc07x_adc.c, 261
 hal_lksmcu_adc.h, 88
 HAL_ADC_SetTriggerSignal
 hal_lks32mc07x_adc.c, 262
 hal_lksmcu_adc.h, 88
 HAL_ADC_SoftTrigger
 hal_lks32mc07x_adc.c, 263

hal_lksmcu_adc.h, 89
 HAL_ADC_TRIGGER_MCPWM
 hal_lksmcu_adc.h, 81
 HAL_ADC_TRIGGER_SOFT
 hal_lksmcu_adc.h, 81
 HAL_ADC_TRIGGER_TIMER
 hal_lksmcu_adc.h, 81
 HAL_ADC_TRIGGER_x
 hal_lksmcu_adc.h, 80
 HAL_ADC_TypeDef, 7
 AMC0, 9
 AMC1, 9
 CFG, 9
 CHN0, 9
 CHN1, 9
 CHN2, 9
 CHN3, 10
 CHNT, 10
 DAT, 10
 DC0, 10
 DC1, 10
 GAIN, 10
 GEN, 11
 HTH, 11
 ICHN, 11
 IDAT, 11
 IE, 11
 IF, 11
 LTH, 12
 null, 12
 null2, 12
 null3, 12
 SWT, 12
 TRIG, 12
 HAL_ADC_UpdateTrig
 hal_lks32mc07x_adc.c, 263
 HAL_ADC_x
 hal_lksmcu_adc.h, 81
 hal_baud
 hal_lks32mc07x_i2c.c, 335
 HAL_CAN_IRQHandler
 hal_lks32mc07x.h, 59
 HAL_CAN_IRQHandler_00
 hal_lks32mc07x.h, 59
 HAL_CL_IRQHandler
 hal_lks32mc07x.h, 59
 HAL_CL_IRQHandler_00
 hal_lks32mc07x.h, 59
 HAL_CLK_FREQ_RC_12MHZ
 hal_lksmcu_clk.h, 94
 HAL_CLK_FREQ_RC_16MHZ
 hal_lksmcu_clk.h, 94
 HAL_CLK_FREQ_RC_18MHZ
 hal_lksmcu_clk.h, 94
 HAL_CLK_FREQ_RC_192MHZ
 hal_lksmcu_clk.h, 94
 HAL_CLK_FREQ_RC_24MHZ
 hal_lksmcu_clk.h, 94
 HAL_CLK_FREQ_RC_32MHZ
 hal_lksmcu_clk.h, 94
 HAL_CLK_FREQ_RC_36MHZ
 hal_lksmcu_clk.h, 94
 HAL_CLK_FREQ_RC_48MHZ
 hal_lksmcu_clk.h, 94
 HAL_CLK_FREQ_RC_4MHZ
 hal_lksmcu_clk.h, 94
 HAL_CLK_FREQ_RC_64MHZ
 hal_lksmcu_clk.h, 94
 HAL_CLK_FREQ_RC_6MHZ
 hal_lksmcu_clk.h, 94
 HAL_CLK_FREQ_RC_72MHZ
 hal_lksmcu_clk.h, 94
 HAL_CLK_FREQ_RC_8MHZ
 hal_lksmcu_clk.h, 94
 HAL_CLK_FREQ_RC_96MHZ
 hal_lksmcu_clk.h, 94
 HAL_CLK_FREQ_RC_9MHZ
 hal_lksmcu_clk.h, 94
 HAL_CLK_FREQ_x
 hal_lksmcu_clk.h, 94
 HAL_CLK_FREQ_XTAL_12MHZ
 hal_lksmcu_clk.h, 94
 HAL_CLK_FREQ_XTAL_192MHZ
 hal_lksmcu_clk.h, 94
 HAL_CLK_FREQ_XTAL_24MHZ
 hal_lksmcu_clk.h, 94
 HAL_CLK_FREQ_XTAL_48MHZ
 hal_lksmcu_clk.h, 94
 HAL_CLK_FREQ_XTAL_6MHZ
 hal_lksmcu_clk.h, 94
 HAL_CLK_FREQ_XTAL_96MHZ
 hal_lksmcu_clk.h, 94
 HAL_CLK_GetMcuFreq
 hal_lks32mc07x_clk.c, 270
 hal_lksmcu_clk.h, 94
 HAL_CLK_mclk
 hal_lks32mc07x_clk.c, 272
 HAL_CLK_SetMcuFreq
 hal_lks32mc07x_clk.c, 271
 hal_lksmcu_clk.h, 95
 HAL_CMP
 hal_lks32mc07x_reg.c, 389
 hal_lks32mc07x_reg.h, 70
 HAL_CMP_0
 hal_lksmcu_cmp.h, 99
 HAL_CMP_1
 hal_lksmcu_cmp.h, 99
 HAL_CMP_2
 hal_lksmcu_cmp.h, 99
 HAL_CMP_3
 hal_lksmcu_cmp.h, 100
 HAL_CMP_4
 hal_lksmcu_cmp.h, 100
 HAL_CMP_5
 hal_lksmcu_cmp.h, 100
 HAL_CMP_CHANNEL_IN_0

[hal_lksmcu_cmp.h](#), 98
 HAL_CMP_CHANNEL_IN_DAC
[hal_lksmcu_cmp.h](#), 98
 HAL_CMP_CHANNEL_IN_MID
[hal_lksmcu_cmp.h](#), 98
 HAL_CMP_CHANNEL_IN_REF
[hal_lksmcu_cmp.h](#), 98
 HAL_CMP_CHANNEL_INx
[hal_lksmcu_cmp.h](#), 98
 HAL_CMP_CHANNEL_IP0
[hal_lksmcu_cmp.h](#), 98
 HAL_CMP_CHANNEL_IP1
[hal_lksmcu_cmp.h](#), 98
 HAL_CMP_CHANNEL_IP2
[hal_lksmcu_cmp.h](#), 98
 HAL_CMP_CHANNEL_IP3
[hal_lksmcu_cmp.h](#), 98
 HAL_CMP_CHANNEL_IP4
[hal_lksmcu_cmp.h](#), 98
 HAL_CMP_CHANNEL_IP5
[hal_lksmcu_cmp.h](#), 99
 HAL_CMP_CHANNEL_IP6
[hal_lksmcu_cmp.h](#), 99
 HAL_CMP_CHANNEL_IP7
[hal_lksmcu_cmp.h](#), 99
 HAL_CMP_CHANNEL_IP_AVSS
[hal_lksmcu_cmp.h](#), 99
 HAL_CMP_CHANNEL_IP_DAC0
[hal_lksmcu_cmp.h](#), 98
 HAL_CMP_CHANNEL_IP_DAC1
[hal_lksmcu_cmp.h](#), 99
 HAL_CMP_CHANNEL_IP_OPA0.IN
[hal_lksmcu_cmp.h](#), 99
 HAL_CMP_CHANNEL_IP_OPA0.IP
[hal_lksmcu_cmp.h](#), 98
 HAL_CMP_CHANNEL_IP_OPA0.OUTN
[hal_lksmcu_cmp.h](#), 99
 HAL_CMP_CHANNEL_IP_OPA0.OUTPUT
[hal_lksmcu_cmp.h](#), 99
 HAL_CMP_CHANNEL_IP_OPA1.IN
[hal_lksmcu_cmp.h](#), 99
 HAL_CMP_CHANNEL_IP_OPA1.IP
[hal_lksmcu_cmp.h](#), 99
 HAL_CMP_CHANNEL_IP_OPA1.OUTN
[hal_lksmcu_cmp.h](#), 99
 HAL_CMP_CHANNEL_IP_OPA1.OUTPUT
[hal_lksmcu_cmp.h](#), 99
 HAL_CMP_CHANNEL_IP_OPA2.IN
[hal_lksmcu_cmp.h](#), 99
 HAL_CMP_CHANNEL_IP_OPA2.IP
[hal_lksmcu_cmp.h](#), 99
 HAL_CMP_CHANNEL_IP_OPA2.OUTN
[hal_lksmcu_cmp.h](#), 99
 HAL_CMP_CHANNEL_IP_OPA2.OUTPUT
[hal_lksmcu_cmp.h](#), 99
 HAL_CMP_CHANNEL_IP_OPA3.IN
[hal_lksmcu_cmp.h](#), 99
 HAL_CMP_CHANNEL_IP_OPA3.IP
[hal_lksmcu_cmp.h](#), 99
 HAL_CMP_CHANNEL_IP_OPA3.OUTN
[hal_lksmcu_cmp.h](#), 99
 HAL_CMP_CHANNEL_IP_OPA3.OUTPUT
[hal_lksmcu_cmp.h](#), 99
 HAL_CMP_CHANNEL_IP_OPA4.IN
[hal_lksmcu_cmp.h](#), 99
 HAL_CMP_CHANNEL_IP_OPA4.IP
[hal_lksmcu_cmp.h](#), 99
 HAL_CMP_CHANNEL_IP_OPA4.OUTN
[hal_lksmcu_cmp.h](#), 99
 HAL_CMP_CHANNEL_IP_OPA4.OUTPUT
[hal_lksmcu_cmp.h](#), 99
 HAL_CMP_CHANNEL_IP_OPA5.IN
[hal_lksmcu_cmp.h](#), 99
 HAL_CMP_CHANNEL_IP_OPA5.IP
[hal_lksmcu_cmp.h](#), 99
 HAL_CMP_CHANNEL_IP_OPA5.OUTN
[hal_lksmcu_cmp.h](#), 99
 HAL_CMP_CHANNEL_IP_OPA5.OUTPUT
[hal_lksmcu_cmp.h](#), 99
 HAL_CMP_CHANNEL_IP_OPA.OUT_CMP0
[hal_lksmcu_cmp.h](#), 99
 HAL_CMP_CHANNEL_IPx
[hal_lksmcu_cmp.h](#), 98
 HAL_CMP_Disable
[hal_lks32mc07x_cmp.c](#), 274
[hal_lksmcu_cmp.h](#), 100
 HAL_CMP_Enable
[hal_lks32mc07x_cmp.c](#), 274
[hal_lksmcu_cmp.h](#), 100
 HAL_CMP_GetData
[hal_lks32mc07x_cmp.c](#), 275
[hal_lksmcu_cmp.h](#), 101
 HAL_CMP_IRQHandler
[hal_lks32mc07x.h](#), 59
 HAL_CMP_IRQHandler_00
[hal_lks32mc07x.h](#), 59
 HAL_CMP_IRQHandler_10
[hal_lks32mc07x.h](#), 60
 HAL_CMP_IRQHandler_20
[hal_lks32mc07x.h](#), 60
 HAL_CMP_IRQHandler_30
[hal_lks32mc07x.h](#), 60
 HAL_CMP_IRQHandler_40
[hal_lks32mc07x.h](#), 60
 HAL_CMP_IRQHandler_50
[hal_lks32mc07x.h](#), 60
 HAL_CMP_Reset
[hal_lks32mc07x_cmp.c](#), 275
[hal_lksmcu_cmp.h](#), 101
 HAL_CMP_SetChannel
[hal_lks32mc07x_cmp.c](#), 276
[hal_lksmcu_cmp.h](#), 102
 HAL_CMP_SetDigitalFilter
[hal_lks32mc07x_cmp.c](#), 276
[hal_lksmcu_cmp.h](#), 102
 HAL_CMP_SetGpioIn

hal_lks32mc07x_cmp.c, 276
 hal_lksmcu_cmp.h, 102
 HAL_CMP_SetGpioOut
 hal_lks32mc07x_cmp.c, 277
 hal_lksmcu_cmp.h, 103
 HAL_CMP_TypeDef, 13
 BLCWIN, 13
 CFG, 13
 DATA, 14
 IE, 14
 IF, 14
 TCLK, 14
 HAL_CMP_x
 hal_lksmcu_cmp.h, 99
 HAL_CRC
 hal_lks32mc07x_reg.c, 390
 hal_lks32mc07x_reg.h, 70
 HAL_CRC_0
 hal_lksmcu_crc.h, 106
 HAL_CRC_Calc
 hal_lks32mc07x_crc.c, 281
 hal_lksmcu_crc.h, 106
 HAL_CRC_Disable
 hal_lks32mc07x_crc.c, 282
 hal_lksmcu_crc.h, 107
 HAL_CRC_Enable
 hal_lks32mc07x_crc.c, 282
 hal_lksmcu_crc.h, 107
 HAL_CRC_MODEL_CRC16_CCITT
 hal_lksmcu_crc.h, 106
 HAL_CRC_MODEL_CRC16_CCITT_FALSE
 hal_lksmcu_crc.h, 106
 HAL_CRC_MODEL_CRC16_DNP
 hal_lksmcu_crc.h, 106
 HAL_CRC_MODEL_CRC16_IBMx1
 hal_lksmcu_crc.h, 106
 HAL_CRC_MODEL_CRC16_MAXIM
 hal_lksmcu_crc.h, 106
 HAL_CRC_MODEL_CRC16_MODBU
 hal_lksmcu_crc.h, 106
 HAL_CRC_MODEL_CRC16_USBx1
 hal_lksmcu_crc.h, 106
 HAL_CRC_MODEL_CRC16_X25
 hal_lksmcu_crc.h, 106
 HAL_CRC_MODEL_CRC16_XMODEM
 hal_lksmcu_crc.h, 106
 HAL_CRC_MODEL_CRC32
 hal_lksmcu_crc.h, 106
 HAL_CRC_MODEL_CRC32_MPEG_2
 hal_lksmcu_crc.h, 106
 HAL_CRC_MODEL_CRC8
 hal_lksmcu_crc.h, 106
 HAL_CRC_MODEL_CRC8_ITU
 hal_lksmcu_crc.h, 106
 HAL_CRC_MODEL_CRC8_MAXIMx
 hal_lksmcu_crc.h, 106
 HAL_CRC_MODEL_CRC8_ROHC
 hal_lksmcu_crc.h, 106
 HAL_CRC_MODEL_x
 hal_lksmcu_crc.h, 105
 HAL_CRC_Reflect
 hal_lks32mc07x_crc.c, 282
 HAL_CRC_Reset
 hal_lks32mc07x_crc.c, 282
 hal_lksmcu_crc.h, 107
 HAL_CRC_TypeDef, 14
 CR, 15
 DR, 15
 INIT, 15
 POL, 15
 HAL_CRC_x
 hal_lksmcu_crc.h, 106
 HAL_DAC_0
 hal_lksmcu_dac.h, 109
 HAL_DAC_1
 hal_lksmcu_dac.h, 109
 HAL_DAC_2
 hal_lksmcu_dac.h, 109
 HAL_DAC_3
 hal_lksmcu_dac.h, 109
 HAL_DAC_Disable
 hal_lks32mc07x_dac.c, 285
 hal_lksmcu_dac.h, 109
 HAL_DAC_Enable
 hal_lks32mc07x_dac.c, 286
 hal_lksmcu_dac.h, 110
 HAL_DAC_Out2GpioDisable
 hal_lks32mc07x_dac.c, 286
 hal_lksmcu_dac.h, 110
 HAL_DAC_Out2GpioEnable
 hal_lks32mc07x_dac.c, 286
 hal_lksmcu_dac.h, 110
 HAL_DAC_Reset
 hal_lks32mc07x_dac.c, 286
 hal_lksmcu_dac.h, 110
 HAL_DAC_SetGpioOut
 hal_lks32mc07x_dac.c, 286
 hal_lksmcu_dac.h, 110
 HAL_DAC_SetOutVal
 hal_lks32mc07x_dac.c, 287
 hal_lksmcu_dac.h, 111
 HAL_DAC_SetOutVttagemV
 hal_lks32mc07x_dac.c, 287
 hal_lksmcu_dac.h, 111
 HAL_DAC_x
 hal_lksmcu_dac.h, 109
 HAL_DMA
 hal_lks32mc07x_reg.c, 390
 hal_lks32mc07x_reg.h, 70
 HAL_DMA_0
 hal_lksmcu_dma.h, 113
 HAL_DMA_1
 hal_lksmcu_dma.h, 113
 HAL_DMA_2
 hal_lksmcu_dma.h, 113
 HAL_DMA_3

hal_lksmcu_dma.h, 113

HAL_DMA_Disable
hal_lks32mc07x_dma.c, 291
hal_lksmcu_dma.h, 113

HAL_DMA_ena
hal_lks32mc07x_dma.c, 293

HAL_DMA_Enable
hal_lks32mc07x_dma.c, 291
hal_lksmcu_dma.h, 113

HAL_DMA_I2cRx
hal_lks32mc07x_dma.c, 291
hal_lksmcu_dma.h, 114

HAL_DMA_I2cTx
hal_lks32mc07x_dma.c, 291
hal_lksmcu_dma.h, 114

HAL_DMA_IRQHandler
hal_lks32mc07x.h, 60

HAL_DMA_IRQHandler_00
hal_lks32mc07x.h, 60

HAL_DMA_MemCopy
hal_lks32mc07x_dma.c, 292
hal_lksmcu_dma.h, 114

HAL_DMA_Reset
hal_lks32mc07x_dma.c, 292
hal_lksmcu_dma.h, 114

HAL_DMA_SpiRx
hal_lks32mc07x_dma.c, 292
hal_lksmcu_dma.h, 114

HAL_DMA_SpiRxTx
hal_lks32mc07x_dma.c, 292
hal_lksmcu_dma.h, 114

HAL_DMA_SpiTx
hal_lks32mc07x_dma.c, 292
hal_lksmcu_dma.h, 115

HAL_DMA_TypeDef, 16
CCR, 16
CTMS, 16
DADR, 16
REN, 16
SADR, 17

HAL_DMA_UartRx
hal_lks32mc07x_dma.c, 293
hal_lksmcu_dma.h, 115

HAL_DMA_UartTx
hal_lks32mc07x_dma.c, 293
hal_lksmcu_dma.h, 115

HAL_DMA_x
hal_lksmcu_dma.h, 113

HAL_DSP
hal_lks32mc07x_reg.c, 390
hal_lks32mc07x_reg.h, 71

HAL_DSP_0
hal_lksmcu_dsp.h, 117

HAL_DSP_Cos
hal_lks32mc07x_dsp.c, 296
hal_lksmcu_dsp.h, 117

HAL_DSP_Disable
hal_lks32mc07x_dsp.c, 296

hal_lksmcu_dsp.h, 117

HAL_DSP_Div
hal_lks32mc07x_dsp.c, 296
hal_lksmcu_dsp.h, 118

HAL_DSP_Enable
hal_lks32mc07x_dsp.c, 296
hal_lksmcu_dsp.h, 118

HAL_DSP_IRQHandler
hal_lks32mc07x.h, 60

HAL_DSP_IRQHandler_00
hal_lks32mc07x.h, 61

HAL_DSP_Reset
hal_lks32mc07x_dsp.c, 297
hal_lksmcu_dsp.h, 118

HAL_DSP_Sin
hal_lks32mc07x_dsp.c, 297
hal_lksmcu_dsp.h, 118

HAL_DSP_SinCos
hal_lks32mc07x_dsp.c, 297
hal_lksmcu_dsp.h, 118

HAL_DSP_Sqrt
hal_lks32mc07x_dsp.c, 297
hal_lksmcu_dsp.h, 118

HAL_DSP_TypeDef, 17
ARCTAN, 18
COS, 18
DID, 18
DIS, 18
MOD, 18
PC, 18
QUO, 18
RAD, 18
REM, 19
SC, 19
SIN, 19
SQRT, 19
THETA, 19
X, 19
Y, 19

HAL_DSP_x
hal_lksmcu_dsp.h, 117

HAL_FLASH
hal_lks32mc07x_reg.c, 390
hal_lks32mc07x_reg.h, 71

HAL_FLASH_0
hal_lksmcu_flash.h, 120

HAL_FLASH_1
hal_lksmcu_flash.h, 120

HAL_FLASH_data
hal_lks32mc07x_flash.c, 308

HAL_FLASH_Disable
hal_lks32mc07x_flash.c, 300
hal_lksmcu_flash.h, 121

HAL_FLASH_DisableCache
hal_lks32mc07x_flash.c, 300
hal_lksmcu_flash.h, 121

HAL_FLASH_Enable
hal_lks32mc07x_flash.c, 300

hal_lksmcu_flash.h, 121
 HAL_FLASH_EnableCache
 hal_lks32mc07x_flash.c, 301
 hal_lksmcu_flash.h, 121
 HAL_FLASH_Erase
 hal_lks32mc07x_flash.c, 301
 hal_lksmcu_flash.h, 122
 HAL_FLASH_Program
 hal_lks32mc07x_flash.c, 302
 hal_lksmcu_flash.h, 122
 HAL_FLASH_Read
 hal_lks32mc07x_flash.c, 302
 hal_lksmcu_flash.h, 123
 HAL_FLASH_ReadSectorSize
 hal_lks32mc07x_flash.c, 303
 hal_lksmcu_flash.h, 124
 HAL_FLASH_ReadUint16
 hal_lks32mc07x_flash.c, 303
 hal_lksmcu_flash.h, 124
 HAL_FLASH_ReadUint32
 hal_lks32mc07x_flash.c, 304
 hal_lksmcu_flash.h, 125
 HAL_FLASH_ReadUint8
 hal_lks32mc07x_flash.c, 304
 hal_lksmcu_flash.h, 125
 HAL_FLASH_Reset
 hal_lks32mc07x_flash.c, 305
 hal_lksmcu_flash.h, 126
 HAL_FLASH_SecurityParameterCrc
 hal_lks32mc07x_flash.c, 305
 HAL_FLASH_SecurityParameterRead
 hal_lks32mc07x_flash.c, 305
 hal_lksmcu_flash.h, 126
 HAL_FLASH_SecurityParameterSave
 hal_lks32mc07x_flash.c, 306
 hal_lksmcu_flash.h, 127
 HAL_FLASH_TypeDef, 20
 ADDR, 20
 CFG, 20
 ERASE, 21
 NADDR, 21
 NCFG, 21
 NERASE, 21
 NKEY, 21
 NRDATA, 21
 NWDATA, 22
 PROTECT, 22
 RDATA, 22
 READY, 22
 SIZE, 22
 WDATA, 22
 HAL_FLASH_Update
 hal_lks32mc07x_flash.c, 307
 hal_lksmcu_flash.h, 127
 HAL_FLASH_x
 hal_lksmcu_flash.h, 120
 HAL_FMAC_IRQHandler_00
 hal_lks32mc07x.h, 61
 HAL_GPIO
 hal_lks32mc07x_reg.c, 390
 hal_lks32mc07x_reg.h, 71
 HAL_GPIO_AF_ADC
 hal_lksmcu_gpio.h, 132
 HAL_GPIO_AF_ADC_TRIG
 hal_lksmcu_gpio.h, 132
 HAL_GPIO_AF_CAN
 hal_lksmcu_gpio.h, 132
 HAL_GPIO_AF_CL
 hal_lksmcu_gpio.h, 132
 HAL_GPIO_AF_CLK
 hal_lksmcu_gpio.h, 132
 HAL_GPIO_AF_CLK_OUT
 hal_lksmcu_gpio.h, 132
 HAL_GPIO_AF_CMP
 hal_lksmcu_gpio.h, 132
 HAL_GPIO_AF_CMP_OUT
 hal_lksmcu_gpio.h, 132
 HAL_GPIO_AF_DAC
 hal_lksmcu_gpio.h, 132
 HAL_GPIO_AF_GPIO
 hal_lksmcu_gpio.h, 132
 HAL_GPIO_AF_HALL
 hal_lksmcu_gpio.h, 132
 HAL_GPIO_AF_I2C
 hal_lksmcu_gpio.h, 132
 HAL_GPIO_AF_MCPWM
 hal_lksmcu_gpio.h, 132
 HAL_GPIO_AF_OPA
 hal_lksmcu_gpio.h, 132
 HAL_GPIO_AF_OPA_OUT
 hal_lksmcu_gpio.h, 132
 HAL_GPIO_AF_QEP
 hal_lksmcu_gpio.h, 132
 HAL_GPIO_AF_SIF
 hal_lksmcu_gpio.h, 132
 HAL_GPIO_AF_SPI
 hal_lksmcu_gpio.h, 132
 HAL_GPIO_AF_TIMER0
 hal_lksmcu_gpio.h, 132
 HAL_GPIO_AF_TIMER1
 hal_lksmcu_gpio.h, 132
 HAL_GPIO_AF_TIMER2
 hal_lksmcu_gpio.h, 132
 HAL_GPIO_AF_TIMER3
 hal_lksmcu_gpio.h, 132
 HAL_GPIO_AF_TIMER4
 hal_lksmcu_gpio.h, 132
 HAL_GPIO_AF_TIMER5
 hal_lksmcu_gpio.h, 132
 HAL_GPIO_AF_UART
 hal_lksmcu_gpio.h, 132
 HAL_GPIO_AFx
 hal_lksmcu_gpio.h, 131
 HAL_GPIO_Disable
 hal_lks32mc07x_gpio.c, 313
 hal_lksmcu_gpio.h, 135

HAL_GPIO_Enable
 hal_iks32mc07x_gpio.c, 313
 hal_iksmcu_gpio.h, 135

HAL_GPIO_GetInData
 hal_iks32mc07x_gpio.c, 313
 hal_iksmcu_gpio.h, 135

HAL_GPIO_GetOutData
 hal_iks32mc07x_gpio.c, 313
 hal_iksmcu_gpio.h, 136

HAL_GPIO_Invert
 hal_iks32mc07x_gpio.c, 314
 hal_iksmcu_gpio.h, 136

HAL_GPIO_IRQHandler
 hal_iks32mc07x.h, 61

HAL_GPIO_IRQHandler_00
 hal_iks32mc07x.h, 61

HAL_GPIO_MODE_AFE
 hal_iksmcu_gpio.h, 132

HAL_GPIO_MODE_IN
 hal_iksmcu_gpio.h, 132

HAL_GPIO_MODE_INOUT
 hal_iksmcu_gpio.h, 132

HAL_GPIO_MODE_INOUT_0
 hal_iksmcu_gpio.h, 132

HAL_GPIO_MODE_INOUT_1
 hal_iksmcu_gpio.h, 133

HAL_GPIO_MODE_INUP
 hal_iksmcu_gpio.h, 132

HAL_GPIO_MODE_INUPOUT
 hal_iksmcu_gpio.h, 132

HAL_GPIO_MODE_INUPOUT_0
 hal_iksmcu_gpio.h, 133

HAL_GPIO_MODE_INUPOUT_1
 hal_iksmcu_gpio.h, 133

HAL_GPIO_MODE_NONE
 hal_iksmcu_gpio.h, 132

HAL_GPIO_MODE_OUT
 hal_iksmcu_gpio.h, 132

HAL_GPIO_MODE_OUT_0
 hal_iksmcu_gpio.h, 132

HAL_GPIO_MODE_OUT_1
 hal_iksmcu_gpio.h, 133

HAL_GPIO_MODE_x
 hal_iksmcu_gpio.h, 132

HAL_GPIO_P0_0
 hal_iksmcu_gpio.h, 133

HAL_GPIO_P0_1
 hal_iksmcu_gpio.h, 133

HAL_GPIO_P0_10
 hal_iksmcu_gpio.h, 133

HAL_GPIO_P0_11
 hal_iksmcu_gpio.h, 133

HAL_GPIO_P0_12
 hal_iksmcu_gpio.h, 133

HAL_GPIO_P0_13
 hal_iksmcu_gpio.h, 133

HAL_GPIO_P0_14
 hal_iksmcu_gpio.h, 133

HAL_GPIO_P0_15
 hal_iksmcu_gpio.h, 133

HAL_GPIO_P0_2
 hal_iksmcu_gpio.h, 133

HAL_GPIO_P0_3
 hal_iksmcu_gpio.h, 133

HAL_GPIO_P0_4
 hal_iksmcu_gpio.h, 133

HAL_GPIO_P0_5
 hal_iksmcu_gpio.h, 133

HAL_GPIO_P0_6
 hal_iksmcu_gpio.h, 133

HAL_GPIO_P0_7
 hal_iksmcu_gpio.h, 133

HAL_GPIO_P0_8
 hal_iksmcu_gpio.h, 133

HAL_GPIO_P0_9
 hal_iksmcu_gpio.h, 133

HAL_GPIO_P1_0
 hal_iksmcu_gpio.h, 133

HAL_GPIO_P1_1
 hal_iksmcu_gpio.h, 133

HAL_GPIO_P1_10
 hal_iksmcu_gpio.h, 133

HAL_GPIO_P1_11
 hal_iksmcu_gpio.h, 133

HAL_GPIO_P1_12
 hal_iksmcu_gpio.h, 133

HAL_GPIO_P1_13
 hal_iksmcu_gpio.h, 133

HAL_GPIO_P1_14
 hal_iksmcu_gpio.h, 134

HAL_GPIO_P1_15
 hal_iksmcu_gpio.h, 134

HAL_GPIO_P1_2
 hal_iksmcu_gpio.h, 133

HAL_GPIO_P1_3
 hal_iksmcu_gpio.h, 133

HAL_GPIO_P1_4
 hal_iksmcu_gpio.h, 133

HAL_GPIO_P1_5
 hal_iksmcu_gpio.h, 133

HAL_GPIO_P1_6
 hal_iksmcu_gpio.h, 133

HAL_GPIO_P1_7
 hal_iksmcu_gpio.h, 133

HAL_GPIO_P1_8
 hal_iksmcu_gpio.h, 133

HAL_GPIO_P1_9
 hal_iksmcu_gpio.h, 133

HAL_GPIO_P2_0
 hal_iksmcu_gpio.h, 134

HAL_GPIO_P2_1
 hal_iksmcu_gpio.h, 134

HAL_GPIO_P2_10
 hal_iksmcu_gpio.h, 134

HAL_GPIO_P2_11
 hal_iksmcu_gpio.h, 134

HAL_GPIO_P2_12
 hal_lksmcu_gpio.h, 134

HAL_GPIO_P2_13
 hal_lksmcu_gpio.h, 134

HAL_GPIO_P2_14
 hal_lksmcu_gpio.h, 134

HAL_GPIO_P2_15
 hal_lksmcu_gpio.h, 134

HAL_GPIO_P2_2
 hal_lksmcu_gpio.h, 134

HAL_GPIO_P2_3
 hal_lksmcu_gpio.h, 134

HAL_GPIO_P2_4
 hal_lksmcu_gpio.h, 134

HAL_GPIO_P2_5
 hal_lksmcu_gpio.h, 134

HAL_GPIO_P2_6
 hal_lksmcu_gpio.h, 134

HAL_GPIO_P2_7
 hal_lksmcu_gpio.h, 134

HAL_GPIO_P2_8
 hal_lksmcu_gpio.h, 134

HAL_GPIO_P2_9
 hal_lksmcu_gpio.h, 134

HAL_GPIO_P3_0
 hal_lksmcu_gpio.h, 134

HAL_GPIO_P3_1
 hal_lksmcu_gpio.h, 134

HAL_GPIO_P3_10
 hal_lksmcu_gpio.h, 134

HAL_GPIO_P3_11
 hal_lksmcu_gpio.h, 134

HAL_GPIO_P3_12
 hal_lksmcu_gpio.h, 134

HAL_GPIO_P3_13
 hal_lksmcu_gpio.h, 134

HAL_GPIO_P3_14
 hal_lksmcu_gpio.h, 134

HAL_GPIO_P3_15
 hal_lksmcu_gpio.h, 134

HAL_GPIO_P3_2
 hal_lksmcu_gpio.h, 134

HAL_GPIO_P3_3
 hal_lksmcu_gpio.h, 134

HAL_GPIO_P3_4
 hal_lksmcu_gpio.h, 134

HAL_GPIO_P3_5
 hal_lksmcu_gpio.h, 134

HAL_GPIO_P3_6
 hal_lksmcu_gpio.h, 134

HAL_GPIO_P3_7
 hal_lksmcu_gpio.h, 134

HAL_GPIO_P3_8
 hal_lksmcu_gpio.h, 134

HAL_GPIO_P3_9
 hal_lksmcu_gpio.h, 134

HAL_GPIO_P4_0
 hal_lksmcu_gpio.h, 134

HAL_GPIO_P4_1
 hal_lksmcu_gpio.h, 134

HAL_GPIO_P4_10
 hal_lksmcu_gpio.h, 134

HAL_GPIO_P4_11
 hal_lksmcu_gpio.h, 134

HAL_GPIO_P4_12
 hal_lksmcu_gpio.h, 134

HAL_GPIO_P4_13
 hal_lksmcu_gpio.h, 134

HAL_GPIO_P4_14
 hal_lksmcu_gpio.h, 134

HAL_GPIO_P4_15
 hal_lksmcu_gpio.h, 134

HAL_GPIO_P4_2
 hal_lksmcu_gpio.h, 134

HAL_GPIO_P4_3
 hal_lksmcu_gpio.h, 134

HAL_GPIO_P4_4
 hal_lksmcu_gpio.h, 134

HAL_GPIO_P4_5
 hal_lksmcu_gpio.h, 134

HAL_GPIO_P4_6
 hal_lksmcu_gpio.h, 134

HAL_GPIO_P4_7
 hal_lksmcu_gpio.h, 134

HAL_GPIO_P4_8
 hal_lksmcu_gpio.h, 134

HAL_GPIO_P4_9
 hal_lksmcu_gpio.h, 134

HAL_GPIO_P5_0
 hal_lksmcu_gpio.h, 135

HAL_GPIO_P5_1
 hal_lksmcu_gpio.h, 135

HAL_GPIO_P5_10
 hal_lksmcu_gpio.h, 135

HAL_GPIO_P5_11
 hal_lksmcu_gpio.h, 135

HAL_GPIO_P5_12
 hal_lksmcu_gpio.h, 135

HAL_GPIO_P5_13
 hal_lksmcu_gpio.h, 135

HAL_GPIO_P5_14
 hal_lksmcu_gpio.h, 135

HAL_GPIO_P5_15
 hal_lksmcu_gpio.h, 135

HAL_GPIO_P5_2
 hal_lksmcu_gpio.h, 135

HAL_GPIO_P5_3
 hal_lksmcu_gpio.h, 135

HAL_GPIO_P5_4
 hal_lksmcu_gpio.h, 135

HAL_GPIO_P5_5
 hal_lksmcu_gpio.h, 135

HAL_GPIO_P5_6
 hal_lksmcu_gpio.h, 135

HAL_GPIO_P5_7
 hal_lksmcu_gpio.h, 135

HAL_GPIO_P5_8
 hal_lksmcu_gpio.h, 135

HAL_GPIO_P5_9
 hal_lksmcu_gpio.h, 135

HAL_GPIO_Px
 hal_lksmcu_gpio.h, 133

HAL_GPIO_Reset
 hal_lks32mc07x_gpio.c, 314
 hal_lksmcu_gpio.h, 137

HAL_GPIO_RstReuseGpioDisable
 hal_lks32mc07x_gpio.c, 314
 hal_lksmcu_gpio.h, 137

HAL_GPIO_RstReuseGpioEnable
 hal_lks32mc07x_gpio.c, 315
 hal_lksmcu_gpio.h, 137

HAL_GPIO_SetAf
 hal_lks32mc07x_gpio.c, 315
 hal_lksmcu_gpio.h, 137

HAL_GPIO_SetHigh
 hal_lks32mc07x_gpio.c, 316
 hal_lksmcu_gpio.h, 138

HAL_GPIO_SetLow
 hal_lks32mc07x_gpio.c, 317
 hal_lksmcu_gpio.h, 139

HAL_GPIO_SetMode
 hal_lks32mc07x_gpio.c, 317
 hal_lksmcu_gpio.h, 139

HAL_GPIO_SwdReuseGpioDisable
 hal_lks32mc07x_gpio.c, 318
 hal_lksmcu_gpio.h, 140

HAL_GPIO_SwdReuseGpioEnable
 hal_lks32mc07x_gpio.c, 318
 hal_lksmcu_gpio.h, 140

HAL_GPIO_TypeDef, 23
 BRR, 24
 BSRR, 24
 F3210, 24
 F7654, 24
 FBA98, 24
 FFEDC, 24
 null1, 24
 PDI, 25
 PDO, 25
 PFLT, 25
 PIE, 25
 PODE, 25
 POE, 25
 PUE, 26

HAL_GPIO_Write
 hal_lks32mc07x_gpio.c, 319
 hal_lksmcu_gpio.h, 141

HAL_HALL
 hal_lks32mc07x_reg.c, 390
 hal_lks32mc07x_reg.h, 71

HAL_HALL_0
 hal_lksmcu_hall.h, 145

HAL_HALL_1
 hal_lksmcu_hall.h, 145

HAL_HALL_Disable
 hal_lks32mc07x_hall.c, 323
 hal_lksmcu_hall.h, 145

HAL_HALL_Enable
 hal_lks32mc07x_hall.c, 323
 hal_lksmcu_hall.h, 145

HAL_HALL_IrqChange
 hal_lks32mc07x_irqfun.c, 347
 hal_lksmcu_irqfun.h, 167

HAL_HALL_IrqChangeClear
 hal_lks32mc07x_irqfun.c, 348
 hal_lksmcu_irqfun.h, 168

HAL_HALL_IrqChangeDisable
 hal_lks32mc07x_irqfun.c, 348
 hal_lksmcu_irqfun.h, 168

HAL_HALL_IrqChangeEnable
 hal_lks32mc07x_irqfun.c, 348
 hal_lksmcu_irqfun.h, 168

HAL_HALL_IrqChangeFlg
 hal_lks32mc07x_irqfun.c, 350
 hal_lksmcu_irqfun.h, 168

HAL_HALL_IRQHandler
 hal_lks32mc07x.h, 61

HAL_HALL_IRQHandler_00
 hal_lks32mc07x.h, 61

HAL_HALL_IRQHandler_10
 hal_lks32mc07x.h, 61

HAL_HALL_IrqOver
 hal_lks32mc07x_irqfun.c, 350
 hal_lksmcu_irqfun.h, 169

HAL_HALL_IrqOverClear
 hal_lks32mc07x_irqfun.c, 350
 hal_lksmcu_irqfun.h, 169

HAL_HALL_IrqOverDisable
 hal_lks32mc07x_irqfun.c, 351
 hal_lksmcu_irqfun.h, 169

HAL_HALL_IrqOverEnable
 hal_lks32mc07x_irqfun.c, 351
 hal_lksmcu_irqfun.h, 170

HAL_HALL_IrqOverFlg
 hal_lks32mc07x_irqfun.c, 351
 hal_lksmcu_irqfun.h, 170

HAL_HALL_ReadHallData
 hal_lks32mc07x_hall.c, 324
 hal_lksmcu_hall.h, 146

HAL_HALL_ReadHallWidth
 hal_lks32mc07x_hall.c, 324
 hal_lksmcu_hall.h, 146

HAL_HALL_ReadTh
 hal_lks32mc07x_hall.c, 324
 hal_lksmcu_hall.h, 146

HAL_HALL_Reset
 hal_lks32mc07x_hall.c, 325
 hal_lksmcu_hall.h, 147

HAL_HALL_SetFilter
 hal_lks32mc07x_hall.c, 325
 hal_lksmcu_hall.h, 147

HAL_HALL_SetFilteruS

hal_iks32mc07x_hall.c, 325
 hal_iksmcu_hall.h, 147
 HAL_HALL_SetFreq
 hal_iks32mc07x_hall.c, 326
 hal_iksmcu_hall.h, 148
 HAL_HALL_SetGpio
 hal_iks32mc07x_hall.c, 327
 hal_iksmcu_hall.h, 149
 HAL_HALL_SetTh
 hal_iks32mc07x_hall.c, 327
 hal_iksmcu_hall.h, 149
 HAL_HALL_StartCounting
 hal_iks32mc07x_hall.c, 328
 hal_iksmcu_hall.h, 150
 HAL_HALL_StopCounting
 hal_iks32mc07x_hall.c, 328
 hal_iksmcu_hall.h, 150
 HAL_HALL_TypeDef, 26
 CFG, 26
 CNT, 26
 INFO, 27
 TH, 27
 WIDTH, 27
 HAL_HALL_x
 hal_iksmcu_hall.h, 145
 HAL_I2C
 hal_iks32mc07x_reg.c, 390
 hal_iks32mc07x_reg.h, 71
 HAL_I2C_0
 hal_iksmcu_i2c.h, 152
 HAL_I2C_1
 hal_iksmcu_i2c.h, 152
 HAL_I2C_BusReset
 hal_iks32mc07x_i2c.c, 331
 hal_iksmcu_i2c.h, 152
 HAL_I2C_Disable
 hal_iks32mc07x_i2c.c, 331
 hal_iksmcu_i2c.h, 153
 HAL_I2C_Enable
 hal_iks32mc07x_i2c.c, 332
 hal_iksmcu_i2c.h, 153
 HAL_I2C_IRQHandler
 hal_iks32mc07x.h, 61
 HAL_I2C_IRQHandler_00
 hal_iks32mc07x.h, 62
 HAL_I2C_IRQHandler_10
 hal_iks32mc07x.h, 62
 HAL_I2C_Reset
 hal_iks32mc07x_i2c.c, 332
 hal_iksmcu_i2c.h, 153
 HAL_I2C_RxData
 hal_iks32mc07x_i2c.c, 332
 hal_iksmcu_i2c.h, 154
 HAL_I2C_SetBaud
 hal_iks32mc07x_i2c.c, 333
 hal_iksmcu_i2c.h, 154
 HAL_I2C_SetGpioScl
 hal_iks32mc07x_i2c.c, 333
 hal_iksmcu_i2c.h, 155
 HAL_I2C_SetGpioSda
 hal_iks32mc07x_i2c.c, 334
 hal_iksmcu_i2c.h, 155
 HAL_I2C_TxData
 hal_iks32mc07x_i2c.c, 335
 hal_iksmcu_i2c.h, 156
 HAL_I2C_TypeDef, 27
 ADDR, 28
 BCR, 28
 BSIZE, 28
 CFG, 28
 DATA, 28
 MSCR, 28
 SCR, 29
 HAL_I2C_x
 hal_iksmcu_i2c.h, 152
 hal_i2cio
 hal_iks32mc07x_i2c.c, 335
 HAL_INIT_DEFAULE_MCLK
 hal_iks32mc07x.h, 62
 HAL_INIT_DEFAULE_MCLK_MS
 hal_iks32mc07x.h, 62
 HAL_INIT_DEFAULE_MCLK_US
 hal_iks32mc07x.h, 62
 HAL_INIT_IRQHandler
 hal_iks32mc07x.h, 62
 HAL_INIT_IRQHandler_00
 hal_iks32mc07x.h, 62
 HAL_INIT_POWER_3.3V
 hal_iksmcu_init.h, 158
 HAL_INIT_POWER_5V
 hal_iksmcu_init.h, 158
 HAL_INIT_POWER_x
 hal_iksmcu_init.h, 157
 HAL_INIT_PowerReady
 hal_iks32mc07x_init.c, 339
 hal_iksmcu_init.h, 158
 HAL_INIT_SysInit
 hal_iks32mc07x_init.c, 339
 hal_iksmcu_init.h, 158
 HAL_IRQ_Disable
 hal_iks32mc07x_irq.c, 340
 hal_iksmcu_irq.h, 160
 HAL_IRQ_Enable
 hal_iks32mc07x_irq.c, 341
 hal_iksmcu_irq.h, 161
 HAL_IRQ_MODULE_ADC_00
 hal_iksmcu_irq.h, 160
 HAL_IRQ_MODULE_ADC_10
 hal_iksmcu_irq.h, 160
 HAL_IRQ_MODULE_CL_00
 hal_iksmcu_irq.h, 160
 HAL_IRQ_MODULE_CMP_00
 hal_iksmcu_irq.h, 160
 HAL_IRQ_MODULE_CMP_10
 hal_iksmcu_irq.h, 160
 HAL_IRQ_MODULE_DMA_00

hal_lksmcu_irq.h, 160
 HAL_IRQ_MODULE_GPIO_00
 hal_lksmcu_irq.h, 160
 HAL_IRQ_MODULE_HALL_00
 hal_lksmcu_irq.h, 160
 HAL_IRQ_MODULE_HardFault
 hal_lksmcu_irq.h, 160
 HAL_IRQ_MODULE_I2C_00
 hal_lksmcu_irq.h, 160
 HAL_IRQ_MODULE_INIT_00
 hal_lksmcu_irq.h, 160
 HAL_IRQ_MODULE_MCPWM_00
 hal_lksmcu_irq.h, 160
 HAL_IRQ_MODULE_MCPWM_01
 hal_lksmcu_irq.h, 160
 HAL_IRQ_MODULE_NonMaskableInt
 hal_lksmcu_irq.h, 159
 HAL_IRQ_MODULE_PendSV
 hal_lksmcu_irq.h, 160
 HAL_IRQ_MODULE_QEP_00
 hal_lksmcu_irq.h, 160
 HAL_IRQ_MODULE_SIF_00
 hal_lksmcu_irq.h, 160
 HAL_IRQ_MODULE_SPI_00
 hal_lksmcu_irq.h, 160
 HAL_IRQ_MODULE_SVCall
 hal_lksmcu_irq.h, 160
 HAL_IRQ_MODULE_SW_00
 hal_lksmcu_irq.h, 160
 HAL_IRQ_MODULE_SysTick
 hal_lksmcu_irq.h, 160
 HAL_IRQ_MODULE_TIMER_00
 hal_lksmcu_irq.h, 160
 HAL_IRQ_MODULE_TIMER_10
 hal_lksmcu_irq.h, 160
 HAL_IRQ_MODULE_TIMER_20
 hal_lksmcu_irq.h, 160
 HAL_IRQ_MODULE_UART_00
 hal_lksmcu_irq.h, 160
 HAL_IRQ_MODULE_UART_10
 hal_lksmcu_irq.h, 160
 HAL_IRQ_MODULE_UART_20
 hal_lksmcu_irq.h, 160
 HAL_IRQ_MODULE_WAKE_00
 hal_lksmcu_irq.h, 160
 HAL_IRQ_MODULE_x
 hal_lksmcu_irq.h, 159
 HAL_IRQ_ModuleDisable
 hal_lks32mc07x_irq.c, 341
 hal_lksmcu_irq.h, 161
 HAL_IRQ_ModuleEnable
 hal_lks32mc07x_irq.c, 341
 hal_lksmcu_irq.h, 161
 HAL_IRQ_Reset
 hal_lks32mc07x_irq.c, 342
 hal_lksmcu_irq.h, 162
 HAL_IWDG
 hal_lks32mc07x_reg.c, 391
 hal_lks32mc07x_reg.h, 71
 HAL_IWDG_0
 hal_lksmcu_iwdg.h, 172
 HAL_IWDG_Disable
 hal_lks32mc07x_iwdg.c, 354
 hal_lksmcu_iwdg.h, 172
 HAL_IWDG_Enable
 hal_lks32mc07x_iwdg.c, 355
 hal_lksmcu_iwdg.h, 173
 HAL_IWDG_Feed
 hal_lks32mc07x_iwdg.c, 355
 hal_lksmcu_iwdg.h, 173
 HAL_IWDG_Reset
 hal_lks32mc07x_iwdg.c, 356
 hal_lksmcu_iwdg.h, 174
 HAL_IWDG_SetResetTimemS
 hal_lks32mc07x_iwdg.c, 356
 hal_lksmcu_iwdg.h, 174
 HAL_IWDG_TypeDef, 29
 CFG, 29
 CLR, 29
 CNT, 30
 PSW, 30
 RTH, 30
 WTH, 30
 HAL_IWDG_x
 hal_lksmcu_iwdg.h, 172
 hal_lks32mc07x.c, 55
 __HAL_LKS32MC05X_C__, 56
 hal_lks32mc07x.h, 56
 HAL_ADC_IRQHandler, 58
 HAL_ADC_IRQHandler_00, 58
 HAL_ADC_IRQHandler_10, 59
 HAL_ADC_IRQHandler_20, 59
 HAL_CAN_IRQHandler, 59
 HAL_CAN_IRQHandler_00, 59
 HAL_CL_IRQHandler, 59
 HAL_CL_IRQHandler_00, 59
 HAL_CMP_IRQHandler, 59
 HAL_CMP_IRQHandler_00, 59
 HAL_CMP_IRQHandler_10, 60
 HAL_CMP_IRQHandler_20, 60
 HAL_CMP_IRQHandler_30, 60
 HAL_CMP_IRQHandler_40, 60
 HAL_CMP_IRQHandler_50, 60
 HAL_DMA_IRQHandler, 60
 HAL_DMA_IRQHandler_00, 60
 HAL_DSP_IRQHandler, 60
 HAL_DSP_IRQHandler_00, 61
 HAL_FMAC_IRQHandler_00, 61
 HAL_GPIO_IRQHandler, 61
 HAL_GPIO_IRQHandler_00, 61
 HAL_HALL_IRQHandler, 61
 HAL_HALL_IRQHandler_00, 61
 HAL_HALL_IRQHandler_10, 61
 HAL_I2C_IRQHandler, 61
 HAL_I2C_IRQHandler_00, 62
 HAL_I2C_IRQHandler_10, 62

- HAL_INIT_DEFAULE_MCLK, 62
- HAL_INIT_DEFAULE_MCLK_MS, 62
- HAL_INIT_DEFAULE_MCLK_US, 62
- HAL_INIT_IRQHandler, 62
- HAL_INIT_IRQHandler_00, 62
- HAL_MCPWM_IRQHandler, 63
- HAL_MCPWM_IRQHandler_00, 63
- HAL_MCPWM_IRQHandler_01, 63
- HAL_MCPWM_IRQHandler_10, 63
- HAL_MCPWM_IRQHandler_11, 63
- HAL_PWRDN_IRQHandler, 63
- HAL_PWRDN_IRQHandler_00, 63
- HAL_QEP_IRQHandler, 63
- HAL_QEP_IRQHandler_00, 64
- HAL_QEP_IRQHandler_10, 64
- HAL_QEP_IRQHandler_20, 64
- HAL_QEP_IRQHandler_30, 64
- HAL_SIF_IRQHandler, 64
- HAL_SIF_IRQHandler_00, 64
- HAL_SPI_IRQHandler, 64
- HAL_SPI_IRQHandler_00, 64
- HAL_SPI_IRQHandler_10, 65
- HAL_SW_IRQHandler, 65
- HAL_SW_IRQHandler_00, 65
- HAL_TIMER_IRQHandler, 65
- HAL_TIMER_IRQHandler_00, 65
- HAL_TIMER_IRQHandler_10, 65
- HAL_TIMER_IRQHandler_20, 65
- HAL_TIMER_IRQHandler_30, 66
- HAL_TIMER_IRQHandler_40, 66
- HAL_UART_IRQHandler, 66
- HAL_UART_IRQHandler_00, 66
- HAL_UART_IRQHandler_10, 66
- HAL_UART_IRQHandler_20, 66
- HAL_WAKE_IRQHandler, 66
- HAL_WAKE_IRQHandler_00, 66
- HAL_WWDG_IRQHandler_00, 67
- hal_lks32mc07x_adc.c
 - adc_soft_reg, 265
 - HAL_ADC_Disable, 255
 - HAL_ADC_Enable, 255
 - HAL_ADC_EndSampling, 255
 - HAL_ADC_GetRange, 256
 - HAL_ADC_GetSamplingValues, 256
 - HAL_ADC_GetValuesUnit, 257
 - HAL_ADC_ManualSampling, 258
 - HAL_ADC_Reset, 258
 - HAL_ADC_SetGpioIn, 259
 - HAL_ADC_SetGpioTriggerOut, 259
 - HAL_ADC_SetReference, 260
 - HAL_ADC_SetSamplingChannel, 260
 - HAL_ADC_SetSamplingQuantity, 261
 - HAL_ADC_SetTriggerSignal, 262
 - HAL_ADC_SoftTrigger, 263
 - HAL_ADC_UpdateTrig, 263
- hal_lks32mc07x_clk.c
 - HAL_CLK_GetMcuFreq, 270
 - HAL_CLK_mclk, 272
 - HAL_CLK_SetMcuFreq, 271
- hal_lks32mc07x_cmp.c
 - HAL_CMP_Disable, 274
 - HAL_CMP_Enable, 274
 - HAL_CMP_GetData, 275
 - HAL_CMP_Reset, 275
 - HAL_CMP_SetChannel, 276
 - HAL_CMP_SetDigitalFilter, 276
 - HAL_CMP_SetGpioIn, 276
 - HAL_CMP_SetGpioOut, 277
- hal_lks32mc07x_crc.c
 - crcModels, 283
 - HAL_CRC_Calc, 281
 - HAL_CRC_Disable, 282
 - HAL_CRC_Enable, 282
 - HAL_CRC_Reflect, 282
 - HAL_CRC_Reset, 282
- hal_lks32mc07x_dac.c
 - HAL_DAC_Disable, 285
 - HAL_DAC_Enable, 286
 - HAL_DAC_Out2GpioDisable, 286
 - HAL_DAC_Out2GpioEnable, 286
 - HAL_DAC_Reset, 286
 - HAL_DAC_SetGpioOut, 286
 - HAL_DAC_SetOutVal, 287
 - HAL_DAC_SetOutVoltageV, 287
- hal_lks32mc07x_dma.c
 - HAL_DMA_Disable, 291
 - HAL_DMA_ena, 293
 - HAL_DMA_Enable, 291
 - HAL_DMA_I2cRx, 291
 - HAL_DMA_I2cTx, 291
 - HAL_DMA_MemCopy, 292
 - HAL_DMA_Reset, 292
 - HAL_DMA_SpiRx, 292
 - HAL_DMA_SpiRxTx, 292
 - HAL_DMA_SpiTx, 292
 - HAL_DMA_UartRx, 293
 - HAL_DMA_UartTx, 293
- hal_lks32mc07x_dsp.c
 - HAL_DSP_Cos, 296
 - HAL_DSP_Disable, 296
 - HAL_DSP_Div, 296
 - HAL_DSP_Enable, 296
 - HAL_DSP_Reset, 297
 - HAL_DSP_Sin, 297
 - HAL_DSP_SinCos, 297
 - HAL_DSP_Sqrt, 297
- hal_lks32mc07x_flash.c
 - HAL_FLASH_data, 308
 - HAL_FLASH_Disable, 300
 - HAL_FLASH_DisableCache, 300
 - HAL_FLASH_Enable, 300
 - HAL_FLASH_EnableCache, 301
 - HAL_FLASH_Erase, 301
 - HAL_FLASH_Program, 302
 - HAL_FLASH_Read, 302
 - HAL_FLASH_ReadSectorSize, 303

- HAL_FLASH_ReadUint16, 303
- HAL_FLASH_ReadUint32, 304
- HAL_FLASH_ReadUint8, 304
- HAL_FLASH_Reset, 305
- HAL_FLASH_SecurityParameterCrc, 305
- HAL_FLASH_SecurityParameterRead, 305
- HAL_FLASH_SecurityParameterSave, 306
- HAL_FLASH_Update, 307
- hal_lks32mc07x_gpio.c
 - HAL_GPIO_Disable, 313
 - HAL_GPIO_Enable, 313
 - HAL_GPIO_GetInData, 313
 - HAL_GPIO_GetOutData, 313
 - HAL_GPIO_Invert, 314
 - HAL_GPIO_Reset, 314
 - HAL_GPIO_RstReuseGpioDisable, 314
 - HAL_GPIO_RstReuseGpioEnable, 315
 - HAL_GPIO_SetAf, 315
 - HAL_GPIO_SetHigh, 316
 - HAL_GPIO_SetLow, 317
 - HAL_GPIO_SetMode, 317
 - HAL_GPIO_SwdReuseGpioDisable, 318
 - HAL_GPIO_SwdReuseGpioEnable, 318
 - HAL_GPIO_Write, 319
- hal_lks32mc07x_hall.c
 - HAL_HALL_Disable, 323
 - HAL_HALL_Enable, 323
 - HAL_HALL_ReadHallData, 324
 - HAL_HALL_ReadHallWidth, 324
 - HAL_HALL_ReadTh, 324
 - HAL_HALL_Reset, 325
 - HAL_HALL_SetFilter, 325
 - HAL_HALL_SetFilteruS, 325
 - HAL_HALL_SetFreq, 326
 - HAL_HALL_SetGpio, 327
 - HAL_HALL_SetTh, 327
 - HAL_HALL_StartCounting, 328
 - HAL_HALL_StopCounting, 328
- hal_lks32mc07x_i2c.c
 - hal_baud, 335
 - HAL_I2C_BusReset, 331
 - HAL_I2C_Disable, 331
 - HAL_I2C_Enable, 332
 - HAL_I2C_Reset, 332
 - HAL_I2C_RxData, 332
 - HAL_I2C_SetBaud, 333
 - HAL_I2C_SetGpioScl, 333
 - HAL_I2C_SetGpioSda, 334
 - HAL_I2C_TxData, 335
 - hal_i2cio, 335
- hal_lks32mc07x_init.c
 - HAL_INIT_PowerReady, 339
 - HAL_INIT_SysInit, 339
- hal_lks32mc07x_irq.c
 - HAL_IRQ_Disable, 340
 - HAL_IRQ_Enable, 341
 - HAL_IRQ_ModuleDisable, 341
 - HAL_IRQ_ModuleEnable, 341
 - HAL_IRQ_Reset, 342
- hal_lks32mc07x_irqfun.c
 - HAL_ADC_IrqSamp1, 344
 - HAL_ADC_IrqSamp1Clear, 345
 - HAL_ADC_IrqSamp1Disable, 345
 - HAL_ADC_IrqSamp1Enable, 345
 - HAL_ADC_IrqSamp1Flg, 346
 - HAL_ADC_IrqSamp2, 346
 - HAL_ADC_IrqSamp2Clear, 346
 - HAL_ADC_IrqSamp2Disable, 347
 - HAL_ADC_IrqSamp2Enable, 347
 - HAL_ADC_IrqSamp2Flg, 347
 - HAL_HALL_IrqChange, 347
 - HAL_HALL_IrqChangeClear, 348
 - HAL_HALL_IrqChangeDisable, 348
 - HAL_HALL_IrqChangeEnable, 348
 - HAL_HALL_IrqChangeFlg, 350
 - HAL_HALL_IrqOver, 350
 - HAL_HALL_IrqOverClear, 350
 - HAL_HALL_IrqOverDisable, 351
 - HAL_HALL_IrqOverEnable, 351
 - HAL_HALL_IrqOverFlg, 351
- hal_lks32mc07x_iwdg.c
 - HAL_IWDG_Disable, 354
 - HAL_IWDG_Enable, 355
 - HAL_IWDG_Feed, 355
 - HAL_IWDG_Reset, 356
 - HAL_IWDG_SetResetTimemS, 356
- hal_lks32mc07x_mcpwm.c
 - HAL_MCPWM_Disable, 359
 - HAL_MCPWM_Enable, 360
 - HAL_MCPWM_GetFreq, 360
 - HAL_MCPWM_GetThCnt, 360
 - HAL_MCPWM_IrqFail, 361
 - HAL_MCPWM_IrqFailClear, 361
 - HAL_MCPWM_IrqFailDisable, 361
 - HAL_MCPWM_IrqFailEnable, 361
 - HAL_MCPWM_IrqOverflow, 362
 - HAL_MCPWM_IrqOverflowClear, 362
 - HAL_MCPWM_IrqOverflowDisable, 362
 - HAL_MCPWM_IrqOverflowEnable, 362
 - HAL_MCPWM_IrqZeroCross, 362
 - HAL_MCPWM_IrqZeroCrossClear, 362
 - HAL_MCPWM_IrqZeroCrossDisable, 362
 - HAL_MCPWM_IrqZeroCrossEnable, 363
 - HAL_MCPWM_Reset, 363
 - HAL_MCPWM_SetAdcTrig, 363
 - HAL_MCPWM_SetAdcTrigQ15, 363
 - HAL_MCPWM_SetDead, 364
 - HAL_MCPWM_SetDefaultState, 364
 - HAL_MCPWM_SetDrivePolarity, 365
 - HAL_MCPWM_SetFailSignal, 366
 - HAL_MCPWM_SetFreq, 366
 - HAL_MCPWM_SetGpioFail, 366
 - HAL_MCPWM_SetGpioOut, 367
 - HAL_MCPWM_SetOutputDisable, 367
 - HAL_MCPWM_SetOutputEnable, 368
 - HAL_MCPWM_SetOutState, 368

- HAL_MCPWM.SetPwm, 369
- HAL_MCPWM.SetPwmQ15, 369
- HAL_MCPWM.StartCounting, 370
- HAL_MCPWM.StopCounting, 370
- hal_MCPWM.UpdatePolarityState, 370
- mcpwm_polarity_state, 372
- hal_iks32mc07x_opa.c
 - HAL_OPA.Disable, 379
 - HAL_OPA.Enable, 379
 - HAL_OPA.Out2GpioDisable, 379
 - HAL_OPA.Out2GpioEnable, 379
 - HAL_OPA.Reset, 379
 - HAL_OPA.SetGain, 380
 - HAL_OPA.SetGpioIn, 380
 - HAL_OPA.SetGpioOut, 380
- hal_iks32mc07x_qep.c
 - HAL_QEP.Disable, 384
 - HAL_QEP.Enable, 384
 - HAL_QEP.ReadCnt, 384
 - HAL_QEP.Reset, 385
 - HAL_QEP.ResetCnt, 385
 - HAL_QEP.SetGpio, 385
 - HAL_QEP.SetMode, 386
 - HAL_QEP.SetTh, 386
- hal_iks32mc07x_reg.c
 - HAL_ADC, 389
 - HAL_CMP, 389
 - HAL_CRC, 390
 - HAL_DMA, 390
 - HAL_DSP, 390
 - HAL_FLASH, 390
 - HAL_GPIO, 390
 - HAL_HALL, 390
 - HAL_I2C, 390
 - HAL_IWDG, 391
 - HAL_MCPWM, 391
 - HAL_QEP, 391
 - HAL_SPI, 391
 - HAL_SYS, 391
 - HAL_TIMER, 391
 - HAL_UART, 391
- hal_iks32mc07x_reg.h
 - HAL_ADC, 70
 - HAL_CMP, 70
 - HAL_CRC, 70
 - HAL_DMA, 70
 - HAL_DSP, 71
 - HAL_FLASH, 71
 - HAL_GPIO, 71
 - HAL_HALL, 71
 - HAL_I2C, 71
 - HAL_IWDG, 71
 - HAL_MCPWM, 71
 - HAL_QEP, 71
 - HAL_SPI, 72
 - HAL_SYS, 72
 - HAL_TIMER, 72
 - HAL_UART, 72
- hal_iks32mc07x_reset.c
 - HAL_RESET.ResetCore, 393
 - HAL_RESET.ResetGlobal, 393
 - HAL_RESET.ResetRegister, 393
- hal_iks32mc07x_spi.c
 - HAL_SPI.Disable, 396
 - HAL_SPI.Enable, 396
 - HAL_SPI.Reset, 396
 - HAL_SPI.RxData, 396
 - HAL_SPI.RxDataWidth16, 398
 - HAL_SPI.RxTxData, 398
 - HAL_SPI.RxTxDataWidth16, 399
 - HAL_SPI.SetBaud, 399
 - HAL_SPI.SetGpioClk, 400
 - HAL_SPI.SetGpioCs, 400
 - HAL_SPI.SetGpioDi, 401
 - HAL_SPI.SetGpioDo, 402
 - HAL_SPI.SetGpioMiso, 402
 - HAL_SPI.SetGpioMosi, 403
 - HAL_SPI.SetMode, 404
 - HAL_SPI.TxData, 404
 - HAL_SPI.TxDataWidth16, 404
- hal_iks32mc07x_test.c
 - HAL_TestRegAddr, 409
- hal_iks32mc07x_timer.c
 - HAL_TIMER.Disable, 411
 - HAL_TIMER.Enable, 411
 - HAL_TIMER.GetFreq, 411
 - HAL_TIMER.GetThCnt, 411
 - HAL_TIMER.IrqCompare0, 412
 - HAL_TIMER.IrqCompare0Clear, 412
 - HAL_TIMER.IrqCompare0Disable, 412
 - HAL_TIMER.IrqCompare0Enable, 412
 - HAL_TIMER.IrqCompare1, 412
 - HAL_TIMER.IrqCompare1Clear, 413
 - HAL_TIMER.IrqCompare1Disable, 413
 - HAL_TIMER.IrqCompare1Enable, 413
 - HAL_TIMER.IrqOverflow, 413
 - HAL_TIMER.IrqOverflowClear, 413
 - HAL_TIMER.IrqOverflowDisable, 413
 - HAL_TIMER.IrqOverflowEnable, 413
 - HAL_TIMER.Reset, 414
 - HAL_TIMER.SetFreq, 414
 - HAL_TIMER.SetGpioIn, 414
 - HAL_TIMER.SetGpioOut, 415
 - HAL_TIMER.SetPeriodCount, 416
 - HAL_TIMER.SetPeriodmS, 416
 - HAL_TIMER.SetPerioduS, 417
 - HAL_TIMER.SetPwm, 417
 - HAL_TIMER.SetPwmQ15, 417
 - HAL_TIMER.SetWorkMode, 418
 - HAL_TIMER.StartCounting, 418
 - HAL_TIMER.StopCounting, 418
- hal_iks32mc07x_tmp.c
 - HAL_TMP.Adc2Temperature, 424
 - HAL_TMP.Adc2Temperature01, 424
 - HAL_TMP.Disable, 425
 - HAL_TMP.Enable, 425

HAL_TMP_GetTemperature, 426
 HAL_TMP_GetTemperature01, 427
 HAL_TMP_Reset, 427
 tmp_coef_a, 428
 tmp_coef_b, 428
 hal_lks32mc07x_trim.c
 HAL_TRIM_Read, 430
 HAL_TRIM_ReadId, 431
 HAL_TRIM_ReadIdCrc32, 432
 HAL_TRIM_ReadMcu07, 432
 hal_lks32mc07x_uart.c
 HAL_UART_Disable, 435
 HAL_UART_Enable, 435
 HAL_UART_IrqCheckErr, 435
 HAL_UART_IrqCheckErrClear, 435
 HAL_UART_IrqCheckErrDisable, 435
 HAL_UART_IrqCheckErrEnable, 435
 HAL_UART_IrqReceiveOver, 436
 HAL_UART_IrqReceiveOverClear, 436
 HAL_UART_IrqReceiveOverDisable, 436
 HAL_UART_IrqReceiveOverEnable, 436
 HAL_UART_IrqSendOver, 436
 HAL_UART_IrqSendOverClear, 436
 HAL_UART_IrqSendOverDisable, 436
 HAL_UART_IrqSendOverEnable, 437
 HAL_UART_IrqStopErr, 437
 HAL_UART_IrqStopErrClear, 437
 HAL_UART_IrqStopErrDisable, 437
 HAL_UART_IrqStopErrEnable, 437
 HAL_UART_ReadBuffer, 437
 HAL_UART_Reset, 437
 HAL_UART_SetBaud, 438
 HAL_UART_SetGpioRx, 438
 HAL_UART_SetGpioTx, 438
 HAL_UART_SetMode, 439
 HAL_UART_TxData, 439
 HAL_UART_WriteBuffer, 440
 hal_lks32mc07x_wake.c
 HAL_WAKE_DeepSleepIoTimeWakeUp, 445
 HAL_WAKE_DeepSleepIoWake, 445
 HAL_WAKE_DeepSleepTimeWakeUp, 445
 hal_lksmcu.h, 68
 hal_lksmcu_adc.h
 HAL_ADC_0, 81
 HAL_ADC_1, 81
 HAL_ADC_2, 81
 HAL_ADC_3, 81
 HAL_ADC_Disable, 81
 HAL_ADC_Enable, 81
 HAL_ADC_EndSampling, 82
 HAL_ADC_GetRange, 82
 HAL_ADC_GetSamplingValues, 83
 HAL_ADC_GetValuesUnit, 83
 HAL_ADC_ManualSampling, 84
 HAL_ADC_PORT_AGND, 79
 HAL_ADC_PORT_CH0, 78
 HAL_ADC_PORT_CH1, 78
 HAL_ADC_PORT_CH10, 78
 HAL_ADC_PORT_CH11, 78
 HAL_ADC_PORT_CH12, 78
 HAL_ADC_PORT_CH13, 78
 HAL_ADC_PORT_CH14, 78
 HAL_ADC_PORT_CH15, 78
 HAL_ADC_PORT_CH16, 79
 HAL_ADC_PORT_CH17, 79
 HAL_ADC_PORT_CH18, 79
 HAL_ADC_PORT_CH19, 79
 HAL_ADC_PORT_CH2, 78
 HAL_ADC_PORT_CH3, 78
 HAL_ADC_PORT_CH4, 78
 HAL_ADC_PORT_CH5, 78
 HAL_ADC_PORT_CH6, 78
 HAL_ADC_PORT_CH7, 78
 HAL_ADC_PORT_CH8, 78
 HAL_ADC_PORT_CH9, 78
 HAL_ADC_PORT_DAC0, 79
 HAL_ADC_PORT_DAC1, 79
 HAL_ADC_PORT_LDO12, 79
 HAL_ADC_PORT_OPA0, 78
 HAL_ADC_PORT_OPA1, 79
 HAL_ADC_PORT_OPA2, 79
 HAL_ADC_PORT_OPA3, 79
 HAL_ADC_PORT_OPA4, 79
 HAL_ADC_PORT_OPA5, 79
 HAL_ADC_PORT_REF, 79
 HAL_ADC_PORT_TMP0, 79
 HAL_ADC_PORT_VDD, 79
 HAL_ADC_PORT_x, 78
 HAL_ADC_REFERENCE_EXTERNAL, 79
 HAL_ADC_REFERENCE_INTERNAL, 79
 HAL_ADC_REFERENCE_x, 79
 HAL_ADC_Reset, 85
 HAL_ADC_SAMP_NUMBER_0, 79
 HAL_ADC_SAMP_NUMBER_1, 79
 HAL_ADC_SAMP_NUMBER_10, 80
 HAL_ADC_SAMP_NUMBER_11, 80
 HAL_ADC_SAMP_NUMBER_12, 80
 HAL_ADC_SAMP_NUMBER_13, 80
 HAL_ADC_SAMP_NUMBER_14, 80
 HAL_ADC_SAMP_NUMBER_15, 80
 HAL_ADC_SAMP_NUMBER_2, 79
 HAL_ADC_SAMP_NUMBER_3, 79
 HAL_ADC_SAMP_NUMBER_4, 79
 HAL_ADC_SAMP_NUMBER_5, 79
 HAL_ADC_SAMP_NUMBER_6, 79
 HAL_ADC_SAMP_NUMBER_7, 80
 HAL_ADC_SAMP_NUMBER_8, 80
 HAL_ADC_SAMP_NUMBER_9, 80
 HAL_ADC_SAMP_NUMBER_MAX, 80
 HAL_ADC_SAMP_NUMBER_x, 79
 HAL_ADC_SAMP_SEQUENCE_0, 80
 HAL_ADC_SAMP_SEQUENCE_1, 80
 HAL_ADC_SAMP_SEQUENCE_10, 80
 HAL_ADC_SAMP_SEQUENCE_11, 80
 HAL_ADC_SAMP_SEQUENCE_12, 80
 HAL_ADC_SAMP_SEQUENCE_13, 80

- HAL_ADC_SAMP_SEQUENCE_14, 80
- HAL_ADC_SAMP_SEQUENCE_15, 80
- HAL_ADC_SAMP_SEQUENCE_16, 80
- HAL_ADC_SAMP_SEQUENCE_17, 80
- HAL_ADC_SAMP_SEQUENCE_18, 80
- HAL_ADC_SAMP_SEQUENCE_19, 80
- HAL_ADC_SAMP_SEQUENCE_2, 80
- HAL_ADC_SAMP_SEQUENCE_3, 80
- HAL_ADC_SAMP_SEQUENCE_4, 80
- HAL_ADC_SAMP_SEQUENCE_5, 80
- HAL_ADC_SAMP_SEQUENCE_6, 80
- HAL_ADC_SAMP_SEQUENCE_7, 80
- HAL_ADC_SAMP_SEQUENCE_8, 80
- HAL_ADC_SAMP_SEQUENCE_9, 80
- HAL_ADC_SAMP_SEQUENCE_x, 80
- HAL_ADC_SetGpioIn, 85
- HAL_ADC_SetGpioTriggerOut, 86
- HAL_ADC_SetReference, 86
- HAL_ADC_SetSamplingChannel, 87
- HAL_ADC_SetSamplingQuantity, 88
- HAL_ADC_SetTriggerSignal, 88
- HAL_ADC_SoftTrigger, 89
- HAL_ADC_TRIGGER_MCPWM, 81
- HAL_ADC_TRIGGER_SOFT, 81
- HAL_ADC_TRIGGER_TIMER, 81
- HAL_ADC_TRIGGER_x, 80
- HAL_ADC_x, 81
- hal_lksmcu_clk.h
 - HAL_CLK_FREQ_RC_12MHZ, 94
 - HAL_CLK_FREQ_RC_16MHZ, 94
 - HAL_CLK_FREQ_RC_18MHZ, 94
 - HAL_CLK_FREQ_RC_192MHZ, 94
 - HAL_CLK_FREQ_RC_24MHZ, 94
 - HAL_CLK_FREQ_RC_32MHZ, 94
 - HAL_CLK_FREQ_RC_36MHZ, 94
 - HAL_CLK_FREQ_RC_48MHZ, 94
 - HAL_CLK_FREQ_RC_4MHZ, 94
 - HAL_CLK_FREQ_RC_64MHZ, 94
 - HAL_CLK_FREQ_RC_6MHZ, 94
 - HAL_CLK_FREQ_RC_72MHZ, 94
 - HAL_CLK_FREQ_RC_8MHZ, 94
 - HAL_CLK_FREQ_RC_96MHZ, 94
 - HAL_CLK_FREQ_RC_9MHZ, 94
 - HAL_CLK_FREQ_x, 94
 - HAL_CLK_FREQ_XTAL_12MHZ, 94
 - HAL_CLK_FREQ_XTAL_192MHZ, 94
 - HAL_CLK_FREQ_XTAL_24MHZ, 94
 - HAL_CLK_FREQ_XTAL_48MHZ, 94
 - HAL_CLK_FREQ_XTAL_6MHZ, 94
 - HAL_CLK_FREQ_XTAL_96MHZ, 94
 - HAL_CLK_GetMcuFreq, 94
 - HAL_CLK_SetMcuFreq, 95
- hal_lksmcu_cmp.h
 - HAL_CMP_0, 99
 - HAL_CMP_1, 99
 - HAL_CMP_2, 99
 - HAL_CMP_3, 100
 - HAL_CMP_4, 100
- HAL_CMP_5, 100
- HAL_CMP_CHANNEL_IN_0, 98
- HAL_CMP_CHANNEL_IN_DAC, 98
- HAL_CMP_CHANNEL_IN_MID, 98
- HAL_CMP_CHANNEL_IN_REF, 98
- HAL_CMP_CHANNEL_INx, 98
- HAL_CMP_CHANNEL_IP0, 98
- HAL_CMP_CHANNEL_IP1, 98
- HAL_CMP_CHANNEL_IP2, 98
- HAL_CMP_CHANNEL_IP3, 98
- HAL_CMP_CHANNEL_IP4, 98
- HAL_CMP_CHANNEL_IP5, 99
- HAL_CMP_CHANNEL_IP6, 99
- HAL_CMP_CHANNEL_IP7, 99
- HAL_CMP_CHANNEL_IP_AVSS, 99
- HAL_CMP_CHANNEL_IP_DAC0, 98
- HAL_CMP_CHANNEL_IP_DAC1, 99
- HAL_CMP_CHANNEL_IP_OPA0_IN, 99
- HAL_CMP_CHANNEL_IP_OPA0_IP, 98
- HAL_CMP_CHANNEL_IP_OPA0_OUTN, 99
- HAL_CMP_CHANNEL_IP_OPA0_OUTP, 99
- HAL_CMP_CHANNEL_IP_OPA1_IN, 99
- HAL_CMP_CHANNEL_IP_OPA1_IP, 99
- HAL_CMP_CHANNEL_IP_OPA1_OUTN, 99
- HAL_CMP_CHANNEL_IP_OPA1_OUTP, 99
- HAL_CMP_CHANNEL_IP_OPA2_IN, 99
- HAL_CMP_CHANNEL_IP_OPA2_IP, 99
- HAL_CMP_CHANNEL_IP_OPA2_OUTN, 99
- HAL_CMP_CHANNEL_IP_OPA2_OUTP, 99
- HAL_CMP_CHANNEL_IP_OPA3_IN, 99
- HAL_CMP_CHANNEL_IP_OPA3_IP, 99
- HAL_CMP_CHANNEL_IP_OPA3_OUTN, 99
- HAL_CMP_CHANNEL_IP_OPA3_OUTP, 99
- HAL_CMP_CHANNEL_IP_OPA4_IN, 99
- HAL_CMP_CHANNEL_IP_OPA4_IP, 99
- HAL_CMP_CHANNEL_IP_OPA4_OUTN, 99
- HAL_CMP_CHANNEL_IP_OPA4_OUTP, 99
- HAL_CMP_CHANNEL_IP_OPA5_IN, 99
- HAL_CMP_CHANNEL_IP_OPA5_IP, 99
- HAL_CMP_CHANNEL_IP_OPA5_OUTN, 99
- HAL_CMP_CHANNEL_IP_OPA5_OUTP, 99
- HAL_CMP_CHANNEL_IP_OPA_OUT_CMP0, 99
- HAL_CMP_CHANNEL_IPx, 98
- HAL_CMP_Disable, 100
- HAL_CMP_Enable, 100
- HAL_CMP_GetData, 101
- HAL_CMP_Reset, 101
- HAL_CMP_SetChannel, 102
- HAL_CMP_SetDigitalFilter, 102
- HAL_CMP_SetGpioIn, 102
- HAL_CMP_SetGpioOut, 103
- HAL_CMP_x, 99
- hal_lksmcu_crc.h
 - HAL_CRC_0, 106
 - HAL_CRC_Calc, 106
 - HAL_CRC_Disable, 107
 - HAL_CRC_Enable, 107
 - HAL_CRC_MODEL_CRC16_CCITT, 106

- HAL_CRC_MODEL_CRC16_CCITT_FALSE, 106
- HAL_CRC_MODEL_CRC16_DNP, 106
- HAL_CRC_MODEL_CRC16_IBMx1, 106
- HAL_CRC_MODEL_CRC16_MAXIM, 106
- HAL_CRC_MODEL_CRC16_MODBU, 106
- HAL_CRC_MODEL_CRC16_USBx1, 106
- HAL_CRC_MODEL_CRC16_X25, 106
- HAL_CRC_MODEL_CRC16_XMODEM, 106
- HAL_CRC_MODEL_CRC32, 106
- HAL_CRC_MODEL_CRC32_MPEG_2, 106
- HAL_CRC_MODEL_CRC8, 106
- HAL_CRC_MODEL_CRC8_ITU, 106
- HAL_CRC_MODEL_CRC8_MAXIMx, 106
- HAL_CRC_MODEL_CRC8_ROHC, 106
- HAL_CRC_MODEL_x, 105
- HAL_CRC_Reset, 107
- HAL_CRC_x, 106
- hal_ksmcu_dac.h
 - HAL_DAC_0, 109
 - HAL_DAC_1, 109
 - HAL_DAC_2, 109
 - HAL_DAC_3, 109
 - HAL_DAC_Disable, 109
 - HAL_DAC_Enable, 110
 - HAL_DAC_Out2GpioDisable, 110
 - HAL_DAC_Out2GpioEnable, 110
 - HAL_DAC_Reset, 110
 - HAL_DAC_SetGpioOut, 110
 - HAL_DAC_SetOutVal, 111
 - HAL_DAC_SetOutVoltageV, 111
 - HAL_DAC_x, 109
- hal_ksmcu_dma.h
 - HAL_DMA_0, 113
 - HAL_DMA_1, 113
 - HAL_DMA_2, 113
 - HAL_DMA_3, 113
 - HAL_DMA_Disable, 113
 - HAL_DMA_Enable, 113
 - HAL_DMA_I2cRx, 114
 - HAL_DMA_I2cTx, 114
 - HAL_DMA_MemCopy, 114
 - HAL_DMA_Reset, 114
 - HAL_DMA_SpiRx, 114
 - HAL_DMA_SpiRxTx, 114
 - HAL_DMA_SpiTx, 115
 - HAL_DMA_UartRx, 115
 - HAL_DMA_UartTx, 115
 - HAL_DMA_x, 113
- hal_ksmcu_dsp.h
 - HAL_DSP_0, 117
 - HAL_DSP_Cos, 117
 - HAL_DSP_Disable, 117
 - HAL_DSP_Div, 118
 - HAL_DSP_Enable, 118
 - HAL_DSP_Reset, 118
 - HAL_DSP_Sin, 118
 - HAL_DSP_SinCos, 118
 - HAL_DSP_Sqrt, 118
 - HAL_DSP_x, 117
- hal_ksmcu_flash.h
 - HAL_FLASH_0, 120
 - HAL_FLASH_1, 120
 - HAL_FLASH_Disable, 121
 - HAL_FLASH_DisableCache, 121
 - HAL_FLASH_Enable, 121
 - HAL_FLASH_EnableCache, 121
 - HAL_FLASH_Erase, 122
 - HAL_FLASH_Program, 122
 - HAL_FLASH_Read, 123
 - HAL_FLASH_ReadSectorSize, 124
 - HAL_FLASH_ReadUint16, 124
 - HAL_FLASH_ReadUint32, 125
 - HAL_FLASH_ReadUint8, 125
 - HAL_FLASH_Reset, 126
 - HAL_FLASH_SecurityParameterRead, 126
 - HAL_FLASH_SecurityParameterSave, 127
 - HAL_FLASH_Update, 127
 - HAL_FLASH_x, 120
- hal_ksmcu_gpio.h
 - HAL_GPIO_AF_ADC, 132
 - HAL_GPIO_AF_ADC_TRIG, 132
 - HAL_GPIO_AF_CAN, 132
 - HAL_GPIO_AF_CL, 132
 - HAL_GPIO_AF_CLK, 132
 - HAL_GPIO_AF_CLK_OUT, 132
 - HAL_GPIO_AF_CMP, 132
 - HAL_GPIO_AF_CMP_OUT, 132
 - HAL_GPIO_AF_DAC, 132
 - HAL_GPIO_AF_GPIO, 132
 - HAL_GPIO_AF_HALL, 132
 - HAL_GPIO_AF_I2C, 132
 - HAL_GPIO_AF_MCPWM, 132
 - HAL_GPIO_AF_OPA, 132
 - HAL_GPIO_AF_OPA_OUT, 132
 - HAL_GPIO_AF_QEP, 132
 - HAL_GPIO_AF_SIF, 132
 - HAL_GPIO_AF_SPI, 132
 - HAL_GPIO_AF_TIMER0, 132
 - HAL_GPIO_AF_TIMER1, 132
 - HAL_GPIO_AF_TIMER2, 132
 - HAL_GPIO_AF_TIMER3, 132
 - HAL_GPIO_AF_TIMER4, 132
 - HAL_GPIO_AF_TIMER5, 132
 - HAL_GPIO_AF_UART, 132
 - HAL_GPIO_AFx, 131
 - HAL_GPIO_Disable, 135
 - HAL_GPIO_Enable, 135
 - HAL_GPIO_GetInData, 135
 - HAL_GPIO_GetOutData, 136
 - HAL_GPIO_Invert, 136
 - HAL_GPIO_MODE_AFE, 132
 - HAL_GPIO_MODE_IN, 132
 - HAL_GPIO_MODE_INOUT, 132
 - HAL_GPIO_MODE_INOUT_0, 132
 - HAL_GPIO_MODE_INOUT_1, 133
 - HAL_GPIO_MODE_INUP, 132

HAL_GPIO_MODE_INUPOUT, 132
HAL_GPIO_MODE_INUPOUT_0, 133
HAL_GPIO_MODE_INUPOUT_1, 133
HAL_GPIO_MODE_NONE, 132
HAL_GPIO_MODE_OUT, 132
HAL_GPIO_MODE_OUT_0, 132
HAL_GPIO_MODE_OUT_1, 133
HAL_GPIO_MODE_x, 132
HAL_GPIO_P0.0, 133
HAL_GPIO_P0.1, 133
HAL_GPIO_P0.10, 133
HAL_GPIO_P0.11, 133
HAL_GPIO_P0.12, 133
HAL_GPIO_P0.13, 133
HAL_GPIO_P0.14, 133
HAL_GPIO_P0.15, 133
HAL_GPIO_P0.2, 133
HAL_GPIO_P0.3, 133
HAL_GPIO_P0.4, 133
HAL_GPIO_P0.5, 133
HAL_GPIO_P0.6, 133
HAL_GPIO_P0.7, 133
HAL_GPIO_P0.8, 133
HAL_GPIO_P0.9, 133
HAL_GPIO_P1.0, 133
HAL_GPIO_P1.1, 133
HAL_GPIO_P1.10, 133
HAL_GPIO_P1.11, 133
HAL_GPIO_P1.12, 133
HAL_GPIO_P1.13, 133
HAL_GPIO_P1.14, 134
HAL_GPIO_P1.15, 134
HAL_GPIO_P1.2, 133
HAL_GPIO_P1.3, 133
HAL_GPIO_P1.4, 133
HAL_GPIO_P1.5, 133
HAL_GPIO_P1.6, 133
HAL_GPIO_P1.7, 133
HAL_GPIO_P1.8, 133
HAL_GPIO_P1.9, 133
HAL_GPIO_P2.0, 134
HAL_GPIO_P2.1, 134
HAL_GPIO_P2.10, 134
HAL_GPIO_P2.11, 134
HAL_GPIO_P2.12, 134
HAL_GPIO_P2.13, 134
HAL_GPIO_P2.14, 134
HAL_GPIO_P2.15, 134
HAL_GPIO_P2.2, 134
HAL_GPIO_P2.3, 134
HAL_GPIO_P2.4, 134
HAL_GPIO_P2.5, 134
HAL_GPIO_P2.6, 134
HAL_GPIO_P2.7, 134
HAL_GPIO_P2.8, 134
HAL_GPIO_P2.9, 134
HAL_GPIO_P3.0, 134
HAL_GPIO_P3.1, 134
HAL_GPIO_P3.10, 134
HAL_GPIO_P3.11, 134
HAL_GPIO_P3.12, 134
HAL_GPIO_P3.13, 134
HAL_GPIO_P3.14, 134
HAL_GPIO_P3.15, 134
HAL_GPIO_P3.2, 134
HAL_GPIO_P3.3, 134
HAL_GPIO_P3.4, 134
HAL_GPIO_P3.5, 134
HAL_GPIO_P3.6, 134
HAL_GPIO_P3.7, 134
HAL_GPIO_P3.8, 134
HAL_GPIO_P3.9, 134
HAL_GPIO_P4.0, 134
HAL_GPIO_P4.1, 134
HAL_GPIO_P4.10, 134
HAL_GPIO_P4.11, 134
HAL_GPIO_P4.12, 134
HAL_GPIO_P4.13, 134
HAL_GPIO_P4.14, 134
HAL_GPIO_P4.15, 134
HAL_GPIO_P4.2, 134
HAL_GPIO_P4.3, 134
HAL_GPIO_P4.4, 134
HAL_GPIO_P4.5, 134
HAL_GPIO_P4.6, 134
HAL_GPIO_P4.7, 134
HAL_GPIO_P4.8, 134
HAL_GPIO_P4.9, 134
HAL_GPIO_P5.0, 135
HAL_GPIO_P5.1, 135
HAL_GPIO_P5.10, 135
HAL_GPIO_P5.11, 135
HAL_GPIO_P5.12, 135
HAL_GPIO_P5.13, 135
HAL_GPIO_P5.14, 135
HAL_GPIO_P5.15, 135
HAL_GPIO_P5.2, 135
HAL_GPIO_P5.3, 135
HAL_GPIO_P5.4, 135
HAL_GPIO_P5.5, 135
HAL_GPIO_P5.6, 135
HAL_GPIO_P5.7, 135
HAL_GPIO_P5.8, 135
HAL_GPIO_P5.9, 135
HAL_GPIO_Px, 133
HAL_GPIO_Reset, 137
HAL_GPIO_RstReuseGpioDisable, 137
HAL_GPIO_RstReuseGpioEnable, 137
HAL_GPIO_SetAf, 137
HAL_GPIO_SetHigh, 138
HAL_GPIO_SetLow, 139
HAL_GPIO_SetMode, 139
HAL_GPIO_SwdReuseGpioDisable, 140
HAL_GPIO_SwdReuseGpioEnable, 140
HAL_GPIO_Write, 141
hal_lksmcu_hall.h

- HAL_HALL_0, 145
- HAL_HALL_1, 145
- HAL_HALL_Disable, 145
- HAL_HALL_Enable, 145
- HAL_HALL_ReadHallData, 146
- HAL_HALL_ReadHallWidth, 146
- HAL_HALL_ReadTh, 146
- HAL_HALL_Reset, 147
- HAL_HALL_SetFilter, 147
- HAL_HALL_SetFilteruS, 147
- HAL_HALL_SetFreq, 148
- HAL_HALL_SetGpio, 149
- HAL_HALL_SetTh, 149
- HAL_HALL_StartCounting, 150
- HAL_HALL_StopCounting, 150
- HAL_HALL_x, 145
- hal_iksmcu_i2c.h
 - HAL_I2C_0, 152
 - HAL_I2C_1, 152
 - HAL_I2C_BusReset, 152
 - HAL_I2C_Disable, 153
 - HAL_I2C_Enable, 153
 - HAL_I2C_Reset, 153
 - HAL_I2C_RxData, 154
 - HAL_I2C_SetBaud, 154
 - HAL_I2C_SetGpioScl, 155
 - HAL_I2C_SetGpioSda, 155
 - HAL_I2C_TxData, 156
 - HAL_I2C_x, 152
- hal_iksmcu_init.h
 - HAL_INIT_POWER_3.3V, 158
 - HAL_INIT_POWER_5V, 158
 - HAL_INIT_POWER_x, 157
 - HAL_INIT_PowerReady, 158
 - HAL_INIT_SysInit, 158
- hal_iksmcu_irq.h
 - HAL_IRQ_Disable, 160
 - HAL_IRQ_Enable, 161
 - HAL_IRQ_MODULE_ADC_00, 160
 - HAL_IRQ_MODULE_ADC_10, 160
 - HAL_IRQ_MODULE_CL_00, 160
 - HAL_IRQ_MODULE_CMP_00, 160
 - HAL_IRQ_MODULE_CMP_10, 160
 - HAL_IRQ_MODULE_DMA_00, 160
 - HAL_IRQ_MODULE_GPIO_00, 160
 - HAL_IRQ_MODULE_HALL_00, 160
 - HAL_IRQ_MODULE_HardFault, 160
 - HAL_IRQ_MODULE_I2C_00, 160
 - HAL_IRQ_MODULE_INIT_00, 160
 - HAL_IRQ_MODULE_MCPWM_00, 160
 - HAL_IRQ_MODULE_MCPWM_01, 160
 - HAL_IRQ_MODULE_NonMaskableInt, 159
 - HAL_IRQ_MODULE_PendSV, 160
 - HAL_IRQ_MODULE_QEP_00, 160
 - HAL_IRQ_MODULE_SIF_00, 160
 - HAL_IRQ_MODULE_SPI_00, 160
 - HAL_IRQ_MODULE_SVCall, 160
 - HAL_IRQ_MODULE_SW_00, 160
 - HAL_IRQ_MODULE_SysTick, 160
 - HAL_IRQ_MODULE_TIMER_00, 160
 - HAL_IRQ_MODULE_TIMER_10, 160
 - HAL_IRQ_MODULE_TIMER_20, 160
 - HAL_IRQ_MODULE_UART_00, 160
 - HAL_IRQ_MODULE_UART_10, 160
 - HAL_IRQ_MODULE_UART_20, 160
 - HAL_IRQ_MODULE_WAKE_00, 160
 - HAL_IRQ_MODULE_x, 159
 - HAL_IRQ_ModuleDisable, 161
 - HAL_IRQ_ModuleEnable, 161
 - HAL_IRQ_Reset, 162
- hal_iksmcu_irqfun.h
 - HAL_ADC_IrqSamp1, 164
 - HAL_ADC_IrqSamp1Clear, 165
 - HAL_ADC_IrqSamp1Disable, 165
 - HAL_ADC_IrqSamp1Enable, 165
 - HAL_ADC_IrqSamp1Flg, 165
 - HAL_ADC_IrqSamp2, 166
 - HAL_ADC_IrqSamp2Clear, 166
 - HAL_ADC_IrqSamp2Disable, 166
 - HAL_ADC_IrqSamp2Enable, 167
 - HAL_ADC_IrqSamp2Flg, 167
 - HAL_HALL_IrqChange, 167
 - HAL_HALL_IrqChangeClear, 168
 - HAL_HALL_IrqChangeDisable, 168
 - HAL_HALL_IrqChangeEnable, 168
 - HAL_HALL_IrqChangeFlg, 168
 - HAL_HALL_IrqOver, 169
 - HAL_HALL_IrqOverClear, 169
 - HAL_HALL_IrqOverDisable, 169
 - HAL_HALL_IrqOverEnable, 170
 - HAL_HALL_IrqOverFlg, 170
- hal_iksmcu_iwdg.h
 - HAL_IWDG_0, 172
 - HAL_IWDG_Disable, 172
 - HAL_IWDG_Enable, 173
 - HAL_IWDG_Feed, 173
 - HAL_IWDG_Reset, 174
 - HAL_IWDG_SetResetTimemS, 174
 - HAL_IWDG_x, 172
- hal_iksmcu_mcpwm.h
 - HAL_MCPWM_0, 179
 - HAL_MCPWM_1, 179
 - HAL_MCPWM_DEFAULT_OUTPUT_STATE_HIGH, 177
 - HAL_MCPWM_DEFAULT_OUTPUT_STATE_LOW, 177
 - HAL_MCPWM_DEFAULT_OUTPUT_STATE_OFF, 177
 - HAL_MCPWM_DEFAULT_OUTPUT_STATE_x, 177
 - HAL_MCPWM_Disable, 179
 - HAL_MCPWM_Enable, 179
 - HAL_MCPWM_FAIL_CMP0_HIGH, 177
 - HAL_MCPWM_FAIL_CMP0_LOW, 177
 - HAL_MCPWM_FAIL_CMP1_HIGH, 177
 - HAL_MCPWM_FAIL_CMP1_LOW, 177
 - HAL_MCPWM_FAIL_IO0_HIGH, 177

- HAL_MCPWM_FAIL_IO0_LOW, 177
- HAL_MCPWM_FAIL_IO1_HIGH, 177
- HAL_MCPWM_FAIL_IO1_LOW, 177
- HAL_MCPWM_FAIL_x, 177
- HAL_MCPWM_GetFreq, 179
- HAL_MCPWM_GetThCnt, 181
- HAL_MCPWM_IrqFail, 181
- HAL_MCPWM_IrqFailClear, 182
- HAL_MCPWM_IrqFailDisable, 182
- HAL_MCPWM_IrqFailEnable, 182
- HAL_MCPWM_IrqOverflow, 182
- HAL_MCPWM_IrqOverflowClear, 182
- HAL_MCPWM_IrqOverflowDisable, 182
- HAL_MCPWM_IrqOverflowEnable, 182
- HAL_MCPWM_IrqZeroCross, 183
- HAL_MCPWM_IrqZeroCrossClear, 183
- HAL_MCPWM_IrqZeroCrossDisable, 183
- HAL_MCPWM_IrqZeroCrossEnable, 183
- HAL_MCPWM_OUTPUT_STATE_HIGH, 178
- HAL_MCPWM_OUTPUT_STATE_HIGH_PWM, 178
- HAL_MCPWM_OUTPUT_STATE_LOW, 178
- HAL_MCPWM_OUTPUT_STATE_LOW_PWM, 178
- HAL_MCPWM_OUTPUT_STATE_OFF, 178
- HAL_MCPWM_OUTPUT_STATE_PWM, 178
- HAL_MCPWM_OUTPUT_STATE_x, 177
- HAL_MCPWM_POLARITY_PH_NH, 178
- HAL_MCPWM_POLARITY_PH_NL, 178
- HAL_MCPWM_POLARITY_PL_NH, 178
- HAL_MCPWM_POLARITY_PL_NL, 178
- HAL_MCPWM_POLARITY_x, 178
- HAL_MCPWM_PORT_0, 178
- HAL_MCPWM_PORT_1, 178
- HAL_MCPWM_PORT_2, 178
- HAL_MCPWM_PORT_3, 178
- HAL_MCPWM_PORT_4, 178
- HAL_MCPWM_PORT_5, 178
- HAL_MCPWM_PORT_x, 178
- HAL_MCPWM_Reset, 183
- HAL_MCPWM_SetAdcTrig, 184
- HAL_MCPWM_SetAdcTrigQ15, 184
- HAL_MCPWM_SetDead, 184
- HAL_MCPWM_SetDefaultState, 185
- HAL_MCPWM_SetDrivePolarity, 185
- HAL_MCPWM_SetFailSignal, 186
- HAL_MCPWM_SetFreq, 186
- HAL_MCPWM_SetGpioFail, 187
- HAL_MCPWM_SetGpioOut, 187
- HAL_MCPWM_SetOutputDisable, 188
- HAL_MCPWM_SetOutputEnable, 188
- HAL_MCPWM_SetOutState, 189
- HAL_MCPWM_SetPwm, 189
- HAL_MCPWM_SetPwmQ15, 190
- HAL_MCPWM_StartCounting, 190
- HAL_MCPWM_StopCounting, 190
- HAL_MCPWM_x, 178
- hal_lksmcu_opa.h
 - HAL_OPA_0, 195
 - HAL_OPA_0B, 195
 - HAL_OPA_1, 195
 - HAL_OPA_2, 195
 - HAL_OPA_3, 195
 - HAL_OPA_4, 195
 - HAL_OPA_5, 195
 - HAL_OPA_Disable, 196
 - HAL_OPA_Enable, 196
 - HAL_OPA_GAIN_160_10, 195
 - HAL_OPA_GAIN_160_40, 195
 - HAL_OPA_GAIN_170_40, 195
 - HAL_OPA_GAIN_170_40_4, 195
 - HAL_OPA_GAIN_170_40_6, 195
 - HAL_OPA_GAIN_180_30, 195
 - HAL_OPA_GAIN_180_30_4, 195
 - HAL_OPA_GAIN_180_30_6, 195
 - HAL_OPA_GAIN_190_20, 195
 - HAL_OPA_GAIN_190_20_4, 195
 - HAL_OPA_GAIN_190_20_6, 195
 - HAL_OPA_GAIN_200_10, 195
 - HAL_OPA_GAIN_200_10_4, 195
 - HAL_OPA_GAIN_200_10_6, 195
 - HAL_OPA_GAIN_20_10, 195
 - HAL_OPA_GAIN_320_10, 195
 - HAL_OPA_GAIN_320_40, 195
 - HAL_OPA_GAIN_320_5, 195
 - HAL_OPA_GAIN_40_10, 195
 - HAL_OPA_GAIN_640_40, 195
 - HAL_OPA_GAIN_660_20, 195
 - HAL_OPA_GAIN_80_10, 195
 - HAL_OPA_GAIN_HIGH, 195
 - HAL_OPA_GAIN_LOW, 195
 - HAL_OPA_GAIN_MEDIUM, 195
 - HAL_OPA_GAIN_x, 194
 - HAL_OPA_OPAX_2_ADCCHN, 194
 - HAL_OPA_Out2GpioDisable, 196
 - HAL_OPA_Out2GpioEnable, 196
 - HAL_OPA_READ_GAIN, 194
 - HAL_OPA_Reset, 196
 - HAL_OPA_SetGain, 197
 - HAL_OPA_SetGpioIn, 197
 - HAL_OPA_SetGpioOut, 197
 - HAL_OPA_x, 195
- hal_lksmcu_qep.h
 - HAL_QEP_0, 201
 - HAL_QEP_1, 201
 - HAL_QEP_2, 201
 - HAL_QEP_3, 201
 - HAL_QEP_Disable, 201
 - HAL_QEP_Enable, 201
 - HAL_QEP_MODE_DUAL_PULSE, 200
 - HAL_QEP_MODE_QUADRATURE, 200
 - HAL_QEP_MODE_SYMBOL_PULSE, 200
 - HAL_QEP_MODE_x, 200
 - HAL_QEP_ReadCnt, 202
 - HAL_QEP_Reset, 202
 - HAL_QEP_ResetCnt, 202
 - HAL_QEP_SetGpio, 203
 - HAL_QEP_SetMode, 203

HAL_QEP_SetTh, 204
 HAL_QEP_x, 200
 hal_ksmcu_reg.h
 HAL_REG_IF_WRITE, 205
 HAL_REG_INVERT, 205
 HAL_REG_READ, 205
 HAL_REG_READ_BIT, 205
 HAL_REG_READ_BIT_POS_MASK, 206
 HAL_REG_RESET, 206
 HAL_REG_SET, 206
 HAL_REG_SYS_LOCK, 206
 HAL_REG_SYS_UNLOCK, 206
 HAL_REG_WRITE, 206
 HAL_REG_WRITE_BIT, 207
 HAL_REG_WRITE_BIT_MASK_POS, 207
 hal_ksmcu_reset.h
 HAL_RESET_ResetCore, 208
 HAL_RESET_ResetGlobal, 208
 HAL_RESET_ResetRegister, 208
 hal_ksmcu_spi.h
 HAL_SPI_0, 211
 HAL_SPI_1, 211
 HAL_SPI_Disable, 211
 HAL_SPI_Enable, 212
 HAL_SPI_MODE_MASTER_LSB_00, 211
 HAL_SPI_MODE_MASTER_LSB_01, 211
 HAL_SPI_MODE_MASTER_LSB_10, 211
 HAL_SPI_MODE_MASTER_LSB_11, 211
 HAL_SPI_MODE_MASTER_MSB_00, 211
 HAL_SPI_MODE_MASTER_MSB_01, 211
 HAL_SPI_MODE_MASTER_MSB_10, 211
 HAL_SPI_MODE_MASTER_MSB_11, 211
 HAL_SPI_MODE_SLAVE_LSB_00, 211
 HAL_SPI_MODE_SLAVE_LSB_01, 211
 HAL_SPI_MODE_SLAVE_LSB_10, 211
 HAL_SPI_MODE_SLAVE_LSB_11, 211
 HAL_SPI_MODE_SLAVE_MSB_00, 211
 HAL_SPI_MODE_SLAVE_MSB_01, 211
 HAL_SPI_MODE_SLAVE_MSB_10, 211
 HAL_SPI_MODE_SLAVE_MSB_11, 211
 HAL_SPI_MODE_x, 210
 HAL_SPI_Reset, 212
 HAL_SPI_RxData, 212
 HAL_SPI_RxDataWidth16, 212
 HAL_SPI_RxTxData, 213
 HAL_SPI_RxTxDataWidth16, 213
 HAL_SPI_SetBaud, 214
 HAL_SPI_SetGpioClk, 214
 HAL_SPI_SetGpioCs, 216
 HAL_SPI_SetGpioDi, 217
 HAL_SPI_SetGpioDo, 217
 HAL_SPI_SetGpioMiso, 218
 HAL_SPI_SetGpioMosi, 219
 HAL_SPI_SetMode, 219
 HAL_SPI_TxData, 220
 HAL_SPI_TxDataWidth16, 220
 HAL_SPI_x, 211
 hal_ksmcu_timer.h
 HAL_TIMER_0, 223
 HAL_TIMER_1, 223
 HAL_TIMER_2, 223
 HAL_TIMER_3, 224
 HAL_TIMER_4, 224
 HAL_TIMER_Disable, 224
 HAL_TIMER_Enable, 224
 HAL_TIMER_GetFreq, 224
 HAL_TIMER_GetThCnt, 224
 HAL_TIMER_IrqCompare0, 225
 HAL_TIMER_IrqCompare0Clear, 225
 HAL_TIMER_IrqCompare0Disable, 225
 HAL_TIMER_IrqCompare0Enable, 225
 HAL_TIMER_IrqCompare1, 225
 HAL_TIMER_IrqCompare1Clear, 226
 HAL_TIMER_IrqCompare1Disable, 226
 HAL_TIMER_IrqCompare1Enable, 226
 HAL_TIMER_IrqOverflow, 226
 HAL_TIMER_IrqOverflowClear, 226
 HAL_TIMER_IrqOverflowDisable, 226
 HAL_TIMER_IrqOverflowEnable, 226
 HAL_TIMER_MODE_COUNT, 223
 HAL_TIMER_MODE_PWMIN, 223
 HAL_TIMER_MODE_PWMOUT, 223
 HAL_TIMER_MODE_x, 223
 HAL_TIMER_Reset, 227
 HAL_TIMER_SetFreq, 227
 HAL_TIMER_SetGpioIn, 227
 HAL_TIMER_SetGpioOut, 228
 HAL_TIMER_SetPeriodCount, 229
 HAL_TIMER_SetPeriodmS, 229
 HAL_TIMER_SetPerioduS, 230
 HAL_TIMER_SetPwm, 230
 HAL_TIMER_SetPwmQ15, 230
 HAL_TIMER_SetWorkMode, 231
 HAL_TIMER_StartCounting, 231
 HAL_TIMER_StopCounting, 231
 HAL_TIMER_x, 223
 hal_ksmcu_tmp.h
 HAL_TMP_0, 234
 HAL_TMP_1, 234
 HAL_TMP_Adc2Temperature, 234
 HAL_TMP_Adc2Temperature01, 235
 HAL_TMP_Disable, 235
 HAL_TMP_Enable, 236
 HAL_TMP_GetTemperature, 236
 HAL_TMP_GetTemperature01, 237
 HAL_TMP_Reset, 238
 HAL_TMP_x, 234
 hal_ksmcu_trim.h
 HAL_TRIM_Read, 239
 HAL_TRIM_ReadId, 240
 HAL_TRIM_ReadIdCrc32, 241
 hal_ksmcu_uart.h
 HAL_UART_0, 244
 HAL_UART_1, 244
 HAL_UART_2, 244
 HAL_UART_Disable, 244

HAL_UART_Enable, 244
 HAL_UART_IrqCheckErr, 244
 HAL_UART_IrqCheckErrClear, 244
 HAL_UART_IrqCheckErrDisable, 244
 HAL_UART_IrqCheckErrEnable, 245
 HAL_UART_IrqReceiveOver, 245
 HAL_UART_IrqReceiveOverClear, 245
 HAL_UART_IrqReceiveOverDisable, 245
 HAL_UART_IrqReceiveOverEnable, 245
 HAL_UART_IrqSendOver, 245
 HAL_UART_IrqSendOverClear, 245
 HAL_UART_IrqSendOverDisable, 246
 HAL_UART_IrqSendOverEnable, 246
 HAL_UART_IrqStopErr, 246
 HAL_UART_IrqStopErrClear, 246
 HAL_UART_IrqStopErrDisable, 246
 HAL_UART_IrqStopErrEnable, 246
 HAL_UART_MODE_LSB_STOP1_EVEN, 243
 HAL_UART_MODE_LSB_STOP1_NONE, 243
 HAL_UART_MODE_LSB_STOP1_ODD, 243
 HAL_UART_MODE_LSB_STOP2_EVEN, 243
 HAL_UART_MODE_LSB_STOP2_NONE, 243
 HAL_UART_MODE_LSB_STOP2_ODD, 243
 HAL_UART_MODE_MSB_STOP1_EVEN, 243
 HAL_UART_MODE_MSB_STOP1_NONE, 243
 HAL_UART_MODE_MSB_STOP1_ODD, 243
 HAL_UART_MODE_MSB_STOP2_EVEN, 243
 HAL_UART_MODE_MSB_STOP2_NONE, 243
 HAL_UART_MODE_MSB_STOP2_ODD, 243
 HAL_UART_MODE_x, 243
 HAL_UART_ReadBuffer, 246
 HAL_UART_Reset, 247
 HAL_UART_SetBaud, 247
 HAL_UART_SetGpioRx, 247
 HAL_UART_SetGpioTx, 247
 HAL_UART_SetMode, 248
 HAL_UART_TxData, 248
 HAL_UART_WriteBuffer, 249
 HAL_UART_x, 243
 hal_iksmcu_wake.h
 HAL_WAKE_DeepSleepIoTimeWakeUp, 251
 HAL_WAKE_DeepSleepIoWake, 252
 HAL_WAKE_DeepSleepTimeWakeUp, 252
 HAL_WAKE_UP_IO_0_High, 251
 HAL_WAKE_UP_IO_0_Low, 251
 HAL_WAKE_UP_IO_1_High, 251
 HAL_WAKE_UP_IO_1_Low, 251
 HAL_WAKE_UP_IO_2_High, 251
 HAL_WAKE_UP_IO_2_Low, 251
 HAL_WAKE_UP_IO_3_High, 251
 HAL_WAKE_UP_IO_3_Low, 251
 HAL_WAKE_UP_IO_4_High, 251
 HAL_WAKE_UP_IO_4_Low, 251
 HAL_WAKE_UP_IO_5_High, 251
 HAL_WAKE_UP_IO_5_Low, 251
 HAL_WAKE_UP_IO_6_High, 251
 HAL_WAKE_UP_IO_6_Low, 251
 HAL_WAKE_UP_IO_7_High, 251
 HAL_WAKE_UP_IO_7_Low, 251
 HAL_WAKE_UP_IO_x, 251
 HAL_MCPWM
 hal_iks32mc07x_reg.c, 391
 hal_iks32mc07x_reg.h, 71
 HAL_MCPWM_0
 hal_iksmcu_mcpwm.h, 179
 HAL_MCPWM_1
 hal_iksmcu_mcpwm.h, 179
 HAL_MCPWM_DEFAULT_OUTPUT_STATE_HIGH
 hal_iksmcu_mcpwm.h, 177
 HAL_MCPWM_DEFAULT_OUTPUT_STATE_LOW
 hal_iksmcu_mcpwm.h, 177
 HAL_MCPWM_DEFAULT_OUTPUT_STATE_OFF
 hal_iksmcu_mcpwm.h, 177
 HAL_MCPWM_DEFAULT_OUTPUT_STATE_x
 hal_iksmcu_mcpwm.h, 177
 HAL_MCPWM_Disable
 hal_iks32mc07x_mcpwm.c, 359
 hal_iksmcu_mcpwm.h, 179
 HAL_MCPWM_Enable
 hal_iks32mc07x_mcpwm.c, 360
 hal_iksmcu_mcpwm.h, 179
 HAL_MCPWM_FAIL_CMP0_HIGH
 hal_iksmcu_mcpwm.h, 177
 HAL_MCPWM_FAIL_CMP0_LOW
 hal_iksmcu_mcpwm.h, 177
 HAL_MCPWM_FAIL_CMP1_HIGH
 hal_iksmcu_mcpwm.h, 177
 HAL_MCPWM_FAIL_CMP1_LOW
 hal_iksmcu_mcpwm.h, 177
 HAL_MCPWM_FAIL_IO0_HIGH
 hal_iksmcu_mcpwm.h, 177
 HAL_MCPWM_FAIL_IO0_LOW
 hal_iksmcu_mcpwm.h, 177
 HAL_MCPWM_FAIL_IO1_HIGH
 hal_iksmcu_mcpwm.h, 177
 HAL_MCPWM_FAIL_IO1_LOW
 hal_iksmcu_mcpwm.h, 177
 HAL_MCPWM_FAIL_x
 hal_iksmcu_mcpwm.h, 177
 HAL_MCPWM_GetFreq
 hal_iks32mc07x_mcpwm.c, 360
 hal_iksmcu_mcpwm.h, 179
 HAL_MCPWM_GetThCnt
 hal_iks32mc07x_mcpwm.c, 360
 hal_iksmcu_mcpwm.h, 181
 HAL_MCPWM_IrqFail
 hal_iks32mc07x_mcpwm.c, 361
 hal_iksmcu_mcpwm.h, 181
 HAL_MCPWM_IrqFailClear
 hal_iks32mc07x_mcpwm.c, 361
 hal_iksmcu_mcpwm.h, 182
 HAL_MCPWM_IrqFailDisable
 hal_iks32mc07x_mcpwm.c, 361
 hal_iksmcu_mcpwm.h, 182
 HAL_MCPWM_IrqFailEnable
 hal_iks32mc07x_mcpwm.c, 361

hal_iksmcu_mcpwm.h, 182
 HAL_MCPWM_IRQHandler
 hal_iks32mc07x.h, 63
 HAL_MCPWM_IRQHandler_00
 hal_iks32mc07x.h, 63
 HAL_MCPWM_IRQHandler_01
 hal_iks32mc07x.h, 63
 HAL_MCPWM_IRQHandler_10
 hal_iks32mc07x.h, 63
 HAL_MCPWM_IRQHandler_11
 hal_iks32mc07x.h, 63
 HAL_MCPWM_IrqOverflow
 hal_iks32mc07x_mcpwm.c, 362
 hal_iksmcu_mcpwm.h, 182
 HAL_MCPWM_IrqOverflowClear
 hal_iks32mc07x_mcpwm.c, 362
 hal_iksmcu_mcpwm.h, 182
 HAL_MCPWM_IrqOverflowDisable
 hal_iks32mc07x_mcpwm.c, 362
 hal_iksmcu_mcpwm.h, 182
 HAL_MCPWM_IrqOverflowEnable
 hal_iks32mc07x_mcpwm.c, 362
 hal_iksmcu_mcpwm.h, 182
 HAL_MCPWM_IrqZeroCross
 hal_iks32mc07x_mcpwm.c, 362
 hal_iksmcu_mcpwm.h, 183
 HAL_MCPWM_IrqZeroCrossClear
 hal_iks32mc07x_mcpwm.c, 362
 hal_iksmcu_mcpwm.h, 183
 HAL_MCPWM_IrqZeroCrossDisable
 hal_iks32mc07x_mcpwm.c, 362
 hal_iksmcu_mcpwm.h, 183
 HAL_MCPWM_IrqZeroCrossEnable
 hal_iks32mc07x_mcpwm.c, 363
 hal_iksmcu_mcpwm.h, 183
 HAL_MCPWM_OUTPUT_STATE_HIGH
 hal_iksmcu_mcpwm.h, 178
 HAL_MCPWM_OUTPUT_STATE_HIGH_PWM
 hal_iksmcu_mcpwm.h, 178
 HAL_MCPWM_OUTPUT_STATE_LOW
 hal_iksmcu_mcpwm.h, 178
 HAL_MCPWM_OUTPUT_STATE_LOW_PWM
 hal_iksmcu_mcpwm.h, 178
 HAL_MCPWM_OUTPUT_STATE_OFF
 hal_iksmcu_mcpwm.h, 178
 HAL_MCPWM_OUTPUT_STATE_PWM
 hal_iksmcu_mcpwm.h, 178
 HAL_MCPWM_OUTPUT_STATE_x
 hal_iksmcu_mcpwm.h, 177
 HAL_MCPWM_POLARITY_PH_NH
 hal_iksmcu_mcpwm.h, 178
 HAL_MCPWM_POLARITY_PH_NL
 hal_iksmcu_mcpwm.h, 178
 HAL_MCPWM_POLARITY_PL_NH
 hal_iksmcu_mcpwm.h, 178
 HAL_MCPWM_POLARITY_PL_NL
 hal_iksmcu_mcpwm.h, 178
 HAL_MCPWM_POLARITY_x
 hal_iksmcu_mcpwm.h, 178
 HAL_MCPWM_PORT_0
 hal_iksmcu_mcpwm.h, 178
 HAL_MCPWM_PORT_1
 hal_iksmcu_mcpwm.h, 178
 HAL_MCPWM_PORT_2
 hal_iksmcu_mcpwm.h, 178
 HAL_MCPWM_PORT_3
 hal_iksmcu_mcpwm.h, 178
 HAL_MCPWM_PORT_4
 hal_iksmcu_mcpwm.h, 178
 HAL_MCPWM_PORT_5
 hal_iksmcu_mcpwm.h, 178
 HAL_MCPWM_PORT_x
 hal_iksmcu_mcpwm.h, 178
 HAL_MCPWM_Reset
 hal_iks32mc07x_mcpwm.c, 363
 hal_iksmcu_mcpwm.h, 183
 HAL_MCPWM_SetAdcTrig
 hal_iks32mc07x_mcpwm.c, 363
 hal_iksmcu_mcpwm.h, 184
 HAL_MCPWM_SetAdcTrigQ15
 hal_iks32mc07x_mcpwm.c, 363
 hal_iksmcu_mcpwm.h, 184
 HAL_MCPWM_SetDead
 hal_iks32mc07x_mcpwm.c, 364
 hal_iksmcu_mcpwm.h, 184
 HAL_MCPWM_SetDefaultState
 hal_iks32mc07x_mcpwm.c, 364
 hal_iksmcu_mcpwm.h, 185
 HAL_MCPWM_SetDrivePolarity
 hal_iks32mc07x_mcpwm.c, 365
 hal_iksmcu_mcpwm.h, 185
 HAL_MCPWM_SetFailSignal
 hal_iks32mc07x_mcpwm.c, 366
 hal_iksmcu_mcpwm.h, 186
 HAL_MCPWM_SetFreq
 hal_iks32mc07x_mcpwm.c, 366
 hal_iksmcu_mcpwm.h, 186
 HAL_MCPWM_SetGpioFail
 hal_iks32mc07x_mcpwm.c, 366
 hal_iksmcu_mcpwm.h, 187
 HAL_MCPWM_SetGpioOut
 hal_iks32mc07x_mcpwm.c, 367
 hal_iksmcu_mcpwm.h, 187
 HAL_MCPWM_SetOutputDisable
 hal_iks32mc07x_mcpwm.c, 367
 hal_iksmcu_mcpwm.h, 188
 HAL_MCPWM_SetOutputEnable
 hal_iks32mc07x_mcpwm.c, 368
 hal_iksmcu_mcpwm.h, 188
 HAL_MCPWM_SetOutState
 hal_iks32mc07x_mcpwm.c, 368
 hal_iksmcu_mcpwm.h, 189
 HAL_MCPWM_SetPwm
 hal_iks32mc07x_mcpwm.c, 369
 hal_iksmcu_mcpwm.h, 189
 HAL_MCPWM_SetPwmQ15

hal_lks32mc07x_mcpwm.c, 369
 hal_lksmcu_mcpwm.h, 190
 HAL_MCPWM_StartCounting
 hal_lks32mc07x_mcpwm.c, 370
 hal_lksmcu_mcpwm.h, 190
 HAL_MCPWM_StopCounting
 hal_lks32mc07x_mcpwm.c, 370
 hal_lksmcu_mcpwm.h, 190
 HAL_MCPWM_TypeDef, 30
 AUEN, 32
 CH_DEF, 32
 CH_MSK, 33
 CNT0, 33
 CNT1, 33
 DTH00, 33
 DTH01, 33
 DTH10, 33
 DTH11, 34
 EIE, 34
 EIF, 34
 EVT0, 34
 EVT1, 34
 FAIL012, 34
 FAIL345, 35
 FCNT, 35
 FLT, 35
 IE0, 35
 IE1, 35
 IF0, 35
 IF1, 36
 IO01, 36
 IO23, 36
 IO45, 36
 null, 36
 PP, 36
 PRT, 37
 RE, 37
 SDCFG, 37
 STT_HYST, 37
 TCLK, 37
 TH, 37
 TH0, 38
 TH1, 38
 TMR0, 38
 TMR1, 38
 TMR2, 38
 TMR3, 38
 UPDATE, 39
 ZCS_DELAY, 39
 hal_MCPWM_UpdatePolarityState
 hal_lks32mc07x_mcpwm.c, 370
 HAL_MCPWM_x
 hal_lksmcu_mcpwm.h, 178
 HAL_OPA_0
 hal_lksmcu_opa.h, 195
 HAL_OPA_0B
 hal_lksmcu_opa.h, 195
 HAL_OPA_1
 hal_lksmcu_opa.h, 195
 HAL_OPA_2
 hal_lksmcu_opa.h, 195
 HAL_OPA_3
 hal_lksmcu_opa.h, 195
 HAL_OPA_4
 hal_lksmcu_opa.h, 195
 HAL_OPA_5
 hal_lksmcu_opa.h, 195
 HAL_OPA_Disable
 hal_lks32mc07x_opa.c, 379
 hal_lksmcu_opa.h, 196
 HAL_OPA_Enable
 hal_lks32mc07x_opa.c, 379
 hal_lksmcu_opa.h, 196
 HAL_OPA_GAIN_160_10
 hal_lksmcu_opa.h, 195
 HAL_OPA_GAIN_160_40
 hal_lksmcu_opa.h, 195
 HAL_OPA_GAIN_170_40
 hal_lksmcu_opa.h, 195
 HAL_OPA_GAIN_170_40_4
 hal_lksmcu_opa.h, 195
 HAL_OPA_GAIN_170_40_6
 hal_lksmcu_opa.h, 195
 HAL_OPA_GAIN_180_30
 hal_lksmcu_opa.h, 195
 HAL_OPA_GAIN_180_30_4
 hal_lksmcu_opa.h, 195
 HAL_OPA_GAIN_180_30_6
 hal_lksmcu_opa.h, 195
 HAL_OPA_GAIN_190_20
 hal_lksmcu_opa.h, 195
 HAL_OPA_GAIN_190_20_4
 hal_lksmcu_opa.h, 195
 HAL_OPA_GAIN_190_20_6
 hal_lksmcu_opa.h, 195
 HAL_OPA_GAIN_200_10
 hal_lksmcu_opa.h, 195
 HAL_OPA_GAIN_200_10_4
 hal_lksmcu_opa.h, 195
 HAL_OPA_GAIN_200_10_6
 hal_lksmcu_opa.h, 195
 HAL_OPA_GAIN_20_10
 hal_lksmcu_opa.h, 195
 HAL_OPA_GAIN_320_10
 hal_lksmcu_opa.h, 195
 HAL_OPA_GAIN_320_40
 hal_lksmcu_opa.h, 195
 HAL_OPA_GAIN_320_5
 hal_lksmcu_opa.h, 195
 HAL_OPA_GAIN_40_10
 hal_lksmcu_opa.h, 195
 HAL_OPA_GAIN_640_40
 hal_lksmcu_opa.h, 195
 HAL_OPA_GAIN_660_20
 hal_lksmcu_opa.h, 195
 HAL_OPA_GAIN_80_10

hal_lksmcu_opa.h, 195
 HAL_OPA_GAIN_HIGH
 hal_lksmcu_opa.h, 195
 HAL_OPA_GAIN_LOW
 hal_lksmcu_opa.h, 195
 HAL_OPA_GAIN_MEDIUM
 hal_lksmcu_opa.h, 195
 HAL_OPA_GAIN_x
 hal_lksmcu_opa.h, 194
 HAL_OPA_OPAX_2_ADCCHN
 hal_lksmcu_opa.h, 194
 HAL_OPA_Out2GpioDisable
 hal_lks32mc07x_opa.c, 379
 hal_lksmcu_opa.h, 196
 HAL_OPA_Out2GpioEnable
 hal_lks32mc07x_opa.c, 379
 hal_lksmcu_opa.h, 196
 HAL_OPA_READ_GAIN
 hal_lksmcu_opa.h, 194
 HAL_OPA_Reset
 hal_lks32mc07x_opa.c, 379
 hal_lksmcu_opa.h, 196
 HAL_OPA_SetGain
 hal_lks32mc07x_opa.c, 380
 hal_lksmcu_opa.h, 197
 HAL_OPA_SetGpioIn
 hal_lks32mc07x_opa.c, 380
 hal_lksmcu_opa.h, 197
 HAL_OPA_SetGpioOut
 hal_lks32mc07x_opa.c, 380
 hal_lksmcu_opa.h, 197
 HAL_OPA_x
 hal_lksmcu_opa.h, 195
 HAL_PWRDN_IRQHandler
 hal_lks32mc07x.h, 63
 HAL_PWRDN_IRQHandler_00
 hal_lks32mc07x.h, 63
 HAL_QEP
 hal_lks32mc07x_reg.c, 391
 hal_lks32mc07x_reg.h, 71
 HAL_QEP_0
 hal_lksmcu_qep.h, 201
 HAL_QEP_1
 hal_lksmcu_qep.h, 201
 HAL_QEP_2
 hal_lksmcu_qep.h, 201
 HAL_QEP_3
 hal_lksmcu_qep.h, 201
 HAL_QEP_Disable
 hal_lks32mc07x_qep.c, 384
 hal_lksmcu_qep.h, 201
 HAL_QEP_Enable
 hal_lks32mc07x_qep.c, 384
 hal_lksmcu_qep.h, 201
 HAL_QEP_IRQHandler
 hal_lks32mc07x.h, 63
 HAL_QEP_IRQHandler_00
 hal_lks32mc07x.h, 64
 HAL_QEP_IRQHandler_10
 hal_lks32mc07x.h, 64
 HAL_QEP_IRQHandler_20
 hal_lks32mc07x.h, 64
 HAL_QEP_IRQHandler_30
 hal_lks32mc07x.h, 64
 HAL_QEP_MODE_DUAL_PULSE
 hal_lksmcu_qep.h, 200
 HAL_QEP_MODE_QUADRATURE
 hal_lksmcu_qep.h, 200
 HAL_QEP_MODE_SYMBOL_PULSE
 hal_lksmcu_qep.h, 200
 HAL_QEP_MODE_x
 hal_lksmcu_qep.h, 200
 HAL_QEP_ReadCnt
 hal_lks32mc07x_qep.c, 384
 hal_lksmcu_qep.h, 202
 HAL_QEP_Reset
 hal_lks32mc07x_qep.c, 385
 hal_lksmcu_qep.h, 202
 HAL_QEP_ResetCnt
 hal_lks32mc07x_qep.c, 385
 hal_lksmcu_qep.h, 202
 HAL_QEP_SetGpio
 hal_lks32mc07x_qep.c, 385
 hal_lksmcu_qep.h, 203
 HAL_QEP_SetMode
 hal_lks32mc07x_qep.c, 386
 hal_lksmcu_qep.h, 203
 HAL_QEP_SetTh
 hal_lks32mc07x_qep.c, 386
 hal_lksmcu_qep.h, 204
 HAL_QEP_TypeDef, 39
 CFG, 40
 CNT, 40
 IE, 40
 IF, 40
 TH, 40
 HAL_QEP_x
 hal_lksmcu_qep.h, 200
 HAL_REG_IF_WRITE
 hal_lksmcu_reg.h, 205
 HAL_REG_INVERT
 hal_lksmcu_reg.h, 205
 HAL_REG_READ
 hal_lksmcu_reg.h, 205
 HAL_REG_READ_BIT
 hal_lksmcu_reg.h, 205
 HAL_REG_READ_BIT_POS_MASK
 hal_lksmcu_reg.h, 206
 HAL_REG_RESET
 hal_lksmcu_reg.h, 206
 HAL_REG_SET
 hal_lksmcu_reg.h, 206
 HAL_REG_SYS_LOCK
 hal_lksmcu_reg.h, 206
 HAL_REG_SYS_UNLOCK
 hal_lksmcu_reg.h, 206

HAL_REG_WRITE
 hal_lksmcu_reg.h, 206

HAL_REG_WRITE_BIT
 hal_lksmcu_reg.h, 207

HAL_REG_WRITE_BIT_MASK_POS
 hal_lksmcu_reg.h, 207

HAL_RESET_ResetCore
 hal_lks32mc07x_reset.c, 393
 hal_lksmcu_reset.h, 208

HAL_RESET_ResetGlobal
 hal_lks32mc07x_reset.c, 393
 hal_lksmcu_reset.h, 208

HAL_RESET_ResetRegister
 hal_lks32mc07x_reset.c, 393
 hal_lksmcu_reset.h, 208

HAL_SIF_IRQHandler
 hal_lks32mc07x.h, 64

HAL_SIF_IRQHandler_00
 hal_lks32mc07x.h, 64

HAL_SPI
 hal_lks32mc07x_reg.c, 391
 hal_lks32mc07x_reg.h, 72

HAL_SPI_0
 hal_lksmcu_spi.h, 211

HAL_SPI_1
 hal_lksmcu_spi.h, 211

HAL_SPI_Disable
 hal_lks32mc07x_spi.c, 396
 hal_lksmcu_spi.h, 211

HAL_SPI_Enable
 hal_lks32mc07x_spi.c, 396
 hal_lksmcu_spi.h, 212

HAL_SPI_IRQHandler
 hal_lks32mc07x.h, 64

HAL_SPI_IRQHandler_00
 hal_lks32mc07x.h, 64

HAL_SPI_IRQHandler_10
 hal_lks32mc07x.h, 65

HAL_SPI_MODE_MASTER_LSB_00
 hal_lksmcu_spi.h, 211

HAL_SPI_MODE_MASTER_LSB_01
 hal_lksmcu_spi.h, 211

HAL_SPI_MODE_MASTER_LSB_10
 hal_lksmcu_spi.h, 211

HAL_SPI_MODE_MASTER_LSB_11
 hal_lksmcu_spi.h, 211

HAL_SPI_MODE_MASTER_MSB_00
 hal_lksmcu_spi.h, 211

HAL_SPI_MODE_MASTER_MSB_01
 hal_lksmcu_spi.h, 211

HAL_SPI_MODE_MASTER_MSB_10
 hal_lksmcu_spi.h, 211

HAL_SPI_MODE_MASTER_MSB_11
 hal_lksmcu_spi.h, 211

HAL_SPI_MODE_SLAVE_LSB_00
 hal_lksmcu_spi.h, 211

HAL_SPI_MODE_SLAVE_LSB_01
 hal_lksmcu_spi.h, 211

HAL_SPI_MODE_SLAVE_LSB_10
 hal_lksmcu_spi.h, 211

HAL_SPI_MODE_SLAVE_LSB_11
 hal_lksmcu_spi.h, 211

HAL_SPI_MODE_SLAVE_MSB_00
 hal_lksmcu_spi.h, 211

HAL_SPI_MODE_SLAVE_MSB_01
 hal_lksmcu_spi.h, 211

HAL_SPI_MODE_SLAVE_MSB_10
 hal_lksmcu_spi.h, 211

HAL_SPI_MODE_SLAVE_MSB_11
 hal_lksmcu_spi.h, 211

HAL_SPI_MODE_x
 hal_lksmcu_spi.h, 210

HAL_SPI_Reset
 hal_lks32mc07x_spi.c, 396
 hal_lksmcu_spi.h, 212

HAL_SPI_RxData
 hal_lks32mc07x_spi.c, 396
 hal_lksmcu_spi.h, 212

HAL_SPI_RxDataWidth16
 hal_lks32mc07x_spi.c, 398
 hal_lksmcu_spi.h, 212

HAL_SPI_RxTxData
 hal_lks32mc07x_spi.c, 398
 hal_lksmcu_spi.h, 213

HAL_SPI_RxTxDataWidth16
 hal_lks32mc07x_spi.c, 399
 hal_lksmcu_spi.h, 213

HAL_SPI_SetBaud
 hal_lks32mc07x_spi.c, 399
 hal_lksmcu_spi.h, 214

HAL_SPI_SetGpioClk
 hal_lks32mc07x_spi.c, 400
 hal_lksmcu_spi.h, 214

HAL_SPI_SetGpioCs
 hal_lks32mc07x_spi.c, 400
 hal_lksmcu_spi.h, 216

HAL_SPI_SetGpioDi
 hal_lks32mc07x_spi.c, 401
 hal_lksmcu_spi.h, 217

HAL_SPI_SetGpioDo
 hal_lks32mc07x_spi.c, 402
 hal_lksmcu_spi.h, 217

HAL_SPI_SetGpioMiso
 hal_lks32mc07x_spi.c, 402
 hal_lksmcu_spi.h, 218

HAL_SPI_SetGpioMosi
 hal_lks32mc07x_spi.c, 403
 hal_lksmcu_spi.h, 219

HAL_SPI_SetMode
 hal_lks32mc07x_spi.c, 404
 hal_lksmcu_spi.h, 219

HAL_SPI_TxData
 hal_lks32mc07x_spi.c, 404
 hal_lksmcu_spi.h, 220

HAL_SPI_TxDataWidth16
 hal_lks32mc07x_spi.c, 404

- hal_ksmcu_spi.h, 220
- HAL_SPI_TypeDef, 40
 - BAUD, 41
 - CFG, 41
 - IE, 41
 - RXDATA, 41
 - SIZE, 42
 - TXDATA, 42
- HAL_SPI_x
 - hal_ksmcu_spi.h, 211
- HAL_SW_IRQHandler
 - hal_ks32mc07x.h, 65
- HAL_SW_IRQHandler_00
 - hal_ks32mc07x.h, 65
- HAL_SYS
 - hal_ks32mc07x_reg.c, 391
 - hal_ks32mc07x_reg.h, 72
- HAL_SYS_TypeDef, 42
 - ADC, 43
 - CLK_CFG, 43
 - CLK_DIV0, 44
 - CLK_DIV1, 44
 - CLK_DIV2, 44
 - CLK_FEN, 44
 - DAC0, 44
 - DAC0_AMC, 44
 - DAC0_DC, 44
 - DAC1, 44
 - DAC1_AMC, 45
 - DAC1_DC, 45
 - DAC_CTRL, 45
 - DBG, 45
 - DBG_CFG, 45
 - FLSE, 45
 - FLSP, 45
 - FLST, 45
 - INFO, 46
 - IO_CFG, 46
 - MBIST, 46
 - null0, 46
 - null1, 46
 - null10, 46
 - null11, 46
 - null12, 46
 - null13, 47
 - null2, 47
 - null3, 47
 - null4, 47
 - null5, 47
 - null6, 47
 - null7, 47
 - null8, 47
 - null9, 48
 - REG0, 48
 - REG1, 48
 - REG2, 48
 - REG3, 48
 - REG4, 48
 - REG5, 48
 - REG6, 48
 - REG7, 49
 - REG8, 49
 - REG9, 49
 - REGA, 49
 - REGB, 49
 - REGC, 49
 - SBIST, 49
 - SFT_RST, 49
 - TRIM, 50
 - WR_PROTECT, 50
- HAL_TestRegAddr
 - hal_ks32mc07x_test.c, 409
- HAL_TIMER
 - hal_ks32mc07x_reg.c, 391
 - hal_ks32mc07x_reg.h, 72
- HAL_TIMER_0
 - hal_ksmcu_timer.h, 223
- HAL_TIMER_1
 - hal_ksmcu_timer.h, 223
- HAL_TIMER_2
 - hal_ksmcu_timer.h, 223
- HAL_TIMER_3
 - hal_ksmcu_timer.h, 224
- HAL_TIMER_4
 - hal_ksmcu_timer.h, 224
- HAL_TIMER_Disable
 - hal_ks32mc07x_timer.c, 411
 - hal_ksmcu_timer.h, 224
- HAL_TIMER_Enable
 - hal_ks32mc07x_timer.c, 411
 - hal_ksmcu_timer.h, 224
- HAL_TIMER_GetFreq
 - hal_ks32mc07x_timer.c, 411
 - hal_ksmcu_timer.h, 224
- HAL_TIMER_GetThCnt
 - hal_ks32mc07x_timer.c, 411
 - hal_ksmcu_timer.h, 224
- HAL_TIMER_IrqCompare0
 - hal_ks32mc07x_timer.c, 412
 - hal_ksmcu_timer.h, 225
- HAL_TIMER_IrqCompare0Clear
 - hal_ks32mc07x_timer.c, 412
 - hal_ksmcu_timer.h, 225
- HAL_TIMER_IrqCompare0Disable
 - hal_ks32mc07x_timer.c, 412
 - hal_ksmcu_timer.h, 225
- HAL_TIMER_IrqCompare0Enable
 - hal_ks32mc07x_timer.c, 412
 - hal_ksmcu_timer.h, 225
- HAL_TIMER_IrqCompare1
 - hal_ks32mc07x_timer.c, 412
 - hal_ksmcu_timer.h, 225
- HAL_TIMER_IrqCompare1Clear
 - hal_ks32mc07x_timer.c, 413
 - hal_ksmcu_timer.h, 226
- HAL_TIMER_IrqCompare1Disable

hal_lks32mc07x_timer.c, 413
 hal_lksmcu_timer.h, 226
 HAL_TIMER_IrqCompare1Enable
 hal_lks32mc07x_timer.c, 413
 hal_lksmcu_timer.h, 226
 HAL_TIMER_IRQHandler
 hal_lks32mc07x.h, 65
 HAL_TIMER_IRQHandler_00
 hal_lks32mc07x.h, 65
 HAL_TIMER_IRQHandler_10
 hal_lks32mc07x.h, 65
 HAL_TIMER_IRQHandler_20
 hal_lks32mc07x.h, 65
 HAL_TIMER_IRQHandler_30
 hal_lks32mc07x.h, 66
 HAL_TIMER_IRQHandler_40
 hal_lks32mc07x.h, 66
 HAL_TIMER_IrqOverflow
 hal_lks32mc07x_timer.c, 413
 hal_lksmcu_timer.h, 226
 HAL_TIMER_IrqOverflowClear
 hal_lks32mc07x_timer.c, 413
 hal_lksmcu_timer.h, 226
 HAL_TIMER_IrqOverflowDisable
 hal_lks32mc07x_timer.c, 413
 hal_lksmcu_timer.h, 226
 HAL_TIMER_IrqOverflowEnable
 hal_lks32mc07x_timer.c, 413
 hal_lksmcu_timer.h, 226
 HAL_TIMER_MODE_COUNT
 hal_lksmcu_timer.h, 223
 HAL_TIMER_MODE_PWMIN
 hal_lksmcu_timer.h, 223
 HAL_TIMER_MODE_PWMOUT
 hal_lksmcu_timer.h, 223
 HAL_TIMER_MODE_x
 hal_lksmcu_timer.h, 223
 HAL_TIMER_Reset
 hal_lks32mc07x_timer.c, 414
 hal_lksmcu_timer.h, 227
 HAL_TIMER_SetFreq
 hal_lks32mc07x_timer.c, 414
 hal_lksmcu_timer.h, 227
 HAL_TIMER_SetGpioIn
 hal_lks32mc07x_timer.c, 414
 hal_lksmcu_timer.h, 227
 HAL_TIMER_SetGpioOut
 hal_lks32mc07x_timer.c, 415
 hal_lksmcu_timer.h, 228
 HAL_TIMER_SetPeriodCount
 hal_lks32mc07x_timer.c, 416
 hal_lksmcu_timer.h, 229
 HAL_TIMER_SetPeriodmS
 hal_lks32mc07x_timer.c, 416
 hal_lksmcu_timer.h, 229
 HAL_TIMER_SetPerioduS
 hal_lks32mc07x_timer.c, 417
 hal_lksmcu_timer.h, 230
 HAL_TIMER_SetPwm
 hal_lks32mc07x_timer.c, 417
 hal_lksmcu_timer.h, 230
 HAL_TIMER_SetPwmQ15
 hal_lks32mc07x_timer.c, 417
 hal_lksmcu_timer.h, 230
 HAL_TIMER_SetWorkMode
 hal_lks32mc07x_timer.c, 418
 hal_lksmcu_timer.h, 231
 HAL_TIMER_StartCounting
 hal_lks32mc07x_timer.c, 418
 hal_lksmcu_timer.h, 231
 HAL_TIMER_StopCounting
 hal_lks32mc07x_timer.c, 418
 hal_lksmcu_timer.h, 231
 HAL_TIMER_TypeDef, 50
 CFG, 51
 CMP0, 51
 CMP1, 51
 CNT, 51
 EVT, 51
 FLT, 51
 IE, 52
 IF, 52
 IO, 52
 TH, 52
 HAL_TIMER_x
 hal_lksmcu_timer.h, 223
 HAL_TMP_0
 hal_lksmcu_tmp.h, 234
 HAL_TMP_1
 hal_lksmcu_tmp.h, 234
 HAL_TMP_Adc2Temperature
 hal_lks32mc07x_tmp.c, 424
 hal_lksmcu_tmp.h, 234
 HAL_TMP_Adc2Temperature01
 hal_lks32mc07x_tmp.c, 424
 hal_lksmcu_tmp.h, 235
 HAL_TMP_Disable
 hal_lks32mc07x_tmp.c, 425
 hal_lksmcu_tmp.h, 235
 HAL_TMP_Enable
 hal_lks32mc07x_tmp.c, 425
 hal_lksmcu_tmp.h, 236
 HAL_TMP_GetTemperature
 hal_lks32mc07x_tmp.c, 426
 hal_lksmcu_tmp.h, 236
 HAL_TMP_GetTemperature01
 hal_lks32mc07x_tmp.c, 427
 hal_lksmcu_tmp.h, 237
 HAL_TMP_Reset
 hal_lks32mc07x_tmp.c, 427
 hal_lksmcu_tmp.h, 238
 HAL_TMP_x
 hal_lksmcu_tmp.h, 234
 HAL_TRIM_Read
 hal_lks32mc07x_trim.c, 430
 hal_lksmcu_trim.h, 239

HAL_TRIM_ReadId
 hal_lks32mc07x_trim.c, 431
 hal_lksmcu_trim.h, 240

HAL_TRIM_ReadIdCrc32
 hal_lks32mc07x_trim.c, 432
 hal_lksmcu_trim.h, 241

HAL_TRIM_ReadMcu07
 hal_lks32mc07x_trim.c, 432

HAL_UART
 hal_lks32mc07x_reg.c, 391
 hal_lks32mc07x_reg.h, 72

HAL_UART_0
 hal_lksmcu_uart.h, 244

HAL_UART_1
 hal_lksmcu_uart.h, 244

HAL_UART_2
 hal_lksmcu_uart.h, 244

HAL_UART_Disable
 hal_lks32mc07x_uart.c, 435
 hal_lksmcu_uart.h, 244

HAL_UART_Enable
 hal_lks32mc07x_uart.c, 435
 hal_lksmcu_uart.h, 244

HAL_UART_IrqCheckErr
 hal_lks32mc07x_uart.c, 435
 hal_lksmcu_uart.h, 244

HAL_UART_IrqCheckErrClear
 hal_lks32mc07x_uart.c, 435
 hal_lksmcu_uart.h, 244

HAL_UART_IrqCheckErrDisable
 hal_lks32mc07x_uart.c, 435
 hal_lksmcu_uart.h, 244

HAL_UART_IrqCheckErrEnable
 hal_lks32mc07x_uart.c, 435
 hal_lksmcu_uart.h, 245

HAL_UART_IRQHandler
 hal_lks32mc07x.h, 66

HAL_UART_IRQHandler_00
 hal_lks32mc07x.h, 66

HAL_UART_IRQHandler_10
 hal_lks32mc07x.h, 66

HAL_UART_IRQHandler_20
 hal_lks32mc07x.h, 66

HAL_UART_IrqReceiveOver
 hal_lks32mc07x_uart.c, 436
 hal_lksmcu_uart.h, 245

HAL_UART_IrqReceiveOverClear
 hal_lks32mc07x_uart.c, 436
 hal_lksmcu_uart.h, 245

HAL_UART_IrqReceiveOverDisable
 hal_lks32mc07x_uart.c, 436
 hal_lksmcu_uart.h, 245

HAL_UART_IrqReceiveOverEnable
 hal_lks32mc07x_uart.c, 436
 hal_lksmcu_uart.h, 245

HAL_UART_IrqSendOver
 hal_lks32mc07x_uart.c, 436
 hal_lksmcu_uart.h, 245

HAL_UART_IrqSendOverClear
 hal_lks32mc07x_uart.c, 436
 hal_lksmcu_uart.h, 245

HAL_UART_IrqSendOverDisable
 hal_lks32mc07x_uart.c, 436
 hal_lksmcu_uart.h, 246

HAL_UART_IrqSendOverEnable
 hal_lks32mc07x_uart.c, 437
 hal_lksmcu_uart.h, 246

HAL_UART_IrqStopErr
 hal_lks32mc07x_uart.c, 437
 hal_lksmcu_uart.h, 246

HAL_UART_IrqStopErrClear
 hal_lks32mc07x_uart.c, 437
 hal_lksmcu_uart.h, 246

HAL_UART_IrqStopErrDisable
 hal_lks32mc07x_uart.c, 437
 hal_lksmcu_uart.h, 246

HAL_UART_IrqStopErrEnable
 hal_lks32mc07x_uart.c, 437
 hal_lksmcu_uart.h, 246

HAL_UART_MODE_LSB_STOP1_EVEN
 hal_lksmcu_uart.h, 243

HAL_UART_MODE_LSB_STOP1_NONE
 hal_lksmcu_uart.h, 243

HAL_UART_MODE_LSB_STOP1_ODD
 hal_lksmcu_uart.h, 243

HAL_UART_MODE_LSB_STOP2_EVEN
 hal_lksmcu_uart.h, 243

HAL_UART_MODE_LSB_STOP2_NONE
 hal_lksmcu_uart.h, 243

HAL_UART_MODE_LSB_STOP2_ODD
 hal_lksmcu_uart.h, 243

HAL_UART_MODE_MSB_STOP1_EVEN
 hal_lksmcu_uart.h, 243

HAL_UART_MODE_MSB_STOP1_NONE
 hal_lksmcu_uart.h, 243

HAL_UART_MODE_MSB_STOP1_ODD
 hal_lksmcu_uart.h, 243

HAL_UART_MODE_MSB_STOP2_EVEN
 hal_lksmcu_uart.h, 243

HAL_UART_MODE_MSB_STOP2_NONE
 hal_lksmcu_uart.h, 243

HAL_UART_MODE_MSB_STOP2_ODD
 hal_lksmcu_uart.h, 243

HAL_UART_MODE_x
 hal_lksmcu_uart.h, 243

HAL_UART_ReadBuffer
 hal_lks32mc07x_uart.c, 437
 hal_lksmcu_uart.h, 246

HAL_UART_Reset
 hal_lks32mc07x_uart.c, 437
 hal_lksmcu_uart.h, 247

HAL_UART_SetBaud
 hal_lks32mc07x_uart.c, 438
 hal_lksmcu_uart.h, 247

HAL_UART_SetGpioRx
 hal_lks32mc07x_uart.c, 438

- hal_lksmcu_uart.h, 247
- HAL_UART_SetGpioTx
 - hal_lks32mc07x_uart.c, 438
 - hal_lksmcu_uart.h, 247
- HAL_UART_SetMode
 - hal_lks32mc07x_uart.c, 439
 - hal_lksmcu_uart.h, 248
- HAL_UART_TxData
 - hal_lks32mc07x_uart.c, 439
 - hal_lksmcu_uart.h, 248
- HAL_UART_TypeDef, 52
 - ADR, 53
 - BUFF, 53
 - CTRL, 53
 - DIVH, 54
 - DIVL, 54
 - IE, 54
 - IF, 54
 - IOC, 54
 - RE, 54
 - STT, 54
- HAL_UART_WriteBuffer
 - hal_lks32mc07x_uart.c, 440
 - hal_lksmcu_uart.h, 249
- HAL_UART_x
 - hal_lksmcu_uart.h, 243
- HAL_WAKE_DeepSleepToTimeWakeUp
 - hal_lks32mc07x_wake.c, 445
 - hal_lksmcu_wake.h, 251
- HAL_WAKE_DeepSleepToWake
 - hal_lks32mc07x_wake.c, 445
 - hal_lksmcu_wake.h, 252
- HAL_WAKE_DeepSleepTimeWakeUp
 - hal_lks32mc07x_wake.c, 445
 - hal_lksmcu_wake.h, 252
- HAL_WAKE_IRQHandler
 - hal_lks32mc07x.h, 66
- HAL_WAKE_IRQHandler_00
 - hal_lks32mc07x.h, 66
- HAL_WAKE_UP_IO_0_High
 - hal_lksmcu_wake.h, 251
- HAL_WAKE_UP_IO_0_Low
 - hal_lksmcu_wake.h, 251
- HAL_WAKE_UP_IO_1_High
 - hal_lksmcu_wake.h, 251
- HAL_WAKE_UP_IO_1_Low
 - hal_lksmcu_wake.h, 251
- HAL_WAKE_UP_IO_2_High
 - hal_lksmcu_wake.h, 251
- HAL_WAKE_UP_IO_2_Low
 - hal_lksmcu_wake.h, 251
- HAL_WAKE_UP_IO_3_High
 - hal_lksmcu_wake.h, 251
- HAL_WAKE_UP_IO_3_Low
 - hal_lksmcu_wake.h, 251
- HAL_WAKE_UP_IO_4_High
 - hal_lksmcu_wake.h, 251
- HAL_WAKE_UP_IO_4_Low
 - hal_lksmcu_wake.h, 251
- HAL_WAKE_UP_IO_5_High
 - hal_lksmcu_wake.h, 251
- HAL_WAKE_UP_IO_5_Low
 - hal_lksmcu_wake.h, 251
- HAL_WAKE_UP_IO_6_High
 - hal_lksmcu_wake.h, 251
- HAL_WAKE_UP_IO_6_Low
 - hal_lksmcu_wake.h, 251
- HAL_WAKE_UP_IO_7_High
 - hal_lksmcu_wake.h, 251
- HAL_WAKE_UP_IO_7_Low
 - hal_lksmcu_wake.h, 251
- HAL_WAKE_UP_IO_x
 - hal_lksmcu_wake.h, 251
- HAL_WWDG_IRQHandler_00
 - hal_lks32mc07x.h, 67
- HTH
 - HAL_ADC_TypeDef, 11
- ICHN
 - HAL_ADC_TypeDef, 11
- IDAT
 - HAL_ADC_TypeDef, 11
- IE
 - CMP_TypeDef, 6
 - HAL_ADC_TypeDef, 11
 - HAL_CMP_TypeDef, 14
 - HAL_QEP_TypeDef, 40
 - HAL_SPI_TypeDef, 41
 - HAL_TIMER_TypeDef, 52
 - HAL_UART_TypeDef, 54
- IE0
 - HAL_MCPWM_TypeDef, 35
- IE1
 - HAL_MCPWM_TypeDef, 35
- IF
 - CMP_TypeDef, 6
 - HAL_ADC_TypeDef, 11
 - HAL_CMP_TypeDef, 14
 - HAL_QEP_TypeDef, 40
 - HAL_TIMER_TypeDef, 52
 - HAL_UART_TypeDef, 54
- IF0
 - HAL_MCPWM_TypeDef, 35
- IF1
 - HAL_MCPWM_TypeDef, 36
- include/hal_lks32mc07x_reg.h, 69, 72
- include/hal_lksmcu_adc.h, 76, 90
- include/hal_lksmcu_can.h, 92
- include/hal_lksmcu_clk.h, 93, 96
- include/hal_lksmcu_cmp.h, 96, 104
- include/hal_lksmcu_crc.h, 105, 108
- include/hal_lksmcu_dac.h, 108, 112
- include/hal_lksmcu_dma.h, 112, 116
- include/hal_lksmcu_dsp.h, 116, 119
- include/hal_lksmcu_flash.h, 119, 128
- include/hal_lksmcu_gpio.h, 129, 141
- include/hal_lksmcu_hall.h, 143, 150

include/hal_lksmcu_i2c.h, 151, 157
 include/hal_lksmcu_init.h, 157, 158
 include/hal_lksmcu_irq.h, 158, 162
 include/hal_lksmcu_irqfun.h, 163, 170
 include/hal_lksmcu_iwdg.h, 171, 174
 include/hal_lksmcu_mcpwm.h, 175, 191
 include/hal_lksmcu_opa.h, 192, 198
 include/hal_lksmcu_qep.h, 199, 204
 include/hal_lksmcu_reg.h, 204, 207
 include/hal_lksmcu_reset.h, 207, 209
 include/hal_lksmcu_spi.h, 209, 221
 include/hal_lksmcu_timer.h, 221, 232
 include/hal_lksmcu_tmp.h, 232, 238
 include/hal_lksmcu_trim.h, 239, 241
 include/hal_lksmcu_uart.h, 242, 249
 include/hal_lksmcu_wake.h, 250, 253
 INFO
 HAL_HALL_TypeDef, 27
 HAL_SYS_TypeDef, 46
 INIT
 HAL_CRC_TypeDef, 15
 initValue
 CRC_Model_Params, 7
 IO
 HAL_TIMER_TypeDef, 52
 IO01
 HAL_MCPWM_TypeDef, 36
 IO23
 HAL_MCPWM_TypeDef, 36
 IO45
 HAL_MCPWM_TypeDef, 36
 IO_CFG
 HAL_SYS_TypeDef, 46
 IOC
 HAL_UART_TypeDef, 54

 LTH
 HAL_ADC_TypeDef, 12

 MBIST
 HAL_SYS_TypeDef, 46
 mcpwm_polarity_state
 hal_lks32mc07x_mcpwm.c, 372
 MOD
 HAL_DSP_TypeDef, 18
 MSCR
 HAL_I2C_TypeDef, 28

 NADDR
 HAL_FLASH_TypeDef, 21
 NCFG
 HAL_FLASH_TypeDef, 21
 NERASE
 HAL_FLASH_TypeDef, 21
 NKEY
 HAL_FLASH_TypeDef, 21
 NRDATA
 HAL_FLASH_TypeDef, 21
 null
 HAL_ADC_TypeDef, 12
 HAL_MCPWM_TypeDef, 36
 null0
 HAL_SYS_TypeDef, 46
 null1
 HAL_GPIO_TypeDef, 24
 HAL_SYS_TypeDef, 46
 null10
 HAL_SYS_TypeDef, 46
 null11
 HAL_SYS_TypeDef, 46
 null12
 HAL_SYS_TypeDef, 46
 null13
 HAL_SYS_TypeDef, 47
 null2
 HAL_ADC_TypeDef, 12
 HAL_SYS_TypeDef, 47
 null3
 HAL_ADC_TypeDef, 12
 HAL_SYS_TypeDef, 47
 null4
 HAL_SYS_TypeDef, 47
 null5
 HAL_SYS_TypeDef, 47
 null6
 HAL_SYS_TypeDef, 47
 null7
 HAL_SYS_TypeDef, 47
 null8
 HAL_SYS_TypeDef, 47
 null9
 HAL_SYS_TypeDef, 48
 NWDATA
 HAL_FLASH_TypeDef, 22

 PC
 HAL_DSP_TypeDef, 18
 PDI
 HAL_GPIO_TypeDef, 25
 PDO
 HAL_GPIO_TypeDef, 25
 PFLT
 HAL_GPIO_TypeDef, 25
 PIE
 HAL_GPIO_TypeDef, 25
 PODE
 HAL_GPIO_TypeDef, 25
 POE
 HAL_GPIO_TypeDef, 25
 POL
 HAL_CRC_TypeDef, 15
 polynomial
 CRC_Model_Params, 7
 PP
 HAL_MCPWM_TypeDef, 36
 PROTECT
 HAL_FLASH_TypeDef, 22
 PRT

HAL_MCPWM_TypeDef, 37
 PSW
 HAL_IWDG_TypeDef, 30
 PUE
 HAL_GPIO_TypeDef, 26
 QUO
 HAL_DSP_TypeDef, 18
 RAD
 HAL_DSP_TypeDef, 18
 RDATA
 HAL_FLASH_TypeDef, 22
 RE
 HAL_MCPWM_TypeDef, 37
 HAL_UART_TypeDef, 54
 readme.md, 253
 READY
 HAL_FLASH_TypeDef, 22
 reflectIn
 CRC_Model_Params, 7
 reflectOut
 CRC_Model_Params, 7
 REG0
 HAL_SYS_TypeDef, 48
 REG1
 HAL_SYS_TypeDef, 48
 REG2
 HAL_SYS_TypeDef, 48
 REG3
 HAL_SYS_TypeDef, 48
 REG4
 HAL_SYS_TypeDef, 48
 REG5
 HAL_SYS_TypeDef, 48
 REG6
 HAL_SYS_TypeDef, 48
 REG7
 HAL_SYS_TypeDef, 49
 REG8
 HAL_SYS_TypeDef, 49
 REG9
 HAL_SYS_TypeDef, 49
 REGA
 HAL_SYS_TypeDef, 49
 REGB
 HAL_SYS_TypeDef, 49
 REGC
 HAL_SYS_TypeDef, 49
 REM
 HAL_DSP_TypeDef, 19
 REN
 HAL_DMA_TypeDef, 16
 RTH
 HAL_IWDG_TypeDef, 30
 RXDATA
 HAL_SPI_TypeDef, 41
 SADR
 HAL_DMA_TypeDef, 17
 SBIST
 HAL_SYS_TypeDef, 49
 SC
 HAL_DSP_TypeDef, 19
 SCR
 HAL_I2C_TypeDef, 29
 SDCFG
 HAL_MCPWM_TypeDef, 37
 SFT_RST
 HAL_SYS_TypeDef, 49
 SIN
 HAL_DSP_TypeDef, 19
 SIZE
 HAL_FLASH_TypeDef, 22
 HAL_SPI_TypeDef, 42
 source/hal_lks32mc07x_adc.c, 253, 265
 source/hal_lks32mc07x_clk.c, 269, 272
 source/hal_lks32mc07x_cmp.c, 273, 278
 source/hal_lks32mc07x_crc.c, 280, 283
 source/hal_lks32mc07x_dac.c, 284, 288
 source/hal_lks32mc07x_dma.c, 290, 294
 source/hal_lks32mc07x_dsp.c, 295, 298
 source/hal_lks32mc07x_flash.c, 298, 308
 source/hal_lks32mc07x_gpio.c, 311, 319
 source/hal_lks32mc07x_hall.c, 322, 328
 source/hal_lks32mc07x_i2c.c, 330, 336
 source/hal_lks32mc07x_init.c, 338, 339
 source/hal_lks32mc07x_irq.c, 340, 342
 source/hal_lks32mc07x_irqfun.c, 343, 352
 source/hal_lks32mc07x_iwdg.c, 353, 357
 source/hal_lks32mc07x_mcpwm.c, 357, 373
 source/hal_lks32mc07x_opa.c, 377, 381
 source/hal_lks32mc07x_qep.c, 383, 387
 source/hal_lks32mc07x_reg.c, 388, 392
 source/hal_lks32mc07x_reset.c, 392, 394
 source/hal_lks32mc07x_spi.c, 394, 405
 source/hal_lks32mc07x_test.c, 408, 409
 source/hal_lks32mc07x_timer.c, 409, 419
 source/hal_lks32mc07x_tmp.c, 422, 428
 source/hal_lks32mc07x_trim.c, 429, 433
 source/hal_lks32mc07x_uart.c, 434, 440
 source/hal_lks32mc07x_wake.c, 444, 446
 SQRT
 HAL_DSP_TypeDef, 19
 STT
 HAL_UART_TypeDef, 54
 STT_HYST
 HAL_MCPWM_TypeDef, 37
 SWT
 HAL_ADC_TypeDef, 12
 TCLK
 CMP_TypeDef, 6
 HAL_CMP_TypeDef, 14
 HAL_MCPWM_TypeDef, 37
 TH
 HAL_HALL_TypeDef, 27
 HAL_MCPWM_TypeDef, 37

HAL_QEP_TypeDef, [40](#)
HAL_TIMER_TypeDef, [52](#)
TH0
 HAL_MCPWM_TypeDef, [38](#)
TH1
 HAL_MCPWM_TypeDef, [38](#)
THETA
 HAL_DSP_TypeDef, [19](#)
tmp_coef_a
 hal_lks32mc07x_tmp.c, [428](#)
tmp_coef_b
 hal_lks32mc07x_tmp.c, [428](#)
TMR0
 HAL_MCPWM_TypeDef, [38](#)
TMR1
 HAL_MCPWM_TypeDef, [38](#)
TMR2
 HAL_MCPWM_TypeDef, [38](#)
TMR3
 HAL_MCPWM_TypeDef, [38](#)
TRIG
 HAL_ADC_TypeDef, [12](#)
TRIM
 HAL_SYS_TypeDef, [50](#)
TXDATA
 HAL_SPI_TypeDef, [42](#)

UPDATE
 HAL_MCPWM_TypeDef, [39](#)

WDATA
 HAL_FLASH_TypeDef, [22](#)
WIDTH
 HAL_HALL_TypeDef, [27](#)
WR_PROTECT
 HAL_SYS_TypeDef, [50](#)
WTH
 HAL_IWDG_TypeDef, [30](#)

X
 HAL_DSP_TypeDef, [19](#)
xorOut
 CRC_Model_Params, [7](#)

Y
 HAL_DSP_TypeDef, [19](#)

ZCS_DELAY
 HAL_MCPWM_TypeDef, [39](#)